Clean Code & Software Architecture

A Complete Guide

Write Maintainable Code and Design Scalable Systems -- With Java and Python

2025 Edition • 32 Chapters

Table of Contents

Chapter 1 — Why Clean Code Matters
Chapter 2 — Meaningful Names
Chapter 3 — Functions
Chapter 4 — Comments
Chapter 5 — Formatting
Chapter 6 — Error Handling
Chapter 7 — Classes
Chapter 8 — SOLID: Single Responsibility Principle
Chapter 9 — SOLID: Open-Closed Principle
Chapter 10 — SOLID: Liskov Substitution Principle
Chapter 11 — SOLID: Interface Segregation Principle
Chapter 12 — SOLID: Dependency Inversion Principle
Chapter 13 — DRY, KISS, and YAGNI
Chapter 14 — Code Smells
Chapter 15 — Refactoring Techniques
Chapter 16 — Unit Testing Principles
Chapter 17 — Test-Driven Development
Chapter 18 — Clean Architecture Introduction
Chapter 19 — Layered Architecture
Chapter 20 — Hexagonal Architecture
Chapter 21 — Onion Architecture
Chapter 22 — Domain-Driven Design
Chapter 23 — Bounded Contexts
Chapter 24 — Dependency Injection
Chapter 25 — Design for Change
Chapter 26 — API Design Principles
Chapter 27 — Logging and Observability

Chapter 28 — Technical Debt
Chapter 29 — Code Review Culture
Chapter 30 — Legacy Code
Chapter 31 — Project Refactoring
Chapter 32 — Glossary

Chapter 1: Why Clean Code Matters

What You Will Learn

- What clean code actually means and why it matters more than clever code
- The real cost of messy code and how technical debt compounds over time
- Why developers spend 10 times more time reading code than writing it
- The Boy Scout Rule and how it prevents codebases from rotting
- How to recognize the warning signs of a codebase heading toward disaster
- What this book will teach you and how to apply it immediately

Why This Chapter Matters

Every software project starts with enthusiasm. The first few hundred lines flow effortlessly. Features ship fast. The team feels productive. Then, somewhere around month six, something shifts. Simple changes take days instead of hours. Bug fixes introduce new bugs. New team members stare at the screen in confusion. The codebase has become a swamp.

This chapter explains why that happens, how to prevent it, and why investing in clean code is not a luxury but a survival strategy. If you understand the "why" before learning the "how," every technique in this book will stick with you permanently.

What Is Clean Code?

Clean code is code that other humans can read, understand, and maintain. That is the entire definition. Not code that is clever. Not code that uses the most advanced language features.

Not code that runs in the fewest lines possible. Code that a colleague -- or your future self six months from now -- can open and immediately understand.

Here is a quote that belongs on every developer's wall:

> "Any fool can write code that a computer can understand. Good programmers write code that humans can understand." -- Martin Fowler

The computer does not care about your variable names. It does not care about your indentation. It does not care if your function is 500 lines long or 5. It will execute whatever you give it. But your teammates care. Your future self cares. The developer who inherits your project after you leave absolutely cares.

Clean Code Is Not About Perfection

Clean code is not about writing perfect code on the first try. It is about writing code that communicates its intent clearly, that can be changed without fear, and that does not require an archaeological expedition to understand.

Here is how several respected programmers have described it:

```
+-------------------------------------------------------------------+
|                    What Is Clean Code?                             |
+-------------------------------------------------------------------+
|                                                                   |
|  Bjarne Stroustrup    "Elegant and efficient. Clean code does     |
|  (Creator of C++)      one thing well."                           |
|                                                                   |
|  Grady Booch          "Clean code reads like well-written         |
|  (Author of OOA&D)     prose."                                    |
|                                                                   |
|  Dave Thomas          "Clean code can be read and enhanced        |
|  (Pragmatic Prog.)     by a developer other than its author."     |
|                                                                   |
|  Michael Feathers     "Clean code looks like it was written       |
|  (Working w/ Legacy)   by someone who cares."                     |
|                                                                   |
|  Ward Cunningham      "You know you are reading clean code        |
|  (Wiki inventor)       when every routine you read turns out      |
|                        to be pretty much what you expected."       |
|                                                                   |
+-------------------------------------------------------------------+
```

Notice the common theme: readability, simplicity, and care. Nobody mentions performance optimization, design patterns, or line count.

The Real Cost of Messy Code

A Horror Story

Imagine this scenario. It is real, and variations of it happen at companies every single day.

A startup builds a product in six months. Three developers, working fast, shipping features. The code is messy -- they know it -- but they plan to "clean it up later." The product launches successfully. Users love it. The company hires five more developers to add features faster.

But faster never comes.

The new developers spend their first two weeks just trying to understand the codebase. There are no tests. Variable names like `x`, `tmp`, and `data2` are everywhere. Functions are hundreds of lines long. One function called `processStuff()` handles user authentication, database queries, email sending, and logging -- all in 400 lines of spaghetti code.

```
Timeline of a Codebase Disaster
================================

Month 1-6:   [====] Fast development, messy code
             "We will clean it up later."

Month 7-9:   [======] New hires confused, slow onboarding
             "The code is a bit hard to follow."

Month 10-14: [==========] Bug fixes create new bugs
             "I am afraid to touch that file."

Month 15-18: [================] Features take 5x longer
             "We need to rewrite everything."

Month 19-24: [========================] Rewrite begins while
             maintaining old system. Team burns out.

Month 25+:   [====] Rewrite abandoned. Company struggles.
             "We should have written it clean from the start."
```

The cost of messy code is not a one-time payment. It is compound interest on a debt that grows every single day.

Technical Debt: The Snowball Effect

Technical debt is a metaphor borrowed from finance. When you write messy code to save time now, you are borrowing from the future. Like financial debt, it accumulates interest.

```
The Technical Debt Snowball
===========================

     Small mess          Bigger mess         Unmaintainable
     (easy to fix)       (painful to fix)    (rewrite needed)
         .                  ...                 .........
        /|\                /|||\               //|||||||\\\
       / | \              / ||| \             // ||||||| \\\
      /  |  \            /  |||  \           //  |||||||  \\\
     -------            ----------         ------------------

     Day 1               Month 6             Year 2
     "I will fix         "It would take      "Nobody knows
      it tomorrow"        a week to fix"      how this works"
```

Here is what technical debt looks like in practice:

BEFORE: Quick and dirty (borrowing from the future)

Java:

```java
// "It works, ship it!"
public class U {
    public static int c(String s, String t) {
        int r = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == ' ') {
                String w = s.substring(r, i);
                if (w.equals(t)) return i;
                r = i + 1;
            }
        }
        return -1;
    }
}
```

Python:

```python
# "It works, ship it!"
class U:
    @staticmethod
    def c(s, t):
        r = 0
        for i in range(len(s)):
            if s[i] == ' ':
                w = s[r:i]
                if w == t:
                    return i
                r = i + 1
        return -1
```

Can you tell what this code does? What is `U`? What is `c`? What do `s`, `t`, `r`, and `w` mean? What happens if the input is null or empty? This code "works," but it is a ticking time bomb.

AFTER: Clean and clear (investing in the future)

Java:

```java
public class TextSearcher {

    public static int findWordPosition(String text, String targetWord) {
        if (text == null || targetWord == null) {
            return -1;
        }

        String[] words = text.split(" ");
        for (int i = 0; i < words.length; i++) {
            if (words[i].equals(targetWord)) {
                return i;
            }
        }
        return -1;
    }
}
```

Python:

```python
class TextSearcher:

    @staticmethod
    def find_word_position(text: str, target_word: str) -> int:
        if not text or not target_word:
            return -1

        words = text.split(" ")
        for index, word in enumerate(words):
            if word == target_word:
                return index
        return -1
```

Same functionality. But now any developer can understand it in seconds. The class name tells you what it does. The method name tells you what it does. The parameter names tell you what they expect. The code is self-documenting.

The Numbers Do Not Lie

Here are real-world statistics that should make every developer pay attention:

```
+-----------------------------------------------------------------+
|              The Cost of Messy Code                             |
+-----------------------------------------------------------------+
|                                                                 |
|  Fixing a bug in production costs 100x more than fixing         |
|  it during development.                                         |
|                                                                 |
|  Developers spend 60-80% of their time understanding            |
|  existing code, not writing new code.                           |
|                                                                 |
|  A messy codebase reduces team productivity by 40-60%           |
|  within the first year.                                         |
|                                                                 |
|  The average cost of a full rewrite is 2-3x the original        |
|  development cost.                                              |
|                                                                 |
+-----------------------------------------------------------------+
```

The Reading-to-Writing Ratio

This is one of the most important concepts in all of software development, and most developers have never heard of it.

You spend 10 times more time reading code than writing it.

Think about your typical day. You open a file to make a change. Before you type a single character, you read the function you need to modify. You read the functions it calls. You read the class it belongs to. You search for where this function is called from. You read the tests. You read the related configuration. You trace the data flow through multiple files.

All of that is reading. The actual change might be five lines of code. But understanding where those five lines go and what they should do required reading hundreds of lines.

```
A Typical Day of "Writing" Code
===============================

  9:00 AM  [READING]  Open ticket, read requirements
  9:15 AM  [READING]  Find the relevant file
  9:20 AM  [READING]  Read the function to modify
  9:35 AM  [READING]  Read related functions
  9:50 AM  [READING]  Read tests to understand expected behavior
 10:05 AM  [READING]  Read another module that interacts with this one
 10:20 AM  [READING]  Read documentation / comments
 10:30 AM  [WRITING]  Write 12 lines of code       <--- This part
 10:35 AM  [READING]  Read the diff, review your change
```

```
10:40 AM  [READING]  Read test output
10:45 AM  [WRITING]  Fix a small issue, write 3 more lines
10:50 AM  [READING]  Read the final diff before committing
```

The implication is profound: making code easier to read makes everything faster. Not just for you, but for every developer who touches that code after you. If ten developers each save 15 minutes of reading time because your code is clean, you have saved 150 minutes of human productivity from one small investment.

What This Means for You

Every time you are tempted to use a cryptic variable name to save a few keystrokes, remember: you are optimizing the 10% (writing) at the expense of the 90% (reading). That is a bad trade.

BEFORE: Optimized for writing speed

Java:

```java
public double calc(List<E> el) {
    double t = 0;
    for (E e : el) {
        t += e.p * e.q;
        if (e.q > 10) t *= 0.9;
    }
    return t;
}
```

Python:

```python
def calc(el):
    t = 0
    for e in el:
        t += e.p * e.q
        if e.q > 10:
            t *= 0.9
    return t
```

AFTER: Optimized for reading speed

Java:

```java
public double calculateTotalPrice(List<OrderItem> items) {
    double totalPrice = 0;
    for (OrderItem item : items) {
        totalPrice += item.getPrice() * item.getQuantity();
        if (item.getQuantity() > BULK_DISCOUNT_THRESHOLD) {
            totalPrice *= BULK_DISCOUNT_RATE;
        }
    }
    return totalPrice;
}
```

Python:

```python
BULK_DISCOUNT_THRESHOLD = 10
BULK_DISCOUNT_RATE = 0.9

def calculate_total_price(items: list[OrderItem]) -> float:
    total_price = 0.0
    for item in items:
        total_price += item.price * item.quantity
        if item.quantity > BULK_DISCOUNT_THRESHOLD:
            total_price *= BULK_DISCOUNT_RATE
    return total_price
```

The second version takes slightly longer to type. But it saves minutes of reading time every single time someone looks at it. Over the lifetime of a project, that adds up to days or weeks of saved time.

The Boy Scout Rule

The Boy Scouts have a simple rule: "Always leave the campground cleaner than you found it."

Applied to code, this becomes: Always leave the code cleaner than you found it.

You do not need to refactor the entire codebase. You do not need to spend a week on a cleanup sprint. You just need to make one small improvement every time you touch a file.

```
The Boy Scout Rule in Practice
==============================

Before you arrived:
    def calc_val(x, y, z):    # Unclear name
        return x * y - z
```

```
Your task: Fix a bug where discount is applied wrong.

What most developers do:
    def calc_val(x, y, z):    # Fix the bug, leave the mess
        return x * y * z      # Changed - to *

What a Boy Scout developer does:
    def calculate_discounted_price(    # Renamed for clarity
        base_price: float,             # Renamed parameter
        quantity: int,                 # Renamed parameter
        discount: float                # Renamed parameter
    ) -> float:
        return base_price * quantity * discount  # Fixed bug too
```

Over time, thousands of small improvements transform a messy codebase into a clean one. It is like compound interest, but working in your favor instead of against you.

```
The Compound Effect of the Boy Scout Rule
=========================================

Without Boy Scout Rule:

Code Quality  |
  100% |*
       | *
       |  *
       |   *
       |     *
       |       *
       |          *
       |              *
       |                   *
    0% |_________________________*____
       Start                    Time


With Boy Scout Rule:

Code Quality  |
  100% |          *  *  *  *  *  *  *
       |       *
       |     *
       |   *
       |  *
       | *
       |*
       |
       |
    0% |_____________________________
       Start                    Time
```

Practical Boy Scout Improvements

Here are small improvements you can make every time you touch a file:

1. Rename one unclear variable -- `d` becomes `elapsedDays`
2. Extract one long expression into a well-named variable
3. Add a type hint to a function parameter
4. Remove one dead comment that describes what the code obviously does
5. Break one long function into two smaller ones
6. Remove one unused import

None of these take more than two minutes. All of them make the code better for the next person.

Recognizing Messy Code: Warning Signs

How do you know when a codebase is heading toward trouble? Here are the warning signs:

```
+-------------------------------------------------------------------+
|              Code Smell Warning Signs                             |
+-------------------------------------------------------------------+
|                                                                   |
|  Level 1: Yellow Flags                                            |
|  - Variable names like x, tmp, data, result                      |
|  - Functions longer than 30 lines                                 |
|  - Comments that explain "what" instead of "why"                  |
|  - Copy-pasted code blocks                                        |
|                                                                   |
|  Level 2: Orange Flags                                            |
|  - Functions with more than 3 parameters                          |
|  - Deeply nested if/else (3+ levels)                              |
|  - Classes longer than 300 lines                                  |
|  - No tests                                                       |
|                                                                   |
|  Level 3: Red Flags                                               |
|  - "I am afraid to change this code"                              |
|  - Nobody understands how module X works                          |
|  - Simple changes take days instead of hours                      |
|  - Bug fixes regularly introduce new bugs                         |
|  - New team members take months to become productive              |
|                                                                   |
+-------------------------------------------------------------------+
```

A Real-World Comparison

Let us look at a complete example that shows the difference between messy code and clean code. This is a simple feature: process an order and send a confirmation.

BEFORE: The "it works" approach

Java:

```java
public class O {
    public boolean p(Map<String, Object> d) {
        // check stuff
        if (d != null && d.get("items") != null) {
            List<Map<String, Object>> items =
                (List<Map<String, Object>>) d.get("items");
            double t = 0;
            for (Map<String, Object> i : items) {
                double pr = (double) i.get("p");
                int q = (int) i.get("q");
                t += pr * q;
                // apply discount
                if (q > 10) {
                    t = t - (t * 0.1);
                } else if (q > 5) {
                    t = t - (t * 0.05);
                }
            }
            // tax
            t = t * 1.08;
            // save
            try {
                DB db = new DB();
                db.connect("prod-server");
                db.save("orders", d);
                db.save("totals", t);
                // send email
                Email e = new Email();
                e.setTo((String) d.get("email"));
                e.setSubject("Order");
                e.setBody("Total: " + t);
                e.send();
                return true;
            } catch (Exception e) {
                System.out.println("error: " + e);
                return false;
            }
        }
        return false;
    }
}
```

Python:

```python
class O:
    def p(self, d):
        # check stuff
        if d is not None and d.get("items") is not None:
            items = d["items"]
            t = 0
            for i in items:
                pr = i["p"]
                q = i["q"]
                t += pr * q
                # apply discount
                if q > 10:
                    t = t - (t * 0.1)
                elif q > 5:
                    t = t - (t * 0.05)
            # tax
            t = t * 1.08
            # save
            try:
                db = DB()
                db.connect("prod-server")
                db.save("orders", d)
                db.save("totals", t)
                # send email
                e = Email()
                e.to = d["email"]
                e.subject = "Order"
                e.body = f"Total: {t}"
                e.send()
                return True
            except Exception as e:
                print(f"error: {e}")
                return False
        return False
```

Problems with this code:

- Class `O` and method `p` reveal nothing about purpose
- Uses raw Maps/dicts instead of proper types
- Single-letter variable names everywhere
- Mixes business logic, database access, and email sending
- Hard-coded database server and tax rate
- Swallows exceptions with a useless print statement
- No input validation
- Discount logic is buried and unexplained

AFTER: The clean approach

Java:

```java
public class OrderProcessor {

    private static final double TAX_RATE = 0.08;
    private static final int LARGE_BULK_THRESHOLD = 10;
    private static final int SMALL_BULK_THRESHOLD = 5;
    private static final double LARGE_BULK_DISCOUNT = 0.10;
    private static final double SMALL_BULK_DISCOUNT = 0.05;

    private final OrderRepository orderRepository;
    private final EmailService emailService;

    public OrderProcessor(OrderRepository orderRepository,
                          EmailService emailService) {
        this.orderRepository = orderRepository;
        this.emailService = emailService;
    }

    public OrderResult processOrder(Order order) {
        validateOrder(order);
        double total = calculateTotal(order.getItems());
        orderRepository.save(order, total);
        emailService.sendOrderConfirmation(order.getCustomerEmail(), total);
        return new OrderResult(order.getId(), total);
    }

    private void validateOrder(Order order) {
        if (order == null || order.getItems().isEmpty()) {
            throw new InvalidOrderException("Order must have at least one item");
        }
    }

    private double calculateTotal(List<OrderItem> items) {
        double subtotal = 0;
        for (OrderItem item : items) {
            subtotal += calculateItemTotal(item);
        }
        return applyTax(subtotal);
    }

    private double calculateItemTotal(OrderItem item) {
        double itemTotal = item.getPrice() * item.getQuantity();
        double discount = determineDiscount(item.getQuantity());
        return itemTotal * (1 - discount);
    }

    private double determineDiscount(int quantity) {
        if (quantity > LARGE_BULK_THRESHOLD) {
            return LARGE_BULK_DISCOUNT;
        } else if (quantity > SMALL_BULK_THRESHOLD) {
            return SMALL_BULK_DISCOUNT;
        }
        return 0;
    }
```

```
    private double applyTax(double subtotal) {
        return subtotal * (1 + TAX_RATE);
    }
}
```

Python:

```python
from dataclasses import dataclass

TAX_RATE = 0.08
LARGE_BULK_THRESHOLD = 10
SMALL_BULK_THRESHOLD = 5
LARGE_BULK_DISCOUNT = 0.10
SMALL_BULK_DISCOUNT = 0.05


@dataclass
class OrderResult:
    order_id: str
    total: float


class OrderProcessor:

    def __init__(self, order_repository, email_service):
        self.order_repository = order_repository
        self.email_service = email_service

    def process_order(self, order) -> OrderResult:
        self._validate_order(order)
        total = self._calculate_total(order.items)
        self.order_repository.save(order, total)
        self.email_service.send_order_confirmation(order.customer_email, total)
        return OrderResult(order.id, total)

    def _validate_order(self, order):
        if order is None or not order.items:
            raise InvalidOrderError("Order must have at least one item")

    def _calculate_total(self, items) -> float:
        subtotal = sum(self._calculate_item_total(item) for item in items)
        return self._apply_tax(subtotal)

    def _calculate_item_total(self, item) -> float:
        item_total = item.price * item.quantity
        discount = self._determine_discount(item.quantity)
        return item_total * (1 - discount)

    def _determine_discount(self, quantity: int) -> float:
        if quantity > LARGE_BULK_THRESHOLD:
            return LARGE_BULK_DISCOUNT
        elif quantity > SMALL_BULK_THRESHOLD:
```

```
            return SMALL_BULK_DISCOUNT
        return 0.0

    def _apply_tax(self, subtotal: float) -> float:
        return subtotal * (1 + TAX_RATE)
```

The clean version is longer, yes. But look at what you gain:

- Every class, method, and variable name tells you exactly what it does
- Each method does one thing and does it well
- Business rules (tax rate, discount thresholds) are named constants
- Dependencies are injected, making testing easy
- Errors are handled with meaningful exceptions
- You can understand any single method without reading any other method

---

What This Book Covers

This book is organized as a journey from the fundamentals of clean code to the architecture of entire systems.

```
Book Structure
===============

Part 1: Clean Code Fundamentals (Chapters 1-7)
+-----------------------------------------------+
| Naming | Functions | Comments | Formatting    |
| Error Handling | Classes                      |
+-----------------------------------------------+
           |
           v
Part 2: Design Principles (Chapters 8-14)
+-----------------------------------------------+
| SOLID Principles | DRY, KISS, YAGNI           |
| Code Smells | Refactoring                     |
+-----------------------------------------------+
           |
           v
Part 3: Testing (Chapters 15-17)
+-----------------------------------------------+
| Unit Testing | Test-Driven Development        |
+-----------------------------------------------+
           |
           v
Part 4: Architecture (Chapters 18-26)
```

```
+-----------------------------------------------+
| Clean Architecture | Layered | Hexagonal      |
| Onion | DDD | Dependency Injection | APIs     |
+-----------------------------------------------+
           |
           v
Part 5: Professional Practice (Chapters 27–32)
+-----------------------------------------------+
| Logging | Technical Debt | Code Reviews        |
| Legacy Code | Project Refactoring | Glossary   |
+-----------------------------------------------+
```

Every chapter follows the same practical format:

- Concepts explained simply with real-world analogies
- Bad code shown first, then transformed into clean code
- Both Java and Python examples side by side
- Practice exercises so you can apply what you learn immediately

---

Common Mistakes

1. "I will clean it up later." Later never comes. Write it clean the first time or accept that it will stay messy forever.
2. "Clean code is too slow to write." It takes slightly longer to write but saves enormous time in reading, debugging, and maintaining. The math is overwhelmingly in favor of clean code.
3. "Performance matters more than readability." In 99% of cases, readability matters more. Optimize only after profiling identifies an actual bottleneck. Premature optimization is the root of all evil.
4. "My code is obvious to me." It is obvious to you right now, while the context is fresh. It will not be obvious to you in six months, and it is definitely not obvious to your teammates today.
5. "We do not have time for clean code." You do not have time for messy code. Every minute spent deciphering messy code is a minute not spent delivering value.

Best Practices

1. Treat code as communication. You are not writing instructions for a machine. You are writing a document that other humans will read and modify.
2. Apply the Boy Scout Rule daily. Every time you touch a file, leave it a little cleaner. Small improvements compound into massive gains.
3. Name things as if the next reader is a beginner. Do not assume context. Spell it out. Use full words. Be explicit.
4. Write code that does not need comments. If your code requires a comment to explain what it does, refactor the code until the comment is unnecessary.
5. Invest in readability over cleverness. The cleverest code is the hardest to maintain. Simple, boring code is a gift to your future self.
6. Measure the cost of messy code. Track how long bug fixes take, how long onboarding takes, and how often changes break unrelated features. These metrics make the case for clean code better than any argument.

Quick Summary

Clean code is code that humans can read and maintain. It is not about perfection or clever tricks. It is about clarity, simplicity, and care.

Messy code creates technical debt that compounds over time, slowing development and increasing bugs. Since developers spend 10 times more time reading code than writing it, investing in readability pays massive dividends.

The Boy Scout Rule -- always leave the code cleaner than you found it -- is the simplest and most effective strategy for improving a codebase over time.

Key Points

- Clean code is defined by readability and maintainability, not by cleverness or brevity
- Technical debt is like financial debt: it accumulates compound interest that eventually becomes crushing
- The 10:1 reading-to-writing ratio means readability improvements have 10x impact
- The Boy Scout Rule (leave code cleaner than you found it) prevents codebases from rotting

- Clean code takes slightly longer to write but saves enormous time in maintenance and debugging
- Every code smell warning sign you ignore today becomes a bigger problem tomorrow
- Clean code is not a luxury or a nice-to-have; it is a professional responsibility

---

Practice Questions

1. You are reviewing a pull request and find a function called `doStuff()` with single-letter variable names. The author says "it works and we need to ship today." What would you say, and why?
2. Your team lead argues that spending time on clean code is wasteful because "we can always refactor later." Using the technical debt metaphor, explain why this reasoning is flawed.
3. Calculate the time savings: if a function is read by 8 developers, 3 times each per month, and clean code saves 5 minutes per reading versus messy code, how many hours per year does clean code save for just that one function?
4. A developer on your team writes very concise code using advanced language features that most team members do not understand. The code is technically correct and performant. Is this clean code? Why or why not?
5. You join a project with 50,000 lines of messy code. Applying the Boy Scout Rule, describe a realistic plan for improving the codebase over six months without a dedicated refactoring sprint.

Exercises

Exercise 1: Identify the Mess

Look at this code and list every problem you can find. Then rewrite it to be clean.

Java:

```java
public class D {
    public static String f(int a, int b, int c) {
        String r = "";
```

```
        if (a < 10) r += "0";
        r += a + "/";
        if (b < 10) r += "0";
        r += b + "/";
        r += c;
        return r;
    }
}
```

Python:

```python
class D:
    @staticmethod
    def f(a, b, c):
        r = ""
        if a < 10:
            r += "0"
        r += str(a) + "/"
        if b < 10:
            r += "0"
        r += str(b) + "/"
        r += str(c)
        return r
```

Hint: What do `D`, `f`, `a`, `b`, `c`, and `r` represent? What would make this code instantly understandable?

Exercise 2: The Boy Scout Improvement

Take a file from any project you have worked on (or the exercise above) and make exactly three Boy Scout improvements without changing its behavior. Document what you changed and why.

Exercise 3: Cost of Confusion

Find a piece of code (yours or open source) that took you more than 5 minutes to understand. Write down:

- How long it took you to understand it
- What made it confusing
- How you would rewrite it to be immediately clear
- Estimate how many other developers have also spent time being confused by this same code

What Is Next?

Now that you understand why clean code matters, it is time to learn the first and most impactful skill: choosing good names. Chapter 2 will teach you how meaningful names can transform confusing code into self-documenting code. You will see over ten real examples of bad names transformed into good ones, and you will learn the rules that make naming feel effortless.

Good naming is the single highest-leverage clean code skill. If you learn nothing else from this book, learn to name things well.

Chapter 2: Meaningful Names

What You Will Learn

- How to choose names that reveal the intention behind your code
- Why misleading names are worse than unclear names
- The difference between meaningful distinctions and noise words
- Rules for naming classes, methods, variables, and constants
- How to avoid common naming traps that confuse other developers
- Over ten real before/after naming transformations

Why This Chapter Matters

Naming is the most frequent activity in programming. You name variables, functions, classes, packages, files, directories, parameters, and constants. You do it so often that you might not think much about it. But names are the primary way code communicates with human readers.

A good name eliminates the need for a comment. A bad name creates confusion that comments cannot fully repair. Since you spend 10 times more time reading code than writing it (as we learned in Chapter 1), every good name you choose saves reading time for every developer who encounters it -- including your future self.

Naming is hard. It is acknowledged as one of the two hardest problems in computer science (along with cache invalidation and off-by-one errors). But it gets easier with practice and a set of clear rules. This chapter gives you those rules.

Rule 1: Use Intention-Revealing Names

The name of a variable, function, or class should tell you three things:

1. Why it exists
2. What it does
3. How it is used

If a name requires a comment to explain it, the name is not good enough.

BEFORE: Names that hide intention

Java:

```java
int d; // elapsed time in days
```

Python:

```python
d = 0  # elapsed time in days
```

AFTER: Names that reveal intention

Java:

```java
int elapsedTimeInDays;
int daysSinceCreation;
int daysSinceModification;
int fileAgeInDays;
```

Python:

```python
elapsed_time_in_days = 0
days_since_creation = 0
days_since_modification = 0
file_age_in_days = 0
```

The comment is gone because the name carries all the information. Notice that each alternative name carries a slightly different meaning. The right choice depends on the specific context, and that precision is exactly the point.

A Complete Example

Here is a larger example that shows how intention-revealing names transform unreadable code into clear code.

BEFORE: What does this do?

Java:

```java
public List<int[]> getThem() {
    List<int[]> list1 = new ArrayList<>();
    for (int[] x : theList) {
        if (x[0] == 4) {
            list1.add(x);
        }
    }
    return list1;
}
```

Python:

```python
def get_them(self):
    list1 = []
    for x in the_list:
        if x[0] == 4:
            list1.append(x)
    return list1
```

This code is simple -- just a filtered list. But you cannot tell what it does without knowing:

- What is `theList`?
- What is the significance of `x[0]`?
- What does `4` mean?
- What do you do with the returned list?

Now imagine this is a minesweeper game:

AFTER: Intention revealed through names

Java:

```java
public List<Cell> getFlaggedCells() {
    List<Cell> flaggedCells = new ArrayList<>();
    for (Cell cell : gameBoard) {
        if (cell.isFlagged()) {
            flaggedCells.add(cell);
        }
    }
    return flaggedCells;
}
```

Python:

```python
def get_flagged_cells(self) -> list[Cell]:
    flagged_cells = []
    for cell in self.game_board:
        if cell.is_flagged():
            flagged_cells.append(cell)
    return flagged_cells
```

The code has not become more complex. It has become clear. Every name tells you what it represents and why it matters.

Rule 2: Avoid Disinformation

Do not use names that mean something different from what they actually represent. This is worse than unclear names because they actively mislead.

BEFORE: Disinforming names

Java:

```java
// BAD: It is not a List, it is a Map
Map<String, Account> accountList = new HashMap<>();

// BAD: hp could mean hit points, horsepower, or Hewlett-Packard
int hp = calculateValue();

// BAD: looks like the letter O or the number 0
int O = calculateWeight();
int l = 1;  // looks like the number 1
if (O == l) { ... }
```

Python:

```python
# BAD: It is not a list, it is a dictionary
account_list = {}

# BAD: hp is ambiguous
hp = calculate_value()

# BAD: O looks like 0, l looks like 1
```

```
O = calculate_weight()
l = 1
if O == l: ...
```

AFTER: Honest names

Java:

```java
Map<String, Account> accountsByName = new HashMap<>();
int horsePower = calculateHorsePower();
int outputWeight = calculateWeight();
int lineCount = 1;
```

Python:

```python
accounts_by_name = {}
horse_power = calculate_horse_power()
output_weight = calculate_weight()
line_count = 1
```

```
+-------------------------------------------------------------+
|              Common Disinformation Traps                    |
+-------------------------------------------------------------+
|                                                             |
|  accountList     -->  Use when it IS a List/list            |
|                       Otherwise: accounts, accountMap,      |
|                       accountsByName                        |
|                                                             |
|  userGroup       -->  Is it a UI group? Permission group?   |
|                       Team? Be specific.                    |
|                                                             |
|  controller      -->  MVC controller? Game controller?      |
|                       Hardware controller? Name it fully.   |
|                                                             |
|  XYZController       Avoid names that differ in only        |
|  XYZControllerFor    small ways -- they are easy to         |
|  XYZHandlerController  confuse at a glance.                 |
|                                                             |
+-------------------------------------------------------------+
```

Rule 3: Make Meaningful Distinctions

If names must be different, they must also mean something different. Do not add noise words just to satisfy the compiler.

BEFORE: Noise word distinctions

Java:

```java
// What is the difference between these?
public void copyChars(char[] a1, char[] a2) {
    for (int i = 0; i < a1.length; i++) {
        a2[i] = a1[i];
    }
}

// What is the difference between these classes?
class ProductInfo { }
class ProductData { }
class ProductDetails { }

// What is the difference between these methods?
String getName();
String getNameString();  // What else would it be? An integer?

// What is the difference between these variables?
String customerName;
String customerNameVariable;
```

Python:

```python
# What is the difference between a1 and a2?
def copy_chars(a1, a2):
    for i in range(len(a1)):
        a2[i] = a1[i]

# What is the difference?
class ProductInfo: ...
class ProductData: ...
class ProductDetails: ...
```

AFTER: Meaningful distinctions

Java:

```java
public void copyChars(char[] source, char[] destination) {
    for (int i = 0; i < source.length; i++) {
        destination[i] = source[i];
    }
}
```

Python:

```python
def copy_chars(source: list[str], destination: list[str]):
    for i in range(len(source)):
        destination[i] = source[i]
```

The rule is simple: if you cannot explain the difference between two names without looking at their implementations, at least one of the names is wrong.

```
Noise Words That Add No Meaning
================================

  "Info"     -->  ProductInfo vs Product? No real difference.
  "Data"     -->  UserData vs User? No real difference.
  "Object"   -->  ConfigObject vs Config? No real difference.
  "Variable" -->  nameVariable vs name? Obvious padding.
  "String"   -->  nameString vs name? Type is already clear.
  "Manager"  -->  Sometimes useful, but often a sign that the
                  class does too many things.
```

Rule 4: Use Pronounceable Names

If you cannot pronounce a name, you cannot discuss it without sounding foolish. This matters more than you might think, because programming is a social activity. You discuss code in reviews, pair programming, and meetings.

BEFORE: Unpronounceable names

Java:

```java
class DtaRcrd102 {
    private Date genymdhms;    // generation year month day hour minute second
    private Date modymdhms;    // modification timestamp
    private String pszqint;    // what?
}
```

Python:

```python
class DtaRcrd102:
    def __init__(self):
```

```
        self.genymdhms = None    # generation date
        self.modymdhms = None    # modification date
        self.pszqint = None      # ???
```

Try saying this in a code review: "Hey, look at the gen-why-em-dee-aitch-em-ess field on the dee-tee-ay-arr-see-arr-dee-one-oh-two class." Nobody wants to have that conversation.

AFTER: Pronounceable names

Java:

```java
class CustomerRecord {
    private Date generationTimestamp;
    private Date modificationTimestamp;
    private String recordId;
}
```

Python:

```python
class CustomerRecord:
    def __init__(self):
        self.generation_timestamp = None
        self.modification_timestamp = None
        self.record_id = None
```

Now you can say: "Hey, look at the generation timestamp on the customer record." Natural conversation.

Rule 5: Use Searchable Names

Single-letter names and numeric constants are impossible to search for in a codebase.

BEFORE: Unsearchable names

Java:

```java
for (int j = 0; j < 34; j++) {
    s += (t[j] * 4) / 5;
}
```

Python:

```python
for j in range(34):
    s += (t[j] * 4) / 5
```

Try searching for `j` in a project. You will find thousands of matches. Try searching for `34`. What does it mean? Why is it 34?

AFTER: Searchable names

Java:

```java
int realDaysPerIdealDay = 4;
int WORK_DAYS_PER_WEEK = 5;
int NUMBER_OF_TASKS = 34;

for (int taskIndex = 0; taskIndex < NUMBER_OF_TASKS; taskIndex++) {
    int realTaskDays = taskEstimate[taskIndex] * realDaysPerIdealDay;
    int realTaskWeeks = realTaskDays / WORK_DAYS_PER_WEEK;
    totalWeeks += realTaskWeeks;
}
```

Python:

```python
REAL_DAYS_PER_IDEAL_DAY = 4
WORK_DAYS_PER_WEEK = 5
NUMBER_OF_TASKS = 34

for task_index in range(NUMBER_OF_TASKS):
    real_task_days = task_estimate[task_index] * REAL_DAYS_PER_IDEAL_DAY
    real_task_weeks = real_task_days // WORK_DAYS_PER_WEEK
    total_weeks += real_task_weeks
```

Now you can search for `WORK_DAYS_PER_WEEK` and find every place it is used. If the value changes, you change it in one place.

```
Searchability Rule of Thumb
============================

  Single-letter variables  -->  ONLY acceptable as loop
                                counters in very short loops
                                (i, j in a 3-line loop)

  Magic numbers (7, 86400) -->  ALWAYS use named constants
                                SECONDS_PER_DAY = 86400
                                MAX_RETRY_COUNT = 7

  Short abbreviations      -->  Use full words unless the
                                abbreviation is universally
```

```
                                    understood (url, html, id)
```

Rule 6: Class Names Should Be Nouns

Classes represent things, so they should be named with nouns or noun phrases. Avoid verbs in class names.

BEFORE: Bad class names

Java:

```java
class ProcessOrders { }      // Verb -- sounds like a function
class ManageData { }         // Verb -- sounds like a function
class Calculate { }          // Verb -- sounds like a function
class Info { }               // Too vague -- info about what?
class Processor { }          // Too vague -- processes what?
class Data { }               // Too vague -- what data?
```

AFTER: Good class names

Java:

```java
class Order { }              // Clear noun
class OrderProcessor { }     // Noun phrase (the thing that processes)
class Customer { }           // Clear noun
class ShippingCalculator { } // Noun phrase
class PaymentGateway { }     // Noun phrase
class InvoiceRepository { }  // Noun phrase
```

Python:

```python
class Order: ...
class OrderProcessor: ...
class Customer: ...
class ShippingCalculator: ...
class PaymentGateway: ...
class InvoiceRepository: ...
```

Rule 7: Method Names Should Be Verbs

Methods represent actions, so they should be named with verbs or verb phrases.

BEFORE: Bad method names

Java:

```java
public double calculation();       // Noun -- what calculation?
public void data();                // Noun -- what about data?
public boolean valid();            // Adjective -- validate? is valid?
public User user(int id);          // Noun -- get user? create user?
```

AFTER: Good method names

Java:

```java
public double calculateTotal();
public void saveData();
public boolean isValid();
public User findUserById(int id);
```

Python:

```python
def calculate_total(self) -> float: ...
def save_data(self): ...
def is_valid(self) -> bool: ...
def find_user_by_id(self, user_id: int) -> User: ...
```

```
Method Naming Conventions
=========================

  Getters:     getName()  / get_name()
  Setters:     setName()  / set_name()
  Booleans:    isActive() / is_active()
               hasPermission() / has_permission()
               canExecute() / can_execute()
  Actions:     save(), delete(), send(), calculate()
  Conversions: toString() / to_string()
               toJson() / to_json()
  Factories:   createOrder() / create_order()
               fromJson() / from_json()
```

Rule 8: Do Not Be Cute

Resist the temptation to use clever, funny, or culturally specific names. Not everyone shares your sense of humor, and jokes get stale fast.

BEFORE: "Clever" names

Java:

```java
public void whack() { }          // means delete
public void eatMyShorts() { }    // means abort
public void nuke(List items) { } // means clear all
public int getCheeseburger() { } // means... what?
public void rockAndRoll() { }    // means start processing
```

AFTER: Professional names

Java:

```java
public void delete() { }
public void abort() { }
public void clearAll(List items) { }
public int getMenuItemCount() { }
public void startProcessing() { }
```

Python:

```python
def delete(self): ...
def abort(self): ...
def clear_all(self, items: list): ...
def get_menu_item_count(self) -> int: ...
def start_processing(self): ...
```

The rule: say what you mean. Mean what you say. Code is not the place for inside jokes.

Rule 9: One Word Per Concept

Pick one word for one abstract concept and stick with it throughout the codebase. Do not use `fetch`, `retrieve`, and `get` to mean the same thing in different classes.

BEFORE: Inconsistent terminology

Java:

```java
class UserController {
    public User fetchUser(int id) { ... }
}

class OrderController {
    public Order retrieveOrder(int id) { ... }
}

class ProductController {
    public Product getProduct(int id) { ... }
}

class InvoiceController {
    public Invoice obtainInvoice(int id) { ... }
}
```

A developer looking at this codebase will wonder: Is there a difference between fetch, retrieve, get, and obtain? Do they have different semantics? Different error handling? The answer is usually no -- they all mean the same thing, and the inconsistency creates unnecessary confusion.

AFTER: Consistent terminology

Java:

```java
class UserController {
    public User getUser(int id) { ... }
}

class OrderController {
    public Order getOrder(int id) { ... }
}

class ProductController {
    public Product getProduct(int id) { ... }
}

class InvoiceController {
    public Invoice getInvoice(int id) { ... }
}
```

Python:

```python
class UserController:
    def get_user(self, user_id: int) -> User: ...
```

```
class OrderController:
    def get_order(self, order_id: int) -> Order: ...

class ProductController:
    def get_product(self, product_id: int) -> Product: ...

class InvoiceController:
    def get_invoice(self, invoice_id: int) -> Invoice: ...
```

```
Establish a Project Vocabulary
==============================

  Pick ONE word for each concept:

  +-------------------+---------------------+
  | Concept           | Chosen Word         |
  +-------------------+---------------------+
  | Get single item   | get                 |
  | Get multiple items| list / findAll      |
  | Create new        | create              |
  | Modify existing   | update              |
  | Remove            | delete              |
  | Check existence   | exists              |
  | Validate          | validate            |
  | Convert           | toX / fromX         |
  +-------------------+---------------------+

  Document this in your team's style guide.
```

Rule 10: Use Solution Domain Names When Appropriate

Your readers are programmers. Use computer science terms when they are the clearest option. Do not invent new names for well-known patterns.

Java:

```java
// GOOD: Programmers know what these terms mean
class AccountRepository { }     // Repository pattern
class OrderFactory { }          // Factory pattern
class EventQueue { }            // Queue data structure
class UserObserver { }          // Observer pattern
```

```
class JobScheduler { }          // Scheduler concept
class RequestVisitor { }        // Visitor pattern
```

Python:

```python
class AccountRepository: ...    # Repository pattern
class OrderFactory: ...         # Factory pattern
class EventQueue: ...           # Queue data structure
class UserObserver: ...         # Observer pattern
class JobScheduler: ...         # Scheduler concept
```

If there is no programmer term that fits, use a name from the business domain. If the users call it a "shopping cart," call it `ShoppingCart`, not `ItemAccumulator`.

The Big Before/After: Ten Naming Fixes

Here is a comprehensive example showing ten bad naming choices fixed in a single class.

BEFORE: A naming disaster

Java:

```java
public class Mgr {                                      // 1. Abbreviated class name
    private List<String[]> lst;                          // 2. Cryptic variable name
    private int cnt;                                     // 3. Unnecessary abbreviation

    public void proc(String[] a) {                       // 4. Verb abbreviation, unclear...
        String n = a[0];                                 // 5. Single letter variable
        String val = a[1];                               // 6. Abbreviated, ambiguous
        int flag = Integer.parseInt(a[2]);               // 7. "flag" means nothing

        if (flag == 1) {                                 // 8. Magic number
            for (int j = 0; j < lst.size(); j++) {      // 9. Generic loop variable is OK...
                String[] tmp = lst.get(j);               // 10. "tmp" is lazy
                if (tmp[0].equals(n)) {
                    tmp[1] = val;
                    cnt++;
                    return;
                }
            }
        }
    }
}
```

Python:

```python
class Mgr:                                              # 1. Abbreviated class name
    def __init__(self):
        self.lst = []                                    # 2. Cryptic variable name
        self.cnt = 0                                     # 3. Unnecessary abbreviation

    def proc(self, a):                                   # 4. Verb abbreviation, unclear ...
        n = a[0]                                         # 5. Single letter variable
        val = a[1]                                       # 6. Abbreviated, ambiguous
        flag = int(a[2])                                 # 7. "flag" means nothing

        if flag == 1:                                    # 8. Magic number
            for j in range(len(self.lst)):               # 9. Generic loop variable
                tmp = self.lst[j]                        # 10. "tmp" is lazy
                if tmp[0] == n:
                    tmp[1] = val
                    self.cnt += 1
                    return
```

AFTER: Every name tells a story

Java:

```java
public class EmployeeManager {                                      // 1. Full descriptiv...
    private List<Employee> employees;                               // 2. Clear collectio...
    private int updateCount;                                        // 3. Full word, clea...

    private static final int UPDATE_OPERATION = 1;                  // 8. Named constant

    public void processEmployeeUpdate(String[] rawRecord) {         // 4. Full verb phras...
        String employeeName = rawRecord[0];                         // 5. Descriptive var...
        String newDepartment = rawRecord[1];                        // 6. Specific and clear
        int operationType = Integer.parseInt(rawRecord[2]);         // 7. Meaningful name

        if (operationType == UPDATE_OPERATION) {                    // 8. Reads like English
            for (Employee employee : employees) {                   // 9+10. Proper type,...
                if (employee.getName().equals(employeeName)) {
                    employee.setDepartment(newDepartment);
                    updateCount++;
                    return;
                }
            }
        }
    }
}
```

Python:

```python
class EmployeeManager:                                          # 1. Full descriptive...
    UPDATE_OPERATION = 1                                        # 8. Named constant

    def __init__(self):
        self.employees: list[Employee] = []                     # 2. Clear collection...
        self.update_count: int = 0                              # 3. Full word, clear...

    def process_employee_update(self, raw_record: list[str]):   # 4. Full verb phrase
        employee_name = raw_record[0]                           # 5. Descriptive vari...
        new_department = raw_record[1]                          # 6. Specific and clear
        operation_type = int(raw_record[2])                     # 7. Meaningful name

        if operation_type == self.UPDATE_OPERATION:              # 8. Reads like English
            for employee in self.employees:                     # 9+10. Clear loop va...
                if employee.name == employee_name:
                    employee.department = new_department
                    self.update_count += 1
                    return
```

Here is a summary of all ten fixes:

```
+----+------------------------+-------------------------+
| #  | Before                 | After                   |
+----+------------------------+-------------------------+
|  1 | Mgr                    | EmployeeManager         |
|  2 | lst                    | employees               |
|  3 | cnt                    | updateCount             |
|  4 | proc(String[] a)       | processEmployeeUpdate() |
|  5 | n                      | employeeName            |
|  6 | val                    | newDepartment           |
|  7 | flag                   | operationType           |
|  8 | 1 (magic number)       | UPDATE_OPERATION        |
|  9 | j (loop counter)       | (enhanced for loop)     |
| 10 | tmp                    | employee                |
+----+------------------------+-------------------------+
```

Real-World Scenario: Naming in a REST API

Here is a realistic scenario showing how naming affects an entire layer of an application.

BEFORE: An API controller with poor names

Java:

```java
@RestController
public class UC {

    @Autowired
    private Svc s;

    @GetMapping("/u")
    public Resp g(@RequestParam int i) {
        Obj o = s.f(i);
        if (o == null) {
            return new Resp(404, "nf");
        }
        return new Resp(200, o.conv());
    }

    @PostMapping("/u")
    public Resp c(@RequestBody Map<String, Object> d) {
        int r = s.mk(d);
        return new Resp(201, String.valueOf(r));
    }

    @DeleteMapping("/u")
    public Resp d(@RequestParam int i) {
        boolean ok = s.rm(i);
        return ok ? new Resp(200, "ok") : new Resp(404, "nf");
    }
}
```

Python (Flask):

```python
class UC:
    def __init__(self, s):
        self.s = s

    def g(self, i):
        o = self.s.f(i)
        if o is None:
            return {"status": 404, "msg": "nf"}
        return {"status": 200, "data": o.conv()}

    def c(self, d):
        r = self.s.mk(d)
        return {"status": 201, "id": str(r)}

    def d(self, i):
        ok = self.s.rm(i)
        return {"status": 200} if ok else {"status": 404, "msg": "nf"}
```

AFTER: Clear, professional API controller

Java:

```java
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{userId}")
    public ResponseEntity<UserResponse> getUser(@PathVariable int userId) {
        User user = userService.findById(userId);
        if (user == null) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(user.toResponse());
    }

    @PostMapping
    public ResponseEntity<UserResponse> createUser(
            @RequestBody CreateUserRequest request) {
        User createdUser = userService.create(request);
        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(createdUser.toResponse());
    }

    @DeleteMapping("/{userId}")
    public ResponseEntity<Void> deleteUser(@PathVariable int userId) {
        boolean deleted = userService.deleteById(userId);
        if (!deleted) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok().build();
    }
}
```

Python (Flask):

```python
class UserController:

    def __init__(self, user_service: UserService):
        self.user_service = user_service

    def get_user(self, user_id: int) -> dict:
        user = self.user_service.find_by_id(user_id)
        if user is None:
            return {"error": "User not found"}, 404
        return user.to_response(), 200

    def create_user(self, create_request: dict) -> dict:
```

```
        created_user = self.user_service.create(create_request)
        return created_user.to_response(), 201

    def delete_user(self, user_id: int) -> dict:
        deleted = self.user_service.delete_by_id(user_id)
        if not deleted:
            return {"error": "User not found"}, 404
        return {}, 200
```

The difference is dramatic. The clean version reads like documentation. Every name tells you exactly what is happening, what the inputs are, and what the outputs mean.

Naming Conventions by Language

Java and Python have different naming conventions. Respecting these conventions makes your code feel natural to developers in each language.

```
+-------------------+-------------------+-------------------+
| Element           | Java Convention   | Python Convention  |
+-------------------+-------------------+-------------------+
| Class             | PascalCase        | PascalCase         |
|                   | OrderProcessor    | OrderProcessor     |
+-------------------+-------------------+-------------------+
| Method/Function   | camelCase         | snake_case         |
|                   | calculateTotal()  | calculate_total()  |
+-------------------+-------------------+-------------------+
| Variable          | camelCase         | snake_case         |
|                   | totalPrice        | total_price        |
+-------------------+-------------------+-------------------+
| Constant          | UPPER_SNAKE_CASE  | UPPER_SNAKE_CASE   |
|                   | MAX_RETRY_COUNT   | MAX_RETRY_COUNT    |
+-------------------+-------------------+-------------------+
| Package/Module    | lowercase         | lowercase          |
|                   | com.app.orders    | orders             |
+-------------------+-------------------+-------------------+
| Boolean variable  | isActive          | is_active          |
|                   | hasPermission     | has_permission     |
+-------------------+-------------------+-------------------+
| Private member    | this.field        | _field (prefix _)  |
|                   | (no prefix)       | self._field        |
+-------------------+-------------------+-------------------+
| Interface (Java)  | Readable          | N/A (use ABC or    |
|                   | Serializable      |  Protocol)         |
+-------------------+-------------------+-------------------+
```

Common Mistakes

1. Using abbreviations to save typing. `usr` instead of `user`, `msg` instead of `message`, `btn` instead of `button`. Your IDE has autocomplete. Use full words.
2. Encoding types in names. `strName`, `intCount`, `lstItems`. Modern languages and IDEs show you the type. The name should describe the purpose, not the type (this is known as Hungarian notation, and it is outdated).
3. Using `temp`, `tmp`, `data`, `info`, `stuff`, `thing`. These names carry zero information. Every variable holds data. Every variable is temporary in some sense.
4. Using different names for the same concept. If you call it `customer` in one file and `client` in another and `user` in a third, readers will wonder if these are three different things.
5. Being afraid to rename. Modern IDEs have robust refactoring tools. Renaming a variable across an entire project takes seconds. Do not live with a bad name because "it would be too much work to change."
6. Naming booleans without a question-word prefix. `active` does not read well in conditions. `isActive` reads like English: `if (isActive)` versus `if (active)`.

Best Practices

1. Spend time choosing names. A few minutes spent finding the right name saves hours of confusion later. It is one of the highest-leverage activities in programming.
2. Read your code out loud. If you cannot explain what a variable represents in plain English, the name is wrong. "We get the, um, thing from the list" means your names need work.
3. Use the language of the business domain. If your users say "invoice," do not call it `BillingDocument`. If they say "shopping cart," do not call it `ItemCollection`.
4. Establish and document team conventions. Write down which word your team uses for each concept (get vs fetch, create vs make, delete vs remove). Consistency matters more than any particular choice.
5. Rename fearlessly and often. When you find a better name, change it immediately. The cost of renaming is seconds; the cost of a bad name is confusion that lasts forever.
6. Let the name length match the scope. A loop counter `i` in a three-line loop is fine. A class-level field or a function parameter should have a descriptive name.

Quick Summary

Names are the primary tool for making code readable. Intention-revealing names eliminate the need for comments. Consistent naming across a codebase reduces cognitive load. The time you invest in choosing good names is repaid every time anyone reads your code.

The core rules are: reveal intention, avoid disinformation, make meaningful distinctions, use pronounceable and searchable names, use nouns for classes and verbs for methods, be consistent, and never be clever at the expense of clarity.

Key Points

- A name should tell you why something exists, what it does, and how it is used -- without requiring a comment
- Misleading names (like `accountList` for a Map) are worse than unclear names because they actively deceive
- Noise words (Info, Data, Object, Variable) add length without adding meaning
- Pronounceable names enable productive conversations about code
- Searchable names (named constants instead of magic numbers) make code navigable
- Classes are nouns, methods are verbs -- this convention makes code read like prose
- Pick one word per concept and use it consistently across the entire codebase
- The time spent choosing a good name pays for itself many times over

---

Practice Questions

1. You encounter a variable named `d` with a comment `// number of days since last login`. Provide three possible good names for this variable and explain when each would be most appropriate.
2. A codebase has three service classes: `UserManager`, `OrderService`, and `ProductHandler`. All three do similar things (CRUD operations). What is wrong with this naming, and how would you fix it?

3. You find a method called `check()` that returns a boolean. Without seeing the implementation, list three possible things this method might do. Then provide three better names that each remove the ambiguity.
4. Your team uses `get`, `fetch`, `retrieve`, `find`, and `load` interchangeably to mean "get data from the database." Design a naming convention that distinguishes between these operations when there are genuine differences, and consolidates them when there are not.
5. Explain why the name `AbstractBaseFactoryManagerServiceImpl` is problematic, even though each word in it has a clear meaning.

Exercises

Exercise 1: Name That Variable

For each cryptic variable, provide a clear, intention-revealing replacement:

```
a) int x = 86400;
b) String s = "USD";
c) double r = 0.15;
d) boolean f = true;
e) List<String> l = getItems();
f) int n = employees.size();
g) Map<String, Integer> m = loadData();
h) String t = formatOutput();
```

Exercise 2: Refactor This Class

Rename every identifier in this class to be clean and clear:

Java:

```java
public class DM {
    private Connection c;

    public List<Map<String, Object>> q(String s) {
        List<Map<String, Object>> r = new ArrayList<>();
        try {
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery(s);
            while (rs.next()) {
```

```
                Map<String, Object> row = new HashMap<>();
                int cc = rs.getMetaData().getColumnCount();
                for (int i = 1; i <= cc; i++) {
                    String cn = rs.getMetaData().getColumnName(i);
                    row.put(cn, rs.getObject(i));
                }
                r.add(row);
            }
        } catch (Exception e) {
            System.out.println(e);
        }
        return r;
    }
}
```

Exercise 3: Naming Audit

Open any project you have worked on. Find five variables, three methods, and two classes with names that could be improved. For each one:

- Write the current name
- Write a better name
- Explain why the new name is better
- Use your IDE to rename it (if it is your project)

What Is Next?

Now that you know how to name things well, it is time to learn how to structure the code that lives inside those well-named containers. Chapter 3 covers functions -- how to keep them small, focused, and easy to understand. You will see a monster 100-line function broken down into clean, small functions that each do one thing well. The combination of good names and good functions is the foundation of all clean code.

Chapter 3: Functions -- The Building Blocks of Clean Code

What You Will Learn

- Why functions should be small -- ideally 5 to 15 lines -- and how this makes code dramatically easier to read
- The "do one thing" principle and how to tell when a function is doing too much
- How to keep functions at a single level of abstraction
- Why descriptive function names are worth every character
- The ideal number of function arguments (spoiler: fewer is better)
- Why flag arguments are a code smell and what to use instead
- How to eliminate hidden side effects that create bugs
- A complete before/after transformation of a 100-line monster function

Why This Chapter Matters

Functions are the first line of organization in any program. They are the verbs of your codebase -- the actions your software performs. Every single thing your application does happens inside a function. If your functions are bloated, confusing, and tangled, your entire codebase inherits that chaos.

Consider this: the average developer reads 10 to 20 functions for every one they write. When each function is a 100-line maze with six parameters and three hidden side effects, reading becomes an exhausting puzzle. But when each function is short, well-named, and does exactly one thing, reading code feels like reading a well-organized recipe.

This chapter teaches you how to write functions that other developers will thank you for.

Small Functions

The first rule of functions is that they should be small. The second rule is that they should be smaller than that.

How small? In most cases, a function should be between 5 and 15 lines of code. If it grows beyond 20 lines, it is almost certainly doing too much and should be broken apart.

This might sound extreme if you are used to writing functions that span 50, 100, or even 200 lines. But think about it from the reader's perspective. A 10-line function can be understood in a single glance. A 100-line function requires scrolling, mental bookmarking, and deep concentration.

```
+-------------------------------------------------------------+
|              Function Size and Readability                  |
+-------------------------------------------------------------+
|                                                             |
|   5 lines   -->  Understood in seconds                      |
|  10 lines   -->  Understood in a glance                     |
|  20 lines   -->  Understood in a minute                     |
|  50 lines   -->  Requires careful reading                   |
| 100 lines   -->  Requires scrolling and mental tracking     |
| 200+ lines  -->  Requires a map and a prayer                |
|                                                             |
+-------------------------------------------------------------+
```

BEFORE: A Long Function

Java:

```java
public void processOrder(Order order) {
    // Validate order
    if (order == null) {
        throw new IllegalArgumentException("Order cannot be null");
    }
    if (order.getItems() == null || order.getItems().isEmpty()) {
        throw new IllegalArgumentException("Order must have items");
    }
    if (order.getCustomer() == null) {
        throw new IllegalArgumentException("Order must have a customer");
    }
```

```
    // Calculate totals
    double subtotal = 0;
    for (OrderItem item : order.getItems()) {
        subtotal += item.getPrice() * item.getQuantity();
    }
    double tax = subtotal * 0.08;
    double total = subtotal + tax;
    order.setSubtotal(subtotal);
    order.setTax(tax);
    order.setTotal(total);

    // Apply discount
    if (order.getCustomer().isPreferred()) {
        order.setTotal(order.getTotal() * 0.9);
    }

    // Save to database
    Connection conn = DriverManager.getConnection(DB_URL);
    PreparedStatement stmt = conn.prepareStatement(
        "INSERT INTO orders (customer_id, total) VALUES (?, ?)"
    );
    stmt.setLong(1, order.getCustomer().getId());
    stmt.setDouble(2, order.getTotal());
    stmt.executeUpdate();
    conn.close();

    // Send confirmation email
    String subject = "Order Confirmation #" + order.getId();
    String body = "Thank you for your order. Total: $" + order.getTotal();
    EmailService.send(order.getCustomer().getEmail(), subject, body);
}
```

This function is about 35 lines, and it does at least five different things: validation, calculation, discount logic, database persistence, and email notification. Imagine what happens when this grows to 100+ lines with more business rules.

AFTER: Small Functions That Do One Thing

Java:

```java
public void processOrder(Order order) {
    validateOrder(order);
    calculateTotals(order);
    applyDiscounts(order);
    saveOrder(order);
    sendConfirmation(order);
}

private void validateOrder(Order order) {
```

```
    Objects.requireNonNull(order, "Order cannot be null");
    requireNonEmpty(order.getItems(), "Order must have items");
    Objects.requireNonNull(order.getCustomer(), "Order must have a customer");
}

private void calculateTotals(Order order) {
    double subtotal = order.getItems().stream()
        .mapToDouble(item -> item.getPrice() * item.getQuantity())
        .sum();
    order.setSubtotal(subtotal);
    order.setTax(subtotal * TAX_RATE);
    order.setTotal(subtotal + order.getTax());
}

private void applyDiscounts(Order order) {
    if (order.getCustomer().isPreferred()) {
        order.setTotal(order.getTotal() * PREFERRED_DISCOUNT);
    }
}

private void saveOrder(Order order) {
    orderRepository.save(order);
}

private void sendConfirmation(Order order) {
    emailService.sendOrderConfirmation(order);
}
```

Now `processOrder` reads like a table of contents. Each helper function is small, focused, and independently testable.

Python equivalent:

```python
def process_order(order):
    validate_order(order)
    calculate_totals(order)
    apply_discounts(order)
    save_order(order)
    send_confirmation(order)


def validate_order(order):
    if order is None:
        raise ValueError("Order cannot be None")
    if not order.items:
        raise ValueError("Order must have items")
    if order.customer is None:
        raise ValueError("Order must have a customer")


def calculate_totals(order):
```

```
    order.subtotal = sum(
        item.price * item.quantity for item in order.items
    )
    order.tax = order.subtotal * TAX_RATE
    order.total = order.subtotal + order.tax


def apply_discounts(order):
    if order.customer.is_preferred:
        order.total *= PREFERRED_DISCOUNT


def save_order(order):
    order_repository.save(order)


def send_confirmation(order):
    email_service.send_order_confirmation(order)
```

Do One Thing

A function should do one thing. It should do it well. It should do it only.

But how do you define "one thing"? Here is a practical test:

The "AND" Test: Describe what the function does. If your description includes the word "and" or "or," the function is doing more than one thing.

- "This function validates the order AND calculates the total AND saves it to the database." -- Three things. Break it apart.
- "This function calculates the order total." -- One thing. Keep it.

The Extraction Test: Try to extract another function from it with a name that is not merely a restatement of the original. If you can, the original function is doing more than one thing.

```
+-------------------------------------------------------------+
|                  The "One Thing" Test                       |
+-------------------------------------------------------------+
|                                                             |
|   Can you describe the function without using "AND"?        |
|                                                             |
|   YES --> The function probably does one thing     [OK]     |
|   NO  --> The function does too much            [SPLIT]     |
|                                                             |
|   Can you extract a meaningful sub-function?                |
|                                                             |
```

```
|   YES --> The original does more than one thing  [SPLIT]  |
|   NO  --> The function is properly focused          [OK]  |
|                                                           |
+-----------------------------------------------------------+
```

Sections Within Functions Are a Warning Sign

If you find yourself putting blank lines or comments inside a function to separate "sections," that function is doing multiple things. Each section should be its own function.

BEFORE: A function with internal sections

Python:

```python
def generate_report(employees):
    # Filter active employees
    active = [e for e in employees if e.is_active]

    # Sort by department
    active.sort(key=lambda e: e.department)

    # Build report lines
    lines = []
    current_dept = None
    for emp in active:
        if emp.department != current_dept:
            current_dept = emp.department
            lines.append(f"\n=== {current_dept} ===")
        lines.append(f"  {emp.name}: {emp.role}")

    # Write to file
    with open("report.txt", "w") as f:
        f.write("\n".join(lines))
```

AFTER: Each section becomes a function

Python:

```python
def generate_report(employees):
    active_employees = filter_active(employees)
    sorted_employees = sort_by_department(active_employees)
    report_content = build_report(sorted_employees)
    write_report(report_content)


def filter_active(employees):
    return [e for e in employees if e.is_active]
```

```
def sort_by_department(employees):
    return sorted(employees, key=lambda e: e.department)


def build_report(employees):
    lines = []
    current_dept = None
    for emp in employees:
        if emp.department != current_dept:
            current_dept = emp.department
            lines.append(f"\n=== {current_dept} ===")
        lines.append(f"  {emp.name}: {emp.role}")
    return "\n".join(lines)


def write_report(content, filename="report.txt"):
    with open(filename, "w") as f:
        f.write(content)
```

One Level of Abstraction per Function

Each function should operate at a single level of abstraction. Mixing high-level business logic with low-level implementation details makes code jarring to read.

Think of it like giving driving directions. You would not say: "Drive to the grocery store, engage the clutch pedal with your left foot while simultaneously rotating the ignition key clockwise, and pick up some milk." The directions mix high-level intent (drive to the store, buy milk) with low-level mechanics (clutch and ignition).

```
+-------------------------------------------------------------+
|             Abstraction Level Hierarchy                     |
+-------------------------------------------------------------+
|                                                             |
|  HIGH       processOrder()                                  |
|              |                                              |
|  MEDIUM      +-- validateOrder()                            |
|              +-- calculateTotals()                          |
|              +-- saveOrder()                                |
|              |    |                                         |
|  LOW         |    +-- prepareStatement()                    |
|              |    +-- executeUpdate()                       |
|              +-- sendConfirmation()                         |
|                   |                                         |
|  LOW              +-- formatEmailBody()                     |
```

```
|                   +-- connectToSmtpServer()              |
|                                                          |
+----------------------------------------------------------+
```

The Stepdown Rule

Code should read like a top-down narrative. Every function should be followed by those at the next level of abstraction. This is called the Stepdown Rule -- you read the program by descending one level of abstraction at a time.

BEFORE: Mixed abstraction levels

Java:

```java
public void registerUser(String name, String email, String password) {
    // High level: validate
    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException("Name required");
    }
    // Low level: regex check
    if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
        throw new IllegalArgumentException("Invalid email");
    }
    // Low level: hashing
    byte[] salt = new byte[16];
    new SecureRandom().nextBytes(salt);
    String hashedPassword = Base64.getEncoder().encodeToString(
        MessageDigest.getInstance("SHA-256")
            .digest((password + Base64.getEncoder().encodeToString(salt))
            .getBytes())
    );
    // High level: save
    User user = new User(name, email, hashedPassword);
    userRepository.save(user);
}
```

AFTER: Consistent abstraction level

Java:

```java
public void registerUser(String name, String email, String password) {
    validateRegistration(name, email, password);
    String hashedPassword = hashPassword(password);
    User user = new User(name, email, hashedPassword);
    userRepository.save(user);
}
```

```
private void validateRegistration(String name, String email, String password) {
    requireNonBlank(name, "Name");
    requireValidEmail(email);
    requireStrongPassword(password);
}

private String hashPassword(String password) {
    byte[] salt = generateSalt();
    return passwordHasher.hash(password, salt);
}
```

Use Descriptive Names

A long descriptive name is better than a short enigmatic name. A long descriptive name is better than a long descriptive comment.

Do not be afraid of long function names. Your IDE will autocomplete them. What matters is that a reader understands what the function does without reading its body.

```
+-------------------------------------------------------------+
|                Naming Comparison Table                       |
+-------------------------------------------------------------+
|                                                             |
|  BAD                      |  GOOD                           |
|  -------------------------|---------------------------------|
|  calc()                   |  calculateMonthlyPayment()      |
|  process()                |  processRefundRequest()         |
|  handle()                 |  handleExpiredSubscription()     |
|  doIt()                   |  sendWelcomeEmail()             |
|  run()                    |  runDailyInventoryCheck()       |
|  check()                  |  checkPasswordStrength()        |
|  update()                 |  updateShippingAddress()        |
|  get()                    |  getActiveSubscribers()         |
|                                                             |
+-------------------------------------------------------------+
```

Naming Conventions for Functions

Java: Use camelCase verbs. Start with a verb that describes the action: `calculateTax()`, `isValid()`, `hasPermission()`, `sendEmail()`.

Python: Use snake_case verbs. Same verb-first principle: `calculate_tax()`, `is_valid()`, `has_permission()`, `send_email()`.

The verb tells you what the function does. The noun tells you what it operates on. Together they form a clear action: `validateEmail()`, `parseJsonResponse()`, `formatCurrency()`.

Be Consistent

If you call one function `fetchCustomer()`, do not call the equivalent function for orders `retrieveOrder()` or `getProduct()`. Pick one convention -- `fetch`, `get`, or `retrieve` -- and stick with it throughout the codebase.

Function Arguments

The ideal number of arguments for a function is zero. Next comes one. Then two. Three arguments should be avoided. More than three requires very special justification -- and then should not be used anyway.

Why? Each argument increases the cognitive load on the reader. Each argument is one more thing to understand, remember, and get right when calling the function.

```
+-------------------------------------------------------------+
|              Argument Count Guidelines                      |
+-------------------------------------------------------------+
|                                                             |
|  0 args  (niladic)   -->  Ideal                             |
|  1 arg   (monadic)   -->  Good, very common                 |
|  2 args  (dyadic)    -->  Acceptable, use carefully          |
|  3 args  (triadic)   -->  Avoid if possible                  |
|  4+ args (polyadic)  -->  Almost always wrong                |
|                                                             |
+-------------------------------------------------------------+
```

Common Monadic Forms

There are three common reasons to pass a single argument:

1. Asking a question: `boolean fileExists("MyFile")` -- You pass something and get a boolean answer.
2. Transforming it: `InputStream fileOpen("MyFile")` -- You pass something and get a transformed version back.
3. Processing an event: `passwordAttemptFailed(attemptCount)` -- You pass something to signal an event.

Dyadic Functions

Two arguments are fine when they have a natural ordering or relationship:

```java
// Natural pair -- coordinates
Point p = new Point(0, 0);

// Natural pair -- expected vs actual
assertEquals(expected, actual);

// Problematic -- which is which?
writeField(outputStream, name);  // Better: outputStream.writeField(name)
```

Reduce Arguments with Objects

When a function needs many arguments, it is often a sign that some of those arguments should be grouped into an object.

BEFORE: Too many arguments

Java:

```java
Circle makeCircle(double x, double y, double radius) {
    // ...
}
```

AFTER: Arguments grouped into an object

Java:

```java
Circle makeCircle(Point center, double radius) {
    // ...
}
```

Python:

```python
# BEFORE
def create_user(first_name, last_name, email, phone, street,
                city, state, zip_code):
    pass

# AFTER
def create_user(name: PersonName, contact: ContactInfo,
                address: Address):
    pass
```

No Flag Arguments

Flag arguments -- boolean parameters that change a function's behavior -- are ugly. They loudly proclaim that the function does more than one thing. It does one thing if the flag is true and another thing if the flag is false.

BEFORE: Flag argument

Java:

```java
public void render(boolean isSuite) {
    if (isSuite) {
        // ... render suite header ...
    } else {
        // ... render single test header ...
    }
    // ... common rendering ...
    if (isSuite) {
        // ... render suite footer ...
    } else {
        // ... render single test footer ...
    }
}
```

AFTER: Two separate functions

Java:

```java
public void renderSuite() {
    renderSuiteHeader();
    renderBody();
    renderSuiteFooter();
}

public void renderSingleTest() {
    renderSingleTestHeader();
    renderBody();
    renderSingleTestFooter();
}
```

Python:

```python
# BEFORE
def create_report(data, include_charts=False):
    # Does two different things based on flag
    pass

# AFTER
def create_text_report(data):
    pass

def create_visual_report(data):
    report = create_text_report(data)
    add_charts(report, data)
    return report
```

When you see `render(true)` in calling code, you have to look up the function to understand what `true` means. But `renderSuite()` is immediately clear.

No Side Effects

A side effect is when a function promises to do one thing but quietly does something else too. Side effects are lies. Your function promises to do one thing, but it also does other hidden things.

BEFORE: A function with a hidden side effect

Java:

```java
public boolean checkPassword(String userName, String password) {
    User user = userRepository.findByName(userName);
    if (user != null) {
        String encodedPhrase = cryptographer.encrypt(password, user.getSalt());
        if (encodedPhrase.equals(user.getEncryptedPassword())) {
            Session.initialize();  // <-- SIDE EFFECT!
            return true;
        }
    }
    return false;
}
```

The function is called `checkPassword`. It says it checks a password and returns true or false. But it also initializes the session. A developer calling `checkPassword` during a routine validation might accidentally erase session data.

AFTER: Side effect removed

Java:

```java
public boolean checkPassword(String userName, String password) {
    User user = userRepository.findByName(userName);
    if (user == null) {
        return false;
    }
    String encodedPhrase = cryptographer.encrypt(password, user.getSalt());
    return encodedPhrase.equals(user.getEncryptedPassword());
}

// Session initialization happens explicitly in the login flow
public void login(String userName, String password) {
    if (checkPassword(userName, password)) {
        session.initialize();
    }
}
```

Python:

```python
# BEFORE: Hidden side effect
def check_password(username, password):
    user = user_repository.find_by_name(username)
    if user and hasher.verify(password, user.hashed_password):
        session.initialize()  # Hidden side effect!
        return True
    return False


# AFTER: No side effects
def check_password(username, password):
    user = user_repository.find_by_name(username)
    if user is None:
        return False
    return hasher.verify(password, user.hashed_password)


def login(username, password):
    if check_password(username, password):
        session.initialize()
```

Output Arguments

Arguments should be inputs to a function, not outputs. Avoid using arguments to return results.

BEFORE: Output argument

Java:

```java
// What does this do? Does it append 's' to the footer?
// Or does it append the footer to 's'?
appendFooter(s);
```

AFTER: Method on the object

Java:

```java
// Clear and obvious
report.appendFooter();
```

Command-Query Separation

Functions should either do something (a command) or answer something (a query), but not both.

BEFORE: Mixed command and query

Java:

```java
// Returns true if the attribute existed and was set successfully
public boolean set(String attribute, String value) {
    // ...
}

// Confusing in calling code:
if (set("username", "bob")) {
    // Was it already set? Did the set succeed? Unclear.
}
```

AFTER: Separated command and query

Java:

```java
if (attributeExists("username")) {
    setAttribute("username", "bob");
}
```

Python:

```python
# BEFORE: Mixed command and query
def set_value(key, value):
    """Sets value and returns True if key existed."""
    pass

# AFTER: Separated
def has_key(key):
    return key in _store

def set_value(key, value):
    _store[key] = value
```

Prefer Exceptions to Returning Error Codes

When you return error codes from a function, you create a problem: the caller must deal with the error immediately, leading to deeply nested structures.

BEFORE: Error codes lead to deep nesting

Java:

```java
if (deletePage(page) == E_OK) {
    if (registry.deleteReference(page.name) == E_OK) {
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK) {
            logger.log("page deleted");
        } else {
            logger.log("configKey not deleted");
        }
    } else {
        logger.log("deleteReference failed");
    }
} else {
    logger.log("delete failed");
    return E_ERROR;
}
```

AFTER: Exceptions simplify the flow

Java:

```java
try {
    deletePage(page);
    registry.deleteReference(page.name);
```

```
    configKeys.deleteKey(page.name.makeKey());
} catch (Exception e) {
    logger.log(e.getMessage());
}
```

Extract Try/Catch Blocks

`try/catch` blocks are ugly. They mix error processing with normal processing. Extract the bodies into their own functions.

Java:

```java
public void delete(Page page) {
    try {
        deletePageAndReferences(page);
    } catch (Exception e) {
        logError(e);
    }
}

private void deletePageAndReferences(Page page) throws Exception {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
```

The Complete Monster Refactoring

Here is a real-world-style 100-line monster function, followed by its clean refactoring.

BEFORE: The 100-Line Monster

Java:

```java
public String generateInvoice(int customerId, List<Integer> productIds,
        String discountCode, boolean sendEmail, String format) {
    // Look up customer
    Connection conn = null;
    String invoice = "";
    try {
        conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
```

```
        PreparedStatement ps = conn.prepareStatement(
            "SELECT * FROM customers WHERE id = ?");
        ps.setInt(1, customerId);
        ResultSet rs = ps.executeQuery();
        if (!rs.next()) {
            return "ERROR: Customer not found";
        }
        String custName = rs.getString("name");
        String custEmail = rs.getString("email");
        String custAddress = rs.getString("address");

        // Look up products and calculate total
        double subtotal = 0;
        List<String> lines = new ArrayList<>();
        for (int pid : productIds) {
            PreparedStatement ps2 = conn.prepareStatement(
                "SELECT * FROM products WHERE id = ?");
            ps2.setInt(1, pid);
            ResultSet rs2 = ps2.executeQuery();
            if (rs2.next()) {
                String pName = rs2.getString("name");
                double pPrice = rs2.getDouble("price");
                int stock = rs2.getInt("stock");
                if (stock <= 0) {
                    lines.add(pName + " - OUT OF STOCK");
                } else {
                    lines.add(pName + " - $" + pPrice);
                    subtotal += pPrice;
                    // Update stock
                    PreparedStatement ps3 = conn.prepareStatement(
                        "UPDATE products SET stock = stock - 1 WHERE id = ?");
                    ps3.setInt(1, pid);
                    ps3.executeUpdate();
                }
            }
        }

        // Apply discount
        double discount = 0;
        if (discountCode != null && !discountCode.isEmpty()) {
            PreparedStatement ps4 = conn.prepareStatement(
                "SELECT percentage FROM discounts WHERE code = ? AND active = 1");
            ps4.setString(1, discountCode);
            ResultSet rs4 = ps4.executeQuery();
            if (rs4.next()) {
                discount = rs4.getDouble("percentage");
            }
        }
        double discountAmount = subtotal * (discount / 100.0);
        double tax = (subtotal - discountAmount) * 0.08;
        double total = subtotal - discountAmount + tax;

        // Format invoice
        if (format.equals("HTML")) {
            invoice = "<html><body><h1>Invoice</h1>";
```

```
            invoice += "<p>Customer: " + custName + "</p>";
            invoice += "<p>Address: " + custAddress + "</p>";
            for (String line : lines) {
                invoice += "<p>" + line + "</p>";
            }
            invoice += "<p>Subtotal: $" + subtotal + "</p>";
            if (discount > 0) {
                invoice += "<p>Discount: -$" + discountAmount + "</p>";
            }
            invoice += "<p>Tax: $" + tax + "</p>";
            invoice += "<p><strong>Total: $" + total + "</strong></p>";
            invoice += "</body></html>";
        } else {
            invoice = "INVOICE\n";
            invoice += "Customer: " + custName + "\n";
            invoice += "Address: " + custAddress + "\n";
            invoice += "---\n";
            for (String line : lines) {
                invoice += line + "\n";
            }
            invoice += "---\n";
            invoice += "Subtotal: $" + subtotal + "\n";
            if (discount > 0) {
                invoice += "Discount: -$" + discountAmount + "\n";
            }
            invoice += "Tax: $" + tax + "\n";
            invoice += "Total: $" + total + "\n";
        }

        // Send email
        if (sendEmail) {
            Properties props = new Properties();
            props.put("mail.smtp.host", "smtp.company.com");
            Session session = Session.getInstance(props);
            MimeMessage msg = new MimeMessage(session);
            msg.setFrom(new InternetAddress("billing@company.com"));
            msg.setRecipient(Message.RecipientType.TO,
                new InternetAddress(custEmail));
            msg.setSubject("Your Invoice");
            msg.setContent(invoice, format.equals("HTML")
                ? "text/html" : "text/plain");
            Transport.send(msg);
        }

        // Save invoice
        PreparedStatement ps5 = conn.prepareStatement(
            "INSERT INTO invoices (customer_id, total, content) VALUES (?, ?, ?)");
        ps5.setInt(1, customerId);
        ps5.setDouble(2, total);
        ps5.setString(3, invoice);
        ps5.executeUpdate();

    } catch (Exception e) {
        return "ERROR: " + e.getMessage();
    } finally {
```

```
        if (conn != null) try { conn.close(); } catch (Exception e) {}
    }
    return invoice;
}
```

This function has every problem we have discussed: it is enormous, it does at least seven different things, it mixes abstraction levels, it has five arguments including a flag, it has side effects (updating stock, sending email, saving to the database), and it returns error codes as strings.

AFTER: Refactored Into Clean Functions

Java:

```java
public class InvoiceService {

    private final CustomerRepository customerRepository;
    private final ProductRepository productRepository;
    private final DiscountRepository discountRepository;
    private final InvoiceRepository invoiceRepository;
    private final EmailService emailService;
    private final TaxCalculator taxCalculator;

    public Invoice generateInvoice(InvoiceRequest request) {
        Customer customer = findCustomer(request.getCustomerId());
        List<InvoiceLine> lines = buildInvoiceLines(request.getProductIds());
        PriceSummary pricing = calculatePricing(lines, request.getDiscountCode());
        Invoice invoice = createInvoice(customer, lines, pricing);
        invoiceRepository.save(invoice);
        return invoice;
    }

    private Customer findCustomer(int customerId) {
        return customerRepository.findById(customerId)
            .orElseThrow(() -> new CustomerNotFoundException(customerId));
    }

    private List<InvoiceLine> buildInvoiceLines(List<Integer> productIds) {
        return productIds.stream()
            .map(this::createInvoiceLine)
            .collect(Collectors.toList());
    }

    private InvoiceLine createInvoiceLine(int productId) {
        Product product = productRepository.findById(productId)
            .orElseThrow(() -> new ProductNotFoundException(productId));

        if (product.isOutOfStock()) {
            return InvoiceLine.outOfStock(product);
```

```
        }

        productRepository.decrementStock(productId);
        return InvoiceLine.forProduct(product);
    }

    private PriceSummary calculatePricing(List<InvoiceLine> lines,
                                          String discountCode) {
        double subtotal = lines.stream()
            .filter(InvoiceLine::isAvailable)
            .mapToDouble(InvoiceLine::getPrice)
            .sum();

        double discountPercent = discountRepository
            .findActiveDiscount(discountCode)
            .orElse(0.0);

        double discountAmount = subtotal * (discountPercent / 100.0);
        double tax = taxCalculator.calculate(subtotal - discountAmount);
        double total = subtotal - discountAmount + tax;

        return new PriceSummary(subtotal, discountAmount, tax, total);
    }

    private Invoice createInvoice(Customer customer,
                                  List<InvoiceLine> lines,
                                  PriceSummary pricing) {
        return new Invoice(customer, lines, pricing);
    }
}

// Email sending is now a separate concern
public class InvoiceEmailSender {
    public void sendInvoice(Invoice invoice) {
        emailService.send(
            invoice.getCustomer().getEmail(),
            "Your Invoice",
            invoiceFormatter.format(invoice)
        );
    }
}

// Formatting is a separate concern
public class InvoiceFormatter {
    public String formatAsText(Invoice invoice) { /* ... */ }
    public String formatAsHtml(Invoice invoice) { /* ... */ }
}
```

```
+-------------------------------------------------------------+
|        Monster Function vs Clean Refactoring                |
+-------------------------------------------------------------+
|                                                             |
|  BEFORE                     |  AFTER                        |
```

```
|  100+ lines, 1 function   |  10 functions, 5-15 lines    |
|  5 arguments + flag        |  1 argument (request object) |
|  7 responsibilities        |  1 responsibility each       |
|  Mixed abstractions        |  Consistent levels           |
|  String error codes        |  Proper exceptions           |
|  Hidden side effects       |  Explicit operations         |
|  Untestable                |  Fully testable              |
|  Hard to change            |  Easy to extend              |
|                                                           |
+-----------------------------------------------------------+
```

Common Mistakes

1. Writing functions that are too long -- If you need to scroll to see the whole function, it is too long. Break it apart.
2. Using vague names like `process()`, `handle()`, `manage()` -- These names tell the reader nothing. Be specific about what is being processed, handled, or managed.
3. Passing boolean flags -- Every flag argument doubles the function's responsibilities. Create separate functions instead.
4. Mixing abstraction levels -- Do not put SQL queries next to business logic. Do not put HTTP headers next to domain calculations. Each function should live at one level.
5. Creating hidden side effects -- If a function modifies global state, writes to a file, sends an email, or changes a database, that action should be visible in the function's name or handled at the appropriate level.
6. Returning error codes instead of throwing exceptions -- Error codes force callers into deeply nested if-else chains. Use exceptions and let the error handling happen at the right level.
7. Writing functions with too many arguments -- If you need more than three arguments, consider creating a parameter object.

Best Practices

1. Start with a working mess, then refactor -- Nobody writes perfectly clean functions on the first pass. Write the code that works, then extract, rename, and restructure until each function does one thing.

2. Apply the newspaper metaphor -- High-level functions at the top, lower-level details below. The reader should be able to skim the public methods and understand the story without diving into the details.
3. Name functions after what they do, not how they do it -- `calculateShippingCost()` is better than `iterateOverItemsAndSumWeights()`.
4. Keep argument counts low -- Zero, one, or two arguments. Group related arguments into objects.
5. Write functions that can be read without context -- A well-named function with well-named parameters should be understandable without reading its implementation.
6. Separate commands from queries -- Functions that change state should not return values. Functions that return values should not change state.
7. Extract try/catch bodies -- The error handling and the logic should live in separate functions.
8. Test your functions independently -- If a function is hard to test, it is doing too much. Difficulty testing is a design signal.

Quick Summary

Functions should be small (5 to 15 lines), do exactly one thing, operate at a single level of abstraction, and have descriptive names. They should take as few arguments as possible, never use flag arguments, and have no hidden side effects. Prefer exceptions to error codes. When you encounter a monster function, refactor it into a collection of small, focused functions that read like a story.

Key Points

- Functions should be small. Really small. Five to fifteen lines is the sweet spot.
- Each function should do one thing and do it well.
- Functions should maintain a single level of abstraction. Do not mix high-level business logic with low-level implementation details.
- Descriptive names eliminate the need for comments. Use verbs that describe the action.
- The ideal argument count is zero. One and two are acceptable. Three or more is a code smell.
- Boolean flag arguments announce that a function does two things. Split it.

- Side effects are hidden behaviors that surprise callers. Make all effects visible.
- Commands (change state) and queries (return values) should be separate functions.
- Exceptions are cleaner than error codes for error handling.
- Nobody writes clean functions on the first try. Write the mess, make it work, then clean it up.

Practice Questions

1. You encounter a function called `processData()` that is 80 lines long. It validates input, transforms data, writes to a database, and sends a notification. Describe step by step how you would refactor it following the principles from this chapter.
2. Why is `render(boolean isSuite)` worse than having two separate functions `renderSuite()` and `renderSingleTest()`? What principle does the flag argument violate?
3. Explain the difference between a "command" and a "query" in the context of Command-Query Separation. Give an example of a function that violates this principle and show how to fix it.
4. A colleague argues that writing many small functions creates "too many files" and "too much jumping around." How would you respond to this concern?
5. Look at this function signature: `createUser(String firstName, String lastName, String email, String phone, String street, String city, String state, String zipCode)`. What is wrong with it and how would you improve it?

Exercises

Exercise 1: Extract and Name

Take this function and refactor it into small, well-named functions:

```python
def handle_order(order_data):
    if not order_data.get("items"):
        print("Error: no items")
        return None
    total = 0
    for item in order_data["items"]:
```

```
        if item["quantity"] <= 0:
            print(f"Error: bad quantity for {item['name']}")
            return None
        total += item["price"] * item["quantity"]
    if total > 100:
        total *= 0.9  # 10% discount for large orders
    tax = total * 0.07
    total += tax
    print(f"Order total: ${total:.2f}")
    with open("orders.log", "a") as f:
        f.write(f"Order: ${total:.2f}\n")
    return total
```

Exercise 2: Eliminate the Flag

Refactor this Java function to eliminate the boolean parameter:

```java
public List<Employee> getEmployees(boolean includeTerminated) {
    List<Employee> result = new ArrayList<>();
    for (Employee emp : allEmployees) {
        if (includeTerminated || emp.isActive()) {
            result.add(emp);
        }
    }
    return result;
}
```

Exercise 3: Reduce Arguments

This function has too many arguments. Refactor it using a parameter object:

```java
public void sendEmail(String to, String from, String subject,
                      String body, String cc, String bcc,
                      boolean isHtml, int priority) {
    // ...
}
```

What Is Next?

You now know how to write functions that are small, focused, and expressive. But even clean functions can be obscured by bad comments -- or, conversely, clarified by good ones. In

Chapter 4, we will explore the art of commenting: when comments help, when they hurt, and how to write code so clear that most comments become unnecessary. You will learn why self-documenting code is the best kind of documentation.

Chapter 4: Comments -- When They Help and When They Hurt

What You Will Learn

- Why the best comment is the one you did not need to write
- The difference between good comments and bad comments with concrete examples
- Which types of comments actively improve a codebase
- Which types of comments are code smells in disguise
- How to write self-documenting code that eliminates the need for most comments
- Why commented-out code is one of the worst things you can leave in a codebase
- Before/after transformations showing comments replaced by clearer code

Why This Chapter Matters

Comments are a double-edged sword. A good comment can save a developer hours of confusion. A bad comment can waste hours or, worse, actively mislead. The problem is that most comments in most codebases fall into the "bad" category.

Here is the uncomfortable truth: comments are, at best, a necessary evil. If our programming languages were expressive enough, if we were skilled enough to write code that expresses our intent clearly, we would not need comments at all.

Every time you write a comment, you should feel a twinge of failure. Not because commenting is wrong, but because it means you failed to express yourself in code. Before you write a comment, ask yourself: "Can I rename this variable, extract this function, or restructure this logic so the comment becomes unnecessary?"

Sometimes the answer is no, and a comment is genuinely the best tool. This chapter teaches you to tell the difference.

The Problem with Comments

Comments do not make up for bad code. If you write confusing code and then add a comment to explain it, you have two problems: confusing code and a comment that will eventually become outdated and lie to the next developer.

```
+-------------------------------------------------------------+
|              The Comment Lifecycle                          |
+-------------------------------------------------------------+
|                                                             |
|  1. Developer writes code and a comment explaining it       |
|  2. Code works correctly. Comment is accurate.              |
|  3. Months pass. Requirements change.                       |
|  4. Developer modifies the code.                            |
|  5. Developer forgets to update the comment.                |
|  6. Comment now lies about what the code does.              |
|  7. Next developer reads the comment, trusts it.            |
|  8. Next developer wastes hours debugging based on the      |
|     comment's outdated information.                         |
|                                                             |
|  The older a comment gets, the more likely it is wrong.     |
|                                                             |
+-------------------------------------------------------------+
```

Code Is the Only Source of Truth

Code gets compiled or interpreted. Code gets tested. Code gets executed. Comments do none of these things. When the code and the comment disagree, the comment is always wrong.

BEFORE: Comment explains unclear code

Java:

```java
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65)) {
    // ...
}
```

AFTER: Code explains itself

Java:

```java
if (employee.isEligibleForFullBenefits()) {
    // ...
}
```

The comment vanished because the code now says exactly what the comment used to say. The extracted method `isEligibleForFullBenefits()` carries all the meaning, and unlike a comment, it will cause a compile error if the class changes in incompatible ways.

Good Comments

Some comments are necessary and valuable. Here are the types worth keeping.

Legal Comments

Copyright and license headers are sometimes required by corporate or legal standards. These belong at the top of files.

Java:

```java
/*
 * Copyright (c) 2024 Acme Corporation
 * Licensed under the Apache License, Version 2.0
 * See LICENSE file for details.
 */
```

Python:

```python
# Copyright (c) 2024 Acme Corporation
# Licensed under the Apache License, Version 2.0
# See LICENSE file for details.
```

Keep them short. Reference an external license file rather than including the full license text.

Informative Comments

Sometimes a comment provides useful information that cannot be easily expressed in code.

Java:

```java
// Format: kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

The regex is inherently hard to read. The comment clarifying the format is genuinely helpful.

Explanation of Intent

Sometimes a comment explains why a decision was made, not what the code does.

Java:

```java
// We sort by priority descending because the payment processor
// applies the first matching discount rule, and we want the
// highest-value discount to be checked first.
Collections.sort(discountRules, Collections.reverseOrder());
```

Python:

```python
# Using insertion sort here instead of the built-in sort because
# the list is nearly sorted (only 1-2 elements move per update)
# and insertion sort is O(n) for nearly-sorted data.
insertion_sort(almost_sorted_data)
```

These comments explain why, not what. The code already shows what happens. The comment explains the reasoning behind the choice.

Clarification

When dealing with code you cannot alter (library calls, legacy APIs), clarification comments help translate obscure operations.

Java:

```java
assertTrue(a.compareTo(b) != 0);    // a != b
assertTrue(a.compareTo(b) == -1);   // a < b
assertTrue(b.compareTo(a) == 1);    // b > a
```

Warning of Consequences

Comments that warn other developers about consequences are valuable.

Java:

```java
// WARNING: This test takes approximately 30 minutes to run.
// It performs a full integration test against the payment provider's
// staging environment. Only run manually before releases.
@Test
public void testPaymentProviderIntegration() {
    // ...
}
```

Python:

```python
# WARNING: This function is NOT thread-safe. It modifies
# shared state without locking. Use ThreadSafeCounter instead
# if calling from multiple threads.
def increment_counter():
    global _counter
    _counter += 1
```

TODO Comments

`TODO` comments are acceptable as temporary markers for work that needs to be done but cannot be completed right now.

Java:

```java
// TODO: Extract discount calculation into a strategy pattern
//       when we add the third discount type. -- Jane, 2024-03-15
public double calculateDiscount(Order order) {
    // current simple implementation
}
```

Python:

```python
# TODO: Replace with async HTTP calls when we upgrade to Python 3.12
#       Tracking issue: PROJ-1234
def fetch_all_prices(product_ids):
    return [fetch_price(pid) for pid in product_ids]
```

Good TODO comments include who left them, when, and ideally a ticket reference. They should be reviewed and cleaned up regularly.

Javadoc for Public APIs

Documentation comments on public APIs are essential. They form the contract between the code and its users.

Java:

```java
/**
 * Calculates the shipping cost for the given order.
 *
 * The cost is determined by the total weight of all items,
 * the shipping destination zone, and the selected speed.
 *
 * @param order  the order containing items with weights
 * @param zone   the destination shipping zone (1-5)
 * @param speed  the shipping speed (STANDARD, EXPRESS, OVERNIGHT)
 * @return the shipping cost in dollars, always non-negative
 * @throws InvalidZoneException if zone is outside range 1-5
 */
public BigDecimal calculateShippingCost(Order order, int zone,
                                        ShippingSpeed speed) {
    // ...
}
```

Python:

```python
def calculate_shipping_cost(order, zone, speed):
    """Calculate the shipping cost for the given order.

    The cost is determined by the total weight of all items,
    the shipping destination zone, and the selected speed.

    Args:
        order: The order containing items with weights.
        zone: The destination shipping zone (1-5).
        speed: The shipping speed (STANDARD, EXPRESS, OVERNIGHT).

    Returns:
        The shipping cost in dollars as a Decimal, always non-negative.

    Raises:
        InvalidZoneError: If zone is outside range 1-5.
    """
    pass
```

```
+-------------------------------------------------------------+
|                   Good Comments Summary                     |
+-------------------------------------------------------------+
|                                                             |
```

```
|  Type                |  Purpose                            |
|  --------------------|------------------------------------ |
|  Legal               |  Copyright, license headers         |
|  Informative         |  Clarify obscure constructs         |
|  Intent              |  Explain WHY, not WHAT              |
|  Clarification       |  Translate library/API calls        |
|  Warning             |  Consequences of changes            |
|  TODO                |  Temporary work markers             |
|  Javadoc / Docstring |  Public API documentation           |
|                                                            |
+------------------------------------------------------------+
```

Bad Comments

Far more common than good comments are bad comments. These clutter the code, mislead developers, and create maintenance burden. Here are the types to eliminate.

Redundant Comments

A comment that says exactly what the code says is pure noise.

BEFORE: Redundant comments

Java:

```java
// The day of the month
private int dayOfMonth;

// Returns the day of the month
public int getDayOfMonth() {
    return dayOfMonth;
}

// Sets the day of the month
public void setDayOfMonth(int dayOfMonth) {
    this.dayOfMonth = dayOfMonth;
}

// The name
private String name;

// Default constructor
public Employee() {
}
```

AFTER: No comments needed

Java:

```java
private int dayOfMonth;
private String name;

public int getDayOfMonth() {
    return dayOfMonth;
}

public void setDayOfMonth(int dayOfMonth) {
    this.dayOfMonth = dayOfMonth;
}

public Employee() {
}
```

Every comment in the "before" version was pure noise. The code already communicated everything the comments said. Removing them makes the code cleaner and shorter.

Python:

```python
# BAD: Redundant
class Employee:
    def __init__(self):
        self.name = ""  # the name of the employee
        self.age = 0    # the age of the employee

# GOOD: Self-evident
class Employee:
    def __init__(self):
        self.name = ""
        self.age = 0
```

Misleading Comments

Worse than redundant comments are comments that lie. They were accurate once but the code changed and nobody updated the comment.

Java:

```java
// Returns true if the user is active
public boolean isActive(User user) {
    // Code was later modified to also check subscription status
    return user.getStatus() == Status.ACTIVE
```

```
        && user.getSubscription() != null
        && !user.getSubscription().isExpired();
}
```

The comment says this checks if the user is active. But the code also checks subscription status. A developer reading only the comment would miss the subscription requirement.

Journal Comments

Some developers maintain a changelog at the top of every file. This was useful before version control existed. Today it is pointless clutter -- that is what `git log` is for.

BEFORE: Journal comments

Java:

```java
/*
 * Changes (from 01-Oct-2023)
 * --------------------------
 * 01-Oct-2023 : Added calculateTotal method (Mike)
 * 15-Oct-2023 : Fixed tax calculation bug (Sarah)
 * 02-Nov-2023 : Added discount support (Mike)
 * 14-Nov-2023 : Refactored to use streams (Jane)
 * 03-Dec-2023 : Fixed null pointer in discount (Sarah)
 * 18-Dec-2023 : Added support for bulk pricing (Mike)
 * 05-Jan-2024 : Performance optimization (Jane)
 */
```

AFTER: Use version control

Delete the entire journal. Every single entry is already captured in your Git history with more detail (diffs, timestamps, branches, associated pull requests).

Noise Comments

Comments that restate the obvious add nothing but visual noise.

Java:

```java
/** The name. */
private String name;

/** The version. */
private String version;
```

```
/** The info. */
private String info;

/** Default constructor. */
public MyClass() {
}

/** Returns the name. */
public String getName() {
    return name;
}
```

Python:

```python
def get_name(self):
    """Returns the name."""  # Adds nothing
    return self.name
```

These comments train developers to ignore comments entirely, which means they will also ignore the rare genuinely useful comment.

Mandated Comments

Some organizations require Javadoc on every function and every variable. This creates a tsunami of useless comments.

Java:

```java
/**
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param duration The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int duration) {
    // ...
}
```

Every `@param` tag merely restates the parameter name. This adds no value and creates a maintenance burden -- if you rename a parameter, you must also update the comment. Require documentation for public APIs, not for every function.

Commented-Out Code

This is one of the worst offenses in any codebase.

BEFORE: Commented-out code

Java:

```java
public double calculateTotal(List<Item> items) {
    double total = 0;
    for (Item item : items) {
        total += item.getPrice();
        // total += item.getPrice() * item.getQuantity();
        // if (item.isOnSale()) {
        //     total -= item.getDiscount();
        // }
    }
    // applyBulkDiscount(total, items.size());
    // logCalculation(total);
    return total;
}
```

Python:

```python
def calculate_total(items):
    total = 0
    for item in items:
        total += item.price
        # total += item.price * item.quantity
        # if item.is_on_sale:
        #     total -= item.discount
    # apply_bulk_discount(total, len(items))
    # log_calculation(total)
    return total
```

Why is this so bad?

1. Nobody will delete it. Other developers see commented-out code and think "someone left this for a reason" and are afraid to remove it.
2. It accumulates. Over months, more and more commented-out code piles up.
3. It is confusing. Is the commented code the old way? A feature in progress? A debugging artifact?
4. Version control already has it. If you need the old code, use `git log` or `git show`.

AFTER: Delete it

Java:

```java
public double calculateTotal(List<Item> items) {
    double total = 0;
```

```
    for (Item item : items) {
        total += item.getPrice();
    }
    return total;
}
```

If you genuinely need the old code someday, Git has it. Delete the commented-out code without hesitation.

Position Markers and Banners

Java:

```java
// ==================== GETTERS ====================

public String getName() { return name; }
public int getAge() { return age; }

// ==================== SETTERS ====================

public void setName(String name) { this.name = name; }
public void setAge(int age) { this.age = age; }

// ==================== BUSINESS LOGIC ==============
```

If you feel the need for section markers, your class is too large. Extract separate classes instead of organizing a monolith with banners.

Closing Brace Comments

Java:

```java
if (condition) {
    while (running) {
        for (Item item : items) {
            if (item.isValid()) {
                // ... lots of code ...
            } // if item is valid
        } // for each item
    } // while running
} // if condition
```

If your nesting is so deep that you need comments on closing braces to keep track, your function is too long. Extract smaller functions instead.

Attribution Comments

```java
// Added by Mike, March 2024
public void calculateDiscount() { ... }
```

This is what `git blame` is for. Attribution comments get outdated as code gets modified by other developers.

```
+-------------------------------------------------------------+
|                   Bad Comments Summary                      |
+-------------------------------------------------------------+
|                                                             |
|  Type               |  Why It Is Bad                        |
|  -------------------|---------------------------------------|
|  Redundant          |  Says what code already says          |
|  Misleading         |  Lies about what code does            |
|  Journal            |  Duplicates version control           |
|  Noise              |  States the obvious, trains ignoring  |
|  Mandated           |  Creates maintenance burden           |
|  Commented-out      |  Clutters, confuses, never deleted    |
|  Position markers   |  Masks classes that are too large     |
|  Closing brace      |  Masks functions that are too long    |
|  Attribution        |  Duplicates git blame                 |
|                                                             |
+-------------------------------------------------------------+
```

Self-Documenting Code

The ultimate goal is code that documents itself. Every time you are tempted to write a comment, try these alternatives first.

Replace Comments with Better Names

BEFORE:

Python:

```python
# Check if the user can access premium features
if user.plan == "premium" and user.is_active and not user.is_suspended:
    show_premium_features()
```

AFTER:

Python:

```python
if user.can_access_premium_features():
    show_premium_features()
```

Replace Comments with Extracted Functions

BEFORE:

Java:

```java
// Calculate the price after applying the seasonal discount
// and the loyalty bonus, but cap the total discount at 30%
double discount = seasonalRate + loyaltyBonus;
if (discount > 0.30) {
    discount = 0.30;
}
double finalPrice = basePrice * (1 - discount);
```

AFTER:

Java:

```java
double finalPrice = calculateDiscountedPrice(basePrice,
    seasonalRate, loyaltyBonus);

private double calculateDiscountedPrice(double basePrice,
        double seasonalRate, double loyaltyBonus) {
    double totalDiscount = Math.min(
        seasonalRate + loyaltyBonus, MAX_DISCOUNT_RATE);
    return basePrice * (1 - totalDiscount);
}
```

Replace Comments with Constants

BEFORE:

Python:

```python
# Timeout after 30 seconds
time.sleep(30)

# Maximum retry attempts
```

```
for i in range(3):
    pass
```

AFTER:

Python:

```python
CONNECTION_TIMEOUT_SECONDS = 30
MAX_RETRY_ATTEMPTS = 3

time.sleep(CONNECTION_TIMEOUT_SECONDS)

for attempt in range(MAX_RETRY_ATTEMPTS):
    pass
```

Replace Comments with Better Structure

BEFORE:

Java:

```java
public void processApplication(Application app) {
    // Step 1: Validate
    if (app.getName() == null) { throw new Exception("Name required"); }
    if (app.getAge() < 18) { throw new Exception("Must be 18+"); }

    // Step 2: Calculate score
    int score = app.getGpa() * 10 + app.getTestScore();

    // Step 3: Determine result
    String result;
    if (score > 90) {
        result = "ACCEPTED";
    } else if (score > 70) {
        result = "WAITLISTED";
    } else {
        result = "REJECTED";
    }

    // Step 4: Save
    repository.save(app, result);

    // Step 5: Notify
    emailService.send(app.getEmail(), result);
}
```

AFTER:

Java:

```java
public void processApplication(Application app) {
    validateApplication(app);
    int score = calculateScore(app);
    AdmissionResult result = determineResult(score);
    repository.save(app, result);
    notifyApplicant(app, result);
}
```

The "step" comments disappeared because the extracted functions tell the same story.

```
+-------------------------------------------------------------+
|           Self-Documenting Code Strategies                  |
+-------------------------------------------------------------+
|                                                             |
|  Instead of...          |  Try...                           |
|  -----------------------|---------------------------------- |
|  // Check if valid      |  if (isValid())                   |
|  // Max retries = 3     |  MAX_RETRIES = 3                  |
|  // Calculate discount  |  calculateDiscount()              |
|  // User is admin       |  user.isAdmin()                   |
|  // Step 1, Step 2...   |  Extract methods                  |
|  // Complex condition   |  Extract to named boolean         |
|                                                             |
+-------------------------------------------------------------+
```

A Complete Before/After Example

BEFORE: Comment-Heavy Code

Python:

```python
class OrderProcessor:
    def process(self, order_data):
        # Validate the order data
        # Check that we have all required fields
        if "customer_id" not in order_data:
            raise ValueError("Missing customer ID")  # customer ID is required
        if "items" not in order_data or len(order_data["items"]) == 0:
            raise ValueError("No items")  # must have at least one item

        # Get customer from database
```

```
        # We use the customer repository to look up by ID
        customer = self.customer_repo.find(order_data["customer_id"])
        if customer is None:
            raise ValueError("Customer not found")  # invalid customer

        # Calculate the total
        total = 0  # running total
        for item in order_data["items"]:
            # Get the product price
            product = self.product_repo.find(item["product_id"])
            # Multiply price by quantity
            total += product.price * item["quantity"]

        # Apply discount if customer is preferred
        # Preferred customers get 10% off
        if customer.is_preferred:
            total = total * 0.9  # 10% discount

        # Add tax (8%)
        total = total * 1.08  # 8% tax rate

        # Save the order
        # order = Order(customer, total)
        # self.order_repo.save(order)
        order = Order(customer_id=customer.id, total=total,
                      items=order_data["items"])
        self.order_repo.save(order)

        # Send confirmation email
        # self.email_service.send_simple(customer.email, total)
        self.email_service.send_confirmation(customer.email, order)

        return order  # return the saved order
```

This code has redundant comments, commented-out code, and comments that simply restate what the code does.

AFTER: Self-Documenting Code

Python:

```python
TAX_RATE = 0.08
PREFERRED_CUSTOMER_DISCOUNT = 0.10


class OrderProcessor:
    def process(self, order_data):
        self._validate(order_data)
        customer = self._find_customer(order_data["customer_id"])
        total = self._calculate_total(order_data["items"])
```

```
        total = self._apply_preferred_discount(total, customer)
        total = self._apply_tax(total)
        order = self._create_and_save_order(customer, total, order_data["items"])
        self.email_service.send_confirmation(customer.email, order)
        return order

    def _validate(self, order_data):
        if "customer_id" not in order_data:
            raise ValueError("Missing customer ID")
        if not order_data.get("items"):
            raise ValueError("Order must contain at least one item")

    def _find_customer(self, customer_id):
        customer = self.customer_repo.find(customer_id)
        if customer is None:
            raise CustomerNotFoundError(customer_id)
        return customer

    def _calculate_total(self, items):
        return sum(
            self.product_repo.find(item["product_id"]).price * item["quantity"]
            for item in items
        )

    def _apply_preferred_discount(self, total, customer):
        if customer.is_preferred:
            return total * (1 - PREFERRED_CUSTOMER_DISCOUNT)
        return total

    def _apply_tax(self, total):
        return total * (1 + TAX_RATE)

    def _create_and_save_order(self, customer, total, items):
        order = Order(customer_id=customer.id, total=total, items=items)
        self.order_repo.save(order)
        return order
```

The refactored version has zero comments but is far more readable. Every function name explains what happens. Constants replace magic numbers. Commented-out code is gone. The main `process()` method reads like a recipe.

Common Mistakes

1. Writing comments instead of improving the code -- If you need a comment to explain code, the code should probably be rewritten to be clearer.
2. Keeping commented-out code -- Delete it. Git remembers everything. Other developers will be afraid to remove it, so it will live forever if you leave it.

3. Writing journal comments at the top of files -- Use version control. That is literally what it is for.
4. Adding redundant Javadoc that restates parameter names -- Only document public APIs, and add information beyond what the signature already tells you.
5. Explaining "what" instead of "why" -- Good comments explain why a decision was made, not what the code does. The code already shows what it does.
6. Leaving TODO comments forever -- TODOs are temporary. Set a deadline, tie them to a ticket, and review them regularly.
7. Using comments to organize a function into sections -- If a function needs section headers, it is too long. Extract functions instead.

Best Practices

1. Try to express yourself in code first -- Before writing a comment, ask if you can rename a variable, extract a function, or introduce a constant to make the comment unnecessary.
2. When you must comment, explain why, not what -- The code shows what happens. A good comment explains the reasoning behind a non-obvious decision.
3. Keep comments close to the code they describe -- A comment that is five lines away from the code it explains might as well not exist.
4. Delete commented-out code immediately -- No exceptions. If it is important, it is in version control.
5. Review and clean up TODOs regularly -- Make TODO review part of your sprint process.
6. Document public APIs thoroughly -- This is the one place where more documentation is almost always better.
7. Treat misleading comments as bugs -- A comment that lies about the code is worse than no comment at all. When you find one, fix or delete it.
8. Use constants instead of magic number comments -- Replace `sleep(86400) // seconds in a day` with `sleep(SECONDS_PER_DAY)`.

Quick Summary

The best comment is the one you did not need to write because your code was clear enough on its own. Good comments explain why (not what), provide legal notices, warn about consequences, mark TODOs with accountability, and document public APIs. Bad comments

restate the obvious, lie about the code, maintain change journals, add noise, and worst of all, leave commented-out code rotting in the codebase. Before writing a comment, always try to make the code itself more expressive through better names, extracted functions, and meaningful constants.

Key Points

- Comments are a sign that the code failed to express itself. Always try code-level solutions first.
- Good comments explain why decisions were made, not what the code does.
- Legal comments, intent explanations, warnings, TODOs, and public API docs are the main categories of useful comments.
- Redundant comments that restate the code are pure noise and train developers to ignore all comments.
- Misleading comments are worse than no comments because they actively deceive.
- Journal comments belong in version control, not in source files.
- Commented-out code must be deleted immediately. Git remembers everything.
- Self-documenting code uses descriptive names, extracted functions, and named constants to eliminate the need for comments.
- Mandated comments on every function create a maintenance burden with no value.
- If your code needs section-header comments, your function is too long.

Practice Questions

1. You find a function with 15 lines of code and 12 lines of comments. What does this ratio suggest, and what would you do about it?
2. A colleague argues that commenting every function is "good practice for documentation." How would you respond? What is the difference between documentation and noise?
3. You encounter this comment in a codebase: `// IMPORTANT: Do not remove this sleep(500) -- it prevents a race condition with the payment processor`. Is this a good or bad comment? Explain your reasoning.
4. Explain three strategies for making code self-documenting. For each strategy, give a before/after example.

5. Why are closing brace comments (like `} // end if` or `} // end for`) a code smell? What do they indicate about the function?

Exercises

Exercise 1: Comment Cleanup

Take this code and remove all bad comments while preserving any good ones. Refactor the code to be self-documenting where possible.

```python
# User class
class User:
    # Constructor
    def __init__(self, name, email, age):
        self.name = name      # the user's name
        self.email = email    # the user's email
        self.age = age        # the user's age
        self.active = True    # whether user is active

    # Check if the user is an adult (18 or older)
    def check(self):
        # Return true if age >= 18
        return self.age >= 18

    # Deactivate the user
    # Added by Mike on 2023-10-15
    # Modified by Sarah on 2023-11-02
    def deactivate(self):
        self.active = False   # set active to false
        # self.send_notification()
        # self.log_deactivation()
```

Exercise 2: Self-Documenting Transformation

Rewrite this code so that all comments become unnecessary:

```java
public boolean validate(String input) {
    // Check if null or empty
    if (input == null || input.trim().isEmpty()) {
        return false;
    }
    // Must be between 3 and 50 characters
```

```
    if (input.length() < 3 || input.length() > 50) {
        return false;
    }
    // Must start with a letter
    if (!Character.isLetter(input.charAt(0))) {
        return false;
    }
    // Can only contain letters, numbers, and underscores
    if (!input.matches("[a-zA-Z0-9_]+")) {
        return false;
    }
    return true;
}
```

Exercise 3: Comment Audit

Review one of your own source files. Categorize each comment as one of: legal, informative, intent, warning, TODO, redundant, misleading, noise, journal, or commented-out code. Delete the bad ones and refactor the code to make the redundant ones unnecessary.

What Is Next?

You now know when comments help and when they hurt, and how to write code that speaks for itself. In Chapter 5, we turn to another aspect of code communication: formatting. Just as punctuation and paragraphs make prose readable, consistent formatting makes code scannable and maintainable. You will learn the newspaper metaphor, vertical and horizontal formatting rules, and how to use tools like Prettier, Black, and Checkstyle to enforce consistency automatically.

Chapter 5: Formatting -- The Silent Communicator

What You Will Learn

- Why code formatting is a communication tool, not a cosmetic preference
- The newspaper metaphor and how it guides vertical formatting decisions
- How vertical openness and density make code scannable at a glance
- Horizontal formatting rules including line length and indentation
- Why team rules always beat personal preferences
- How to use linters and formatters to enforce consistency automatically
- Practical before-and-after transformations in Java and Python

Why This Chapter Matters

You will spend far more time reading code than writing it. Every study on developer behavior confirms this. If code formatting is inconsistent, every reader has to decode the layout before they can understand the logic. That cognitive overhead adds up to hours, days, and weeks of lost productivity across a team.

Formatting is not about making code pretty. It is about making code communicable. A well-formatted codebase tells you where things are, what belongs together, and what is separate -- all before you read a single keyword. A poorly formatted codebase makes every file feel like a puzzle.

This chapter will give you concrete rules and tools to make formatting automatic, consistent, and effortless.

The Newspaper Metaphor

Think about how a newspaper article is structured. At the top, you get a headline that tells you what the article is about. Below that, a summary paragraph gives you the key facts. As you read further down, you get more and more detail. You can stop reading at any point and still have a useful understanding of the story.

Your source code files should work the same way.

```
+-----------------------------------------------+
|          THE NEWSPAPER METAPHOR               |
+-----------------------------------------------+
|                                               |
|  HEADLINE          --> Class name / file name |
|                                               |
|  Summary paragraph --> Public methods,        |
|                        high-level API         |
|                                               |
|  Supporting details --> Private methods,       |
|                        helper functions       |
|                                               |
|  Fine print         --> Low-level utilities,   |
|                        constants, config      |
|                                               |
+-----------------------------------------------+
|  Reader can stop at any level and still       |
|  understand the purpose of the code.          |
+-----------------------------------------------+
```

Headline First, Details Later

The name of the file or class is your headline. The first few methods should be the high-level public interface -- the "what" of the code. As you scroll down, the methods become more detailed, more specific, more "how."

BEFORE -- Details before headline:

```java
// Java -- BAD: Implementation details at the top, purpose buried below
public class ReportGenerator {

    private String sanitizeInput(String raw) {
        return raw.replaceAll("[^a-zA-Z0-9 ]", "").trim();
    }
```

```
    private List<String> splitIntoLines(String data) {
        return Arrays.asList(data.split("\n"));
    }

    private String formatLine(String line, int width) {
        return String.format("%-" + width + "s", line);
    }

    private void writeToFile(String content, String path) {
        try (FileWriter writer = new FileWriter(path)) {
            writer.write(content);
        } catch (IOException e) {
            throw new RuntimeException("Failed to write report", e);
        }
    }

    // The actual public method is buried at the bottom
    public void generateReport(String rawData, String outputPath) {
        String clean = sanitizeInput(rawData);
        List<String> lines = splitIntoLines(clean);
        StringBuilder report = new StringBuilder();
        for (String line : lines) {
            report.append(formatLine(line, 80)).append("\n");
        }
        writeToFile(report.toString(), outputPath);
    }
}
```

AFTER -- Newspaper order:

```java
// Java -- GOOD: Public purpose at the top, details below
public class ReportGenerator {

    public void generateReport(String rawData, String outputPath) {
        String clean = sanitizeInput(rawData);
        List<String> lines = splitIntoLines(clean);
        StringBuilder report = new StringBuilder();
        for (String line : lines) {
            report.append(formatLine(line, 80)).append("\n");
        }
        writeToFile(report.toString(), outputPath);
    }

    private String sanitizeInput(String raw) {
        return raw.replaceAll("[^a-zA-Z0-9 ]", "").trim();
    }

    private List<String> splitIntoLines(String data) {
        return Arrays.asList(data.split("\n"));
    }
```

```
    private String formatLine(String line, int width) {
        return String.format("%-" + width + "s", line);
    }

    private void writeToFile(String content, String path) {
        try (FileWriter writer = new FileWriter(path)) {
            writer.write(content);
        } catch (IOException e) {
            throw new RuntimeException("Failed to write report", e);
        }
    }
}
```

The same principle applies in Python:

```python
# Python -- GOOD: Public methods first, helpers below
class ReportGenerator:

    def generate_report(self, raw_data: str, output_path: str) -> None:
        """Generate a formatted report from raw data."""
        clean = self._sanitize_input(raw_data)
        lines = self._split_into_lines(clean)
        formatted = [self._format_line(line, 80) for line in lines]
        self._write_to_file("\n".join(formatted), output_path)

    def _sanitize_input(self, raw: str) -> str:
        import re
        return re.sub(r"[^a-zA-Z0-9 ]", "", raw).strip()

    def _split_into_lines(self, data: str) -> list[str]:
        return data.split("\n")

    def _format_line(self, line: str, width: int) -> str:
        return line.ljust(width)

    def _write_to_file(self, content: str, path: str) -> None:
        with open(path, "w") as f:
            f.write(content)
```

Notice how in both languages, you can read just the first method and immediately understand what this class does. The details are there if you need them, but they do not get in the way.

Vertical Formatting

Vertical formatting is about how you use vertical space -- blank lines, grouping, and ordering -- to communicate structure.

Vertical Openness: Blank Lines Between Concepts

Blank lines are visual separators. They tell the reader: "This thought is done. A new thought begins." Without blank lines, code becomes a wall of text. With too many, it becomes scattered and hard to follow.

BEFORE -- No vertical openness:

```java
// Java -- BAD: Everything crammed together
public class UserService {
    private final UserRepository repository;
    private final EmailService emailService;
    private final Logger logger;
    public UserService(UserRepository repository, EmailService emailService) {
        this.repository = repository;
        this.emailService = emailService;
        this.logger = LoggerFactory.getLogger(UserService.class);
    }
    public User createUser(String name, String email) {
        validateEmail(email);
        User user = new User(name, email);
        repository.save(user);
        emailService.sendWelcome(user);
        logger.info("Created user: {}", email);
        return user;
    }
    public void deleteUser(long id) {
        User user = repository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
        repository.delete(user);
        logger.info("Deleted user: {}", id);
    }
    private void validateEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new InvalidEmailException(email);
        }
    }
}
```

AFTER -- Vertical openness between concepts:

```java
// Java -- GOOD: Blank lines separate distinct concepts
public class UserService {

    private final UserRepository repository;
    private final EmailService emailService;
    private final Logger logger;

    public UserService(UserRepository repository, EmailService emailService) {
```

```
        this.repository = repository;
        this.emailService = emailService;
        this.logger = LoggerFactory.getLogger(UserService.class);
    }

    public User createUser(String name, String email) {
        validateEmail(email);

        User user = new User(name, email);
        repository.save(user);

        emailService.sendWelcome(user);
        logger.info("Created user: {}", email);

        return user;
    }

    public void deleteUser(long id) {
        User user = repository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));

        repository.delete(user);
        logger.info("Deleted user: {}", id);
    }

    private void validateEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new InvalidEmailException(email);
        }
    }
}
```

The difference is dramatic. In the "after" version, your eyes can quickly scan and identify: fields, constructor, public methods, private methods. Each concept has breathing room.

Vertical Density: Related Code Together

While blank lines separate unrelated concepts, you should keep related lines close together. Do not scatter things that belong together.

BEFORE -- Related things separated:

```python
# Python -- BAD: Related code scattered across the file

class OrderProcessor:

    def __init__(self):
        self.tax_rate = 0.08
```

```
    # ... 50 lines of other methods ...

    def calculate_subtotal(self, items: list[dict]) -> float:
        return sum(item["price"] * item["quantity"] for item in items)

    # ... 30 more lines of other methods ...

    def calculate_tax(self, subtotal: float) -> float:
        return subtotal * self.tax_rate

    # ... 20 more lines of other methods ...

    def calculate_total(self, items: list[dict]) -> float:
        subtotal = self.calculate_subtotal(items)
        tax = self.calculate_tax(subtotal)
        return subtotal + tax
```

AFTER -- Related things together:

```python
# Python -- GOOD: Calculation methods grouped together

class OrderProcessor:

    def __init__(self):
        self.tax_rate = 0.08

    def calculate_total(self, items: list[dict]) -> float:
        subtotal = self.calculate_subtotal(items)
        tax = self.calculate_tax(subtotal)
        return subtotal + tax

    def calculate_subtotal(self, items: list[dict]) -> float:
        return sum(item["price"] * item["quantity"] for item in items)

    def calculate_tax(self, subtotal: float) -> float:
        return subtotal * self.tax_rate
```

The caller (`calculate_total`) is near the methods it calls. A reader exploring the total calculation can see all three methods without scrolling.

Vertical Distance: Declarations Close to Usage

Variables should be declared as close to their first use as possible. In Java, this means declaring variables right before the loop or block that uses them, not at the top of a long method.

```java
// Java -- BAD: Variable declared far from use
public void processOrders(List<Order> orders) {
    double totalRevenue = 0;       // Declared here...
    int failedCount = 0;           // ...but used 15 lines later
    Logger auditLogger = getAuditLogger();

    for (Order order : orders) {
        // ... 10 lines of validation ...
        totalRevenue += order.getAmount();  // First use, far from declaration
    }

    // failedCount used much later...
}

// Java -- GOOD: Variable declared close to use
public void processOrders(List<Order> orders) {
    Logger auditLogger = getAuditLogger();

    double totalRevenue = 0;
    for (Order order : orders) {
        // ... validation ...
        totalRevenue += order.getAmount();
    }

    int failedCount = countFailedOrders(orders);
    auditLogger.info("Failed: {}", failedCount);
}
```

Horizontal Formatting

Horizontal formatting is about how you use horizontal space: line length, alignment, indentation, and whitespace within a line.

Line Length: The 80-120 Character Rule

The ideal line length is a subject of passionate debate. Here is the practical answer: keep lines between 80 and 120 characters. Anything beyond 120 forces horizontal scrolling, makes side-by-side diffs painful, and breaks readability on smaller screens.

```
+-------------------------------------------------------------+
|              LINE LENGTH GUIDE                               |
+-------------------------------------------------------------+
|                                                              |
|  0          40          80         120         160           |
|  |----------|-----------|----------|-----------|            |
```

```
|  [    Sweet spot: 80-100 chars    ]                        |
|  [       Acceptable: up to 120    ........]                |
|  [       Avoid: beyond 120        ...................]     |
|                                                             |
|  Why?                                                       |
|  - Fits on most screens without scrolling                   |
|  - Works with side-by-side diff tools                       |
|  - Readable in code review interfaces                       |
|  - Allows split-screen editing                              |
|                                                             |
+-------------------------------------------------------------+
```

BEFORE -- Lines too long:

```java
// Java -- BAD: Horizontal scrolling required
public List<CustomerOrderSummary> getActiveCustomerOrderSummariesByDateRangeAndStatus(L...
    return customerRepository.findAllByRegistrationDateBetweenAndStatusEquals(startDate...
}
```

AFTER -- Properly broken lines:

```java
// Java -- GOOD: Each concept on its own line
public List<CustomerOrderSummary> getActiveCustomerOrderSummaries(
        LocalDate startDate,
        LocalDate endDate,
        OrderStatus status,
        boolean includeArchived) {

    return customerRepository
        .findByDateRangeAndStatus(startDate, endDate, status)
        .stream()
        .filter(customer -> includeArchived || !customer.isArchived())
        .map(customer -> buildSummary(customer, status))
        .collect(Collectors.toList());
}

private CustomerOrderSummary buildSummary(Customer customer, OrderStatus status) {
    long orderCount = customer.getOrders().stream()
        .filter(order -> order.getStatus() == status)
        .count();

    return new CustomerOrderSummary(
        customer.getId(),
        customer.getName(),
        orderCount
    );
}
```

The same principle in Python:

```python
# Python -- BAD: One massive line
def get_active_summaries(start, end, status, include_archived):
    return [{"id": c.id, "name": c.name, "count": len([o for o in c.orders if o.status ...

# Python -- GOOD: Broken into readable lines
def get_active_summaries(
    start: date,
    end: date,
    status: str,
    include_archived: bool = False,
) -> list[dict]:
    customers = repo.find_by_date_range(start, end, status)

    return [
        build_summary(customer, status)
        for customer in customers
        if include_archived or not customer.archived
    ]


def build_summary(customer: Customer, status: str) -> dict:
    matching_orders = [
        order for order in customer.orders
        if order.status == status
    ]

    return {
        "id": customer.id,
        "name": customer.name,
        "count": len(matching_orders),
    }
```

Horizontal Whitespace: Use It to Show Association

Whitespace within a line communicates which things are related and which are separate.

```java
// Java -- Whitespace shows association
private void measureLine(String line) {
    lineCount++;                          // No space around ++ (tight binding)
    int lineSize = line.length();         // Space around = (assignment)
    totalChars += lineSize;               // Space around += (assignment)
    lineWidthHistogram.addLine(lineSize); // No space before ( (method call)
}

// Operators: space around low-precedence, no space around high-precedence
int result = a*b + c*d;    // Multiplication binds tighter than addition
double det = b*b - 4*a*c;  // Visual grouping matches mathematical grouping
```

```python
# Python -- Consistent whitespace
def calculate_score(hits: int, misses: int, bonus: float) -> float:
    base_score = hits*10 - misses*5   # Tight binding for arithmetic
    return base_score + bonus         # Space around + for clarity
```

Indentation: The Structure Visualizer

Indentation reveals the hierarchical structure of your code. Without it, you cannot tell what is inside a loop, a conditional, a method, or a class.

BEFORE -- Broken indentation:

```python
# Python -- BAD: Inconsistent indentation
class DataProcessor:
  def process(self, data):
        for item in data:
            if item.is_valid():
              result = self.transform(item)
                  self.save(result)
          else:
            self.log_error(item)
```

AFTER -- Consistent indentation:

```python
# Python -- GOOD: Consistent 4-space indentation (PEP 8)
class DataProcessor:

    def process(self, data: list) -> None:
        for item in data:
            if item.is_valid():
                result = self.transform(item)
                self.save(result)
            else:
                self.log_error(item)
```

The tabs vs. spaces debate: Pick one. The industry has largely settled on spaces (4 spaces for Python per PEP 8, 4 spaces for Java by convention, 2 spaces for JavaScript by convention). What matters is that everyone on the team uses the same setting. Configure your editor and never think about it again.

Team Rules Beat Personal Preferences

This is the most important formatting rule in this entire chapter:

A team's formatting rules always override your personal preferences.

It does not matter if you prefer tabs over spaces, or 2-space indentation over 4-space, or opening braces on the same line versus the next line. What matters is consistency across the codebase. A codebase where every file looks different is harder to read than one where every file follows a convention you do not personally love.

```
+-------------------------------------------------------------+
|           PERSONAL PREFERENCE vs TEAM STANDARD              |
+-------------------------------------------------------------+
|                                                             |
|  Developer A:  "I prefer tabs and 80-char lines."          |
|  Developer B:  "I prefer spaces and 120-char lines."       |
|  Developer C:  "I prefer K&R braces."                      |
|  Developer D:  "I prefer Allman braces."                   |
|                                                             |
|  Result if everyone follows their preference:               |
|  --> Every file looks different                             |
|  --> Git diffs are full of formatting noise                 |
|  --> Code reviews waste time on style arguments             |
|                                                             |
|  Result if everyone follows team standard:                  |
|  --> Every file looks the same                              |
|  --> Git diffs show only real changes                       |
|  --> Code reviews focus on logic and design                 |
|                                                             |
+-------------------------------------------------------------+
```

How to Establish Team Rules

1. Pick a well-known style guide as your starting point (Google Java Style, PEP 8, Airbnb JavaScript)
2. Discuss and document any deviations the team agrees on
3. Configure an automated formatter to enforce the rules
4. Run the formatter as part of your CI/CD pipeline
5. Never argue about formatting in a code review again

Linters and Formatters: Automate Your Style

The best formatting is formatting you never have to think about. Modern tools can enforce style rules automatically.

The Tooling Landscape

```
+-------------------------------------------------------------+
|              FORMATTING TOOLS BY LANGUAGE                    |
+-------------------------------------------------------------+
|                                                              |
|  Language    | Formatter        | Linter                    |
|  ------------|------------------|---------------------------|
|  Python      | Black            | Pylint, Ruff, Flake8      |
|  Java        | google-java-fmt  | Checkstyle, PMD           |
|  JavaScript  | Prettier         | ESLint                    |
|  TypeScript  | Prettier         | ESLint, TSLint (legacy)   |
|  Go          | gofmt            | (built-in)                |
|  Rust        | rustfmt          | clippy                    |
|                                                              |
+-------------------------------------------------------------+
|                                                              |
|  Formatter = automatically rewrites code to match style      |
|  Linter    = reports violations (may or may not auto-fix)    |
|                                                              |
+-------------------------------------------------------------+
```

Formatter vs. Linter: What Is the Difference?

A formatter automatically rewrites your code to conform to style rules. You run it, and your code is reformatted. No decisions, no arguments. Black for Python is famously "uncompromising" -- it gives you almost zero configuration options, and that is the point.

A linter analyzes your code for potential problems -- not just style issues but also bugs, complexity, and anti-patterns. Linters often report issues for you to fix manually, though many support auto-fix for simple problems.

Python: Black in Action

```python
# Python -- BEFORE Black formats it
def calculate_price(base_price,discount_percentage,tax_rate,
    is_member):
    if is_member==True:
        discount=discount_percentage+5
    else:
        discount=discount_percentage
    subtotal=base_price*(1-discount/100)
    total=subtotal*(1+tax_rate/100)
    return   total
```

```python
# Python -- AFTER Black formats it
def calculate_price(
    base_price,
    discount_percentage,
    tax_rate,
    is_member,
):
    if is_member:
        discount = discount_percentage + 5
    else:
        discount = discount_percentage

    subtotal = base_price * (1 - discount / 100)
    total = subtotal * (1 + tax_rate / 100)
    return total
```

Black fixed: spacing around operators, trailing commas, parameter alignment, the `== True` anti-pattern is flagged by linters, and consistent blank lines.

Java: Checkstyle Configuration

```xml
<!-- checkstyle.xml -- Example configuration -->
<module name="Checker">
    <module name="TreeWalker">
        <module name="Indentation">
            <property name="basicOffset" value="4"/>
        </module>
        <module name="LineLength">
            <property name="max" value="120"/>
        </module>
        <module name="NeedBraces"/>
        <module name="LeftCurly">
            <property name="option" value="eol"/>
        </module>
    </module>
</module>
```

Setting Up Pre-Commit Hooks

The best way to enforce formatting is to make it impossible to commit unformatted code:

```bash
# .pre-commit-config.yaml (Python project using pre-commit framework)
repos:
  - repo: https://github.com/psf/black
    rev: 24.3.0
    hooks:
```

```
      - id: black
  - repo: https://github.com/charliermarsh/ruff-pre-commit
    rev: v0.3.0
    hooks:
      - id: ruff
        args: [--fix]
```

```bash
# For Java (using Maven)
# pom.xml plugin that runs Checkstyle on every build
mvn checkstyle:check
```

Complete Before/After: A Messy File Cleaned Up

Let us look at a complete transformation. This is a realistic example of code you might encounter in a real project.

BEFORE: The Mess

```java
// Java -- BEFORE: Multiple formatting violations
public class    invoiceCalculator{
private static final double TAX=0.08;
private static final double MEMBER_DISCOUNT=0.05;
    public double calculateInvoiceTotal(List<LineItem> items,boolean isMember,String co...
double subtotal=0;
for(LineItem item:items){if(item.getQuantity()>0){
subtotal+=item.getPrice()*item.getQuantity();}else{System.out.println("invalid quantity...
double discount=0;
if(isMember){discount=MEMBER_DISCOUNT;}
if(couponCode!=null&&!couponCode.isEmpty()){if(couponCode.equals("SAVE10")){discount+=0...
double afterDiscount=subtotal*(1-discount);
        double tax=afterDiscount*TAX;
return afterDiscount+tax;
}
public void printInvoice(List<LineItem> items,boolean isMember,String coupon){
double total=calculateInvoiceTotal(items,isMember,coupon);
System.out.println("=== INVOICE ===");
for(LineItem item:items){System.out.println(item.getName()+": $"+item.getPrice()+" x "+...
System.out.println("Total: $"+String.format("%.2f",total));
System.out.println("===============");
}
}
```

AFTER: Clean and Readable

```java
// Java -- AFTER: Properly formatted
public class InvoiceCalculator {

    private static final double TAX_RATE = 0.08;
    private static final double MEMBER_DISCOUNT = 0.05;

    private static final Map<String, Double> COUPON_DISCOUNTS = Map.of(
        "SAVE10", 0.10,
        "SAVE20", 0.20
    );

    public double calculateInvoiceTotal(
            List<LineItem> items,
            boolean isMember,
            String couponCode) {

        double subtotal = calculateSubtotal(items);
        double discount = calculateDiscount(isMember, couponCode);
        double afterDiscount = subtotal * (1 - discount);
        double tax = afterDiscount * TAX_RATE;

        return afterDiscount + tax;
    }

    public void printInvoice(
            List<LineItem> items,
            boolean isMember,
            String couponCode) {

        double total = calculateInvoiceTotal(items, isMember, couponCode);

        System.out.println("=== INVOICE ===");
        for (LineItem item : items) {
            System.out.printf(
                "%s: $%.2f x %d%n",
                item.getName(),
                item.getPrice(),
                item.getQuantity()
            );
        }
        System.out.printf("Total: $%.2f%n", total);
        System.out.println("===============");
    }

    private double calculateSubtotal(List<LineItem> items) {
        double subtotal = 0;

        for (LineItem item : items) {
            if (item.getQuantity() <= 0) {
                throw new IllegalArgumentException(
                    "Invalid quantity for item: " + item.getName()
```

```
                );
            }
            subtotal += item.getPrice() * item.getQuantity();
        }

        return subtotal;
    }

    private double calculateDiscount(boolean isMember, String couponCode) {
        double discount = 0;

        if (isMember) {
            discount = MEMBER_DISCOUNT;
        }

        if (couponCode != null && !couponCode.isEmpty()) {
            Double couponDiscount = COUPON_DISCOUNTS.get(couponCode);
            if (couponDiscount == null) {
                throw new IllegalArgumentException(
                    "Invalid coupon code: " + couponCode
                );
            }
            discount += couponDiscount;
        }

        return discount;
    }
}
```

The same transformation in Python:

BEFORE: Python Mess

```python
# Python -- BEFORE: Formatting chaos
class   invoiceCalculator:
    TAX=0.08
    MEMBER_DISCOUNT=0.05
    def calculateTotal(self,items,is_member,coupon_code):
        subtotal=0
        for item in items:
            if item['qty']>0:
                    subtotal+=item['price']*item['qty']
            else:
                print("bad qty")
        discount=0
        if is_member==True:discount=self.MEMBER_DISCOUNT
        if coupon_code!=None:
          if coupon_code=="SAVE10":discount+=0.10
          elif coupon_code=="SAVE20":discount+=0.20
          else:print("bad coupon")
        return (subtotal*(1-discount))*(1+self.TAX)
```

AFTER: Python Clean

```python
# Python — AFTER: PEP 8 compliant, Black-formatted
class InvoiceCalculator:

    TAX_RATE = 0.08
    MEMBER_DISCOUNT = 0.05

    COUPON_DISCOUNTS = {
        "SAVE10": 0.10,
        "SAVE20": 0.20,
    }

    def calculate_total(
        self,
        items: list[dict],
        is_member: bool,
        coupon_code: str | None = None,
    ) -> float:
        subtotal = self._calculate_subtotal(items)
        discount = self._calculate_discount(is_member, coupon_code)
        after_discount = subtotal * (1 - discount)
        tax = after_discount * self.TAX_RATE

        return after_discount + tax

    def _calculate_subtotal(self, items: list[dict]) -> float:
        subtotal = 0.0

        for item in items:
            if item["qty"] <= 0:
                raise ValueError(
                    f"Invalid quantity for item: {item['name']}"
                )
            subtotal += item["price"] * item["qty"]

        return subtotal

    def _calculate_discount(
        self,
        is_member: bool,
        coupon_code: str | None,
    ) -> float:
        discount = 0.0

        if is_member:
            discount = self.MEMBER_DISCOUNT

        if coupon_code:
            coupon_discount = self.COUPON_DISCOUNTS.get(coupon_code)
            if coupon_discount is None:
                raise ValueError(f"Invalid coupon code: {coupon_code}")
            discount += coupon_discount
```

```
        return discount
```

Common Mistakes

1. No formatter configured. Teams that rely on "everyone will follow the style guide" always end up with inconsistent code. Automate it.
2. Arguing about formatting in code reviews. If your team is spending review time on formatting, you have not set up your tooling correctly. Let the formatter handle it.
3. Giant commits that mix formatting and logic changes. If you need to reformat a file, do it in a separate commit with the message "Apply formatter" so that logic changes are easy to review in isolation.
4. Ignoring vertical formatting. Many developers get horizontal formatting right (indentation, line length) but neglect vertical formatting (blank lines, ordering, grouping). Both matter.
5. Over-formatting. Do not add blank lines between every single line of code. Blank lines should separate concepts, not every statement. Three related assignments can stay together.
6. Using different formatters in the same project. If half the team uses Prettier with one config and the other half uses a different config, you get endless formatting battles in version control.

Best Practices

1. Configure a formatter on day one of every project. Do not wait until the codebase is large.
2. Use pre-commit hooks to enforce formatting. Make it impossible to commit unformatted code.
3. Follow the newspaper metaphor. Public interface at the top, implementation details below.
4. Use blank lines to separate concepts, not individual lines. Group related code together.
5. Keep lines under 120 characters. Aim for 80-100 as the sweet spot.
6. Declare variables close to their first use. Do not declare everything at the top of a method.

7. Adopt a well-known style guide. PEP 8 for Python, Google Java Style for Java. Do not invent your own.
8. Make formatting changes in separate commits. Never mix formatting and logic in the same commit.

Quick Summary

```
+-------------------------------------------------------------+
|              FORMATTING CHEAT SHEET                         |
+-------------------------------------------------------------+
|                                                             |
|  VERTICAL:                                                  |
|    - Newspaper metaphor (headline -> details)               |
|    - Blank lines between concepts (openness)                |
|    - Related code together (density)                        |
|    - Variables declared near first use                      |
|                                                             |
|  HORIZONTAL:                                                |
|    - Line length: 80-120 characters                         |
|    - Consistent indentation (spaces, not tabs)              |
|    - Whitespace shows association                           |
|    - Break long lines at logical points                     |
|                                                             |
|  TEAM:                                                      |
|    - Team rules > personal preference                       |
|    - Adopt a standard style guide                           |
|    - Automate with formatters (Black, Prettier)             |
|    - Enforce with pre-commit hooks and CI                   |
|                                                             |
+-------------------------------------------------------------+
```

Key Points

- Formatting is a communication tool that makes code scannable and understandable before you read a single keyword
- The newspaper metaphor means putting high-level public methods first and implementation details below
- Vertical openness (blank lines between concepts) and vertical density (related code together) work together to create readable structure
- Horizontal formatting should keep lines under 120 characters with consistent indentation

- Team formatting rules always override personal preferences because consistency across a codebase is more valuable than any individual style choice
- Formatters like Black and Prettier automate style enforcement so developers never need to argue about formatting
- Formatting changes should always be in separate commits from logic changes

---

Practice Questions

1. Explain the newspaper metaphor for code formatting. How does it guide the ordering of methods within a class?
2. What is the difference between vertical openness and vertical density? Give an example of when you would use each.
3. Why should team formatting rules take priority over personal preferences? What problems arise when every developer follows their own style?
4. What is the difference between a formatter and a linter? Name one of each for Python and one of each for Java.
5. You are reviewing a pull request that contains both a new feature and a codebase-wide reformatting. What would you tell the author?

Exercises

Exercise 1: Reformat a Messy File

Take the following code and apply proper formatting rules. Fix vertical openness, vertical density, horizontal line length, indentation, and method ordering.

```python
class  reportBuilder:
  def _header(self):return "=== REPORT ==="
  def generate(self,data,format_type):
        result=self._header()+"\n"
        for item in data:
            if format_type=="csv":result+=",".join(str(v) for v in item.values())+"\n"
            elif format_type=="table":
                    result+="|".join(f"{str(v):>15}" for v in item.values())+"\n"
            else:result+= str(item)+"\n"
```

```
        result+=self._footer()
        return result
    def _footer(self):return "=== END ===="
```

Exercise 2: Configure a Formatter

Set up Black for a Python project with a `pyproject.toml` configuration. Include settings for line length of 100 characters and target Python 3.11. Then set up a pre-commit hook that runs Black automatically before every commit.

Exercise 3: Evaluate Formatting Choices

Look at an open-source project on GitHub in a language you use regularly. Examine three different files and answer: Does the project follow consistent formatting? What style guide does it appear to follow? Is there a formatter configured? Check for `.prettierrc`, `pyproject.toml`, `checkstyle.xml`, or similar configuration files.

What Is Next?

Now that you know how to make code visually consistent and scannable, the next chapter tackles a topic that can make or break your application's reliability: error handling. You will learn why exceptions beat error codes, how to build a clean exception hierarchy, and how to separate error handling from business logic so that neither clutters the other.

Chapter 6: Error Handling -- Keeping the Happy Path Clean

What You Will Learn

- Why exceptions are superior to error codes for communicating failure
- How to use unchecked exceptions effectively and when checked exceptions cause harm
- How to provide meaningful context in your exceptions so debugging is fast
- How to define exception classes based on the caller's needs, not the implementation
- Why returning null is a bug factory and what to return instead
- The try-catch-finally structure and how to use it without cluttering your code
- How to build a custom exception hierarchy for your application
- The fail fast principle and why it saves you hours of debugging
- How to separate error handling from business logic cleanly

Why This Chapter Matters

Error handling is one of those topics that separates amateur code from professional code. In amateur code, errors are an afterthought -- sprinkled in as `if (result == -1)` checks, swallowed with empty catch blocks, or ignored entirely until production explodes. In professional code, error handling is a first-class concern that is designed into the architecture from the beginning.

The challenge is doing error handling without turning every function into a tangled mess of try-catch blocks and null checks. This chapter teaches you how to handle errors thoroughly while keeping your code readable and your business logic clean.

Exceptions vs. Error Codes: Why Exceptions Win

In the early days of programming, functions communicated failure by returning special values. A function might return `-1` for an error, `0` for success, or `null` to indicate "not found." This approach has a fatal flaw: the caller can ignore the error code without the compiler complaining.

```
+-------------------------------------------------------------+
|         ERROR CODES vs EXCEPTIONS                           |
+-------------------------------------------------------------+
|                                                             |
|  Error Codes:                                               |
|  +--------+    returns -1     +--------+                   |
|  | Caller | <--------------- | Method |                   |
|  +--------+                   +--------+                   |
|  Caller can IGNORE the -1.                                  |
|  No compiler warning. Bug hides silently.                   |
|                                                             |
|  Exceptions:                                                |
|  +--------+    throws Error   +--------+                   |
|  | Caller | <======X========= | Method |                   |
|  +--------+                   +--------+                   |
|  Caller MUST handle or propagate.                           |
|  Cannot be silently ignored.                                |
|  Stack trace pinpoints the failure.                         |
|                                                             |
+-------------------------------------------------------------+
```

BEFORE: Error Code Spaghetti (Java)

```java
// Java -- BAD: Error codes obscure the business logic
public class OrderService {

    public int processOrder(Order order) {
        int validationResult = validateOrder(order);
        if (validationResult != 0) {
            return validationResult;  // -1 = invalid items, -2 = no stock
        }

        int paymentResult = chargePayment(order);
        if (paymentResult != 0) {
            return paymentResult;  // -3 = card declined, -4 = network error
        }
```

```
        int shippingResult = arrangeShipping(order);
        if (shippingResult != 0) {
            // Undo payment because shipping failed
            int refundResult = refundPayment(order);
            if (refundResult != 0) {
                return -99;  // Catastrophic: payment taken but cannot refund
            }
            return shippingResult;  // -5 = no carriers, -6 = invalid address
        }

        int notifyResult = notifyCustomer(order);
        if (notifyResult != 0) {
            // Shipping is done, so we log the notification failure
            // but do not roll back. Or do we? Who knows.
            logError("Notification failed: " + notifyResult);
        }

        return 0;  // Success... probably
    }
}
```

What is wrong with this code? The business logic -- validate, charge, ship, notify -- is buried under a mountain of error-checking boilerplate. Every caller of `processOrder` has to remember what `-1`, `-2`, `-3`, `-4`, `-5`, `-6`, and `-99` mean. Nothing enforces that. Nothing prevents a caller from ignoring the return value entirely.

AFTER: Clean Exception Handling (Java)

```java
// Java -- GOOD: Exceptions separate error handling from business logic
public class OrderService {

    public void processOrder(Order order) {
        validateOrder(order);
        chargePayment(order);

        try {
            arrangeShipping(order);
        } catch (ShippingException e) {
            refundPayment(order);
            throw new OrderProcessingException(
                "Shipping failed after payment. Refund issued.", e
            );
        }

        try {
            notifyCustomer(order);
        } catch (NotificationException e) {
            logger.warn("Customer notification failed for order {}",
                order.getId(), e);
```

```
            // Non-critical: order still succeeds
        }
    }
}
```

The happy path reads like a story: validate, charge, ship, notify. Error handling is present but does not obscure the flow. Each exception carries its own meaning -- you do not need a lookup table to understand `ShippingException`.

BEFORE: Error Code Spaghetti (Python)

```python
# Python -- BAD: Error codes returned as tuples
class OrderService:

    def process_order(self, order):
        success, error = self.validate_order(order)
        if not success:
            return False, error

        success, error = self.charge_payment(order)
        if not success:
            return False, f"Payment failed: {error}"

        success, error = self.arrange_shipping(order)
        if not success:
            refund_ok, refund_err = self.refund_payment(order)
            if not refund_ok:
                return False, f"CRITICAL: {error} AND refund failed: {refund_err}"
            return False, f"Shipping failed: {error}. Payment refunded."

        success, error = self.notify_customer(order)
        if not success:
            print(f"Warning: notification failed: {error}")
            # Continue anyway? Maybe?

        return True, None
```

AFTER: Clean Exception Handling (Python)

```python
# Python -- GOOD: Exceptions keep the happy path clean
class OrderService:

    def process_order(self, order: Order) -> None:
        self.validate_order(order)
        self.charge_payment(order)
```

```
        try:
            self.arrange_shipping(order)
        except ShippingError as e:
            self.refund_payment(order)
            raise OrderProcessingError(
                f"Shipping failed after payment. Refund issued."
            ) from e

        try:
            self.notify_customer(order)
        except NotificationError:
            logger.warning(
                "Customer notification failed for order %s",
                order.id,
            )
```

Use Unchecked Exceptions

Java has two kinds of exceptions: checked (you must declare them with `throws` or catch them) and unchecked (subclasses of `RuntimeException`, no declaration required). Checked exceptions seemed like a good idea -- the compiler forces you to handle errors! -- but in practice they cause significant problems.

```
+-------------------------------------------------------------+
|       CHECKED vs UNCHECKED EXCEPTIONS                       |
+-------------------------------------------------------------+
|                                                             |
|  Checked (Java):                                            |
|    - Must be declared in method signature                   |
|    - Every caller must handle or re-declare                 |
|    - Changes propagate through entire call chain            |
|    - Example: IOException, SQLException                     |
|                                                             |
|  Unchecked (Java):                                          |
|    - No declaration required                                |
|    - Caller handles only if they choose to                  |
|    - Changes do not break calling code                      |
|    - Example: IllegalArgumentException, RuntimeException    |
|                                                             |
|  Python:                                                    |
|    - ALL exceptions are unchecked                           |
|    - No throws declaration in method signatures             |
|    - Document expected exceptions in docstrings             |
|                                                             |
+-------------------------------------------------------------+
```

The problem with checked exceptions is that they violate the Open-Closed Principle. If a low-level method adds a new checked exception, every method in the call chain above it must be modified to either catch or declare that exception. A single change at the bottom can cause modifications to dozens of files.

```java
// Java -- Checked exception chain reaction
// If parseData adds a new checked exception, ALL callers must change:

// Level 3 (low-level)
public Data parseData(String raw) throws ParseException, ValidationException {
    // Adding FormatException here forces changes in Levels 2 and 1
}

// Level 2 (mid-level) -- must declare the new exception
public Report generateReport(String input)
        throws ParseException, ValidationException {
    Data data = parseData(input);
    return buildReport(data);
}

// Level 1 (high-level) -- must also change
public void handleRequest(Request request)
        throws ParseException, ValidationException {
    Report report = generateReport(request.getData());
    sendResponse(report);
}
```

Prefer unchecked exceptions. Use `RuntimeException` subclasses in Java. In Python, all exceptions are already unchecked.

```java
// Java -- GOOD: Unchecked exception, no chain reaction
public class ParseException extends RuntimeException {

    public ParseException(String message) {
        super(message);
    }

    public ParseException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Provide Context with Exceptions

When an exception occurs, the developer debugging it needs to know three things: what happened, where it happened, and why it happened. A good exception message answers all three.

BAD exception messages:

```java
throw new RuntimeException("Error");
throw new RuntimeException("Failed");
throw new IllegalArgumentException("Invalid input");
```

GOOD exception messages:

```java
throw new OrderNotFoundException(
    "Order with ID 12345 not found in database 'orders_prod'"
);

throw new PaymentDeclinedException(
    "Payment of $49.99 declined for customer C-789. "
    + "Reason: insufficient funds. Card ending in 4242."
);

throw new ConfigurationException(
    "Required environment variable 'DATABASE_URL' is not set. "
    + "Set it in .env or as a system environment variable."
);
```

```python
# Python -- GOOD: Context-rich exceptions
raise OrderNotFoundError(
    f"Order with ID {order_id} not found in database '{db_name}'"
)

raise PaymentDeclinedError(
    f"Payment of ${amount:.2f} declined for customer {customer_id}. "
    f"Reason: {decline_reason}. Card ending in {card_last_four}."
)

raise ConfigurationError(
    "Required environment variable 'DATABASE_URL' is not set. "
    "Set it in .env or as a system environment variable."
)
```

Preserve the Cause Chain

When you catch an exception and throw a new one, always include the original exception as the cause. This preserves the full stack trace for debugging.

```java
// Java -- GOOD: Preserve the cause
try {
    database.save(order);
} catch (SQLException e) {
    throw new OrderPersistenceException(
        "Failed to save order " + order.getId(), e  // 'e' is the cause
    );
}
```

```python
# Python -- GOOD: Use 'from' to chain exceptions
try:
    database.save(order)
except DatabaseError as e:
    raise OrderPersistenceError(
        f"Failed to save order {order.id}"
    ) from e  # Preserves the original traceback
```

Define Exception Classes by the Caller's Needs

When you design your exception hierarchy, think about what the caller needs to do when an error occurs, not about what went wrong internally.

BAD: Exceptions organized by internal cause:

```java
// Java -- BAD: Caller has to know about internal details
try {
    port.open();
} catch (DeviceResponseException e) {
    reportPortError(e);
    logger.error("Device response error", e);
} catch (ATM1212UnlockedException e) {
    reportPortError(e);
    logger.error("Unlock error", e);
} catch (GMXError e) {
    reportPortError(e);
    logger.error("GMX error", e);
}
```

All three catch blocks do the same thing. The caller does not care whether the device responded incorrectly or used the wrong protocol. The caller just needs to know: did the port open or not?

GOOD: One exception that wraps the details:

```java
// Java -- GOOD: Wrap third-party exceptions
public class LocalPort {

    private final ACMEPort innerPort;

    public void open() {
        try {
            innerPort.open();
        } catch (DeviceResponseException e) {
            throw new PortDeviceFailure(e);
        } catch (ATM1212UnlockedException e) {
            throw new PortDeviceFailure(e);
        } catch (GMXError e) {
            throw new PortDeviceFailure(e);
        }
    }
}

// Now the caller has one clean catch:
try {
    port.open();
} catch (PortDeviceFailure e) {
    reportError(e);
    logger.error("Failed to open port", e);
}
```

```python
# Python -- GOOD: Wrap third-party exceptions
class LocalPort:

    def __init__(self, inner_port):
        self._inner_port = inner_port

    def open(self) -> None:
        try:
            self._inner_port.open()
        except (DeviceResponseError, UnlockedError, GMXError) as e:
            raise PortDeviceFailure(
                f"Failed to open port: {e}"
            ) from e


# Caller sees one clean exception
try:
    port.open()
except PortDeviceFailure as e:
    report_error(e)
    logger.error("Failed to open port", exc_info=True)
```

Do Not Return Null

Returning `null` (Java) or `None` (Python) to indicate absence is one of the most common sources of bugs. Every caller must remember to check for null, and if even one forgets, you get a `NullPointerException` at runtime.

```
+-------------------------------------------------------------+
|           THE NULL PROBLEM                                  |
+-------------------------------------------------------------+
|                                                             |
|  Method returns null                                        |
|       |                                                     |
|       v                                                     |
|  Caller A checks for null    --> OK                         |
|  Caller B checks for null    --> OK                         |
|  Caller C forgets to check   --> NullPointerException       |
|  Caller D forgets to check   --> NullPointerException       |
|                                                             |
|  The more callers, the more likely someone forgets.         |
|  null is a ticking time bomb in your API.                   |
|                                                             |
+-------------------------------------------------------------+
```

BEFORE: Null Returns

```java
// Java -- BAD: Returning null
public class UserRepository {

    public User findByEmail(String email) {
        // Returns null if not found
        return database.query("SELECT * FROM users WHERE email = ?", email);
    }
}

// Every caller must do this -- and many will forget:
User user = repository.findByEmail(email);
if (user != null) {
    sendWelcomeEmail(user);  // Safe
} else {
    // Handle not found
}

// The inevitable bug:
User user = repository.findByEmail(email);
sendWelcomeEmail(user);  // NullPointerException if not found
```

AFTER: Optional and Empty Collections

```java
// Java -- GOOD: Return Optional for single values
public class UserRepository {

    public Optional<User> findByEmail(String email) {
        User user = database.query(
            "SELECT * FROM users WHERE email = ?", email
        );
        return Optional.ofNullable(user);
    }
}

// Caller is forced to handle the absence:
repository.findByEmail(email)
    .ifPresentOrElse(
        user -> sendWelcomeEmail(user),
        () -> logger.warn("User not found: {}", email)
    );

// Or with a default:
User user = repository.findByEmail(email)
    .orElseThrow(() -> new UserNotFoundException(email));
```

```java
// Java -- GOOD: Return empty collection, never null
public List<Order> findOrdersByCustomer(String customerId) {
    List<Order> orders = database.query(
        "SELECT * FROM orders WHERE customer_id = ?", customerId
    );
    return orders != null ? orders : Collections.emptyList();
}

// Caller can always iterate safely -- no null check needed:
for (Order order : findOrdersByCustomer(customerId)) {
    processOrder(order);
}
```

In Python, the same principles apply:

```python
# Python -- BAD: Returning None
def find_user_by_email(email: str):
    user = database.query("SELECT * FROM users WHERE email = %s", email)
    return user  # Could be None


# Python -- GOOD: Raise an exception when absence is an error
def find_user_by_email(email: str) -> User:
    user = database.query("SELECT * FROM users WHERE email = %s", email)
    if user is None:
        raise UserNotFoundError(f"No user with email: {email}")
    return user
```

```
# Python -- GOOD: Return a sentinel or use Optional typing when absence is normal
def find_user_by_email(email: str) -> User | None:
    """Returns the User or None if not found."""
    return database.query("SELECT * FROM users WHERE email = %s", email)


# Python -- GOOD: Return empty collection, never None
def find_orders_by_customer(customer_id: str) -> list[Order]:
    orders = database.query(
        "SELECT * FROM orders WHERE customer_id = %s", customer_id
    )
    return orders or []
```

Do Not Pass Null

If returning null is bad, passing null as an argument is worse. When a method receives a null argument, it has to decide what to do. Most of the time, the right response is to blow up immediately.

```java
// Java -- BAD: Accepting null and trying to handle it
public double calculateArea(Double width, Double height) {
    if (width == null) {
        width = 0.0;  // Silent corruption of intent
    }
    if (height == null) {
        height = 0.0;
    }
    return width * height;
}

// Java -- GOOD: Reject null immediately
public double calculateArea(double width, double height) {
    if (width <= 0 || height <= 0) {
        throw new IllegalArgumentException(
            String.format(
                "Width and height must be positive. Got width=%f, height=%f",
                width, height
            )
        );
    }
    return width * height;
}
```

```python
# Python -- BAD: Silently accepting None
def calculate_area(width, height):
    width = width or 0
    height = height or 0
    return width * height  # Returns 0 instead of failing -- hides bugs


# Python -- GOOD: Fail fast on invalid input
def calculate_area(width: float, height: float) -> float:
    if width <= 0 or height <= 0:
        raise ValueError(
            f"Width and height must be positive. "
            f"Got width={width}, height={height}"
        )
    return width * height
```

Try-Catch-Finally Structure

The `try-catch-finally` (Java) or `try-except-finally` (Python) structure is your primary tool for error handling. Use it correctly:

- try: The happy path code that might throw an exception
- catch/except: What to do when a specific error occurs
- finally: Cleanup that must happen regardless of success or failure

```java
// Java -- Clean try-catch-finally
public String readFile(String path) {
    BufferedReader reader = null;
    try {
        reader = new BufferedReader(new FileReader(path));
        StringBuilder content = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            content.append(line).append("\n");
        }
        return content.toString();
    } catch (FileNotFoundException e) {
        throw new ConfigFileException(
            "Configuration file not found: " + path, e
        );
    } catch (IOException e) {
        throw new ConfigFileException(
            "Error reading configuration file: " + path, e
        );
    } finally {
```

```
        if (reader != null) {
            try {
                reader.close();
            } catch (IOException e) {
                logger.warn("Failed to close reader for: {}", path, e);
            }
        }
    }
}

// Java -- Even cleaner with try-with-resources
public String readFile(String path) {
    try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        StringBuilder content = new StringBuilder();
        String line;
        while ((line = reader.readLine()) != null) {
            content.append(line).append("\n");
        }
        return content.toString();
    } catch (FileNotFoundException e) {
        throw new ConfigFileException(
            "Configuration file not found: " + path, e
        );
    } catch (IOException e) {
        throw new ConfigFileException(
            "Error reading configuration file: " + path, e
        );
    }
}
```

```python
# Python -- Clean try-except-finally
def read_file(path: str) -> str:
    try:
        with open(path) as f:
            return f.read()
    except FileNotFoundError:
        raise ConfigFileError(
            f"Configuration file not found: {path}"
        )
    except PermissionError:
        raise ConfigFileError(
            f"Permission denied reading: {path}"
        )
    except OSError as e:
        raise ConfigFileError(
            f"Error reading configuration file: {path}"
        ) from e
```

Custom Exception Hierarchy

For any non-trivial application, you should build a hierarchy of custom exceptions. This lets callers decide how specifically they want to handle errors.

```
+-------------------------------------------------------------+
|           EXCEPTION HIERARCHY EXAMPLE                       |
+-------------------------------------------------------------+
|                                                             |
|  AppException (base for all app exceptions)                 |
|    |                                                        |
|    +-- OrderException                                       |
|    |     +-- OrderNotFoundException                         |
|    |     +-- OrderProcessingException                       |
|    |     +-- PaymentDeclinedException                       |
|    |                                                        |
|    +-- UserException                                        |
|    |     +-- UserNotFoundException                          |
|    |     +-- DuplicateEmailException                        |
|    |     +-- InvalidCredentialsException                    |
|    |                                                        |
|    +-- InfrastructureException                              |
|          +-- DatabaseConnectionException                    |
|          +-- ExternalServiceException                       |
|          +-- ConfigurationException                         |
|                                                             |
+-------------------------------------------------------------+
```

Java Implementation

```java
// Java -- Base exception for the application
public class AppException extends RuntimeException {

    private final String errorCode;

    public AppException(String message, String errorCode) {
        super(message);
        this.errorCode = errorCode;
    }

    public AppException(String message, String errorCode, Throwable cause) {
        super(message, cause);
        this.errorCode = errorCode;
    }

    public String getErrorCode() {
        return errorCode;
    }
```

```
}

// Domain-specific exceptions
public class OrderNotFoundException extends AppException {

    public OrderNotFoundException(long orderId) {
        super(
            "Order not found: " + orderId,
            "ORDER_NOT_FOUND"
        );
    }
}

public class PaymentDeclinedException extends AppException {

    public PaymentDeclinedException(String reason, String cardLastFour) {
        super(
            String.format(
                "Payment declined. Reason: %s. Card ending in %s.",
                reason, cardLastFour
            ),
            "PAYMENT_DECLINED"
        );
    }
}
```

Python Implementation

```python
# Python -- Custom exception hierarchy

class AppError(Exception):
    """Base exception for the application."""

    def __init__(self, message: str, error_code: str):
        super().__init__(message)
        self.error_code = error_code


class OrderError(AppError):
    """Base for order-related errors."""
    pass


class OrderNotFoundError(OrderError):

    def __init__(self, order_id: int):
        super().__init__(
            f"Order not found: {order_id}",
            "ORDER_NOT_FOUND",
        )
```

```
        self.order_id = order_id


class PaymentDeclinedError(OrderError):

    def __init__(self, reason: str, card_last_four: str):
        super().__init__(
            f"Payment declined. Reason: {reason}. "
            f"Card ending in {card_last_four}.",
            "PAYMENT_DECLINED",
        )
        self.reason = reason
        self.card_last_four = card_last_four
```

The Fail Fast Principle

Fail fast means: if something is wrong, stop immediately with a clear error rather than continuing with corrupted state. The earlier you detect and report a problem, the easier it is to fix.

```
+-------------------------------------------------------------+
|              FAIL FAST vs FAIL SLOW                          |
+-------------------------------------------------------------+
|                                                              |
|  Fail Slow (BAD):                                           |
|  Input --> Process --> Process --> Process --> ERROR!        |
|  "Something went wrong somewhere... but where?"              |
|                                                              |
|  Fail Fast (GOOD):                                          |
|  Input --> VALIDATE --> ERROR!                               |
|  "Invalid email format: 'not-an-email'"                     |
|                                                              |
|  Fail fast = less debugging, clearer errors, safer code     |
|                                                              |
+-------------------------------------------------------------+
```

```java
// Java -- GOOD: Fail fast with guard clauses
public class TransferService {

    public void transfer(Account from, Account to, BigDecimal amount) {
        // Validate everything before doing anything
        Objects.requireNonNull(from, "Source account must not be null");
        Objects.requireNonNull(to, "Target account must not be null");
        Objects.requireNonNull(amount, "Amount must not be null");
```

```
        if (amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException(
                "Transfer amount must be positive: " + amount
            );
        }

        if (from.equals(to)) {
            throw new IllegalArgumentException(
                "Cannot transfer to the same account: " + from.getId()
            );
        }

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException(from.getId(), amount);
        }

        // Happy path -- all validation passed
        from.debit(amount);
        to.credit(amount);
    }
}
```

```python
# Python -- GOOD: Fail fast with guard clauses
class TransferService:

    def transfer(
        self,
        from_account: Account,
        to_account: Account,
        amount: Decimal,
    ) -> None:
        if amount <= 0:
            raise ValueError(
                f"Transfer amount must be positive: {amount}"
            )

        if from_account == to_account:
            raise ValueError(
                f"Cannot transfer to the same account: "
                f"{from_account.id}"
            )

        if from_account.balance < amount:
            raise InsufficientFundsError(from_account.id, amount)

        # Happy path -- all validation passed
        from_account.debit(amount)
        to_account.credit(amount)
```

Error Handling vs. Business Logic Separation

The most important principle in this chapter: error handling and business logic should live in separate places. When they are interleaved, both become hard to understand.

BEFORE: Error Handling Mixed with Business Logic

```java
// Java -- BAD: Error handling and business logic tangled
public class ReportService {

    public Report generateMonthlyReport(int year, int month) {
        List<Transaction> transactions = null;
        try {
            transactions = transactionRepo.findByMonth(year, month);
        } catch (DatabaseException e) {
            logger.error("DB error", e);
            return null;
        }

        if (transactions == null || transactions.isEmpty()) {
            return new Report(year, month, 0, 0, Collections.emptyList());
        }

        double totalRevenue = 0;
        double totalExpenses = 0;
        List<ReportLine> lines = new ArrayList<>();

        for (Transaction t : transactions) {
            try {
                if (t.getType() == TransactionType.REVENUE) {
                    double converted = 0;
                    try {
                        converted = currencyService.convert(
                            t.getAmount(), t.getCurrency(), "USD"
                        );
                    } catch (CurrencyConversionException e) {
                        logger.warn("Currency conversion failed", e);
                        converted = t.getAmount();  // Use unconverted
                    }
                    totalRevenue += converted;
                    lines.add(new ReportLine(t, converted));
                } else {
                    double converted = 0;
                    try {
                        converted = currencyService.convert(
                            t.getAmount(), t.getCurrency(), "USD"
                        );
                    } catch (CurrencyConversionException e) {
                        logger.warn("Currency conversion failed", e);
                        converted = t.getAmount();
```

```
                    }
                    totalExpenses += converted;
                    lines.add(new ReportLine(t, converted));
                }
            } catch (Exception e) {
                logger.error("Error processing transaction: " + t.getId(), e);
            }
        }

        return new Report(year, month, totalRevenue, totalExpenses, lines);
    }
}
```

AFTER: Separated Concerns

```java
// Java -- GOOD: Business logic is clean, error handling is separate
public class ReportService {

    public Report generateMonthlyReport(int year, int month) {
        List<Transaction> transactions = fetchTransactions(year, month);

        if (transactions.isEmpty()) {
            return Report.empty(year, month);
        }

        return buildReport(year, month, transactions);
    }

    private Report buildReport(
            int year,
            int month,
            List<Transaction> transactions) {

        double totalRevenue = 0;
        double totalExpenses = 0;
        List<ReportLine> lines = new ArrayList<>();

        for (Transaction transaction : transactions) {
            double amount = convertToUsd(transaction);
            lines.add(new ReportLine(transaction, amount));

            if (transaction.getType() == TransactionType.REVENUE) {
                totalRevenue += amount;
            } else {
                totalExpenses += amount;
            }
        }

        return new Report(year, month, totalRevenue, totalExpenses, lines);
    }
```

```
    private List<Transaction> fetchTransactions(int year, int month) {
        try {
            return transactionRepo.findByMonth(year, month);
        } catch (DatabaseException e) {
            throw new ReportGenerationException(
                "Failed to fetch transactions for " + year + "-" + month, e
            );
        }
    }

    private double convertToUsd(Transaction transaction) {
        try {
            return currencyService.convert(
                transaction.getAmount(),
                transaction.getCurrency(),
                "USD"
            );
        } catch (CurrencyConversionException e) {
            logger.warn(
                "Currency conversion failed for transaction {}. "
                + "Using original amount.",
                transaction.getId(), e
            );
            return transaction.getAmount();
        }
    }
}
```

The Python version follows the same separation:

```python
# Python -- GOOD: Business logic clean, error handling in helper methods
class ReportService:

    def generate_monthly_report(self, year: int, month: int) -> Report:
        transactions = self._fetch_transactions(year, month)

        if not transactions:
            return Report.empty(year, month)

        return self._build_report(year, month, transactions)

    def _build_report(
        self,
        year: int,
        month: int,
        transactions: list[Transaction],
    ) -> Report:
        total_revenue = 0.0
        total_expenses = 0.0
        lines = []
```

```
        for transaction in transactions:
            amount = self._convert_to_usd(transaction)
            lines.append(ReportLine(transaction, amount))

            if transaction.type == TransactionType.REVENUE:
                total_revenue += amount
            else:
                total_expenses += amount

        return Report(year, month, total_revenue, total_expenses, lines)

    def _fetch_transactions(
        self, year: int, month: int
    ) -> list[Transaction]:
        try:
            return self.transaction_repo.find_by_month(year, month)
        except DatabaseError as e:
            raise ReportGenerationError(
                f"Failed to fetch transactions for {year}-{month}"
            ) from e

    def _convert_to_usd(self, transaction: Transaction) -> float:
        try:
            return self.currency_service.convert(
                transaction.amount, transaction.currency, "USD"
            )
        except CurrencyConversionError:
            logger.warning(
                "Currency conversion failed for transaction %s. "
                "Using original amount.",
                transaction.id,
            )
            return transaction.amount
```

Common Mistakes

1. Empty catch blocks. Catching an exception and doing nothing with it is almost always a bug. If you truly want to ignore an exception, add a comment explaining why.
2. Catching Exception (the base class). Catching every possible exception hides bugs. Catch specific exceptions that you know how to handle.
3. Using exceptions for control flow. Exceptions should represent exceptional conditions, not normal branching. Do not use try-catch as a replacement for if-else.
4. Returning null from methods. Return `Optional`, throw an exception, or return an empty collection instead.

5. Logging and re-throwing the same exception. This produces duplicate log entries. Either log it and handle it, or re-throw it for a higher-level handler to log.
6. Exception messages without context. "Error occurred" tells you nothing. Include the what, where, and why.
7. Swallowing the cause chain. When wrapping exceptions, always pass the original as the cause (`throw new X(msg, e)` in Java, `raise X() from e` in Python).

Best Practices

1. Use exceptions instead of error codes. They cannot be silently ignored, carry stack traces, and keep the happy path clean.
2. Prefer unchecked exceptions. Checked exceptions cause cascading changes through the call chain.
3. Provide rich context in exception messages. Include IDs, values, and expected vs. actual states.
4. Define exceptions by the caller's needs. One `PortDeviceFailure` is better than five low-level device exceptions if the caller handles them all the same way.
5. Never return null. Use `Optional`, empty collections, or throw exceptions for truly exceptional cases.
6. Fail fast. Validate inputs at the boundary and throw immediately if something is wrong.
7. Separate error handling from business logic. Extract error-prone operations into helper methods that handle their own errors.
8. Build a custom exception hierarchy. Start with a base `AppException` and create domain-specific subclasses.

Quick Summary

```
+-------------------------------------------------------------+
|              ERROR HANDLING CHEAT SHEET                      |
+-------------------------------------------------------------+
|                                                              |
|  DO:                                                         |
|    - Use exceptions, not error codes                         |
|    - Use unchecked exceptions (RuntimeException)             |
|    - Provide context: what, where, why                       |
```

```
|    - Return Optional or empty collections, not null        |
|    - Fail fast with guard clauses                          |
|    - Separate error handling from business logic           |
|    - Preserve the cause chain (throw ... from e)           |
|    - Build a custom exception hierarchy                    |
|                                                            |
|  DON'T:                                                    |
|    - Swallow exceptions with empty catch blocks            |
|    - Catch the base Exception class                        |
|    - Use exceptions for normal control flow                |
|    - Return null from methods                              |
|    - Pass null as arguments                                |
|    - Log AND re-throw the same exception                   |
|    - Write exception messages without context              |
|                                                            |
+------------------------------------------------------------+
```

Key Points

- Exceptions are superior to error codes because they cannot be silently ignored and they separate the happy path from error handling
- Unchecked exceptions (RuntimeException in Java, all Python exceptions) avoid cascading changes through the call chain
- Exception messages must include context: what failed, what values were involved, and what was expected
- Design exception classes around what the caller needs to do, not what went wrong internally
- Returning null forces every caller to add null checks and guarantees that someone will forget -- use Optional or empty collections instead
- The fail fast principle means validating early and failing with clear messages before corrupted state can spread
- Separating error handling from business logic keeps both readable -- extract error-prone calls into helper methods

---

Practice Questions

1. Explain why error codes are problematic compared to exceptions. Give a specific example of how a caller might silently ignore an error code.

2. What is the difference between checked and unchecked exceptions in Java? Why do most modern Java developers prefer unchecked exceptions?
3. What three pieces of information should every exception message contain? Write an example exception message for a failed database connection.
4. Why is returning null from a method considered bad practice? What are three alternatives?
5. Explain the fail fast principle. Why is it better to fail immediately at the boundary than to continue processing with invalid data?

Exercises

Exercise 1: Refactor Error Codes to Exceptions

Take the following error-code-based code and refactor it to use exceptions. Create appropriate custom exception classes and ensure the happy path reads cleanly.

```python
def register_user(username, email, password):
    if not username:
        return {"error": "USERNAME_EMPTY", "success": False}
    if len(password) < 8:
        return {"error": "PASSWORD_TOO_SHORT", "success": False}
    existing = db.find_user_by_email(email)
    if existing is not None:
        return {"error": "EMAIL_TAKEN", "success": False}
    result = db.create_user(username, email, password)
    if result == -1:
        return {"error": "DB_ERROR", "success": False}
    email_result = send_welcome_email(email)
    if email_result == -1:
        return {"error": "EMAIL_SEND_FAILED", "success": False}
    return {"success": True, "user_id": result}
```

Exercise 2: Build an Exception Hierarchy

Design and implement a custom exception hierarchy for an e-commerce application. Include at least three levels of specificity (base, domain, specific) and cover at least two domains (orders and users). Each exception should accept relevant context in its constructor and produce a clear, debuggable message.

Exercise 3: Eliminate Null Returns

Find three methods in one of your own projects (or an open-source project) that return null. For each one, determine the appropriate replacement: Optional, empty collection, or exception. Refactor the method and update one of its callers.

What Is Next?

With clean error handling in place, your code can fail gracefully and communicate problems clearly. The next few chapters dive into the SOLID principles -- five foundational rules that guide how you structure classes and their relationships. You will start with the Single Responsibility Principle and build up to the full set, learning how each principle prevents a specific category of design problems.

Chapter 7: Classes -- Building Blocks of Clean Software

What You Will Learn

By the end of this chapter, you will be able to:

- Design small, focused classes with a single responsibility
- Understand and measure class cohesion
- Recognize the God Class anti-pattern and refactor it
- Apply the "one reason to change" principle to class design
- Use encapsulation to protect class internals
- Apply the Tell, Don't Ask principle
- Organize classes for change and extensibility
- Split large, unfocused classes into clean, maintainable components

Why This Chapter Matters

Classes are the primary organizational unit in object-oriented software. A well-designed class is easy to understand, easy to test, and easy to change. A poorly designed class becomes a maintenance nightmare that slows down entire teams.

Consider this: you join a new team and open a file called `ApplicationManager.java`. It is 2,400 lines long. It handles user authentication, database connections, email sending, report generation, and logging. Where do you start? How do you add a feature without breaking something else? How do you write a test for just one behavior?

This chapter teaches you to avoid that situation entirely. You will learn how to design classes that are small, focused, and easy to work with. These principles apply whether you are writing

Java, Python, or any other object-oriented language.

The Problem With Large Classes

Most developers have encountered a class like this at some point in their career. It starts small, with a clear purpose. Over months and years, developers add "just one more method" until the class becomes a tangled mess of unrelated responsibilities.

```
┌──────────────────────────────────────────────┐
│            UserManager (God Class)           │
├──────────────────────────────────────────────┤
│ - dbConnection                               │
│ - emailClient                                │
│ - logger                                     │
│ - cache                                      │
│ - passwordEncoder                            │
│ - templateEngine                             │
│ - reportFormatter                            │
│ - auditTrail                                 │
├──────────────────────────────────────────────┤
│ + createUser()                               │
│ + deleteUser()                               │
│ + findUserById()                             │
│ + findUserByEmail()                          │
│ + authenticateUser()                         │
│ + resetPassword()                            │
│ + sendWelcomeEmail()                         │
│ + sendPasswordResetEmail()                   │
│ + generateMonthlyReport()                    │
│ + exportUsersToCsv()                         │
│ + logUserActivity()                          │
│ + cacheUserSession()                         │
│ + invalidateCache()                          │
│ + validateUserInput()                        │
│ + formatUserForDisplay()                     │
│ + calculateUserStatistics()                  │
│ + ... (30 more methods)                      │
└──────────────────────────────────────────────┘
```

This is the God Class anti-pattern. One class that tries to do everything. It knows too much, does too much, and changes for too many reasons.

Why God Classes Are Harmful

1. Hard to understand. A developer must read hundreds or thousands of lines to grasp what the class does.
2. Hard to test. Testing one behavior requires setting up dependencies for all behaviors. Want to test password reset? You still need a database connection, email client, cache, and logger.
3. High risk of bugs. Changing the email logic might accidentally break the reporting logic because they share instance variables.
4. Merge conflicts. Multiple developers working on different features all edit the same file, creating constant merge conflicts.
5. Hard to reuse. You cannot use just the email functionality without dragging along the database, caching, and reporting code.

Small Classes: The First Rule

The first rule of classes is that they should be small. The second rule is that they should be smaller than that.

But "small" does not mean counting lines of code. A 200-line class with a clear, single purpose is better than a 50-line class that mixes two unrelated responsibilities. Size is measured in responsibilities, not lines.

A Class Should Have One Reason to Change

This is the most important principle in class design. Every class should have exactly one reason to change. If you can think of more than one reason why a class might need to be modified, it has more than one responsibility.

Ask yourself: "Who would request changes to this class?" If the answer includes multiple people or teams (the security team, the marketing team, the database administrators), the class has too many responsibilities.

```
┌──────────────────────────────────────────────┐
│              One Reason to Change            │
├──────────────────────────────────────────────┤
│                                              │
│  UserRepository  -- changes when the         │
│                     database schema changes  │
│                                              │
│  EmailService    -- changes when email       │
│                     requirements change      │
```

```
│                                                │
│  PasswordHasher  -- changes when security      │
│                     policy changes             │
│                                                │
│  UserFormatter   -- changes when display       │
│                     requirements change        │
│                                                │
└────────────────────────────────────────────────┘
```

Each class has one actor (one stakeholder) who would request changes. The database team changes `UserRepository`. The security team changes `PasswordHasher`. The UI team changes `UserFormatter`. No class changes for two different reasons.

Cohesion: The Measure of a Good Class

Cohesion measures how closely the methods and instance variables of a class are related. In a highly cohesive class, every method uses most of the instance variables. In a low-cohesion class, methods are grouped together but operate on different subsets of data.

High Cohesion Example

Java:

```java
// HIGH COHESION: Every method uses the core instance variables
public class Rectangle {
    private double width;
    private double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double area() {
        return width * height;   // uses both variables
    }

    public double perimeter() {
        return 2 * (width + height);  // uses both variables
    }

    public double diagonal() {
        return Math.sqrt(width * width + height * height);  // uses both
```

```
    }

    public boolean isSquare() {
        return Double.compare(width, height) == 0;  // uses both
    }
}
```

Python:

```python
# HIGH COHESION: Every method uses the core instance variables
class Rectangle:
    def __init__(self, width: float, height: float):
        self._width = width
        self._height = height

    def area(self) -> float:
        return self._width * self._height  # uses both

    def perimeter(self) -> float:
        return 2 * (self._width + self._height)  # uses both

    def diagonal(self) -> float:
        return (self._width ** 2 + self._height ** 2) ** 0.5  # uses both

    def is_square(self) -> bool:
        return self._width == self._height  # uses both
```

Every method in this class operates on `width` and `height`. The class is highly cohesive. All its parts work together toward a single purpose: representing a rectangle.

Low Cohesion Example

Java:

```java
// LOW COHESION: Methods use different subsets of variables
public class UserManager {
    private Database db;
    private EmailClient emailClient;
    private PasswordEncoder encoder;
    private Logger logger;

    public User findUser(int id) {
        return db.query("SELECT * FROM users WHERE id = ?", id);
        // uses: db
    }

    public void sendWelcomeEmail(User user) {
        emailClient.send(user.getEmail(), "Welcome!", buildTemplate(user));
```

```
        // uses: emailClient
    }

    public String hashPassword(String password) {
        return encoder.encode(password);
        // uses: encoder
    }

    public void logActivity(String message) {
        logger.info(message);
        // uses: logger
    }
}
```

Each method uses only one of the four instance variables. The methods are not related to each other. They are stuffed into the same class out of convenience, not because they belong together. This is a sign that the class should be split.

Measuring Cohesion

A simple way to measure cohesion: for each method, count how many instance variables it uses. Then calculate the average.

```
Cohesion Score = (Total variable uses across all methods)
                 / (Number of methods * Number of instance variables)

Rectangle class:
  - 4 methods, 2 variables
  - Each method uses 2 variables
  - Score = (4 * 2) / (4 * 2) = 1.0  (perfect cohesion)

UserManager class:
  - 4 methods, 4 variables
  - Each method uses 1 variable
  - Score = (4 * 1) / (4 * 4) = 0.25  (low cohesion)
```

A score close to 1.0 means high cohesion. A score below 0.5 suggests the class should be split.

Before and After: Refactoring a God Class

Let us take a realistic God Class and refactor it step by step.

BEFORE: The God Class (500+ lines condensed)

Java:

```java
public class OrderService {
    private Connection dbConnection;
    private JavaMailSender mailSender;
    private Logger logger;
    private TaxCalculator taxCalc;
    private PdfGenerator pdfGen;
    private InventorySystem inventory;

    // ---- Order CRUD ----
    public Order createOrder(Customer customer, List<Item> items) {
        logger.info("Creating order for " + customer.getName());
        Order order = new Order();
        order.setCustomer(customer);
        order.setItems(items);
        order.setStatus("PENDING");

        double subtotal = 0;
        for (Item item : items) {
            subtotal += item.getPrice() * item.getQuantity();
        }
        double tax = taxCalc.calculate(subtotal, customer.getState());
        order.setSubtotal(subtotal);
        order.setTax(tax);
        order.setTotal(subtotal + tax);

        // Save to database
        PreparedStatement stmt = dbConnection.prepareStatement(
            "INSERT INTO orders (customer_id, subtotal, tax, total, status) " +
            "VALUES (?, ?, ?, ?, ?)"
        );
        stmt.setInt(1, customer.getId());
        stmt.setDouble(2, subtotal);
        stmt.setDouble(3, tax);
        stmt.setDouble(4, subtotal + tax);
        stmt.setString(5, "PENDING");
        stmt.executeUpdate();

        // Update inventory
        for (Item item : items) {
            inventory.decreaseStock(item.getId(), item.getQuantity());
        }

        // Send confirmation email
        String emailBody = "<html><body>"
            + "<h1>Order Confirmation</h1>"
            + "<p>Thank you, " + customer.getName() + "!</p>"
            + "<p>Total: $" + order.getTotal() + "</p>"
            + "</body></html>";
        mailSender.send(customer.getEmail(), "Order Confirmation", emailBody);
```

```
        logger.info("Order created successfully: " + order.getId());
        return order;
    }

    public Order findOrderById(int id) {
        PreparedStatement stmt = dbConnection.prepareStatement(
            "SELECT * FROM orders WHERE id = ?"
        );
        stmt.setInt(1, id);
        ResultSet rs = stmt.executeQuery();
        // ... map result set to Order object (20+ lines)
        return order;
    }

    public void cancelOrder(int orderId) {
        Order order = findOrderById(orderId);
        order.setStatus("CANCELLED");
        // Update database
        PreparedStatement stmt = dbConnection.prepareStatement(
            "UPDATE orders SET status = ? WHERE id = ?"
        );
        stmt.setString(1, "CANCELLED");
        stmt.setInt(2, orderId);
        stmt.executeUpdate();

        // Restore inventory
        for (Item item : order.getItems()) {
            inventory.increaseStock(item.getId(), item.getQuantity());
        }

        // Send cancellation email
        mailSender.send(order.getCustomer().getEmail(),
            "Order Cancelled",
            "Your order #" + orderId + " has been cancelled.");

        logger.info("Order cancelled: " + orderId);
    }

    // ---- Reporting ----
    public byte[] generateInvoicePdf(int orderId) {
        Order order = findOrderById(orderId);
        return pdfGen.createInvoice(order);
    }

    public List<Order> getMonthlyReport(int month, int year) {
        PreparedStatement stmt = dbConnection.prepareStatement(
            "SELECT * FROM orders WHERE MONTH(created_at) = ? AND YEAR(created_at) = ?"
        );
        // ... query and build report (30+ lines)
        return orders;
    }

    public String exportOrdersToCsv(List<Order> orders) {
        StringBuilder csv = new StringBuilder("id,customer,total,status\n");
```

```
        for (Order order : orders) {
            csv.append(order.getId()).append(",")
               .append(order.getCustomer().getName()).append(",")
               .append(order.getTotal()).append(",")
               .append(order.getStatus()).append("\n");
        }
        return csv.toString();
    }

    // ---- Tax calculation ----
    public double calculateTax(double amount, String state) {
        return taxCalc.calculate(amount, state);
    }

    // ---- Email templates ----
    private String buildConfirmationEmail(Order order) { /* 20 lines */ }
    private String buildCancellationEmail(Order order) { /* 15 lines */ }
    private String buildShippingEmail(Order order) { /* 20 lines */ }

    // ... 200 more lines of mixed concerns
}
```

This class changes when:

- The database schema changes (CRUD operations)
- Email templates change (email formatting)
- Tax rules change (tax calculation)
- Report formats change (reporting)
- Inventory rules change (stock management)

Five reasons to change means five responsibilities. That is four too many.

AFTER: Focused Classes

Java:

```java
// Responsibility: Persisting and retrieving orders
public class OrderRepository {
    private final Connection dbConnection;

    public OrderRepository(Connection dbConnection) {
        this.dbConnection = dbConnection;
    }

    public Order save(Order order) {
        PreparedStatement stmt = dbConnection.prepareStatement(
            "INSERT INTO orders (customer_id, subtotal, tax, total, status) " +
            "VALUES (?, ?, ?, ?, ?)"
```

```
        );
        stmt.setInt(1, order.getCustomer().getId());
        stmt.setDouble(2, order.getSubtotal());
        stmt.setDouble(3, order.getTax());
        stmt.setDouble(4, order.getTotal());
        stmt.setString(5, order.getStatus());
        stmt.executeUpdate();
        return order;
    }

    public Order findById(int id) {
        PreparedStatement stmt = dbConnection.prepareStatement(
            "SELECT * FROM orders WHERE id = ?"
        );
        stmt.setInt(1, id);
        ResultSet rs = stmt.executeQuery();
        return mapToOrder(rs);
    }

    public void updateStatus(int orderId, String status) {
        PreparedStatement stmt = dbConnection.prepareStatement(
            "UPDATE orders SET status = ? WHERE id = ?"
        );
        stmt.setString(1, status);
        stmt.setInt(2, orderId);
        stmt.executeUpdate();
    }

    private Order mapToOrder(ResultSet rs) {
        // mapping logic
    }
}
```

```java
// Responsibility: Calculating order totals and tax
public class OrderPriceCalculator {
    private final TaxCalculator taxCalculator;

    public OrderPriceCalculator(TaxCalculator taxCalculator) {
        this.taxCalculator = taxCalculator;
    }

    public OrderTotal calculate(List<Item> items, String state) {
        double subtotal = items.stream()
            .mapToDouble(item -> item.getPrice() * item.getQuantity())
            .sum();
        double tax = taxCalculator.calculate(subtotal, state);
        return new OrderTotal(subtotal, tax, subtotal + tax);
    }
}
```

```java
// Responsibility: Sending order-related emails
public class OrderNotificationService {
    private final JavaMailSender mailSender;
    private final OrderEmailTemplates templates;

    public OrderNotificationService(JavaMailSender mailSender,
                                    OrderEmailTemplates templates) {
        this.mailSender = mailSender;
        this.templates = templates;
    }

    public void sendConfirmation(Order order) {
        String body = templates.confirmation(order);
        mailSender.send(order.getCustomer().getEmail(),
            "Order Confirmation", body);
    }

    public void sendCancellation(Order order) {
        String body = templates.cancellation(order);
        mailSender.send(order.getCustomer().getEmail(),
            "Order Cancelled", body);
    }
}
```

```java
// Responsibility: Managing inventory levels
public class InventoryManager {
    private final InventorySystem inventory;

    public InventoryManager(InventorySystem inventory) {
        this.inventory = inventory;
    }

    public void reserveItems(List<Item> items) {
        for (Item item : items) {
            inventory.decreaseStock(item.getId(), item.getQuantity());
        }
    }

    public void releaseItems(List<Item> items) {
        for (Item item : items) {
            inventory.increaseStock(item.getId(), item.getQuantity());
        }
    }
}
```

```java
// Responsibility: Generating order reports
public class OrderReportService {
    private final OrderRepository orderRepository;
    private final PdfGenerator pdfGenerator;
```

```
    public OrderReportService(OrderRepository orderRepository,
                              PdfGenerator pdfGenerator) {
        this.orderRepository = orderRepository;
        this.pdfGenerator = pdfGenerator;
    }

    public byte[] generateInvoice(int orderId) {
        Order order = orderRepository.findById(orderId);
        return pdfGenerator.createInvoice(order);
    }

    public String exportToCsv(List<Order> orders) {
        StringBuilder csv = new StringBuilder("id,customer,total,status\n");
        for (Order order : orders) {
            csv.append(order.getId()).append(",")
               .append(order.getCustomer().getName()).append(",")
               .append(order.getTotal()).append(",")
               .append(order.getStatus()).append("\n");
        }
        return csv.toString();
    }
}
```

```java
// Responsibility: Coordinating the order creation workflow
public class OrderService {
    private final OrderRepository repository;
    private final OrderPriceCalculator calculator;
    private final OrderNotificationService notifications;
    private final InventoryManager inventoryManager;
    private final Logger logger;

    public OrderService(OrderRepository repository,
                        OrderPriceCalculator calculator,
                        OrderNotificationService notifications,
                        InventoryManager inventoryManager,
                        Logger logger) {
        this.repository = repository;
        this.calculator = calculator;
        this.notifications = notifications;
        this.inventoryManager = inventoryManager;
        this.logger = logger;
    }

    public Order createOrder(Customer customer, List<Item> items) {
        logger.info("Creating order for " + customer.getName());

        OrderTotal total = calculator.calculate(items, customer.getState());

        Order order = new Order(customer, items, total);
        repository.save(order);

        inventoryManager.reserveItems(items);
```

```
        notifications.sendConfirmation(order);

        logger.info("Order created: " + order.getId());
        return order;
    }

    public void cancelOrder(int orderId) {
        Order order = repository.findById(orderId);
        order.setStatus("CANCELLED");
        repository.updateStatus(orderId, "CANCELLED");

        inventoryManager.releaseItems(order.getItems());
        notifications.sendCancellation(order);

        logger.info("Order cancelled: " + orderId);
    }
}
```

The Refactored Structure

```
BEFORE:                                AFTER:
┌──────────────────────┐      ┌──────────────────────┐
│                      │      │    OrderService      │
│   OrderService       │      │  (coordination only) │
│                      │      └──────────────────────┘
│  - DB queries        │                 │
│  - Tax calculation   │      ┌──────────┼──────────────┐
│  - Email sending     │      │          │              │
│  - Inventory mgmt    │      ▼          ▼              ▼
│  - PDF generation    │
│  - CSV export        │   ┌──────┐ ┌──────────┐ ┌──────┐
│  - Email templates   │   │ Order │ │ Order    │ │ Inventory│
│  - Reporting         │   │ Repo  │ │ Price    │ │ Manager  │
│  - Logging           │   │       │ │ Calculator│ │      │
│  - ... 40+ methods   │   └──────┘ └──────────┘ └──────┘
│                      │
│  500+ lines          │      ┌──────────────┐ ┌──────────┐
│                      │      │ Order        │ │ Order Report │
└──────────────────────┘      │ Notification │ │ Service  │
                              │ Service      │ │          │
                              └──────────────┘ └──────────┘
```

Python version of the refactored code:

```python
class OrderRepository:
    """Persists and retrieves orders from the database."""

    def __init__(self, db_connection):
        self._db = db_connection
```

```
    def save(self, order: Order) -> Order:
        self._db.execute(
            "INSERT INTO orders (customer_id, subtotal, tax, total, status) "
            "VALUES (?, ?, ?, ?, ?)",
            (order.customer.id, order.subtotal, order.tax,
             order.total, order.status)
        )
        return order

    def find_by_id(self, order_id: int) -> Order:
        row = self._db.execute(
            "SELECT * FROM orders WHERE id = ?", (order_id,)
        ).fetchone()
        return self._map_to_order(row)

    def update_status(self, order_id: int, status: str) -> None:
        self._db.execute(
            "UPDATE orders SET status = ? WHERE id = ?",
            (status, order_id)
        )


class OrderPriceCalculator:
    """Calculates order totals including tax."""

    def __init__(self, tax_calculator):
        self._tax_calculator = tax_calculator

    def calculate(self, items: list, state: str) -> OrderTotal:
        subtotal = sum(item.price * item.quantity for item in items)
        tax = self._tax_calculator.calculate(subtotal, state)
        return OrderTotal(subtotal, tax, subtotal + tax)


class OrderNotificationService:
    """Sends order-related email notifications."""

    def __init__(self, mail_sender, templates):
        self._mail_sender = mail_sender
        self._templates = templates

    def send_confirmation(self, order: Order) -> None:
        body = self._templates.confirmation(order)
        self._mail_sender.send(order.customer.email,
                               "Order Confirmation", body)

    def send_cancellation(self, order: Order) -> None:
        body = self._templates.cancellation(order)
        self._mail_sender.send(order.customer.email,
                               "Order Cancelled", body)


class OrderService:
    """Coordinates the order creation workflow."""
```

```
    def __init__(self, repository, calculator, notifications,
                 inventory_manager, logger):
        self._repository = repository
        self._calculator = calculator
        self._notifications = notifications
        self._inventory = inventory_manager
        self._logger = logger

    def create_order(self, customer, items) -> Order:
        self._logger.info(f"Creating order for {customer.name}")

        total = self._calculator.calculate(items, customer.state)
        order = Order(customer, items, total)
        self._repository.save(order)

        self._inventory.reserve_items(items)
        self._notifications.send_confirmation(order)

        self._logger.info(f"Order created: {order.id}")
        return order

    def cancel_order(self, order_id: int) -> None:
        order = self._repository.find_by_id(order_id)
        order.status = "CANCELLED"
        self._repository.update_status(order_id, "CANCELLED")

        self._inventory.release_items(order.items)
        self._notifications.send_cancellation(order)

        self._logger.info(f"Order cancelled: {order_id}")
```

Encapsulation

Encapsulation means hiding a class's internal details and exposing only what other classes need to know. It is not just about making fields `private`. It is about protecting the integrity of your objects.

Why Encapsulation Matters

Without encapsulation, any code anywhere can reach into your object and change its state in ways you did not anticipate:

Java -- BEFORE (no encapsulation):

```java
public class BankAccount {
    public double balance;    // anyone can modify this directly
    public String status;     // anyone can set this to anything
}

// Somewhere else in the codebase...
account.balance = -1000;     // negative balance? No validation!
account.status = "banana";   // nonsensical status? No protection!
```

Java -- AFTER (with encapsulation):

```java
public class BankAccount {
    private double balance;
    private AccountStatus status;

    public BankAccount(double initialBalance) {
        if (initialBalance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative");
        }
        this.balance = initialBalance;
        this.status = AccountStatus.ACTIVE;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        this.balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }
        if (amount > balance) {
            throw new InsufficientFundsException("Not enough funds");
        }
        this.balance -= amount;
    }

    public double getBalance() {
        return balance;    // read-only access
    }
}
```

Python:

```python
class BankAccount:
    def __init__(self, initial_balance: float):
```

```
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative")
        self._balance = initial_balance
        self._status = AccountStatus.ACTIVE

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self._balance += amount

    def withdraw(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive")
        if amount > self._balance:
            raise InsufficientFundsError("Not enough funds")
        self._balance -= amount

    @property
    def balance(self) -> float:
        return self._balance  # read-only access
```

Now the `BankAccount` class controls how its state changes. No outside code can set a negative balance or an invalid status. The class enforces its own rules.

Tell, Don't Ask

The Tell, Don't Ask principle says: instead of asking an object for its data and then making decisions based on that data, tell the object what to do and let it make the decision itself.

BEFORE: Ask, Then Decide (Violating Tell, Don't Ask)

Java:

```java
// Asking the object for data, then deciding externally
public class OrderProcessor {
    public void processDiscount(Order order) {
        // ASK for internal state
        double total = order.getTotal();
        String customerType = order.getCustomer().getType();
        int itemCount = order.getItems().size();

        // DECIDE based on that state
        double discount = 0;
        if (customerType.equals("PREMIUM") && total > 100) {
```

```
            discount = total * 0.15;
        } else if (customerType.equals("REGULAR") && itemCount > 5) {
            discount = total * 0.05;
        }

        // MODIFY the object externally
        order.setDiscount(discount);
        order.setTotal(total - discount);
    }
}
```

The problem: the discount logic lives outside the `Order` class. If another part of the codebase also calculates discounts, you get duplicated logic. The `Order` class has no control over how its own discount is calculated.

AFTER: Tell the Object What to Do

Java:

```java
public class Order {
    private double subtotal;
    private double discount;
    private Customer customer;
    private List<Item> items;

    // TELL the order to apply its discount
    public void applyDiscount() {
        if (customer.isPremium() && subtotal > 100) {
            this.discount = subtotal * 0.15;
        } else if (items.size() > 5) {
            this.discount = subtotal * 0.05;
        } else {
            this.discount = 0;
        }
    }

    public double getTotal() {
        return subtotal - discount;
    }
}

// Usage:
order.applyDiscount();   // Tell, don't ask
double total = order.getTotal();
```

Python:

```python
class Order:
    def __init__(self, customer, items):
        self._customer = customer
        self._items = items
        self._subtotal = sum(item.price * item.quantity for item in items)
        self._discount = 0.0

    def apply_discount(self) -> None:
        """Tell the order to calculate its own discount."""
        if self._customer.is_premium and self._subtotal > 100:
            self._discount = self._subtotal * 0.15
        elif len(self._items) > 5:
            self._discount = self._subtotal * 0.05
        else:
            self._discount = 0.0

    @property
    def total(self) -> float:
        return self._subtotal - self._discount


# Usage:
order.apply_discount()   # Tell, don't ask
print(order.total)
```

The discount logic now lives where it belongs: inside the `Order` class. The class knows its own rules and enforces them.

Spotting Tell, Don't Ask Violations

Look for this pattern in your code:

```
obj.getX()           # ASK
if (someCondition)   # DECIDE externally
obj.setY(newValue)   # MODIFY externally
```

This pattern means the logic belongs inside the object, not outside it. Refactor it to:

```
obj.doSomething()    # TELL
```

Organizing Classes for Change

Well-organized classes make future changes easy. Poorly organized classes make every change risky.

The Newspaper Metaphor

Organize your class like a newspaper article:

1. Public constants at the top (the headline)
2. Private variables next (the subheading)
3. Constructors (the lead paragraph)
4. Public methods (the body of the article)
5. Private methods right after the public method that calls them (supporting details)

Java:

```java
public class UserService {
    // 1. Constants
    private static final int MAX_LOGIN_ATTEMPTS = 5;
    private static final int LOCKOUT_DURATION_MINUTES = 30;

    // 2. Instance variables
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    // 3. Constructor
    public UserService(UserRepository userRepository,
                       PasswordEncoder passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }

    // 4. Public methods
    public User register(String email, String password) {
        validateEmail(email);
        String hashedPassword = passwordEncoder.encode(password);
        User user = new User(email, hashedPassword);
        return userRepository.save(user);
    }

    public boolean authenticate(String email, String password) {
        User user = userRepository.findByEmail(email);
        if (user == null) {
            return false;
        }
        if (isLockedOut(user)) {
            throw new AccountLockedException("Account is locked");
        }
        return passwordEncoder.matches(password, user.getPasswordHash());
    }
```

```
    // 5. Private methods (near the public methods that use them)
    private void validateEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
    }

    private boolean isLockedOut(User user) {
        return user.getFailedLoginAttempts() >= MAX_LOGIN_ATTEMPTS
            && user.getLastFailedLogin()
                   .plusMinutes(LOCKOUT_DURATION_MINUTES)
                   .isAfter(LocalDateTime.now());
    }
}
```

Open for Extension

When you design classes, think about what is likely to change. Isolate those parts so they can be modified or extended without touching existing code.

```
┌──────────────────────────────────────────────────┐
│            Designing for Change                  │
├──────────────────────────────────────────────────┤
│                                                  │
│  Ask: "What is likely to change?"                │
│                                                  │
│  - Data source?    --> Repository interface      │
│  - Business rules? --> Strategy pattern          │
│  - Output format?  --> Formatter interface       │
│  - Notification?   --> Notification interface    │
│                                                  │
│  Isolate the volatile part behind an interface.  │
│  The rest of the code stays stable.              │
│                                                  │
└──────────────────────────────────────────────────┘
```

Real-World Scenario: The Growing User Class

You are building a web application. In the first sprint, you create a `User` class:

```python
class User:
    def __init__(self, name, email, password):
        self.name = name
        self.email = email
        self.password = password
```

Simple and clean. Then the requirements grow:

- Sprint 2: "We need email validation." You add `validate_email()`.
- Sprint 3: "We need password hashing." You add `hash_password()`.
- Sprint 4: "We need to send welcome emails." You add `send_welcome_email()`.
- Sprint 5: "We need to generate profile URLs." You add `generate_profile_url()`.
- Sprint 6: "We need activity logging." You add `log_activity()`.

After six sprints, your `User` class is doing five unrelated things. The fix: split it early.

```python
# Data representation
class User:
    def __init__(self, name: str, email: str, password_hash: str):
        self.name = name
        self.email = email
        self.password_hash = password_hash


# Validation logic
class UserValidator:
    @staticmethod
    def validate_email(email: str) -> bool:
        import re
        pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
        return bool(re.match(pattern, email))

    @staticmethod
    def validate_password(password: str) -> bool:
        return len(password) >= 8


# Password handling
class PasswordService:
    def __init__(self, hasher):
        self._hasher = hasher

    def hash_password(self, password: str) -> str:
        return self._hasher.hash(password)

    def verify_password(self, password: str, hashed: str) -> bool:
        return self._hasher.verify(password, hashed)
```

```python
# Email notifications
class UserNotificationService:
    def __init__(self, email_sender):
        self._sender = email_sender

    def send_welcome_email(self, user: User) -> None:
        self._sender.send(
            to=user.email,
            subject="Welcome!",
            body=f"Hello {user.name}, welcome aboard!"
        )


# Profile URLs
class ProfileUrlGenerator:
    def __init__(self, base_url: str):
        self._base_url = base_url

    def generate(self, user: User) -> str:
        slug = user.name.lower().replace(" ", "-")
        return f"{self._base_url}/users/{slug}"
```

Each class has one job. Each can be tested independently. Each changes for exactly one reason.

Common Mistakes

Mistake 1: Classes Named "Manager," "Handler," "Processor," or "Utility"

These vague names are a red flag. They give the class permission to do anything. A `DataManager` can manage data in any way, so developers keep adding unrelated methods.

Fix: Name classes after what they specifically do: `OrderRepository`, `PasswordHasher`, `InvoiceGenerator`, `EmailSender`.

Mistake 2: Creating Getters and Setters for Everything

Automatically generating getters and setters for every field defeats the purpose of encapsulation. You are just adding extra steps to access public fields.

Fix: Only expose what is necessary. Prefer methods that perform actions (`deposit`, `withdraw`) over methods that expose raw state (`getBalance`, `setBalance`).

Mistake 3: Making Everything Static

Static methods and static state create hidden dependencies and make testing difficult. A class full of static methods is really just a disguised set of global functions.

Fix: Use instance methods. If a class has no state, consider whether it should be a class at all, or whether its methods belong on another class.

Mistake 4: Feature Envy

A method that uses more data from another class than from its own class is exhibiting Feature Envy. The method probably belongs in the other class.

```java
// Feature Envy: this method belongs in Customer, not OrderProcessor
public class OrderProcessor {
    public double calculateShipping(Customer customer) {
        if (customer.getCity().equals("NYC") &&
            customer.isPremium() &&
            customer.getOrderCount() > 10) {
            return 0.0;
        }
        return customer.getAddress().getState().getShippingRate();
    }
}
```

Fix: Move the method to `Customer`:

```java
public class Customer {
    public double calculateShipping() {
        if (city.equals("NYC") && isPremium && orderCount > 10) {
            return 0.0;
        }
        return address.getState().getShippingRate();
    }
}
```

Mistake 5: Splitting Too Aggressively

Not every class needs to be 10 lines long. Splitting a class into 15 tiny classes that constantly call each other is worse than having one slightly large but cohesive class. Split only when you identify genuinely separate responsibilities.

Best Practices

1. Keep classes small by responsibility, not by line count. A class with one responsibility and 150 lines is better than a class with three responsibilities and 50 lines.
2. Aim for high cohesion. Every method should use most of the class's instance variables. If you see clusters of methods that use different variables, those clusters are separate classes waiting to be extracted.
3. Name classes after what they do, not what they contain. `InvoiceGenerator` is better than `InvoiceData`. `OrderValidator` is better than `OrderHelper`.
4. Follow Tell, Don't Ask. Push behavior into the objects that own the data. If you find yourself calling `getX()` followed by an `if` statement, that logic probably belongs inside the object.
5. Design for one reason to change. Ask: "Who would request a change to this class?" If the answer is more than one person or team, the class has too many responsibilities.
6. Encapsulate aggressively. Make fields private. Expose behavior, not data. Let the class protect its own invariants.
7. Organize code like a newspaper. Constants at the top, then variables, then constructors, then public methods, then private helper methods.
8. Prefer composition over inheritance. Instead of extending a base class to gain functionality, inject collaborating objects. This keeps classes focused and flexible.

Quick Summary

Classes are the fundamental building blocks of object-oriented software. Clean classes are small, focused, and cohesive. Each class has exactly one reason to change. The God Class anti-pattern, where one class absorbs dozens of unrelated responsibilities, is one of the most common sources of complexity in real-world codebases. Encapsulation protects a class's internal state, and the Tell, Don't Ask principle pushes behavior to the objects that own the data. When you split a God Class, each resulting class is easier to understand, easier to test, and easier to change independently.

Key Points

- A class should have one reason to change -- one responsibility, one stakeholder.

- Cohesion measures how closely a class's methods and variables are related. High cohesion means every method uses most instance variables.
- The God Class anti-pattern happens when a class accumulates unrelated responsibilities over time. It is hard to understand, test, and change.
- Encapsulation means hiding internal state and exposing behavior. Make fields private and control access through methods.
- Tell, Don't Ask means telling an object what to do instead of querying its state and making decisions externally.
- Classes should be organized like a newspaper: public interface first, private details last.
- Split classes when you identify separate responsibilities, not just to reduce line count.
- Vague names like "Manager" or "Helper" are a sign that the class has too many responsibilities.

Practice Questions

1. You find a class called `ReportManager` with methods: `generatePdfReport()`, `sendReportByEmail()`, `saveReportToDatabase()`, `formatReportAsHtml()`, and `scheduleReportGeneration()`. How many responsibilities does this class have? How would you split it?
2. A colleague argues that creating many small classes increases complexity because there are more files to navigate. How would you respond? What are the tradeoffs?
3. Consider a `ShoppingCart` class with fields `items`, `discountCode`, and `shippingAddress`. It has methods `addItem()`, `removeItem()`, `calculateTotal()`, `applyDiscount()`, and `calculateShipping()`. Is this class cohesive? Should it be split? Why or why not?
4. What is the difference between encapsulation and simply making fields private? Give an example where making a field private without proper encapsulation still leads to problems.
5. Look at this code and identify the Tell, Don't Ask violation:

```java
if (employee.getVacationDays() > 0 && employee.getStatus().equals("ACTIVE")) {
    employee.setVacationDays(employee.getVacationDays() - 1);
    employee.setLastVacation(LocalDate.now());
}
```

Exercises

Exercise 1: Identify and Split a God Class

Below is a `StudentManager` class. Identify its separate responsibilities and refactor it into focused classes.

```python
class StudentManager:
    def __init__(self, db, email_client, pdf_generator):
        self.db = db
        self.email_client = email_client
        self.pdf_generator = pdf_generator

    def add_student(self, name, email, grade):
        self.db.insert("students", {"name": name, "email": email, "grade": grade})

    def find_student(self, student_id):
        return self.db.query("SELECT * FROM students WHERE id = ?", student_id)

    def calculate_gpa(self, student_id):
        grades = self.db.query("SELECT grade FROM courses WHERE student_id = ?", studen...
        return sum(grades) / len(grades)

    def send_report_card(self, student_id):
        student = self.find_student(student_id)
        gpa = self.calculate_gpa(student_id)
        self.email_client.send(student["email"], "Report Card", f"GPA: {gpa}")

    def generate_transcript(self, student_id):
        student = self.find_student(student_id)
        return self.pdf_generator.create(student)

    def get_honor_roll(self):
        return self.db.query("SELECT * FROM students WHERE gpa > 3.5")
```

Exercise 2: Apply Tell, Don't Ask

Refactor this code to follow the Tell, Don't Ask principle:

```java
public class TemperatureMonitor {
    public void checkTemperature(Thermostat thermostat) {
        double temp = thermostat.getCurrentTemperature();
        double max = thermostat.getMaxTemperature();
        double min = thermostat.getMinTemperature();

        if (temp > max) {
            thermostat.setMode("COOLING");
```

```
            thermostat.setFanSpeed(3);
        } else if (temp < min) {
            thermostat.setMode("HEATING");
            thermostat.setFanSpeed(2);
        } else {
            thermostat.setMode("IDLE");
            thermostat.setFanSpeed(0);
        }
    }
}
```

Exercise 3: Measure Cohesion

For the following class, calculate the cohesion score using the formula from this chapter. Then refactor it into classes with higher cohesion.

```java
public class FileProcessor {
    private String inputPath;
    private String outputPath;
    private Logger logger;
    private EmailClient notifier;

    public String readFile() { /* uses inputPath */ }
    public void writeFile(String content) { /* uses outputPath */ }
    public void logOperation(String msg) { /* uses logger */ }
    public void notifyComplete(String recipient) { /* uses notifier */ }
}
```

What Is Next?

Now that you understand how to design small, focused classes with a single responsibility, you are ready to explore this concept in greater depth. The next chapter dives into the Single Responsibility Principle (SRP) -- the first of the five SOLID principles. You will learn Uncle Bob's precise definition, including the actor-based perspective that makes SRP more actionable, and you will see detailed examples of identifying and separating responsibilities in real-world code.

Chapter 8: The Single Responsibility Principle (SRP)

What You Will Learn

By the end of this chapter, you will be able to:

- Define the Single Responsibility Principle using Uncle Bob's actor-based definition
- Distinguish between "a class should do one thing" and "a class should have one reason to change"
- Identify the actors (stakeholders) that drive changes to a class
- Recognize the symptoms of SRP violations in real-world code
- Refactor classes that violate SRP into focused, maintainable components
- Apply SRP at the method, class, and module level
- Understand how SRP relates to cohesion from the previous chapter

Why This Chapter Matters

The Single Responsibility Principle is the most frequently cited and most frequently misunderstood principle in software design. Many developers think it means "a class should do one thing." That definition is too vague to be useful. Does "one thing" mean one method? One algorithm? One feature?

Uncle Bob (Robert C. Martin) refined the definition to something far more actionable: "A module should be responsible to one, and only one, actor." An actor is a person, team, or stakeholder who might request changes to the code.

This distinction matters because it shifts your thinking from "what does this code do" to "who would ask me to change this code." When a class serves multiple actors, changes requested

by one actor can accidentally break functionality needed by another. SRP prevents this by ensuring that each class has exactly one reason to change.

The Definition: Getting It Right

The Common (Incomplete) Definition

> "A class should have only one responsibility."

This is the version most developers learn first. It is not wrong, but it is imprecise. What counts as "one responsibility"? Different developers will draw the line in different places.

Uncle Bob's Actor-Based Definition

> "A module should be responsible to one, and only one, actor."

An actor is a group of users or stakeholders who want the system to change in the same way. The Chief Financial Officer (CFO) cares about financial reports. The Chief Technology Officer (CTO) cares about system architecture. The Chief Operating Officer (COO) cares about operational workflows. Each is a different actor.

Putting the Two Definitions Together

```
┌──────────────────────────────────────────────────────────┐
│              SRP: Two Perspectives                       │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  VERSION 1: "One reason to change"                       │
│  Focus: What triggers a modification to this class?      │
│                                                          │
│  VERSION 2: "Responsible to one actor"                   │
│  Focus: Who requests changes to this class?              │
│                                                          │
│  Both say the same thing from different angles.          │
│  Version 2 is more actionable because it gives           │
│  you a concrete question to ask:                         │
│  "Who would ask me to change this code?"                 │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

The Classic Example: The Employee Class

Uncle Bob uses this example in Clean Architecture to illustrate why SRP matters at a deep level.

BEFORE: Three Actors, One Class

```java
public class Employee {
    private String name;
    private double baseSalary;
    private int hoursWorked;

    // Used by the CFO's team (accounting/payroll)
    public double calculatePay() {
        double regularPay = baseSalary;
        double overtimePay = calculateOvertimeHours() * (baseSalary / 160) * 1.5;
        return regularPay + overtimePay;
    }

    // Used by the COO's team (operations/HR)
    public void reportHours() {
        System.out.println(name + " worked " + hoursWorked + " hours");
        System.out.println("Regular: " + calculateRegularHours());
        System.out.println("Overtime: " + calculateOvertimeHours());
    }

    // Used by the CTO's team (database/technical)
    public void save() {
        // Save employee data to the database
        Database.execute("INSERT INTO employees ...", this);
    }

    // Shared helper method -- this is the danger zone
    private int calculateOvertimeHours() {
        return Math.max(0, hoursWorked - 40);
    }

    private int calculateRegularHours() {
        return Math.min(hoursWorked, 40);
    }
}
```

Why This Is Dangerous

Three different actors depend on this class:

```
            ┌──────────┐
            │   CFO    │ -- cares about calculatePay()
            └──────────┘
                 │
            ┌────▼──────────────────┐
            │                       │
            │     Employee          │
            │                       │
            │  calculatePay()       │◄── CFO's team
            │  reportHours()        │◄── COO's team
            │  save()               │◄── CTO's team
            │                       │
            │  calculateOvertime()  │◄── shared helper!
            │                       │
            └───────────────────────┘
                 ▲          ▲
            ┌────┴─────┐ ┌────┴─────┐
            │   COO    │ │   CTO    │
            └──────────┘ └──────────┘
```

Now imagine this scenario:

1. The CFO's team asks you to change how overtime is calculated for payroll purposes. They want overtime to start after 45 hours instead of 40.
2. You modify `calculateOvertimeHours()` to use 45.
3. The COO's team runs their hours report the next day. The report now shows overtime starting at 45 hours. But the COO's team still uses the 40-hour threshold for operational planning. Their reports are now wrong.
4. Nobody notices the bug until the end of the month when operational reports do not match expectations.

This is the core danger of SRP violations: changes requested by one actor accidentally affect another actor. The shared `calculateOvertimeHours()` method is used by both `calculatePay()` and `reportHours()`, but each actor has different rules for what counts as overtime.

AFTER: One Class Per Actor

Java:

```java
// Responsible to: CFO's team (payroll/accounting)
public class PayCalculator {
    private static final int PAYROLL_OVERTIME_THRESHOLD = 45;

    public double calculatePay(Employee employee) {
        double regularPay = employee.getBaseSalary();
        int overtimeHours = Math.max(0,
```

```
            employee.getHoursWorked() - PAYROLL_OVERTIME_THRESHOLD);
        double overtimePay = overtimeHours
            * (employee.getBaseSalary() / 160) * 1.5;
        return regularPay + overtimePay;
    }
}

// Responsible to: COO's team (operations/HR)
public class HoursReporter {
    private static final int OPERATIONS_OVERTIME_THRESHOLD = 40;

    public HoursReport generateReport(Employee employee) {
        int regular = Math.min(employee.getHoursWorked(),
            OPERATIONS_OVERTIME_THRESHOLD);
        int overtime = Math.max(0,
            employee.getHoursWorked() - OPERATIONS_OVERTIME_THRESHOLD);
        return new HoursReport(employee.getName(), regular, overtime);
    }
}

// Responsible to: CTO's team (technical/infrastructure)
public class EmployeeRepository {
    private final Database database;

    public EmployeeRepository(Database database) {
        this.database = database;
    }

    public void save(Employee employee) {
        database.execute("INSERT INTO employees ...", employee);
    }

    public Employee findById(int id) {
        return database.query("SELECT * FROM employees WHERE id = ?", id);
    }
}

// The Employee class is now just a data holder
public class Employee {
    private String name;
    private double baseSalary;
    private int hoursWorked;

    public Employee(String name, double baseSalary, int hoursWorked) {
        this.name = name;
        this.baseSalary = baseSalary;
        this.hoursWorked = hoursWorked;
    }

    public String getName() { return name; }
    public double getBaseSalary() { return baseSalary; }
    public int getHoursWorked() { return hoursWorked; }
}
```

Python:

```python
# Responsible to: CFO's team (payroll/accounting)
class PayCalculator:
    PAYROLL_OVERTIME_THRESHOLD = 45

    def calculate_pay(self, employee: Employee) -> float:
        regular_pay = employee.base_salary
        overtime_hours = max(0,
            employee.hours_worked - self.PAYROLL_OVERTIME_THRESHOLD)
        overtime_pay = overtime_hours * (employee.base_salary / 160) * 1.5
        return regular_pay + overtime_pay


# Responsible to: COO's team (operations/HR)
class HoursReporter:
    OPERATIONS_OVERTIME_THRESHOLD = 40

    def generate_report(self, employee: Employee) -> HoursReport:
        regular = min(employee.hours_worked,
                      self.OPERATIONS_OVERTIME_THRESHOLD)
        overtime = max(0,
            employee.hours_worked - self.OPERATIONS_OVERTIME_THRESHOLD)
        return HoursReport(employee.name, regular, overtime)


# Responsible to: CTO's team (technical/infrastructure)
class EmployeeRepository:
    def __init__(self, database):
        self._database = database

    def save(self, employee: Employee) -> None:
        self._database.execute("INSERT INTO employees ...", employee)

    def find_by_id(self, employee_id: int) -> Employee:
        return self._database.query(
            "SELECT * FROM employees WHERE id = ?", employee_id
        )


# Data holder -- no business logic
class Employee:
    def __init__(self, name: str, base_salary: float, hours_worked: int):
        self.name = name
        self.base_salary = base_salary
        self.hours_worked = hours_worked
```

Now each class changes for exactly one reason, requested by exactly one actor:

```
┌──────────┐          ┌──────────────────┐
│   CFO    │──────────►│  PayCalculator   │
└──────────┘          └──────────────────┘


┌──────────┐          ┌──────────────────┐
```

```
│   COO    │──────►│  HoursReporter    │
└──────────┘       └───────────────────┘

┌──────────┐       ┌───────────────────┐
│   CTO    │──────►│ EmployeeRepository│
└──────────┘       └───────────────────┘
```

The CFO's overtime rule change no longer affects the COO's reports. Each actor's logic is isolated.

Identifying Responsibilities: A Practical Method

Here is a step-by-step method for finding SRP violations in your code:

Step 1: List the Methods

Write down every public method in the class.

Step 2: Group by Actor

For each method, ask: "Who would request a change to this method?" Group methods by their actor.

Step 3: Check for Multiple Groups

If you find two or more groups, the class violates SRP. Each group should become its own class.

Worked Example

```
UserService class methods:
  - registerUser()          --> Product team
  - authenticateUser()      --> Security team
  - sendVerificationEmail() --> Marketing team
  - generateApiToken()      --> Security team
  - updateUserProfile()     --> Product team
  - logUserActivity()       --> Operations team
  - exportUserData()        --> Compliance team

Groups found:
```

```
  Product team:    registerUser(), updateUserProfile()
  Security team:   authenticateUser(), generateApiToken()
  Marketing team:  sendVerificationEmail()
  Operations team: logUserActivity()
  Compliance team: exportUserData()

Result: 5 actors = 5 classes needed
```

The refactored classes:

```
┌──────────────────────────┐
│   UserRegistration       │  <-- Product team
│   Service                │
├──────────────────────────┤
│ + registerUser()         │
│ + updateUserProfile()    │
└──────────────────────────┘


┌──────────────────────────┐
│   Authentication         │  <-- Security team
│   Service                │
├──────────────────────────┤
│ + authenticateUser()     │
│ + generateApiToken()     │
└──────────────────────────┘


┌──────────────────────────┐
│   UserNotification       │  <-- Marketing team
│   Service                │
├──────────────────────────┤
│ + sendVerificationEmail()│
└──────────────────────────┘


┌──────────────────────────┐
│   ActivityLogger         │  <-- Operations team
├──────────────────────────┤
│ + logUserActivity()      │
└──────────────────────────┘


┌──────────────────────────┐
│   UserDataExporter       │  <-- Compliance team
├──────────────────────────┤
│ + exportUserData()       │
└──────────────────────────┘
```

Before and After: UserService Refactoring

BEFORE: UserService That Does Everything

Java:

```java
public class UserService {
    private final UserRepository userRepo;
    private final PasswordEncoder encoder;
    private final JavaMailSender mailSender;
    private final Logger logger;
    private final JwtTokenProvider tokenProvider;
    private final TemplateEngine templateEngine;

    public UserService(UserRepository userRepo, PasswordEncoder encoder,
                       JavaMailSender mailSender, Logger logger,
                       JwtTokenProvider tokenProvider,
                       TemplateEngine templateEngine) {
        this.userRepo = userRepo;
        this.encoder = encoder;
        this.mailSender = mailSender;
        this.logger = logger;
        this.tokenProvider = tokenProvider;
        this.templateEngine = templateEngine;
    }

    // --- Authentication (Security team) ---
    public AuthResult authenticate(String email, String password) {
        User user = userRepo.findByEmail(email);
        if (user == null) {
            logger.warn("Login attempt for unknown email: " + email);
            return AuthResult.failure("Invalid credentials");
        }
        if (!encoder.matches(password, user.getPasswordHash())) {
            user.incrementFailedLogins();
            userRepo.save(user);
            logger.warn("Failed login for: " + email);
            return AuthResult.failure("Invalid credentials");
        }
        String token = tokenProvider.generateToken(user);
        logger.info("Successful login for: " + email);
        return AuthResult.success(token);
    }

    // --- Registration (Product team) ---
    public User register(String name, String email, String password) {
        if (userRepo.findByEmail(email) != null) {
            throw new DuplicateEmailException("Email already registered");
        }
        String hashedPassword = encoder.encode(password);
        User user = new User(name, email, hashedPassword);
        userRepo.save(user);

        // Send welcome email (Marketing team concern!)
        String body = templateEngine.render("welcome-email",
```

```
            Map.of("name", name));
        mailSender.send(email, "Welcome!", body);

        logger.info("New user registered: " + email);
        return user;
    }

    // --- Email notifications (Marketing team) ---
    public void sendPasswordResetEmail(String email) {
        User user = userRepo.findByEmail(email);
        if (user == null) return;

        String token = tokenProvider.generateResetToken(user);
        String body = templateEngine.render("password-reset",
            Map.of("token", token, "name", user.getName()));
        mailSender.send(email, "Password Reset", body);

        logger.info("Password reset email sent to: " + email);
    }

    // --- Logging (Operations team) ---
    public void logUserActivity(int userId, String activity) {
        logger.info("User " + userId + ": " + activity);
        userRepo.saveActivityLog(userId, activity, LocalDateTime.now());
    }
}
```

This class has four actors: the security team, the product team, the marketing team, and the operations team. It changes for four different reasons.

AFTER: Focused Classes

Java:

```java
// Actor: Security team
// Reason to change: authentication rules, token policies, lockout policies
public class AuthenticationService {
    private final UserRepository userRepo;
    private final PasswordEncoder encoder;
    private final JwtTokenProvider tokenProvider;
    private final Logger logger;

    public AuthenticationService(UserRepository userRepo,
                                 PasswordEncoder encoder,
                                 JwtTokenProvider tokenProvider,
                                 Logger logger) {
        this.userRepo = userRepo;
        this.encoder = encoder;
        this.tokenProvider = tokenProvider;
        this.logger = logger;
```

```
    }

    public AuthResult authenticate(String email, String password) {
        User user = userRepo.findByEmail(email);
        if (user == null) {
            logger.warn("Login attempt for unknown email: " + email);
            return AuthResult.failure("Invalid credentials");
        }
        if (!encoder.matches(password, user.getPasswordHash())) {
            user.incrementFailedLogins();
            userRepo.save(user);
            logger.warn("Failed login for: " + email);
            return AuthResult.failure("Invalid credentials");
        }
        String token = tokenProvider.generateToken(user);
        logger.info("Successful login for: " + email);
        return AuthResult.success(token);
    }
}
```

```java
// Actor: Product team
// Reason to change: registration requirements, profile fields, validation rules
public class UserRegistrationService {
    private final UserRepository userRepo;
    private final PasswordEncoder encoder;
    private final UserNotificationService notifications;
    private final Logger logger;

    public UserRegistrationService(UserRepository userRepo,
                                   PasswordEncoder encoder,
                                   UserNotificationService notifications,
                                   Logger logger) {
        this.userRepo = userRepo;
        this.encoder = encoder;
        this.notifications = notifications;
        this.logger = logger;
    }

    public User register(String name, String email, String password) {
        if (userRepo.findByEmail(email) != null) {
            throw new DuplicateEmailException("Email already registered");
        }
        String hashedPassword = encoder.encode(password);
        User user = new User(name, email, hashedPassword);
        userRepo.save(user);

        notifications.sendWelcomeEmail(user);

        logger.info("New user registered: " + email);
        return user;
    }
}
```

```java
// Actor: Marketing team
// Reason to change: email templates, email providers, notification channels
public class UserNotificationService {
    private final JavaMailSender mailSender;
    private final TemplateEngine templateEngine;
    private final JwtTokenProvider tokenProvider;

    public UserNotificationService(JavaMailSender mailSender,
                                   TemplateEngine templateEngine,
                                   JwtTokenProvider tokenProvider) {
        this.mailSender = mailSender;
        this.templateEngine = templateEngine;
        this.tokenProvider = tokenProvider;
    }

    public void sendWelcomeEmail(User user) {
        String body = templateEngine.render("welcome-email",
            Map.of("name", user.getName()));
        mailSender.send(user.getEmail(), "Welcome!", body);
    }

    public void sendPasswordResetEmail(User user) {
        String token = tokenProvider.generateResetToken(user);
        String body = templateEngine.render("password-reset",
            Map.of("token", token, "name", user.getName()));
        mailSender.send(user.getEmail(), "Password Reset", body);
    }
}
```

```java
// Actor: Operations team
// Reason to change: logging format, audit requirements, monitoring needs
public class UserActivityLogger {
    private final Logger logger;
    private final UserRepository userRepo;

    public UserActivityLogger(Logger logger, UserRepository userRepo) {
        this.logger = logger;
        this.userRepo = userRepo;
    }

    public void logActivity(int userId, String activity) {
        logger.info("User " + userId + ": " + activity);
        userRepo.saveActivityLog(userId, activity, LocalDateTime.now());
    }
}
```

Python version:

```python
# Actor: Security team
class AuthenticationService:
```

```
    def __init__(self, user_repo, password_encoder, token_provider, logger):
        self._user_repo = user_repo
        self._encoder = password_encoder
        self._token_provider = token_provider
        self._logger = logger

    def authenticate(self, email: str, password: str) -> AuthResult:
        user = self._user_repo.find_by_email(email)
        if user is None:
            self._logger.warning(f"Login attempt for unknown email: {email}")
            return AuthResult.failure("Invalid credentials")

        if not self._encoder.matches(password, user.password_hash):
            user.increment_failed_logins()
            self._user_repo.save(user)
            self._logger.warning(f"Failed login for: {email}")
            return AuthResult.failure("Invalid credentials")

        token = self._token_provider.generate_token(user)
        self._logger.info(f"Successful login for: {email}")
        return AuthResult.success(token)


# Actor: Product team
class UserRegistrationService:
    def __init__(self, user_repo, password_encoder, notifications, logger):
        self._user_repo = user_repo
        self._encoder = password_encoder
        self._notifications = notifications
        self._logger = logger

    def register(self, name: str, email: str, password: str) -> User:
        if self._user_repo.find_by_email(email) is not None:
            raise DuplicateEmailError("Email already registered")

        hashed_password = self._encoder.encode(password)
        user = User(name, email, hashed_password)
        self._user_repo.save(user)

        self._notifications.send_welcome_email(user)

        self._logger.info(f"New user registered: {email}")
        return user


# Actor: Marketing team
class UserNotificationService:
    def __init__(self, mail_sender, template_engine, token_provider):
        self._mail_sender = mail_sender
        self._template_engine = template_engine
        self._token_provider = token_provider

    def send_welcome_email(self, user: User) -> None:
        body = self._template_engine.render(
            "welcome-email", {"name": user.name}
```

```
        )
        self._mail_sender.send(user.email, "Welcome!", body)

    def send_password_reset_email(self, user: User) -> None:
        token = self._token_provider.generate_reset_token(user)
        body = self._template_engine.render(
            "password-reset", {"token": token, "name": user.name}
        )
        self._mail_sender.send(user.email, "Password Reset", body)


# Actor: Operations team
class UserActivityLogger:
    def __init__(self, logger, user_repo):
        self._logger = logger
        self._user_repo = user_repo

    def log_activity(self, user_id: int, activity: str) -> None:
        self._logger.info(f"User {user_id}: {activity}")
        self._user_repo.save_activity_log(
            user_id, activity, datetime.now()
        )
```

Symptoms of SRP Violation

How do you know when a class violates SRP without doing a full actor analysis? Look for these symptoms:

Symptom 1: The Class Has Many Unrelated Import Statements

```java
import java.sql.*;              // database
import javax.mail.*;            // email
import com.itextpdf.*;          // PDF generation
import org.apache.poi.*;        // Excel export
import com.stripe.api.*;        // payment processing
import org.slf4j.*;             // logging
```

If your class imports libraries for databases, email, PDFs, and payment processing, it is doing too many things.

Symptom 2: The Class Changes Frequently for Different Reasons

Look at your version control history. If the commit messages for a single file include:

- "Fix tax calculation bug"
- "Update email template"
- "Change database query for performance"
- "Add new report format"

Four different kinds of changes to one file means four responsibilities.

Symptom 3: Hard to Name the Class

If you struggle to name a class without using vague words like "Manager," "Processor," "Handler," or "Service," it probably does too much. A class with a clear responsibility has a clear name.

```
Vague names (red flag):     Specific names (good sign):
  DataManager                 OrderRepository
  RequestHandler              PasswordHasher
  UserProcessor               InvoiceGenerator
  SystemService               ShippingCalculator
  ApplicationHelper           EmailTemplateRenderer
```

Symptom 4: Large Constructor

If the constructor takes more than three or four dependencies, the class probably has too many responsibilities. Each dependency often corresponds to a different concern.

```java
// Red flag: 7 dependencies = likely 3-4 responsibilities
public UserService(UserRepository repo,
                   PasswordEncoder encoder,
                   EmailSender emailSender,
                   SmsService smsService,
                   PaymentGateway paymentGateway,
                   AuditLogger auditLogger,
                   CacheManager cacheManager) {
    // ...
}
```

Symptom 5: Methods That Do Not Use Each Other

In a class with a single responsibility, methods tend to call each other or operate on the same data. If a class has clusters of methods that never interact, those clusters are separate responsibilities.

```
┌──────────────────────────────────┐
│         UserService              │
│                                  │
│  Cluster A:         Cluster B:   │
│  ┌──────────────┐  ┌──────────────┐│
│  │ register()   │  │ sendEmail()  │
│  │ validate()   │  │ formatMsg()  │
│  │ saveUser()   │  │ getTempl()   │
│  └──────────────┘  └──────────────┘│
│                                  │
│  These clusters never interact.  │
│  They should be separate classes.│
└──────────────────────────────────┘
```

SRP at Different Levels

SRP applies not just to classes, but to methods and modules as well.

SRP at the Method Level

A method should do one thing. If a method name requires "and" to describe what it does, it should be split.

BEFORE:

```python
def create_user_and_send_email(name, email, password):
    # Validate input
    if not email or "@" not in email:
        raise ValueError("Invalid email")
    if len(password) < 8:
        raise ValueError("Password too short")

    # Create user
    hashed = hash_password(password)
    user = User(name, email, hashed)
    db.save(user)

    # Send email
    body = f"Welcome {name}!"
    email_client.send(email, "Welcome", body)

    return user
```

AFTER:

```python
def create_user(name: str, email: str, password: str) -> User:
    validate_registration_input(email, password)
    hashed = hash_password(password)
    user = User(name, email, hashed)
    db.save(user)
    return user

def validate_registration_input(email: str, password: str) -> None:
    if not email or "@" not in email:
        raise ValueError("Invalid email")
    if len(password) < 8:
        raise ValueError("Password too short")

def send_welcome_email(user: User) -> None:
    body = f"Welcome {user.name}!"
    email_client.send(user.email, "Welcome", body)

# The caller coordinates:
user = create_user(name, email, password)
send_welcome_email(user)
```

SRP at the Module/Package Level

At a larger scale, SRP means that a module or package should serve one purpose.

```
project/
  auth/                 # Authentication module
    authentication.py
    token_manager.py
    password_hasher.py

  notifications/        # Notification module
    email_sender.py
    sms_sender.py
    templates.py

  billing/              # Billing module
    invoice_generator.py
    payment_processor.py
    tax_calculator.py

  users/                # User management module
    user_repository.py
    user_validator.py
    profile_service.py
```

Each package has a clear purpose. The auth package changes when authentication requirements change. The billing package changes when billing requirements change. They do not interfere with each other.

Real-World Scenario: The Report Generator

You inherit a `ReportGenerator` class that creates reports for three departments:

```python
class ReportGenerator:
    def __init__(self, db, pdf_engine, email_sender):
        self.db = db
        self.pdf_engine = pdf_engine
        self.email_sender = email_sender

    def generate_sales_report(self, month, year):
        """Sales team wants: revenue by region, top products."""
        data = self.db.query(
            "SELECT region, SUM(amount) FROM sales "
            "WHERE month = ? AND year = ? GROUP BY region",
            month, year
        )
        pdf = self.pdf_engine.create_report(
            title=f"Sales Report - {month}/{year}",
            data=data,
            chart_type="bar"
        )
        self.email_sender.send("sales@company.com",
                               "Monthly Sales Report", pdf)
        return pdf

    def generate_inventory_report(self, warehouse_id):
        """Operations team wants: stock levels, reorder alerts."""
        data = self.db.query(
            "SELECT product, quantity, reorder_level FROM inventory "
            "WHERE warehouse_id = ?", warehouse_id
        )
        alerts = [row for row in data if row["quantity"] < row["reorder_level"]]
        pdf = self.pdf_engine.create_report(
            title=f"Inventory Report - Warehouse {warehouse_id}",
            data=data,
            alerts=alerts
        )
        self.email_sender.send("ops@company.com",
                               "Inventory Report", pdf)
        return pdf

    def generate_hr_report(self, department):
        """HR team wants: headcount, turnover rate, open positions."""
        data = self.db.query(
            "SELECT * FROM employees WHERE department = ?", department
        )
        turnover = self.db.query(
```

```
            "SELECT COUNT(*) FROM departures WHERE department = ? "
            "AND year = YEAR(CURDATE())", department
        )
        pdf = self.pdf_engine.create_report(
            title=f"HR Report - {department}",
            data=data,
            turnover_rate=turnover
        )
        self.email_sender.send("hr@company.com", "HR Report", pdf)
        return pdf
```

Three actors (sales, operations, HR) means three classes:

```python
class SalesReportGenerator:
    def __init__(self, sales_repo, pdf_engine, email_sender):
        self._repo = sales_repo
        self._pdf = pdf_engine
        self._email = email_sender

    def generate(self, month: int, year: int) -> bytes:
        data = self._repo.get_revenue_by_region(month, year)
        pdf = self._pdf.create_report(
            title=f"Sales Report - {month}/{year}",
            data=data, chart_type="bar"
        )
        self._email.send("sales@company.com",
                         "Monthly Sales Report", pdf)
        return pdf


class InventoryReportGenerator:
    def __init__(self, inventory_repo, pdf_engine, email_sender):
        self._repo = inventory_repo
        self._pdf = pdf_engine
        self._email = email_sender

    def generate(self, warehouse_id: int) -> bytes:
        data = self._repo.get_stock_levels(warehouse_id)
        alerts = [item for item in data
                  if item.quantity < item.reorder_level]
        pdf = self._pdf.create_report(
            title=f"Inventory Report - Warehouse {warehouse_id}",
            data=data, alerts=alerts
        )
        self._email.send("ops@company.com",
                         "Inventory Report", pdf)
        return pdf


class HrReportGenerator:
    def __init__(self, hr_repo, pdf_engine, email_sender):
        self._repo = hr_repo
```

```
        self._pdf = pdf_engine
        self._email = email_sender

    def generate(self, department: str) -> bytes:
        data = self._repo.get_employees(department)
        turnover = self._repo.get_turnover_rate(department)
        pdf = self._pdf.create_report(
            title=f"HR Report - {department}",
            data=data, turnover_rate=turnover
        )
        self._email.send("hr@company.com", "HR Report", pdf)
        return pdf
```

Now when the sales team wants a new chart type, only `SalesReportGenerator` changes. When HR wants to add a diversity metric, only `HrReportGenerator` changes.

Common Mistakes

Mistake 1: Taking SRP Too Far

SRP does not mean "one method per class." A class with a single responsibility can have many methods, as long as they all serve the same actor and change for the same reason.

```java
// This is fine! All methods serve the same actor (tax accountants)
// and change for the same reason (tax rules change)
public class TaxCalculator {
    public double calculateIncomeTax(double income) { ... }
    public double calculateSalesTax(double amount, String state) { ... }
    public double calculatePropertyTax(double value, String county) { ... }
    public double calculateCapitalGainsTax(double gains) { ... }
    public TaxBracket determineBracket(double income) { ... }
}
```

Mistake 2: Confusing SRP with the Interface Segregation Principle

SRP is about who requests changes (actors). ISP is about what clients depend on (interfaces). A class can satisfy SRP while violating ISP, and vice versa.

Mistake 3: Splitting by Technical Layer Instead of by Actor

Some developers split by layer (controller, service, repository) but keep all actors in each layer:

```
// WRONG: Split by layer, but UserService still serves 4 actors
UserController  -->  UserService  -->  UserRepository
                     (auth + email + logging + registration)
```

```
// RIGHT: Split by actor within each layer
AuthController  -->  AuthService     -->  UserRepository
RegController   -->  RegistrationSvc -->  UserRepository
```

Mistake 4: Ignoring SRP Because "It Works"

Code that violates SRP works fine... until it does not. The problems emerge over time: merge conflicts, accidental side effects, difficulty testing. By the time the pain is obvious, the code is much harder to fix.

Best Practices

1. Ask "who would request a change?" for every class you write. If the answer is more than one actor, consider splitting.
2. Watch for classes with many unrelated dependencies. A constructor with six parameters from different domains (database, email, PDF, payments) is a red flag.
3. Check your version control history. If a file changes frequently for different reasons, it violates SRP.
4. Name classes precisely. A good name like `ShippingCostCalculator` makes SRP violations obvious. A vague name like `OrderHelper` hides them.
5. Apply SRP at the method level first. Methods that do one thing naturally lead to classes that have one responsibility.
6. Use a facade or coordinator when needed. After splitting a class, you may need a thin coordinator class that delegates to the focused classes. This is fine; the coordinator itself has one responsibility: orchestration.
7. Do not split prematurely. If a class is small, has few methods, and is unlikely to attract new responsibilities, leave it alone. Split when you feel the pain.

Quick Summary

The Single Responsibility Principle states that a class should be responsible to one, and only one, actor. An actor is a stakeholder who would request changes to the code. When a class serves multiple actors, changes requested by one can accidentally break functionality needed by another. You identify SRP violations by listing a class's methods, grouping them by actor, and checking whether multiple groups exist. Symptoms include many unrelated imports, frequent changes for different reasons, vague class names, large constructors, and clusters of methods that do not interact. SRP applies at the method, class, and module levels.

Key Points

- SRP means one actor, one reason to change -- not "one method per class."
- Uncle Bob's definition: "A module should be responsible to one, and only one, actor."
- Actors are stakeholders (people, teams, roles) who request changes to the system.
- The danger of SRP violations: changes for one actor accidentally break functionality for another actor.
- Symptoms of SRP violations: many unrelated imports, vague class names, large constructors, clusters of non-interacting methods, and frequent changes for different reasons.
- Apply SRP at three levels: method, class, and module/package.
- Do not over-apply SRP. A class with 10 methods can still have one responsibility if all methods serve the same actor.

Practice Questions

1. A class called `PaymentProcessor` has these methods: `chargeCard()`, `refund()`, `generateReceipt()`, `sendReceiptEmail()`, and `logTransaction()`. Identify the actors and suggest how to split this class.
2. Your colleague says: "SRP means every class should have exactly one method." How would you correct this misunderstanding?
3. Look at your current project. Find one class that serves multiple actors. Which actors does it serve? How would you refactor it?

4. A `NotificationService` handles email, SMS, and push notifications. Does this violate SRP? It depends. Explain under what circumstances it does and does not violate SRP.
5. Why is "a class should do one thing" an incomplete definition of SRP? Give an example where this definition leads to a wrong conclusion.

Exercises

Exercise 1: Actor Analysis

Analyze the following class. Identify each actor, which methods they own, and how you would split the class.

```java
public class ProductService {
    public Product createProduct(String name, double price) { ... }
    public void updatePrice(int productId, double newPrice) { ... }
    public List<Product> searchProducts(String query) { ... }
    public byte[] generateProductCatalog() { ... }
    public void syncWithWarehouse(int productId) { ... }
    public void applyBulkDiscount(List<Integer> productIds, double pct) { ... }
    public ProductAnalytics getAnalytics(int productId) { ... }
}
```

Exercise 2: Refactor a Python Class

Refactor this class to follow SRP. Create separate classes for each responsibility.

```python
class DocumentManager:
    def __init__(self, storage, converter, notifier, logger):
        self.storage = storage
        self.converter = converter
        self.notifier = notifier
        self.logger = logger

    def upload(self, file_path, user_id):
        content = open(file_path, 'rb').read()
        doc_id = self.storage.save(content, user_id)
        self.logger.info(f"Document {doc_id} uploaded by {user_id}")
        self.notifier.notify(user_id, f"Document {doc_id} uploaded")
        return doc_id

    def convert_to_pdf(self, doc_id):
```

```
        content = self.storage.load(doc_id)
        pdf = self.converter.to_pdf(content)
        self.storage.save(pdf, doc_id + "_pdf")
        return doc_id + "_pdf"

    def share(self, doc_id, recipient_email):
        link = self.storage.generate_link(doc_id)
        self.notifier.send_email(recipient_email, "Shared doc", link)
        self.logger.info(f"Document {doc_id} shared with {recipient_email}")
```

Exercise 3: Commit History Analysis

Look at the git log for a file in your project that changes frequently. Categorize each commit message by the type of change (bug fix, feature, UI change, database change, etc.). If you find more than two categories, the file likely violates SRP. Propose how you would split it.

What Is Next?

With SRP in your toolkit, you can ensure that each class changes for exactly one reason. The next chapter explores the Open-Closed Principle (OCP) -- the idea that software should be open for extension but closed for modification. You will learn how to design classes that can gain new behavior without changing existing code, using techniques like the Strategy pattern and polymorphism.

Chapter 9: The Open-Closed Principle -- Extend Without Modifying

What You Will Learn

- What the Open-Closed Principle (OCP) means in practical terms
- How to add new behavior to a system without changing existing, tested code
- The Strategy pattern as the primary tool for achieving OCP
- How to refactor if/else chains for different types into polymorphic designs
- How plugin architectures leverage OCP at a larger scale
- How Python's duck typing naturally supports OCP
- When OCP is overkill and you should keep things simple

Why This Chapter Matters

Every time you modify an existing class to add a new feature, you risk breaking something that already works. If a class handles credit card payments and you edit it to also handle PayPal, you might accidentally break the credit card logic. Multiply this across a team of developers all modifying the same files, and you have a recipe for constant regressions.

The Open-Closed Principle eliminates this risk. It teaches you to design your code so that adding a new payment method means creating a new class, not touching any existing class. The existing code stays untouched, tested, and reliable. The new code is isolated and easy to verify.

This is not theoretical. This principle directly determines whether your team can ship new features quickly or gets bogged down fixing regressions every sprint.

The Open-Closed Principle Defined

The Open-Closed Principle was coined by Bertrand Meyer in 1988:

> Software entities (classes, modules, functions) should be open for extension but closed for modification.

What does this mean?

- Open for extension: You can add new behavior. New features, new types, new rules.
- Closed for modification: You do not change existing, working code to add that new behavior.

```
+-------------------------------------------------------------+
|           THE OPEN-CLOSED PRINCIPLE                         |
+-------------------------------------------------------------+
|                                                             |
|  CLOSED for modification:                                   |
|  +---------------------------+                              |
|  |  Existing Code            |  <-- Do NOT change this    |
|  |  (tested, working,        |                             |
|  |   deployed, reliable)     |                             |
|  +---------------------------+                              |
|              |                                              |
|              | extends via abstraction                      |
|              v                                              |
|  OPEN for extension:                                        |
|  +---------------------------+                              |
|  |  New Code                 |  <-- ADD this               |
|  |  (new feature, new type,  |                             |
|  |   new behavior)           |                             |
|  +---------------------------+                              |
|                                                             |
|  Existing tests still pass.                                 |
|  New feature has its own tests.                             |
|  Nobody touched the old code.                               |
|                                                             |
+-------------------------------------------------------------+
```

The Problem: If/Else Chains That Keep Growing

The most common OCP violation is an if/else chain (or switch statement) that grows every time you add a new type or variant.

BEFORE: The Payment Type Problem (Java)

```java
// Java -- BAD: Every new payment type requires modifying this class
public class PaymentProcessor {

    public PaymentResult processPayment(Order order, String paymentType) {

        if (paymentType.equals("CREDIT_CARD")) {
            CreditCardGateway gateway = new CreditCardGateway();
            boolean success = gateway.charge(
                order.getCreditCardNumber(),
                order.getAmount()
            );
            if (success) {
                return new PaymentResult(true, "Credit card charged");
            } else {
                return new PaymentResult(false, "Credit card declined");
            }

        } else if (paymentType.equals("PAYPAL")) {
            PayPalClient client = new PayPalClient();
            String token = client.authenticate(order.getPaypalEmail());
            boolean success = client.executePayment(token, order.getAmount());
            if (success) {
                return new PaymentResult(true, "PayPal payment completed");
            } else {
                return new PaymentResult(false, "PayPal payment failed");
            }

        } else if (paymentType.equals("BITCOIN")) {
            BitcoinWallet wallet = new BitcoinWallet();
            String txHash = wallet.sendPayment(
                order.getBitcoinAddress(),
                order.getAmount()
            );
            if (txHash != null) {
                return new PaymentResult(true, "Bitcoin tx: " + txHash);
            } else {
                return new PaymentResult(false, "Bitcoin payment failed");
            }

        } else {
            throw new UnsupportedPaymentException(
                "Unknown payment type: " + paymentType
            );
```

```
        }
    }
}
```

What happens when the business asks you to add Apple Pay? You open `PaymentProcessor`, add another `else if` block, and risk breaking the credit card, PayPal, and Bitcoin logic that already works. This class will grow forever.

BEFORE: The Same Problem in Python

```python
# Python -- BAD: Growing if/elif chain
class PaymentProcessor:

    def process_payment(self, order, payment_type: str) -> dict:

        if payment_type == "credit_card":
            gateway = CreditCardGateway()
            success = gateway.charge(
                order.credit_card_number, order.amount
            )
            return {
                "success": success,
                "message": "Credit card charged" if success
                           else "Credit card declined",
            }

        elif payment_type == "paypal":
            client = PayPalClient()
            token = client.authenticate(order.paypal_email)
            success = client.execute_payment(token, order.amount)
            return {
                "success": success,
                "message": "PayPal completed" if success
                           else "PayPal failed",
            }

        elif payment_type == "bitcoin":
            wallet = BitcoinWallet()
            tx_hash = wallet.send_payment(
                order.bitcoin_address, order.amount
            )
            return {
                "success": tx_hash is not None,
                "message": f"Bitcoin tx: {tx_hash}" if tx_hash
                           else "Bitcoin failed",
            }

        else:
            raise ValueError(f"Unknown payment type: {payment_type}")
```

The Solution: Strategy Pattern Enables OCP

The Strategy pattern extracts each variant into its own class that implements a common interface. The processor delegates to whatever strategy it receives, without knowing the details.

```
+-------------------------------------------------------------+
|           STRATEGY PATTERN FOR OCP                          |
+-------------------------------------------------------------+
|                                                             |
|  +--------------------+                                     |
|  | PaymentProcessor   |    uses     +--------------------+  |
|  |                    |------------>| PaymentStrategy    |  |
|  | process(order,     |             | (interface)        |  |
|  |   strategy)        |             +--------------------+  |
|  +--------------------+             | + pay(order)       |  |
|                                     +--------------------+  |
|                                       ^    ^    ^    ^      |
|                                       |    |    |    |      |
|                    +---------+-------+    |    |    +----+  |
|                    |         |            |    |         |  |
|               +--------+ +--------+ +--------+ +--------+ |
|               |Credit  | |PayPal  | |Bitcoin | |ApplePay| |
|               |Card    | |Strategy| |Strategy| |Strategy| |
|               |Strategy| |        | |        | |        | |
|               +--------+ +--------+ +--------+ +--------+ |
|                                                             |
|  To add Apple Pay:                                          |
|    1. Create ApplePayStrategy (NEW class)                   |
|    2. PaymentProcessor: UNCHANGED                           |
|    3. CreditCardStrategy: UNCHANGED                         |
|    4. PayPalStrategy: UNCHANGED                             |
|    5. BitcoinStrategy: UNCHANGED                            |
|                                                             |
+-------------------------------------------------------------+
```

AFTER: Strategy Pattern (Java)

```java
// Java -- GOOD: OCP-compliant with Strategy pattern

// Step 1: Define the abstraction
public interface PaymentStrategy {

    PaymentResult pay(Order order);
```

```
    String getPaymentType();
}

// Step 2: Implement each strategy
public class CreditCardPayment implements PaymentStrategy {

    private final CreditCardGateway gateway;

    public CreditCardPayment(CreditCardGateway gateway) {
        this.gateway = gateway;
    }

    @Override
    public PaymentResult pay(Order order) {
        boolean success = gateway.charge(
            order.getCreditCardNumber(),
            order.getAmount()
        );
        return new PaymentResult(
            success,
            success ? "Credit card charged" : "Credit card declined"
        );
    }

    @Override
    public String getPaymentType() {
        return "CREDIT_CARD";
    }
}

public class PayPalPayment implements PaymentStrategy {

    private final PayPalClient client;

    public PayPalPayment(PayPalClient client) {
        this.client = client;
    }

    @Override
    public PaymentResult pay(Order order) {
        String token = client.authenticate(order.getPaypalEmail());
        boolean success = client.executePayment(token, order.getAmount());
        return new PaymentResult(
            success,
            success ? "PayPal payment completed" : "PayPal payment failed"
        );
    }

    @Override
    public String getPaymentType() {
        return "PAYPAL";
    }
}
```

```
public class BitcoinPayment implements PaymentStrategy {

    private final BitcoinWallet wallet;

    public BitcoinPayment(BitcoinWallet wallet) {
        this.wallet = wallet;
    }

    @Override
    public PaymentResult pay(Order order) {
        String txHash = wallet.sendPayment(
            order.getBitcoinAddress(),
            order.getAmount()
        );
        boolean success = txHash != null;
        return new PaymentResult(
            success,
            success ? "Bitcoin tx: " + txHash : "Bitcoin payment failed"
        );
    }

    @Override
    public String getPaymentType() {
        return "BITCOIN";
    }
}

// Step 3: The processor is now closed for modification
public class PaymentProcessor {

    public PaymentResult processPayment(
            Order order,
            PaymentStrategy strategy) {

        return strategy.pay(order);
    }
}
```

Now adding Apple Pay requires zero changes to existing code:

```java
// Java -- Adding Apple Pay: create a NEW class, change NOTHING else
public class ApplePayPayment implements PaymentStrategy {

    private final ApplePayService service;

    public ApplePayPayment(ApplePayService service) {
        this.service = service;
    }

    @Override
    public PaymentResult pay(Order order) {
        String receipt = service.processPayment(
```

```
            order.getApplePayToken(),
            order.getAmount()
        );
        boolean success = receipt != null;
        return new PaymentResult(
            success,
            success ? "Apple Pay receipt: " + receipt
                    : "Apple Pay failed"
        );
    }

    @Override
    public String getPaymentType() {
        return "APPLE_PAY";
    }
}
```

AFTER: Strategy Pattern (Python)

```python
# Python -- GOOD: OCP-compliant with Strategy pattern
from abc import ABC, abstractmethod


class PaymentStrategy(ABC):
    """Abstraction that all payment methods implement."""

    @abstractmethod
    def pay(self, order: Order) -> PaymentResult:
        pass


class CreditCardPayment(PaymentStrategy):

    def __init__(self, gateway: CreditCardGateway):
        self._gateway = gateway

    def pay(self, order: Order) -> PaymentResult:
        success = self._gateway.charge(
            order.credit_card_number, order.amount
        )
        return PaymentResult(
            success=success,
            message="Credit card charged" if success
                    else "Credit card declined",
        )


class PayPalPayment(PaymentStrategy):

    def __init__(self, client: PayPalClient):
```

```
        self._client = client

    def pay(self, order: Order) -> PaymentResult:
        token = self._client.authenticate(order.paypal_email)
        success = self._client.execute_payment(token, order.amount)
        return PaymentResult(
            success=success,
            message="PayPal completed" if success
                    else "PayPal failed",
        )


class BitcoinPayment(PaymentStrategy):

    def __init__(self, wallet: BitcoinWallet):
        self._wallet = wallet

    def pay(self, order: Order) -> PaymentResult:
        tx_hash = self._wallet.send_payment(
            order.bitcoin_address, order.amount
        )
        return PaymentResult(
            success=tx_hash is not None,
            message=f"Bitcoin tx: {tx_hash}" if tx_hash
                    else "Bitcoin failed",
        )


# The processor is closed for modification
class PaymentProcessor:

    def process_payment(
        self,
        order: Order,
        strategy: PaymentStrategy,
    ) -> PaymentResult:
        return strategy.pay(order)
```

Adding Apple Pay in Python:

```python
# Python -- Adding Apple Pay: new class, nothing else changes
class ApplePayPayment(PaymentStrategy):

    def __init__(self, service: ApplePayService):
        self._service = service

    def pay(self, order: Order) -> PaymentResult:
        receipt = self._service.process_payment(
            order.apple_pay_token, order.amount
        )
        return PaymentResult(
            success=receipt is not None,
```

```
            message=f"Apple Pay receipt: {receipt}" if receipt
                    else "Apple Pay failed",
        )
```

Polymorphism Over Conditionals

The Strategy pattern is one instance of a broader principle: prefer polymorphism over conditionals. Whenever you have an if/else or switch that checks the type of something to decide what to do, consider replacing it with polymorphic dispatch.

```
+-------------------------------------------------------------+
|     CONDITIONALS vs POLYMORPHISM                            |
+-------------------------------------------------------------+
|                                                             |
|  Conditional approach:                                      |
|  if (shape == "circle")     draw circle                     |
|  else if (shape == "rect")  draw rectangle                  |
|  else if (shape == "tri")   draw triangle                   |
|  --> Adding a new shape means finding every if/else         |
|      chain in the entire codebase and updating it.          |
|                                                             |
|  Polymorphic approach:                                      |
|  shape.draw()               // each shape knows how         |
|  --> Adding a new shape means creating one new class.       |
|      Zero other classes change.                             |
|                                                             |
+-------------------------------------------------------------+
```

Another Example: Discount Calculation

```java
// Java -- BAD: Conditional discount calculation
public double calculateDiscount(Customer customer, double amount) {
    if (customer.getType().equals("REGULAR")) {
        return 0;
    } else if (customer.getType().equals("PREMIUM")) {
        return amount * 0.10;
    } else if (customer.getType().equals("VIP")) {
        return amount * 0.20;
    } else if (customer.getType().equals("EMPLOYEE")) {
        return amount * 0.30;
    }
    return 0;
```

```
}

// Java -- GOOD: Polymorphic discount calculation
public interface DiscountPolicy {
    double calculateDiscount(double amount);
}

public class RegularDiscount implements DiscountPolicy {
    public double calculateDiscount(double amount) { return 0; }
}

public class PremiumDiscount implements DiscountPolicy {
    public double calculateDiscount(double amount) {
        return amount * 0.10;
    }
}

public class VipDiscount implements DiscountPolicy {
    public double calculateDiscount(double amount) {
        return amount * 0.20;
    }
}

// Usage: customer carries its own discount policy
double discount = customer.getDiscountPolicy().calculateDiscount(amount);
```

Plugin Architecture: OCP at Scale

The Strategy pattern is OCP applied to a single decision point. A plugin architecture is OCP applied to an entire system. The core application defines extension points, and plugins provide implementations without modifying the core.

```
+-------------------------------------------------------------+
|           PLUGIN ARCHITECTURE                               |
+-------------------------------------------------------------+
|                                                             |
|  +-------------------------------------------------+     |
|  |              CORE APPLICATION                    |     |
|  |                                                  |     |
|  |  +-----------+  +-----------+  +-----------+     |     |
|  |  | Extension |  | Extension |  | Extension |     |     |
|  |  | Point A   |  | Point B   |  | Point C   |     |     |
|  |  +-----------+  +-----------+  +-----------+     |     |
|  |       ^              ^              ^             |     |
|  +-------|--------------|--------------|-------------+     |
|          |              |              |                   |
|     +---------+    +---------+    +---------+              |
```

```
|     | Plugin  |    | Plugin  |    | Plugin  |              |
|     | A1      |    | B1      |    | C1      |              |
|     +---------+    +---------+    +---------+              |
|     | Plugin  |    | Plugin  |                             |
|     | A2      |    | B2      |                             |
|     +---------+    +---------+                             |
|                                                            |
|  Core never changes. Plugins are added independently.      |
|                                                            |
+------------------------------------------------------------+
```

Real-world examples of plugin architectures:

- IDEs: VS Code extensions, IntelliJ plugins
- Build tools: Maven plugins, Webpack loaders
- Web frameworks: Django middleware, Spring Boot starters
- Browsers: Chrome extensions, Firefox add-ons

Registry Pattern for Dynamic Strategy Selection

In real applications, you often need to select a strategy at runtime based on some input. A registry pattern lets you add strategies without modifying the selection logic.

```java
// Java -- Registry pattern for strategy selection
public class PaymentStrategyRegistry {

    private final Map<String, PaymentStrategy> strategies = new HashMap<>();

    public void register(PaymentStrategy strategy) {
        strategies.put(strategy.getPaymentType(), strategy);
    }

    public PaymentStrategy getStrategy(String paymentType) {
        PaymentStrategy strategy = strategies.get(paymentType);
        if (strategy == null) {
            throw new UnsupportedPaymentException(
                "No strategy registered for: " + paymentType
            );
        }
        return strategy;
    }
}

// Configuration: register strategies at startup
PaymentStrategyRegistry registry = new PaymentStrategyRegistry();
registry.register(new CreditCardPayment(new CreditCardGateway()));
registry.register(new PayPalPayment(new PayPalClient()));
registry.register(new BitcoinPayment(new BitcoinWallet()));
```

```
// Adding a new payment type: just register it. No code changes.
registry.register(new ApplePayPayment(new ApplePayService()));
```

```python
# Python -- Registry pattern
class PaymentStrategyRegistry:

    def __init__(self):
        self._strategies: dict[str, PaymentStrategy] = {}

    def register(self, payment_type: str, strategy: PaymentStrategy):
        self._strategies[payment_type] = strategy

    def get_strategy(self, payment_type: str) -> PaymentStrategy:
        strategy = self._strategies.get(payment_type)
        if strategy is None:
            raise UnsupportedPaymentError(
                f"No strategy registered for: {payment_type}"
            )
        return strategy


# Configuration at startup
registry = PaymentStrategyRegistry()
registry.register("credit_card", CreditCardPayment(CreditCardGateway()))
registry.register("paypal", PayPalPayment(PayPalClient()))
registry.register("bitcoin", BitcoinPayment(BitcoinWallet()))
registry.register("apple_pay", ApplePayPayment(ApplePayService()))
```

OCP in Python: Duck Typing Helps

Python's duck typing makes OCP even easier. You do not always need a formal abstract base class. If an object has the right methods, it works.

```python
# Python -- Duck typing enables OCP naturally

# No formal interface needed -- just define the method
class EmailNotifier:
    def notify(self, user: User, message: str) -> None:
        send_email(user.email, message)


class SlackNotifier:
    def notify(self, user: User, message: str) -> None:
        post_to_slack(user.slack_id, message)
```

```
class SmsNotifier:
    def notify(self, user: User, message: str) -> None:
        send_sms(user.phone, message)


# The service works with anything that has a notify() method
class AlertService:

    def __init__(self, notifiers: list):
        self._notifiers = notifiers

    def send_alert(self, user: User, message: str) -> None:
        for notifier in self._notifiers:
            notifier.notify(user, message)


# Adding a new notifier requires zero changes to AlertService
class TeamsNotifier:
    def notify(self, user: User, message: str) -> None:
        post_to_teams(user.teams_id, message)
```

However, if you want to be explicit about the contract (recommended for larger projects), use Python's `Protocol`:

```python
# Python -- Protocol for explicit duck typing
from typing import Protocol


class Notifier(Protocol):
    def notify(self, user: User, message: str) -> None: ...


class AlertService:

    def __init__(self, notifiers: list[Notifier]):
        self._notifiers = notifiers

    def send_alert(self, user: User, message: str) -> None:
        for notifier in self._notifiers:
            notifier.notify(user, message)
```

When OCP Is Overkill

OCP is powerful, but applying it everywhere leads to over-engineered code. Here are signs that OCP is overkill:

1. The if/else has only two branches and is unlikely to grow. If your application will only ever have two user roles (admin and user), a simple if/else is fine.
2. The variation point is in only one place. OCP shines when the same type check appears in multiple places. If it only exists in one function, the overhead of an interface and multiple classes may not be worth it.
3. You are speculating about future requirements. Do not create extension points for features nobody has asked for. Apply OCP when you see an actual pattern of change, not a hypothetical one.
4. The code is a short script or prototype. If the code will live for a few days, formatting it with full OCP compliance is wasted effort.

The rule of three: If you have modified the same if/else chain three times to add a new case, it is time to refactor to OCP. Before that, keep it simple.

Common Mistakes

1. Applying OCP preemptively to everything. Not every conditional needs a strategy pattern. Apply OCP where you see repeated change.
2. Modifying the interface when adding a new strategy. If adding a new payment type requires changing the `PaymentStrategy` interface, you have not achieved OCP.
3. Forgetting the registry/factory. The strategies need to be wired together somewhere. Without a registry or factory, the caller still needs to know about all the concrete types.
4. Using string type checks instead of polymorphism. If you see `instanceof` checks or string comparisons to decide behavior, you have an OCP violation.
5. Over-abstracting with too many layers. One interface with concrete implementations is usually enough. Do not add abstract factory, builder, and decorator on top unless you genuinely need them.

Best Practices

1. Identify the axis of change. What is the thing that keeps changing? Payment types? Export formats? Notification channels? That is where to apply OCP.

2. Define a clean interface for the variation point. The interface should represent the caller's needs, not the implementation details.
3. Use the Strategy pattern for single-method behaviors. If the variation is one operation, Strategy is the right pattern.
4. Use a registry for runtime strategy selection. Map type identifiers to strategy instances at configuration time.
5. Apply the Rule of Three. Wait until you have seen the pattern repeat before investing in OCP infrastructure.
6. Keep strategies independent. Each strategy should be fully self-contained. Strategies should not know about or depend on each other.

Quick Summary

```
+-----------------------------------------------------------+
|              OCP CHEAT SHEET                               |
+-----------------------------------------------------------+
|                                                            |
|  PRINCIPLE:                                                |
|    Open for extension, closed for modification             |
|    Add new behavior by adding new code, not changing       |
|    existing code                                           |
|                                                            |
|  KEY PATTERN:                                              |
|    Strategy pattern: define an interface, implement it      |
|    once per variant, let the processor delegate             |
|                                                            |
|  SIGNS YOU NEED OCP:                                       |
|    - if/else or switch on a type that keeps growing         |
|    - Same type check scattered across multiple classes      |
|    - Adding a new type requires changes to many files       |
|                                                            |
|  SIGNS OCP IS OVERKILL:                                    |
|    - Only 2 branches, unlikely to grow                     |
|    - Type check in only one place                          |
|    - Speculating about future requirements                 |
|                                                            |
+-----------------------------------------------------------+
```

Key Points

- The Open-Closed Principle says code should be open for extension (add new behavior) but closed for modification (do not change existing code)
- If/else chains that check types and grow with each new variant are the classic OCP violation
- The Strategy pattern solves this by defining an interface and creating one implementation per variant
- Polymorphism replaces conditionals: instead of checking what type something is, let each type implement its own behavior
- A registry pattern enables runtime selection of strategies without modifying the selection logic
- Python's duck typing and Protocol make OCP natural without heavy interface hierarchies
- Apply the Rule of Three: wait until an if/else chain has been modified three times before refactoring to OCP

Practice Questions

1. Explain the Open-Closed Principle in your own words. What does "open for extension" mean, and what does "closed for modification" mean?
2. Why is an if/else chain that checks types considered an OCP violation? What specific problems does it cause as the codebase grows?
3. How does the Strategy pattern enable OCP? Draw or describe the relationship between the context class, the strategy interface, and the concrete strategies.
4. What is the Rule of Three, and how does it help you decide when to apply OCP?
5. How does Python's duck typing relate to OCP? Can you achieve OCP in Python without defining an abstract base class?

Exercises

Exercise 1: Refactor to Strategy Pattern

Refactor the following shipping cost calculator from a conditional chain to a Strategy pattern in both Java and Python:

```java
public double calculateShippingCost(String method, double weight) {
    if (method.equals("STANDARD")) {
        return weight * 1.50;
    } else if (method.equals("EXPRESS")) {
        return weight * 3.00 + 5.00;
    } else if (method.equals("OVERNIGHT")) {
        return weight * 5.00 + 15.00;
    } else if (method.equals("INTERNATIONAL")) {
        return weight * 8.00 + 25.00;
    }
    throw new IllegalArgumentException("Unknown method: " + method);
}
```

Exercise 2: Build a Plugin Registry

Create a file export system that supports multiple formats (CSV, JSON, XML). Design it so that adding a new format (such as YAML) requires only creating a new class and registering it. Include a registry that maps format names to exporter instances.

Exercise 3: Identify OCP Violations

Look at a codebase you work with regularly. Find three if/else or switch statements that check a type or category to decide behavior. For each one, assess whether it would benefit from an OCP refactoring or whether it is simple enough to leave as is. Justify your decision.

What Is Next?

The Open-Closed Principle taught you how to extend behavior without modifying existing code. The next chapter covers the Liskov Substitution Principle, which ensures that your extensions actually work correctly. It answers a critical question: when you create a subclass, can it truly replace the parent class everywhere without breaking anything? You will discover why a Square is not always a valid Rectangle, and why a Penguin that throws an exception when asked to fly violates a fundamental design rule.

Chapter 10: The Liskov Substitution Principle -- Subtypes Must Keep Their Promises

What You Will Learn

- What the Liskov Substitution Principle (LSP) means and why it is essential for reliable inheritance
- The Rectangle/Square problem explained step by step, showing how intuitive inheritance can be wrong
- Behavioral subtyping: what promises a subtype must keep
- Pre-conditions and post-conditions and how they constrain subtype behavior
- How to fix broken inheritance hierarchies using composition
- The Bird/Penguin fly() problem as a practical before-and-after example
- How duck typing in Python relates to LSP
- How to detect LSP violations in your own code

Why This Chapter Matters

Inheritance is one of the most powerful tools in object-oriented programming, and also one of the most misused. Developers create subclasses based on real-world "is-a" relationships (a Square is a Rectangle, a Penguin is a Bird) without checking whether the subclass actually behaves correctly in every place the parent class is used.

When a subclass violates the behavioral contract of its parent, every piece of code that works with the parent type becomes unreliable. Tests that pass with the parent mysteriously fail with

the subclass. Functions that work correctly for years suddenly break when someone passes a new subtype. These are some of the hardest bugs to track down because the code looks correct at the local level.

The Liskov Substitution Principle gives you a precise rule for checking whether your inheritance is safe.

The Liskov Substitution Principle Defined

Barbara Liskov defined this principle in 1987:

> If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program.

In simpler terms: if your code works with a parent type, it must also work correctly with any subtype of that parent, without the code knowing or caring which subtype it received.

```
+-------------------------------------------------------------+
|           THE LISKOV SUBSTITUTION PRINCIPLE                 |
+-------------------------------------------------------------+
|                                                             |
|  function doSomething(T parent) {                           |
|      parent.method();   // Works correctly with T           |
|  }                                                          |
|                                                             |
|  If S extends T, then:                                      |
|    doSomething(new S())  // MUST also work correctly        |
|                                                             |
|  "Correctly" means:                                         |
|    - Same behavior the caller expects                       |
|    - No surprising exceptions                               |
|    - No violated invariants                                 |
|    - No broken post-conditions                              |
|                                                             |
+-------------------------------------------------------------+
```

The Rectangle/Square Problem: Step by Step

This is the classic example. Mathematically, a square is a rectangle. So it seems natural to make `Square` a subclass of `Rectangle`. But this innocent-looking hierarchy is a trap.

Step 1: Define Rectangle

```java
// Java -- A simple Rectangle class
public class Rectangle {

    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getWidth() {
        return width;
    }

    public int getHeight() {
        return height;
    }

    public int getArea() {
        return width * height;
    }
}
```

Step 2: Make Square Extend Rectangle

A square is a rectangle where width equals height. So we override the setters to enforce this constraint:

```java
// Java -- Square as a subtype of Rectangle
public class Square extends Rectangle {

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;  // Keep width == height
    }

    @Override
    public void setHeight(int height) {
        this.width = height;  // Keep width == height
        this.height = height;
    }
}
```

Step 3: Write Code That Uses Rectangle

```java
// Java -- This function works with any Rectangle
public void resize(Rectangle rect) {
    rect.setWidth(5);
    rect.setHeight(10);

    // A reasonable expectation for a Rectangle:
    assert rect.getArea() == 50;  // 5 * 10 = 50
}
```

Step 4: Watch It Break

```java
Rectangle rect = new Rectangle();
resize(rect);  // area = 50. Correct.

Rectangle square = new Square();
resize(square);  // area = 100! setHeight(10) also set width to 10.
                 // The assertion fails.
```

```
+-------------------------------------------------------------+
|     THE RECTANGLE/SQUARE PROBLEM                            |
+-------------------------------------------------------------+
|                                                             |
|  Rectangle:                                                 |
|  +----------+                                               |
|  |          |  setWidth(5)  -> width=5, height unchanged   |
|  |   5x10   |  setHeight(10) -> height=10, width unchanged|
|  |          |  area = 5 * 10 = 50  (correct)              |
|  +----------+                                               |
|                                                             |
|  Square (pretending to be Rectangle):                       |
|  +----------+                                               |
|  |          |  setWidth(5)  -> width=5, height=5           |
|  |  10x10   |  setHeight(10) -> width=10, height=10       |
|  |          |  area = 10 * 10 = 100  (WRONG!)             |
|  +----------+                                               |
|                                                             |
|  The caller expected setWidth and setHeight to be           |
|  independent. Square violates that expectation.             |
|                                                             |
+-------------------------------------------------------------+
```

The problem is not mathematical. Mathematically, a square is a rectangle. The problem is behavioral. `Rectangle` has an implicit contract: `setWidth` changes only the width, and `setHeight` changes only the height. `Square` violates this contract.

The Fix: Separate Types

```java
// Java -- GOOD: Rectangle and Square are independent shapes
public interface Shape {
    int getArea();
}

public class Rectangle implements Shape {
    private final int width;
    private final int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

public class Square implements Shape {
    private final int side;

    public Square(int side) {
        this.side = side;
    }

    public int getArea() {
        return side * side;
    }
}
```

Now there is no inheritance relationship. Both implement `Shape`, and neither violates any behavioral contract.

The Bird/Penguin Problem: Before and After

Here is a more practical example. You have a `Bird` class with a `fly()` method. Then you add `Penguin`, which is a bird -- but penguins cannot fly.

BEFORE: Broken Inheritance

```java
// Java -- BAD: Penguin violates Bird's behavioral contract
public class Bird {

    private String name;

    public Bird(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void fly() {
        System.out.println(name + " is flying!");
    }

    public void eat() {
        System.out.println(name + " is eating.");
    }
}

public class Penguin extends Bird {

    public Penguin(String name) {
        super(name);
    }

    @Override
    public void fly() {
        throw new UnsupportedOperationException(
            "Penguins cannot fly!"
        );
    }
}
```

```
+-------------------------------------------------------------+
|     THE BIRD/PENGUIN PROBLEM                                |
+-------------------------------------------------------------+
|                                                             |
|         Bird                                                |
|        /    \                                               |
|    Eagle    Penguin                                         |
|                                                             |
|  Code that uses Bird:                                       |
|                                                             |
|  void migrateFlock(List<Bird> birds) {                      |
|      for (Bird bird : birds) {                              |
|          bird.fly();  // BOOM! Penguin throws exception     |
|      }                                                      |
|  }                                                          |
|                                                             |
|  The caller has every right to expect that Bird.fly()       |
```

```
|  works. Penguin breaks that expectation.                  |
|                                                           |
+-----------------------------------------------------------+
```

The Problem in Python

```python
# Python -- BAD: Same LSP violation
class Bird:

    def __init__(self, name: str):
        self.name = name

    def fly(self) -> None:
        print(f"{self.name} is flying!")

    def eat(self) -> None:
        print(f"{self.name} is eating.")


class Penguin(Bird):

    def fly(self) -> None:
        raise NotImplementedError("Penguins cannot fly!")


# This function expects all birds to fly
def migrate_flock(birds: list[Bird]) -> None:
    for bird in birds:
        bird.fly()  # Crashes on Penguin
```

AFTER: Fixed with Composition and Proper Abstractions

The fix is to recognize that not all birds can fly. Separate the concept of "bird" from the ability "can fly."

```java
// Java -- GOOD: Separate the ability to fly from being a bird

public abstract class Bird {

    private final String name;

    protected Bird(String name) {
        this.name = name;
    }

    public String getName() {
```

```
        return name;
    }

    public abstract void eat();
}

// Interface for birds that can fly
public interface Flyable {
    void fly();
}

// Interface for birds that can swim
public interface Swimmable {
    void swim();
}

public class Eagle extends Bird implements Flyable {

    public Eagle(String name) {
        super(name);
    }

    @Override
    public void eat() {
        System.out.println(getName() + " hunts fish.");
    }

    @Override
    public void fly() {
        System.out.println(getName() + " soars through the sky!");
    }
}

public class Penguin extends Bird implements Swimmable {

    public Penguin(String name) {
        super(name);
    }

    @Override
    public void eat() {
        System.out.println(getName() + " eats fish from the sea.");
    }

    @Override
    public void swim() {
        System.out.println(getName() + " swims gracefully!");
    }
}

public class Duck extends Bird implements Flyable, Swimmable {

    public Duck(String name) {
        super(name);
    }
```

```
    @Override
    public void eat() {
        System.out.println(getName() + " eats bread.");
    }

    @Override
    public void fly() {
        System.out.println(getName() + " flies over the pond.");
    }

    @Override
    public void swim() {
        System.out.println(getName() + " paddles on the lake.");
    }
}
```

```
+-------------------------------------------------------------+
|     FIXED BIRD HIERARCHY                                     |
+-------------------------------------------------------------+
|                                                              |
|           Bird (abstract)                                    |
|          /      |      \                                     |
|      Eagle    Duck    Penguin                                |
|        |       |  |      |                                   |
|     Flyable  Flyable Swimmable                               |
|             Swimmable                                        |
|                                                              |
|  void migrateFlock(List<Flyable> flyers) {                   |
|      for (Flyable f : flyers) {                              |
|          f.fly();  // SAFE: only Flyable birds are here      |
|      }                                                       |
|  }                                                           |
|                                                              |
|  Eagle and Duck go in. Penguin does not.                     |
|  No exceptions. No surprises.                                |
|                                                              |
+-------------------------------------------------------------+
```

Now the `migrateFlock` function takes `List<Flyable>` instead of `List<Bird>`. Penguins simply are not passed to it. The type system prevents the mistake.

Python Version: Fixed

```python
# Python -- GOOD: Composition-based design
from abc import ABC, abstractmethod
from typing import Protocol
```

```
class Bird(ABC):

    def __init__(self, name: str):
        self.name = name

    @abstractmethod
    def eat(self) -> None:
        pass


class Flyable(Protocol):
    name: str
    def fly(self) -> None: ...


class Swimmable(Protocol):
    name: str
    def swim(self) -> None: ...


class Eagle(Bird):

    def eat(self) -> None:
        print(f"{self.name} hunts fish.")

    def fly(self) -> None:
        print(f"{self.name} soars through the sky!")


class Penguin(Bird):

    def eat(self) -> None:
        print(f"{self.name} eats fish from the sea.")

    def swim(self) -> None:
        print(f"{self.name} swims gracefully!")


class Duck(Bird):

    def eat(self) -> None:
        print(f"{self.name} eats bread.")

    def fly(self) -> None:
        print(f"{self.name} flies over the pond.")

    def swim(self) -> None:
        print(f"{self.name} paddles on the lake.")


def migrate_flock(flyers: list[Flyable]) -> None:
    for flyer in flyers:
        flyer.fly()  # Safe: only objects with fly() are accepted
```

Pre-Conditions and Post-Conditions

The Liskov Substitution Principle can be stated precisely in terms of conditions:

- Pre-conditions: What must be true before a method is called
- Post-conditions: What is guaranteed to be true after a method returns
- Invariants: What is always true about the object's state

The rule is:

```
+-------------------------------------------------------------+
|     LSP CONDITIONS RULE                                     |
+-------------------------------------------------------------+
|                                                             |
|  A subtype may:                                             |
|    - WEAKEN pre-conditions (accept more inputs)             |
|    - STRENGTHEN post-conditions (guarantee more outputs)    |
|                                                             |
|  A subtype must NOT:                                        |
|    - STRENGTHEN pre-conditions (reject valid inputs)        |
|    - WEAKEN post-conditions (guarantee less)                |
|                                                             |
+-------------------------------------------------------------+
```

Example: Weakening Pre-Conditions Is Safe

```java
// Java -- Base class: accepts positive integers only
public class Validator {
    public boolean validate(int value) {
        if (value <= 0) {
            throw new IllegalArgumentException("Must be positive");
        }
        return value < 1000;
    }
}

// Subclass: accepts all integers (WEAKER pre-condition = OK)
public class LenientValidator extends Validator {
    @Override
    public boolean validate(int value) {
        // Accepts zero and negative values too -- this is safe
        // because any code that passed positive values still works
        return Math.abs(value) < 1000;
    }
}
```

Example: Strengthening Pre-Conditions Breaks LSP

```java
// Java -- BAD: Subclass rejects inputs the parent accepts
public class StrictValidator extends Validator {
    @Override
    public boolean validate(int value) {
        if (value <= 0 || value > 500) {  // STRICTER than parent!
            throw new IllegalArgumentException("Must be between 1 and 500");
        }
        return true;
    }
}

// Code that works with Validator (values up to 999) breaks with StrictValidator
Validator v = new StrictValidator();
v.validate(750);  // BOOM! Parent would accept this, subclass rejects it
```

LSP Violations in the Wild

Here are common real-world LSP violations to watch for:

1. Throwing Exceptions the Parent Does Not

```python
# Python -- BAD: Subclass throws where parent does not
class FileStorage:
    def save(self, key: str, data: bytes) -> None:
        with open(f"/data/{key}", "wb") as f:
            f.write(data)


class ReadOnlyStorage(FileStorage):
    def save(self, key: str, data: bytes) -> None:
        raise PermissionError("Storage is read-only!")  # LSP violation
```

2. Ignoring Method Parameters

```java
// Java -- BAD: Subclass ignores parameters
public class Cache {
```

```
    public void put(String key, Object value, Duration ttl) {
        // Stores with the given TTL
        store.put(key, value, ttl);
    }
}

public class PermanentCache extends Cache {
    @Override
    public void put(String key, Object value, Duration ttl) {
        // Ignores TTL entirely -- values never expire
        // Callers who rely on TTL will be surprised
        store.put(key, value, Duration.ofMillis(Long.MAX_VALUE));
    }
}
```

3. Returning Different Types

```python
# Python -- BAD: Subclass returns different type
class UserRepository:
    def find_by_id(self, user_id: int) -> User:
        return self.db.query(User, user_id)


class CachingUserRepository(UserRepository):
    def find_by_id(self, user_id: int) -> dict:  # Returns dict, not User!
        cached = self.cache.get(user_id)
        if cached:
            return cached  # This is a dict, not a User object
        return super().find_by_id(user_id)
```

Duck Typing and LSP in Python

Python's duck typing means LSP is about behavior, not formal inheritance. Two classes that do not share a parent can still satisfy LSP if they implement the same behavioral contract.

```python
# Python -- Duck typing: LSP is about behavior, not hierarchy

class InMemoryStore:
    def __init__(self):
        self._data = {}

    def get(self, key: str) -> str | None:
```

```
        return self._data.get(key)

    def put(self, key: str, value: str) -> None:
        self._data[key] = value


class RedisStore:
    def __init__(self, client):
        self._client = client

    def get(self, key: str) -> str | None:
        return self._client.get(key)

    def put(self, key: str, value: str) -> None:
        self._client.set(key, value)


# Both work interchangeably -- LSP is satisfied through duck typing
def cache_user_profile(store, user_id: str, profile: str) -> None:
    store.put(f"user:{user_id}", profile)

def get_user_profile(store, user_id: str) -> str | None:
    return store.get(f"user:{user_id}")
```

Neither class inherits from the other, and neither inherits from a common base class. But both satisfy the same behavioral contract (get and put with the same signatures and semantics), so they are interchangeable. This is LSP through duck typing.

How to Detect LSP Violations

Here is a checklist for spotting LSP problems:

```
+-------------------------------------------------------------+
|     LSP VIOLATION DETECTOR                                  |
+-------------------------------------------------------------+
|                                                             |
|  Check for these red flags:                                 |
|                                                             |
|  [ ] Does the subclass throw exceptions the parent         |
|      does not declare?                                      |
|                                                             |
|  [ ] Does the subclass ignore parameters that the          |
|      parent uses?                                           |
|                                                             |
|  [ ] Does the subclass return a different type than         |
|      the parent?                                            |
```

```
|                                                            |
|  [ ] Does the subclass have a no-op implementation         |
|      of a parent method?                                   |
|                                                            |
|  [ ] Does code using the parent type need isinstance       |
|      checks to work with the subclass?                     |
|                                                            |
|  [ ] Does the subclass violate the parent's invariants?    |
|      (e.g., Square breaking Rectangle's independent        |
|       width/height)                                        |
|                                                            |
|  If you checked any box, you likely have an LSP violation. |
|                                                            |
+------------------------------------------------------------+
```

The instanceof smell: If you ever find yourself writing `if (obj instanceof SpecificSubclass)` to handle a subtype differently, that subtype is not properly substitutable.

```java
// Java -- BAD: instanceof checks indicate LSP violations
public void feedAnimal(Animal animal) {
    if (animal instanceof Snake) {
        // Snakes eat differently -- this is an LSP smell
        ((Snake) animal).swallowWhole(food);
    } else {
        animal.eat(food);
    }
}
```

Common Mistakes

1. Inheriting based on "is-a" in the real world. A square "is a" rectangle mathematically, but it is not a valid subtype behaviorally. Design based on behavioral compatibility, not real-world taxonomy.
2. Empty or no-op overrides. Overriding a method to do nothing is almost always an LSP violation. If the subtype cannot fulfill the method's contract, it should not inherit it.
3. Throwing UnsupportedOperationException. This is the Penguin.fly() anti-pattern. If a subtype cannot support an operation, it should not inherit the interface that declares it.
4. Adding pre-conditions in the subtype. If the parent accepts any positive number, the subtype cannot suddenly require numbers under 100.
5. Confusing code reuse with subtyping. Do not use inheritance just to reuse some methods from a parent class. If the subtype is not fully substitutable, use composition instead.

Best Practices

1. Design by contract. Define clear pre-conditions, post-conditions, and invariants for your base types. Verify that every subtype honors them.
2. Prefer composition over inheritance. If you are unsure whether a subtype relationship is valid, use composition. It is always safe.
3. Use interfaces to define capabilities. Instead of a deep class hierarchy, use small interfaces (Flyable, Swimmable) that describe what an object can do.
4. Write tests against the base type. If your tests pass with the parent class but fail with a subclass, you have an LSP violation. Every subclass should pass every test written for the parent.
5. Avoid empty overrides. If a subclass method does nothing, refactor the hierarchy so the method does not exist in that branch.
6. Use the Liskov test. For every subclass, ask: "Can I use this subclass everywhere the parent is used without any caller knowing or caring?" If the answer is no, refactor.

Quick Summary

```
+-------------------------------------------------------------+
|              LSP CHEAT SHEET                                |
+-------------------------------------------------------------+
|                                                             |
|  PRINCIPLE:                                                 |
|    Subtypes must be substitutable for their base types      |
|    without altering program correctness                     |
|                                                             |
|  KEY RULE:                                                  |
|    - Subtypes may WEAKEN pre-conditions                     |
|    - Subtypes may STRENGTHEN post-conditions                |
|    - Subtypes must NOT do the reverse                       |
|                                                             |
|  RED FLAGS:                                                 |
|    - UnsupportedOperationException in a subclass            |
|    - instanceof checks in code using the base type          |
|    - No-op method overrides                                 |
|    - Subclass ignoring parent method parameters             |
|                                                             |
|  FIXES:                                                     |
```

```
|    - Use composition instead of inheritance                |
|    - Use interfaces for capabilities (Flyable, Swimmable)  |
|    - Make separate types instead of forcing hierarchy      |
|                                                            |
+------------------------------------------------------------+
```

Key Points

- The Liskov Substitution Principle requires that any subtype can replace its parent type without breaking the program
- The Rectangle/Square problem shows that real-world "is-a" relationships do not always translate into valid inheritance hierarchies
- A subtype violates LSP when it throws unexpected exceptions, ignores parameters, returns different types, or has no-op implementations of parent methods
- Pre-conditions can be weakened (accept more) but not strengthened (reject more) in subtypes; post-conditions can be strengthened but not weakened
- The fix for LSP violations is usually composition over inheritance: separate capabilities into interfaces and let each type implement only what it genuinely supports
- Python's duck typing means LSP is about behavioral compatibility, not formal class hierarchy
- The instanceof check is a code smell that often indicates an LSP violation

---

Practice Questions

1. Explain the Liskov Substitution Principle in your own words. Why is it important for code that works with polymorphism?
2. Walk through the Rectangle/Square problem step by step. What implicit contract does Rectangle have that Square violates?
3. What is the difference between strengthening and weakening pre-conditions? Which is safe for a subtype to do, and which violates LSP?
4. Why is throwing UnsupportedOperationException in a subclass method an LSP violation? What should you do instead?
5. How does Python's duck typing relate to LSP? Can you have an LSP violation in Python even without formal inheritance?

Exercises

Exercise 1: Fix the Bird Hierarchy

The following code has an LSP violation. Refactor it using interfaces and composition so that all types are properly substitutable.

```python
class Vehicle:
    def start_engine(self):
        print("Engine started")

    def accelerate(self):
        print("Accelerating")

    def refuel(self):
        print("Refueling with gasoline")

class ElectricCar(Vehicle):
    def refuel(self):
        raise NotImplementedError("Electric cars don't use fuel!")

    def recharge(self):
        print("Charging battery")
```

Exercise 2: Write Substitutability Tests

Write a set of tests that verify LSP for a `Stack` class. The tests should pass for any valid implementation of `Stack` (array-based, linked-list-based, etc.). Then create a `BoundedStack` subclass with a maximum capacity and check whether it passes all the original tests. If it does not, explain the violation and fix it.

Exercise 3: Detect LSP Violations in Real Code

Find an open-source Java or Python project and look for classes that use inheritance. Identify at least two potential LSP violations using the checklist from this chapter. Describe what the violation is and how you would fix it.

What Is Next?

The Liskov Substitution Principle ensures your subtypes can safely replace their parents. The next chapter covers the Interface Segregation Principle, which addresses a different problem: what happens when an interface grows too large and forces its implementors to provide methods they do not need. You will learn how to split fat interfaces into focused, role-based interfaces that keep your code flexible and your implementations clean.

Chapter 11: The Interface Segregation Principle -- Keep Interfaces Focused

What You Will Learn

- What the Interface Segregation Principle (ISP) means and why fat interfaces cause real problems
- The "forced dependency" problem: when classes must implement methods they do not need
- How to split a fat interface into focused, role-based interfaces
- The IWorker problem: before and after refactoring with work(), eat(), and sleep()
- How Python ABCs and Protocols support ISP
- When a single large interface is actually fine and splitting would be over-engineering
- How ISP applies beyond classes to REST APIs and module design

Why This Chapter Matters

Imagine you are building a robot class for a factory simulation. You inherit from an `IWorker` interface that requires you to implement `work()`, `eat()`, and `sleep()`. Your robot works, but it does not eat or sleep. You are forced to write empty method bodies or throw exceptions for methods that have nothing to do with your class.

This is not a minor annoyance. It creates three serious problems:

1. Misleading API: Anyone looking at the robot class sees `eat()` and `sleep()` and assumes the robot supports those operations.

2. Coupling: If someone adds `dream()` to the `IWorker` interface, your robot class must be updated even though dreaming is irrelevant to robots.
3. Testing burden: You must write tests for methods that do nothing, or worse, document why they throw exceptions.

The Interface Segregation Principle eliminates these problems by ensuring no class is forced to depend on methods it does not use.

The Interface Segregation Principle Defined

Robert C. Martin stated the principle as:

> No client should be forced to depend on methods it does not use.

In practical terms: prefer many small, focused interfaces over one large, general-purpose interface. Each interface should represent a single role or capability.

```
+-------------------------------------------------------------+
|     THE INTERFACE SEGREGATION PRINCIPLE                      |
+-------------------------------------------------------------+
|                                                              |
|  FAT INTERFACE (BAD):                                        |
|  +------------------------------+                            |
|  |         IWorker              |                            |
|  |------------------------------|                            |
|  |  + work()                    |                            |
|  |  + eat()                     |  <-- Robot must            |
|  |  + sleep()                   |      implement all         |
|  |  + takeBreak()               |      even if irrelevant    |
|  |  + getPaycheck()             |                            |
|  +------------------------------+                            |
|                                                              |
|  SEGREGATED INTERFACES (GOOD):                               |
|  +-----------+  +-----------+  +-----------+                |
|  | IWorkable |  | IFeedable |  | ISleepable|                |
|  |-----------|  |-----------|  |-----------|                |
|  | + work()  |  | + eat()   |  | + sleep() |                |
|  +-----------+  +-----------+  +-----------+                |
|       ^              ^              ^                        |
|       |              |              |                        |
|    Robot         Human          Human                        |
|  (work only)   (all three)    (all three)                    |
|                                                              |
+-------------------------------------------------------------+
```

The IWorker Problem: Before and After

BEFORE: Fat Interface Forces Unwanted Dependencies (Java)

```java
// Java -- BAD: Fat interface that mixes unrelated responsibilities
public interface IWorker {

    void work();

    void eat();

    void sleep();
}

public class HumanWorker implements IWorker {

    @Override
    public void work() {
        System.out.println("Human working on tasks.");
    }

    @Override
    public void eat() {
        System.out.println("Human eating lunch.");
    }

    @Override
    public void sleep() {
        System.out.println("Human sleeping for 8 hours.");
    }
}

public class RobotWorker implements IWorker {

    @Override
    public void work() {
        System.out.println("Robot performing tasks efficiently.");
    }

    @Override
    public void eat() {
        // Robot does not eat. What do we put here?
        throw new UnsupportedOperationException("Robots don't eat!");
    }

    @Override
    public void sleep() {
        // Robot does not sleep. Same problem.
        throw new UnsupportedOperationException("Robots don't sleep!");
    }
}
```

This is also an LSP violation. If code iterates over a list of `IWorker` instances and calls `eat()`, the robot will throw an exception. The interface promises something the implementation cannot deliver.

AFTER: Segregated Interfaces (Java)

```java
// Java -- GOOD: Small, focused interfaces

public interface Workable {
    void work();
}

public interface Feedable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class HumanWorker implements Workable, Feedable, Sleepable {

    @Override
    public void work() {
        System.out.println("Human working on tasks.");
    }

    @Override
    public void eat() {
        System.out.println("Human eating lunch.");
    }

    @Override
    public void sleep() {
        System.out.println("Human sleeping for 8 hours.");
    }
}

public class RobotWorker implements Workable {

    @Override
    public void work() {
        System.out.println("Robot performing tasks efficiently.");
    }

    // No eat(). No sleep(). Clean and honest.
}
```

Now the code that manages work schedules takes `Workable`:

```java
// Java -- Clean: each function depends only on what it needs
public class WorkScheduler {

    public void assignTasks(List<Workable> workers) {
        for (Workable worker : workers) {
            worker.work();  // Safe for both humans and robots
        }
    }
}

public class CafeteriaManager {

    public void serveLunch(List<Feedable> hungryWorkers) {
        for (Feedable worker : hungryWorkers) {
            worker.eat();  // Only gets workers that actually eat
        }
    }
}
```

BEFORE: Fat Interface in Python

```python
# Python -- BAD: Fat interface
from abc import ABC, abstractmethod


class IWorker(ABC):

    @abstractmethod
    def work(self) -> None:
        pass

    @abstractmethod
    def eat(self) -> None:
        pass

    @abstractmethod
    def sleep(self) -> None:
        pass


class HumanWorker(IWorker):

    def work(self) -> None:
        print("Human working on tasks.")

    def eat(self) -> None:
        print("Human eating lunch.")

    def sleep(self) -> None:
        print("Human sleeping for 8 hours.")
```

```
class RobotWorker(IWorker):

    def work(self) -> None:
        print("Robot performing tasks efficiently.")

    def eat(self) -> None:
        pass  # Empty implementation -- dishonest

    def sleep(self) -> None:
        pass  # Empty implementation -- dishonest
```

AFTER: Segregated Interfaces in Python

```python
# Python -- GOOD: Segregated ABCs
from abc import ABC, abstractmethod


class Workable(ABC):
    @abstractmethod
    def work(self) -> None:
        pass


class Feedable(ABC):
    @abstractmethod
    def eat(self) -> None:
        pass


class Sleepable(ABC):
    @abstractmethod
    def sleep(self) -> None:
        pass


class HumanWorker(Workable, Feedable, Sleepable):

    def work(self) -> None:
        print("Human working on tasks.")

    def eat(self) -> None:
        print("Human eating lunch.")

    def sleep(self) -> None:
        print("Human sleeping for 8 hours.")


class RobotWorker(Workable):
```

```
    def work(self) -> None:
        print("Robot performing tasks efficiently.")
```

Python ABCs vs. Protocols for ISP

Python gives you two tools for defining interfaces: Abstract Base Classes (ABCs) and Protocols. Both work well for ISP, but they serve different purposes.

ABCs: Explicit, Enforced at Instantiation

```python
# Python -- ABCs enforce implementation at instantiation time
from abc import ABC, abstractmethod


class Printable(ABC):
    @abstractmethod
    def to_string(self) -> str:
        pass


class Invoice(Printable):
    def to_string(self) -> str:
        return "Invoice #12345"


class BadInvoice(Printable):
    pass  # TypeError raised when you try to instantiate this
```

Protocols: Structural, No Inheritance Required

```python
# Python -- Protocols use structural typing (duck typing with type hints)
from typing import Protocol


class Printable(Protocol):
    def to_string(self) -> str: ...


class Invoice:
    """Does NOT inherit from Printable, but satisfies its contract."""
```

```
    def to_string(self) -> str:
        return "Invoice #12345"


def print_document(doc: Printable) -> None:
    print(doc.to_string())


# This works because Invoice has a to_string() method
print_document(Invoice())  # No inheritance needed
```

When to use which:

```
+-------------------------------------------------------------+
|     ABCs vs PROTOCOLS FOR ISP                               |
+-------------------------------------------------------------+
|                                                             |
|  Use ABCs when:                                             |
|    - You want instantiation-time enforcement                |
|    - You have shared default implementations                |
|    - Your team prefers explicit "implements" relationships  |
|                                                             |
|  Use Protocols when:                                        |
|    - You want duck typing with type checker support         |
|    - You are working with third-party classes you cannot    |
|      modify                                                 |
|    - You prefer minimal coupling (no inheritance needed)    |
|                                                             |
+-------------------------------------------------------------+
```

Role Interfaces

The segregated interfaces in the IWorker example are called role interfaces. Each interface represents a specific role an object can play, rather than defining everything the object is.

```
+-------------------------------------------------------------+
|     ROLE INTERFACES                                         |
+-------------------------------------------------------------+
|                                                             |
|  Instead of one big "I am everything" interface:            |
|                                                             |
|    IEmployee: work(), eat(), sleep(), getPaycheck(),        |
|               requestVacation(), attendMeeting(),           |
|               submitTimesheet()                             |
```

```
|                                                            |
|  Define focused roles:                                     |
|                                                            |
|    Workable:     work()                                    |
|    Feedable:     eat()                                     |
|    Payable:      getPaycheck()                             |
|    Schedulable:  attendMeeting(), requestVacation()        |
|    Trackable:    submitTimesheet()                         |
|                                                            |
|  Each client depends only on the roles it cares about.     |
|                                                            |
+------------------------------------------------------------+
```

A more realistic example:

```java
// Java -- Role interfaces for a document management system

public interface Readable {
    String getContent();
}

public interface Writable {
    void setContent(String content);
}

public interface Printable {
    void print();
}

public interface Versionable {
    int getVersion();
    void incrementVersion();
    List<String> getHistory();
}

// A full document supports all roles
public class Document implements Readable, Writable, Printable, Versionable {
    // Implements all four interfaces
}

// A read-only view supports only reading
public class ReadOnlyDocument implements Readable {
    // Only implements getContent()
}

// A printer service only needs Printable
public class PrintService {
    public void printAll(List<Printable> documents) {
        for (Printable doc : documents) {
            doc.print();
        }
    }
```

```
}

// A version control service only needs Versionable
public class VersionControl {
    public void commit(Versionable document) {
        document.incrementVersion();
    }
}
```

When One Big Interface Is Actually Fine

ISP does not mean you should always split every interface into single-method fragments. Sometimes a larger interface is the right choice:

1. Cohesive Operations That Always Go Together

```java
// Java -- This is fine as one interface
public interface Iterator<T> {
    boolean hasNext();
    T next();
}

// These two methods are inseparable.
// You will never need hasNext() without next().
// Splitting them would be pointless.
```

2. Standard Library Interfaces

The `List` interface in Java has dozens of methods (`add`, `remove`, `get`, `size`, `contains`, etc.). This is fine because all implementations genuinely support all operations. If they do not (like `Collections.unmodifiableList()`), that is an LSP problem, not an ISP problem.

3. Small Teams with Simple Domains

If you have three classes that all implement the exact same five methods, and you do not anticipate adding a class that would only need some of them, keeping one five-method interface is simpler than creating five single-method interfaces.

The test: Would splitting the interface result in some implementors implementing fewer interfaces? If yes, split it. If every implementor would implement all the sub-interfaces anyway,

a single interface is fine.

ISP in REST APIs

ISP applies beyond class interfaces. In REST API design, the same principle says: do not force clients to receive data they do not need.

BEFORE: Fat API Response

```json
// BAD: Client only needs name and email, gets everything
GET /api/users/123

{
    "id": 123,
    "name": "Jane Smith",
    "email": "jane@example.com",
    "passwordHash": "bcrypt$...",
    "socialSecurityNumber": "123-45-6789",
    "internalNotes": "Flagged for review",
    "loginHistory": [...2000 entries...],
    "fullOrderHistory": [...500 entries...],
    "paymentMethods": [...],
    "adminFlags": {...}
}
```

AFTER: Focused API Responses

```json
// GOOD: Separate endpoints or field selection

// Public profile (what most clients need)
GET /api/users/123/profile
{
    "id": 123,
    "name": "Jane Smith",
    "email": "jane@example.com"
}

// Order history (only for order-related features)
GET /api/users/123/orders
{
    "orders": [...]
}
```

```
// Admin view (only for admin dashboards)
GET /api/admin/users/123
{
    "id": 123,
    "internalNotes": "...",
    "adminFlags": {...}
}
```

Or use field selection (GraphQL-style):

```
GET /api/users/123?fields=name,email
```

```
+-------------------------------------------------------------+
|     ISP IN API DESIGN                                       |
+-------------------------------------------------------------+
|                                                             |
|  FAT ENDPOINT:                                              |
|  GET /api/users/123                                         |
|  Returns: 50 fields, 3 nested objects, 2000-entry array    |
|  Every client gets everything, uses 3 fields                |
|                                                             |
|  SEGREGATED ENDPOINTS:                                      |
|  GET /api/users/123/profile     -> 5 fields                 |
|  GET /api/users/123/orders      -> order list only          |
|  GET /api/users/123/preferences -> preferences only         |
|                                                             |
|  Each client requests only what it needs.                   |
|  Less bandwidth. Less coupling. Less risk of leaking        |
|  sensitive data.                                            |
|                                                             |
+-------------------------------------------------------------+
```

A Larger Example: Printer Interface

BEFORE: Fat Printer Interface

```java
// Java -- BAD: Every printer must implement every feature
public interface IPrinter {
    void print(Document doc);
    void scan(Document doc);
```

```
    void fax(Document doc);
    void staple(Document doc);
    void collate(Document doc);
}

public class BasicPrinter implements IPrinter {

    @Override
    public void print(Document doc) {
        System.out.println("Printing document...");
    }

    @Override
    public void scan(Document doc) {
        throw new UnsupportedOperationException("Cannot scan");
    }

    @Override
    public void fax(Document doc) {
        throw new UnsupportedOperationException("Cannot fax");
    }

    @Override
    public void staple(Document doc) {
        throw new UnsupportedOperationException("Cannot staple");
    }

    @Override
    public void collate(Document doc) {
        throw new UnsupportedOperationException("Cannot collate");
    }
}
```

AFTER: Segregated Printer Interfaces

```java
// Java -- GOOD: Focused interfaces for printer capabilities
public interface Printer {
    void print(Document doc);
}

public interface Scanner {
    void scan(Document doc);
}

public interface Fax {
    void fax(Document doc);
}

public interface Finisher {
    void staple(Document doc);
```

```
    void collate(Document doc);
}

// Basic printer: only prints
public class BasicPrinter implements Printer {
    @Override
    public void print(Document doc) {
        System.out.println("Printing document...");
    }
}

// All-in-one: implements all capabilities
public class AllInOnePrinter implements Printer, Scanner, Fax, Finisher {

    @Override
    public void print(Document doc) {
        System.out.println("Printing document...");
    }

    @Override
    public void scan(Document doc) {
        System.out.println("Scanning document...");
    }

    @Override
    public void fax(Document doc) {
        System.out.println("Faxing document...");
    }

    @Override
    public void staple(Document doc) {
        System.out.println("Stapling document...");
    }

    @Override
    public void collate(Document doc) {
        System.out.println("Collating document...");
    }
}
```

```python
# Python -- GOOD: Segregated printer protocols
from typing import Protocol


class Printer(Protocol):
    def print_doc(self, doc: Document) -> None: ...


class Scanner(Protocol):
    def scan(self, doc: Document) -> None: ...
```

```
class Fax(Protocol):
    def fax(self, doc: Document) -> None: ...


class BasicPrinter:
    def print_doc(self, doc: Document) -> None:
        print("Printing document...")


class AllInOnePrinter:
    def print_doc(self, doc: Document) -> None:
        print("Printing document...")

    def scan(self, doc: Document) -> None:
        print("Scanning document...")

    def fax(self, doc: Document) -> None:
        print("Faxing document...")


def print_batch(printer: Printer, docs: list[Document]) -> None:
    for doc in docs:
        printer.print_doc(doc)  # Works with BasicPrinter or AllInOnePrinter
```

Common Mistakes

1. Creating one giant interface and making every class implement it. This is the core ISP violation. If any implementor has empty methods or throws UnsupportedOperationException, the interface is too fat.
2. Splitting interfaces too aggressively. Do not create a separate interface for every single method. Group methods that are genuinely cohesive and always used together.
3. Ignoring ISP in API design. Returning 50 fields when the client needs 3 is the API-level equivalent of a fat interface.
4. Using marker interfaces instead of capability interfaces. An empty `IAdvancedPrinter` interface that signals capabilities through its name rather than its methods is a code smell.
5. Confusing ISP with SRP. SRP is about classes having one reason to change. ISP is about interfaces not forcing clients to depend on methods they do not use. They are related but distinct.

Best Practices

1. Design interfaces from the client's perspective. Ask "What does this client need?" not "What can this object do?"
2. Use role interfaces. Each interface should represent one role or capability.
3. Apply the no-empty-methods test. If an implementor must leave a method body empty or throw an exception, the interface should be split.
4. Group cohesive operations. `hasNext()` and `next()` belong together. Do not split things that always go together.
5. Apply ISP to APIs. Use focused endpoints, field selection, or separate views for different client needs.
6. In Python, prefer Protocols for ISP. They provide structural typing without requiring inheritance, which is the cleanest way to implement ISP.

Quick Summary

```
+-------------------------------------------------------------+
|              ISP CHEAT SHEET                                |
+-------------------------------------------------------------+
|                                                             |
|  PRINCIPLE:                                                 |
|    No client should be forced to depend on methods          |
|    it does not use                                          |
|                                                             |
|  FAT INTERFACE SIGNS:                                       |
|    - Classes with empty method implementations              |
|    - UnsupportedOperationException in subclasses            |
|    - Clients using only 2 of 10 methods                     |
|                                                             |
|  SOLUTION:                                                  |
|    - Split into role interfaces (Workable, Feedable)        |
|    - Each client depends on only the interfaces it needs    |
|    - Classes implement only the interfaces they support     |
|                                                             |
|  TOOLS:                                                     |
|    - Java: interface keyword                                |
|    - Python: ABC (explicit) or Protocol (structural)        |
|                                                             |
|  ALSO APPLIES TO:                                           |
|    - REST APIs (focused endpoints over fat responses)       |
|    - Module exports (export only what clients need)         |
|    - Configuration objects (focused config over global)     |
|                                                             |
```

```
+-----------------------------------------------------------+
```

Key Points

- The Interface Segregation Principle says no class should be forced to implement methods it does not use
- Fat interfaces cause empty implementations, UnsupportedOperationException, misleading APIs, and unnecessary coupling
- Split fat interfaces into focused role interfaces where each interface represents a single capability
- Python offers ABCs (explicit enforcement) and Protocols (structural typing) as two tools for ISP
- Do not over-split: if two methods are always used together, they belong in the same interface
- ISP extends beyond classes to REST API design -- do not return unnecessary fields to clients
- The test for whether to split: would any implementor need only a subset of the methods? If yes, split

---

Practice Questions

1. Explain the Interface Segregation Principle in your own words. What problem does it solve?
2. What are the concrete problems caused by a fat interface? List at least three.
3. In the IWorker example, why is it wrong for RobotWorker to implement eat() as a no-op? What are the consequences?
4. What is the difference between Python ABCs and Protocols for implementing ISP? When would you choose one over the other?
5. Give an example of when a large interface is acceptable and should NOT be split. What criteria make a large interface cohesive?

Exercises

Exercise 1: Refactor a Fat Interface

The following interface is too fat. Split it into focused role interfaces and update the implementing classes.

```java
public interface ISmartDevice {
    void makeCall(String number);
    void sendSMS(String number, String message);
    void browseWeb(String url);
    void takePhoto();
    void playMusic(String track);
    void setAlarm(String time);
}

public class Smartphone implements ISmartDevice {
    // Implements everything
}

public class BasicPhone implements ISmartDevice {
    // Can only make calls and send SMS
    // Must throw exceptions for browseWeb, takePhoto, playMusic
}

public class SmartSpeaker implements ISmartDevice {
    // Can only play music and set alarms
    // Must throw exceptions for calls, SMS, web, photos
}
```

Exercise 2: Apply ISP to an API

You have a user API endpoint that returns the following response for every request. Design segregated endpoints or a field selection mechanism that follows ISP.

```json
{
    "id": 1,
    "username": "jsmith",
    "email": "j@example.com",
    "passwordHash": "...",
    "profile": { "bio": "...", "avatar": "..." },
    "orders": [{ ... }, { ... }],
    "paymentMethods": [{ ... }],
    "loginHistory": [{ ... }, { ... }],
    "preferences": { "theme": "dark", "language": "en" }
}
```

Exercise 3: ISP in Python with Protocols

Rewrite the printer example from this chapter using Python Protocols instead of ABCs. Create a function that works with any Printer, a function that works with any Scanner, and verify that BasicPrinter can be passed to the print function but not the scan function (use mypy or pyright for type checking).

What Is Next?

The Interface Segregation Principle taught you to keep interfaces focused so that clients only depend on what they actually use. The next chapter covers the final SOLID principle: the Dependency Inversion Principle. It addresses the direction of dependencies -- why high-level business logic should never depend directly on low-level infrastructure, and how inverting that dependency makes your code testable, flexible, and modular.

Chapter 12: The Dependency Inversion Principle -- Depend on Abstractions, Not Details

What You Will Learn

- What the Dependency Inversion Principle (DIP) means and why the direction of dependencies matters
- Why high-level business logic should never depend directly on low-level infrastructure
- How to invert dependencies using interfaces (ports)
- Dependency injection as the primary mechanism for implementing DIP
- IoC containers in Java (Spring @Autowired) and Python (dependency-injector)
- Abstract factories as a DIP pattern
- How DIP enables testing by letting you inject mock implementations
- Before and after: OrderService with direct MySQL dependency refactored to use a DatabasePort

Why This Chapter Matters

Consider this scenario: your `OrderService` class directly imports and creates a `MySQLDatabase` object. Everything works perfectly. Then one day, the team decides to switch from MySQL to PostgreSQL. Or you need to write unit tests for `OrderService` but cannot because it connects to a real database.

The problem is not MySQL. The problem is the direction of the dependency. Your high-level business logic (processing orders) depends directly on a low-level detail (which specific

database to use). When the detail changes, the business logic must change too.

The Dependency Inversion Principle reverses this relationship. Instead of the business logic reaching down to the database, both the business logic and the database depend on an abstraction (an interface). The business logic says "I need something that can save orders." The database says "I can save orders." Neither knows about the other. This makes your code testable, flexible, and resilient to change.

The Dependency Inversion Principle Defined

Robert C. Martin stated two rules:

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.

2. Abstractions should not depend on details. Details should depend on abstractions.

```
+-------------------------------------------------------------+
|     DEPENDENCY INVERSION PRINCIPLE                          |
+-------------------------------------------------------------+
|                                                             |
|  BEFORE (without DIP):                                      |
|                                                             |
|  +----------------+         +-------------------+          |
|  | OrderService   |-------->| MySQLDatabase     |          |
|  | (high-level)   |         | (low-level)       |          |
|  +----------------+         +-------------------+          |
|                                                             |
|  High-level depends on low-level.                           |
|  Change MySQL --> change OrderService.                      |
|  Cannot test OrderService without MySQL.                    |
|                                                             |
|  AFTER (with DIP):                                          |
|                                                             |
|  +----------------+         +-------------------+          |
|  | OrderService   |-------->| DatabasePort      |          |
|  | (high-level)   |         | (abstraction)     |          |
|  +----------------+         +-------------------+          |
|                                    ^                        |
|                                    |                        |
|                             +-------------------+           |
|                             | MySQLDatabase     |           |
|                             | (low-level)       |           |
|                             +-------------------+           |
|                                                             |
|  Both depend on the abstraction.                            |
```

```
|  Change MySQL --> only change MySQLDatabase.                |
|  Test OrderService with a mock DatabasePort.                |
|                                                             |
+-------------------------------------------------------------+
```

Notice how the dependency arrow changed direction. Before DIP, the arrow pointed from high-level to low-level. After DIP, the low-level module points up toward the abstraction. This is the "inversion."

The Problem: Direct Dependencies

BEFORE: OrderService Coupled to MySQL (Java)

```java
// Java -- BAD: High-level module depends directly on low-level module
public class OrderService {

    private final MySQLDatabase database;
    private final StripePaymentGateway paymentGateway;
    private final SendGridEmailService emailService;

    public OrderService() {
        // Creates its own dependencies -- tightly coupled
        this.database = new MySQLDatabase(
            "jdbc:mysql://prod-server:3306/orders"
        );
        this.paymentGateway = new StripePaymentGateway("sk_live_xxx");
        this.emailService = new SendGridEmailService("sg_api_key_xxx");
    }

    public void placeOrder(Order order) {
        // Validate
        if (order.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order has no items");
        }

        // Charge payment through Stripe specifically
        paymentGateway.charge(
            order.getCreditCard(),
            order.getTotal()
        );

        // Save to MySQL specifically
        database.executeUpdate(
            "INSERT INTO orders (id, customer_id, total) VALUES (?, ?, ?)",
```

```
            order.getId(),
            order.getCustomerId(),
            order.getTotal()
        );

        // Send email through SendGrid specifically
        emailService.send(
            order.getCustomerEmail(),
            "Order Confirmation",
            "Your order #" + order.getId() + " has been placed."
        );
    }
}
```

Problems with this code:

1. Cannot test without real services. Unit testing `placeOrder` requires a MySQL server, a Stripe account, and a SendGrid account.
2. Cannot swap implementations. Switching from MySQL to PostgreSQL requires modifying `OrderService`.
3. Cannot reuse. This class is welded to three specific vendors.
4. Violates SRP. `OrderService` knows about MySQL connection strings, Stripe API keys, and SendGrid configuration.

BEFORE: Same Problem in Python

```python
# Python -- BAD: Direct dependency on specific implementations
import mysql.connector
import stripe
from sendgrid import SendGridAPIClient


class OrderService:

    def __init__(self):
        self.db = mysql.connector.connect(
            host="prod-server",
            database="orders",
            user="admin",
            password="secret",
        )
        stripe.api_key = "sk_live_xxx"
        self.email_client = SendGridAPIClient("sg_api_key_xxx")

    def place_order(self, order: Order) -> None:
        if not order.items:
            raise ValueError("Order has no items")
```

```
        # Coupled to Stripe
        stripe.Charge.create(
            amount=int(order.total * 100),
            currency="usd",
            source=order.credit_card_token,
        )

        # Coupled to MySQL
        cursor = self.db.cursor()
        cursor.execute(
            "INSERT INTO orders (id, customer_id, total) "
            "VALUES (%s, %s, %s)",
            (order.id, order.customer_id, order.total),
        )
        self.db.commit()

        # Coupled to SendGrid
        self.email_client.send(
            to_emails=order.customer_email,
            subject="Order Confirmation",
            html_content=f"Your order #{order.id} has been placed.",
        )
```

The Solution: Depend on Abstractions

Step 1: Define the Abstractions (Ports)

The abstractions represent what the high-level module needs, not how the low-level module works.

```java
// Java -- Define abstractions (ports)

public interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String orderId);
    List<Order> findByCustomer(String customerId);
}

public interface PaymentGateway {
    PaymentResult charge(String paymentToken, BigDecimal amount);
    void refund(String transactionId);
}

public interface NotificationService {
```

```
    void sendOrderConfirmation(Order order);
}
```

```python
# Python -- Define abstractions
from abc import ABC, abstractmethod


class OrderRepository(ABC):
    @abstractmethod
    def save(self, order: Order) -> None:
        pass

    @abstractmethod
    def find_by_id(self, order_id: str) -> Order | None:
        pass

    @abstractmethod
    def find_by_customer(self, customer_id: str) -> list[Order]:
        pass


class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, payment_token: str, amount: float) -> PaymentResult:
        pass

    @abstractmethod
    def refund(self, transaction_id: str) -> None:
        pass


class NotificationService(ABC):
    @abstractmethod
    def send_order_confirmation(self, order: Order) -> None:
        pass
```

Step 2: Rewrite the High-Level Module Against Abstractions

```java
// Java -- GOOD: OrderService depends only on abstractions
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentGateway paymentGateway;
    private final NotificationService notificationService;

    // Dependencies are INJECTED, not created internally
    public OrderService(
            OrderRepository orderRepository,
```

```
            PaymentGateway paymentGateway,
            NotificationService notificationService) {

        this.orderRepository = orderRepository;
        this.paymentGateway = paymentGateway;
        this.notificationService = notificationService;
    }

    public void placeOrder(Order order) {
        if (order.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order has no items");
        }

        PaymentResult payment = paymentGateway.charge(
            order.getPaymentToken(),
            order.getTotal()
        );

        order.setTransactionId(payment.getTransactionId());
        orderRepository.save(order);

        notificationService.sendOrderConfirmation(order);
    }
}
```

```python
# Python -- GOOD: OrderService depends only on abstractions
class OrderService:

    def __init__(
        self,
        order_repository: OrderRepository,
        payment_gateway: PaymentGateway,
        notification_service: NotificationService,
    ):
        self._order_repository = order_repository
        self._payment_gateway = payment_gateway
        self._notification_service = notification_service

    def place_order(self, order: Order) -> None:
        if not order.items:
            raise ValueError("Order has no items")

        payment = self._payment_gateway.charge(
            order.payment_token, order.total
        )

        order.transaction_id = payment.transaction_id
        self._order_repository.save(order)

        self._notification_service.send_order_confirmation(order)
```

Notice that `OrderService` no longer mentions MySQL, Stripe, SendGrid, or any specific technology. It works with abstractions.

Step 3: Implement the Abstractions (Adapters)

```java
// Java -- Low-level implementations (adapters)

public class MySQLOrderRepository implements OrderRepository {

    private final DataSource dataSource;

    public MySQLOrderRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public void save(Order order) {
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement stmt = conn.prepareStatement(
                "INSERT INTO orders (id, customer_id, total, tx_id) "
                + "VALUES (?, ?, ?, ?)"
            );
            stmt.setString(1, order.getId());
            stmt.setString(2, order.getCustomerId());
            stmt.setBigDecimal(3, order.getTotal());
            stmt.setString(4, order.getTransactionId());
            stmt.executeUpdate();
        } catch (SQLException e) {
            throw new RepositoryException(
                "Failed to save order: " + order.getId(), e
            );
        }
    }

    @Override
    public Optional<Order> findById(String orderId) {
        // MySQL-specific implementation
    }

    @Override
    public List<Order> findByCustomer(String customerId) {
        // MySQL-specific implementation
    }
}

public class StripePaymentGateway implements PaymentGateway {

    private final String apiKey;

    public StripePaymentGateway(String apiKey) {
        this.apiKey = apiKey;
    }

    @Override
    public PaymentResult charge(String paymentToken, BigDecimal amount) {
        // Stripe-specific implementation
```

```
        Stripe.apiKey = this.apiKey;
        Charge charge = Charge.create(Map.of(
            "amount", amount.multiply(BigDecimal.valueOf(100)).intValue(),
            "currency", "usd",
            "source", paymentToken
        ));
        return new PaymentResult(true, charge.getId());
    }

    @Override
    public void refund(String transactionId) {
        // Stripe-specific refund implementation
    }
}
```

```python
# Python -- Low-level implementations (adapters)
import mysql.connector


class MySQLOrderRepository(OrderRepository):

    def __init__(self, connection_config: dict):
        self._config = connection_config

    def save(self, order: Order) -> None:
        conn = mysql.connector.connect(**self._config)
        try:
            cursor = conn.cursor()
            cursor.execute(
                "INSERT INTO orders (id, customer_id, total, tx_id) "
                "VALUES (%s, %s, %s, %s)",
                (order.id, order.customer_id, order.total,
                 order.transaction_id),
            )
            conn.commit()
        except Exception as e:
            raise RepositoryError(
                f"Failed to save order: {order.id}"
            ) from e
        finally:
            conn.close()

    def find_by_id(self, order_id: str) -> Order | None:
        # MySQL-specific implementation
        pass

    def find_by_customer(self, customer_id: str) -> list[Order]:
        # MySQL-specific implementation
        pass


class StripePaymentGateway(PaymentGateway):
```

```
    def __init__(self, api_key: str):
        self._api_key = api_key

    def charge(self, payment_token: str, amount: float) -> PaymentResult:
        import stripe
        stripe.api_key = self._api_key
        charge = stripe.Charge.create(
            amount=int(amount * 100),
            currency="usd",
            source=payment_token,
        )
        return PaymentResult(success=True, transaction_id=charge.id)

    def refund(self, transaction_id: str) -> None:
        # Stripe-specific refund implementation
        pass
```

The Dependency Arrows: Before and After

```
+-------------------------------------------------------------+
|     DEPENDENCY DIRECTION: BEFORE vs AFTER                   |
+-------------------------------------------------------------+
|                                                             |
|  BEFORE (Dependencies point downward):                      |
|                                                             |
|    OrderService                                             |
|        |                                                    |
|        +---------> MySQLDatabase                            |
|        +---------> StripeGateway                            |
|        +---------> SendGridEmail                            |
|                                                             |
|  High-level knows about low-level details.                  |
|  Change flows UPWARD (breaking high-level code).            |
|                                                             |
|  ------------------------------------------------------- |
|                                                             |
|  AFTER (Dependencies point toward abstractions):            |
|                                                             |
|    OrderService                                             |
|        |                                                    |
|        +---------> OrderRepository (interface)              |
|        |                  ^                                |
|        |                  |                                |
|        |           MySQLOrderRepository                     |
|        |                                                    |
|        +---------> PaymentGateway (interface)               |
|        |                  ^                                |
```

```
|        |                  |                                |
|        |           StripePaymentGateway                    |
|        |                                                   |
|        +---------> NotificationService (interface)         |
|                          ^                                |
|                          |                                |
|                   EmailNotificationService                |
|                                                            |
|  Everything depends on abstractions.                       |
|  Change is isolated to the adapter that changes.           |
|                                                            |
+------------------------------------------------------------+
```

Dependency Injection: The Mechanism

DIP tells you what to depend on (abstractions). Dependency Injection (DI) tells you how to provide those abstractions to the classes that need them. There are three common forms:

Constructor Injection (Preferred)

```java
// Java -- Constructor injection
public class OrderService {

    private final OrderRepository repository;

    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

```python
# Python -- Constructor injection
class OrderService:

    def __init__(self, repository: OrderRepository):
        self._repository = repository
```

Method Injection (For One-Off Dependencies)

```java
// Java -- Method injection: dependency used only in one method
public class ReportGenerator {

    public Report generate(DataSource dataSource) {
        // dataSource is only needed for this method
    }
}
```

Setter Injection (Least Preferred)

```java
// Java -- Setter injection: allows changing dependency at runtime
public class OrderService {

    private OrderRepository repository;

    public void setRepository(OrderRepository repository) {
        this.repository = repository;
    }
}
// Problem: object is in invalid state between construction and setRepository()
```

Constructor injection is preferred because it makes dependencies explicit, ensures the object is always in a valid state, and makes the class immutable.

IoC Containers

In a real application, you might have dozens of classes with dependencies. Wiring them together manually becomes tedious. IoC (Inversion of Control) containers automate this wiring.

Java: Spring Framework

```java
// Java -- Spring @Autowired wires dependencies automatically

@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentGateway paymentGateway;
    private final NotificationService notificationService;
```

```
    @Autowired
    public OrderService(
            OrderRepository orderRepository,
            PaymentGateway paymentGateway,
            NotificationService notificationService) {

        this.orderRepository = orderRepository;
        this.paymentGateway = paymentGateway;
        this.notificationService = notificationService;
    }

    public void placeOrder(Order order) {
        // Business logic -- no infrastructure concerns
    }
}

@Repository
public class MySQLOrderRepository implements OrderRepository {
    // Spring finds this because it implements OrderRepository
}

@Component
public class StripePaymentGateway implements PaymentGateway {
    // Spring finds this because it implements PaymentGateway
}

@Component
public class EmailNotificationService implements NotificationService {
    // Spring finds this because it implements NotificationService
}
```

Spring scans for classes annotated with `@Service`, `@Repository`, and `@Component`, figures out which class implements which interface, and automatically wires everything together.

Python: dependency-injector

```python
# Python -- Using dependency-injector library
from dependency_injector import containers, providers


class Container(containers.DeclarativeContainer):

    config = providers.Configuration()

    # Low-level modules
    order_repository = providers.Singleton(
        MySQLOrderRepository,
        connection_config=config.database,
    )
```

```
    payment_gateway = providers.Singleton(
        StripePaymentGateway,
        api_key=config.stripe_api_key,
    )

    notification_service = providers.Singleton(
        EmailNotificationService,
        smtp_host=config.smtp_host,
    )

    # High-level module -- wired automatically
    order_service = providers.Factory(
        OrderService,
        order_repository=order_repository,
        payment_gateway=payment_gateway,
        notification_service=notification_service,
    )


# Usage
container = Container()
container.config.from_yaml("config.yaml")

service = container.order_service()
service.place_order(order)
```

Simple Manual Wiring (No Framework)

You do not always need a framework. For smaller applications, manual wiring at the application entry point is perfectly fine:

```python
# Python -- Simple manual dependency injection
def create_app():
    """Wire everything together at the top level."""
    db_config = load_config("database")
    stripe_key = load_config("stripe_api_key")

    repository = MySQLOrderRepository(db_config)
    gateway = StripePaymentGateway(stripe_key)
    notifier = EmailNotificationService(load_config("smtp"))

    order_service = OrderService(repository, gateway, notifier)

    return order_service
```

```java
// Java -- Simple manual wiring
public class Application {
```

```
    public static void main(String[] args) {
        // Wire everything at the entry point
        DataSource dataSource = createDataSource();
        OrderRepository repository = new MySQLOrderRepository(dataSource);
        PaymentGateway gateway = new StripePaymentGateway("sk_live_xxx");
        NotificationService notifier = new EmailNotificationService();

        OrderService service = new OrderService(
            repository, gateway, notifier
        );

        // Use the service
        service.placeOrder(order);
    }
}
```

DIP Enables Testing

The biggest practical benefit of DIP is testability. When your business logic depends on abstractions, you can inject test doubles (mocks, stubs, fakes) instead of real infrastructure.

```java
// Java -- Testing with mock dependencies
public class OrderServiceTest {

    @Test
    public void placeOrder_shouldSaveOrderAfterPayment() {
        // Create test doubles
        OrderRepository mockRepo = mock(OrderRepository.class);
        PaymentGateway mockGateway = mock(PaymentGateway.class);
        NotificationService mockNotifier = mock(NotificationService.class);

        when(mockGateway.charge(anyString(), any(BigDecimal.class)))
            .thenReturn(new PaymentResult(true, "tx_123"));

        // Inject test doubles
        OrderService service = new OrderService(
            mockRepo, mockGateway, mockNotifier
        );

        // Test
        Order order = new Order("order_1", "cust_1",
            List.of(new Item("Widget", 1)),
            "tok_visa", BigDecimal.valueOf(29.99));

        service.placeOrder(order);
```

```
        // Verify
        verify(mockGateway).charge("tok_visa", BigDecimal.valueOf(29.99));
        verify(mockRepo).save(order);
        verify(mockNotifier).sendOrderConfirmation(order);
    }

    @Test
    public void placeOrder_shouldNotSaveIfPaymentFails() {
        OrderRepository mockRepo = mock(OrderRepository.class);
        PaymentGateway mockGateway = mock(PaymentGateway.class);
        NotificationService mockNotifier = mock(NotificationService.class);

        when(mockGateway.charge(anyString(), any(BigDecimal.class)))
            .thenThrow(new PaymentException("Card declined"));

        OrderService service = new OrderService(
            mockRepo, mockGateway, mockNotifier
        );

        Order order = new Order("order_1", "cust_1",
            List.of(new Item("Widget", 1)),
            "tok_declined", BigDecimal.valueOf(29.99));

        assertThrows(PaymentException.class,
            () -> service.placeOrder(order));

        verify(mockRepo, never()).save(any());
        verify(mockNotifier, never()).sendOrderConfirmation(any());
    }
}
```

```python
# Python -- Testing with mock dependencies
from unittest.mock import Mock, MagicMock
import pytest


class TestOrderService:

    def test_place_order_saves_after_payment(self):
        # Create test doubles
        mock_repo = Mock(spec=OrderRepository)
        mock_gateway = Mock(spec=PaymentGateway)
        mock_notifier = Mock(spec=NotificationService)

        mock_gateway.charge.return_value = PaymentResult(
            success=True, transaction_id="tx_123"
        )

        # Inject test doubles
        service = OrderService(mock_repo, mock_gateway, mock_notifier)
```

```
        order = Order(
            id="order_1",
            customer_id="cust_1",
            items=[Item("Widget", 1)],
            payment_token="tok_visa",
            total=29.99,
        )

        service.place_order(order)

        # Verify
        mock_gateway.charge.assert_called_once_with("tok_visa", 29.99)
        mock_repo.save.assert_called_once_with(order)
        mock_notifier.send_order_confirmation.assert_called_once_with(order)

    def test_place_order_does_not_save_if_payment_fails(self):
        mock_repo = Mock(spec=OrderRepository)
        mock_gateway = Mock(spec=PaymentGateway)
        mock_notifier = Mock(spec=NotificationService)

        mock_gateway.charge.side_effect = PaymentError("Card declined")

        service = OrderService(mock_repo, mock_gateway, mock_notifier)

        order = Order(
            id="order_1",
            customer_id="cust_1",
            items=[Item("Widget", 1)],
            payment_token="tok_declined",
            total=29.99,
        )

        with pytest.raises(PaymentError):
            service.place_order(order)

        mock_repo.save.assert_not_called()
        mock_notifier.send_order_confirmation.assert_not_called()
```

Without DIP, these tests would require a running MySQL database, a Stripe account, and an email server. With DIP, they run in milliseconds with zero infrastructure.

Abstract Factories: Creating Families of Objects

Sometimes you need to create objects at runtime, but you still want to depend on abstractions. An abstract factory lets you do this.

```java
// Java -- Abstract factory for DIP-compliant object creation
public interface DatabaseFactory {
    Connection createConnection();
    QueryBuilder createQueryBuilder();
    MigrationRunner createMigrationRunner();
}

public class MySQLDatabaseFactory implements DatabaseFactory {

    @Override
    public Connection createConnection() {
        return new MySQLConnection("jdbc:mysql://...");
    }

    @Override
    public QueryBuilder createQueryBuilder() {
        return new MySQLQueryBuilder();
    }

    @Override
    public MigrationRunner createMigrationRunner() {
        return new MySQLMigrationRunner();
    }
}

public class PostgresDatabaseFactory implements DatabaseFactory {

    @Override
    public Connection createConnection() {
        return new PostgresConnection("jdbc:postgresql://...");
    }

    @Override
    public QueryBuilder createQueryBuilder() {
        return new PostgresQueryBuilder();
    }

    @Override
    public MigrationRunner createMigrationRunner() {
        return new PostgresMigrationRunner();
    }
}

// High-level code uses the factory abstraction
public class DataService {

    private final DatabaseFactory dbFactory;

    public DataService(DatabaseFactory dbFactory) {
        this.dbFactory = dbFactory;
    }

    public void runMigrations() {
        Connection conn = dbFactory.createConnection();
```

```
        MigrationRunner runner = dbFactory.createMigrationRunner();
        runner.migrate(conn);
    }
}
```

```python
# Python -- Abstract factory
from abc import ABC, abstractmethod


class DatabaseFactory(ABC):
    @abstractmethod
    def create_connection(self) -> Connection:
        pass

    @abstractmethod
    def create_query_builder(self) -> QueryBuilder:
        pass


class MySQLDatabaseFactory(DatabaseFactory):

    def create_connection(self) -> Connection:
        return MySQLConnection("mysql://...")

    def create_query_builder(self) -> QueryBuilder:
        return MySQLQueryBuilder()


class PostgresDatabaseFactory(DatabaseFactory):

    def create_connection(self) -> Connection:
        return PostgresConnection("postgresql://...")

    def create_query_builder(self) -> QueryBuilder:
        return PostgresQueryBuilder()


class DataService:

    def __init__(self, db_factory: DatabaseFactory):
        self._db_factory = db_factory

    def execute_query(self, query_spec) -> list:
        conn = self._db_factory.create_connection()
        builder = self._db_factory.create_query_builder()
        query = builder.build(query_spec)
        return conn.execute(query)
```

Common Mistakes

1. Depending on abstractions but creating concrete classes internally. If your constructor does `this.repo = new MySQLRepository()`, you have DIP in name only. The dependency must be injected from outside.
2. Creating an abstraction for every class. Not everything needs an interface. DIP matters at architectural boundaries -- between business logic and infrastructure, between your code and third-party libraries. A utility class that formats strings does not need an interface.
3. Leaking implementation details through the abstraction. If your `OrderRepository` interface has a method called `executeMySQLQuery()`, the abstraction is not abstract. Define the interface in terms of business operations: `save()`, `findById()`, `delete()`.
4. Using a DI container as a service locator. Do not pass the container itself around and have classes pull dependencies from it. This hides dependencies and makes code harder to understand.
5. Circular dependencies. If A depends on B's abstraction and B depends on A's abstraction, you have a design problem that DIP cannot fix. Refactor to remove the cycle.

Best Practices

1. Define abstractions in the high-level module. The `OrderRepository` interface should live in the same package/module as `OrderService`, not in the database package. This ensures the high-level module owns its own abstractions.
2. Use constructor injection. It makes dependencies explicit, enables immutability, and makes invalid states impossible.
3. Apply DIP at architectural boundaries. Invert dependencies between your business logic and databases, APIs, file systems, email services, and other infrastructure.
4. Keep abstractions stable. The interface should change rarely. If it changes frequently, the abstraction is wrong.
5. Wire dependencies at the composition root. The entry point of your application (main function, Spring configuration, container setup) is where all the concrete implementations get wired together. Business logic classes should never create their own dependencies.
6. Start without a DI framework. Manual constructor injection is simple and clear. Add a framework only when the wiring becomes complex enough to justify it.

Quick Summary

```
+-------------------------------------------------------------+
|              DIP CHEAT SHEET                                |
+-------------------------------------------------------------+
|                                                             |
|  PRINCIPLE:                                                 |
|    High-level modules should not depend on low-level        |
|    modules. Both should depend on abstractions.             |
|                                                             |
|  HOW TO APPLY:                                              |
|    1. Define an interface for what the high-level module    |
|       needs (OrderRepository, PaymentGateway)               |
|    2. High-level module depends on the interface            |
|    3. Low-level module implements the interface             |
|    4. Inject the implementation via constructor             |
|                                                             |
|  KEY BENEFIT:                                               |
|    Testability. Inject mocks instead of real databases,     |
|    APIs, and services. Tests run in milliseconds.           |
|                                                             |
|  TOOLS:                                                     |
|    Java: Spring (@Autowired, @Service, @Repository)         |
|    Python: dependency-injector, or manual wiring            |
|                                                             |
|  WHERE TO APPLY:                                            |
|    At architectural boundaries:                             |
|    - Business logic <--> Database                           |
|    - Business logic <--> External APIs                      |
|    - Business logic <--> File system                        |
|    - Business logic <--> Email/notification services        |
|                                                             |
+-------------------------------------------------------------+
```

Key Points

- The Dependency Inversion Principle says high-level modules should not depend on low-level modules; both should depend on abstractions
- "Inversion" means the dependency arrow flips: instead of business logic pointing to infrastructure, infrastructure points toward the abstraction that business logic defines
- Dependency injection is the mechanism for implementing DIP: pass abstractions into constructors instead of creating concrete classes internally
- DIP enables testability because you can inject mock implementations instead of real databases and APIs

- IoC containers like Spring and dependency-injector automate the wiring, but manual constructor injection works well for smaller applications
- Define abstractions from the high-level module's perspective, using business language, not infrastructure language
- Apply DIP at architectural boundaries between business logic and infrastructure, not between every pair of classes

---

Practice Questions

1. Explain the Dependency Inversion Principle in your own words. What does "inversion" refer to specifically?
2. What is the difference between Dependency Inversion (the principle) and Dependency Injection (the technique)? How are they related?
3. Why is constructor injection preferred over setter injection? What problems does setter injection create?
4. How does DIP enable unit testing? Give a specific example of how you would test a service that depends on a database without DIP, and then with DIP.
5. Where should the abstraction (interface) live -- in the high-level module's package or the low-level module's package? Why does this matter?

Exercises

Exercise 1: Invert the Dependencies

Refactor the following class to follow DIP. Define appropriate abstractions, implement them, and update the class to use constructor injection.

```python
import requests
import json


class WeatherReporter:

    def get_daily_report(self, city: str) -> str:
        # Directly coupled to OpenWeatherMap API
```

```
        response = requests.get(
            f"https://api.openweathermap.org/data/2.5/weather"
            f"?q={city}&appid=hardcoded_api_key"
        )
        data = response.json()

        # Directly coupled to file system
        with open(f"/reports/{city}_weather.json", "w") as f:
            json.dump(data, f)

        # Directly coupled to print
        report = f"Weather in {city}: {data['main']['temp']}K"
        print(report)
        return report
```

Exercise 2: Write Tests with Mock Dependencies

Using the refactored code from Exercise 1, write unit tests that verify:

1. The reporter fetches weather data from the weather service
2. The reporter saves the data using the storage service
3. The reporter formats and returns a readable report
4. If the weather service fails, the reporter raises an appropriate error

Use mock objects -- no real HTTP calls or file I/O.

Exercise 3: Compare Manual Wiring vs. Container

Set up the OrderService example from this chapter twice: once with manual constructor injection in a `main()` function, and once using a DI container (Spring for Java or dependency-injector for Python). Compare the two approaches in terms of boilerplate code, readability, and flexibility. Which would you choose for a project with 10 services? With 100 services?

What Is Next?

You have now completed all five SOLID principles. Together, they provide a foundation for building software that is easy to understand, easy to change, and easy to test. The next chapters build on this foundation with additional principles (DRY, KISS, YAGNI) and practical techniques (code smells, refactoring patterns, unit testing) that turn these principles into daily practice.

Chapter 13: DRY, KISS, and YAGNI -- The Three Pillars of Pragmatic Code

What You Will Learn

- What the DRY principle really means and when duplication is actually acceptable
- How to apply KISS to write the simplest solution that genuinely works
- Why YAGNI saves you from building features nobody asked for
- How to recognize over-engineering and premature abstraction
- The Rule of Three for deciding when to eliminate duplication
- Practical before-and-after transformations for each principle

Why This Chapter Matters

Every developer eventually writes code that is too clever, too abstract, or too duplicated. These three principles -- DRY, KISS, and YAGNI -- act as guardrails that keep your code practical and maintainable. They sound simple, but applying them well requires judgment. Misapplying them causes just as many problems as ignoring them. This chapter gives you that judgment.

13.1 DRY -- Don't Repeat Yourself

The Core Idea

DRY was coined by Andy Hunt and Dave Thomas in "The Pragmatic Programmer." It states:

> Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.

Notice: DRY is about knowledge, not just code. Two identical lines of code are not necessarily a DRY violation if they represent different pieces of knowledge.

What DRY Actually Means

```
+----------------------------------------------------+
|              DRY IS ABOUT KNOWLEDGE                |
+----------------------------------------------------+
|                                                    |
|  Same code, same reason to change  -->  Violation  |
|  Same code, different reasons      -->  Acceptable |
|  Different code, same knowledge    -->  Violation   |
|                                                    |
+----------------------------------------------------+
```

BEFORE: DRY Violation (Java)

```java
public class OrderService {

    public double calculateOrderTotal(Order order) {
        double total = 0;
        for (OrderItem item : order.getItems()) {
            total += item.getPrice() * item.getQuantity();
        }
        // Apply 10% tax
        total = total * 1.10;
        return total;
    }

    public double calculateInvoiceTotal(Invoice invoice) {
        double total = 0;
        for (InvoiceItem item : invoice.getItems()) {
            total += item.getPrice() * item.getQuantity();
        }
        // Apply 10% tax
        total = total * 1.10;
        return total;
    }

    public String generateOrderSummary(Order order) {
        double total = 0;
        for (OrderItem item : order.getItems()) {
```

```
            total += item.getPrice() * item.getQuantity();
        }
        // Apply 10% tax
        total = total * 1.10;
        return "Order total: $" + String.format("%.2f", total);
    }
}
```

The tax rate (10%) and the line-item calculation logic appear three times. If the tax rate changes, you must find and update all three locations.

AFTER: DRY Applied (Java)

```java
public class OrderService {

    private static final double TAX_RATE = 0.10;

    private double calculateLineItemTotal(List<? extends LineItem> items) {
        double subtotal = 0;
        for (LineItem item : items) {
            subtotal += item.getPrice() * item.getQuantity();
        }
        return subtotal;
    }

    private double applyTax(double subtotal) {
        return subtotal * (1 + TAX_RATE);
    }

    public double calculateOrderTotal(Order order) {
        return applyTax(calculateLineItemTotal(order.getItems()));
    }

    public double calculateInvoiceTotal(Invoice invoice) {
        return applyTax(calculateLineItemTotal(invoice.getItems()));
    }

    public String generateOrderSummary(Order order) {
        double total = calculateOrderTotal(order);
        return "Order total: $" + String.format("%.2f", total);
    }
}
```

Now the tax rate lives in one place, and the line-item calculation is shared through a common interface.

BEFORE: DRY Violation (Python)

```python
def send_welcome_email(user):
    subject = f"Welcome, {user.name}!"
    body = f"Hi {user.name},\n\nWelcome to our platform."
    smtp = smtplib.SMTP("mail.example.com", 587)
    smtp.starttls()
    smtp.login("noreply@example.com", "secret123")
    smtp.sendmail("noreply@example.com", user.email,
                  f"Subject: {subject}\n\n{body}")
    smtp.quit()

def send_password_reset_email(user, reset_link):
    subject = "Password Reset"
    body = f"Hi {user.name},\n\nClick here to reset: {reset_link}"
    smtp = smtplib.SMTP("mail.example.com", 587)
    smtp.starttls()
    smtp.login("noreply@example.com", "secret123")
    smtp.sendmail("noreply@example.com", user.email,
                  f"Subject: {subject}\n\n{body}")
    smtp.quit()

def send_order_confirmation(user, order_id):
    subject = "Order Confirmed"
    body = f"Hi {user.name},\n\nYour order #{order_id} is confirmed."
    smtp = smtplib.SMTP("mail.example.com", 587)
    smtp.starttls()
    smtp.login("noreply@example.com", "secret123")
    smtp.sendmail("noreply@example.com", user.email,
                  f"Subject: {subject}\n\n{body}")
    smtp.quit()
```

The SMTP connection logic is repeated three times. If you change the mail server, you update three places.

AFTER: DRY Applied (Python)

```python
class EmailService:
    def __init__(self, host="mail.example.com", port=587,
                 sender="noreply@example.com", password="secret123"):
        self.host = host
        self.port = port
        self.sender = sender
        self.password = password

    def send(self, recipient, subject, body):
        smtp = smtplib.SMTP(self.host, self.port)
        smtp.starttls()
        smtp.login(self.sender, self.password)
        smtp.sendmail(self.sender, recipient,
                      f"Subject: {subject}\n\n{body}")
        smtp.quit()
```

```
email_service = EmailService()

def send_welcome_email(user):
    email_service.send(user.email, f"Welcome, {user.name}!",
                       f"Hi {user.name},\n\nWelcome to our platform.")

def send_password_reset_email(user, reset_link):
    email_service.send(user.email, "Password Reset",
                       f"Hi {user.name},\n\nClick here to reset: {reset_link}")

def send_order_confirmation(user, order_id):
    email_service.send(user.email, "Order Confirmed",
                       f"Hi {user.name},\n\nYour order #{order_id} is confirmed.")
```

When Duplication Is Acceptable

Not all duplication is bad. Here are cases where keeping duplicated code is the right call:

1. Coincidental duplication. Two pieces of code look the same today but represent different business concepts that will evolve independently.

```python
# These look similar but serve DIFFERENT business rules
def calculate_employee_bonus(salary):
    return salary * 0.10  # HR policy: 10% bonus

def calculate_sales_tax(price):
    return price * 0.10  # Tax law: 10% tax rate
```

Merging these into a single `apply_ten_percent()` function would be wrong. When the tax rate changes to 12%, you do not want to accidentally change bonuses too.

2. Across bounded contexts. In microservices or modular systems, some duplication between services is healthier than coupling them through shared libraries.

3. Test code. Tests should be readable and self-contained. A little duplication in tests is far better than fragile shared test helpers.

The Rule of Three

> The first time you write something, just write it. The second time, wince but duplicate it. The third time, refactor.

This rule prevents premature abstraction. You need at least three examples of duplication to understand the true pattern before you extract it.

```
Occurrence 1:  Write it.
                  |
Occurrence 2:  Duplicate it. Note the similarity.
                  |
Occurrence 3:  NOW refactor. You understand the pattern.
```

13.2 KISS -- Keep It Simple, Stupid

The Core Idea

KISS says: choose the simplest solution that correctly solves the problem. Complexity is a cost. Every line of clever code is a line that a future developer (including future you) must understand.

The Simplicity Spectrum

```
Too Simple          Just Right           Over-Engineered
(doesn't work)     (KISS sweet spot)    (works but nobody
                                         can maintain it)
    |                    |                    |
    +--------------------+---------------------+

Example:            Example:              Example:
No validation       If/else check         Custom validation
at all              with clear message    framework with
                                          plugin system and
                                          XML configuration
```

BEFORE: Over-Complicated (Java)

```java
// Someone wanted a "flexible" way to check if a number is even
public class NumberClassifierFactory {

    public interface NumberClassificationStrategy {
        boolean classify(int number);
    }

    public static class EvenClassificationStrategy
            implements NumberClassificationStrategy {
```

```
        @Override
        public boolean classify(int number) {
            return number % 2 == 0;
        }
    }

    public static NumberClassificationStrategy createStrategy(String type) {
        switch (type) {
            case "even": return new EvenClassificationStrategy();
            default: throw new IllegalArgumentException("Unknown type: " + type);
        }
    }
}

// Usage
NumberClassificationStrategy strategy =
    NumberClassifierFactory.createStrategy("even");
boolean result = strategy.classify(42);
```

AFTER: KISS Applied (Java)

```java
public class NumberUtils {
    public static boolean isEven(int number) {
        return number % 2 == 0;
    }
}

// Usage
boolean result = NumberUtils.isEven(42);
```

Same functionality. One method instead of an entire factory-strategy hierarchy.

BEFORE: Over-Complicated (Python)

```python
from abc import ABC, abstractmethod

class DataTransformerBase(ABC):
    @abstractmethod
    def transform(self, data):
        pass

class UpperCaseTransformer(DataTransformerBase):
    def transform(self, data):
        return data.upper()

class LowerCaseTransformer(DataTransformerBase):
```

```
    def transform(self, data):
        return data.lower()

class TransformerPipeline:
    def __init__(self):
        self.transformers = []

    def add_transformer(self, transformer):
        self.transformers.append(transformer)

    def execute(self, data):
        result = data
        for t in self.transformers:
            result = t.transform(result)
        return result

# Usage: just to convert a string to uppercase
pipeline = TransformerPipeline()
pipeline.add_transformer(UpperCaseTransformer())
result = pipeline.execute("hello world")
```

AFTER: KISS Applied (Python)

```python
result = "hello world".upper()
```

One line. No classes. No pipeline. The built-in method does exactly what we need.

Real-World Scenario: Configuration Loading

A team needs to load application configuration from a file.

Over-engineered approach:

```
  +-------------------+     +-----------------+     +------------------+
  | ConfigLoaderFactory| --> | ConfigParser    | --> | ConfigValidator  |
  +-------------------+     | Interface       |     | Interface        |
           |                +-----------------+     +------------------+
           |                   |          |              |
           v                   v          v              v
  +------------------+  +----------+ +----------+ +------------+
  | ConfigBuilder    |  | YAMLImpl | | JSONImpl | | SchemaValid|
  +------------------+  +----------+ +----------+ +------------+
```

KISS approach:

```python
import json

def load_config(filepath="config.json"):
    with open(filepath) as f:
        return json.load(f)

config = load_config()
```

Start simple. Add YAML support only when someone actually needs it. Add validation only when bad configs cause real problems.

13.3 YAGNI -- You Aren't Gonna Need It

The Core Idea

YAGNI comes from Extreme Programming. It states: do not add functionality until it is necessary. The cost of building unused features is enormous:

```
  Cost of a Feature You Don't Need:
  +-----------------------------------------+
  |  Time to build it              (wasted)   |
  |  Time to test it               (wasted)   |
  |  Time to maintain it           (ongoing)  |
  |  Complexity it adds            (ongoing)  |
  |  Bugs it might introduce       (ongoing)  |
  |  Time to understand it later   (ongoing)  |
  +-----------------------------------------+
```

Studies show that roughly 65% of features in software products are rarely or never used.

BEFORE: YAGNI Violation (Java)

A team needs a simple user registration system. One developer anticipates "future needs."

```java
public class UserService {

    // Current requirement: register users with email and password
    // "Future-proofed" with things nobody asked for

    private final UserRepository userRepository;
    private final EventBus eventBus;
```

```
    private final AuditLogService auditLogService;
    private final NotificationPreferenceService notifPrefService;
    private final UserProfileImageService imageService;
    private final SocialMediaLinkingService socialService;
    private final TwoFactorAuthService twoFactorService;
    private final GamificationService gamificationService;

    public User registerUser(RegistrationRequest request) {
        // Validate (required)
        validateEmail(request.getEmail());
        validatePassword(request.getPassword());

        // Create user (required)
        User user = new User(request.getEmail(),
                             hashPassword(request.getPassword()));
        userRepository.save(user);

        // --- Everything below is YAGNI ---

        // Set up default notification preferences
        notifPrefService.createDefaults(user);

        // Generate default avatar
        imageService.generateDefaultAvatar(user);

        // Set up gamification profile
        gamificationService.initializeProfile(user);

        // Prepare 2FA enrollment (not required yet)
        twoFactorService.prepareEnrollment(user);

        // Link social accounts (nobody asked for this)
        socialService.checkAndLink(user);

        // Publish event (no consumers exist yet)
        eventBus.publish(new UserRegisteredEvent(user));

        // Audit log (no compliance requirement yet)
        auditLogService.log("USER_REGISTERED", user.getId());

        return user;
    }
}
```

AFTER: YAGNI Applied (Java)

```java
public class UserService {

    private final UserRepository userRepository;
```

```
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User registerUser(String email, String password) {
        validateEmail(email);
        validatePassword(password);

        User user = new User(email, hashPassword(password));
        userRepository.save(user);
        return user;
    }

    private void validateEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email address");
        }
    }

    private void validatePassword(String password) {
        if (password == null || password.length() < 8) {
            throw new IllegalArgumentException(
                "Password must be at least 8 characters");
        }
    }

    private String hashPassword(String password) {
        return BCrypt.hashpw(password, BCrypt.gensalt());
    }
}
```

Clean. Does exactly what the requirements say. When someone actually needs 2FA, gamification, or social linking, you add it then -- with the benefit of understanding the real requirements.

BEFORE: YAGNI Violation (Python)

```python
class DataProcessor:
    """Processes data from CSV files."""

    def __init__(self, source_type="csv"):
        self.source_type = source_type
        # "Supporting" formats nobody uses yet
        self.parsers = {
            "csv": self._parse_csv,
            "json": self._parse_json,       # Not needed yet
            "xml": self._parse_xml,         # Not needed yet
            "yaml": self._parse_yaml,       # Not needed yet
            "excel": self._parse_excel,     # Not needed yet
```

```
            "parquet": self._parse_parquet,  # Not needed yet
        }
        self.output_formats = ["csv", "json", "xml", "html", "pdf"]
        self.compression_types = ["gzip", "bzip2", "lzma", "zip"]

    def process(self, filepath):
        parser = self.parsers.get(self.source_type)
        if not parser:
            raise ValueError(f"Unsupported format: {self.source_type}")
        return parser(filepath)

    def _parse_csv(self, filepath):
        # Actual implementation
        import csv
        with open(filepath) as f:
            return list(csv.DictReader(f))

    def _parse_json(self, filepath):
        raise NotImplementedError("Coming soon!")

    def _parse_xml(self, filepath):
        raise NotImplementedError("Coming soon!")

    def _parse_yaml(self, filepath):
        raise NotImplementedError("Coming soon!")

    def _parse_excel(self, filepath):
        raise NotImplementedError("Coming soon!")

    def _parse_parquet(self, filepath):
        raise NotImplementedError("Coming soon!")
```

Half of these methods are stubs that throw `NotImplementedError`. They add noise, give a false impression of capability, and clutter the interface.

AFTER: YAGNI Applied (Python)

```python
import csv

class CsvProcessor:
    """Processes data from CSV files."""

    def process(self, filepath):
        with open(filepath) as f:
            return list(csv.DictReader(f))
```

When someone needs JSON support, add a `JsonProcessor`. When multiple formats are in use, then consider a common interface. Not before.

Over-Engineering: A Real-World Story

Imagine a team building an internal tool for 20 users. They spend three months building:

- A plugin architecture for custom data sources
- A role-based access control system with eight permission levels
- A caching layer with configurable eviction strategies
- An internationalization system supporting 12 languages
- A theming engine with custom CSS compilation

The actual need was a form that reads from one database and displays results in a table for 20 English-speaking employees in the same department.

```
What was needed:          What was built:

+--------+  +-------+    +--------+  +--------+  +--------+
| Form   |->| Table |    | Plugin |  | RBAC   |  | Cache  |
+--------+  +-------+    | System |  | System |  | Layer  |
                          +--------+  +--------+  +--------+
                          +--------+  +--------+  +--------+
                          | i18n   |  | Theme  |  | API    |
                          | Engine |  | Engine |  | Gateway|
                          +--------+  +--------+  +--------+

1 week of work             3 months of work
Maintained by anyone       Maintained by specialists
```

Premature Abstraction

Premature abstraction is one of the most common YAGNI violations. It happens when you create interfaces, abstract classes, or generic solutions before you have enough concrete examples.

```java
// Premature: only one payment method exists
public interface PaymentProcessor {
    PaymentResult process(Payment payment);
}

public class CreditCardProcessor implements PaymentProcessor {
    public PaymentResult process(Payment payment) {
        // ... credit card logic
    }
}

// When you only have one implementation, the interface
// adds complexity without benefit. Just use the class directly.
```

Better approach: Write the concrete class first. When a second payment method is needed, then extract the interface.

13.4 How the Three Principles Work Together

```
+-----------------------------------------------------------+
|               The Pragmatic Code Triangle                 |
|                                                           |
|              DRY                                          |
|             /   \          "Is this knowledge             |
|            /     \          duplicated?"                   |
|           /       \                                       |
|          /  GOOD   \                                      |
|         /   CODE    \                                     |
|        /             \                                    |
|     KISS ----------- YAGNI                                |
|  "Is this the       "Do I actually                        |
|   simplest way?"     need this?"                          |
+-----------------------------------------------------------+
```

Decision flow when writing code:

```
Need to implement something?
       |
       v
Do I actually need this now? ----NO----> Don't build it (YAGNI)
       |
      YES
       |
       v
Is there a simpler way? --------YES----> Use the simpler way (KISS)
       |
      NO (this is the simplest)
       |
       v
Does this duplicate existing
knowledge? ---------------------YES----> Reuse existing code (DRY)
       |                                  (if Rule of Three is met)
      NO
       |
       v
Write the code.
```

Balancing the Principles

Sometimes the principles conflict:

Situation	DRY Says	KISS Says	YAGNI Says	Best Choice
Two similar functions	Merge them	Keep them separate if simpler	Only merge if both are needed	Depends on Rule of Three
Adding an interface	Extract common behavior	Skip the interface	Don't add until second implementation	Wait for second use case
Shared utility library	Centralize common code	Keep it local if simpler	Don't build a library for one use	Start local, extract later

---

Common Mistakes

1. Treating DRY as "never duplicate any code." DRY is about knowledge duplication, not textual duplication. Two functions that look the same but represent different business rules should stay separate.
2. Applying KISS to mean "no abstractions ever." KISS does not mean dumb code. It means the right level of abstraction for the current problem. A well-named function is simpler than inline code, even though it adds an abstraction.
3. Using YAGNI as an excuse for no planning. YAGNI means don't build features you don't need. It does not mean don't think about architecture or design. You should still write extensible code -- just don't implement the extensions.
4. Refactoring too early. Extracting a shared function after seeing duplication only twice often leads to the wrong abstraction. Wait for three occurrences.
5. Confusing "simple" with "easy." Simple means fewer concepts and moving parts. Easy means familiar. A regex might be easy for you but it is not simple. A well-named function with clear logic is simple even if you have to write more lines.

Best Practices

1. Apply the Rule of Three before extracting shared code. Tolerate duplication until you see the pattern clearly in three places.

2. Start with the simplest implementation that works. You can always add complexity later. You rarely remove it.
3. Ask "who asked for this?" before building any feature. If the answer is "nobody yet," stop.
4. Prefer boring technology. A simple database query is better than an in-memory cache you need to invalidate. A flat file is better than a message queue for batch processing 10 records.
5. Delete code you do not need. Commented-out code, unused functions, dead feature flags -- remove them. Version control remembers everything.
6. Review for over-engineering during code reviews. Ask teammates: "Is there a simpler way to do this?"
7. Write code for the current requirements. Design for extensibility, but implement only what is needed now.

Quick Summary

Principle	Meaning	Misuse
DRY	One source of truth for each piece of knowledge	Merging unrelated code that happens to look similar
KISS	Choose the simplest correct solution	Avoiding all abstractions; writing spaghetti code
YAGNI	Don't build features until they are needed	Skipping all design and planning

---

Key Points

- DRY is about knowledge duplication, not code duplication. Two identical lines can be acceptable if they represent independent concepts.
- KISS means choosing the right level of complexity -- not the lowest level of complexity.
- YAGNI protects you from building features that may never be used, saving time and reducing maintenance burden.

- The Rule of Three is your guide for when to eliminate duplication: tolerate it until the third occurrence.
- Over-engineering and premature abstraction cause more damage than the problems they try to prevent.
- These three principles work together as a decision framework for every line of code you write.

Practice Questions

1. A colleague duplicates a 15-line validation function in two microservices. They argue that DRY requires creating a shared library. What factors would you consider before agreeing or disagreeing?
2. You are asked to build a reporting feature. The product manager says, "Make it support PDF, CSV, and Excel export." Your analytics show 95% of users only download CSV. How would you apply YAGNI here?
3. A junior developer writes a `GenericRepositoryFactoryProvider<T, ID>` class to wrap a simple database query. The application has exactly one entity type. What would you say in a code review?
4. You find three functions in different modules that each parse a date string in the format "YYYY-MM-DD". Should you extract a shared utility? What questions would you ask first?
5. How does the Rule of Three help prevent premature abstraction? Give an example where extracting after two occurrences would lead to the wrong abstraction.

Exercises

Exercise 1: Identify the Violations

Look at the following code and identify which principles (DRY, KISS, YAGNI) are violated:

```python
class UserManager:
    def create_user(self, name, email):
        # Validate email
        if "@" not in email:
```

```
            raise ValueError("Invalid email")
        user = {"name": name, "email": email}
        self.save_to_db(user)
        self.save_to_cache(user)        # No cache exists yet
        self.save_to_search_index(user)  # No search feature yet
        self.send_to_analytics(user)     # No analytics yet
        self.sync_to_backup_db(user)     # No backup DB yet
        return user

    def update_user(self, user_id, name, email):
        # Validate email (same code as above)
        if "@" not in email:
            raise ValueError("Invalid email")
        user = {"id": user_id, "name": name, "email": email}
        self.save_to_db(user)
        self.save_to_cache(user)
        self.save_to_search_index(user)
        self.send_to_analytics(user)
        self.sync_to_backup_db(user)
        return user
```

Rewrite this code applying all three principles correctly.

Exercise 2: Simplify the Over-Engineered Solution

Refactor this Java code to follow KISS:

```java
public class StringReverserStrategyExecutorService {
    public interface ReverseStrategy {
        String reverse(String input);
    }

    public static class RecursiveReverseStrategy implements ReverseStrategy {
        public String reverse(String input) {
            if (input.isEmpty()) return input;
            return reverse(input.substring(1)) + input.charAt(0);
        }
    }

    public String executeReverse(String input, ReverseStrategy strategy) {
        return strategy.reverse(input);
    }
}
```

Exercise 3: Apply the Rule of Three

You discover the following date formatting code in three different files of your project:

```python
# In order_service.py
order_date = datetime.strptime(date_str, "%Y-%m-%d").strftime("%B %d, %Y")

# In report_generator.py
report_date = datetime.strptime(date_str, "%Y-%m-%d").strftime("%B %d, %Y")

# In notification_service.py
notify_date = datetime.strptime(date_str, "%Y-%m-%d").strftime("%B %d, %Y")
```

This is the third occurrence. Extract an appropriate shared utility. Consider: where should it live? What should it be named? How would you ensure the original callers are updated?

What Is Next?

Now that you understand the principles that guide clean code decisions, the next chapter dives into Code Smells -- the warning signs that tell you something is wrong with your code. You will learn to recognize 13 common code smells, understand why they matter, and know exactly how to fix each one.

Chapter 14: Code Smells -- Recognizing the Warning Signs

What You Will Learn

- What code smells are and why they matter more than bugs
- How to identify the ten most common code smells in real projects
- The difference between a smell and a genuine design problem
- Concrete before-and-after fixes for every smell covered
- When a smell is acceptable and when it demands immediate action
- How smells compound into architectural rot if left unchecked

Why This Chapter Matters

Bugs crash your program. Code smells slowly rot it from the inside. A codebase with many smells still "works," but every change takes longer, every fix introduces new problems, and onboarding new developers feels like teaching them a foreign language. Martin Fowler and Kent Beck cataloged smells as early warning signals -- patterns that suggest deeper design problems. Learning to spot them is the first step toward writing code that stays healthy over months and years.

14.1 What Is a Code Smell?

A code smell is a surface-level indicator that usually corresponds to a deeper problem in the system. It is not a bug. The code compiles, passes tests, and runs correctly. But something about its structure makes it harder to understand, extend, or maintain.

```
+-------------------------------------------------------+
|                  CODE SMELL SPECTRUM                  |
+-------------------------------------------------------+
|                                                       |
|  Clean Code  ------>  Mild Smell  ------>  Deep Rot   |
|                                                       |
|  Easy to read        Takes extra effort    Nobody     |
|  Easy to change       to understand        wants to   |
|  Easy to test        Risky to modify       touch it   |
|                                                       |
+-------------------------------------------------------+
```

The Golden Rule

Not every smell requires immediate action. Use judgment:

- Fix now if you are already working in that area of code
- Note it if you spot it during a code review but the change is unrelated
- Ignore it if the code is stable, rarely touched, and well-tested

---

14.2 Long Method

A method that tries to do too much. When you need comments inside a method to explain what different sections do, you almost certainly have a long method smell.

How to Detect It

- The method is longer than 20 to 30 lines
- You need to scroll to see the whole thing
- It has multiple levels of nesting
- You see comments separating "sections" within the method

BEFORE: Long Method (Java)

```java
public class ReportGenerator {

    public String generateMonthlyReport(List<Sale> sales, int month, int year) {
        // Filter sales for the given month
        List<Sale> filtered = new ArrayList<>();
```

```
        for (Sale sale : sales) {
            if (sale.getDate().getMonthValue() == month
                    && sale.getDate().getYear() == year) {
                filtered.add(sale);
            }
        }

        // Calculate totals
        double totalRevenue = 0;
        double totalTax = 0;
        int totalItems = 0;
        for (Sale sale : filtered) {
            totalRevenue += sale.getAmount();
            totalTax += sale.getAmount() * 0.08;
            totalItems += sale.getItemCount();
        }

        // Find top seller
        Sale topSale = null;
        for (Sale sale : filtered) {
            if (topSale == null || sale.getAmount() > topSale.getAmount()) {
                topSale = sale;
            }
        }

        // Build report string
        StringBuilder report = new StringBuilder();
        report.append("Monthly Report: ").append(month).append("/").append(year).append...
        report.append("Total Revenue: $").append(String.format("%.2f", totalRevenue)).a...
        report.append("Total Tax: $").append(String.format("%.2f", totalTax)).append("\...
        report.append("Total Items Sold: ").append(totalItems).append("\n");
        if (topSale != null) {
            report.append("Top Sale: $").append(String.format("%.2f", topSale.getAmount...
        }
        return report.toString();
    }
}
```

AFTER: Extracted Methods (Java)

```java
public class ReportGenerator {

    public String generateMonthlyReport(List<Sale> sales, int month, int year) {
        List<Sale> monthlySales = filterByMonth(sales, month, year);
        ReportData data = calculateTotals(monthlySales);
        Sale topSale = findTopSale(monthlySales);
        return buildReportString(month, year, data, topSale);
    }

    private List<Sale> filterByMonth(List<Sale> sales, int month, int year) {
```

```
        List<Sale> filtered = new ArrayList<>();
        for (Sale sale : sales) {
            if (sale.getDate().getMonthValue() == month
                    && sale.getDate().getYear() == year) {
                filtered.add(sale);
            }
        }
        return filtered;
    }

    private ReportData calculateTotals(List<Sale> sales) {
        double revenue = 0;
        double tax = 0;
        int items = 0;
        for (Sale sale : sales) {
            revenue += sale.getAmount();
            tax += sale.getAmount() * 0.08;
            items += sale.getItemCount();
        }
        return new ReportData(revenue, tax, items);
    }

    private Sale findTopSale(List<Sale> sales) {
        Sale topSale = null;
        for (Sale sale : sales) {
            if (topSale == null || sale.getAmount() > topSale.getAmount()) {
                topSale = sale;
            }
        }
        return topSale;
    }

    private String buildReportString(int month, int year, ReportData data, Sale topSale) {
        StringBuilder report = new StringBuilder();
        report.append("Monthly Report: ").append(month).append("/").append(year).append...
        report.append("Total Revenue: $").append(String.format("%.2f", data.revenue)).a...
        report.append("Total Tax: $").append(String.format("%.2f", data.tax)).append("\...
        report.append("Total Items Sold: ").append(data.totalItems).append("\n");
        if (topSale != null) {
            report.append("Top Sale: $").append(String.format("%.2f", topSale.getAmount...
        }
        return report.toString();
    }
}
```

BEFORE: Long Method (Python)

```python
def process_order(order_data):
    # Validate
    if not order_data.get("customer_id"):
```

```
        raise ValueError("Missing customer ID")
    if not order_data.get("items"):
        raise ValueError("No items in order")
    for item in order_data["items"]:
        if item["quantity"] <= 0:
            raise ValueError(f"Invalid quantity for {item['name']}")
        if item["price"] < 0:
            raise ValueError(f"Invalid price for {item['name']}")

    # Calculate totals
    subtotal = 0
    for item in order_data["items"]:
        subtotal += item["price"] * item["quantity"]
    tax = subtotal * 0.08
    discount = 0
    if subtotal > 100:
        discount = subtotal * 0.10
    total = subtotal + tax - discount

    # Create order record
    order = {
        "customer_id": order_data["customer_id"],
        "items": order_data["items"],
        "subtotal": subtotal,
        "tax": tax,
        "discount": discount,
        "total": total,
        "status": "pending"
    }
    return order
```

AFTER: Extracted Functions (Python)

```python
def process_order(order_data):
    validate_order(order_data)
    totals = calculate_totals(order_data["items"])
    return create_order_record(order_data, totals)


def validate_order(order_data):
    if not order_data.get("customer_id"):
        raise ValueError("Missing customer ID")
    if not order_data.get("items"):
        raise ValueError("No items in order")
    for item in order_data["items"]:
        if item["quantity"] <= 0:
            raise ValueError(f"Invalid quantity for {item['name']}")
        if item["price"] < 0:
            raise ValueError(f"Invalid price for {item['name']}")
```

```
def calculate_totals(items):
    subtotal = sum(item["price"] * item["quantity"] for item in items)
    tax = subtotal * 0.08
    discount = subtotal * 0.10 if subtotal > 100 else 0
    return {"subtotal": subtotal, "tax": tax, "discount": discount,
            "total": subtotal + tax - discount}


def create_order_record(order_data, totals):
    return {
        "customer_id": order_data["customer_id"],
        "items": order_data["items"],
        **totals,
        "status": "pending"
    }
```

14.3 Large Class

A class that has too many responsibilities. It accumulates fields, methods, and inner classes until it becomes a "god object" that knows everything and does everything.

How to Detect It

- The class has more than 200 to 300 lines
- It has fields that are only used by a subset of its methods
- Its name includes "Manager," "Handler," "Processor," or "Util" with many unrelated methods
- It implements more than two or three unrelated interfaces

BEFORE: Large Class (Java)

```java
public class UserManager {
    private Database db;
    private EmailService emailService;
    private Logger logger;

    public User createUser(String name, String email) { /* ... */ }
    public User findUser(int id) { /* ... */ }
    public void updateUser(User user) { /* ... */ }
    public void deleteUser(int id) { /* ... */ }
```

```
    public void sendWelcomeEmail(User user) { /* ... */ }
    public void sendPasswordReset(User user) { /* ... */ }
    public void sendNewsletter(List<User> users) { /* ... */ }

    public boolean validatePassword(String password) { /* ... */ }
    public String hashPassword(String password) { /* ... */ }
    public boolean checkPassword(String raw, String hashed) { /* ... */ }

    public String generateReport(List<User> users) { /* ... */ }
    public void exportToCsv(List<User> users, String path) { /* ... */ }
}
```

AFTER: Split Into Focused Classes (Java)

```java
public class UserRepository {
    private Database db;

    public User create(String name, String email) { /* ... */ }
    public User findById(int id) { /* ... */ }
    public void update(User user) { /* ... */ }
    public void delete(int id) { /* ... */ }
}

public class UserNotificationService {
    private EmailService emailService;

    public void sendWelcomeEmail(User user) { /* ... */ }
    public void sendPasswordReset(User user) { /* ... */ }
    public void sendNewsletter(List<User> users) { /* ... */ }
}

public class PasswordService {
    public boolean validate(String password) { /* ... */ }
    public String hash(String password) { /* ... */ }
    public boolean verify(String raw, String hashed) { /* ... */ }
}

public class UserReportService {
    public String generateReport(List<User> users) { /* ... */ }
    public void exportToCsv(List<User> users, String path) { /* ... */ }
}
```

```
  +------------------+         +--------------------+
  |   UserManager    |         |  UserRepository    |
  |  (God Object)    |  --->   +--------------------+
  |                  |         | UserNotification   |
  |  12+ methods     |         +--------------------+
  |  4 concerns      |         | PasswordService    |
```

```
  |                  |         +--------------------+
  +------------------+         | UserReportService  |
                               +--------------------+
      BEFORE                         AFTER
```

BEFORE: Large Class (Python)

```python
class ShopManager:
    def __init__(self, db):
        self.db = db

    # Product methods
    def add_product(self, name, price): ...
    def remove_product(self, product_id): ...
    def update_price(self, product_id, new_price): ...

    # Order methods
    def create_order(self, customer_id, items): ...
    def cancel_order(self, order_id): ...
    def get_order_status(self, order_id): ...

    # Customer methods
    def register_customer(self, name, email): ...
    def deactivate_customer(self, customer_id): ...

    # Reporting methods
    def daily_sales_report(self): ...
    def inventory_report(self): ...
```

AFTER: Focused Classes (Python)

```python
class ProductCatalog:
    def __init__(self, db):
        self.db = db

    def add(self, name, price): ...
    def remove(self, product_id): ...
    def update_price(self, product_id, new_price): ...


class OrderService:
    def __init__(self, db):
        self.db = db

    def create(self, customer_id, items): ...
    def cancel(self, order_id): ...
```

```
    def get_status(self, order_id): ...


class CustomerService:
    def __init__(self, db):
        self.db = db

    def register(self, name, email): ...
    def deactivate(self, customer_id): ...


class ReportGenerator:
    def __init__(self, db):
        self.db = db

    def daily_sales(self): ...
    def inventory(self): ...
```

14.4 Feature Envy

A method that uses data from another class more than from its own class. It "envies" the features of the other class and should probably live there instead.

How to Detect It

- A method calls multiple getters on another object
- It accesses little or nothing from its own class
- Moving the method to the other class would eliminate multiple getter calls

BEFORE: Feature Envy (Java)

```java
public class OrderPrinter {

    public String formatOrder(Order order) {
        StringBuilder sb = new StringBuilder();
        sb.append("Customer: ").append(order.getCustomer().getName()).append("\n");
        sb.append("Email: ").append(order.getCustomer().getEmail()).append("\n");
        sb.append("Address: ").append(order.getCustomer().getAddress().getStreet())
          .append(", ").append(order.getCustomer().getAddress().getCity())
          .append(", ").append(order.getCustomer().getAddress().getZip()).append("\n");
        sb.append("Total: $").append(String.format("%.2f", order.getTotal()));
        return sb.toString();
```

```
    }
}
```

The method calls `order.getCustomer()` four times and digs into `getAddress()` three times. It clearly envies the Customer and Address classes.

AFTER: Move Behavior to the Right Class (Java)

```java
public class Customer {
    private String name;
    private String email;
    private Address address;

    public String formatContactInfo() {
        return "Customer: " + name + "\n"
             + "Email: " + email + "\n"
             + "Address: " + address.format();
    }
}

public class Address {
    private String street;
    private String city;
    private String zip;

    public String format() {
        return street + ", " + city + ", " + zip;
    }
}

public class OrderPrinter {

    public String formatOrder(Order order) {
        return order.getCustomer().formatContactInfo() + "\n"
             + "Total: $" + String.format("%.2f", order.getTotal());
    }
}
```

BEFORE: Feature Envy (Python)

```python
class InvoiceMailer:

    def build_email_body(self, invoice):
        customer = invoice.customer
        lines = [
            f"Dear {customer.first_name} {customer.last_name},",
```

```
            f"Your invoice #{invoice.number} is ready.",
            f"Amount: ${invoice.amount:.2f}",
            f"Due: {invoice.due_date.strftime('%B %d, %Y')}",
            f"Send payment to: {customer.payment_address}",
        ]
        return "\n".join(lines)
```

AFTER: Let the Objects Format Themselves (Python)

```python
class Customer:
    def __init__(self, first_name, last_name, payment_address):
        self.first_name = first_name
        self.last_name = last_name
        self.payment_address = payment_address

    def full_name(self):
        return f"{self.first_name} {self.last_name}"


class Invoice:
    def __init__(self, number, amount, due_date, customer):
        self.number = number
        self.amount = amount
        self.due_date = due_date
        self.customer = customer

    def summary(self):
        return (f"Invoice #{self.number}\n"
                f"Amount: ${self.amount:.2f}\n"
                f"Due: {self.due_date.strftime('%B %d, %Y')}")


class InvoiceMailer:

    def build_email_body(self, invoice):
        return (f"Dear {invoice.customer.full_name()},\n"
                f"{invoice.summary()}\n"
                f"Send payment to: {invoice.customer.payment_address}")
```

14.5 Data Clumps

Groups of data items that travel together across multiple methods or classes. If you always pass the same three parameters together, they probably belong in their own object.

How to Detect It

- The same group of parameters appears in multiple method signatures
- Multiple classes have the same group of fields
- You find yourself passing the same variables together repeatedly

BEFORE: Data Clumps (Java)

```java
public class ShippingCalculator {

    public double calculate(String street, String city, String state, String zip,
                            double weight) {
        double distance = lookupDistance(city, state, zip);
        return distance * weight * 0.05;
    }

    public boolean validateAddress(String street, String city, String state,
                                   String zip) {
        return street != null && city != null && state != null && zip != null;
    }

    public String formatLabel(String name, String street, String city,
                              String state, String zip) {
        return name + "\n" + street + "\n" + city + ", " + state + " " + zip;
    }
}
```

The four address fields appear in every method. They form a data clump.

AFTER: Introduce a Parameter Object (Java)

```java
public class Address {
    private final String street;
    private final String city;
    private final String state;
    private final String zip;

    public Address(String street, String city, String state, String zip) {
        this.street = street;
        this.city = city;
        this.state = state;
        this.zip = zip;
    }

    public boolean isValid() {
        return street != null && city != null && state != null && zip != null;
    }
```

```
    public String format() {
        return street + "\n" + city + ", " + state + " " + zip;
    }

    // Getters omitted for brevity
}

public class ShippingCalculator {

    public double calculate(Address address, double weight) {
        double distance = lookupDistance(address);
        return distance * weight * 0.05;
    }

    public String formatLabel(String name, Address address) {
        return name + "\n" + address.format();
    }
}
```

BEFORE: Data Clumps (Python)

```python
def create_event(title, start_year, start_month, start_day,
                 end_year, end_month, end_day):
    ...

def is_event_today(start_year, start_month, start_day):
    ...

def format_date_range(start_year, start_month, start_day,
                      end_year, end_month, end_day):
    ...
```

AFTER: Use a Data Class (Python)

```python
from dataclasses import dataclass
from datetime import date


@dataclass
class DateRange:
    start: date
    end: date

    def includes_today(self):
        return self.start <= date.today() <= self.end
```

```
    def format(self):
        return f"{self.start.strftime('%b %d')} - {self.end.strftime('%b %d, %Y')}"


def create_event(title, date_range: DateRange):
    ...

def format_date_range(date_range: DateRange):
    return date_range.format()
```

14.6 Primitive Obsession

Using primitive types (strings, integers, booleans) instead of small objects to represent domain concepts. An email address is not just a string. Money is not just a double. A phone number is not just an integer.

How to Detect It

- String variables with names like `email`, `phone`, `currency`, `zipCode`
- Validation logic for the same primitive repeated across the codebase
- Methods that accept a `String` or `int` when the value has specific rules

BEFORE: Primitive Obsession (Java)

```java
public class User {
    private String email;       // Just a string?
    private String phoneNumber; // Any format?
    private int age;            // Negative age?

    public void setEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new IllegalArgumentException("Invalid email");
        }
        this.email = email;
    }

    public void setPhoneNumber(String phone) {
        if (phone == null || phone.length() < 10) {
            throw new IllegalArgumentException("Invalid phone");
        }
        this.phoneNumber = phone;
```

```
    }

    public void setAge(int age) {
        if (age < 0 || age > 150) {
            throw new IllegalArgumentException("Invalid age");
        }
        this.age = age;
    }
}
```

Every place that uses these values must repeat the same validation logic.

AFTER: Value Objects (Java)

```java
public class Email {
    private final String value;

    public Email(String value) {
        if (value == null || !value.contains("@")) {
            throw new IllegalArgumentException("Invalid email: " + value);
        }
        this.value = value.toLowerCase().trim();
    }

    public String getValue() { return value; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Email)) return false;
        return value.equals(((Email) o).value);
    }

    @Override
    public int hashCode() { return value.hashCode(); }
}

public class PhoneNumber {
    private final String value;

    public PhoneNumber(String value) {
        String digits = value.replaceAll("[^0-9]", "");
        if (digits.length() < 10 || digits.length() > 15) {
            throw new IllegalArgumentException("Invalid phone number");
        }
        this.value = digits;
    }

    public String format() {
        return "(" + value.substring(0, 3) + ") "
```

```
                + value.substring(3, 6) + "-" + value.substring(6);
    }
}

public class User {
    private Email email;
    private PhoneNumber phoneNumber;
    private Age age;

    // Validation is now handled by the value objects themselves
}
```

BEFORE: Primitive Obsession (Python)

```python
def transfer_money(from_account, to_account, amount, currency):
    if currency not in ("USD", "EUR", "GBP"):
        raise ValueError("Unsupported currency")
    if amount <= 0:
        raise ValueError("Amount must be positive")
    # Transfer logic...
```

AFTER: Value Objects (Python)

```python
from dataclasses import dataclass
from enum import Enum


class Currency(Enum):
    USD = "USD"
    EUR = "EUR"
    GBP = "GBP"


@dataclass(frozen=True)
class Money:
    amount: float
    currency: Currency

    def __post_init__(self):
        if self.amount <= 0:
            raise ValueError("Amount must be positive")

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("Cannot add different currencies")
        return Money(self.amount + other.amount, self.currency)
```

```
    def __str__(self):
        symbols = {Currency.USD: "$", Currency.EUR: "EUR", Currency.GBP: "GBP"}
        return f"{symbols[self.currency]}{self.amount:.2f}"


def transfer_money(from_account, to_account, money: Money):
    # No validation needed -- Money enforces its own rules
    ...
```

14.7 Switch Statements (and Long If-Else Chains)

A switch or long if-else chain that selects behavior based on a type code. If the same switch appears in multiple places, adding a new type means updating every switch.

How to Detect It

- The same `switch` or `if-else` chain appears in more than one method
- Cases are added frequently as new types are introduced
- Each branch does something significantly different

BEFORE: Switch Statement Smell (Java)

```java
public class PaymentProcessor {

    public double calculateFee(String paymentType, double amount) {
        switch (paymentType) {
            case "CREDIT_CARD": return amount * 0.03;
            case "DEBIT_CARD":  return amount * 0.01;
            case "BANK_TRANSFER": return 1.50;
            case "PAYPAL":      return amount * 0.029 + 0.30;
            default: throw new IllegalArgumentException("Unknown type");
        }
    }

    public String getConfirmationMessage(String paymentType) {
        switch (paymentType) {
            case "CREDIT_CARD": return "Charged to your credit card.";
            case "DEBIT_CARD":  return "Debited from your bank account.";
            case "BANK_TRANSFER": return "Transfer initiated.";
            case "PAYPAL":      return "PayPal payment confirmed.";
            default: throw new IllegalArgumentException("Unknown type");
```

```
        }
    }
}
```

AFTER: Replace with Polymorphism (Java)

```java
public interface PaymentMethod {
    double calculateFee(double amount);
    String getConfirmationMessage();
}

public class CreditCardPayment implements PaymentMethod {
    public double calculateFee(double amount) { return amount * 0.03; }
    public String getConfirmationMessage() { return "Charged to your credit card."; }
}

public class DebitCardPayment implements PaymentMethod {
    public double calculateFee(double amount) { return amount * 0.01; }
    public String getConfirmationMessage() { return "Debited from your bank account."; }
}

public class BankTransferPayment implements PaymentMethod {
    public double calculateFee(double amount) { return 1.50; }
    public String getConfirmationMessage() { return "Transfer initiated."; }
}

public class PayPalPayment implements PaymentMethod {
    public double calculateFee(double amount) { return amount * 0.029 + 0.30; }
    public String getConfirmationMessage() { return "PayPal payment confirmed."; }
}

// Usage: paymentMethod.calculateFee(amount);
// Adding a new type = adding a new class. No existing code changes.
```

BEFORE: Long If-Else Chain (Python)

```python
def get_shipping_cost(method, weight):
    if method == "standard":
        return 5.99 + weight * 0.50
    elif method == "express":
        return 12.99 + weight * 0.75
    elif method == "overnight":
        return 24.99 + weight * 1.00
    elif method == "drone":
        return 19.99 + weight * 1.25
    else:
        raise ValueError(f"Unknown shipping method: {method}")
```

AFTER: Strategy Pattern (Python)

```python
from dataclasses import dataclass
from abc import ABC, abstractmethod


class ShippingStrategy(ABC):
    @abstractmethod
    def cost(self, weight: float) -> float: ...


class StandardShipping(ShippingStrategy):
    def cost(self, weight): return 5.99 + weight * 0.50

class ExpressShipping(ShippingStrategy):
    def cost(self, weight): return 12.99 + weight * 0.75

class OvernightShipping(ShippingStrategy):
    def cost(self, weight): return 24.99 + weight * 1.00

class DroneShipping(ShippingStrategy):
    def cost(self, weight): return 19.99 + weight * 1.25


STRATEGIES = {
    "standard": StandardShipping(),
    "express": ExpressShipping(),
    "overnight": OvernightShipping(),
    "drone": DroneShipping(),
}

def get_shipping_cost(method: str, weight: float) -> float:
    strategy = STRATEGIES.get(method)
    if not strategy:
        raise ValueError(f"Unknown shipping method: {method}")
    return strategy.cost(weight)
```

14.8 Lazy Class

A class that does too little to justify its existence. It might have been created "just in case" or left over after a refactoring removed most of its responsibilities.

How to Detect It

- The class has only one or two trivial methods
- It delegates everything to another class

- Removing it and inlining its logic would simplify the codebase

BEFORE: Lazy Class (Java)

```java
public class StringFormatter {
    public String toUpperCase(String input) {
        return input.toUpperCase();
    }
}

// Used as:
StringFormatter formatter = new StringFormatter();
String result = formatter.toUpperCase(name);
```

This class adds nothing over using `String.toUpperCase()` directly.

AFTER: Inline the Class (Java)

```java
// Just use the method directly
String result = name.toUpperCase();
```

When Lazy Classes Are Acceptable

Sometimes a thin wrapper class exists for a good reason:

- It represents a domain concept (even if the implementation is trivial today)
- It provides a seam for testing
- It is part of an interface you do not control

BEFORE: Lazy Class (Python)

```python
class MathHelper:
    def add(self, a, b):
        return a + b

    def multiply(self, a, b):
        return a * b
```

AFTER: Just Use the Operators (Python)

```python
# No class needed. Use a + b and a * b directly.
total = price + tax
cost = quantity * unit_price
```

14.9 Speculative Generality

Code that was written "just in case" someone might need it in the future. Abstract classes with a single subclass, parameters that are never used, hooks that nobody calls.

How to Detect It

- Abstract classes or interfaces with only one implementation
- Methods with unused parameters
- Code paths protected by flags that are never toggled
- Complex frameworks wrapping simple operations

BEFORE: Speculative Generality (Java)

```java
public abstract class AbstractDataProcessor<T, R, E extends Exception> {
    protected abstract R preProcess(T input) throws E;
    protected abstract R process(R preprocessed) throws E;
    protected abstract R postProcess(R processed) throws E;
    protected abstract void onError(E error);
    protected abstract void onComplete(R result);

    public final R execute(T input) throws E {
        try {
            R step1 = preProcess(input);
            R step2 = process(step1);
            R result = postProcess(step2);
            onComplete(result);
            return result;
        } catch (Exception e) {
            onError((E) e);
            throw e;
        }
    }
}

// The only implementation:
public class CsvParser extends AbstractDataProcessor<String, List<String[]>, IOExceptio...
    // ... implements all five abstract methods for one simple use case
}
```

AFTER: Just Write What You Need (Java)

```java
public class CsvParser {

    public List<String[]> parse(String filePath) throws IOException {
        List<String[]> rows = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(filePath))) {
            String line;
            while ((line = reader.readLine()) != null) {
                rows.add(line.split(","));
            }
        }
        return rows;
    }
}

// If you later need a second parser, THEN consider an abstraction.
```

BEFORE: Speculative Generality (Python)

```python
class BaseNotifier(ABC):
    @abstractmethod
    def validate_recipient(self, recipient): ...

    @abstractmethod
    def format_message(self, message): ...

    @abstractmethod
    def send(self, recipient, message): ...

    @abstractmethod
    def on_success(self, recipient): ...

    @abstractmethod
    def on_failure(self, recipient, error): ...


# Only implementation:
class EmailNotifier(BaseNotifier):
    # Must implement all five methods for a simple email
    ...
```

AFTER: Simple and Direct (Python)

```python
class EmailNotifier:
    def __init__(self, smtp_client):
        self.smtp_client = smtp_client

    def send(self, to_email, subject, body):
        self.smtp_client.send_message(to_email, subject, body)

# Add an abstraction layer ONLY when you have a second notifier type.
```

14.10 Message Chains

A series of calls like `a.getB().getC().getD().doSomething()`. The caller must know the entire chain of object relationships, creating tight coupling to the internal structure.

How to Detect It

- Chains of three or more getter calls: `obj.getA().getB().getC()`
- Any change to an intermediate object breaks all callers
- The chain exposes internal structure that should be hidden

BEFORE: Message Chain (Java)

```java
public class OrderShipper {

    public String getShippingLabel(Order order) {
        String street = order.getCustomer().getAddress().getStreet();
        String city = order.getCustomer().getAddress().getCity();
        String zip = order.getCustomer().getAddress().getZipCode();
        String country = order.getCustomer().getAddress().getCountry();
        return street + "\n" + city + " " + zip + "\n" + country;
    }
}
```

AFTER: Hide the Chain (Java)

```java
public class Order {
    private Customer customer;
```

```
    public String getShippingLabel() {
        return customer.getShippingLabel();
    }
}

public class Customer {
    private Address address;

    public String getShippingLabel() {
        return address.format();
    }
}

public class Address {
    private String street, city, zipCode, country;

    public String format() {
        return street + "\n" + city + " " + zipCode + "\n" + country;
    }
}

// Now the caller only needs:
public class OrderShipper {
    public String getShippingLabel(Order order) {
        return order.getShippingLabel();
    }
}
```

BEFORE: Message Chain (Python)

```python
# Reaching deep into nested objects
city = company.get_ceo().get_assistant().get_contact_info().get_address().city
```

AFTER: Ask, Don't Dig (Python)

```python
# Each object exposes only what callers need
class Company:
    def ceo_assistant_city(self):
        return self._ceo.assistant_city()

class CEO:
    def assistant_city(self):
        return self._assistant.city()

class Assistant:
    def city(self):
```

```
        return self._contact_info.city

# Caller:
city = company.ceo_assistant_city()
```

> Note: The Law of Demeter says "only talk to your immediate friends." Message chains violate this by talking to friends of friends of friends.

14.11 Middle Man

The opposite of message chains. A class that delegates almost everything to another class, adding no value of its own. It exists only to forward calls.

How to Detect It

- Most methods in the class just call the same method on another object
- The class adds no logic, validation, or transformation
- Callers would be just as well off talking directly to the delegate

BEFORE: Middle Man (Java)

```java
public class Department {
    private Manager manager;

    public Employee getWorker(int id)   { return manager.getWorker(id); }
    public String getBudget()           { return manager.getBudget(); }
    public List<Project> getProjects()  { return manager.getProjects(); }
    public void addProject(Project p)   { manager.addProject(p); }
    public void removeProject(int id)   { manager.removeProject(id); }
}
```

Every method just forwards to `manager`. The `Department` class is pure overhead.

AFTER: Remove the Middle Man (Java)

```java
// Let callers talk to the manager directly
Manager manager = department.getManager();
manager.getWorker(id);
manager.getBudget();
```

Or, if `Department` should exist as a concept, give it real behavior:

```java
public class Department {
    private Manager manager;
    private String name;
    private Budget budget;

    public boolean isOverBudget() {
        return manager.getSpending() > budget.getLimit();
    }

    public List<Employee> getAvailableWorkers() {
        return manager.getWorkers().stream()
                .filter(w -> !w.isOnLeave())
                .collect(Collectors.toList());
    }
}
```

BEFORE: Middle Man (Python)

```python
class Facade:
    def __init__(self, service):
        self._service = service

    def get_data(self):
        return self._service.get_data()

    def save_data(self, data):
        return self._service.save_data(data)

    def delete_data(self, id):
        return self._service.delete_data(id)
```

AFTER: Use the Service Directly (Python)

```python
# Instead of creating a Facade, just use the service:
service = DataService()
service.get_data()
service.save_data(data)
```

14.12 Smell Interaction Map

Smells rarely appear alone. One smell often leads to or masks another:

```
+-------------------+       +---------------------+
|   Long Method     | ----> | Feature Envy        |
+-------------------+       +---------------------+
        |                           |
        v                           v
+-------------------+       +---------------------+
|   Large Class     | ----> | Data Clumps         |
+-------------------+       +---------------------+
        |                           |
        v                           v
+-------------------+       +---------------------+
| Switch Statements | ----> | Primitive Obsession |
+-------------------+       +---------------------+
        |
        v
+-------------------+
|  Speculative       |
|  Generality        |
+-------------------+

Arrows indicate: "often leads to" or "co-occurs with"
```

Common Mistakes

1. Treating all smells as equally urgent. Some smells in rarely-touched code are harmless. Focus on smells in code you change frequently.
2. Refactoring smells without tests. Always have test coverage before you start moving code around. Refactoring without tests is gambling.
3. Over-correcting a smell. Splitting a 25-line method into ten 3-line methods can make code harder to follow, not easier.
4. Creating value objects for everything. Not every string needs its own class. Apply primitive obsession fixes where validation and formatting are genuinely repeated.
5. Confusing delegation with the middle man smell. Delegation is healthy when the delegating class adds value. It is a smell only when it adds nothing.

Best Practices

1. Use the Boy Scout Rule. Leave code cleaner than you found it. Fix one smell each time you visit a file.
2. Track smells in code reviews. Mention smells as suggestions, not blockers, unless they introduce risk.
3. Prioritize smells in hot spots. Use version control history to find frequently changed files -- those benefit most from smell removal.
4. Refactor in small steps. Address one smell at a time. Run tests after each step.
5. Document intentional smells. If a smell exists for a good reason, add a comment explaining why.

Quick Summary

Smell	Core Problem	Primary Fix
Long Method	Too many responsibilities in one method	Extract Method
Large Class	Too many responsibilities in one class	Extract Class
Feature Envy	Method uses another class's data excessively	Move Method
Data Clumps	Same group of values passed together repeatedly	Introduce Parameter Object
Primitive Obsession	Using primitives instead of domain types	Create Value Objects
Switch Statements	Type-based branching duplicated across methods	Replace with Polymorphism
Lazy Class	Class does too little to justify existence	Inline Class
Speculative Generality	Abstractions built for hypothetical future needs	Remove unused abstractions
Message Chains	Long chains of getter calls exposing internals	Hide Delegate
Middle Man	Class delegates everything without adding value	Remove Middle Man

Key Points

- A code smell is a surface indicator of a deeper design problem, not a bug.
- The ten smells covered here account for the vast majority of structural problems in real codebases.
- Not every smell needs fixing. Prioritize smells in frequently changed code.
- Most smell fixes involve moving behavior to where it belongs (closer to the data it uses).
- Smells compound over time. A long method inside a large class with feature envy is exponentially harder to fix than any single smell alone.

Practice Questions

1. You find a method that calls `customer.getProfile().getPreferences().getNotificationSettings().isEmailEnabled()`. Which smell is this, and how would you fix it?
2. A class called `ApplicationHelper` has 45 methods covering logging, email, file I/O, and date formatting. Which smell does it exhibit? What is your first step to fix it?
3. Three methods each accept parameters `(double latitude, double longitude, double altitude)`. Which smell is this? Write a value object class to fix it.
4. You see an abstract class `AbstractValidator<T>` with exactly one subclass: `EmailValidator`. The team says they "might need more validators later." Should you keep the abstract class or remove it? Explain your reasoning.
5. What is the difference between the middle man smell and the message chain smell? Can they exist in the same codebase simultaneously?

Exercises

Exercise 1: Smell Safari

Take a codebase you work with (or an open-source project) and find at least three different code smells from this chapter. For each one, write down:

- Which smell it is

- Why it qualifies as that smell
- What refactoring you would apply
- Whether it is urgent or can wait

Exercise 2: Fix the Smells

Given this code, identify ALL the smells and refactor it:

```java
public class EverythingManager {
    public String processTransaction(String type, String customerName,
            String customerEmail, String customerPhone,
            double amount, String currency) {
        // Validate customer
        if (customerName == null) throw new RuntimeException("No name");
        if (!customerEmail.contains("@")) throw new RuntimeException("Bad email");

        double fee;
        if (type.equals("purchase")) {
            fee = amount * 0.02;
        } else if (type.equals("refund")) {
            fee = 0;
        } else if (type.equals("transfer")) {
            fee = 1.50;
        } else {
            throw new RuntimeException("Unknown type");
        }

        double total = amount + fee;
        String confirmation = "Transaction: " + type + "\n"
            + "Customer: " + customerName + "\n"
            + "Email: " + customerEmail + "\n"
            + "Phone: " + customerPhone + "\n"
            + "Amount: " + currency + amount + "\n"
            + "Fee: " + currency + fee + "\n"
            + "Total: " + currency + total;
        return confirmation;
    }
}
```

Identify: long method, data clumps (customer fields), primitive obsession (email, phone, money), switch statement smell, and large class tendencies.

Exercise 3: Smell or Not a Smell?

For each scenario, decide whether it is a code smell and explain why or why not:

1. A `Logger` class that wraps `System.out.println` with timestamp formatting
2. A 50-line method that performs a single complex mathematical calculation with no branches

3. An `enum` with a switch statement that handles all cases in exactly one place in the codebase

---

What Is Next?

Now that you can recognize code smells, the next chapter teaches you the specific techniques to fix them. Chapter 15: Refactoring Techniques gives you a toolbox of safe, systematic transformations that turn smelly code into clean code -- one step at a time.

Chapter 15: Refactoring Techniques -- Your Toolbox for Clean Code

What You Will Learn

- Seven essential refactoring techniques that solve the most common code smells
- How to apply each technique safely with step-by-step before-and-after examples
- The critical role of tests in making refactoring safe
- When each technique is appropriate and when it is overkill
- How to chain multiple techniques to transform messy code into clean code
- Refactoring in both Java and Python with idiomatic patterns for each language

Why This Chapter Matters

Recognizing code smells (Chapter 14) is only half the battle. You also need the skill to fix them safely. Refactoring is the disciplined process of restructuring existing code without changing its external behavior. The key word is "disciplined" -- refactoring is not hacking at code until it looks different. Each technique here has a clear intent, a clear procedure, and clear before-and-after states. Master these seven techniques, and you can handle the vast majority of code quality improvements you will encounter in your career.

The Safety Net: Tests Before Refactoring

Before applying any refactoring technique, you need a safety net.

```
+-----------------------------------------------+
|         THE REFACTORING SAFETY RULE           |
+-----------------------------------------------+
|                                               |
|  1. Verify existing tests pass                |
|  2. Apply ONE refactoring step                |
|  3. Run tests again                           |
|  4. If tests pass --> commit                  |
|  5. If tests fail --> revert and try again    |
|                                               |
|  NEVER skip step 1. NEVER combine steps.      |
+-----------------------------------------------+
```

If there are no tests, write characterization tests first. These tests capture the current behavior (even if it is wrong) so you can verify that your refactoring does not change it.

15.1 Extract Method

Intent

Take a fragment of code inside a long method and turn it into its own method with a descriptive name.

When to Use It

- A method is too long to understand at a glance
- A comment explains what the next block of code does (the method name replaces the comment)
- The same code fragment appears in multiple places

BEFORE: Java

```java
public class InvoiceProcessor {

    public void printInvoice(Invoice invoice) {
        System.out.println("=== INVOICE ===");
        System.out.println("Date: " + invoice.getDate());

        // Print line items
        double total = 0;
```

```
        for (LineItem item : invoice.getItems()) {
            double lineTotal = item.getQuantity() * item.getPrice();
            System.out.println(item.getName() + " x" + item.getQuantity()
                    + " = $" + String.format("%.2f", lineTotal));
            total += lineTotal;
        }

        // Apply discount
        double discount = 0;
        if (total > 500) {
            discount = total * 0.10;
        } else if (total > 200) {
            discount = total * 0.05;
        }

        // Print summary
        System.out.println("Subtotal: $" + String.format("%.2f", total));
        System.out.println("Discount: $" + String.format("%.2f", discount));
        System.out.println("Total: $" + String.format("%.2f", total - discount));
    }
}
```

AFTER: Java

```java
public class InvoiceProcessor {

    public void printInvoice(Invoice invoice) {
        printHeader(invoice);
        double subtotal = printLineItems(invoice.getItems());
        double discount = calculateDiscount(subtotal);
        printSummary(subtotal, discount);
    }

    private void printHeader(Invoice invoice) {
        System.out.println("=== INVOICE ===");
        System.out.println("Date: " + invoice.getDate());
    }

    private double printLineItems(List<LineItem> items) {
        double total = 0;
        for (LineItem item : items) {
            double lineTotal = item.getQuantity() * item.getPrice();
            System.out.println(item.getName() + " x" + item.getQuantity()
                    + " = $" + String.format("%.2f", lineTotal));
            total += lineTotal;
        }
        return total;
    }

    private double calculateDiscount(double total) {
```

```
        if (total > 500) return total * 0.10;
        if (total > 200) return total * 0.05;
        return 0;
    }

    private void printSummary(double subtotal, double discount) {
        System.out.println("Subtotal: $" + String.format("%.2f", subtotal));
        System.out.println("Discount: $" + String.format("%.2f", discount));
        System.out.println("Total: $" + String.format("%.2f", subtotal - discount));
    }
}
```

BEFORE: Python

```python
def generate_report(employees):
    report_lines = []

    # Calculate statistics
    total_salary = sum(e.salary for e in employees)
    avg_salary = total_salary / len(employees) if employees else 0
    max_salary = max((e.salary for e in employees), default=0)
    min_salary = min((e.salary for e in employees), default=0)

    # Group by department
    departments = {}
    for emp in employees:
        dept = emp.department
        if dept not in departments:
            departments[dept] = []
        departments[dept].append(emp)

    # Build report
    report_lines.append(f"Total Employees: {len(employees)}")
    report_lines.append(f"Average Salary: ${avg_salary:,.2f}")
    report_lines.append(f"Salary Range: ${min_salary:,.2f} - ${max_salary:,.2f}")
    report_lines.append("")
    for dept, members in sorted(departments.items()):
        report_lines.append(f"  {dept}: {len(members)} employees")

    return "\n".join(report_lines)
```

AFTER: Python

```python
def generate_report(employees):
    stats = calculate_salary_stats(employees)
    departments = group_by_department(employees)
```

```
    return build_report(stats, departments, len(employees))


def calculate_salary_stats(employees):
    if not employees:
        return {"total": 0, "average": 0, "max": 0, "min": 0}
    salaries = [e.salary for e in employees]
    total = sum(salaries)
    return {
        "total": total,
        "average": total / len(employees),
        "max": max(salaries),
        "min": min(salaries),
    }


def group_by_department(employees):
    departments = {}
    for emp in employees:
        departments.setdefault(emp.department, []).append(emp)
    return departments


def build_report(stats, departments, count):
    lines = [
        f"Total Employees: {count}",
        f"Average Salary: ${stats['average']:,.2f}",
        f"Salary Range: ${stats['min']:,.2f} - ${stats['max']:,.2f}",
        "",
    ]
    for dept, members in sorted(departments.items()):
        lines.append(f"  {dept}: {len(members)} employees")
    return "\n".join(lines)
```

Step-by-Step Procedure

1. Create a new method with a name that describes what the code does, not how
2. Copy the code fragment into the new method
3. Identify local variables used in the fragment -- pass them as parameters
4. Identify local variables modified in the fragment -- return them
5. Replace the original fragment with a call to the new method
6. Run tests

15.2 Extract Class

Intent

Take a class that is doing the work of two and split it into two classes, each with a single responsibility.

When to Use It

- A class has fields or methods that form a natural subgroup
- You can name the subgroup with a meaningful domain term
- The class has more than one reason to change

BEFORE: Java

```java
public class Employee {
    private String name;
    private String employeeId;

    // Address fields -- these belong together
    private String street;
    private String city;
    private String state;
    private String zipCode;

    // Phone fields -- these belong together
    private String areaCode;
    private String phoneNumber;

    public String getFullAddress() {
        return street + "\n" + city + ", " + state + " " + zipCode;
    }

    public String getFormattedPhone() {
        return "(" + areaCode + ") " + phoneNumber;
    }

    // ... employee-specific methods
}
```

AFTER: Java

```java
public class Address {
    private String street;
    private String city;
    private String state;
    private String zipCode;
```

```
    public Address(String street, String city, String state, String zipCode) {
        this.street = street;
        this.city = city;
        this.state = state;
        this.zipCode = zipCode;
    }

    public String format() {
        return street + "\n" + city + ", " + state + " " + zipCode;
    }
}

public class PhoneNumber {
    private String areaCode;
    private String number;

    public PhoneNumber(String areaCode, String number) {
        this.areaCode = areaCode;
        this.number = number;
    }

    public String format() {
        return "(" + areaCode + ") " + number;
    }
}

public class Employee {
    private String name;
    private String employeeId;
    private Address address;
    private PhoneNumber phone;

    public String getFullAddress() { return address.format(); }
    public String getFormattedPhone() { return phone.format(); }
}
```

BEFORE: Python

```python
class Order:
    def __init__(self, order_id, customer_name, customer_email,
                 customer_tier, items):
        self.order_id = order_id
        self.customer_name = customer_name
        self.customer_email = customer_email
        self.customer_tier = customer_tier
        self.items = items

    def customer_display(self):
        return f"{self.customer_name} ({self.customer_email})"
```

```
    def customer_discount_rate(self):
        rates = {"gold": 0.15, "silver": 0.10, "bronze": 0.05}
        return rates.get(self.customer_tier, 0)

    def total(self):
        raw = sum(i.price * i.quantity for i in self.items)
        return raw * (1 - self.customer_discount_rate())
```

AFTER: Python

```python
from dataclasses import dataclass


@dataclass
class Customer:
    name: str
    email: str
    tier: str

    def display(self):
        return f"{self.name} ({self.email})"

    def discount_rate(self):
        rates = {"gold": 0.15, "silver": 0.10, "bronze": 0.05}
        return rates.get(self.tier, 0)


class Order:
    def __init__(self, order_id, customer: Customer, items):
        self.order_id = order_id
        self.customer = customer
        self.items = items

    def total(self):
        raw = sum(i.price * i.quantity for i in self.items)
        return raw * (1 - self.customer.discount_rate())
```

Step-by-Step Procedure

1. Identify the group of fields and methods that belong together
2. Create a new class with a descriptive name
3. Move the related fields to the new class
4. Move the related methods to the new class
5. Replace the old fields with a single reference to the new class
6. Update all callers to use the new structure

7. Run tests

15.3 Rename (Method, Variable, Class)

Intent

Change the name of a method, variable, or class to better communicate its purpose. This is the simplest refactoring, yet one of the most impactful.

When to Use It

- A name is misleading, abbreviated, or unclear
- You had to read the implementation to understand what a method does
- A name no longer reflects what the code actually does after changes

BEFORE: Java

```java
public class Proc {
    private List<Map<String, Object>> d;

    public double calc(Map<String, Object> r) {
        double t = 0;
        for (Map<String, Object> i : d) {
            double p = (double) i.get("p");
            int q = (int) i.get("q");
            t += p * q;
        }
        double tx = t * 0.08;
        return t + tx;
    }

    public boolean chk(String s) {
        return s != null && s.length() > 0;
    }
}
```

AFTER: Java

```java
public class OrderCalculator {
    private List<Map<String, Object>> lineItems;

    public double calculateTotal(Map<String, Object> order) {
        double subtotal = 0;
        for (Map<String, Object> item : lineItems) {
            double price = (double) item.get("price");
            int quantity = (int) item.get("quantity");
            subtotal += price * quantity;
        }
        double tax = subtotal * 0.08;
        return subtotal + tax;
    }

    public boolean isNotEmpty(String value) {
        return value != null && value.length() > 0;
    }
}
```

BEFORE: Python

```python
def proc(lst, n):
    r = []
    for x in lst:
        if x.s == n:
            r.append(x)
    return r

def fmt(u):
    return f"{u.fn} {u.ln} <{u.e}>"
```

AFTER: Python

```python
def filter_by_status(tasks, status):
    return [task for task in tasks if task.status == status]

def format_user_display(user):
    return f"{user.first_name} {user.last_name} <{user.email}>"
```

Naming Guidelines

```
  +-------------------------------------------------------+
  |              GOOD NAMING CHECKLIST                    |
  +-------------------------------------------------------+
```

```
  |                                                      |
  |  [x]  Reveals intent: calculateTotal, not calc       |
  |  [x]  Uses domain language: Invoice, not DataObj     |
  |  [x]  Pronounceable: customer, not cstmr             |
  |  [x]  Searchable: MAX_RETRY_COUNT, not 3             |
  |  [x]  No misleading abbreviations                    |
  |  [x]  Consistent with codebase conventions           |
  |                                                      |
  +------------------------------------------------------+
```

15.4 Move Method

Intent

Move a method from one class to another where it is a better fit -- typically to the class whose data it uses most.

When to Use It

- A method uses more data from another class than from its own (feature envy)
- Moving the method would reduce coupling between classes
- The method does not make sense as a responsibility of its current class

BEFORE: Java

```java
public class Account {
    private double balance;
    private String type; // "savings" or "checking"

    public double getBalance() { return balance; }
    public String getType() { return type; }
}

public class BankReport {

    public double calculateInterest(Account account) {
        if (account.getType().equals("savings")) {
            return account.getBalance() * 0.04;
        } else if (account.getType().equals("checking")) {
            return account.getBalance() * 0.01;
        }
```

```
        return 0;
    }

    public boolean isEligibleForLoan(Account account) {
        return account.getBalance() > 1000
            && account.getType().equals("savings");
    }
}
```

Both methods in `BankReport` depend entirely on `Account` data. They belong in `Account`.

AFTER: Java

```java
public class Account {
    private double balance;
    private String type;

    public double calculateInterest() {
        if ("savings".equals(type)) return balance * 0.04;
        if ("checking".equals(type)) return balance * 0.01;
        return 0;
    }

    public boolean isEligibleForLoan() {
        return balance > 1000 && "savings".equals(type);
    }

    public double getBalance() { return balance; }
    public String getType() { return type; }
}

public class BankReport {
    // BankReport now calls account.calculateInterest()
    // instead of doing the calculation itself
}
```

BEFORE: Python

```python
class Rectangle:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
```

```
class GeometryUtils:
    def area(self, rect: Rectangle):
        return rect.width * rect.height

    def perimeter(self, rect: Rectangle):
        return 2 * (rect.width + rect.height)

    def contains_point(self, rect: Rectangle, px, py):
        return (rect.x <= px <= rect.x + rect.width
                and rect.y <= py <= rect.y + rect.height)
```

AFTER: Python

```python
class Rectangle:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def contains_point(self, px, py):
        return (self.x <= px <= self.x + self.width
                and self.y <= py <= self.y + self.height)
```

Step-by-Step Procedure

1. Examine all features (fields, methods) used by the method
2. Check if there is another method in the source class that depends on this one
3. Declare the method in the target class
4. Copy the body and adjust references (use `this`/`self` for the target class)
5. Turn the old method into a delegating method (or remove it and update callers)
6. Run tests

15.5 Replace Conditional with Polymorphism

Intent

Replace a conditional (if-else chain or switch statement) that selects behavior based on type with polymorphism. Each branch becomes a separate class that implements a shared interface.

When to Use It

- The same conditional appears in multiple methods
- New types are added frequently, requiring updates to every conditional
- Each branch contains significantly different logic

BEFORE: Java

```java
public class AreaCalculator {

    public double calculateArea(String shape, double[] dimensions) {
        switch (shape) {
            case "circle":
                return Math.PI * dimensions[0] * dimensions[0];
            case "rectangle":
                return dimensions[0] * dimensions[1];
            case "triangle":
                return 0.5 * dimensions[0] * dimensions[1];
            default:
                throw new IllegalArgumentException("Unknown shape: " + shape);
        }
    }

    public String describe(String shape) {
        switch (shape) {
            case "circle":    return "A round shape with a radius";
            case "rectangle": return "A four-sided shape with width and height";
            case "triangle":  return "A three-sided shape with base and height";
            default: throw new IllegalArgumentException("Unknown shape");
        }
    }
}
```

AFTER: Java

```java
public interface Shape {
    double area();
    String describe();
```

```
}

public class Circle implements Shape {
    private final double radius;

    public Circle(double radius) { this.radius = radius; }

    public double area() { return Math.PI * radius * radius; }
    public String describe() { return "A round shape with radius " + radius; }
}

public class Rectangle implements Shape {
    private final double width;
    private final double height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    public double area() { return width * height; }
    public String describe() {
        return "A four-sided shape (" + width + " x " + height + ")";
    }
}

public class Triangle implements Shape {
    private final double base;
    private final double height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    public double area() { return 0.5 * base * height; }
    public String describe() {
        return "A three-sided shape (base=" + base + ", height=" + height + ")";
    }
}

// Usage:
// Shape shape = new Circle(5);
// shape.area();       // No switch needed
// shape.describe();   // No switch needed
```

```
  BEFORE                          AFTER
  +------------------+            +-----------+
  | AreaCalculator   |            | <<Shape>> |
  |                  |            +-----------+
  | switch(shape)    |            | area()    |
  |   circle: ...    |    --->    | describe()|
  |   rect: ...      |            +-----+-----+
```

```
  |   tri: ...       |                  |
  +------------------+        +---------+---------+
                              |         |         |
                           +-------+ +-------+ +--------+
                           |Circle | | Rect  | |Triangle|
                           +-------+ +-------+ +--------+
```

BEFORE: Python

```python
class NotificationSender:

    def send(self, notification_type, recipient, message):
        if notification_type == "email":
            print(f"Sending email to {recipient}: {message}")
            # email-specific logic
        elif notification_type == "sms":
            if len(message) > 160:
                message = message[:157] + "..."
            print(f"Sending SMS to {recipient}: {message}")
        elif notification_type == "push":
            print(f"Sending push notification to {recipient}: {message}")
            # push-specific logic
        else:
            raise ValueError(f"Unknown type: {notification_type}")

    def cost(self, notification_type):
        if notification_type == "email":
            return 0.001
        elif notification_type == "sms":
            return 0.05
        elif notification_type == "push":
            return 0.002
        else:
            raise ValueError(f"Unknown type: {notification_type}")
```

AFTER: Python

```python
from abc import ABC, abstractmethod


class Notification(ABC):
    @abstractmethod
    def send(self, recipient, message): ...

    @abstractmethod
    def cost(self) -> float: ...
```

```
class EmailNotification(Notification):
    def send(self, recipient, message):
        print(f"Sending email to {recipient}: {message}")

    def cost(self):
        return 0.001


class SmsNotification(Notification):
    def send(self, recipient, message):
        if len(message) > 160:
            message = message[:157] + "..."
        print(f"Sending SMS to {recipient}: {message}")

    def cost(self):
        return 0.05


class PushNotification(Notification):
    def send(self, recipient, message):
        print(f"Sending push to {recipient}: {message}")

    def cost(self):
        return 0.002


# Adding a new type = adding a new class. No existing code changes.
```

15.6 Introduce Parameter Object

Intent

Replace a recurring group of parameters with a single object that carries them together.

When to Use It

- The same group of parameters appears in multiple method signatures
- Parameters that always travel together suggest a missing concept in your domain
- Method signatures are getting unwieldy (more than three or four parameters)

BEFORE: Java

```java
public class EventService {

    public List<Event> findEvents(LocalDate startDate, LocalDate endDate,
                                  String city, int maxResults) {
        // ...
    }

    public int countEvents(LocalDate startDate, LocalDate endDate,
                           String city) {
        // ...
    }

    public List<Event> findPopularEvents(LocalDate startDate, LocalDate endDate,
                                         String city, int minAttendees) {
        // ...
    }
}
```

AFTER: Java

```java
public class EventSearchCriteria {
    private final LocalDate startDate;
    private final LocalDate endDate;
    private final String city;

    public EventSearchCriteria(LocalDate startDate, LocalDate endDate, String city) {
        if (endDate.isBefore(startDate)) {
            throw new IllegalArgumentException("End date must be after start date");
        }
        this.startDate = startDate;
        this.endDate = endDate;
        this.city = city;
    }

    public int durationInDays() {
        return (int) ChronoUnit.DAYS.between(startDate, endDate);
    }

    // Getters...
}

public class EventService {

    public List<Event> findEvents(EventSearchCriteria criteria, int maxResults) {
        // ...
    }

    public int countEvents(EventSearchCriteria criteria) {
        // ...
    }
```

```
    public List<Event> findPopularEvents(EventSearchCriteria criteria,
                                         int minAttendees) {
        // ...
    }
}
```

Notice how the parameter object also became a natural home for the `durationInDays()` method and date validation logic.

BEFORE: Python

```python
def create_user(first_name, last_name, email, phone, role, department):
    ...

def update_user(user_id, first_name, last_name, email, phone, role, department):
    ...

def format_user(first_name, last_name, email, role):
    return f"{first_name} {last_name} ({role}) - {email}"
```

AFTER: Python

```python
from dataclasses import dataclass


@dataclass
class UserInfo:
    first_name: str
    last_name: str
    email: str
    phone: str
    role: str
    department: str

    def display_name(self):
        return f"{self.first_name} {self.last_name}"

    def format(self):
        return f"{self.display_name()} ({self.role}) - {self.email}"


def create_user(info: UserInfo):
    ...

def update_user(user_id, info: UserInfo):
    ...
```

15.7 Replace Magic Number with Named Constant

Intent

Replace a literal number (or string) that has a particular meaning with a named constant that explains that meaning.

When to Use It

- A number appears in the code without explanation
- The same literal value appears in multiple places
- Changing the value would require hunting through the codebase

BEFORE: Java

```java
public class TaxCalculator {

    public double calculateTax(double income) {
        if (income <= 10275) {
            return income * 0.10;
        } else if (income <= 41775) {
            return 1027.50 + (income - 10275) * 0.12;
        } else if (income <= 89075) {
            return 4807.50 + (income - 41775) * 0.22;
        } else {
            return 15213.50 + (income - 89075) * 0.24;
        }
    }
}
```

What do these numbers mean? Where do they come from? If tax brackets change, how many places must you update?

AFTER: Java

```java
public class TaxCalculator {

    private static final double BRACKET_1_LIMIT = 10_275.00;
```

```
    private static final double BRACKET_2_LIMIT = 41_775.00;
    private static final double BRACKET_3_LIMIT = 89_075.00;

    private static final double RATE_BRACKET_1 = 0.10;
    private static final double RATE_BRACKET_2 = 0.12;
    private static final double RATE_BRACKET_3 = 0.22;
    private static final double RATE_BRACKET_4 = 0.24;

    private static final double BASE_TAX_BRACKET_2 = 1_027.50;
    private static final double BASE_TAX_BRACKET_3 = 4_807.50;
    private static final double BASE_TAX_BRACKET_4 = 15_213.50;

    public double calculateTax(double income) {
        if (income <= BRACKET_1_LIMIT) {
            return income * RATE_BRACKET_1;
        } else if (income <= BRACKET_2_LIMIT) {
            return BASE_TAX_BRACKET_2 + (income - BRACKET_1_LIMIT) * RATE_BRACKET_2;
        } else if (income <= BRACKET_3_LIMIT) {
            return BASE_TAX_BRACKET_3 + (income - BRACKET_2_LIMIT) * RATE_BRACKET_3;
        } else {
            return BASE_TAX_BRACKET_4 + (income - BRACKET_3_LIMIT) * RATE_BRACKET_4;
        }
    }
}
```

BEFORE: Python

```python
def calculate_shipping(weight, distance):
    if weight > 50:
        return distance * 0.75 + 15.00
    elif weight > 20:
        return distance * 0.50 + 8.00
    else:
        return distance * 0.25 + 3.00

def is_eligible_for_free_shipping(order_total):
    return order_total >= 75.00

def apply_loyalty_discount(price, years):
    if years >= 5:
        return price * 0.85
    elif years >= 2:
        return price * 0.90
    return price
```

AFTER: Python

```python
# Shipping thresholds
HEAVY_PACKAGE_WEIGHT_KG = 50
MEDIUM_PACKAGE_WEIGHT_KG = 20

HEAVY_RATE_PER_KM = 0.75
MEDIUM_RATE_PER_KM = 0.50
LIGHT_RATE_PER_KM = 0.25

HEAVY_BASE_FEE = 15.00
MEDIUM_BASE_FEE = 8.00
LIGHT_BASE_FEE = 3.00

FREE_SHIPPING_THRESHOLD = 75.00

# Loyalty discounts
GOLD_LOYALTY_YEARS = 5
SILVER_LOYALTY_YEARS = 2
GOLD_DISCOUNT_MULTIPLIER = 0.85
SILVER_DISCOUNT_MULTIPLIER = 0.90


def calculate_shipping(weight, distance):
    if weight > HEAVY_PACKAGE_WEIGHT_KG:
        return distance * HEAVY_RATE_PER_KM + HEAVY_BASE_FEE
    elif weight > MEDIUM_PACKAGE_WEIGHT_KG:
        return distance * MEDIUM_RATE_PER_KM + MEDIUM_BASE_FEE
    else:
        return distance * LIGHT_RATE_PER_KM + LIGHT_BASE_FEE


def is_eligible_for_free_shipping(order_total):
    return order_total >= FREE_SHIPPING_THRESHOLD


def apply_loyalty_discount(price, years):
    if years >= GOLD_LOYALTY_YEARS:
        return price * GOLD_DISCOUNT_MULTIPLIER
    elif years >= SILVER_LOYALTY_YEARS:
        return price * SILVER_DISCOUNT_MULTIPLIER
    return price
```

When NOT to Replace

Some numbers are self-explanatory and do not need a constant:

```java
// These are fine as literals:
int[] pair = new int[2];          // 2 elements in a pair is obvious
double half = total / 2.0;        // Dividing by 2 is obvious
String empty = "";                // Empty string is obvious
```

15.8 Chaining Techniques Together

Real-world refactoring rarely uses a single technique in isolation. Here is an example that combines multiple techniques:

BEFORE: Multiple Smells (Java)

```java
public class ReportManager {
    public String generate(List<Map<String, Object>> data, String type,
                           int year, int month, int day,
                           double threshold) {
        // Filter
        List<Map<String, Object>> filtered = new ArrayList<>();
        for (Map<String, Object> row : data) {
            double val = (double) row.get("value");
            if (val >= threshold) {
                filtered.add(row);
            }
        }

        // Format
        StringBuilder sb = new StringBuilder();
        if (type.equals("summary")) {
            double total = 0;
            for (Map<String, Object> row : filtered) {
                total += (double) row.get("value");
            }
            sb.append("Summary for ").append(month).append("/")
              .append(day).append("/").append(year).append("\n");
            sb.append("Total: ").append(total).append("\n");
            sb.append("Count: ").append(filtered.size());
        } else if (type.equals("detail")) {
            sb.append("Detail for ").append(month).append("/")
              .append(day).append("/").append(year).append("\n");
            for (Map<String, Object> row : filtered) {
                sb.append("  ").append(row.get("name"))
                  .append(": ").append(row.get("value")).append("\n");
            }
        }
        return sb.toString();
    }
}
```

AFTER: Multiple Techniques Applied

```java
// Technique: Introduce Parameter Object (date clump)
public class ReportDate {
    private final int year, month, day;

    public ReportDate(int year, int month, int day) {
        this.year = year; this.month = month; this.day = day;
    }

    public String format() {
        return month + "/" + day + "/" + year;
    }
}

// Technique: Replace Conditional with Polymorphism
public interface ReportFormatter {
    String format(List<Map<String, Object>> data, ReportDate date);
}

public class SummaryFormatter implements ReportFormatter {
    public String format(List<Map<String, Object>> data, ReportDate date) {
        double total = data.stream()
                .mapToDouble(r -> (double) r.get("value")).sum();
        return "Summary for " + date.format() + "\n"
             + "Total: " + total + "\n"
             + "Count: " + data.size();
    }
}

public class DetailFormatter implements ReportFormatter {
    public String format(List<Map<String, Object>> data, ReportDate date) {
        StringBuilder sb = new StringBuilder();
        sb.append("Detail for ").append(date.format()).append("\n");
        for (Map<String, Object> row : data) {
            sb.append("  ").append(row.get("name"))
              .append(": ").append(row.get("value")).append("\n");
        }
        return sb.toString();
    }
}

// Technique: Extract Method (filtering)
public class ReportGenerator {
    private final ReportFormatter formatter;

    public ReportGenerator(ReportFormatter formatter) {
        this.formatter = formatter;
    }

    public String generate(List<Map<String, Object>> data,
                           ReportDate date, double threshold) {
        List<Map<String, Object>> filtered = filterByThreshold(data, threshold);
        return formatter.format(filtered, date);
    }
```

```
    private List<Map<String, Object>> filterByThreshold(
            List<Map<String, Object>> data, double threshold) {
        return data.stream()
                .filter(row -> (double) row.get("value") >= threshold)
                .collect(Collectors.toList());
    }
}
```

Common Mistakes

1. Refactoring without tests. Every refactoring technique assumes you can verify behavior has not changed. Without tests, you are just editing code and hoping.
2. Doing too much at once. Apply one technique, run tests, commit. Then apply the next. Combining multiple changes into one big step makes it impossible to isolate problems.
3. Over-extracting. Not every three-line block deserves its own method. Extract only when the method name adds clarity.
4. Renaming to satisfy a style guide but losing meaning. A name like `processData` renamed to `handleData` is not a refactoring -- it is just shuffling words. Rename to reveal intent.
5. Creating parameter objects with no behavior. A parameter object that is just a data carrier misses the opportunity to move validation and formatting logic into the object.

Best Practices

1. Refactor on Green. Only refactor when all tests pass. If tests are failing, fix the bug first.
2. Use your IDE's refactoring tools. Modern IDEs (IntelliJ, VS Code, PyCharm) automate Extract Method, Rename, and Move Method safely. Use them instead of manual editing.
3. Commit after each successful refactoring. Small commits let you revert a single change if something goes wrong.
4. Follow the Boy Scout Rule. Refactor a little bit every time you touch a file. Small, continuous improvements are more effective than big refactoring sprints.
5. Refactor toward a goal. Do not refactor aimlessly. Have a specific smell you are removing or a specific design you are moving toward.

Quick Summary

Technique	Fixes This Smell	Core Action
Extract Method	Long Method	Pull code into a named method
Extract Class	Large Class	Split class into two
Rename	Unclear names	Change name to reveal intent
Move Method	Feature Envy	Move method to the class it envies
Replace Conditional with Polymorphism	Switch Statements	Each branch becomes a class
Introduce Parameter Object	Data Clumps	Group related parameters into a class
Replace Magic Number with Constant	Mysterious literals	Give meaningful names to numbers

Key Points

- Refactoring changes code structure without changing behavior. Tests verify this.
- Each technique targets a specific smell. Knowing which technique to apply is as important as knowing how.
- Small steps with frequent test runs are the safest way to refactor.
- IDE refactoring tools eliminate an entire category of errors. Use them.
- Real refactoring combines multiple techniques. Master each one individually, then learn to chain them.

Practice Questions

1. You have a 60-line method with three distinct sections separated by comments. Which refactoring technique should you apply first? What would you name the extracted methods?

2. A method `calculatePrice(String itemType, double basePrice, int quantity, boolean isMember, String couponCode)` has five parameters. Which technique would reduce this to a cleaner signature? Design the parameter object.
3. You find this code: `if (status == 1) ... else if (status == 2) ... else if (status == 3) ...`. The same pattern appears in four different methods. Which technique fixes this? Outline the class hierarchy you would create.
4. A class `DataProcessor` has methods that call `record.getAccount().getOwner().getEmail()` frequently. Which smell is this, and which two refactoring techniques could fix it?
5. When is it acceptable to NOT replace a magic number with a named constant? Give two examples.

Exercises

Exercise 1: Extract and Rename

Take this function and apply Extract Method and Rename to make it readable:

```python
def p(d):
    r = []
    for x in d:
        if x["t"] == "A" and x["v"] > 100:
            r.append({"n": x["n"], "v": x["v"] * 1.1})
        elif x["t"] == "B":
            r.append({"n": x["n"], "v": x["v"] * 0.9})
    s = sum(i["v"] for i in r)
    return {"items": r, "total": s}
```

Exercise 2: Full Refactoring

Refactor this Java class using at least three different techniques from this chapter:

```java
public class OrderHandler {
    public String handle(String cType, String cName, String cEmail,
                         String item, int qty, double price, String promoCode) {
        double total = qty * price;
        double discount = 0;
        if (promoCode != null) {
            if (promoCode.equals("SAVE10")) discount = total * 0.10;
```

```
            else if (promoCode.equals("SAVE20")) discount = total * 0.20;
            else if (promoCode.equals("HALF")) discount = total * 0.50;
        }
        if (cType.equals("VIP")) discount += total * 0.05;
        total = total - discount;
        double tax = total * 0.0825;
        return "Order for " + cName + " (" + cEmail + ")\n"
            + item + " x" + qty + "\n"
            + "Subtotal: $" + total + "\n"
            + "Tax: $" + tax + "\n"
            + "Total: $" + (total + tax);
    }
}
```

Identify the smells first, then apply the appropriate technique for each one.

Exercise 3: Refactoring Plan

You inherit a 500-line class called `ApplicationController` that handles user authentication, request routing, input validation, and response formatting. Write a refactoring plan that uses techniques from this chapter. List the steps in order, and explain why you chose that order.

What Is Next?

Refactoring makes existing code better. But how do you know the code works correctly before, during, and after refactoring? The answer is testing. Chapter 16: Unit Testing Principles teaches you how to write tests that are fast, reliable, and meaningful -- the safety net that makes refactoring possible.

Chapter 16: Unit Testing Principles -- Building Your Safety Net

What You Will Learn

- What a unit test is and what makes it different from other kinds of tests
- How to structure every test using the Arrange-Act-Assert (AAA) pattern
- The FIRST properties that separate great tests from fragile ones
- When and how to use test doubles: mocks, stubs, spies, and fakes
- What code coverage really tells you (and what it does not)
- How the testing pyramid guides your testing strategy

Why This Chapter Matters

Code without tests is code you cannot confidently change. Every refactoring technique from Chapter 15 assumes you have a safety net that catches regressions instantly. Unit tests are the foundation of that safety net. They run in milliseconds, pinpoint exactly what broke, and serve as living documentation of how your code is supposed to behave. Writing good tests is a skill as important as writing good production code. Bad tests are worse than no tests -- they give false confidence and break every time you refactor.

16.1 What Is a Unit Test?

A unit test verifies the behavior of a small, isolated piece of code -- typically a single method or function -- in a controlled environment.

```
+-------------------------------------------------------+
|                  UNIT TEST PROPERTIES                 |
+-------------------------------------------------------+
|                                                       |
|  Fast         Runs in milliseconds, not seconds       |
|  Isolated     Tests one thing, no external deps       |
|  Repeatable   Same result every time, any machine     |
|  Self-checking  Pass/fail without human inspection    |
|  Timely       Written close to the code it tests      |
|                                                       |
+-------------------------------------------------------+
```

What a "Unit" Means

In Java, a unit is typically a single method of a class. In Python, it is typically a single function or method. The exact boundary is less important than the principle: test one behavior at a time.

16.2 The AAA Pattern (Arrange-Act-Assert)

Every well-structured test follows three phases:

```
+-----------+     +-------+     +----------+
|  ARRANGE  | --> |  ACT  | --> |  ASSERT  |
+-----------+     +-------+     +----------+
|  Set up   |     | Call  |     | Verify   |
|  test data |    | the   |     | the      |
|  and deps  |    | method |    | result   |
+-----------+     +-------+     +----------+
```

AAA in Java (JUnit 5)

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ShoppingCartTest {

    @Test
    void addItem_increasesTotalByItemPrice() {
        // Arrange
```

```
        ShoppingCart cart = new ShoppingCart();
        Product apple = new Product("Apple", 1.50);

        // Act
        cart.addItem(apple, 3);

        // Assert
        assertEquals(4.50, cart.getTotal(), 0.001);
    }

    @Test
    void addItem_incrementsItemCount() {
        // Arrange
        ShoppingCart cart = new ShoppingCart();
        Product apple = new Product("Apple", 1.50);

        // Act
        cart.addItem(apple, 3);

        // Assert
        assertEquals(3, cart.getItemCount());
    }

    @Test
    void removeItem_decreasesTotal() {
        // Arrange
        ShoppingCart cart = new ShoppingCart();
        Product apple = new Product("Apple", 1.50);
        cart.addItem(apple, 3);

        // Act
        cart.removeItem(apple, 1);

        // Assert
        assertEquals(3.00, cart.getTotal(), 0.001);
        assertEquals(2, cart.getItemCount());
    }
}
```

AAA in Python (pytest)

```python
import pytest
from shopping_cart import ShoppingCart, Product


class TestShoppingCart:

    def test_add_item_increases_total(self):
        # Arrange
        cart = ShoppingCart()
```

```
        apple = Product("Apple", 1.50)

        # Act
        cart.add_item(apple, quantity=3)

        # Assert
        assert cart.total == pytest.approx(4.50)

    def test_add_item_increments_count(self):
        # Arrange
        cart = ShoppingCart()
        apple = Product("Apple", 1.50)

        # Act
        cart.add_item(apple, quantity=3)

        # Assert
        assert cart.item_count == 3

    def test_remove_item_decreases_total(self):
        # Arrange
        cart = ShoppingCart()
        apple = Product("Apple", 1.50)
        cart.add_item(apple, quantity=3)

        # Act
        cart.remove_item(apple, quantity=1)

        # Assert
        assert cart.total == pytest.approx(3.00)
        assert cart.item_count == 2
```

Why AAA Works

- Readability: Anyone can see what is being tested at a glance
- Consistency: Every test has the same structure, reducing cognitive load
- Debugging: When a test fails, you immediately know whether the problem is in the setup, the action, or the expectation

---

16.3 Test One Thing

Each test should verify exactly one behavior. This does not mean one assertion -- it means one logical concept.

BEFORE: Testing Too Many Things

```java
@Test
void testEverything() {
    Calculator calc = new Calculator();

    // Test addition
    assertEquals(5, calc.add(2, 3));

    // Test subtraction
    assertEquals(1, calc.subtract(3, 2));

    // Test division
    assertEquals(2.5, calc.divide(5, 2), 0.001);

    // Test division by zero
    assertThrows(ArithmeticException.class, () -> calc.divide(5, 0));
}
```

If `subtract` fails, you do not know whether `divide` also fails because the test stops at the first failure.

AFTER: One Behavior Per Test

```java
@Test
void add_returnsSumOfTwoNumbers() {
    Calculator calc = new Calculator();
    assertEquals(5, calc.add(2, 3));
}

@Test
void subtract_returnsDifference() {
    Calculator calc = new Calculator();
    assertEquals(1, calc.subtract(3, 2));
}

@Test
void divide_returnsQuotient() {
    Calculator calc = new Calculator();
    assertEquals(2.5, calc.divide(5, 2), 0.001);
}

@Test
void divide_byZeroThrowsException() {
    Calculator calc = new Calculator();
    assertThrows(ArithmeticException.class, () -> calc.divide(5, 0));
}
```

BEFORE: Testing Too Many Things (Python)

```python
def test_user_service():
    service = UserService()

    user = service.create("Alice", "alice@example.com")
    assert user.name == "Alice"

    found = service.find_by_email("alice@example.com")
    assert found.name == "Alice"

    service.deactivate(user.id)
    assert not service.is_active(user.id)
```

AFTER: One Behavior Per Test (Python)

```python
def test_create_user_sets_name():
    service = UserService()
    user = service.create("Alice", "alice@example.com")
    assert user.name == "Alice"


def test_find_by_email_returns_matching_user():
    service = UserService()
    service.create("Alice", "alice@example.com")
    found = service.find_by_email("alice@example.com")
    assert found.name == "Alice"


def test_deactivate_makes_user_inactive():
    service = UserService()
    user = service.create("Alice", "alice@example.com")
    service.deactivate(user.id)
    assert not service.is_active(user.id)
```

Test Naming Convention

Good test names describe the scenario and expected outcome:

```
  Method Under Test     Scenario              Expected Result
  +--------------+  +------------------+  +-------------------+
  | add          |  | twoPositiveNums  |  | returnsSum        |
  | divide       |  | byZero           |  | throwsException   |
  | applyDiscount|  | expiredCoupon    |  | returnsFullPrice  |
  +--------------+  +------------------+  +-------------------+
```

16.4 FIRST Properties of Good Tests

The FIRST acronym captures the essential properties of well-designed unit tests:

F -- Fast

Tests must run in milliseconds. If your test suite takes minutes, developers stop running it. Fast tests enable the rapid feedback loop that makes TDD and refactoring possible.

```java
// SLOW: Hits a real database
@Test
void findUser_slow() {
    Database db = new Database("jdbc:mysql://localhost:3306/test");
    UserRepository repo = new UserRepository(db);
    User user = repo.findById(1);
    assertNotNull(user);
}

// FAST: Uses an in-memory substitute
@Test
void findUser_fast() {
    UserRepository repo = new UserRepository(new InMemoryDatabase());
    repo.save(new User(1, "Alice"));
    User user = repo.findById(1);
    assertEquals("Alice", user.getName());
}
```

I -- Isolated (Independent)

Tests must not depend on each other. Each test sets up its own state, runs, and cleans up after itself. Running tests in any order must produce the same result.

```python
# BAD: Tests depend on shared state
shared_list = []

def test_add():
    shared_list.append("item")
    assert len(shared_list) == 1  # Fails if test_add runs twice

def test_remove():
    shared_list.pop()             # Fails if test_add did not run first
    assert len(shared_list) == 0


# GOOD: Each test creates its own state
def test_add():
```

```
    items = []
    items.append("item")
    assert len(items) == 1

def test_remove():
    items = ["item"]
    items.pop()
    assert len(items) == 0
```

R -- Repeatable

Tests must produce the same result regardless of environment, time, or network state. Avoid dependencies on the current date, random numbers, or external services.

```java
// BAD: Depends on current time
@Test
void isExpired_today() {
    Coupon coupon = new Coupon("SAVE10", LocalDate.now().minusDays(1));
    assertTrue(coupon.isExpired());  // What if the clock changes?
}

// GOOD: Uses a fixed date
@Test
void isExpired_whenPastExpirationDate() {
    LocalDate fixedToday = LocalDate.of(2024, 6, 15);
    Coupon coupon = new Coupon("SAVE10", LocalDate.of(2024, 6, 14));
    assertTrue(coupon.isExpired(fixedToday));
}
```

S -- Self-Validating

Tests must produce a clear pass or fail result. No manual inspection of log files or console output.

```python
# BAD: Requires manual inspection
def test_format_report():
    report = format_report(sample_data)
    print(report)  # Developer must read this and decide if it looks right

# GOOD: Automated assertion
def test_format_report_includes_header():
    report = format_report(sample_data)
    assert "Monthly Sales Report" in report

def test_format_report_includes_total():
```

```
    report = format_report(sample_data)
    assert "$1,234.56" in report
```

T -- Timely

Write tests close in time to the production code they test. Ideally, write them before (TDD, Chapter 17) or immediately after.

16.5 Test Doubles: Mock, Stub, Spy, and Fake

When your code depends on external systems (databases, APIs, file systems), you replace those dependencies with test doubles to keep tests fast and isolated.

```
+-------------------------------------------------------+
|                  TEST DOUBLE TYPES                    |
+-------------------------------------------------------+
|                                                       |
|  Stub    Returns canned answers to calls              |
|  Mock    Verifies that specific calls were made       |
|  Spy     Records calls for later verification         |
|  Fake    Working implementation (simplified)          |
|                                                       |
+-------------------------------------------------------+
|                                                       |
|  Dummy   Passed around but never used                 |
|                                                       |
+-------------------------------------------------------+
```

Stub: Providing Canned Answers

A stub returns predefined answers. It does not verify how it was called.

Java (Mockito):

```java
@Test
void getWeatherSummary_returnsFormattedString() {
    // Arrange: Create a stub that returns a fixed temperature
    WeatherService weatherStub = mock(WeatherService.class);
    when(weatherStub.getTemperature("NYC")).thenReturn(72.0);
```

```
    WeatherReporter reporter = new WeatherReporter(weatherStub);

    // Act
    String summary = reporter.getSummary("NYC");

    // Assert
    assertEquals("NYC: 72.0 F", summary);
}
```

Python (unittest.mock):

```python
from unittest.mock import Mock

def test_get_weather_summary():
    # Arrange: Create a stub
    weather_service = Mock()
    weather_service.get_temperature.return_value = 72.0

    reporter = WeatherReporter(weather_service)

    # Act
    summary = reporter.get_summary("NYC")

    # Assert
    assert summary == "NYC: 72.0 F"
```

Mock: Verifying Interactions

A mock checks that specific methods were called with specific arguments.

Java (Mockito):

```java
@Test
void placeOrder_sendsConfirmationEmail() {
    // Arrange
    EmailService emailMock = mock(EmailService.class);
    OrderService orderService = new OrderService(emailMock);
    Order order = new Order("Alice", "alice@example.com", 99.99);

    // Act
    orderService.placeOrder(order);

    // Assert: Verify the email was sent
    verify(emailMock).sendEmail(
        eq("alice@example.com"),
        eq("Order Confirmation"),
        contains("99.99")
```

```
    );
}
```

Python (unittest.mock):

```python
from unittest.mock import Mock, call

def test_place_order_sends_confirmation_email():
    # Arrange
    email_service = Mock()
    order_service = OrderService(email_service)
    order = Order("Alice", "alice@example.com", 99.99)

    # Act
    order_service.place_order(order)

    # Assert
    email_service.send_email.assert_called_once_with(
        "alice@example.com",
        "Order Confirmation",
        "Your order total: $99.99"
    )
```

Spy: Recording Calls

A spy wraps a real object and records calls to it, letting you verify interactions while using real behavior.

Java (Mockito):

```java
@Test
void addToCart_logsTheAction() {
    // Arrange
    Logger loggerSpy = spy(new ConsoleLogger());
    ShoppingCart cart = new ShoppingCart(loggerSpy);
    Product apple = new Product("Apple", 1.50);

    // Act
    cart.addItem(apple, 2);

    // Assert: Real logging happened, AND we can verify it
    verify(loggerSpy).log("Added 2x Apple to cart");
}
```

Python:

```python
from unittest.mock import patch

def test_add_to_cart_logs_action():
    cart = ShoppingCart()
    apple = Product("Apple", 1.50)

    with patch.object(cart, 'logger', wraps=cart.logger) as spy_logger:
        cart.add_item(apple, 2)
        spy_logger.log.assert_called_with("Added 2x Apple to cart")
```

Fake: Simplified Implementation

A fake is a working implementation that takes shortcuts. An in-memory database is a common fake.

Java:

```java
public class InMemoryUserRepository implements UserRepository {
    private final Map<Integer, User> store = new HashMap<>();
    private int nextId = 1;

    @Override
    public User save(User user) {
        user.setId(nextId++);
        store.put(user.getId(), user);
        return user;
    }

    @Override
    public User findById(int id) {
        return store.get(id);
    }

    @Override
    public List<User> findAll() {
        return new ArrayList<>(store.values());
    }
}

// Usage in tests:
@Test
void saveAndRetrieveUser() {
    UserRepository repo = new InMemoryUserRepository();
    User saved = repo.save(new User("Alice"));
    User found = repo.findById(saved.getId());
    assertEquals("Alice", found.getName());
}
```

Python:

```python
class InMemoryUserRepository:
    def __init__(self):
        self._store = {}
        self._next_id = 1

    def save(self, user):
        user.id = self._next_id
        self._next_id += 1
        self._store[user.id] = user
        return user

    def find_by_id(self, user_id):
        return self._store.get(user_id)

    def find_all(self):
        return list(self._store.values())


# Usage in tests:
def test_save_and_retrieve_user():
    repo = InMemoryUserRepository()
    saved = repo.save(User("Alice"))
    found = repo.find_by_id(saved.id)
    assert found.name == "Alice"
```

Choosing the Right Double

```
  +-------------------------------------------------------------+
  |  QUESTION                           | USE THIS              |
  +-------------------------------------------------------------+
  |  Need fixed return values?          | Stub                  |
  |  Need to verify a call was made?    | Mock                  |
  |  Need real behavior + recording?    | Spy                   |
  |  Need a working in-memory system?   | Fake                  |
  |  Need to fill a parameter?          | Dummy (null/empty)    |
  +-------------------------------------------------------------+
```

16.6 Code Coverage

Code coverage measures how much of your production code is executed by your tests. It is reported as a percentage.

Types of Coverage

```
+-------------------------------------------------------------+
|  TYPE                | WHAT IT MEASURES                     |
+-------------------------------------------------------------+
|  Line coverage       | Lines executed / total lines         |
|  Branch coverage     | Branches taken / total branches      |
|  Method coverage     | Methods called / total methods       |
|  Condition coverage  | Boolean conditions evaluated         |
+-------------------------------------------------------------+
```

What Coverage Tells You

Coverage tells you what code your tests execute. It does NOT tell you:

- Whether the tests actually verify correct behavior
- Whether edge cases are covered
- Whether the design is good

The Coverage Trap

```java
// This test achieves 100% line coverage of calculateDiscount():
@Test
void calculateDiscount_runsWithoutError() {
    Calculator calc = new Calculator();
    calc.calculateDiscount(100, "SAVE10");
    // No assertion! The test passes even if the result is wrong.
}
```

This test is worthless despite achieving 100% coverage. Always assert meaningful outcomes.

Practical Coverage Guidelines

```
+-----------------------------------------------+
|          COVERAGE SWEET SPOT                  |
+-----------------------------------------------+
|                                               |
|  0-30%    Dangerously low                     |
|  30-60%   Many gaps, high risk                |
|  60-80%   Reasonable for most projects        |
|  80-90%   Strong coverage, good confidence    |
|  90-100%  Diminishing returns past 90%        |
|                                               |
|  Target: 70-85% with meaningful assertions    |
+-----------------------------------------------+
```

Coverage in Practice (Python with pytest-cov)

```bash
# Run tests with coverage report
pytest --cov=src --cov-report=term-missing

# Output example:
# Name                    Stmts   Miss  Cover   Missing
# -------------------------------------------------
# src/calculator.py          24      3    88%   42-44
# src/cart.py                 31      0   100%
# src/order.py               45     12    73%   23-34
# -------------------------------------------------
# TOTAL                     100     15    85%
```

Coverage in Practice (Java with JaCoCo)

```xml
<!-- Maven pom.xml -->
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
        <execution>
            <goals>
                <goal>prepare-agent</goal>
            </goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

16.7 The Testing Pyramid

The testing pyramid describes the ideal distribution of test types in a project:

```
           /\
          /  \
```

```
        / E2E\         Few, slow, expensive
       /------\
      /        \
     /Integration\     Some, moderate speed
    /--------------\
   /                \
  /    Unit Tests    \  Many, fast, cheap
 /____________________\
```

Layer Breakdown

Layer	Count	Speed	Cost to Write	What It Tests
Unit	Many (hundreds)	Milliseconds	Low	Individual functions and methods
Integration	Some (dozens)	Seconds	Medium	Components working together
End-to-End	Few (handful)	Minutes	High	Full user workflows

Why the Pyramid Shape?

- Unit tests are fast and cheap. Write many of them. They catch most bugs.
- Integration tests verify that components work together (e.g., your code + the database). Write enough to cover critical paths.
- End-to-end tests verify the complete system from the user's perspective. Write few because they are slow, brittle, and expensive to maintain.

Anti-Pattern: The Ice Cream Cone

```
 ____________________
\                    /
 \    E2E Tests    /     Many slow E2E tests
  \--------------/
   \            /
    \Integration\        Some integration
     \----------/
      \        /
       \ Unit /            Few unit tests
        \    /
         \  /
          \/
```

```
This is the ICE CREAM CONE anti-pattern.
Tests are slow, brittle, and expensive.
```

Teams that rely mostly on E2E tests have slow feedback loops, flaky test suites, and developers who stop trusting their tests.

Practical Example: Testing a User Registration Feature

```
+-----------------------------------------------------+
|  E2E (1-2 tests)                                    |
|    User fills form, clicks submit, sees success    |
+-----------------------------------------------------+
|  Integration (3-5 tests)                            |
|    UserService + Database: save and retrieve user   |
|    UserService + EmailService: sends welcome email  |
+-----------------------------------------------------+
|  Unit (10-20 tests)                                 |
|    Email validation logic                           |
|    Password strength checker                        |
|    Username uniqueness check (with fake repo)       |
|    Welcome email template generation                |
|    Input sanitization                               |
+-----------------------------------------------------+
```

16.8 Writing Your First Test Suite

Here is a complete example of a well-tested class in both Java and Python.

The Production Code (Java)

```java
public class StringCalculator {

    public int add(String numbers) {
        if (numbers.isEmpty()) {
            return 0;
        }
        String delimiter = ",";
        String body = numbers;

        if (numbers.startsWith("//")) {
            int newlineIndex = numbers.indexOf("\n");
```

```
            delimiter = numbers.substring(2, newlineIndex);
            body = numbers.substring(newlineIndex + 1);
        }

        String[] parts = body.split(delimiter + "|\n");
        int sum = 0;
        for (String part : parts) {
            int value = Integer.parseInt(part.trim());
            if (value < 0) {
                throw new IllegalArgumentException("Negatives not allowed: " + value);
            }
            if (value <= 1000) {
                sum += value;
            }
        }
        return sum;
    }
}
```

The Test Suite (Java)

```java
class StringCalculatorTest {

    private StringCalculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new StringCalculator();
    }

    @Test
    void add_emptyString_returnsZero() {
        assertEquals(0, calculator.add(""));
    }

    @Test
    void add_singleNumber_returnsThatNumber() {
        assertEquals(5, calculator.add("5"));
    }

    @Test
    void add_twoNumbers_returnsSum() {
        assertEquals(8, calculator.add("3,5"));
    }

    @Test
    void add_multipleNumbers_returnsSum() {
        assertEquals(15, calculator.add("1,2,3,4,5"));
    }
```

```
    @Test
    void add_newlineDelimiter_works() {
        assertEquals(6, calculator.add("1\n2,3"));
    }

    @Test
    void add_customDelimiter_works() {
        assertEquals(7, calculator.add("//;\n3;4"));
    }

    @Test
    void add_negativeNumber_throwsException() {
        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> calculator.add("1,-2,3")
        );
        assertTrue(ex.getMessage().contains("-2"));
    }

    @Test
    void add_numbersAbove1000_ignored() {
        assertEquals(2, calculator.add("2,1001"));
    }
}
```

The Production Code (Python)

```python
class StringCalculator:

    def add(self, numbers: str) -> int:
        if not numbers:
            return 0

        delimiter = ","
        body = numbers

        if numbers.startswith("//"):
            newline_index = numbers.index("\n")
            delimiter = numbers[2:newline_index]
            body = numbers[newline_index + 1:]

        import re
        parts = re.split(f"{re.escape(delimiter)}|\n", body)
        total = 0
        for part in parts:
            value = int(part.strip())
            if value < 0:
                raise ValueError(f"Negatives not allowed: {value}")
            if value <= 1000:
                total += value
        return total
```

The Test Suite (Python)

```python
import pytest
from string_calculator import StringCalculator


@pytest.fixture
def calc():
    return StringCalculator()


def test_add_empty_string_returns_zero(calc):
    assert calc.add("") == 0


def test_add_single_number(calc):
    assert calc.add("5") == 5


def test_add_two_numbers(calc):
    assert calc.add("3,5") == 8


def test_add_multiple_numbers(calc):
    assert calc.add("1,2,3,4,5") == 15


def test_add_newline_delimiter(calc):
    assert calc.add("1\n2,3") == 6


def test_add_custom_delimiter(calc):
    assert calc.add("//;\n3;4") == 7


def test_add_negative_raises_error(calc):
    with pytest.raises(ValueError, match="-2"):
        calc.add("1,-2,3")


def test_add_ignores_numbers_above_1000(calc):
    assert calc.add("2,1001") == 2
```

Common Mistakes

1. Testing implementation, not behavior. Do not test that a method calls a private helper. Test the observable outcome.

2. Overusing mocks. If every test is full of mocks, your design might have too many dependencies. Consider simplifying the design first.
3. Fragile tests that break on refactoring. If renaming a private method breaks tests, the tests are coupled to implementation details.
4. Chasing 100% coverage. The last 10% often tests getters, setters, and trivial code. The effort is better spent writing integration tests for critical paths.
5. Ignoring test readability. Tests are documentation. If a test is hard to read, it is hard to trust and hard to maintain.

Best Practices

1. Follow AAA consistently. Arrange, Act, Assert -- with blank lines separating each section in longer tests.
2. Use descriptive test names. `test_divide_by_zero_raises_error` is infinitely better than `test3`.
3. Keep tests independent. Each test must create its own state. Never rely on test execution order.
4. Prefer stubs over mocks when possible. Stubs test outcomes. Mocks test interactions. Outcome-based tests are more resilient to refactoring.
5. Run tests frequently. After every change. Make it a habit, not a chore.

Quick Summary

Concept	Purpose
AAA Pattern	Structure every test into Arrange, Act, Assert
Test One Thing	Each test verifies a single behavior
FIRST Properties	Fast, Isolated, Repeatable, Self-validating, Timely
Stub	Returns canned answers
Mock	Verifies interactions
Spy	Records calls on a real object

Fake	Simplified working implementation
Code Coverage	Measures executed code, not correctness
Testing Pyramid	Many unit, some integration, few E2E

Key Points

- Unit tests are the foundation of your testing strategy. They are fast, focused, and cheap.
- The AAA pattern gives every test a consistent, readable structure.
- Test doubles let you isolate the code under test from external dependencies.
- Code coverage is a useful metric but a terrible goal. High coverage with no assertions is useless.
- The testing pyramid guides you toward a balanced, maintainable test suite.

Practice Questions

1. What is the difference between a mock and a stub? In what situation would you choose one over the other?
2. A test suite achieves 95% code coverage but has no assertions -- every test just calls methods and discards the result. What is wrong with this suite? What does its coverage number actually mean?
3. Your test for a `sendEmail` method calls a real SMTP server. Which FIRST property does this violate? How would you fix it?
4. You have a method that generates a report with today's date in the header. How would you make this testable and repeatable?
5. Explain why the "ice cream cone" anti-pattern leads to slow development cycles. What happens when most tests are E2E tests?

Exercises

Exercise 1: Write a Test Suite

Write a complete test suite for this Python class using the AAA pattern and pytest:

```python
class PasswordValidator:
    def validate(self, password: str) -> list[str]:
        errors = []
        if len(password) < 8:
            errors.append("Must be at least 8 characters")
        if not any(c.isupper() for c in password):
            errors.append("Must contain an uppercase letter")
        if not any(c.isdigit() for c in password):
            errors.append("Must contain a digit")
        return errors
```

Write at least five tests covering valid passwords, each individual rule, and the combination of multiple failures.

Exercise 2: Introduce a Test Double

Given this class, write tests using a stub for the `WeatherApi`:

```java
public class FarmingAdvisor {
    private WeatherApi weatherApi;

    public FarmingAdvisor(WeatherApi weatherApi) {
        this.weatherApi = weatherApi;
    }

    public String advise(String location) {
        double temp = weatherApi.getTemperature(location);
        double rain = weatherApi.getRainProbability(location);

        if (temp > 35) return "Too hot to plant.";
        if (temp < 5) return "Too cold to plant.";
        if (rain > 0.7) return "Delay planting, heavy rain expected.";
        return "Good conditions for planting.";
    }
}
```

Exercise 3: Coverage Analysis

Run your tests from Exercise 1 with coverage enabled. Identify any uncovered lines. Add tests until you reach at least 90% branch coverage. Then explain: does 90% coverage mean the code is bug-free?

What Is Next?

You now know how to write tests that verify your code works. But what if you wrote the tests first -- before writing any production code? That is the idea behind Test-Driven Development (TDD). Chapter 17 walks you through the Red-Green-Refactor cycle and shows you how writing tests first leads to cleaner, simpler designs.

Chapter 17: Test-Driven Development -- Writing Tests First

What You Will Learn

- The Red-Green-Refactor cycle and why each step matters
- Why writing tests before production code leads to better design
- A complete TDD walkthrough building a calculator from scratch
- How to take the smallest possible steps in TDD
- When TDD helps the most and when it is overkill
- Common TDD mistakes and how to avoid them

Why This Chapter Matters

Most developers learn to write code first and tests later -- if they write tests at all. Test-Driven Development flips this order. You write a failing test, then write just enough code to pass it, then clean up. This feels backward at first, but it produces code that is testable by design, contains no unnecessary complexity, and has complete test coverage from day one. TDD is not just a testing technique -- it is a design technique that happens to produce tests.

17.1 The Red-Green-Refactor Cycle

TDD follows a strict three-step cycle:

```
          +-------+
          |  RED  |  Write a test that FAILS
```

```
         +---+---+
             |
             v
        +--------+
        | GREEN  |  Write the SIMPLEST code to pass
        +---+----+
            |
            v
     +-----------+
     | REFACTOR  |  Clean up without changing behavior
     +-----+-----+
           |
           +-----------> Back to RED
```

Step 1: RED -- Write a Failing Test

Write a test for the behavior you want. Run it. It must fail. If it passes, either your test is wrong or the behavior already exists.

Why this matters: A test that you never saw fail might be testing nothing. The red step proves the test can detect a problem.

Step 2: GREEN -- Make It Pass

Write the absolute minimum production code to make the failing test pass. Do not worry about elegance. Do not handle edge cases you have not written tests for yet. Just make it green.

Why this matters: Writing only enough code to pass forces you to take small, safe steps and avoid speculative generality.

Step 3: REFACTOR -- Clean Up

Now that the test passes, improve the code structure. Remove duplication, rename variables, extract methods. Run the tests after every change to make sure nothing breaks.

Why this matters: You now have a green safety net. Refactoring under green tests is safe and confident.

The Discipline

```
   +-------------------------------------------------------+
   |              TDD RULES OF THE GAME                    |
   +-------------------------------------------------------+
   |                                                       |
```

```
|  1. Do NOT write production code without a failing    |
|     test that demands it.                             |
|                                                       |
|  2. Do NOT write more of a test than is sufficient    |
|     to fail (including compile failures).             |
|                                                       |
|  3. Do NOT write more production code than is         |
|     sufficient to pass the currently failing test.    |
|                                                       |
+-------------------------------------------------------+
```

17.2 Why Test First?

Tests First vs Tests After

```
TESTS AFTER                          TESTS FIRST (TDD)
+-------------------+                +-------------------+
| Write all code    |                | Write ONE test    |
+-------------------+                +-------------------+
| "Now I should     |                | Write JUST ENOUGH|
|  write tests..." |                | code to pass      |
+-------------------+                +-------------------+
| Tests feel like   |                | Refactor          |
| a chore           |                +-------------------+
+-------------------+                | Repeat            |
| Skip edge cases   |                +-------------------+
+-------------------+                | Tests feel like   |
| Hard-to-test      |                | a design tool     |
| design emerges    |                +-------------------+
+-------------------+
```

Benefits of TDD

1. 100% coverage by construction. Every line of production code exists because a test demanded it.
2. Simpler design. You only write what is needed. YAGNI is built into the process.
3. Immediate feedback. You know within seconds if your change works.
4. Confidence to refactor. The test suite is always comprehensive because you never wrote code without a test.
5. Living documentation. The tests describe every behavior the system supports.

The Design Benefit

When you write tests first, you are forced to think about how the code will be used before you think about how it will be implemented. This naturally leads to:

- Smaller, more focused methods
- Fewer dependencies (because dependencies make testing hard)
- Better interfaces (because you design the API from the caller's perspective)

17.3 TDD Walkthrough: Building a Calculator

Let us build a simple calculator using strict TDD. We will follow every Red-Green-Refactor cycle so you can see how the code evolves.

Iteration 1: Addition of Two Numbers

RED -- Write the failing test (Python):

```python
# test_calculator.py
from calculator import Calculator


def test_add_two_numbers():
    calc = Calculator()
    result = calc.add(2, 3)
    assert result == 5
```

Run: `pytest test_calculator.py` -- FAILS (no module named `calculator`).

GREEN -- Write the minimum code:

```python
# calculator.py
class Calculator:
    def add(self, a, b):
        return a + b
```

Run: `pytest test_calculator.py` -- PASSES.

REFACTOR: Nothing to clean up yet. Move on.

Iteration 2: Subtraction

RED:

```python
def test_subtract_two_numbers():
    calc = Calculator()
    result = calc.subtract(10, 4)
    assert result == 6
```

Run: FAILS (`Calculator` has no method `subtract`).

GREEN:

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b
```

Run: PASSES.

REFACTOR: Still clean. Move on.

Iteration 3: Multiplication

RED:

```python
def test_multiply_two_numbers():
    calc = Calculator()
    result = calc.multiply(3, 7)
    assert result == 21
```

GREEN:

```python
def multiply(self, a, b):
    return a * b
```

Run: PASSES. REFACTOR: Nothing needed.

Iteration 4: Division

RED:

```python
def test_divide_two_numbers():
    calc = Calculator()
    result = calc.divide(10, 4)
    assert result == 2.5
```

GREEN:

```python
def divide(self, a, b):
    return a / b
```

Run: PASSES.

Iteration 5: Division by Zero

RED:

```python
import pytest

def test_divide_by_zero_raises_error():
    calc = Calculator()
    with pytest.raises(ValueError, match="Cannot divide by zero"):
        calc.divide(10, 0)
```

Run: FAILS (raises `ZeroDivisionError`, not `ValueError`).

GREEN:

```python
def divide(self, a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b
```

Run: PASSES.

Iteration 6: Chained Operations (Expression)

RED:

```python
def test_evaluate_simple_expression():
    calc = Calculator()
    # 2 + 3 * 4 should give 14 (multiply first, then add)
    result = calc.evaluate("2 + 3 * 4")
    assert result == 14
```

Run: FAILS (no `evaluate` method).

GREEN -- Start simple:

```python
def evaluate(self, expression):
    tokens = expression.split()
    # Handle single operations first
    if len(tokens) == 3:
        left = float(tokens[0])
        op = tokens[1]
        right = float(tokens[2])
        return self._apply_operator(op, left, right)

    # Handle operator precedence: find * or / first
    # Process multiplication and division first
    values = []
    operators = []
    i = 0
    while i < len(tokens):
        if i % 2 == 0:
            values.append(float(tokens[i]))
        else:
            operators.append(tokens[i])
        i += 1

    # First pass: handle * and /
    j = 0
    while j < len(operators):
        if operators[j] in ('*', '/'):
            result = self._apply_operator(operators[j], values[j], values[j + 1])
            values[j] = result
            values.pop(j + 1)
            operators.pop(j)
        else:
            j += 1

    # Second pass: handle + and -
    result = values[0]
    for j, op in enumerate(operators):
        result = self._apply_operator(op, result, values[j + 1])
    return result

def _apply_operator(self, op, left, right):
```

```
    if op == '+': return self.add(left, right)
    if op == '-': return self.subtract(left, right)
    if op == '*': return self.multiply(left, right)
    if op == '/': return self.divide(left, right)
    raise ValueError(f"Unknown operator: {op}")
```

Run: PASSES.

REFACTOR: The `evaluate` method is getting long. Extract methods:

```python
def evaluate(self, expression):
    values, operators = self._parse_expression(expression)
    values, operators = self._apply_high_precedence(values, operators)
    return self._apply_remaining(values, operators)

def _parse_expression(self, expression):
    tokens = expression.split()
    values = [float(tokens[i]) for i in range(0, len(tokens), 2)]
    operators = [tokens[i] for i in range(1, len(tokens), 2)]
    return values, operators

def _apply_high_precedence(self, values, operators):
    j = 0
    while j < len(operators):
        if operators[j] in ('*', '/'):
            result = self._apply_operator(operators[j], values[j], values[j + 1])
            values[j] = result
            values.pop(j + 1)
            operators.pop(j)
        else:
            j += 1
    return values, operators

def _apply_remaining(self, values, operators):
    result = values[0]
    for j, op in enumerate(operators):
        result = self._apply_operator(op, result, values[j + 1])
    return result
```

Run: All tests still PASS. The refactoring is safe.

The Same Walkthrough in Java

Here is how the same TDD flow looks in Java with JUnit 5:

RED:

```java
class CalculatorTest {

    @Test
    void add_twoNumbers_returnsSum() {
        Calculator calc = new Calculator();
        assertEquals(5, calc.add(2, 3));
    }
}
```

GREEN:

```java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Continue the cycle for each operation:

```java
@Test
void subtract_twoNumbers_returnsDifference() {
    Calculator calc = new Calculator();
    assertEquals(6, calc.subtract(10, 4));
}

@Test
void multiply_twoNumbers_returnsProduct() {
    Calculator calc = new Calculator();
    assertEquals(21, calc.multiply(3, 7));
}

@Test
void divide_twoNumbers_returnsQuotient() {
    Calculator calc = new Calculator();
    assertEquals(2.5, calc.divide(10, 4), 0.001);
}

@Test
void divide_byZero_throwsException() {
    Calculator calc = new Calculator();
    assertThrows(IllegalArgumentException.class,
        () -> calc.divide(10, 0));
}
```

Production code after all cycles:

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public double divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Cannot divide by zero");
        }
        return (double) a / b;
    }
}
```

17.4 The Transformation Priority Premise

Kent Beck and Robert C. Martin suggest taking the simplest possible transformations when going from Red to Green. The priority order (from simplest to most complex):

```
+-------------------------------------------------------+
|         TRANSFORMATION PRIORITY (SIMPLEST FIRST)      |
+-------------------------------------------------------+
|                                                       |
|  1. {} -> nil           Return null/None              |
|  2. nil -> constant     Return a hardcoded value      |
|  3. constant -> variable  Use a parameter             |
|  4. add computation    Add arithmetic                  |
|  5. split flow         Add if/else                     |
|  6. variable -> list   Use a collection               |
|  7. list -> iteration  Add a loop                     |
|  8. iteration -> recursion  Replace loop with recursion|
|                                                       |
+-------------------------------------------------------+
```

Example: FizzBuzz via Transformations

Test 1: Returns "1" for 1

```python
def test_fizzbuzz_1():
    assert fizzbuzz(1) == "1"
```

```python
def fizzbuzz(n):
    return "1"  # Transformation: nil -> constant
```

Test 2: Returns "2" for 2

```python
def test_fizzbuzz_2():
    assert fizzbuzz(2) == "2"
```

```python
def fizzbuzz(n):
    return str(n)  # Transformation: constant -> variable
```

Test 3: Returns "Fizz" for 3

```python
def test_fizzbuzz_3():
    assert fizzbuzz(3) == "Fizz"
```

```python
def fizzbuzz(n):
    if n % 3 == 0:  # Transformation: split flow
        return "Fizz"
    return str(n)
```

Test 4: Returns "Buzz" for 5

```python
def test_fizzbuzz_5():
    assert fizzbuzz(5) == "Buzz"
```

```python
def fizzbuzz(n):
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

Test 5: Returns "FizzBuzz" for 15

```python
def test_fizzbuzz_15():
    assert fizzbuzz(15) == "FizzBuzz"
```

```python
def fizzbuzz(n):
    if n % 15 == 0:
        return "FizzBuzz"
    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"
    return str(n)
```

Each step added only the minimum code needed. The final solution emerged naturally.

17.5 When TDD Helps Most

TDD is most valuable when:

```
+-------------------------------------------------------+
|           TDD SWEET SPOTS                             |
+-------------------------------------------------------+
|                                                       |
|  Business logic      Complex rules with many cases    |
|  Algorithms          Clear inputs and outputs         |
|  Data transformations Parse, convert, validate        |
|  State machines       Well-defined transitions        |
|  Bug fixes            Write a test that reproduces    |
|                       the bug, then fix it            |
|                                                       |
+-------------------------------------------------------+
```

TDD for Bug Fixes

One of the most powerful uses of TDD: when you find a bug, write a test that fails because of the bug BEFORE you fix it. This guarantees:

1. You understand the bug (the test reproduces it)
2. Your fix actually works (the test passes)
3. The bug never comes back (the test stays in the suite)

```python
# Bug report: discount is applied twice for VIP customers

# Step 1: Write a test that exposes the bug
def test_vip_discount_applied_once():
    order = Order(customer_type="VIP", amount=100.00)
    total = order.calculate_total()
    # VIP discount is 10%, so total should be 90.00
    assert total == pytest.approx(90.00)  # FAILS: returns 81.00

# Step 2: Fix the bug
# Step 3: Test passes. Bug can never recur.
```

17.6 When TDD Is Overkill

TDD is not always the right approach:

```
+-------------------------------------------------------+
|           WHEN TDD ADDS LITTLE VALUE                  |
+-------------------------------------------------------+
|                                                       |
|  Exploratory prototypes  You don't know what you're   |
|                          building yet                 |
|                                                       |
|  UI layout code          Visual output is hard to     |
|                          assert meaningfully          |
|                                                       |
|  Glue code               Simple delegation with no    |
|                          logic                        |
|                                                       |
|  One-off scripts         Throwaway code not worth     |
|                          investing in tests           |
|                                                       |
|  Framework boilerplate   Configuration, wiring, etc.  |
|                                                       |
+-------------------------------------------------------+
```

The Pragmatic Approach

You do not have to use TDD for everything. Many experienced developers use TDD for core business logic and algorithms, then write tests-after for simpler code. The important thing is that the code gets tested. TDD is a tool, not a religion.

```
+-------------------------------------------------------+
|  Code Type              Recommended Approach          |
+-------------------------------------------------------+
|  Business rules         TDD (test first)              |
|  Algorithms             TDD (test first)              |
|  Data access layer      Test after (integration)      |
|  UI components          Test after (visual/snapshot)   |
|  Configuration          Test after (smoke tests)       |
|  Prototypes             No tests until design settles  |
+-------------------------------------------------------+
```

17.7 A Complete TDD Session: Building a Password Strength Checker

Let us walk through a realistic TDD session building a feature that a real application might need.

Requirements

- Password must be at least 8 characters
- Must contain at least one uppercase letter
- Must contain at least one lowercase letter
- Must contain at least one digit
- Must contain at least one special character
- Return a strength rating: "weak", "fair", "strong"

TDD Session (Java)

Cycle 1: Empty password is weak

```java
@Test
void emptyPassword_isWeak() {
    PasswordChecker checker = new PasswordChecker();
    assertEquals("weak", checker.checkStrength(""));
}
```

```java
public class PasswordChecker {
    public String checkStrength(String password) {
        return "weak";
    }
}
```

Cycle 2: Short password is weak

```java
@Test
void shortPassword_isWeak() {
    PasswordChecker checker = new PasswordChecker();
    assertEquals("weak", checker.checkStrength("Abc1!"));
}
```

Already passes (returns "weak" for everything). Move on.

Cycle 3: Password meeting all criteria is strong

```java
@Test
void passwordWithAllCriteria_isStrong() {
    PasswordChecker checker = new PasswordChecker();
    assertEquals("strong", checker.checkStrength("MyP@ss1word"));
}
```

FAILS. Now write the logic:

```java
public class PasswordChecker {
    public String checkStrength(String password) {
        int score = 0;
        if (password.length() >= 8) score++;
        if (password.matches(".*[A-Z].*")) score++;
        if (password.matches(".*[a-z].*")) score++;
        if (password.matches(".*[0-9].*")) score++;
        if (password.matches(".*[!@#$%^&*()_+\\-=].*")) score++;

        if (score >= 4) return "strong";
        return "weak";
    }
}
```

Cycle 4: Password meeting some criteria is fair

```java
@Test
void passwordMeetingSomeCriteria_isFair() {
    PasswordChecker checker = new PasswordChecker();
```

```
    assertEquals("fair", checker.checkStrength("Abcdefgh"));
}
```

FAILS (returns "weak" -- score is 3: length + upper + lower). Add "fair":

```java
if (score >= 4) return "strong";
if (score >= 3) return "fair";
return "weak";
```

Cycle 5: Add more edge case tests

```java
@Test
void onlyDigitsAndLength_isWeak() {
    PasswordChecker checker = new PasswordChecker();
    assertEquals("weak", checker.checkStrength("12345678"));
}

@Test
void allLowerWithDigit_isFair() {
    PasswordChecker checker = new PasswordChecker();
    assertEquals("fair", checker.checkStrength("password1"));
}
```

REFACTOR: Extract criteria checks

```java
public class PasswordChecker {

    public String checkStrength(String password) {
        int score = countPassingCriteria(password);
        if (score >= 4) return "strong";
        if (score >= 3) return "fair";
        return "weak";
    }

    private int countPassingCriteria(String password) {
        int score = 0;
        if (hasMinimumLength(password)) score++;
        if (hasUpperCase(password)) score++;
        if (hasLowerCase(password)) score++;
        if (hasDigit(password)) score++;
        if (hasSpecialChar(password)) score++;
        return score;
    }

    private boolean hasMinimumLength(String password) {
        return password.length() >= 8;
    }
```

```
    private boolean hasUpperCase(String password) {
        return password.matches(".*[A-Z].*");
    }

    private boolean hasLowerCase(String password) {
        return password.matches(".*[a-z].*");
    }

    private boolean hasDigit(String password) {
        return password.matches(".*[0-9].*");
    }

    private boolean hasSpecialChar(String password) {
        return password.matches(".*[!@#$%^&*()_+\\-=].*");
    }
}
```

All tests pass. The refactoring is complete. Notice how the final design emerged from the tests -- we did not plan the `countPassingCriteria` method up front.

Common Mistakes

1. Writing too many tests at once. TDD works in single test increments. Writing five failing tests before any production code breaks the rhythm.
2. Writing too much production code. If your green step adds code not demanded by the failing test, you are guessing about future needs.
3. Skipping the refactoring step. Red-Green without Refactor leads to code that works but is messy. The refactoring step is not optional.
4. Testing private methods directly. Test behavior through public methods. Private methods are implementation details that should be free to change.
5. Giving up too early. TDD feels slow at first. The productivity gains come after you build the habit, typically after a few weeks of practice.

Best Practices

1. Start with the simplest possible test. The first test should be almost trivially easy to pass.

2. Take the smallest step that makes a test pass. Resist the urge to write the full solution on the first green step.
3. Run tests after every change. After writing a test, after writing production code, after every refactoring step.
4. Refactor both production code AND test code. Tests deserve the same care as production code. Extract helpers, remove duplication.
5. Use TDD for bug fixes. Write a failing test that reproduces the bug, then fix it. This is one of TDD's highest-value applications.

Quick Summary

Concept	Description
Red	Write a test that fails
Green	Write the minimum code to pass
Refactor	Clean up while tests stay green
Test First	Design the API before the implementation
Transformation Priority	Take the simplest possible step to go green
TDD Sweet Spots	Business logic, algorithms, bug fixes
TDD Overkill	Prototypes, UI layout, throwaway scripts

Key Points

- TDD is a design technique that produces tests, not a testing technique that produces design.
- The Red-Green-Refactor cycle ensures you never write untested code and never write unnecessary code.
- Writing the test first forces you to design the interface from the caller's perspective.
- Take the smallest possible steps. Each cycle should take minutes, not hours.

- TDD is most valuable for business logic and algorithms. Use judgment about when to apply it.

Practice Questions

1. Explain why it is important to see the test FAIL before writing production code. What problem does this prevent?
2. You are writing a sorting algorithm using TDD. What would your first three tests be? (Hint: start with the simplest cases.)
3. A colleague writes a 50-line method in the GREEN step to pass a single test. What TDD principle did they violate, and what should they have done instead?
4. Your team argues that TDD is too slow because writing tests doubles the amount of code. How would you respond?
5. When is it appropriate to write tests AFTER the production code, even on a team that practices TDD?

Exercises

Exercise 1: TDD FizzBuzz in Java

Implement FizzBuzz using strict TDD in Java:

- `fizzbuzz(1)` returns `"1"`
- `fizzbuzz(3)` returns `"Fizz"`
- `fizzbuzz(5)` returns `"Buzz"`
- `fizzbuzz(15)` returns `"FizzBuzz"`

Write one test at a time. Commit after each Green phase. Show your commit history to prove you followed the cycle.

Exercise 2: TDD Roman Numeral Converter

Using TDD in Python, build a function that converts integers to Roman numerals:

- `to_roman(1)` returns `"I"`
- `to_roman(4)` returns `"IV"`

- `to_roman(9)` returns `"IX"`
- `to_roman(2024)` returns `"MMXXIV"`

Start with the simplest case and work up. How many Red-Green-Refactor cycles did you need?

Exercise 3: TDD Bug Fix

Given this buggy code:

```python
def calculate_average(numbers):
    total = 0
    for n in numbers:
        total += n
    return total / len(numbers)
```

There is a bug: it crashes on an empty list. Use TDD to fix it:

1. Write a test that fails because of the bug
2. Fix the bug (return 0 for an empty list)
3. Write additional tests for normal cases
4. Refactor if needed

What Is Next?

You now understand TDD and unit testing. But tests and clean code exist within a larger structure -- the architecture of your application. Chapter 18: Clean Architecture Introduction explains how to organize your entire application so that business rules stay independent from frameworks, databases, and user interfaces.

Chapter 18: Clean Architecture Introduction -- Separating What Matters

What You Will Learn

- What software architecture means and why it matters for every developer
- The principle of Separation of Concerns and how it shapes good architecture
- Why business rules should be independent of frameworks, databases, and UIs
- The Dependency Rule and why dependencies must always point inward
- Uncle Bob's Clean Architecture with its concentric circles
- How to read and apply the Clean Architecture diagram to real projects

Why This Chapter Matters

You can write perfectly clean functions, well-named classes, and comprehensive tests -- and still end up with an application that is painful to change. Why? Because the architecture is tangled. Your business rules depend on your database. Your database schema is embedded in your UI. Changing one thing means changing everything. Clean Architecture solves this by giving you a simple rule: separate your code into layers, and make sure dependencies always point inward toward your business rules. The result is a system where you can swap frameworks, change databases, and rewrite UIs without touching the core logic.

18.1 What Is Software Architecture?

Software architecture is the set of decisions about how your code is organized into components, how those components communicate, and which components depend on which.

```
+-------------------------------------------------------+
|            ARCHITECTURE IS ABOUT BOUNDARIES            |
+-------------------------------------------------------+
|                                                        |
|  Which code knows about which other code?              |
|  What can change without affecting other parts?        |
|  Where do you draw the lines between components?       |
|                                                        |
+-------------------------------------------------------+
```

Architecture Is Not About Tools

A common misconception: "Our architecture is Spring Boot + PostgreSQL + React."

Those are tools, not architecture. Architecture is about how your code is structured so that:

- Business rules do not depend on the UI framework
- Use cases do not depend on the database
- Core logic can be tested without starting a web server

The Cost of Bad Architecture

```
Cost of Change
^
|            ___________
|           /            Bad Architecture
|          /
|         /
|        /
|   ____/
|  /
| /     _______________ Good Architecture
|/______/
+-------------------------> Time
```

With bad architecture, every change gets more expensive over time. With good architecture, the cost of change stays relatively flat.

18.2 Separation of Concerns

The most fundamental principle of architecture: every piece of code should have one reason to exist, and different concerns should live in different places.

What Are "Concerns"?

```
+-------------------------------------------------------------+
|                    COMMON CONCERNS                          |
+-------------------------------------------------------------+
|                                                             |
|  Business logic     What the application does               |
|  Persistence        How data is stored and retrieved        |
|  Presentation       How data is shown to the user           |
|  Communication      How systems talk to each other          |
|  Security           Who can do what                         |
|  Logging            What happened and when                  |
|                                                             |
+-------------------------------------------------------------+
```

BEFORE: Concerns Mixed Together (Java)

```java
public class OrderController {

    @PostMapping("/orders")
    public ResponseEntity<String> createOrder(@RequestBody OrderRequest req) {
        // Validation (business logic)
        if (req.getItems().isEmpty()) {
            return ResponseEntity.badRequest().body("No items");
        }

        // Business rule (business logic)
        double total = 0;
        for (Item item : req.getItems()) {
            total += item.getPrice() * item.getQuantity();
        }
        if (total > 10000) {
            total *= 0.95; // 5% discount for large orders
        }

        // Persistence (data access)
        Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/shop");
        PreparedStatement stmt = conn.prepareStatement(
            "INSERT INTO orders (customer_id, total) VALUES (?, ?)");
        stmt.setInt(1, req.getCustomerId());
        stmt.setDouble(2, total);
        stmt.executeUpdate();

        // Notification (external service)
        HttpClient client = HttpClient.newHttpClient();
```

```
        client.send(HttpRequest.newBuilder()
            .uri(URI.create("https://api.email.com/send"))
            .POST(HttpRequest.BodyPublishers.ofString(
                "{\"to\":\"" + req.getEmail() + "\",\"subject\":\"Order Placed\"}"))
            .build(), HttpResponse.BodyHandlers.ofString());

        // Presentation (response formatting)
        return ResponseEntity.ok("{\"status\":\"created\",\"total\":" + total + "}");
    }
}
```

This single method handles validation, business rules, database access, external API calls, and response formatting. Changing any one concern risks breaking the others.

AFTER: Concerns Separated (Java)

```java
// Business logic -- no framework dependencies
public class OrderService {
    private final OrderRepository orderRepo;
    private final NotificationService notifier;

    public Order createOrder(int customerId, List<Item> items) {
        if (items.isEmpty()) {
            throw new ValidationException("No items in order");
        }
        double total = calculateTotal(items);
        Order order = orderRepo.save(new Order(customerId, total));
        notifier.orderCreated(order);
        return order;
    }

    private double calculateTotal(List<Item> items) {
        double total = items.stream()
            .mapToDouble(i -> i.getPrice() * i.getQuantity())
            .sum();
        if (total > 10000) {
            total *= 0.95;
        }
        return total;
    }
}

// Presentation -- only handles HTTP concerns
@RestController
public class OrderController {
    private final OrderService orderService;

    @PostMapping("/orders")
    public ResponseEntity<OrderResponse> createOrder(@RequestBody OrderRequest req) {
        Order order = orderService.createOrder(req.getCustomerId(), req.getItems());
```

```
        return ResponseEntity.ok(new OrderResponse(order));
    }
}

// Persistence -- only handles data storage
public class JpaOrderRepository implements OrderRepository {
    public Order save(Order order) { /* JPA logic */ }
}

// Notification -- only handles external communication
public class EmailNotificationService implements NotificationService {
    public void orderCreated(Order order) { /* Email logic */ }
}
```

Now each class has one concern. The `OrderService` knows nothing about HTTP, SQL, or email APIs.

18.3 Independence: The Goal of Clean Architecture

Clean Architecture aims to make your system independent of:

```
+-------------------------------------------------------+
|           INDEPENDENCE GOALS                          |
+-------------------------------------------------------+
|                                                       |
|  1. Independent of FRAMEWORKS                         |
|     The framework is a tool, not the architecture.    |
|     You can swap Spring for Micronaut.                |
|                                                       |
|  2. Independent of the UI                             |
|     The UI can change from web to mobile to CLI       |
|     without changing business rules.                  |
|                                                       |
|  3. Independent of the DATABASE                       |
|     You can swap PostgreSQL for MongoDB               |
|     without changing business rules.                  |
|                                                       |
|  4. Independent of EXTERNAL AGENCIES                  |
|     Business rules do not know about the outside      |
|     world (APIs, file systems, etc).                  |
|                                                       |
|  5. TESTABLE                                          |
|     Business rules can be tested without the UI,      |
|     database, web server, or any external element.    |
|                                                       |
+-------------------------------------------------------+
```

18.4 The Dependency Rule

The Dependency Rule is the single most important rule in Clean Architecture:

> Source code dependencies must point only inward, toward higher-level policies.

```
+-------------------------------------------------------------+
|                    THE DEPENDENCY RULE                      |
+-------------------------------------------------------------+
|                                                             |
|  Outer layers can depend on inner layers.                   |
|  Inner layers MUST NOT depend on outer layers.              |
|  Inner layers MUST NOT know that outer layers exist.        |
|                                                             |
+-------------------------------------------------------------+
```

What "Inward" Means

"Inward" means toward the center of the architecture -- toward the most important, most stable, most abstract parts of the system. Business rules are at the center. Frameworks, databases, and UIs are at the edges.

Dependency Direction

```
+-----------------------------------------------------+
|                                                     |
|  Framework  ----depends on---->  Use Case           |
|  Controller ----depends on---->  Business Rule      |
|  Database   ----depends on---->  Entity             |
|  UI         ----depends on---->  Service            |
|                                                     |
|  NEVER the reverse.                                 |
|                                                     |
+-----------------------------------------------------+
```

How Dependency Inversion Makes This Work

The inner layer defines an interface. The outer layer implements it. The inner layer never imports, references, or knows about the outer layer.

Java Example:

```java
// INNER LAYER: defines the interface (what it needs)
public interface OrderRepository {
    Order save(Order order);
    Order findById(int id);
}

// INNER LAYER: uses the interface
public class OrderService {
    private final OrderRepository repository;  // Depends on abstraction

    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }

    public Order placeOrder(Order order) {
        // Business logic here
        return repository.save(order);
    }
}

// OUTER LAYER: implements the interface
public class PostgresOrderRepository implements OrderRepository {
    public Order save(Order order) {
        // SQL INSERT logic
    }
    public Order findById(int id) {
        // SQL SELECT logic
    }
}
```

Python Example:

```python
# INNER LAYER: defines the interface
from abc import ABC, abstractmethod

class OrderRepository(ABC):
    @abstractmethod
    def save(self, order): ...

    @abstractmethod
    def find_by_id(self, order_id): ...


# INNER LAYER: uses the interface
class OrderService:
    def __init__(self, repository: OrderRepository):
        self.repository = repository

    def place_order(self, order):
        # Business logic here
        return self.repository.save(order)
```

```
# OUTER LAYER: implements the interface
class PostgresOrderRepository(OrderRepository):
    def save(self, order):
        # SQL INSERT logic
        ...

    def find_by_id(self, order_id):
        # SQL SELECT logic
        ...
```

The `OrderService` depends on the `OrderRepository` interface, which lives in the same inner layer. It has NO dependency on `PostgresOrderRepository`. You could swap in `MongoOrderRepository` or `InMemoryOrderRepository` without touching `OrderService`.

18.5 The Concentric Circles

Uncle Bob's Clean Architecture is commonly shown as four concentric circles:

```
  +-----------------------------------------------------------+
  |                                                           |
  |    Frameworks & Drivers (outermost)                       |
  |  +-------------------------------------------------------+ |
  |  |                                                       | |
  |  |    Interface Adapters                                 | |
  |  |  +----------------------------------------------------+ |
  |  |  |                                                    | |  |
  |  |  |    Application Business Rules (Use Cases)          | |  |
  |  |  |  +-------------------------------------------------+ |  |
  |  |  |  |                                                 | |  |
  |  |  |  |    Enterprise Business Rules (Entities)         | |  |
  |  |  |  |                                                 | |  |
  |  |  |  +-------------------------------------------------+ |  |
  |  |  |                                                    | |  |
  |  |  +----------------------------------------------------+ |
  |  |                                                       | |
  |  +-------------------------------------------------------+ |
  |                                                           |
  +-----------------------------------------------------------+

              Dependencies point INWARD only -->
```

Layer 1: Entities (Center)

Entities encapsulate the most general and high-level business rules. They are the core domain objects that would exist regardless of the application.

```java
// Entity: pure business rules, no dependencies on anything external
public class LoanApplication {
    private double requestedAmount;
    private double annualIncome;
    private int creditScore;

    public boolean isEligible() {
        return creditScore >= 650
            && requestedAmount <= annualIncome * 3;
    }

    public double calculateRate() {
        if (creditScore >= 750) return 0.035;
        if (creditScore >= 700) return 0.045;
        return 0.065;
    }
}
```

```python
class LoanApplication:
    def __init__(self, requested_amount, annual_income, credit_score):
        self.requested_amount = requested_amount
        self.annual_income = annual_income
        self.credit_score = credit_score

    def is_eligible(self):
        return (self.credit_score >= 650
                and self.requested_amount <= self.annual_income * 3)

    def calculate_rate(self):
        if self.credit_score >= 750:
            return 0.035
        if self.credit_score >= 700:
            return 0.045
        return 0.065
```

Layer 2: Use Cases (Application Business Rules)

Use cases contain application-specific business rules. They orchestrate the flow of data to and from entities and direct them to use their enterprise-level rules.

```java
public class ProcessLoanUseCase {
    private final LoanRepository loanRepo;
    private final CreditCheckService creditCheck;
    private final NotificationService notifier;
```

```
    public ProcessLoanUseCase(LoanRepository loanRepo,
                              CreditCheckService creditCheck,
                              NotificationService notifier) {
        this.loanRepo = loanRepo;
        this.creditCheck = creditCheck;
        this.notifier = notifier;
    }

    public LoanResult process(LoanRequest request) {
        int creditScore = creditCheck.getScore(request.getApplicantId());

        LoanApplication application = new LoanApplication(
            request.getAmount(),
            request.getIncome(),
            creditScore
        );

        if (!application.isEligible()) {
            notifier.loanDenied(request.getApplicantId());
            return LoanResult.denied("Does not meet eligibility criteria");
        }

        double rate = application.calculateRate();
        Loan loan = loanRepo.save(new Loan(request, rate));
        notifier.loanApproved(request.getApplicantId(), loan);
        return LoanResult.approved(loan);
    }
}
```

```python
class ProcessLoanUseCase:
    def __init__(self, loan_repo, credit_check, notifier):
        self.loan_repo = loan_repo
        self.credit_check = credit_check
        self.notifier = notifier

    def process(self, request):
        credit_score = self.credit_check.get_score(request.applicant_id)

        application = LoanApplication(
            request.amount, request.income, credit_score
        )

        if not application.is_eligible():
            self.notifier.loan_denied(request.applicant_id)
            return LoanResult.denied("Does not meet eligibility criteria")

        rate = application.calculate_rate()
        loan = self.loan_repo.save(Loan(request, rate))
        self.notifier.loan_approved(request.applicant_id, loan)
        return LoanResult.approved(loan)
```

Layer 3: Interface Adapters

Interface adapters convert data between the format used by use cases and the format used by external agencies (database, web, etc.).

```java
// Controller: converts HTTP request to use case input
@RestController
public class LoanController {
    private final ProcessLoanUseCase useCase;

    @PostMapping("/loans")
    public ResponseEntity<LoanResponse> apply(@RequestBody LoanHttpRequest httpReq) {
        LoanRequest domainRequest = new LoanRequest(
            httpReq.getApplicantId(),
            httpReq.getAmount(),
            httpReq.getIncome()
        );
        LoanResult result = useCase.process(domainRequest);
        return ResponseEntity.ok(LoanResponse.from(result));
    }
}

// Repository implementation: converts domain objects to database rows
public class JpaLoanRepository implements LoanRepository {
    private final JpaLoanEntityRepository jpaRepo;

    public Loan save(Loan loan) {
        LoanEntity entity = LoanEntity.from(loan);
        jpaRepo.save(entity);
        return loan;
    }
}
```

Layer 4: Frameworks and Drivers (Outermost)

This is where frameworks, tools, and external systems live. Spring Boot, Django, PostgreSQL, RabbitMQ. This layer has the least amount of your code -- it is mostly configuration and glue.

18.6 Data Crossing Boundaries

When data crosses a boundary between layers, it should be in the form that is most convenient for the inner layer. The outer layer does the conversion.

```
+-----------------------------------------------------------------+
|                                                                  |
|  HTTP Request  -->  Controller  -->  Use Case Input DTO         |
|                     (converts)       (simple data object)        |
|                                                                  |
|  Use Case Output DTO  -->  Controller  -->  HTTP Response       |
|  (simple data object)      (converts)                            |
|                                                                  |
|  Entity  -->  Repository  -->  Database Row                     |
|               (converts)                                         |
|                                                                  |
+-----------------------------------------------------------------+
```

Why Not Just Pass Entities Everywhere?

If your controller returns an Entity directly as JSON, then:

- Your API contract is coupled to your domain model
- Adding a field to the entity automatically exposes it in the API
- The entity gains JSON annotations, making it dependent on the web framework

Keep entities clean. Use DTOs (Data Transfer Objects) to cross boundaries.

18.7 The Dependency Rule in Practice

Here is a complete dependency map for a typical Clean Architecture application:

```
+-----------------------------------------------------------------+
|  OUTER: Frameworks & Drivers                                     |
|    Spring Boot, Django, PostgreSQL driver, HTTP client           |
|                         |                                        |
|                         | depends on                            |
|                         v                                        |
|  LAYER 3: Interface Adapters                                     |
|    Controllers, Repositories (impl), Presenters                 |
|                         |                                        |
|                         | depends on                            |
|                         v                                        |
|  LAYER 2: Use Cases                                              |
|    Application services, input/output ports                     |
|                         |                                        |
|                         | depends on                            |
|                         v                                        |
|  LAYER 1: Entities (CENTER)                                      |
```

```
  |    Domain objects, business rules                            |
  |    No dependencies on anything else                          |
  +--------------------------------------------------------------+
```

Testing Benefits

Because inner layers have no outward dependencies, you can test them with no framework, no database, and no web server:

```java
@Test
void processLoan_eligibleApplicant_approvesLoan() {
    // All dependencies are fakes -- no Spring, no database
    LoanRepository fakeRepo = new InMemoryLoanRepository();
    CreditCheckService fakeCredit = applicantId -> 720;
    NotificationService fakeNotifier = new NoOpNotificationService();

    ProcessLoanUseCase useCase = new ProcessLoanUseCase(
        fakeRepo, fakeCredit, fakeNotifier
    );

    LoanRequest request = new LoanRequest("APP-001", 50000, 80000);
    LoanResult result = useCase.process(request);

    assertTrue(result.isApproved());
}
```

```python
def test_process_loan_eligible_applicant():
    fake_repo = InMemoryLoanRepository()
    fake_credit = lambda applicant_id: 720
    fake_notifier = NoOpNotificationService()

    use_case = ProcessLoanUseCase(fake_repo, fake_credit, fake_notifier)

    request = LoanRequest("APP-001", 50000, 80000)
    result = use_case.process(request)

    assert result.is_approved
```

No framework startup. No database connection. Runs in milliseconds.

Common Mistakes

1. Letting entities depend on the framework. Putting `@Entity`, `@Column`, or ORM annotations on domain objects couples your core to the persistence framework.
2. Skipping the interface. Directly importing `PostgresRepository` in your use case instead of depending on a `Repository` interface violates the Dependency Rule.
3. Over-engineering for small projects. A TODO app does not need four layers. Apply Clean Architecture proportionally to the project's complexity.
4. Treating architecture as a one-time decision. Architecture evolves. Start simple and add boundaries as the system grows.
5. Confusing layers with folders. Having folders named `entities/`, `usecases/`, and `adapters/` means nothing if the code inside them has tangled dependencies. The Dependency Rule must be enforced in actual import statements.

Best Practices

1. Start with two layers. Domain (business rules) and Infrastructure (everything else). Split further when needed.
2. Define interfaces in the inner layer. The inner layer declares what it needs. The outer layer provides the implementation.
3. Use dependency injection. Pass dependencies through constructors so that inner layers never create outer-layer objects.
4. Keep entities framework-free. No annotations from Spring, Django, or any ORM on your core domain objects.
5. Test the inner layers first. If your entities and use cases are well-tested, you have high confidence in your business logic regardless of what happens at the edges.

Quick Summary

Concept	Definition
Separation of Concerns	Each component handles one concern
Independence	Business rules do not depend on frameworks, UI, or databases
Dependency Rule	Dependencies point inward only

Entities	Core business rules, no external dependencies
Use Cases	Application-specific business rules
Interface Adapters	Convert data between layers
Frameworks & Drivers	External tools, outermost layer

Key Points

- Architecture determines how expensive it is to change your system over time.
- The Dependency Rule is the single most important principle: inner layers must not depend on outer layers.
- Entities contain enterprise-wide business rules. Use cases contain application-specific rules.
- Interface adapters convert between the formats of inner and outer layers.
- Clean Architecture makes your business logic testable without any framework or infrastructure.

Practice Questions

1. Your `UserService` class imports `org.springframework.stereotype.Service`. Which Clean Architecture layer does it belong in? Does the annotation violate the Dependency Rule?
2. Draw the dependency direction for these components: `UserController`, `UserService`, `UserRepository` (interface), `PostgresUserRepository`. Which component should know about which?
3. You need to switch from PostgreSQL to MongoDB. If you followed Clean Architecture, which layers would need to change? Which would remain untouched?
4. Explain why passing a JPA `@Entity` object directly to a REST controller response violates Clean Architecture principles.
5. A teammate says "Clean Architecture has too many layers for our simple app." How would you respond? When is it appropriate to use fewer layers?

Exercises

Exercise 1: Identify the Violations

This code has multiple Clean Architecture violations. Find them all:

```java
public class OrderService {
    @Autowired
    private JdbcTemplate jdbcTemplate;

    public ResponseEntity<String> processOrder(HttpServletRequest request) {
        String json = request.getReader().lines().collect(Collectors.joining());
        JSONObject data = new JSONObject(json);

        double total = data.getDouble("amount") * 1.08;

        jdbcTemplate.update("INSERT INTO orders VALUES (?, ?)",
            data.getString("customerId"), total);

        return ResponseEntity.ok("Order placed: $" + total);
    }
}
```

Exercise 2: Refactor to Clean Architecture

Take the code from Exercise 1 and refactor it into proper Clean Architecture layers:

- Create an `Order` entity with business logic
- Create a `PlaceOrderUseCase` with application logic
- Create an `OrderRepository` interface and implementation
- Create a controller that only handles HTTP concerns

Exercise 3: Dependency Map

Draw a dependency map for a blog application with these features:

- Users can create and edit posts
- Posts are stored in a database
- An email is sent when a post is published
- Posts are displayed on a web page

Identify the entities, use cases, interface adapters, and frameworks. Draw arrows showing which component depends on which. Verify that all arrows point inward.

What Is Next?

Clean Architecture gives you the principles. The next three chapters show you specific architectural patterns that implement these principles. Chapter 19: Layered Architecture covers the most common pattern -- the three-tier architecture used by the vast majority of business applications.

Chapter 19: Layered Architecture -- The Three-Tier Foundation

What You Will Learn

- The three-tier layered architecture pattern: Presentation, Business, and Data
- What each layer is responsible for and what it should never do
- How layers communicate through well-defined interfaces
- Benefits and drawbacks of the layered approach
- A complete Spring Boot example showing all three layers
- A complete Python (Flask) example for comparison
- When layered architecture is the right choice and when to consider alternatives

Why This Chapter Matters

Layered architecture is the most widely used architectural pattern in the industry. When someone says "we have a controller, a service, and a repository," they are describing layered architecture. It is the default pattern for most web applications, enterprise systems, and APIs. Understanding it deeply is essential because it is likely the pattern you will work with most, and it is also the pattern whose limitations drive the adoption of more advanced architectures like hexagonal and clean architecture.

19.1 What Is Layered Architecture?

Layered architecture divides an application into horizontal layers, each with a distinct responsibility. Each layer only communicates with the layer directly below it.

```
+-----------------------------------------------------+
|                                                     |
|              PRESENTATION LAYER                     |
|         (Controllers, Views, API endpoints)         |
|                                                     |
|                      | calls                       |
|                      v                              |
+-----------------------------------------------------+
|                                                     |
|              BUSINESS LOGIC LAYER                   |
|         (Services, Domain rules, Validation)        |
|                                                     |
|                      | calls                       |
|                      v                              |
+-----------------------------------------------------+
|                                                     |
|              DATA ACCESS LAYER                      |
|         (Repositories, DAOs, ORM, SQL)              |
|                                                     |
+-----------------------------------------------------+
|                                                     |
|              DATABASE                               |
|         (PostgreSQL, MySQL, MongoDB)                |
|                                                     |
+-----------------------------------------------------+
```

The Core Rule

Each layer depends only on the layer directly below it. The Presentation layer calls the Business layer. The Business layer calls the Data Access layer. No layer reaches past its neighbor.

```
+-------------------------------------------------------------+
|              ALLOWED              |    NOT ALLOWED          |
+-------------------------------------------------------------+
|  Presentation -> Business         | Presentation -> Data    |
|  Business -> Data Access          | Data Access -> Business|
|                                   | Business -> Presentation|
+-------------------------------------------------------------+
```

19.2 The Presentation Layer

Responsibility

The Presentation layer handles all interaction with the outside world. For a web application, this means HTTP requests and responses. For a desktop app, this means UI events and screen updates.

What It Should Do

- Accept incoming requests (HTTP, CLI, events)
- Validate input format (is it valid JSON? are required fields present?)
- Call the appropriate business logic
- Format the response for the client
- Handle HTTP status codes, content types, headers

What It Should NOT Do

- Contain business rules or calculations
- Access the database directly
- Know about SQL, table names, or data formats
- Make decisions about data processing

Spring Boot Controller (Java)

```java
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public ResponseEntity<List<ProductResponse>> getAllProducts() {
        List<Product> products = productService.findAll();
        List<ProductResponse> response = products.stream()
                .map(ProductResponse::from)
                .collect(Collectors.toList());
        return ResponseEntity.ok(response);
    }

    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getProduct(@PathVariable Long id) {
        Product product = productService.findById(id);
        return ResponseEntity.ok(ProductResponse.from(product));
```

```
    }

    @PostMapping
    public ResponseEntity<ProductResponse> createProduct(
            @Valid @RequestBody CreateProductRequest request) {
        Product product = productService.create(
                request.getName(),
                request.getPrice(),
                request.getCategory()
        );
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(ProductResponse.from(product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Flask Controller (Python)

```python
from flask import Flask, request, jsonify
from product_service import ProductService

app = Flask(__name__)
product_service = ProductService()


@app.route("/api/products", methods=["GET"])
def get_all_products():
    products = product_service.find_all()
    return jsonify([p.to_dict() for p in products])


@app.route("/api/products/<int:product_id>", methods=["GET"])
def get_product(product_id):
    product = product_service.find_by_id(product_id)
    if not product:
        return jsonify({"error": "Product not found"}), 404
    return jsonify(product.to_dict())


@app.route("/api/products", methods=["POST"])
def create_product():
    data = request.get_json()
    if not data.get("name") or not data.get("price"):
        return jsonify({"error": "Name and price are required"}), 400
```

```
    product = product_service.create(
        name=data["name"],
        price=data["price"],
        category=data.get("category", "general")
    )
    return jsonify(product.to_dict()), 201


@app.route("/api/products/<int:product_id>", methods=["DELETE"])
def delete_product(product_id):
    product_service.delete(product_id)
    return "", 204
```

19.3 The Business Logic Layer

Responsibility

The Business Logic layer contains the rules that define what your application does. It is the heart of the system -- the code that would exist regardless of whether the interface is a web app, mobile app, or CLI.

What It Should Do

- Enforce business rules and constraints
- Orchestrate operations across multiple data sources
- Perform calculations, transformations, and validations
- Define the domain model (entities, value objects)

What It Should NOT Do

- Know about HTTP, HTML, or any presentation format
- Import web framework classes
- Handle request parsing or response formatting
- Contain SQL queries or direct database access

Service Class (Java)

```java
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;

    public ProductService(ProductRepository productRepository,
                          CategoryRepository categoryRepository) {
        this.productRepository = productRepository;
        this.categoryRepository = categoryRepository;
    }

    public List<Product> findAll() {
        return productRepository.findAll();
    }

    public Product findById(Long id) {
        return productRepository.findById(id)
                .orElseThrow(() -> new ProductNotFoundException(
                        "Product not found: " + id));
    }

    public Product create(String name, double price, String categoryName) {
        if (price <= 0) {
            throw new InvalidProductException("Price must be positive");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidProductException("Name cannot be empty");
        }
        if (productRepository.existsByName(name)) {
            throw new DuplicateProductException("Product already exists: " + name);
        }

        Category category = categoryRepository.findByName(categoryName)
                .orElseGet(() -> categoryRepository.save(new Category(categoryName)));

        Product product = new Product(name, price, category);
        return productRepository.save(product);
    }

    public void delete(Long id) {
        Product product = findById(id);
        if (product.hasActiveOrders()) {
            throw new BusinessRuleException(
                    "Cannot delete product with active orders");
        }
        productRepository.delete(product);
    }

    public Product applyDiscount(Long productId, double discountPercent) {
        if (discountPercent < 0 || discountPercent > 50) {
            throw new InvalidProductException(
                    "Discount must be between 0% and 50%");
        }
```

```
        Product product = findById(productId);
        product.applyDiscount(discountPercent);
        return productRepository.save(product);
    }
}
```

Service Class (Python)

```python
class ProductService:

    def __init__(self, product_repo=None, category_repo=None):
        self.product_repo = product_repo or ProductRepository()
        self.category_repo = category_repo or CategoryRepository()

    def find_all(self):
        return self.product_repo.find_all()

    def find_by_id(self, product_id):
        product = self.product_repo.find_by_id(product_id)
        if not product:
            raise ProductNotFoundError(f"Product not found: {product_id}")
        return product

    def create(self, name, price, category="general"):
        if price <= 0:
            raise InvalidProductError("Price must be positive")
        if not name or not name.strip():
            raise InvalidProductError("Name cannot be empty")
        if self.product_repo.exists_by_name(name):
            raise DuplicateProductError(f"Product already exists: {name}")

        cat = self.category_repo.find_by_name(category)
        if not cat:
            cat = self.category_repo.save(Category(category))

        product = Product(name=name, price=price, category=cat)
        return self.product_repo.save(product)

    def delete(self, product_id):
        product = self.find_by_id(product_id)
        if product.has_active_orders():
            raise BusinessRuleError("Cannot delete product with active orders")
        self.product_repo.delete(product)

    def apply_discount(self, product_id, discount_percent):
        if not 0 <= discount_percent <= 50:
            raise InvalidProductError("Discount must be between 0% and 50%")
        product = self.find_by_id(product_id)
        product.apply_discount(discount_percent)
        return self.product_repo.save(product)
```

The Domain Model

```java
public class Product {
    private Long id;
    private String name;
    private double price;
    private double originalPrice;
    private Category category;
    private List<Order> activeOrders;

    public Product(String name, double price, Category category) {
        this.name = name;
        this.price = price;
        this.originalPrice = price;
        this.category = category;
        this.activeOrders = new ArrayList<>();
    }

    public void applyDiscount(double percent) {
        this.price = originalPrice * (1 - percent / 100);
    }

    public boolean hasActiveOrders() {
        return !activeOrders.isEmpty();
    }

    // Getters...
}
```

19.4 The Data Access Layer

Responsibility

The Data Access layer handles all communication with data storage systems. It translates between domain objects and database records.

What It Should Do

- Execute database queries (SQL, NoSQL, etc.)
- Map database rows to domain objects and vice versa
- Handle connection pooling and transaction management
- Provide a clean interface to the business layer

What It Should NOT Do

- Contain business rules
- Make decisions about which data to return based on business logic
- Format data for the UI
- Know about HTTP or any presentation concern

Repository Interface and Implementation (Java / Spring Data)

```java
// Interface (used by the business layer)
public interface ProductRepository {
    List<Product> findAll();
    Optional<Product> findById(Long id);
    boolean existsByName(String name);
    Product save(Product product);
    void delete(Product product);
}

// Implementation using Spring Data JPA
@Repository
public class JpaProductRepository implements ProductRepository {

    private final SpringDataProductRepository springRepo;

    public JpaProductRepository(SpringDataProductRepository springRepo) {
        this.springRepo = springRepo;
    }

    @Override
    public List<Product> findAll() {
        return springRepo.findAll().stream()
                .map(this::toDomain)
                .collect(Collectors.toList());
    }

    @Override
    public Optional<Product> findById(Long id) {
        return springRepo.findById(id).map(this::toDomain);
    }

    @Override
    public boolean existsByName(String name) {
        return springRepo.existsByName(name);
    }

    @Override
    public Product save(Product product) {
        ProductEntity entity = toEntity(product);
        ProductEntity saved = springRepo.save(entity);
        return toDomain(saved);
```

```
    }

    @Override
    public void delete(Product product) {
        springRepo.deleteById(product.getId());
    }

    private Product toDomain(ProductEntity entity) {
        return new Product(entity.getId(), entity.getName(),
                entity.getPrice(), entity.getCategory());
    }

    private ProductEntity toEntity(Product product) {
        return new ProductEntity(product.getId(), product.getName(),
                product.getPrice(), product.getCategory());
    }
}

// Spring Data interface
interface SpringDataProductRepository extends JpaRepository<ProductEntity, Long> {
    boolean existsByName(String name);
}
```

Repository (Python with SQLAlchemy)

```python
from sqlalchemy.orm import Session
from models import ProductModel


class ProductRepository:

    def __init__(self, session: Session = None):
        self.session = session or get_default_session()

    def find_all(self):
        rows = self.session.query(ProductModel).all()
        return [self._to_domain(row) for row in rows]

    def find_by_id(self, product_id):
        row = self.session.query(ProductModel).get(product_id)
        return self._to_domain(row) if row else None

    def exists_by_name(self, name):
        return self.session.query(ProductModel).filter_by(name=name).count() > 0

    def save(self, product):
        entity = self._to_entity(product)
        self.session.add(entity)
        self.session.commit()
        return self._to_domain(entity)
```

```
    def delete(self, product):
        entity = self.session.query(ProductModel).get(product.id)
        if entity:
            self.session.delete(entity)
            self.session.commit()

    def _to_domain(self, entity):
        return Product(
            id=entity.id,
            name=entity.name,
            price=entity.price,
            category=entity.category
        )

    def _to_entity(self, product):
        return ProductModel(
            id=product.id,
            name=product.name,
            price=product.price,
            category=product.category
        )
```

19.5 Complete Spring Boot Example: Project Structure

Here is how a layered architecture looks as a project structure:

```
src/main/java/com/example/shop/
+-- controller/                  <-- Presentation Layer
|   +-- ProductController.java
|   +-- OrderController.java
|   +-- dto/
|       +-- CreateProductRequest.java
|       +-- ProductResponse.java
|
+-- service/                     <-- Business Logic Layer
|   +-- ProductService.java
|   +-- OrderService.java
|   +-- exception/
|       +-- ProductNotFoundException.java
|       +-- InvalidProductException.java
|
+-- domain/                      <-- Domain Model
|   +-- Product.java
|   +-- Order.java
```

```
|   +-- Category.java
|
+-- repository/                  <-- Data Access Layer
|   +-- ProductRepository.java   (interface)
|   +-- JpaProductRepository.java (implementation)
|   +-- entity/
|       +-- ProductEntity.java   (JPA entity)
|
+-- config/                      <-- Configuration
    +-- AppConfig.java
```

Request Flow Through the Layers

```
HTTP Request: POST /api/products
{"name": "Widget", "price": 9.99, "category": "tools"}

     |
     v
+-------------------+
| ProductController |  1. Parse JSON into CreateProductRequest
|  (Presentation)   |  2. Call productService.create(...)
+--------+---------+
         |
         v
+-------------------+
| ProductService    |  3. Validate business rules
|  (Business)       |  4. Create Product domain object
|                   |  5. Call repository.save(product)
+--------+---------+
         |
         v
+-------------------+
| JpaProductRepo    |  6. Convert Product to ProductEntity
|  (Data Access)    |  7. Execute INSERT INTO products ...
|                   |  8. Return saved Product
+--------+---------+
         |
         v
+-------------------+
| ProductService    |  9. Return Product to controller
+--------+---------+
         |
         v
+-------------------+
| ProductController |  10. Convert Product to ProductResponse
|                   |  11. Return HTTP 201 Created
+-------------------+

     |
     v
HTTP Response: 201 Created
```

```
{"id": 42, "name": "Widget", "price": 9.99}
```

19.6 Benefits of Layered Architecture

```
+-------------------------------------------------------+
|                    BENEFITS                           |
+-------------------------------------------------------+
|                                                       |
|  1. SIMPLICITY                                        |
|     Easy to understand. Most developers already       |
|     know it. Low learning curve.                      |
|                                                       |
|  2. SEPARATION OF CONCERNS                            |
|     Each layer has a clear responsibility.             |
|     Changes in one layer rarely affect others.        |
|                                                       |
|  3. TESTABILITY                                       |
|     Business logic can be tested independently        |
|     by mocking the data access layer.                 |
|                                                       |
|  4. TEAM ORGANIZATION                                 |
|     Different developers can work on different        |
|     layers with minimal conflicts.                    |
|                                                       |
|  5. TECHNOLOGY FLEXIBILITY                            |
|     You can change the database without touching      |
|     the presentation layer.                           |
|                                                       |
+-------------------------------------------------------+
```

19.7 Drawbacks of Layered Architecture

```
+-------------------------------------------------------+
|                    DRAWBACKS                          |
+-------------------------------------------------------+
|                                                       |
|  1. LAYER LEAKING                                     |
|     Business logic creeps into controllers.           |
|     SQL logic creeps into services.                   |
|     Discipline is needed to keep layers clean.        |
```

```
  |                                                        |
  |  2. PASS-THROUGH LAYERS                                |
  |     Sometimes a controller just passes data to a       |
  |     service that passes it to a repository.            |
  |     The service layer adds no value.                   |
  |                                                        |
  |  3. DATABASE-DRIVEN DESIGN                             |
  |     The natural tendency is to design from the         |
  |     database up. This makes the domain model           |
  |     mirror the database schema instead of the          |
  |     business concepts.                                 |
  |                                                        |
  |  4. MONOLITHIC TENDENCY                                |
  |     Layers run in the same process. As the app         |
  |     grows, it becomes a large monolith.                |
  |                                                        |
  |  5. DEPENDENCY DIRECTION                               |
  |     Business logic depends on the data access          |
  |     layer, which violates the Dependency Rule          |
  |     from Clean Architecture (Chapter 18).              |
  |                                                        |
  +--------------------------------------------------------+
```

The Dependency Problem

In traditional layered architecture, the business layer depends on the data access layer:

```
Controller  -->  Service  -->  Repository (concrete class)
```

This means the business logic knows about the persistence technology. To fix this, use dependency inversion:

```
Controller  -->  Service  -->  Repository (interface)
                                     ^
                                     |
                             JpaRepository (implementation)
```

Now the service depends on an abstraction, not a concrete implementation. This is how layered architecture evolves toward clean architecture.

19.8 Layer Leaking: The Most Common Problem

Layer leaking happens when code that belongs in one layer ends up in another.

BEFORE: Business Logic in the Controller

```java
@PostMapping("/orders")
public ResponseEntity<String> createOrder(@RequestBody OrderRequest req) {
    // This business logic belongs in the SERVICE layer!
    double total = 0;
    for (Item item : req.getItems()) {
        Product p = productRepository.findById(item.getProductId());
        if (p.getStock() < item.getQuantity()) {
            return ResponseEntity.badRequest().body("Out of stock: " + p.getName());
        }
        total += p.getPrice() * item.getQuantity();
    }
    if (total > 1000) {
        total *= 0.95; // Discount
    }

    // This data access belongs in the REPOSITORY layer!
    jdbcTemplate.update("INSERT INTO orders (customer_id, total) VALUES (?, ?)",
            req.getCustomerId(), total);

    return ResponseEntity.ok("Order created");
}
```

AFTER: Proper Layer Separation

```java
// Controller: only HTTP concerns
@PostMapping("/orders")
public ResponseEntity<OrderResponse> createOrder(@RequestBody OrderRequest req) {
    Order order = orderService.create(req.getCustomerId(), req.getItems());
    return ResponseEntity.status(HttpStatus.CREATED)
            .body(OrderResponse.from(order));
}

// Service: business logic
public Order create(Long customerId, List<OrderItem> items) {
    for (OrderItem item : items) {
        Product product = productRepository.findById(item.getProductId())
                .orElseThrow(() -> new ProductNotFoundException(item.getProductId()));
        if (product.getStock() < item.getQuantity()) {
            throw new InsufficientStockException(product.getName());
        }
    }

    double total = calculateTotal(items);
    Order order = new Order(customerId, items, total);
```

```
    return orderRepository.save(order);
}

private double calculateTotal(List<OrderItem> items) {
    double total = items.stream()
            .mapToDouble(i -> i.getPrice() * i.getQuantity())
            .sum();
    if (total > 1000) {
        total *= 0.95;
    }
    return total;
}
```

Common Mistakes

1. Putting business logic in controllers. The controller's job is to handle HTTP, not to calculate discounts or validate business rules.
2. Calling the repository from the controller. Always go through the service layer, even if the service just passes the call through. This keeps your architecture consistent and makes it easy to add business logic later.
3. Using JPA entities as API responses. This couples your database schema to your API contract. Use DTOs (Data Transfer Objects) to decouple them.
4. Creating one service per controller. Services should be organized around business capabilities, not around UI needs. Multiple controllers can share the same service.
5. Skipping the repository interface. Always define a repository interface in the business layer. This lets you swap implementations and test with fakes.

Best Practices

1. Keep controllers thin. Controllers should parse input, call a service, and format output. Nothing more.
2. Keep services fat with business logic. All business rules, validations, and orchestration belong here.
3. Use DTOs at layer boundaries. Do not let JPA entities, HTTP request objects, or database result sets leak across layers.

4. Define repository interfaces in the business layer. Implement them in the data access layer. This follows the Dependency Inversion Principle.
5. Use exception handling across layers. Business exceptions thrown by services should be caught by controllers and translated into appropriate HTTP responses.

Quick Summary

Layer	Responsibility	Example Classes
Presentation	HTTP, input/output formatting	Controllers, DTOs, Views
Business Logic	Rules, validation, orchestration	Services, Domain objects
Data Access	Storage, retrieval, mapping	Repositories, DAOs, Entities

---

Key Points

- Layered architecture divides the application into Presentation, Business, and Data Access layers.
- Each layer has a single responsibility and communicates only with the layer directly below it.
- The pattern is simple, widely understood, and effective for most business applications.
- The main risk is layer leaking -- business logic in controllers or SQL in services.
- Using interfaces at layer boundaries enables dependency inversion and testability.

Practice Questions

1. A developer puts a `SELECT` query directly in a controller method. Which architectural principle does this violate? Where should the query go?
2. Your service method returns a `ProductEntity` (a JPA-annotated class) directly to the controller, which serializes it to JSON. What problems could this cause?

3. Is it acceptable for the Presentation layer to bypass the Business layer and call the Data Access layer directly? When, if ever, might this be justified?
4. Compare the dependency direction in traditional layered architecture versus Clean Architecture. What is the key difference?
5. Your team has a service class with 50 methods covering users, orders, products, and reports. Which code smell from Chapter 14 does this exhibit? How would you fix it?

Exercises

Exercise 1: Layer Identification

For each of these code snippets, identify which layer it belongs in (Presentation, Business, or Data Access) and explain why:

```java
// Snippet A
if (order.getTotal() > 500 && customer.getLoyaltyYears() >= 2) {
    order.applyDiscount(0.10);
}

// Snippet B
@GetMapping("/users/{id}")
public ResponseEntity<UserDto> getUser(@PathVariable Long id) { ... }

// Snippet C
String sql = "SELECT * FROM products WHERE category = ? AND price < ?";
```

Exercise 2: Build a Three-Layer Application

Build a simple "Task Manager" application with the following layers:

1. Presentation: REST endpoints for creating, listing, and completing tasks
2. Business: Validation (title not empty, due date in the future), status transitions (only "pending" tasks can be marked "complete")
3. Data Access: In-memory list storage (no real database needed)

Implement in either Java (Spring Boot) or Python (Flask). Verify that no layer violates its boundaries.

Exercise 3: Find the Layer Leaks

Review this code and identify all layer violations:

```python
@app.route("/api/orders", methods=["POST"])
def create_order():
    data = request.get_json()

    # Check stock (should be in service)
    conn = sqlite3.connect("shop.db")
    cursor = conn.execute(
        "SELECT stock FROM products WHERE id = ?", (data["product_id"],))
    stock = cursor.fetchone()[0]
    if stock < data["quantity"]:
        return jsonify({"error": "Out of stock"}), 400

    # Calculate total (should be in service)
    total = data["price"] * data["quantity"] * 1.08

    # Save order (should be in repository)
    conn.execute("INSERT INTO orders (product_id, total) VALUES (?, ?)",
                 (data["product_id"], total))
    conn.commit()

    return jsonify({"total": total}), 201
```

Refactor it into proper layers.

What Is Next?

Layered architecture is a solid foundation, but it has a significant limitation: the business layer depends on the data access layer. Chapter 20: Hexagonal Architecture solves this by inverting the dependency direction, placing the domain at the center and pushing all infrastructure to the edges. This makes the core business logic truly independent of databases, frameworks, and external services.

Chapter 20: Hexagonal Architecture -- Ports and Adapters

What You Will Learn

- What hexagonal architecture is and why it was created
- The concepts of ports and adapters and how they differ from layers
- The difference between driving (primary) and driven (secondary) adapters
- How the core domain sits at the center, completely isolated from infrastructure
- A complete before-and-after refactoring from layered to hexagonal
- The testing advantage that hexagonal architecture provides
- When to use hexagonal architecture and when layered is sufficient

Why This Chapter Matters

Layered architecture (Chapter 19) is a great starting point, but it has a structural flaw: the business logic depends on the data access layer. This means your domain code knows about databases, and swapping infrastructure requires changing the core. Hexagonal architecture, introduced by Alistair Cockburn, fixes this by putting the domain at the center and defining clear ports (interfaces) that the outside world plugs into with adapters. The result is a system where the core business logic has zero dependencies on any framework, database, or external service -- making it trivially easy to test, change, and evolve.

20.1 The Core Idea

Hexagonal architecture (also called Ports and Adapters) is built on one principle:

> The application talks to the outside world through ports. The outside world connects to the application through adapters.

The application does not know or care what is on the other side of a port. It could be a real database, an in-memory fake, a test mock, or a completely different technology.

```
+-------------------------------------------------------------+
|                                                             |
|  Traditional Layered:                                       |
|    UI  -->  Business Logic  -->  Database                   |
|    (Business depends on Database)                           |
|                                                             |
|  Hexagonal:                                                 |
|    UI Adapter  -->  [Port]  DOMAIN  [Port]  <--  DB Adapter |
|    (Domain depends on NOTHING external)                     |
|                                                             |
+-------------------------------------------------------------+
```

20.2 The Hexagon Diagram

The hexagonal shape is a visual metaphor -- the number of sides is not significant. What matters is that the domain is at the center and all external systems connect through the edges.

```
                        Driving Side
                    (Primary Adapters)

                  REST         CLI
                Controller    Command
                    \          /
                     \        /
         +-----------\------/------------+
        /             \    /              \
       /    +----------\--/----------+     \
      /     |    Driving Ports       |      \
     /      |   (Input Interfaces)   |       \
    |       +------------------------+        |
    |       |                        |        |
    |       |    DOMAIN CORE         |        |
    |       |                        |        |
    |       |  Entities              |        |
    |       |  Use Cases             |        |
    |       |  Business Rules        |        |
    |       |                        |        |
    |       +------------------------+        |
```

```
      \      |   Driven Ports         |       /
       \     |   (Output Interfaces)  |      /
        \    +----------+--+----------+     /
         \             /    \              /
          +-----------/------\------------+
                     /        \
                    /          \
             Database       Email
             Adapter       Adapter

                   Driven Side
               (Secondary Adapters)
```

20.3 Ports: What the Domain Needs and Offers

A port is an interface defined by the domain. Ports come in two varieties:

Driving Ports (Primary / Input)

Driving ports define how the outside world can use the application. They are the entry points -- the API of your domain.

```java
// Driving port: defines what the application can DO
public interface OrderManagement {
    Order placeOrder(String customerId, List<OrderItem> items);
    Order getOrder(String orderId);
    void cancelOrder(String orderId);
}
```

```python
# Driving port: defines what the application can DO
from abc import ABC, abstractmethod

class OrderManagement(ABC):
    @abstractmethod
    def place_order(self, customer_id, items): ...

    @abstractmethod
    def get_order(self, order_id): ...

    @abstractmethod
    def cancel_order(self, order_id): ...
```

Driven Ports (Secondary / Output)

Driven ports define what the application needs from the outside world. They are the requirements -- the dependencies the domain declares without knowing who satisfies them.

```java
// Driven port: defines what the application NEEDS
public interface OrderRepository {
    Order save(Order order);
    Optional<Order> findById(String id);
    void deleteById(String id);
}

public interface PaymentGateway {
    PaymentResult charge(String customerId, Money amount);
    void refund(String transactionId);
}

public interface NotificationSender {
    void sendOrderConfirmation(String customerId, Order order);
}
```

```python
# Driven ports: define what the application NEEDS
class OrderRepository(ABC):
    @abstractmethod
    def save(self, order): ...

    @abstractmethod
    def find_by_id(self, order_id): ...

    @abstractmethod
    def delete_by_id(self, order_id): ...


class PaymentGateway(ABC):
    @abstractmethod
    def charge(self, customer_id, amount): ...

    @abstractmethod
    def refund(self, transaction_id): ...


class NotificationSender(ABC):
    @abstractmethod
    def send_order_confirmation(self, customer_id, order): ...
```

Port Summary

```
+------------------------------------------------------------+
|                                                            |
|  DRIVING PORTS              DRIVEN PORTS                   |
|  (Input / Primary)          (Output / Secondary)           |
|                                                            |
|  "What I offer"             "What I need"                  |
|  Defined by domain          Defined by domain              |
|  Implemented by domain      Implemented by adapters        |
|  Called by adapters          Called by domain               |
|                                                            |
|  Examples:                  Examples:                      |
|  - OrderManagement          - OrderRepository              |
|  - UserRegistration         - PaymentGateway               |
|  - ReportGeneration         - NotificationSender           |
|                                                            |
+------------------------------------------------------------+
```

20.4 Adapters: Connecting the Outside World

An adapter is a concrete implementation that connects an external system to a port.

Driving Adapters (Primary)

Driving adapters call INTO the domain through driving ports. They adapt external input (HTTP requests, CLI commands, message queue events) into domain operations.

REST Controller Adapter (Java):

```java
@RestController
@RequestMapping("/api/orders")
public class OrderRestAdapter {

    private final OrderManagement orderManagement;  // Driving port

    public OrderRestAdapter(OrderManagement orderManagement) {
        this.orderManagement = orderManagement;
    }

    @PostMapping
    public ResponseEntity<OrderDto> placeOrder(@RequestBody PlaceOrderRequest req) {
        List<OrderItem> items = req.getItems().stream()
                .map(i -> new OrderItem(i.getProductId(), i.getQuantity()))
                .collect(Collectors.toList());
```

```
        Order order = orderManagement.placeOrder(req.getCustomerId(), items);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(OrderDto.from(order));
    }

    @GetMapping("/{id}")
    public ResponseEntity<OrderDto> getOrder(@PathVariable String id) {
        Order order = orderManagement.getOrder(id);
        return ResponseEntity.ok(OrderDto.from(order));
    }
}
```

CLI Adapter (Python):

```python
import click

class OrderCliAdapter:
    def __init__(self, order_management: OrderManagement):
        self.order_management = order_management

    def place_order(self, customer_id, product_id, quantity):
        items = [OrderItem(product_id, quantity)]
        order = self.order_management.place_order(customer_id, items)
        print(f"Order placed: {order.id} - Total: ${order.total:.2f}")

    def get_order(self, order_id):
        order = self.order_management.get_order(order_id)
        print(f"Order {order.id}: {order.status} - ${order.total:.2f}")
```

Driven Adapters (Secondary)

Driven adapters are called BY the domain through driven ports. They adapt domain requests into external operations (database queries, API calls, email sending).

Database Adapter (Java):

```java
@Repository
public class PostgresOrderRepository implements OrderRepository {

    private final JdbcTemplate jdbcTemplate;

    public PostgresOrderRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Override
    public Order save(Order order) {
```

```
        jdbcTemplate.update(
            "INSERT INTO orders (id, customer_id, total, status) VALUES (?, ?, ?, ?)",
            order.getId(), order.getCustomerId(),
            order.getTotal(), order.getStatus().name()
        );
        return order;
    }

    @Override
    public Optional<Order> findById(String id) {
        List<Order> results = jdbcTemplate.query(
            "SELECT * FROM orders WHERE id = ?",
            new Object[]{id},
            (rs, rowNum) -> new Order(
                rs.getString("id"),
                rs.getString("customer_id"),
                rs.getDouble("total"),
                OrderStatus.valueOf(rs.getString("status"))
            )
        );
        return results.stream().findFirst();
    }

    @Override
    public void deleteById(String id) {
        jdbcTemplate.update("DELETE FROM orders WHERE id = ?", id);
    }
}
```

Email Adapter (Python):

```python
import smtplib
from email.message import EmailMessage


class SmtpNotificationAdapter(NotificationSender):

    def __init__(self, smtp_host, smtp_port, sender_email):
        self.smtp_host = smtp_host
        self.smtp_port = smtp_port
        self.sender_email = sender_email

    def send_order_confirmation(self, customer_id, order):
        msg = EmailMessage()
        msg["Subject"] = f"Order Confirmation: {order.id}"
        msg["From"] = self.sender_email
        msg["To"] = self._lookup_email(customer_id)
        msg.set_content(
            f"Your order {order.id} has been placed.\n"
            f"Total: ${order.total:.2f}"
        )
        with smtplib.SMTP(self.smtp_host, self.smtp_port) as server:
            server.send_message(msg)
```

20.5 The Domain Core

The domain core implements the driving ports and uses the driven ports. It contains entities, use cases, and business rules. It has NO dependency on any framework, database, or external system.

Domain Core (Java)

```java
// Entity: pure business rules
public class Order {
    private final String id;
    private final String customerId;
    private final List<OrderItem> items;
    private OrderStatus status;
    private double total;

    public Order(String customerId, List<OrderItem> items) {
        this.id = UUID.randomUUID().toString();
        this.customerId = customerId;
        this.items = new ArrayList<>(items);
        this.status = OrderStatus.PENDING;
        this.total = calculateTotal();
    }

    private double calculateTotal() {
        return items.stream()
                .mapToDouble(item -> item.getPrice() * item.getQuantity())
                .sum();
    }

    public void cancel() {
        if (status == OrderStatus.SHIPPED) {
            throw new BusinessException("Cannot cancel a shipped order");
        }
        this.status = OrderStatus.CANCELLED;
    }

    // Getters...
}

// Use case: implements the driving port
public class OrderService implements OrderManagement {

    private final OrderRepository orderRepo;
    private final PaymentGateway paymentGateway;
```

```
    private final NotificationSender notifier;

    public OrderService(OrderRepository orderRepo,
                        PaymentGateway paymentGateway,
                        NotificationSender notifier) {
        this.orderRepo = orderRepo;
        this.paymentGateway = paymentGateway;
        this.notifier = notifier;
    }

    @Override
    public Order placeOrder(String customerId, List<OrderItem> items) {
        if (items == null || items.isEmpty()) {
            throw new BusinessException("Order must have at least one item");
        }

        Order order = new Order(customerId, items);
        Money amount = Money.of(order.getTotal(), Currency.USD);

        PaymentResult payment = paymentGateway.charge(customerId, amount);
        if (!payment.isSuccessful()) {
            throw new PaymentFailedException("Payment declined");
        }

        Order saved = orderRepo.save(order);
        notifier.sendOrderConfirmation(customerId, saved);
        return saved;
    }

    @Override
    public Order getOrder(String orderId) {
        return orderRepo.findById(orderId)
                .orElseThrow(() -> new OrderNotFoundException(orderId));
    }

    @Override
    public void cancelOrder(String orderId) {
        Order order = getOrder(orderId);
        order.cancel();
        orderRepo.save(order);
        paymentGateway.refund(order.getPaymentTransactionId());
    }
}
```

Domain Core (Python)

```python
import uuid
from enum import Enum
```

```
class OrderStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    SHIPPED = "shipped"
    CANCELLED = "cancelled"


class Order:
    def __init__(self, customer_id, items):
        self.id = str(uuid.uuid4())
        self.customer_id = customer_id
        self.items = list(items)
        self.status = OrderStatus.PENDING
        self.total = self._calculate_total()

    def _calculate_total(self):
        return sum(item.price * item.quantity for item in self.items)

    def cancel(self):
        if self.status == OrderStatus.SHIPPED:
            raise BusinessError("Cannot cancel a shipped order")
        self.status = OrderStatus.CANCELLED


class OrderService(OrderManagement):

    def __init__(self, order_repo: OrderRepository,
                 payment_gateway: PaymentGateway,
                 notifier: NotificationSender):
        self.order_repo = order_repo
        self.payment_gateway = payment_gateway
        self.notifier = notifier

    def place_order(self, customer_id, items):
        if not items:
            raise BusinessError("Order must have at least one item")

        order = Order(customer_id, items)

        payment = self.payment_gateway.charge(customer_id, order.total)
        if not payment.is_successful:
            raise PaymentFailedError("Payment declined")

        saved = self.order_repo.save(order)
        self.notifier.send_order_confirmation(customer_id, saved)
        return saved

    def get_order(self, order_id):
        order = self.order_repo.find_by_id(order_id)
        if not order:
            raise OrderNotFoundError(order_id)
        return order

    def cancel_order(self, order_id):
        order = self.get_order(order_id)
```

```
        order.cancel()
        self.order_repo.save(order)
        self.payment_gateway.refund(order.payment_transaction_id)
```

Notice: the domain core imports NO framework classes, NO database libraries, NO HTTP utilities. It is pure business logic.

20.6 Before and After: Refactoring to Hexagonal

BEFORE: Tightly Coupled (Java)

```java
@Service
public class OrderService {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Autowired
    private RestTemplate restTemplate;

    public Map<String, Object> createOrder(Map<String, Object> requestData) {
        String customerId = (String) requestData.get("customerId");
        double amount = (double) requestData.get("amount");

        // Direct database call
        jdbcTemplate.update(
            "INSERT INTO orders (customer_id, amount, status) VALUES (?, ?, 'PENDING')",
            customerId, amount);

        // Direct HTTP call to payment service
        ResponseEntity<String> paymentResponse = restTemplate.postForEntity(
            "https://payments.example.com/charge",
            Map.of("customerId", customerId, "amount", amount),
            String.class);

        if (!paymentResponse.getStatusCode().is2xxSuccessful()) {
            throw new RuntimeException("Payment failed");
        }

        // Direct email via SMTP
        SimpleMailMessage message = new SimpleMailMessage();
        message.setTo(getCustomerEmail(customerId));
        message.setSubject("Order Confirmation");
        message.setText("Your order for $" + amount + " has been placed.");
        mailSender.send(message);
```

```
        return Map.of("status", "created", "amount", amount);
    }
}
```

Problems with this code:

- Domain logic is tangled with JDBC, HTTP, and SMTP
- Impossible to test without a real database, payment API, and mail server
- Changing the database requires changing the business logic
- Changing the payment provider requires changing the business logic

AFTER: Hexagonal Architecture (Java)

```java
// DRIVEN PORT (defined in domain)
public interface OrderRepository {
    Order save(Order order);
}

public interface PaymentGateway {
    PaymentResult charge(String customerId, double amount);
}

public interface NotificationSender {
    void sendConfirmation(String customerId, double amount);
}

// DOMAIN CORE (no external dependencies)
public class OrderService implements OrderManagement {

    private final OrderRepository orderRepo;
    private final PaymentGateway paymentGateway;
    private final NotificationSender notifier;

    public OrderService(OrderRepository orderRepo,
                        PaymentGateway paymentGateway,
                        NotificationSender notifier) {
        this.orderRepo = orderRepo;
        this.paymentGateway = paymentGateway;
        this.notifier = notifier;
    }

    @Override
    public Order placeOrder(String customerId, double amount) {
        PaymentResult payment = paymentGateway.charge(customerId, amount);
        if (!payment.isSuccessful()) {
            throw new PaymentFailedException("Payment declined");
        }
```

```
        Order order = new Order(customerId, amount);
        Order saved = orderRepo.save(order);
        notifier.sendConfirmation(customerId, amount);
        return saved;
    }
}

// DRIVEN ADAPTERS (implement ports)
public class JdbcOrderAdapter implements OrderRepository {
    private final JdbcTemplate jdbcTemplate;
    public Order save(Order order) {
        jdbcTemplate.update("INSERT INTO orders ...", ...);
        return order;
    }
}

public class HttpPaymentAdapter implements PaymentGateway {
    private final RestTemplate restTemplate;
    public PaymentResult charge(String customerId, double amount) {
        // HTTP call to payment API
        ...
    }
}

public class SmtpNotificationAdapter implements NotificationSender {
    private final JavaMailSender mailSender;
    public void sendConfirmation(String customerId, double amount) {
        // SMTP email sending
        ...
    }
}
```

20.7 The Testing Advantage

The greatest practical benefit of hexagonal architecture is testability. Because the domain has no external dependencies, you can test it with simple in-memory implementations.

Testing the Domain (Java)

```java
class OrderServiceTest {

    private InMemoryOrderRepository orderRepo;
    private FakePaymentGateway paymentGateway;
    private SpyNotificationSender notifier;
```

```
    private OrderService orderService;

    @BeforeEach
    void setUp() {
        orderRepo = new InMemoryOrderRepository();
        paymentGateway = new FakePaymentGateway();
        notifier = new SpyNotificationSender();
        orderService = new OrderService(orderRepo, paymentGateway, notifier);
    }

    @Test
    void placeOrder_successfulPayment_savesOrder() {
        paymentGateway.willSucceed();

        Order order = orderService.placeOrder("CUST-1", 99.99);

        assertNotNull(order.getId());
        assertEquals(99.99, order.getTotal(), 0.01);
        assertTrue(orderRepo.contains(order.getId()));
    }

    @Test
    void placeOrder_successfulPayment_sendsNotification() {
        paymentGateway.willSucceed();

        orderService.placeOrder("CUST-1", 99.99);

        assertTrue(notifier.wasCalled());
        assertEquals("CUST-1", notifier.getLastCustomerId());
    }

    @Test
    void placeOrder_failedPayment_throwsException() {
        paymentGateway.willFail();

        assertThrows(PaymentFailedException.class,
            () -> orderService.placeOrder("CUST-1", 99.99));

        assertTrue(orderRepo.isEmpty());
    }

    @Test
    void placeOrder_failedPayment_doesNotSendNotification() {
        paymentGateway.willFail();

        try {
            orderService.placeOrder("CUST-1", 99.99);
        } catch (PaymentFailedException ignored) {}

        assertFalse(notifier.wasCalled());
    }
}

// Test doubles -- simple in-memory implementations
class InMemoryOrderRepository implements OrderRepository {
```

```
    private final Map<String, Order> store = new HashMap<>();

    public Order save(Order order) {
        store.put(order.getId(), order);
        return order;
    }
    public boolean contains(String id) { return store.containsKey(id); }
    public boolean isEmpty() { return store.isEmpty(); }
}

class FakePaymentGateway implements PaymentGateway {
    private boolean shouldSucceed = true;

    public void willSucceed() { shouldSucceed = true; }
    public void willFail() { shouldSucceed = false; }

    public PaymentResult charge(String customerId, double amount) {
        return new PaymentResult(shouldSucceed);
    }
}

class SpyNotificationSender implements NotificationSender {
    private boolean called = false;
    private String lastCustomerId;

    public void sendConfirmation(String customerId, double amount) {
        called = true;
        lastCustomerId = customerId;
    }
    public boolean wasCalled() { return called; }
    public String getLastCustomerId() { return lastCustomerId; }
}
```

Testing the Domain (Python)

```python
import pytest


class InMemoryOrderRepository(OrderRepository):
    def __init__(self):
        self.store = {}

    def save(self, order):
        self.store[order.id] = order
        return order

    def find_by_id(self, order_id):
        return self.store.get(order_id)
```

```
class FakePaymentGateway(PaymentGateway):
    def __init__(self, should_succeed=True):
        self.should_succeed = should_succeed

    def charge(self, customer_id, amount):
        return PaymentResult(successful=self.should_succeed)

    def refund(self, transaction_id):
        pass


class SpyNotificationSender(NotificationSender):
    def __init__(self):
        self.calls = []

    def send_order_confirmation(self, customer_id, order):
        self.calls.append((customer_id, order))

    @property
    def was_called(self):
        return len(self.calls) > 0


@pytest.fixture
def order_repo():
    return InMemoryOrderRepository()

@pytest.fixture
def payment_gateway():
    return FakePaymentGateway(should_succeed=True)

@pytest.fixture
def notifier():
    return SpyNotificationSender()

@pytest.fixture
def order_service(order_repo, payment_gateway, notifier):
    return OrderService(order_repo, payment_gateway, notifier)


def test_place_order_saves_order(order_service, order_repo):
    items = [OrderItem("PROD-1", 2, 49.99)]
    order = order_service.place_order("CUST-1", items)

    assert order.id in order_repo.store
    assert order.total == pytest.approx(99.98)


def test_place_order_sends_notification(order_service, notifier):
    items = [OrderItem("PROD-1", 1, 29.99)]
    order_service.place_order("CUST-1", items)

    assert notifier.was_called
    assert notifier.calls[0][0] == "CUST-1"
```

```
def test_failed_payment_raises_error(order_repo, notifier):
    failing_gateway = FakePaymentGateway(should_succeed=False)
    service = OrderService(order_repo, failing_gateway, notifier)
    items = [OrderItem("PROD-1", 1, 29.99)]

    with pytest.raises(PaymentFailedError):
        service.place_order("CUST-1", items)

    assert len(order_repo.store) == 0
    assert not notifier.was_called
```

Testing Comparison

```
  +-------------------------------------------------------+
  |                                                       |
  |  LAYERED (tightly coupled)                            |
  |  - Need a running database to test business logic     |
  |  - Need a running payment API to test order flow      |
  |  - Tests take seconds to minutes                      |
  |  - Tests are fragile (external services can be down)  |
  |                                                       |
  |  HEXAGONAL (loosely coupled)                          |
  |  - Test business logic with in-memory fakes           |
  |  - No external services needed                        |
  |  - Tests take milliseconds                            |
  |  - Tests are 100% reliable and repeatable             |
  |                                                       |
  +-------------------------------------------------------+
```

20.8 Project Structure

Here is how a hexagonal architecture project is typically organized:

```
  src/main/java/com/example/orders/
  +-- domain/                        <-- CORE (no external deps)
  |   +-- model/
  |   |   +-- Order.java
  |   |   +-- OrderItem.java
  |   |   +-- OrderStatus.java
  |   +-- port/
  |   |   +-- in/                    <-- Driving ports
  |   |   |   +-- OrderManagement.java
  |   |   +-- out/                   <-- Driven ports
```

```
|   |       +-- OrderRepository.java
|   |       +-- PaymentGateway.java
|   |       +-- NotificationSender.java
|   +-- service/
|       +-- OrderService.java      <-- Implements driving port
|
+-- adapter/                       <-- ADAPTERS (external deps)
    +-- in/                        <-- Driving adapters
    |   +-- rest/
    |   |   +-- OrderRestAdapter.java
    |   |   +-- dto/
    |   |       +-- PlaceOrderRequest.java
    |   |       +-- OrderDto.java
    |   +-- cli/
    |       +-- OrderCliAdapter.java
    +-- out/                       <-- Driven adapters
        +-- persistence/
        |   +-- JdbcOrderAdapter.java
        |   +-- OrderEntity.java
        +-- payment/
        |   +-- HttpPaymentAdapter.java
        +-- notification/
            +-- SmtpNotificationAdapter.java
```

Python Project Structure

```
orders/
+-- domain/                        # CORE
|   +-- __init__.py
|   +-- model.py                   # Order, OrderItem, OrderStatus
|   +-- ports.py                   # All port interfaces
|   +-- service.py                 # OrderService
|   +-- exceptions.py              # BusinessError, etc.
|
+-- adapters/                      # ADAPTERS
|   +-- __init__.py
|   +-- api/
|   |   +-- rest_adapter.py        # Flask/FastAPI routes
|   |   +-- dto.py                 # Request/response models
|   +-- persistence/
|   |   +-- postgres_adapter.py    # PostgresOrderRepository
|   |   +-- in_memory_adapter.py   # InMemoryOrderRepository
|   +-- payment/
|   |   +-- stripe_adapter.py      # StripePaymentGateway
|   +-- notification/
|       +-- email_adapter.py       # SmtpNotificationSender
|
+-- config.py                      # Wiring / dependency injection
+-- main.py                        # Application entry point
```

20.9 Driving vs Driven: The Complete Picture

```
  +-------------------------------------------------------------------+
  |                                                                    |
  |  DRIVING (Primary)                  DRIVEN (Secondary)            |
  |  "Things that USE the app"          "Things the app USES"         |
  |                                                                    |
  |  - REST API controller              - Database repository         |
  |  - CLI command handler              - Payment API client          |
  |  - Message queue consumer           - Email/SMS sender            |
  |  - Scheduled job trigger            - File storage                |
  |  - gRPC server                      - Message queue producer      |
  |  - Test harness                     - External API client         |
  |                                                                    |
  |  Direction: Outside --> IN           Direction: IN --> Outside     |
  |  Calls domain methods               Called BY domain methods      |
  |                                                                    |
  +-------------------------------------------------------------------+
```

Both Types of Adapters for the Same Port

A driven port can have multiple adapters, and you choose which one to use at runtime:

```java
// Production configuration
@Configuration
public class ProductionConfig {
    @Bean
    public OrderRepository orderRepository(JdbcTemplate jdbc) {
        return new PostgresOrderAdapter(jdbc);
    }

    @Bean
    public PaymentGateway paymentGateway() {
        return new StripePaymentAdapter("sk_live_xxx");
    }
}

// Test configuration
public class TestConfig {
    public OrderRepository orderRepository() {
        return new InMemoryOrderRepository();
    }

    public PaymentGateway paymentGateway() {
        return new FakePaymentGateway();
```

```
    }
}
```

```python
# Production
def create_production_app():
    order_repo = PostgresOrderRepository(connection_string)
    payment = StripePaymentGateway(api_key)
    notifier = SmtpNotificationSender(smtp_config)
    service = OrderService(order_repo, payment, notifier)
    return create_flask_app(service)

# Testing
def create_test_service():
    order_repo = InMemoryOrderRepository()
    payment = FakePaymentGateway(should_succeed=True)
    notifier = SpyNotificationSender()
    return OrderService(order_repo, payment, notifier)
```

20.10 When to Use Hexagonal Architecture

```
  +-------------------------------------------------------+
  |  USE HEXAGONAL WHEN:                                  |
  +-------------------------------------------------------+
  |                                                       |
  |  - Business logic is complex and needs isolation      |
  |  - You have multiple input channels (REST + CLI + MQ) |
  |  - You want to swap infrastructure without pain       |
  |  - Testability of business logic is a priority        |
  |  - The project will live for years                    |
  |                                                       |
  +-------------------------------------------------------+
  |  USE SIMPLE LAYERED WHEN:                             |
  +-------------------------------------------------------+
  |                                                       |
  |  - The project is small with little business logic    |
  |  - There is only one input channel                    |
  |  - The team is new to architecture patterns           |
  |  - The project is a prototype or proof of concept     |
  |  - Time-to-market is more important than flexibility  |
  |                                                       |
  +-------------------------------------------------------+
```

Common Mistakes

1. Putting framework annotations in the domain. The domain core must have zero framework dependencies. No `@Service`, no `@Autowired`, no `@Entity` from Spring or Django.
2. Leaking adapter types through ports. A port should not expose `ResultSet`, `HttpResponse`, or any adapter-specific type. Use domain types only.
3. Creating too many ports. Not every method call needs a port. Ports are for boundaries between the domain and the outside world.
4. Skipping ports for "simple" dependencies. Even if your repository is simple today, define a port. It costs almost nothing and pays off immediately in testability.
5. Over-engineering small applications. A 200-line CRUD app does not need hexagonal architecture. Start with layers and evolve when complexity demands it.

Best Practices

1. Keep the domain dependency-free. The domain module should have zero external imports. If you see `import org.springframework` or `import flask` in the domain, something is wrong.
2. Name ports by domain concept, not technology. Use `OrderRepository`, not `OrderDatabaseAccess`. Use `NotificationSender`, not `SmtpEmailer`.
3. Put ports in the domain package. Ports are defined by the domain, not by the adapters.
4. Start with one adapter per port. You can always add more later. The port interface ensures you can.
5. Test the domain first and most thoroughly. If your domain is well-tested, the adapters are thin enough to verify with integration tests.

Quick Summary

Concept	Definition
Port	Interface defined by the domain for communication with the outside

Driving Port	How the outside world uses the domain (input)
Driven Port	What the domain needs from the outside world (output)
Driving Adapter	Converts external input into domain calls (controller, CLI)
Driven Adapter	Implements domain needs with real infrastructure (database, API)
Domain Core	Business logic with zero external dependencies

Key Points

- Hexagonal architecture puts the domain at the center with no outward dependencies.
- Ports are interfaces defined by the domain. Adapters implement those interfaces using real infrastructure.
- Driving adapters call into the domain. Driven adapters are called by the domain.
- The testing advantage is the primary practical benefit: domain logic can be tested in milliseconds with simple in-memory fakes.
- Hexagonal architecture naturally enforces the Dependency Rule from Clean Architecture.

Practice Questions

1. What is the fundamental difference between a port and an adapter? Which one lives in the domain and which one lives outside?
2. A `PaymentService` in the domain directly imports `com.stripe.Stripe`. Does this violate hexagonal architecture? How would you fix it?
3. You need to support both REST and GraphQL for your application. How does hexagonal architecture make this easier compared to layered architecture?
4. Your team has a `UserRepository` interface and a `PostgresUserRepository` implementation. A new requirement says you also need to cache users in Redis. How would you add this in a hexagonal architecture?

5. Explain why hexagonal architecture makes it easier to follow Test-Driven Development (Chapter 17). How do ports help with the RED phase?

Exercises

Exercise 1: Identify Ports and Adapters

Given this requirement: "A library system where users can borrow and return books. Overdue books incur a fine. Users receive email notifications for due dates."

Identify:

- All driving ports (what the system offers)
- All driven ports (what the system needs)
- At least two driving adapters and three driven adapters

Draw the hexagon diagram with your identified components.

Exercise 2: Refactor to Hexagonal

Take this tightly coupled code and refactor it to hexagonal architecture:

```python
import sqlite3
import requests

class BookingService:
    def __init__(self):
        self.db = sqlite3.connect("bookings.db")

    def create_booking(self, user_id, room_id, date):
        # Check availability directly in database
        cursor = self.db.execute(
            "SELECT count(*) FROM bookings WHERE room_id=? AND date=?",
            (room_id, date))
        if cursor.fetchone()[0] > 0:
            raise ValueError("Room not available")

        # Save booking directly
        self.db.execute(
            "INSERT INTO bookings (user_id, room_id, date) VALUES (?, ?, ?)",
            (user_id, room_id, date))
        self.db.commit()

        # Send notification via external API
```

```
        requests.post("https://api.notify.com/send", json={
            "user_id": user_id,
            "message": f"Booking confirmed for room {room_id} on {date}"
        })
```

Create: domain model, driven ports, domain service, and adapter stubs.

Exercise 3: Test the Domain

Using the code you created in Exercise 2, write a complete test suite for the domain service using in-memory fakes. Cover:

- Successful booking
- Room not available
- Notification sent on success
- Notification NOT sent on failure

What Is Next?

Hexagonal architecture gives your application a clean separation between domain logic and infrastructure. The next chapters explore additional architectural patterns that build on these same principles: Onion Architecture (Chapter 21) adds more granularity to the domain layers, and Domain-Driven Design (Chapter 22) provides a rich set of patterns for modeling complex business domains within these architectural boundaries.

Chapter 21: Onion Architecture

What You Will Learn

- What Onion Architecture is and how it organizes code into concentric layers
- The strict dependency rule: dependencies always point inward
- How Domain Model, Domain Services, Application Services, and Infrastructure relate
- Differences between Onion, Hexagonal, and Clean Architecture
- How to structure a real project using Onion Architecture
- When Onion Architecture is the right choice for your project

Why This Chapter Matters

In the previous chapters, you learned about layered architecture and hexagonal architecture. Both solve real problems, but they leave room for ambiguity about how layers should depend on each other. Onion Architecture removes that ambiguity with one unbreakable rule: dependencies always point inward, toward the domain.

This matters because the domain -- your business logic -- is the most valuable and stable part of your application. Databases change. Frameworks change. APIs change. But the core rules of your business change far less often. Onion Architecture protects that core by ensuring nothing in the outside world leaks into it.

If you have ever worked on a project where changing the database forced you to rewrite business logic, or where unit testing required spinning up a web server, you have felt the pain that Onion Architecture prevents.

The Core Idea: Layers Like an Onion

Onion Architecture arranges your code in concentric rings. The innermost ring is the most important and the most protected. Each outer ring depends on the rings inside it, but never the other way around.

```
+-------------------------------------------------------------+
|                     INFRASTRUCTURE                          |
|   (Database, File System, External APIs, UI, Web)           |
|                                                             |
|   +-----------------------------------------------------+   |
|   |              APPLICATION SERVICES                   |   |
|   |   (Use Cases, Orchestration, DTOs, Commands)        |   |
|   |                                                     |   |
|   |   +-------------------------------------------+     |   |
|   |   |           DOMAIN SERVICES                 |     |   |
|   |   |   (Business Rules Spanning Entities)      |     |   |
|   |   |                                           |     |   |
|   |   |   +-----------------------------------+   |     |   |
|   |   |   |         DOMAIN MODEL              |   |     |   |
|   |   |   |   (Entities, Value Objects,       |   |     |   |
|   |   |   |    Repository Interfaces)         |   |     |   |
|   |   |   +-----------------------------------+   |     |   |
|   |   |                                           |     |   |
|   |   +-------------------------------------------+     |   |
|   |                                                     |   |
|   +-----------------------------------------------------+   |
|                                                             |
+-------------------------------------------------------------+

       Dependency Direction: ALWAYS inward -->
```

The Four Layers

Layer 1: Domain Model (Innermost) This is the heart of your application. It contains entities, value objects, and repository interfaces. It has zero dependencies on anything outside itself -- no frameworks, no libraries, no database code.

Layer 2: Domain Services Business logic that does not naturally belong to a single entity lives here. A pricing calculator that needs data from multiple entities, for example. Domain Services depend only on the Domain Model.

Layer 3: Application Services These orchestrate use cases. They receive a command ("place an order"), coordinate domain objects, and return results. They depend on Domain Services and the Domain Model, but not on Infrastructure.

Layer 4: Infrastructure (Outermost) Everything external lives here: database access, file I/O, web frameworks, email services, third-party APIs. Infrastructure implements the interfaces defined in the Domain Model.

The Dependency Rule

This is the single most important rule in Onion Architecture:

> Source code dependencies must point inward. Nothing in an inner layer can know about anything in an outer layer.

This means:

- The Domain Model does not import database libraries
- Domain Services do not reference HTTP controllers
- Application Services do not instantiate database connections
- Only Infrastructure knows about concrete implementations

```
WRONG:
  Domain Model --> imports SQLAlchemy (infrastructure concern)

RIGHT:
  Domain Model --> defines OrderRepository interface
  Infrastructure --> implements OrderRepository using SQLAlchemy
```

Before and After: Seeing the Difference

BEFORE: Everything Depends on the Database

```java
// BEFORE: Domain logic tangled with infrastructure
// Java - Order.java
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class Order {
    private Connection dbConnection;

    public Order(Connection dbConnection) {
        this.dbConnection = dbConnection;
    }
```

```
    public double calculateTotal() {
        double total = 0;
        try {
            PreparedStatement stmt = dbConnection.prepareStatement(
                "SELECT price, quantity FROM order_items WHERE order_id = ?"
            );
            stmt.setLong(1, this.id);
            ResultSet rs = stmt.executeQuery();
            while (rs.next()) {
                total += rs.getDouble("price") * rs.getInt("quantity");
            }
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
        return total;
    }

    public void applyDiscount(double percentage) {
        // Business rule buried inside database code
        if (percentage > 50) {
            throw new IllegalArgumentException("Discount cannot exceed 50%");
        }
        try {
            PreparedStatement stmt = dbConnection.prepareStatement(
                "UPDATE orders SET discount = ? WHERE id = ?"
            );
            stmt.setDouble(1, percentage);
            stmt.setLong(2, this.id);
            stmt.executeUpdate();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

Problems with this approach:

- Cannot test `calculateTotal()` without a real database
- Business rule (max 50% discount) is buried in database code
- Changing databases means rewriting business logic
- The `Order` class has two reasons to change: business rules AND data access

AFTER: Onion Architecture Separates Concerns

```java
// AFTER: Clean separation with Onion Architecture

// === LAYER 1: DOMAIN MODEL ===
```

```
// domain/model/OrderItem.java
public class OrderItem {
    private final String productId;
    private final double price;
    private final int quantity;

    public OrderItem(String productId, double price, int quantity) {
        if (price < 0) throw new IllegalArgumentException("Price cannot be negative");
        if (quantity < 1) throw new IllegalArgumentException("Quantity must be at least...
        this.productId = productId;
        this.price = price;
        this.quantity = quantity;
    }

    public double subtotal() {
        return price * quantity;
    }
}

// domain/model/Order.java
public class Order {
    private final String orderId;
    private final List<OrderItem> items;
    private double discountPercentage;

    public Order(String orderId, List<OrderItem> items) {
        this.orderId = orderId;
        this.items = new ArrayList<>(items);
        this.discountPercentage = 0;
    }

    public double calculateTotal() {
        double subtotal = items.stream()
            .mapToDouble(OrderItem::subtotal)
            .sum();
        return subtotal * (1 - discountPercentage / 100);
    }

    public void applyDiscount(double percentage) {
        if (percentage > 50) {
            throw new IllegalArgumentException("Discount cannot exceed 50%");
        }
        this.discountPercentage = percentage;
    }
}

// domain/model/OrderRepository.java (INTERFACE in the domain)
public interface OrderRepository {
    Order findById(String orderId);
    void save(Order order);
    List<Order> findByCustomerId(String customerId);
}
```

```java
// === LAYER 2: DOMAIN SERVICES ===

// domain/service/PricingService.java
public class PricingService {

    public double calculateLoyaltyDiscount(Order order, int customerOrderCount) {
        // Business rule: loyal customers get bigger discounts
        if (customerOrderCount > 100) return 15.0;
        if (customerOrderCount > 50) return 10.0;
        if (customerOrderCount > 10) return 5.0;
        return 0.0;
    }
}
```

```java
// === LAYER 3: APPLICATION SERVICES ===

// application/PlaceOrderUseCase.java
public class PlaceOrderUseCase {
    private final OrderRepository orderRepository;
    private final PricingService pricingService;

    // Dependencies injected -- no concrete classes, only interfaces/domain types
    public PlaceOrderUseCase(OrderRepository orderRepository,
                             PricingService pricingService) {
        this.orderRepository = orderRepository;
        this.pricingService = pricingService;
    }

    public OrderResult execute(PlaceOrderCommand command) {
        List<OrderItem> items = command.getItems().stream()
            .map(i -> new OrderItem(i.getProductId(), i.getPrice(), i.getQuantity()))
            .collect(Collectors.toList());

        Order order = new Order(generateId(), items);

        double discount = pricingService.calculateLoyaltyDiscount(
            order, command.getCustomerOrderCount()
        );
        order.applyDiscount(discount);

        orderRepository.save(order);

        return new OrderResult(order.getOrderId(), order.calculateTotal());
    }
}
```

```java
// === LAYER 4: INFRASTRUCTURE ===

// infrastructure/persistence/JpaOrderRepository.java
public class JpaOrderRepository implements OrderRepository {
```

```
    private final EntityManager entityManager;

    public JpaOrderRepository(EntityManager entityManager) {
        this.entityManager = entityManager;
    }

    @Override
    public Order findById(String orderId) {
        OrderEntity entity = entityManager.find(OrderEntity.class, orderId);
        return toDomain(entity);
    }

    @Override
    public void save(Order order) {
        OrderEntity entity = toEntity(order);
        entityManager.persist(entity);
    }

    @Override
    public List<Order> findByCustomerId(String customerId) {
        // JPA query implementation
        return entityManager
            .createQuery("SELECT o FROM OrderEntity o WHERE o.customerId = :cid")
            .setParameter("cid", customerId)
            .getResultList()
            .stream()
            .map(this::toDomain)
            .collect(Collectors.toList());
    }
}
```

What changed:

- Domain Model has zero infrastructure dependencies
- Business rules are testable with plain unit tests
- Swapping databases means only changing the Infrastructure layer
- Each layer has a single, clear responsibility

The Same Example in Python

```python
# === LAYER 1: DOMAIN MODEL ===

# domain/model/order.py
from dataclasses import dataclass, field
from typing import List
```

```
from abc import ABC, abstractmethod


@dataclass(frozen=True)
class OrderItem:
    """Value Object -- immutable, identity does not matter."""
    product_id: str
    price: float
    quantity: int

    def __post_init__(self):
        if self.price < 0:
            raise ValueError("Price cannot be negative")
        if self.quantity < 1:
            raise ValueError("Quantity must be at least 1")

    @property
    def subtotal(self) -> float:
        return self.price * self.quantity


class Order:
    """Entity -- has identity (order_id)."""
    def __init__(self, order_id: str, items: List[OrderItem]):
        self.order_id = order_id
        self._items = list(items)
        self._discount_percentage = 0.0

    def calculate_total(self) -> float:
        subtotal = sum(item.subtotal for item in self._items)
        return subtotal * (1 - self._discount_percentage / 100)

    def apply_discount(self, percentage: float) -> None:
        if percentage > 50:
            raise ValueError("Discount cannot exceed 50%")
        self._discount_percentage = percentage


class OrderRepository(ABC):
    """Interface defined in the domain -- implemented in infrastructure."""

    @abstractmethod
    def find_by_id(self, order_id: str) -> Order:
        pass

    @abstractmethod
    def save(self, order: Order) -> None:
        pass
```

```python
# === LAYER 2: DOMAIN SERVICES ===

# domain/service/pricing_service.py
```

```
from domain.model.order import Order


class PricingService:
    def calculate_loyalty_discount(self, order: Order,
                                   customer_order_count: int) -> float:
        if customer_order_count > 100:
            return 15.0
        if customer_order_count > 50:
            return 10.0
        if customer_order_count > 10:
            return 5.0
        return 0.0
```

```python
# === LAYER 3: APPLICATION SERVICES ===

# application/place_order_use_case.py
from dataclasses import dataclass
from typing import List
from domain.model.order import Order, OrderItem, OrderRepository
from domain.service.pricing_service import PricingService
import uuid


@dataclass
class PlaceOrderCommand:
    items: List[dict]
    customer_order_count: int


@dataclass
class OrderResult:
    order_id: str
    total: float


class PlaceOrderUseCase:
    def __init__(self, order_repository: OrderRepository,
                 pricing_service: PricingService):
        self._order_repository = order_repository
        self._pricing_service = pricing_service

    def execute(self, command: PlaceOrderCommand) -> OrderResult:
        items = [
            OrderItem(i["product_id"], i["price"], i["quantity"])
            for i in command.items
        ]

        order = Order(str(uuid.uuid4()), items)

        discount = self._pricing_service.calculate_loyalty_discount(
            order, command.customer_order_count
```

```
        )
        order.apply_discount(discount)

        self._order_repository.save(order)

        return OrderResult(order.order_id, order.calculate_total())
```

```python
# === LAYER 4: INFRASTRUCTURE ===

# infrastructure/persistence/sqlalchemy_order_repository.py
from sqlalchemy.orm import Session
from domain.model.order import Order, OrderItem, OrderRepository


class SqlAlchemyOrderRepository(OrderRepository):
    def __init__(self, session: Session):
        self._session = session

    def find_by_id(self, order_id: str) -> Order:
        entity = self._session.query(OrderEntity).get(order_id)
        return self._to_domain(entity)

    def save(self, order: Order) -> None:
        entity = self._to_entity(order)
        self._session.add(entity)
        self._session.commit()

    def _to_domain(self, entity) -> Order:
        items = [
            OrderItem(ie.product_id, ie.price, ie.quantity)
            for ie in entity.items
        ]
        return Order(entity.order_id, items)

    def _to_entity(self, order: Order):
        # Convert domain object to SQLAlchemy entity
        pass
```

Complete Project Folder Structure

Here is how a real project organized with Onion Architecture looks on disk:

```
my-order-service/
|
+-- domain/                          # LAYERS 1 & 2 (innermost)
|   +-- model/
```

```
|   |   +-- order.py                # Order entity
|   |   +-- order_item.py           # OrderItem value object
|   |   +-- customer.py             # Customer entity
|   |   +-- order_repository.py     # Repository interface
|   |   +-- customer_repository.py  # Repository interface
|   |
|   +-- service/
|       +-- pricing_service.py      # Domain service
|       +-- inventory_checker.py    # Domain service
|
+-- application/                    # LAYER 3
|   +-- use_cases/
|   |   +-- place_order.py          # Use case
|   |   +-- cancel_order.py         # Use case
|   |   +-- get_order_status.py     # Query use case
|   |
|   +-- dto/
|       +-- order_commands.py       # Input DTOs
|       +-- order_results.py        # Output DTOs
|
+-- infrastructure/                 # LAYER 4 (outermost)
|   +-- persistence/
|   |   +-- sqlalchemy_order_repo.py # Repository implementation
|   |   +-- orm_models.py           # Database models
|   |
|   +-- web/
|   |   +-- flask_app.py            # Web framework setup
|   |   +-- order_controller.py     # HTTP endpoints
|   |
|   +-- messaging/
|   |   +-- rabbitmq_publisher.py   # Message queue
|   |
|   +-- config/
|       +-- settings.py             # Configuration
|
+-- tests/
    +-- unit/
    |   +-- test_order.py           # Tests domain model (no infra)
    |   +-- test_pricing_service.py # Tests domain service
    |   +-- test_place_order.py     # Tests use case with mocks
    |
    +-- integration/
        +-- test_order_repository.py # Tests with real database
```

Notice how the `domain/` folder has no imports from `infrastructure/`. You can verify this with a simple grep:

```bash
# This should return ZERO results in a properly structured project
grep -r "from infrastructure" domain/
grep -r "import infrastructure" domain/
```

Onion vs Hexagonal vs Clean Architecture

These three architectures share the same core philosophy -- protect the domain from infrastructure -- but differ in terminology and emphasis.

```
+--------------------+--------------------+---------------------+
|  ONION             |  HEXAGONAL         |  CLEAN              |
|  ARCHITECTURE      |  ARCHITECTURE      |  ARCHITECTURE       |
+--------------------+--------------------+---------------------+
| Domain Model       | Domain Model       | Entities            |
+--------------------+--------------------+---------------------+
| Domain Services    | Domain Services    | Use Cases           |
+--------------------+--------------------+---------------------+
| Application        | Application        | Interface           |
| Services           | Services (Ports)   | Adapters            |
+--------------------+--------------------+---------------------+
| Infrastructure     | Adapters           | Frameworks &        |
|                    |                    | Drivers              |
+--------------------+--------------------+---------------------+
| Emphasizes:        | Emphasizes:        | Emphasizes:         |
| Explicit layers    | Ports and adapters | Dependency rule     |
| with strict        | for pluggable      | across concentric   |
| inward dependency  | external systems   | circles             |
+--------------------+--------------------+---------------------+
| Coined by:         | Coined by:         | Coined by:          |
| Jeffrey Palermo    | Alistair Cockburn  | Robert C. Martin    |
| (2008)             | (2005)             | (2012)              |
+--------------------+--------------------+---------------------+
```

Key Differences:

Aspect	Onion	Hexagonal	Clean
Number of layers	4 explicit	2 (inside/outside)	4 concentric rings
How externals connect	Dependency injection	Ports and adapters	Interface adapters
Primary metaphor	Onion rings	Hexagon with ports	Concentric circles
Domain isolation method	Layer boundaries	Port interfaces	Dependency rule
Practical difference	Most prescriptive about layers	Most flexible about structure	Most prescriptive about dependency direction

The truth: In practice, all three lead to very similar code. The differences are mainly in vocabulary and emphasis. Choose the one whose mental model makes the most sense to

your team.

Real-World Scenario: Hospital Patient Management

Imagine you are building a patient management system for a hospital.

```
+-------------------------------------------------------------+
|  INFRASTRUCTURE                                             |
|  - PostgreSQL for patient records                           |
|  - REST API for doctor-facing app                           |
|  - HL7 FHIR adapter for insurance systems                   |
|  - Email service for appointment reminders                  |
|                                                             |
|  +-----------------------------------------------------+  |
|  |  APPLICATION SERVICES                               |  |
|  |  - ScheduleAppointmentUseCase                       |  |
|  |  - AdmitPatientUseCase                              |  |
|  |  - TransferPatientUseCase                           |  |
|  |  - DischargePatientUseCase                          |  |
|  |                                                     |  |
|  |  +-----------------------------------------------+  |  |
|  |  |  DOMAIN SERVICES                              |  |  |
|  |  |  - BedAllocationService                       |  |  |
|  |  |  - InsuranceVerificationService (interface)   |  |  |
|  |  |  - MedicationInteractionChecker               |  |  |
|  |  |                                               |  |  |
|  |  |  +-------------------------------------+      |  |  |
|  |  |  |  DOMAIN MODEL                       |      |  |  |
|  |  |  |  - Patient (entity)                 |      |  |  |
|  |  |  |  - Appointment (entity)             |      |  |  |
|  |  |  |  - MedicalRecord (entity)           |      |  |  |
|  |  |  |  - Diagnosis (value object)         |      |  |  |
|  |  |  |  - Medication (value object)        |      |  |  |
|  |  |  |  - PatientRepository (interface)    |      |  |  |
|  |  |  +-------------------------------------+      |  |  |
|  |  +-----------------------------------------------+  |  |
|  +-----------------------------------------------------+  |
+-------------------------------------------------------------+
```

The Domain Model knows nothing about PostgreSQL, REST, or HL7. If the hospital switches from PostgreSQL to MongoDB, only the Infrastructure layer changes. If they add a mobile app, they add a new adapter in Infrastructure. The domain stays untouched.

When to Use Onion Architecture

Good fit:

- Medium to large applications with complex business logic
- Projects where the domain is the competitive advantage
- Teams that want clear architectural guidance on where code belongs
- Systems that need to support multiple delivery mechanisms (web, mobile, CLI)
- Long-lived applications where infrastructure will change over time

Not a good fit:

- Simple CRUD applications with little business logic
- Prototypes or throwaway projects
- Very small microservices that are essentially data pipes
- Projects where time-to-market matters more than long-term maintainability

---

Common Mistakes

Mistake 1: Putting business logic in the Application Services layer. Application Services should orchestrate, not decide. If you see `if/else` statements about business rules in a use case class, that logic belongs in the Domain Model or Domain Services.

Mistake 2: Having the Domain Model depend on infrastructure types. Your `Order` entity should never import `@Entity` from JPA or inherit from a SQLAlchemy `Base`. Use separate ORM models in the Infrastructure layer and map between them.

Mistake 3: Making every layer depend on a shared "commons" package. A shared utilities package that every layer imports defeats the purpose. If something is truly shared, it should be in the Domain Model (innermost layer). If it is infrastructure-specific, keep it in Infrastructure.

Mistake 4: Skipping layers. If a controller directly calls a repository without going through a use case, you have bypassed the architecture. Every external request should flow through Application Services.

Mistake 5: Over-engineering simple operations. A simple "get user by ID" query does not need a Domain Service, an Application Service, and three DTOs. Use the architecture where it adds value, and keep simple operations simple.

Best Practices

1. Define interfaces in the domain, implement them in infrastructure. Repository interfaces live in the Domain Model. Concrete database classes live in Infrastructure.
2. Keep the Domain Model free of annotations and decorators. No `@Entity`, no `@Column`, no `@JsonProperty`. These are infrastructure concerns.
3. Use Application Services as the entry point for all operations. Controllers call Application Services, never Domain Services or Repositories directly.
4. Test the domain without infrastructure. If your domain tests require a database, something is wrong. Domain tests should be fast, plain unit tests.
5. Map between layers explicitly. Use DTOs at the Application Services boundary. Use ORM entities at the Infrastructure boundary. The Domain Model uses its own types.
6. Enforce the dependency rule with tooling. Use module dependency checks, architectural fitness functions, or import analysis tools to catch violations early.

Quick Summary

Onion Architecture organizes code into four concentric layers: Domain Model at the center, surrounded by Domain Services, then Application Services, and finally Infrastructure on the outside. The fundamental rule is that dependencies always point inward -- outer layers depend on inner layers, never the reverse. This protects your business logic from infrastructure changes and makes the domain testable in isolation.

Key Points

- Onion Architecture has four explicit layers: Domain Model, Domain Services, Application Services, and Infrastructure
- The dependency rule states that code dependencies must always point inward
- The Domain Model is the innermost layer and has zero external dependencies
- Repository interfaces are defined in the domain but implemented in infrastructure
- Infrastructure (databases, web frameworks, external APIs) lives in the outermost layer

- Onion, Hexagonal, and Clean Architecture share the same core philosophy with different vocabulary
- The architecture shines when business logic is complex and infrastructure is expected to change
- Simple CRUD applications may not benefit from the additional structure

---

Practice Questions

1. A developer adds a `@Entity` JPA annotation to a domain class to "save time." What architectural rule does this violate, and what problems could it cause?
2. Your team has a `PricingService` that calculates discounts. It currently lives in the Application Services layer. A new requirement says the discount calculation needs data from two different entities. Where should `PricingService` live, and why?
3. Draw the dependency arrows for this scenario: `OrderController` calls `PlaceOrderUseCase`, which calls `OrderRepository`. `JpaOrderRepository` implements `OrderRepository`. Which arrows point inward, and which point outward?
4. You are told to add Redis caching to speed up order lookups. Which layer does the caching code belong in? Should the `Order` entity know about Redis?
5. A junior developer argues that having four layers is "over-engineering" for their CRUD app. Are they right? Under what circumstances would you agree with them?

Exercises

Exercise 1: Identify the Layer Violations

The following code has three architectural violations. Find them and explain which layer each piece of code should live in.

```python
# domain/model/invoice.py
from sqlalchemy import Column, Integer, String    # Violation?
from infrastructure.email import send_email        # Violation?

class Invoice:
    def __init__(self, amount, customer_email):
        self.amount = amount
```

```
        self.customer_email = customer_email

    def finalize(self):
        if self.amount > 10000:
            send_email(self.customer_email,          # Violation?
                       "Large invoice requires approval")
        self.status = "FINALIZED"
```

Exercise 2: Restructure a Monolith

Take a simple application (a library book checkout system) and restructure it using Onion Architecture. Define:

- At least two entities (e.g., `Book`, `Member`)
- One value object (e.g., `ISBN`)
- One repository interface
- One domain service (e.g., `LateFeesCalculator`)
- One application service (e.g., `CheckOutBookUseCase`)
- One infrastructure implementation (e.g., in-memory repository)

Write the code so that the domain layer has zero imports from infrastructure.

Exercise 3: Swap the Database

Using your solution from Exercise 2, write a second repository implementation that stores data in a JSON file instead of in memory. Verify that you do not need to change any code in the domain or application layers. This proves your architecture is working correctly.

What Is Next?

Now that you understand how to structure your application with explicit layers and inward dependencies, the next chapter explores Domain-Driven Design (DDD) -- the methodology for modeling the business domain that lives at the center of your onion. DDD gives you the tools to build rich, expressive domain models that capture real business rules, not just data containers.

Chapter 22: Domain-Driven Design

What You Will Learn

- What Domain-Driven Design (DDD) is and why it focuses on the business domain
- How Ubiquitous Language bridges the gap between developers and business experts
- The difference between Entities (identity matters) and Value Objects (value matters)
- How Aggregates enforce consistency boundaries
- What Domain Events are and how they decouple parts of your system
- The role of Repositories and Domain Services
- Why a Rich Domain Model beats an Anemic Model

Why This Chapter Matters

Most software projects fail not because of technical problems, but because the software does not accurately model the business it serves. Developers build what they think the business needs. Business people describe what they think they want. The result is a system that satisfies nobody.

Domain-Driven Design attacks this problem head-on. It says: the most important thing in your software is the model of your business domain, and that model should be built collaboratively by developers and domain experts using the same language.

If you have ever been in a meeting where the business team says "policy" and the development team codes it as "rule," or where "customer" means one thing in billing and another thing in shipping, you have experienced the exact problem DDD solves.

DDD is not a framework or a library. It is a way of thinking about software that puts the business domain at the center of every design decision.

Ubiquitous Language: Speak the Same Words

The foundation of DDD is Ubiquitous Language -- a shared vocabulary that developers and business experts use consistently in conversations, documentation, and code.

```
WITHOUT UBIQUITOUS LANGUAGE:

  Business says:      Developer codes:
  +-----------+       +-------------+
  | "Policy"  | ----> | Rule.java   |
  | "Claim"   | ----> | Request.java|
  | "Holder"  | ----> | User.java   |
  | "Premium" | ----> | Payment.java|
  +-----------+       +-------------+

  Result: Constant translation errors, misunderstandings,
          bugs born from miscommunication.

WITH UBIQUITOUS LANGUAGE:

  Business says:      Developer codes:
  +-----------+       +---------------+
  | "Policy"  | ----> | Policy.java   |
  | "Claim"   | ----> | Claim.java    |
  | "Holder"  | ----> | Holder.java   |
  | "Premium" | ----> | Premium.java  |
  +-----------+       +---------------+

  Result: Code reads like the business speaks.
          Fewer misunderstandings. Faster onboarding.
```

Rules for Ubiquitous Language

1. Use the business terms in your code. If the business calls it a "shipment," do not call it a "delivery" or "package" in code.
2. Challenge vague terms. If someone says "process the order," ask what "process" means. It might mean "validate," "charge payment," and "schedule fulfillment" -- three distinct operations.
3. Update the language as understanding evolves. When the team discovers a better term, rename it everywhere: code, tests, documentation, conversations.
4. Write a glossary. Maintain a shared document that defines each term. Review it regularly.

Entities: Identity Matters

An Entity is a domain object with a unique identity that persists over time. Two entities are the same if they have the same identity, even if all other attributes differ.

The key question: "Are these two objects the same thing if they have the same ID, even if their attributes changed?" If yes, it is an Entity.

Java Example

```java
// ENTITY: Two patients with the same patient ID are the same patient,
// even if their name or address changes.

public class Patient {
    private final PatientId id;       // Identity -- never changes
    private String name;               // Can change
    private String address;            // Can change
    private List<Diagnosis> diagnoses; // Can change

    public Patient(PatientId id, String name) {
        this.id = id;
        this.name = name;
        this.diagnoses = new ArrayList<>();
    }

    public void addDiagnosis(Diagnosis diagnosis) {
        // Business rule: cannot have duplicate active diagnoses
        boolean alreadyActive = diagnoses.stream()
            .anyMatch(d -> d.getCode().equals(diagnosis.getCode())
                        && d.isActive());
        if (alreadyActive) {
            throw new IllegalStateException(
                "Patient already has active diagnosis: " + diagnosis.getCode()
            );
        }
        diagnoses.add(diagnosis);
    }

    public void updateAddress(String newAddress) {
        // Business rule: address changes must be logged for compliance
        this.address = newAddress;
        // The patient is still the SAME patient after this change
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Patient)) return false;
        Patient other = (Patient) o;
        return id.equals(other.id);  // Equality based on IDENTITY only
```

```
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}
```

Python Example

```python
# ENTITY: identity-based equality

class Patient:
    def __init__(self, patient_id: str, name: str):
        self.patient_id = patient_id  # Identity -- never changes
        self.name = name               # Can change
        self.address = None            # Can change
        self._diagnoses: list = []     # Can change

    def add_diagnosis(self, diagnosis) -> None:
        """Business rule: no duplicate active diagnoses."""
        for d in self._diagnoses:
            if d.code == diagnosis.code and d.is_active:
                raise ValueError(
                    f"Patient already has active diagnosis: {diagnosis.code}"
                )
        self._diagnoses.append(diagnosis)

    def __eq__(self, other) -> bool:
        if not isinstance(other, Patient):
            return False
        return self.patient_id == other.patient_id  # Identity only

    def __hash__(self) -> int:
        return hash(self.patient_id)
```

Value Objects: Value Matters

A Value Object has no identity. Two Value Objects are equal if all their attributes are equal. They should be immutable -- once created, they never change.

The key question: "Do I care which specific instance this is, or only what value it holds?" A five-dollar bill is a five-dollar bill. You do not care which one.

Java Example

```java
// VALUE OBJECT: Two Money objects with the same amount and currency
// are completely interchangeable.

public final class Money {
    private final BigDecimal amount;
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        if (amount == null) throw new IllegalArgumentException("Amount required");
        if (currency == null || currency.isBlank())
            throw new IllegalArgumentException("Currency required");
        this.amount = amount;
        this.currency = currency.toUpperCase();
    }

    public Money add(Money other) {
        if (!this.currency.equals(other.currency)) {
            throw new IllegalArgumentException("Cannot add different currencies");
        }
        return new Money(this.amount.add(other.amount), this.currency);
    }

    public Money multiply(int quantity) {
        return new Money(this.amount.multiply(BigDecimal.valueOf(quantity)),
                         this.currency);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Money)) return false;
        Money money = (Money) o;
        return amount.compareTo(money.amount) == 0
            && currency.equals(money.currency);  // ALL attributes compared
    }

    @Override
    public int hashCode() {
        return Objects.hash(amount, currency);
    }
}
```

Python Example

```python
# VALUE OBJECT: immutable, equality based on all attributes

from dataclasses import dataclass
```

```python
from decimal import Decimal


@dataclass(frozen=True)   # frozen=True makes it immutable
class Money:
    amount: Decimal
    currency: str

    def __post_init__(self):
        if self.amount is None:
            raise ValueError("Amount required")
        if not self.currency:
            raise ValueError("Currency required")
        # Normalize currency to uppercase
        object.__setattr__(self, 'currency', self.currency.upper())

    def add(self, other: 'Money') -> 'Money':
        if self.currency != other.currency:
            raise ValueError("Cannot add different currencies")
        return Money(self.amount + other.amount, self.currency)

    def multiply(self, quantity: int) -> 'Money':
        return Money(self.amount * quantity, self.currency)
```

Entity vs Value Object Decision Guide

```
+-----------------------------------------------------------------+
|  Ask yourself:                                                  |
|                                                                 |
|  Does it need a unique ID that persists over time?              |
|    YES --> Entity (Patient, Order, Account)                     |
|    NO  --> Value Object (Money, Address, DateRange)             |
|                                                                 |
|  Can two instances with the same attributes be swapped freely?  |
|    YES --> Value Object                                         |
|    NO  --> Entity                                               |
|                                                                 |
|  Does it change state over its lifetime?                        |
|    YES --> Likely an Entity                                     |
|    NO  --> Likely a Value Object (immutable)                    |
+-----------------------------------------------------------------+
```

Aggregates: The Consistency Boundary

An Aggregate is a cluster of Entities and Value Objects treated as a single unit for data changes. Every Aggregate has a root entity -- the only object that external code is allowed to reference.

Why Aggregates matter: They define the boundary within which all business rules are guaranteed to be consistent. Outside the boundary, consistency is eventual.

```
+-----------------------------------------+
|  ORDER AGGREGATE                        |
|                                         |
|  +--------------------+                 |
|  | Order (ROOT)       |                 |
|  | - orderId          |                 |
|  | - status           |                 |
|  | - customerId       |   Reference by  |
|  +----+---------+-----+   ID only       |
|       |         |         (not by object) |
|       v         v                        |
|  +---------+ +---------+                |
|  |OrderItem| |OrderItem|                |
|  |- product| |- product|                |
|  |- price  | |- price  |                |
|  |- qty    | |- qty    |                |
|  +---------+ +---------+                |
|                                         |
+-----------------------------------------+
  External code accesses OrderItems
  ONLY through the Order root.
```

Aggregate Rules

1. The root entity is the only entry point. External objects never hold references to inner objects.
2. The aggregate enforces all invariants. When you add an item to an order, the Order (root) validates the business rules.
3. One transaction per aggregate. Save or update one aggregate at a time. Cross-aggregate consistency is handled through Domain Events.
4. Reference other aggregates by ID, not by object. An Order holds a `customerId`, not a `Customer` object.

Java Example

```java
// AGGREGATE ROOT: Order controls access to its OrderItems

public class Order {
    private final OrderId id;
    private final CustomerId customerId;  // Reference by ID, not object
    private final List<OrderItem> items;
    private OrderStatus status;
    private Money totalPrice;

    public Order(OrderId id, CustomerId customerId) {
        this.id = id;
        this.customerId = customerId;
        this.items = new ArrayList<>();
        this.status = OrderStatus.DRAFT;
        this.totalPrice = Money.zero("USD");
    }

    public void addItem(String productId, Money price, int quantity) {
        // Invariant: cannot add items to a submitted order
        if (status != OrderStatus.DRAFT) {
            throw new IllegalStateException("Cannot modify a submitted order");
        }
        // Invariant: max 50 items per order
        if (items.size() >= 50) {
            throw new IllegalStateException("Order cannot have more than 50 items");
        }

        OrderItem item = new OrderItem(productId, price, quantity);
        items.add(item);
        recalculateTotal();
    }

    public void submit() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot submit an empty order");
        }
        this.status = OrderStatus.SUBMITTED;
        // This would also raise a domain event (covered next)
    }

    private void recalculateTotal() {
        this.totalPrice = items.stream()
            .map(OrderItem::subtotal)
            .reduce(Money.zero("USD"), Money::add);
    }

    // OrderItem is NOT exposed directly -- return an unmodifiable view
    public List<OrderItem> getItems() {
        return Collections.unmodifiableList(items);
    }
}
```

Python Example

```python
# AGGREGATE ROOT

from enum import Enum
from typing import List
from dataclasses import dataclass


class OrderStatus(Enum):
    DRAFT = "DRAFT"
    SUBMITTED = "SUBMITTED"
    CANCELLED = "CANCELLED"


@dataclass(frozen=True)
class OrderItem:
    product_id: str
    price: float
    quantity: int

    @property
    def subtotal(self) -> float:
        return self.price * self.quantity


class Order:
    """Aggregate Root -- all access to OrderItems goes through Order."""

    def __init__(self, order_id: str, customer_id: str):
        self.order_id = order_id
        self.customer_id = customer_id   # Reference by ID
        self._items: List[OrderItem] = []
        self._status = OrderStatus.DRAFT

    def add_item(self, product_id: str, price: float, quantity: int) -> None:
        if self._status != OrderStatus.DRAFT:
            raise ValueError("Cannot modify a submitted order")
        if len(self._items) >= 50:
            raise ValueError("Order cannot have more than 50 items")

        self._items.append(OrderItem(product_id, price, quantity))

    def submit(self) -> None:
        if not self._items:
            raise ValueError("Cannot submit an empty order")
        self._status = OrderStatus.SUBMITTED

    @property
    def total(self) -> float:
        return sum(item.subtotal for item in self._items)

    @property
    def items(self) -> tuple:
        return tuple(self._items)  # Immutable view
```

Domain Events: Something Important Happened

A Domain Event is a record of something that happened in the domain. Events are named in the past tense: `OrderPlaced`, `PaymentReceived`, `PatientAdmitted`.

Why events matter: They let different parts of the system react to changes without being tightly coupled. When an order is placed, the inventory service needs to know -- but the Order aggregate should not call the inventory service directly.

```
+------------------+    OrderPlaced     +-------------------+
|  Order           | =================> |  Inventory        |
|  Aggregate       |    (event)         |  Service          |
+------------------+                    +-------------------+
                     \
                      \  OrderPlaced    +-------------------+
                       ===============> |  Notification     |
                          (event)       |  Service          |
                                        +-------------------+

The Order does not know about Inventory or Notifications.
It just says "this happened." Others decide what to do.
```

Java Example

```java
// Domain Event
public class OrderPlaced {
    private final String orderId;
    private final String customerId;
    private final double totalAmount;
    private final Instant occurredAt;

    public OrderPlaced(String orderId, String customerId,
                       double totalAmount) {
        this.orderId = orderId;
        this.customerId = customerId;
        this.totalAmount = totalAmount;
        this.occurredAt = Instant.now();
    }

    // Getters omitted for brevity
}

// Aggregate raises events
```

```
public class Order {
    private final List<Object> domainEvents = new ArrayList<>();

    public void submit() {
        if (items.isEmpty()) {
            throw new IllegalStateException("Cannot submit empty order");
        }
        this.status = OrderStatus.SUBMITTED;

        // Record the event -- do not publish yet
        domainEvents.add(new OrderPlaced(
            this.id.toString(),
            this.customerId.toString(),
            this.calculateTotal()
        ));
    }

    public List<Object> getDomainEvents() {
        return Collections.unmodifiableList(domainEvents);
    }

    public void clearDomainEvents() {
        domainEvents.clear();
    }
}

// Application Service publishes events after saving
public class PlaceOrderUseCase {
    public void execute(PlaceOrderCommand command) {
        Order order = createOrder(command);
        order.submit();

        orderRepository.save(order);

        // Publish events after successful save
        for (Object event : order.getDomainEvents()) {
            eventPublisher.publish(event);
        }
        order.clearDomainEvents();
    }
}
```

Python Example

```python
# Domain Event
from dataclasses import dataclass, field
from datetime import datetime


@dataclass(frozen=True)
```

```
class OrderPlaced:
    order_id: str
    customer_id: str
    total_amount: float
    occurred_at: datetime = field(default_factory=datetime.utcnow)


# Aggregate with events
class Order:
    def __init__(self, order_id: str, customer_id: str):
        self.order_id = order_id
        self.customer_id = customer_id
        self._items = []
        self._status = OrderStatus.DRAFT
        self._domain_events = []

    def submit(self) -> None:
        if not self._items:
            raise ValueError("Cannot submit empty order")
        self._status = OrderStatus.SUBMITTED

        self._domain_events.append(
            OrderPlaced(
                order_id=self.order_id,
                customer_id=self.customer_id,
                total_amount=self.total,
            )
        )

    @property
    def domain_events(self) -> list:
        return list(self._domain_events)

    def clear_events(self) -> None:
        self._domain_events.clear()
```

Repositories: Collection-Like Access

A Repository provides a collection-like interface for accessing and persisting Aggregates. It hides the details of the data store. To the domain, a Repository looks like an in-memory collection.

```
Domain thinks:              Reality:
+-----------+               +-----------+       +-----------+
| orders.   |               | Repository|       | Database  |
| find(id)  | ------------> | translates| ----> | SELECT *  |
| orders.   |               | to SQL or |       | FROM ...  |
```

```
| save(o)   |               | API calls |       |           |
+-----------+               +-----------+       +-----------+

The domain never knows a database exists.
```

Java Example

```java
// Interface in the domain layer
public interface OrderRepository {
    Order findById(OrderId id);
    void save(Order order);
    List<Order> findByCustomer(CustomerId customerId);
    void delete(OrderId id);
}

// Usage in Application Service -- no database awareness
public class CancelOrderUseCase {
    private final OrderRepository orders;

    public CancelOrderUseCase(OrderRepository orders) {
        this.orders = orders;
    }

    public void execute(String orderId) {
        Order order = orders.findById(new OrderId(orderId));
        if (order == null) {
            throw new OrderNotFoundException(orderId);
        }
        order.cancel();
        orders.save(order);
    }
}
```

Python Example

```python
# Interface in the domain layer
from abc import ABC, abstractmethod


class OrderRepository(ABC):

    @abstractmethod
    def find_by_id(self, order_id: str) -> Order:
        """Returns the Order or raises NotFoundError."""
        pass
```

```
    @abstractmethod
    def save(self, order: Order) -> None:
        pass

    @abstractmethod
    def find_by_customer(self, customer_id: str) -> list:
        pass


# Usage -- no database awareness
class CancelOrderUseCase:
    def __init__(self, orders: OrderRepository):
        self._orders = orders

    def execute(self, order_id: str) -> None:
        order = self._orders.find_by_id(order_id)
        order.cancel()
        self._orders.save(order)
```

Domain Services: Stateless Business Operations

A Domain Service handles business logic that does not naturally belong to a single Entity or Value Object. It is stateless -- it performs a calculation or validation using data from multiple objects and returns a result.

When to Use a Domain Service

- The operation involves multiple aggregates
- The logic does not belong to any single entity
- The operation is a recognized concept in the business domain

```java
// Domain Service: transfer between accounts involves two aggregates
public class MoneyTransferService {

    public TransferResult transfer(Account source, Account destination,
                                   Money amount) {
        if (!source.hasEnoughFunds(amount)) {
            return TransferResult.insufficientFunds();
        }

        source.debit(amount);
        destination.credit(amount);

        return TransferResult.success(source.getId(),
```

```
                                        destination.getId(), amount);
    }
}
```

```python
# Domain Service
class MoneyTransferService:

    def transfer(self, source: Account, destination: Account,
                 amount: Money) -> TransferResult:
        if not source.has_enough_funds(amount):
            return TransferResult.insufficient_funds()

        source.debit(amount)
        destination.credit(amount)

        return TransferResult.success(
            source.account_id, destination.account_id, amount
        )
```

Rich Domain Model vs Anemic Model

This is one of the most important distinctions in DDD. An Anemic Model is a domain object that is just a bag of getters and setters with no behavior. A Rich Domain Model encapsulates both data and behavior.

BEFORE: Anemic Model (Anti-Pattern)

```java
// ANEMIC MODEL: just data, no behavior
public class BankAccount {
    private String accountId;
    private double balance;
    private String status;

    // Only getters and setters -- no business logic
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

// All logic lives in a "service" that manipulates the object
public class BankAccountService {
```

```
    public void withdraw(BankAccount account, double amount) {
        if (account.getStatus().equals("FROZEN")) {
            throw new RuntimeException("Account is frozen");
        }
        if (account.getBalance() < amount) {
            throw new RuntimeException("Insufficient funds");
        }
        account.setBalance(account.getBalance() - amount);
    }
}
```

Problems with the Anemic Model:

- Business rules are scattered across service classes
- The `BankAccount` can be put into an invalid state (negative balance) by anyone calling `setBalance`
- Rules get duplicated -- every service that touches `BankAccount` must remember to check the status
- The class is just a glorified `HashMap`

AFTER: Rich Domain Model

```java
// RICH MODEL: data AND behavior together
public class BankAccount {
    private final AccountId id;
    private Money balance;
    private AccountStatus status;

    public BankAccount(AccountId id, Money initialDeposit) {
        this.id = id;
        this.balance = initialDeposit;
        this.status = AccountStatus.ACTIVE;
    }

    public void withdraw(Money amount) {
        // Business rules are right here, inside the object
        if (status == AccountStatus.FROZEN) {
            throw new AccountFrozenException(id);
        }
        if (balance.isLessThan(amount)) {
            throw new InsufficientFundsException(id, amount, balance);
        }
        this.balance = balance.subtract(amount);
    }

    public void deposit(Money amount) {
        if (status == AccountStatus.CLOSED) {
            throw new AccountClosedException(id);
```

```
        }
        this.balance = balance.add(amount);
    }

    public void freeze() {
        this.status = AccountStatus.FROZEN;
    }

    // No setBalance() -- the only way to change the balance
    // is through withdraw() or deposit(), which enforce the rules.
    public Money getBalance() {
        return balance;  // Money is immutable, so this is safe
    }
}
```

```python
# RICH MODEL in Python
class BankAccount:
    def __init__(self, account_id: str, initial_deposit: Money):
        self.account_id = account_id
        self._balance = initial_deposit
        self._status = AccountStatus.ACTIVE

    def withdraw(self, amount: Money) -> None:
        if self._status == AccountStatus.FROZEN:
            raise AccountFrozenError(self.account_id)
        if self._balance < amount:
            raise InsufficientFundsError(
                self.account_id, amount, self._balance
            )
        self._balance = self._balance.subtract(amount)

    def deposit(self, amount: Money) -> None:
        if self._status == AccountStatus.CLOSED:
            raise AccountClosedError(self.account_id)
        self._balance = self._balance.add(amount)

    def freeze(self) -> None:
        self._status = AccountStatus.FROZEN

    @property
    def balance(self) -> Money:
        return self._balance   # No setter -- enforced via methods
```

Comparison Table

```
+-----------------------+-------------------+--------------------+
|  Aspect               | Anemic Model      | Rich Domain Model  |
+-----------------------+-------------------+--------------------+
| Business logic lives  | In service classes | Inside the entity  |
```

```
| Data + behavior       | Separated          | Together           |
| Invalid states        | Possible           | Prevented by design|
| Rule duplication      | Common             | Eliminated         |
| Testability           | Must test services | Test the entity    |
| Encapsulation         | Weak (setters)     | Strong (methods)   |
| Code reads like       | Technical steps    | Business language  |
+-----------------------+--------------------+--------------------+
```

Real-World Scenario: Insurance Claims System

An insurance company needs to model their claims process.

Ubiquitous Language glossary:

- Policy: An insurance contract between the insurer and the policyholder
- Claim: A request by the policyholder for payment after a covered event
- Adjuster: The person who evaluates the claim
- Settlement: The agreed payment amount

```
AGGREGATES:

+------------------+     +------------------+     +------------------+
| Policy           |     | Claim            |     | Adjuster         |
| (Aggregate Root) |     | (Aggregate Root) |     | (Aggregate Root) |
|                  |     |                  |     |                  |
| - policyId       |<-id-| - policyId (ref) |     | - adjusterId     |
| - holderId (ref) |     | - claimId        |<-id-| - currentClaim   |
| - coverageItems  |     | - documents      |     |   Id (ref)       |
| - status         |     | - adjusterId(ref)|     | - specialties    |
| - effectiveDate  |     | - settlement     |     |                  |
| - expirationDate |     | - status         |     |                  |
+------------------+     +------------------+     +------------------+

Each aggregate is loaded and saved independently.
Cross-aggregate references are by ID only.

DOMAIN EVENTS:
  ClaimFiled --> triggers assignment to Adjuster
  ClaimApproved --> triggers payment processing
  PolicyExpired --> triggers notification to holder
```

Common Mistakes

Mistake 1: Using DDD for simple CRUD applications. If your application is mostly reading and writing data with little business logic, DDD adds unnecessary complexity. Use it when the domain is genuinely complex.

Mistake 2: Making aggregates too large. An Order aggregate that contains Customer, Product, Warehouse, and ShippingRoute objects is too big. Keep aggregates small and reference other aggregates by ID.

Mistake 3: Ignoring Ubiquitous Language. If the code uses `User` but the business says `Policyholder`, you are losing the primary benefit of DDD. Rename the code.

Mistake 4: Treating Domain Services as a dumping ground. If most of your logic lives in Domain Services rather than Entities, you have an Anemic Model. Push behavior into the entities first.

Mistake 5: Skipping the domain experts. DDD without domain expert involvement is just architecture theater. The whole point is collaborative modeling with people who understand the business.

Best Practices

1. Start with the Ubiquitous Language. Before writing code, spend time with domain experts to establish shared terminology. Write it down.
2. Make Value Objects your default. When in doubt, model something as a Value Object. They are simpler, safer, and easier to test. Only use Entities when identity truly matters.
3. Keep Aggregates small. A good aggregate fits in your head. If you cannot explain what it protects in one sentence, it is too big.
4. Use Domain Events for cross-aggregate communication. Do not have one aggregate reach into another. Raise an event and let the other aggregate react.
5. Put business rules in the domain objects. If you find yourself writing `if (order.getStatus() == ...)` in a service class, that check belongs inside the `Order` class.
6. Test the domain in isolation. Domain tests should be fast, require no database, and read like business specifications.

Quick Summary

Domain-Driven Design is an approach to software development that places the business domain at the center of the design process. It starts with Ubiquitous Language -- a shared vocabulary used by both developers and business experts. The domain model is built from Entities (objects with identity), Value Objects (objects defined by their attributes), and Aggregates (consistency boundaries with a root entity). Domain Events capture significant occurrences. Repositories provide collection-like access to aggregates. Domain Services handle logic spanning multiple objects. A Rich Domain Model encapsulates both data and behavior, preventing invalid states by design.

Key Points

- DDD focuses on modeling the business domain, not the technology
- Ubiquitous Language eliminates translation errors between business and development
- Entities have unique identity; Value Objects are defined by their attributes
- Aggregates are consistency boundaries with a single root entity
- Domain Events decouple parts of the system by broadcasting that something happened
- Repositories hide persistence details behind a collection-like interface
- Domain Services handle stateless operations spanning multiple entities
- Rich Domain Models encapsulate data and behavior; Anemic Models are an anti-pattern
- DDD is most valuable for complex domains -- do not use it for simple CRUD

Practice Questions

1. You are modeling an e-commerce system. Is a shipping address an Entity or a Value Object? Explain your reasoning.
2. An `Order` aggregate currently includes a `Customer` object loaded from the database. What is wrong with this design, and how would you fix it?
3. A developer puts the rule "orders over $500 require manager approval" in the `OrderController` class. Where should this rule live according to DDD, and why?
4. Explain why an Anemic Model makes it easier to create objects in invalid states. Give a specific example.

5. When should you use a Domain Service instead of putting logic directly in an Entity?

Exercises

Exercise 1: Model a Library System

Using DDD concepts, model a library system with the following requirements:

- Members can borrow books
- A member can have at most 5 books checked out
- Books have ISBNs and titles
- Overdue books incur a daily fine

Identify: Which classes are Entities? Which are Value Objects? What is the Aggregate Root? What Domain Events would you raise?

Exercise 2: Anemic to Rich

Refactor this anemic model into a rich domain model:

```java
public class Subscription {
    private String id;
    private String plan;      // "BASIC", "PRO", "ENTERPRISE"
    private String status;    // "ACTIVE", "CANCELLED", "EXPIRED"
    private LocalDate startDate;
    private LocalDate endDate;

    // Only getters and setters
}

public class SubscriptionService {
    public void cancel(Subscription sub) {
        if (sub.getStatus().equals("CANCELLED")) {
            throw new RuntimeException("Already cancelled");
        }
        sub.setStatus("CANCELLED");
        sub.setEndDate(LocalDate.now());
    }

    public void upgrade(Subscription sub, String newPlan) {
        if (!sub.getStatus().equals("ACTIVE")) {
            throw new RuntimeException("Can only upgrade active subscriptions");
        }
        sub.setPlan(newPlan);
```

```
    }
}
```

Move the business rules into the `Subscription` class. Remove the setters. Make invalid states impossible.

Exercise 3: Design Domain Events

For an online food delivery system, identify at least four Domain Events. For each event, describe:

- The event name (past tense)
- What data it carries
- What other parts of the system might react to it

What Is Next?

DDD works best when you clearly define the boundaries of your domain models. In large systems, different parts of the business use the same words to mean different things -- "customer" in sales is not the same as "customer" in support. The next chapter on Bounded Contexts shows you how to draw these boundaries, prevent naming collisions, and build systems where each team has its own clean, focused model.

Chapter 23: Bounded Contexts

What You Will Learn

- What Bounded Contexts are and why they are the most important pattern in Domain-Driven Design
- How to identify boundaries where the same word means different things
- Context Mapping: how bounded contexts relate to each other
- The Anti-Corruption Layer pattern for protecting your model from external systems
- How bounded contexts naturally align with microservices
- Practical strategies for defining and enforcing context boundaries

Why This Chapter Matters

In the previous chapter on Domain-Driven Design, we learned about Ubiquitous Language -- the shared vocabulary between developers and domain experts. But here is a critical insight that many teams miss: Ubiquitous Language only works within a boundary.

In a real business, the word "product" means one thing to the catalog team, another to the warehouse team, and something entirely different to the accounting team. Trying to create a single `Product` class that satisfies all three teams creates a monster -- a class with dozens of fields, half of which are irrelevant to any given use case.

Bounded Contexts solve this problem. They say: each part of your system gets its own model, its own language, and its own boundary. Inside that boundary, the model is consistent and clear. Between boundaries, you translate explicitly.

If you have ever seen a codebase where one "god object" is passed everywhere and every team adds their own fields to it, you have seen what happens without Bounded Contexts. This chapter shows you a better way.

The Problem: One Word, Many Meanings

Consider an e-commerce company. The word "Customer" appears everywhere, but it means different things to different teams:

```
THE SAME WORD, DIFFERENT MEANINGS:

  Sales Team:                Shipping Team:             Billing Team:
  +------------------+      +------------------+       +------------------+
  | Customer         |      | Customer         |       | Customer         |
  |------------------|      |------------------|       |------------------|
  | - name           |      | - name           |       | - name           |
  | - email          |      | - shippingAddr   |       | - billingAddr    |
  | - preferences    |      | - deliveryNotes  |       | - paymentMethod  |
  | - leadSource     |      | - accessCode     |       | - creditLimit    |
  | - purchaseHist   |      | - preferredTime  |       | - taxId          |
  | - loyaltyTier    |      | - signatureReq   |       | - invoiceHistory |
  +------------------+      +------------------+       +------------------+

  If you merge all of these into ONE Customer class:

  +----------------------------------+
  | Customer (God Object)            |
  |----------------------------------|
  | - name, email, preferences       |
  | - leadSource, purchaseHistory    |
  | - loyaltyTier                    |
  | - shippingAddress                |
  | - deliveryNotes, accessCode      |
  | - preferredDeliveryTime          |
  | - signatureRequired              |
  | - billingAddress                 |
  | - paymentMethod, creditLimit     |
  | - taxId, invoiceHistory          |
  |----------------------------------|
  | 12+ fields, most irrelevant to   |
  | any single use case              |
  +----------------------------------+
```

This merged class is a maintenance nightmare. Every change risks breaking unrelated features. Every developer must understand fields they do not use. Testing requires setting up irrelevant data.

What Is a Bounded Context?

A Bounded Context is an explicit boundary within which a particular domain model applies. Inside the boundary, every term has exactly one meaning, and the model is internally consistent.

```
BOUNDED CONTEXTS: Each context owns its own model.

  +----------------------+    +----------------------+    +----------------------+
  |   SALES CONTEXT      |    |  SHIPPING CONTEXT    |    |  BILLING CONTEXT     |
  |                      |    |                      |    |                      |
  | Customer:            |    | Recipient:           |    | Account:             |
  |  - name              |    |  - name              |    |  - name              |
  |  - email             |    |  - address           |    |  - billingAddress    |
  |  - preferences       |    |  - deliveryNotes     |    |  - paymentMethod     |
  |  - leadSource        |    |  - accessCode        |    |  - creditLimit       |
  |  - loyaltyTier       |    |  - preferredTime     |    |  - taxId             |
  |                      |    |                      |    |                      |
  | Order:               |    | Shipment:            |    | Invoice:             |
  |  - items             |    |  - packages          |    |  - lineItems         |
  |  - total             |    |  - carrier           |    |  - taxes             |
  |  - status            |    |  - trackingNumber    |    |  - amountDue         |
  +----------------------+    +----------------------+    +----------------------+

  Key insight: "Customer" in Sales, "Recipient" in Shipping, and
  "Account" in Billing are DIFFERENT models of the same real person.
  Each model contains only what that context needs.
```

Key Properties of a Bounded Context

1. Own language. Terms inside the context have precise, unambiguous meanings.
2. Own model. Classes, entities, and value objects are defined for this context only.
3. Explicit boundary. You know exactly what is inside and outside the context.
4. Independence. Changes inside one context should not force changes in another.

Identifying Bounded Contexts

How do you find the boundaries? Here are practical techniques:

1. Listen for Language Conflicts

When the same word means different things to different people, you have found a boundary.

```
LANGUAGE CONFLICT SIGNALS:

  "When you say 'order,' do you mean..."
    Sales:    "The customer's purchase request"
    Shipping: "The instruction to ship specific packages"
    Billing:  "The financial transaction to invoice"

  Each of these is a different concept that happens to share a name.
  Each belongs in its own Bounded Context.
```

2. Look for Different Lifecycles

If two concepts change at different rates or for different reasons, they probably belong in different contexts.

```
LIFECYCLE DIFFERENCES:

  Product Catalog Context:         Inventory Context:
  +---------------------+         +---------------------+
  | Product             |         | StockItem           |
  | - Changes when      |         | - Changes when      |
  |   marketing updates |         |   warehouse receives |
  |   descriptions      |         |   or ships items    |
  | - Updated by: Content|        | - Updated by:       |
  |   team, monthly     |         |   Warehouse, hourly |
  +---------------------+         +---------------------+

  Same physical product, different models, different change rates.
```

3. Look for Different Stakeholders

Different teams or departments often indicate different bounded contexts.

Context Mapping: How Contexts Relate

Bounded Contexts do not exist in isolation. They must communicate. A Context Map shows how contexts relate to each other and who has power in the relationship.

```
E-COMMERCE CONTEXT MAP:

  +-----------------+          +-------------------+
```

```
  |                  |  Order   |                   |
  |  SALES           |  Placed  |  ORDER            |
  |  CONTEXT         |--------->|  FULFILLMENT      |
  |                  |          |  CONTEXT           |
  | (Upstream)       |          |  (Downstream)     |
  +------------------+          +-------------------+
         |                              |
         |                              |
         | Customer                     | Shipment
         | Created                      | Ready
         v                              v
  +------------------+          +-------------------+
  |                  |          |                   |
  |  CUSTOMER        |          |  SHIPPING         |
  |  MANAGEMENT      |          |  CONTEXT          |
  |  CONTEXT         |          |                   |
  +------------------+          +-------------------+
         |                              |
         |                              |
         | Account                      | Delivery
         | Info                         | Confirmed
         v                              v
  +------------------+          +-------------------+
  |                  |  Invoice |                   |
  |  BILLING         |<--------|  NOTIFICATION     |
  |  CONTEXT         |  Request |  CONTEXT          |
  |                  |          |                   |
  +------------------+          +-------------------+

  Arrows show the direction of data flow and events.
  Upstream contexts produce data. Downstream contexts consume it.
```

Common Relationship Patterns

1. Customer-Supplier (Upstream-Downstream)

The upstream context provides data that the downstream context needs. The upstream team should accommodate reasonable requests from the downstream team.

```java
// UPSTREAM: Sales Context publishes an event when an order is placed
public class OrderPlacedEvent {
    private final String orderId;
    private final String customerId;
    private final List<OrderLineItem> items;
    private final Money totalAmount;
    private final Instant occurredAt;

    public OrderPlacedEvent(String orderId, String customerId,
                            List<OrderLineItem> items, Money totalAmount) {
```

```
        this.orderId = orderId;
        this.customerId = customerId;
        this.items = List.copyOf(items);
        this.totalAmount = totalAmount;
        this.occurredAt = Instant.now();
    }

    // Getters omitted for brevity
}

// DOWNSTREAM: Fulfillment Context consumes the event
// and translates it into its own model
public class FulfillmentOrderCreator {

    public FulfillmentOrder createFrom(OrderPlacedEvent event) {
        // Translate from Sales language to Fulfillment language
        List<PickItem> pickList = event.getItems().stream()
            .map(item -> new PickItem(
                item.getSku(),           // Sales calls it "productId"
                item.getQuantity(),      // Fulfillment calls it "pickQuantity"
                lookupWarehouseLocation(item.getSku())
            ))
            .collect(Collectors.toList());

        return new FulfillmentOrder(
            event.getOrderId(),
            pickList,
            FulfillmentPriority.fromAmount(event.getTotalAmount())
        );
    }
}
```

```python
# UPSTREAM: Sales Context publishes an event
from dataclasses import dataclass, field
from datetime import datetime
from typing import List

@dataclass(frozen=True)
class OrderPlacedEvent:
    order_id: str
    customer_id: str
    items: List[dict]
    total_amount: float
    occurred_at: datetime = field(default_factory=datetime.utcnow)


# DOWNSTREAM: Fulfillment Context translates it
class FulfillmentOrderCreator:

    def create_from(self, event: OrderPlacedEvent) -> "FulfillmentOrder":
        """Translate from Sales language to Fulfillment language."""
        pick_list = [
```

```
            PickItem(
                sku=item["product_id"],
                pick_quantity=item["quantity"],
                location=self._lookup_warehouse_location(item["product_id"])
            )
            for item in event.items
        ]

        return FulfillmentOrder(
            order_id=event.order_id,
            pick_list=pick_list,
            priority=FulfillmentPriority.from_amount(event.total_amount)
        )

    def _lookup_warehouse_location(self, sku: str) -> str:
        # Look up where this item is stored
        ...
```

2. Shared Kernel

Two contexts share a small, common model that both teams agree on and maintain together.

```
SHARED KERNEL:

  +------------------+     +------------------+
  |                  |     |                  |
  |  SALES           |     |  BILLING         |
  |  CONTEXT         |     |  CONTEXT         |
  |                  |     |                  |
  |       +----------+-----+----------+       |
  |       |    SHARED KERNEL          |       |
  |       |  - Money value object     |       |
  |       |  - Currency enum          |       |
  |       |  - Address value object   |       |
  |       +---------------------------+       |
  |                  |     |                  |
  +------------------+     +------------------+

  WARNING: Keep the shared kernel SMALL.
  The larger it grows, the more coupling you create.
```

3. Conformist

The downstream context conforms to the upstream context's model without any translation. This is common when integrating with external APIs you cannot change.

4. Anti-Corruption Layer (ACL)

The downstream context creates a translation layer to protect its model from the upstream context's model.

The Anti-Corruption Layer Pattern

The Anti-Corruption Layer is one of the most important patterns for maintaining clean boundaries. It acts as a translator between two contexts, ensuring that one context's model does not leak into another.

```
ANTI-CORRUPTION LAYER:

  +------------------+     +---+     +------------------+
  |                  |     |   |     |                  |
  |  EXTERNAL        |     | A |     |  YOUR            |
  |  PAYMENT         |---->| C |---->|  BILLING         |
  |  GATEWAY         |     | L |     |  CONTEXT         |
  |                  |     |   |     |                  |
  |  Their model:    |     |   |     |  Your model:     |
  |  - txn_id        |     | T |     |  - paymentId     |
  |  - amt_cents     |     | R |     |  - amount (Money)|
  |  - ccy_code      |     | A |     |  - currency      |
  |  - stat          |     | N |     |  - status        |
  |  - cust_ref      |     | S |     |  - customerId    |
  |  - err_cd        |     | L |     |  - failureReason |
  |                  |     | A |     |                  |
  |  Ugly, cryptic   |     | T |     |  Clean, clear    |
  |  abbreviations   |     | E |     |  domain language |
  +------------------+     +---+     +------------------+
```

Java Example

```java
// EXTERNAL SYSTEM: Payment gateway response (their model, their language)
// You cannot change this -- it comes from a third-party API
public class PayGatewayResponse {
    public String txn_id;
    public int amt_cents;
    public String ccy_code;
    public String stat;       // "OK", "FAIL", "PEND"
    public String cust_ref;
    public String err_cd;     // "E001", "E002", etc.
}

// ANTI-CORRUPTION LAYER: Translates external model to your domain model
public class PaymentGatewayAdapter {

    public PaymentResult translate(PayGatewayResponse response) {
```

```
        return new PaymentResult(
            new PaymentId(response.txn_id),
            Money.ofMinorUnits(response.amt_cents,
                               Currency.fromCode(response.ccy_code)),
            translateStatus(response.stat),
            new CustomerId(response.cust_ref),
            translateError(response.err_cd)
        );
    }

    private PaymentStatus translateStatus(String stat) {
        return switch (stat) {
            case "OK"   -> PaymentStatus.COMPLETED;
            case "FAIL" -> PaymentStatus.FAILED;
            case "PEND" -> PaymentStatus.PENDING;
            default     -> throw new UnknownPaymentStatusException(stat);
        };
    }

    private FailureReason translateError(String errorCode) {
        if (errorCode == null) return null;
        return switch (errorCode) {
            case "E001" -> FailureReason.INSUFFICIENT_FUNDS;
            case "E002" -> FailureReason.CARD_EXPIRED;
            case "E003" -> FailureReason.FRAUD_SUSPECTED;
            default     -> FailureReason.UNKNOWN;
        };
    }
}

// YOUR DOMAIN MODEL: Clean, expressive, uses your Ubiquitous Language
public record PaymentResult(
    PaymentId paymentId,
    Money amount,
    PaymentStatus status,
    CustomerId customerId,
    FailureReason failureReason
) {
    public boolean isSuccessful() {
        return status == PaymentStatus.COMPLETED;
    }

    public boolean requiresRetry() {
        return status == PaymentStatus.FAILED
            && failureReason != FailureReason.FRAUD_SUSPECTED;
    }
}
```

```python
# EXTERNAL SYSTEM: Payment gateway response (their ugly model)
# You cannot change this -- it comes from a third-party API

# ANTI-CORRUPTION LAYER: Translates external to your domain
```

```python
from dataclasses import dataclass
from enum import Enum
from typing import Optional

class PaymentStatus(Enum):
    COMPLETED = "completed"
    FAILED = "failed"
    PENDING = "pending"

class FailureReason(Enum):
    INSUFFICIENT_FUNDS = "insufficient_funds"
    CARD_EXPIRED = "card_expired"
    FRAUD_SUSPECTED = "fraud_suspected"
    UNKNOWN = "unknown"

@dataclass(frozen=True)
class PaymentResult:
    """Your clean domain model."""
    payment_id: str
    amount_cents: int
    currency: str
    status: PaymentStatus
    customer_id: str
    failure_reason: Optional[FailureReason] = None

    @property
    def is_successful(self) -> bool:
        return self.status == PaymentStatus.COMPLETED

    @property
    def requires_retry(self) -> bool:
        return (self.status == PaymentStatus.FAILED
                and self.failure_reason != FailureReason.FRAUD_SUSPECTED)


class PaymentGatewayAdapter:
    """Anti-Corruption Layer: translates external model to domain model."""

    STATUS_MAP = {
        "OK": PaymentStatus.COMPLETED,
        "FAIL": PaymentStatus.FAILED,
        "PEND": PaymentStatus.PENDING,
    }

    ERROR_MAP = {
        "E001": FailureReason.INSUFFICIENT_FUNDS,
        "E002": FailureReason.CARD_EXPIRED,
        "E003": FailureReason.FRAUD_SUSPECTED,
    }

    def translate(self, response: dict) -> PaymentResult:
        status = self.STATUS_MAP.get(response["stat"])
        if status is None:
            raise ValueError(f"Unknown payment status: {response['stat']}")
```

```
        failure_reason = None
        if response.get("err_cd"):
            failure_reason = self.ERROR_MAP.get(
                response["err_cd"], FailureReason.UNKNOWN
            )

        return PaymentResult(
            payment_id=response["txn_id"],
            amount_cents=response["amt_cents"],
            currency=response["ccy_code"],
            status=status,
            customer_id=response["cust_ref"],
            failure_reason=failure_reason,
        )
```

Why the Anti-Corruption Layer Matters

Without an ACL, the external system's model leaks into your code:

```
WITHOUT ACL (Bad):

  Your service methods use external types directly:

  def process_payment(self, gateway_response):
      if gateway_response["stat"] == "OK":        # External language
          amount = gateway_response["amt_cents"]    # External naming
          # Your code is now coupled to their abbreviations

WITH ACL (Good):

  Your service methods use your own domain types:

  def process_payment(self, result: PaymentResult):
      if result.is_successful:                      # Your language
          amount = result.amount_cents               # Your naming
          # Your code is clean and independent
```

Bounded Contexts and Microservices

There is a natural alignment between bounded contexts and microservices, but they are not the same thing.

```
BOUNDED CONTEXTS vs MICROSERVICES:

  +-------------------------------------------------------+
  |  Bounded Context = LOGICAL boundary                   |
  |  (A model boundary in your domain)                    |
  |                                                       |
  |  Microservice = DEPLOYMENT boundary                   |
  |  (A separately deployable unit)                       |
  +-------------------------------------------------------+

  IDEAL ALIGNMENT:

  +-------------------+    +-------------------+    +-------------------+
  |  Sales Context    |    |  Shipping Context|    |  Billing Context |
  |  = Sales Service |    |  = Ship Service  |    |  = Bill Service  |
  +-------------------+    +-------------------+    +-------------------+
       One bounded context per microservice

  COMMON MISTAKE:

  +-------------------------------------------+
  |  One Big Monolith Service                 |
  |  +----------+  +----------+  +----------+|
  |  | Sales    |  | Shipping |  | Billing  ||
  |  | Context  |  | Context  |  | Context  ||
  |  +----------+  +----------+  +----------+|
  +-------------------------------------------+
       Multiple contexts in one service (OK for starting)

  WORSE MISTAKE:

  +----------+  +----------+  +----------+
  | Service1 |  | Service2 |  | Service3 |
  | Part of  |  | Part of  |  | Part of  |
  | Sales    |  | Sales    |  | Sales    |
  +----------+  +----------+  +----------+
       One context split across multiple services (BAD)
```

Guidelines for Alignment

1. Start with a monolith and identify bounded contexts first. Do not jump to microservices.
2. One bounded context per microservice is the ideal target.
3. Never split one bounded context across multiple microservices. This creates distributed coupling -- the worst kind.
4. Multiple bounded contexts in one deployment is acceptable when starting out (modular monolith).

Practical Example: E-Commerce System

Let us design the bounded contexts for an e-commerce system step by step.

Step 1: Identify the Sub-Domains

```
E-COMMERCE SUB-DOMAINS:

  +---------------+  +---------------+  +---------------+
  |   Product     |  |   Ordering    |  |   Shipping    |
  |   Catalog     |  |               |  |               |
  +---------------+  +---------------+  +---------------+

  +---------------+  +---------------+  +---------------+
  |   Payment     |  |   Customer    |  |   Inventory   |
  |   Processing  |  |   Management  |  |               |
  +---------------+  +---------------+  +---------------+
```

Step 2: Define the Models in Each Context

```java
// PRODUCT CATALOG CONTEXT
// Here, a "Product" is rich with marketing information
public class Product {
    private final ProductId id;
    private String name;
    private String description;
    private List<String> images;
    private List<Category> categories;
    private List<ProductAttribute> attributes;  // color, size, material
    private Money listPrice;

    public void updateDescription(String description) {
        // Marketing team updates this
        this.description = description;
    }
}

// INVENTORY CONTEXT
// Here, the same physical product is a "StockItem" -- only quantity matters
public class StockItem {
    private final String sku;
    private int quantityOnHand;
    private int quantityReserved;
    private WarehouseLocation location;

    public int availableQuantity() {
        return quantityOnHand - quantityReserved;
```

```
    }

    public void reserve(int quantity) {
        if (quantity > availableQuantity()) {
            throw new InsufficientStockException(sku, quantity, availableQuantity());
        }
        this.quantityReserved += quantity;
    }

    public void confirmPickup(int quantity) {
        this.quantityOnHand -= quantity;
        this.quantityReserved -= quantity;
    }
}

// ORDERING CONTEXT
// Here, it is an "OrderItem" -- just a line in an order
public class OrderItem {
    private final String productId;
    private final String productName;   // Snapshot at time of order
    private final Money unitPrice;      // Snapshot at time of order
    private final int quantity;

    public Money lineTotal() {
        return unitPrice.multiply(quantity);
    }
}
```

```python
# PRODUCT CATALOG CONTEXT
@dataclass
class Product:
    """In the catalog context, a product has rich marketing data."""
    id: str
    name: str
    description: str
    images: list[str]
    categories: list[str]
    attributes: dict[str, str]  # color, size, material
    list_price: float


# INVENTORY CONTEXT
@dataclass
class StockItem:
    """In inventory context, same physical product is just quantity."""
    sku: str
    quantity_on_hand: int
    quantity_reserved: int
    warehouse_location: str

    @property
    def available_quantity(self) -> int:
```

```
        return self.quantity_on_hand - self.quantity_reserved

    def reserve(self, quantity: int) -> None:
        if quantity > self.available_quantity:
            raise InsufficientStockError(
                f"Cannot reserve {quantity} of {self.sku}. "
                f"Only {self.available_quantity} available."
            )
        self.quantity_reserved += quantity


# ORDERING CONTEXT
@dataclass(frozen=True)
class OrderItem:
    """In ordering context, it is a line item with a price snapshot."""
    product_id: str
    product_name: str   # Snapshot at time of order
    unit_price: float   # Snapshot at time of order
    quantity: int

    @property
    def line_total(self) -> float:
        return self.unit_price * self.quantity
```

Step 3: Define How Contexts Communicate

```java
// When an order is placed in the ORDERING CONTEXT,
// it publishes an event for other contexts to react to.

// ORDERING CONTEXT publishes:
public record OrderPlacedEvent(
    String orderId,
    String customerId,
    List<OrderItemSnapshot> items,
    Money totalAmount,
    Instant occurredAt
) {}

// INVENTORY CONTEXT listens and reserves stock:
public class InventoryEventHandler {
    private final StockItemRepository stockItems;

    public void onOrderPlaced(OrderPlacedEvent event) {
        for (OrderItemSnapshot item : event.items()) {
            StockItem stock = stockItems.findBySku(item.sku());
            stock.reserve(item.quantity());
            stockItems.save(stock);
        }
    }
}
```

```
// PAYMENT CONTEXT listens and initiates payment:
public class PaymentEventHandler {
    private final PaymentService payments;

    public void onOrderPlaced(OrderPlacedEvent event) {
        payments.initiatePayment(
            new PaymentRequest(
                event.orderId(),
                event.customerId(),
                event.totalAmount()
            )
        );
    }
}
```

```python
# INVENTORY CONTEXT listens and reserves stock
class InventoryEventHandler:
    def __init__(self, stock_repository):
        self.stock_repository = stock_repository

    def on_order_placed(self, event: dict) -> None:
        for item in event["items"]:
            stock = self.stock_repository.find_by_sku(item["sku"])
            stock.reserve(item["quantity"])
            self.stock_repository.save(stock)


# PAYMENT CONTEXT listens and initiates payment
class PaymentEventHandler:
    def __init__(self, payment_service):
        self.payment_service = payment_service

    def on_order_placed(self, event: dict) -> None:
        self.payment_service.initiate_payment(
            order_id=event["order_id"],
            customer_id=event["customer_id"],
            amount=event["total_amount"],
        )
```

Enforcing Boundaries in Code

Bounded contexts are only useful if you enforce their boundaries in your code. Here are practical techniques:

Package/Module Structure

```
JAVA PROJECT STRUCTURE:

  com.shop/
  +-- catalog/                    # Product Catalog Context
  |   +-- domain/
  |   |   +-- Product.java
  |   |   +-- Category.java
  |   +-- application/
  |   |   +-- ProductService.java
  |   +-- infrastructure/
  |       +-- ProductRepository.java
  |
  +-- ordering/                   # Ordering Context
  |   +-- domain/
  |   |   +-- Order.java
  |   |   +-- OrderItem.java
  |   +-- application/
  |   |   +-- OrderService.java
  |   +-- infrastructure/
  |       +-- OrderRepository.java
  |
  +-- inventory/                  # Inventory Context
  |   +-- domain/
  |   |   +-- StockItem.java
  |   +-- application/
  |   |   +-- StockService.java
  |   +-- infrastructure/
  |       +-- StockRepository.java
  |
  +-- shared/                     # Shared Kernel (keep small!)
      +-- Money.java
      +-- Currency.java


PYTHON PROJECT STRUCTURE:

  shop/
  +-- catalog/                    # Product Catalog Context
  |   +-- __init__.py
  |   +-- domain.py
  |   +-- services.py
  |   +-- repository.py
  |
  +-- ordering/                   # Ordering Context
  |   +-- __init__.py
  |   +-- domain.py
  |   +-- services.py
  |   +-- repository.py
  |
  +-- inventory/                  # Inventory Context
  |   +-- __init__.py
  |   +-- domain.py
  |   +-- services.py
  |   +-- repository.py
```

```
  |
  +-- shared/                     # Shared Kernel (keep small!)
      +-- __init__.py
      +-- money.py
```

Rules for Boundary Enforcement

1. No direct imports across contexts. The ordering context should never import from the inventory context's domain.
2. Communicate through events or defined interfaces. Use published events, APIs, or shared contracts.
3. Each context has its own database schema (ideally). No shared tables between contexts.
4. Use architecture fitness functions to automatically check for boundary violations in CI.

```java
// BAD: Direct dependency across context boundaries
// ordering/application/OrderService.java
import com.shop.inventory.domain.StockItem;  // VIOLATION!

public class OrderService {
    public void placeOrder(Order order) {
        // Directly reaching into another context
        StockItem stock = stockRepo.findBySku(order.getSku());  // WRONG
        stock.reserve(order.getQuantity());
    }
}

// GOOD: Communicate through an event or interface
// ordering/application/OrderService.java
public class OrderService {
    private final EventPublisher events;

    public void placeOrder(Order order) {
        order.place();
        orderRepo.save(order);
        // Publish event -- inventory context will handle reservation
        events.publish(new OrderPlacedEvent(
            order.getId(), order.getItems()
        ));
    }
}
```

Common Mistakes

Mistake 1: Making Bounded Contexts Too Small

Creating a bounded context for every entity leads to excessive complexity and constant inter-context communication.

Fix: A bounded context should encompass a cohesive set of related concepts, not a single class. If two concepts always change together, they belong in the same context.

Mistake 2: Sharing a Database Between Contexts

When two contexts read and write the same tables, changes in one context can break the other.

Fix: Each context should own its data. Use events to synchronize data between contexts.

Mistake 3: No Anti-Corruption Layer for External Systems

Directly using an external API's response types in your domain model couples your code to their design decisions.

Fix: Always create an ACL when integrating with external systems. Translate their model into your domain language.

Mistake 4: Letting the Shared Kernel Grow

Starting with a small shared kernel is fine. But over time, teams add more and more to it, creating tight coupling between contexts.

Fix: Regularly review the shared kernel. If it grows beyond a few value objects, split it or move things into individual contexts.

Mistake 5: Ignoring Organizational Boundaries

Bounded contexts should align with team boundaries (Conway's Law). A single team should own a single bounded context.

Fix: Map your contexts to your team structure. If two teams are working on one context, either split the context or merge the teams.

Best Practices

1. Start with the domain, not the technology. Identify bounded contexts through domain analysis, not by choosing microservices first.
2. Use events for inter-context communication. Events decouple contexts better than direct calls.
3. Keep the shared kernel minimal. Only share what is truly common (Money, Currency, basic value objects).
4. Build an Anti-Corruption Layer for every external integration. Never let external models leak into your domain.
5. Enforce boundaries with package structure and build rules. Make violations visible and hard to introduce.
6. Align contexts with team ownership. One team, one context. This follows Conway's Law.
7. Document your context map. Keep it visible and up-to-date as a team reference.
8. Evolve boundaries as understanding deepens. Your first context map will be wrong. Refine it as you learn.

Quick Summary

```
BOUNDED CONTEXTS AT A GLANCE:

  What:    An explicit boundary within which a domain model applies
  Why:     The same word means different things in different parts
           of a business. One model cannot serve all needs.
  How:     Define separate models per context. Communicate through
           events and translation layers.

  Key patterns:
  +---------------------------+----------------------------------+
  | Pattern                   | When to use                      |
  +---------------------------+----------------------------------+
  | Customer-Supplier         | Upstream provides, downstream    |
  |                           | consumes. Teams can negotiate.   |
  | Shared Kernel             | Two contexts share a small,      |
  |                           | jointly maintained model.         |
  | Anti-Corruption Layer     | Protect your model from an       |
  |                           | external or legacy system.        |
  | Conformist                | You cannot change the upstream.   |
  |                           | You just conform to their model. |
  | Separate Ways             | No integration needed. Contexts  |
  |                           | are completely independent.       |
  +---------------------------+----------------------------------+
```

Key Points

- A Bounded Context is an explicit boundary where a particular domain model applies and every term has one clear meaning.
- The same real-world concept (like "Customer") can and should have different representations in different contexts.
- Context Mapping shows how bounded contexts relate: Customer-Supplier, Shared Kernel, Anti-Corruption Layer, Conformist.
- The Anti-Corruption Layer translates between an external model and your domain model, protecting your code from external design decisions.
- Bounded contexts align naturally with microservices, but start with a modular monolith and extract services later.
- Enforce boundaries through package structure, build rules, and architectural fitness functions.
- Communicate between contexts using events, not direct calls or shared databases.

Practice Questions

1. You are building a hospital system. The word "Patient" is used by the admissions desk, the medical staff, and the billing department. Each group needs different information about the patient. How would you model this using bounded contexts?
2. Your team integrates with a third-party shipping API that returns responses with cryptic field names like `shp_stat`, `dlv_dt`, and `rcpt_nm`. How would you protect your domain model from this external API's design?
3. Two bounded contexts both need to work with monetary values (amounts with currency). Should you duplicate the `Money` class in each context, or share it? What are the trade-offs?
4. Your e-commerce system has a Product Catalog context and an Inventory context. A developer proposes that both contexts should read from the same `products` database table. What problems could this cause, and what would you suggest instead?
5. A team has split their Order Management bounded context into three microservices: one for creating orders, one for processing payments, and one for shipping. Each service needs to access the Order entity. What is wrong with this design?

Exercises

Exercise 1: Context Map for a University System

Draw a context map for a university system with these sub-domains: Student Enrollment, Course Management, Grading, Financial Aid, and Library. For each pair of related contexts, identify the relationship type (Customer-Supplier, Shared Kernel, ACL, etc.) and explain why.

Exercise 2: Build an Anti-Corruption Layer

You integrate with a legacy HR system that returns employee data in this format:

```json
{
  "emp_no": "E12345",
  "f_nm": "Jane",
  "l_nm": "Smith",
  "dept_cd": "ENG",
  "sal_grade": 7,
  "mgr_emp_no": "E00042",
  "hire_dt": "20190315"
}
```

Write an Anti-Corruption Layer (in Java or Python) that translates this into a clean `Employee` domain object with proper field names, parsed dates, and meaningful types.

Exercise 3: Identify Bounded Contexts

You are building a food delivery platform. Stakeholder interviews reveal these concepts: Restaurant Menu, Customer Orders, Delivery Tracking, Payment Processing, Restaurant Ratings, Driver Management, and Promotions. Group these into bounded contexts, define the key entity in each context, and explain how the contexts communicate.

What Is Next?

Now that you understand how to define clear boundaries between different parts of your domain, the next chapter explores Dependency Injection -- a technique for managing dependencies within a bounded context. You will learn how constructor injection, DI containers, and the dependency inversion principle work together to make your code testable, flexible, and loosely coupled.

Chapter 24: Dependency Injection

What You Will Learn

- What Dependency Injection (DI) is and why it is essential for clean, testable code
- The three types of injection: constructor, setter, and interface injection
- Why constructor injection is the preferred approach
- How DI containers work in Spring (Java) and Python (dependency-injector)
- Why the Service Locator is an anti-pattern
- How DI transforms untestable code into easily testable code
- Practical before-and-after refactoring examples

Why This Chapter Matters

Imagine you are building a house. Every room has its own plumbing, electrical wiring, and ventilation. Now imagine that every room manufactures its own pipes, wires, and fans from raw materials. That would be absurd -- rooms should receive the infrastructure they need, not build it themselves.

Yet this is exactly what most beginners do in code. Classes create their own dependencies using `new` -- they manufacture the objects they need instead of receiving them. This makes testing impossible (you cannot substitute a fake database), changes expensive (swapping implementations requires editing every class), and the code rigid.

Dependency Injection says: a class should receive its dependencies from the outside, not create them itself. This one principle transforms code from rigid and untestable to flexible and clean.

If you have ever struggled to write a unit test because the class you are testing creates a real database connection inside itself, Dependency Injection is the solution.

The Problem: Hard-Coded Dependencies

Look at this common pattern in beginner code:

Java -- Before (Hard-Coded Dependencies)

```java
// BAD: OrderService creates its own dependencies
public class OrderService {

    // Hard-coded dependency -- created inside the class
    private final OrderRepository repository = new MySqlOrderRepository();
    private final EmailService emailService = new SmtpEmailService();
    private final PaymentGateway paymentGateway = new StripePaymentGateway();

    public void placeOrder(Order order) {
        paymentGateway.charge(order.getCustomerId(), order.getTotal());
        repository.save(order);
        emailService.sendConfirmation(order.getCustomerId(), order.getId());
    }
}
```

Python -- Before (Hard-Coded Dependencies)

```python
# BAD: OrderService creates its own dependencies
class OrderService:
    def __init__(self):
        # Hard-coded dependencies -- created inside the class
        self.repository = MySqlOrderRepository()
        self.email_service = SmtpEmailService()
        self.payment_gateway = StripePaymentGateway()

    def place_order(self, order):
        self.payment_gateway.charge(order.customer_id, order.total)
        self.repository.save(order)
        self.email_service.send_confirmation(order.customer_id, order.id)
```

What Is Wrong With This?

```
PROBLEMS WITH HARD-CODED DEPENDENCIES:

  1. UNTESTABLE
     +-------------------+
     | OrderService      |
     |  creates          |
```

```
     |  +-------------+  |    To test OrderService, you MUST
     |  | Real MySQL  |  |    have a running MySQL database,
     |  | Database    |  |    a working SMTP server, and a
     |  +-------------+  |    valid Stripe account.
     |  +-------------+  |
     |  | Real SMTP   |  |    That is not a unit test.
     |  | Server      |  |    That is an integration test.
     |  +-------------+  |
     |  +-------------+  |
     |  | Real Stripe |  |
     |  | Gateway     |  |
     |  +-------------+  |
     +-------------------+

  2. RIGID
     Swapping MySQL for PostgreSQL requires editing OrderService.
     Swapping Stripe for PayPal requires editing OrderService.
     Every change ripples through the codebase.

  3. VIOLATES DEPENDENCY INVERSION
     High-level policy (OrderService) depends on
     low-level details (MySqlOrderRepository).
```

The Solution: Dependency Injection

Dependency Injection means passing dependencies to a class from the outside, rather than having the class create them internally.

Java -- After (Constructor Injection)

```java
// GOOD: Dependencies are injected through the constructor
public class OrderService {

    private final OrderRepository repository;
    private final EmailService emailService;
    private final PaymentGateway paymentGateway;

    // Dependencies come IN through the constructor
    public OrderService(OrderRepository repository,
                        EmailService emailService,
                        PaymentGateway paymentGateway) {
        this.repository = repository;
        this.emailService = emailService;
        this.paymentGateway = paymentGateway;
    }
```

```
    public void placeOrder(Order order) {
        paymentGateway.charge(order.getCustomerId(), order.getTotal());
        repository.save(order);
        emailService.sendConfirmation(order.getCustomerId(), order.getId());
    }
}
```

Python -- After (Constructor Injection)

```python
# GOOD: Dependencies are injected through the constructor
class OrderService:
    def __init__(
        self,
        repository: OrderRepository,
        email_service: EmailService,
        payment_gateway: PaymentGateway,
    ):
        # Dependencies come IN through the constructor
        self.repository = repository
        self.email_service = email_service
        self.payment_gateway = payment_gateway

    def place_order(self, order):
        self.payment_gateway.charge(order.customer_id, order.total)
        self.repository.save(order)
        self.email_service.send_confirmation(order.customer_id, order.id)
```

What Changed?

```
BEFORE vs AFTER:

  BEFORE (Hard-coded):            AFTER (Injected):

  +-------------------+           +-------------------+
  | OrderService      |           | OrderService      |
  |  CREATES:         |           |  RECEIVES:        |
  |  MySqlRepo        |           |  OrderRepository  | <-- interface
  |  SmtpEmail        |           |  EmailService     | <-- interface
  |  StripeGateway    |           |  PaymentGateway   | <-- interface
  +-------------------+           +-------------------+
                                        ^   ^   ^
                                        |   |   |
  The class knows                  Caller provides
  exactly which                    any implementation
  implementations                  that fits the
  it uses.                         interface.
```

Types of Dependency Injection

1. Constructor Injection (Preferred)

Dependencies are provided through the constructor. The object cannot be created without all its dependencies.

```java
// CONSTRUCTOR INJECTION -- the gold standard
public class NotificationService {

    private final EmailSender emailSender;
    private final SmsSender smsSender;

    // All dependencies declared upfront. Cannot create the object without them.
    public NotificationService(EmailSender emailSender, SmsSender smsSender) {
        this.emailSender = Objects.requireNonNull(emailSender);
        this.smsSender = Objects.requireNonNull(smsSender);
    }

    public void notify(User user, String message) {
        if (user.prefersEmail()) {
            emailSender.send(user.getEmail(), message);
        } else {
            smsSender.send(user.getPhone(), message);
        }
    }
}
```

```python
# CONSTRUCTOR INJECTION -- the gold standard
class NotificationService:
    def __init__(self, email_sender: EmailSender, sms_sender: SmsSender):
        if email_sender is None or sms_sender is None:
            raise ValueError("Dependencies cannot be None")
        self.email_sender = email_sender
        self.sms_sender = sms_sender

    def notify(self, user, message: str) -> None:
        if user.prefers_email:
            self.email_sender.send(user.email, message)
        else:
            self.sms_sender.send(user.phone, message)
```

Why constructor injection is preferred:

- Completeness. The object is fully initialized after construction. No half-created objects.

- Immutability. Dependencies can be final/readonly. They never change after creation.
- Visibility. All dependencies are declared in one place -- the constructor signature.
- Fail fast. Missing dependencies cause an error at construction time, not at runtime.

2. Setter Injection

Dependencies are provided through setter methods after construction. The object can exist without all dependencies set.

```java
// SETTER INJECTION -- use only for optional dependencies
public class ReportGenerator {

    private ReportFormatter formatter;  // Optional dependency

    // Required dependency via constructor
    private final DataSource dataSource;

    public ReportGenerator(DataSource dataSource) {
        this.dataSource = dataSource;
        this.formatter = new DefaultFormatter();  // sensible default
    }

    // Optional: caller can override the default formatter
    public void setFormatter(ReportFormatter formatter) {
        this.formatter = formatter;
    }

    public Report generate() {
        Data data = dataSource.fetch();
        return formatter.format(data);
    }
}
```

```python
# SETTER INJECTION -- use only for optional dependencies
class ReportGenerator:
    def __init__(self, data_source: DataSource):
        self.data_source = data_source               # required
        self.formatter = DefaultFormatter()           # sensible default

    def set_formatter(self, formatter: ReportFormatter):
        """Optional: override the default formatter."""
        self.formatter = formatter

    def generate(self) -> Report:
        data = self.data_source.fetch()
        return self.formatter.format(data)
```

When to use setter injection:

- The dependency is truly optional and has a sensible default.
- The dependency may need to change after construction (rare).

Risks of setter injection:

- Object may be used before all dependencies are set.
- Dependencies are mutable, which can lead to bugs.
- Harder to reason about the object's state.

3. Interface Injection

The dependency provides an injector method that the client must implement. This is rarely used in modern code.

```java
// INTERFACE INJECTION -- rarely used, shown for completeness
public interface LoggerAware {
    void setLogger(Logger logger);
}

public class UserService implements LoggerAware {
    private Logger logger;

    @Override
    public void setLogger(Logger logger) {
        this.logger = logger;
    }
}
```

Comparison of Injection Types

```
INJECTION TYPE COMPARISON:

  +-------------------+------------+-----------+-------------+
  | Criteria          | Constructor| Setter    | Interface   |
  +-------------------+------------+-----------+-------------+
  | Completeness      | YES        | NO        | NO          |
  | Immutability      | YES        | NO        | NO          |
  | Required deps     | CLEAR      | UNCLEAR   | UNCLEAR     |
  | Optional deps     | AWKWARD    | GOOD      | AWKWARD     |
  | Testability       | EXCELLENT  | GOOD      | GOOD        |
  | Recommended       | DEFAULT    | SOMETIMES | RARELY      |
  +-------------------+------------+-----------+-------------+

  Rule of thumb:
```

```
  - Use CONSTRUCTOR injection for required dependencies (90% of cases)
  - Use SETTER injection for optional dependencies (10% of cases)
  - Avoid interface injection
```

The Testing Benefit

The single biggest benefit of Dependency Injection is testability. When dependencies are injected, you can substitute fakes, mocks, or stubs during testing.

Java -- Testing with Injected Dependencies

```java
// BEFORE DI: Testing is nearly impossible
// You would need a real MySQL database, SMTP server, and Stripe account

// AFTER DI: Testing is trivial
class OrderServiceTest {

    @Test
    void placeOrder_chargesPaymentAndSavesOrder() {
        // Create test doubles
        FakeOrderRepository fakeRepo = new FakeOrderRepository();
        FakeEmailService fakeEmail = new FakeEmailService();
        FakePaymentGateway fakePayment = new FakePaymentGateway();

        // Inject test doubles
        OrderService service = new OrderService(fakeRepo, fakeEmail, fakePayment);

        // Act
        Order order = new Order("customer-1", List.of(
            new OrderItem("widget", 2, Money.of(10))
        ));
        service.placeOrder(order);

        // Assert
        assertTrue(fakePayment.wasCharged("customer-1", Money.of(20)));
        assertTrue(fakeRepo.wasSaved(order));
        assertTrue(fakeEmail.wasSentTo("customer-1"));
    }
}

// Simple fake for testing -- no framework needed
class FakeOrderRepository implements OrderRepository {
    private final List<Order> saved = new ArrayList<>();

    @Override
```

```
    public void save(Order order) {
        saved.add(order);
    }

    public boolean wasSaved(Order order) {
        return saved.contains(order);
    }
}
```

Python -- Testing with Injected Dependencies

```python
# AFTER DI: Testing is trivial
def test_place_order_charges_payment_and_saves():
    # Create test doubles
    fake_repo = FakeOrderRepository()
    fake_email = FakeEmailService()
    fake_payment = FakePaymentGateway()

    # Inject test doubles
    service = OrderService(fake_repo, fake_email, fake_payment)

    # Act
    order = Order(customer_id="customer-1", items=[
        OrderItem("widget", quantity=2, unit_price=10.0)
    ])
    service.place_order(order)

    # Assert
    assert fake_payment.was_charged("customer-1", 20.0)
    assert fake_repo.was_saved(order)
    assert fake_email.was_sent_to("customer-1")


# Simple fake for testing
class FakeOrderRepository:
    def __init__(self):
        self.saved_orders = []

    def save(self, order):
        self.saved_orders.append(order)

    def was_saved(self, order) -> bool:
        return order in self.saved_orders
```

```
TESTING WITHOUT DI vs WITH DI:

  WITHOUT DI:                        WITH DI:
  +-------------------+              +--------------------+
  | Test              |              | Test               |
```

```
  | +---------------+ |             | +---------------+ |
  | | OrderService  | |             | | OrderService  | |
  | | +---+---+---+ | |             | +---+-----------+ |
  | | |DB |SMTP|$$| | |             |     |   |   |     |
  | | +---+---+---+ | |             |  +--+  ++-+ +--+  |
  | +---------------+ |             |  |Fake||Fake||Fake| |
  +-------------------+             |  |Repo||Mail||Pay | |
                                    |  +----++----++----+ |
  Requires:                         +-------------------+
  - Running database
  - Running mail server              Requires:
  - Payment API credentials          - Nothing! Just objects.
  - Network access
  - Minutes to run                    Runs in milliseconds.
```

DI Containers

In a small application, you can wire dependencies manually. But as the application grows, manual wiring becomes tedious. DI containers automate the wiring.

Spring Framework (Java)

Spring is the most popular DI container in Java. It can automatically discover and inject dependencies.

```java
// Step 1: Define your interfaces
public interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String id);
}

public interface EmailService {
    void sendConfirmation(String customerId, String orderId);
}

// Step 2: Implement them and mark as Spring components
@Repository  // Tells Spring: "This is a bean, manage it for me"
public class JpaOrderRepository implements OrderRepository {
    @Override
    public void save(Order order) {
        // JPA implementation
    }

    @Override
```

```
    public Optional<Order> findById(String id) {
        // JPA implementation
        return Optional.empty();
    }
}

@Component
public class SmtpEmailService implements EmailService {
    @Override
    public void sendConfirmation(String customerId, String orderId) {
        // SMTP implementation
    }
}

// Step 3: Declare dependencies via constructor -- Spring injects automatically
@Service
public class OrderService {

    private final OrderRepository repository;
    private final EmailService emailService;

    // Spring sees this constructor and automatically injects
    // the matching beans (JpaOrderRepository, SmtpEmailService)
    public OrderService(OrderRepository repository,
                        EmailService emailService) {
        this.repository = repository;
        this.emailService = emailService;
    }

    public void placeOrder(Order order) {
        repository.save(order);
        emailService.sendConfirmation(order.getCustomerId(), order.getId());
    }
}
```

```
HOW SPRING DI WORKS:

  1. Application starts
  2. Spring scans for @Component, @Service, @Repository, etc.
  3. Spring creates instances (beans) of each
  4. Spring looks at constructors and matches parameters to beans
  5. Spring injects the matching beans automatically

  +------------------+
  | Spring Container |
  |                  |
  | Beans:           |     OrderService needs OrderRepository?
  | - JpaOrderRepo   |---> Here, take JpaOrderRepo.
  | - SmtpEmailSvc   |
  | - OrderService   |     OrderService needs EmailService?
  |                  |---> Here, take SmtpEmailSvc.
  +------------------+
```

Python Dependency Injection

Python has several approaches. Here is a clean pattern using `dependency-injector`:

```python
# Approach 1: Manual wiring (good for small apps)
def create_app():
    """Composition root: wire everything together in one place."""
    # Create implementations
    repository = PostgresOrderRepository(connection_string="...")
    email_service = SmtpEmailService(smtp_host="mail.example.com")
    payment_gateway = StripePaymentGateway(api_key="...")

    # Wire them together
    order_service = OrderService(
        repository=repository,
        email_service=email_service,
        payment_gateway=payment_gateway,
    )

    return Application(order_service=order_service)


# Approach 2: Using dependency-injector library
from dependency_injector import containers, providers

class Container(containers.DeclarativeContainer):
    """DI Container: defines how to create and wire objects."""

    config = providers.Configuration()

    # Infrastructure
    order_repository = providers.Singleton(
        PostgresOrderRepository,
        connection_string=config.db.connection_string,
    )

    email_service = providers.Singleton(
        SmtpEmailService,
        smtp_host=config.email.smtp_host,
    )

    payment_gateway = providers.Singleton(
        StripePaymentGateway,
        api_key=config.payment.api_key,
    )

    # Application services -- automatically wired
    order_service = providers.Factory(
        OrderService,
        repository=order_repository,
        email_service=email_service,
        payment_gateway=payment_gateway,
    )
```

```
# Usage
container = Container()
container.config.from_yaml("config.yml")

order_service = container.order_service()
order_service.place_order(order)

# For testing: override specific dependencies
with container.order_repository.override(FakeOrderRepository()):
    test_service = container.order_service()
    # test_service uses FakeOrderRepository but real email and payment
```

Python with FastAPI

```python
# FastAPI has built-in dependency injection
from fastapi import Depends, FastAPI

app = FastAPI()

def get_repository() -> OrderRepository:
    return PostgresOrderRepository(connection_string="...")

def get_email_service() -> EmailService:
    return SmtpEmailService(smtp_host="mail.example.com")

def get_order_service(
    repo: OrderRepository = Depends(get_repository),
    email: EmailService = Depends(get_email_service),
) -> OrderService:
    return OrderService(repository=repo, email_service=email)


@app.post("/orders")
def create_order(
    order_data: OrderRequest,
    service: OrderService = Depends(get_order_service),
):
    service.place_order(order_data.to_order())
    return {"status": "created"}


# Testing: override dependencies
def test_create_order():
    app.dependency_overrides[get_repository] = lambda: FakeOrderRepository()
    app.dependency_overrides[get_email_service] = lambda: FakeEmailService()

    client = TestClient(app)
    response = client.post("/orders", json={"item": "widget", "qty": 1})
```

```
    assert response.status_code == 200

    app.dependency_overrides.clear()
```

The Service Locator Anti-Pattern

The Service Locator pattern looks like it solves the same problem as DI, but it introduces hidden dependencies and makes code harder to understand and test.

```java
// SERVICE LOCATOR ANTI-PATTERN
// Dependencies are HIDDEN -- you cannot see them from the constructor

public class OrderService {

    public void placeOrder(Order order) {
        // Dependencies are fetched from a global registry
        OrderRepository repo = ServiceLocator.get(OrderRepository.class);
        EmailService email = ServiceLocator.get(EmailService.class);
        PaymentGateway payment = ServiceLocator.get(PaymentGateway.class);

        payment.charge(order.getCustomerId(), order.getTotal());
        repo.save(order);
        email.sendConfirmation(order.getCustomerId(), order.getId());
    }
}
```

```python
# SERVICE LOCATOR ANTI-PATTERN
class OrderService:
    def place_order(self, order):
        # Dependencies are HIDDEN inside the method
        repo = ServiceLocator.get(OrderRepository)
        email = ServiceLocator.get(EmailService)
        payment = ServiceLocator.get(PaymentGateway)

        payment.charge(order.customer_id, order.total)
        repo.save(order)
        email.send_confirmation(order.customer_id, order.id)
```

Why Service Locator Is an Anti-Pattern

```
SERVICE LOCATOR vs DEPENDENCY INJECTION:
```

```
SERVICE LOCATOR:                  DEPENDENCY INJECTION:

class OrderService:               class OrderService:
    def place_order(self):            def __init__(self, repo, email):
        repo = SL.get(Repo)               self.repo = repo
        email = SL.get(Email)             self.email = email
        ...                           ...

Problems:                         Benefits:
+-------------------------------+ +-------------------------------+
| 1. Hidden dependencies        | | 1. Explicit dependencies      |
|    Cannot see what the class  | |    Constructor tells you       |
|    needs without reading the  | |    exactly what is needed.     |
|    method body.               | |                               |
|                               | |                               |
| 2. Global mutable state       | | 2. No global state            |
|    ServiceLocator is a global | |    Dependencies are local      |
|    registry that any code     | |    to each object.             |
|    can modify.                | |                               |
|                               | |                               |
| 3. Hard to test               | | 3. Easy to test               |
|    Must configure the global  | |    Just pass test doubles     |
|    locator before every test. | |    to the constructor.         |
|                               | |                               |
| 4. Runtime errors             | | 4. Compile-time safety        |
|    Fails at runtime if a      | |    Missing dependency is a    |
|    service is not registered. | |    compile error (or TypeError)|
+-------------------------------+ +-------------------------------+
```

The Composition Root

Where do you actually create and wire all the dependencies? In a single place called the Composition Root -- the entry point of your application.

```java
// COMPOSITION ROOT: Wire everything together at the application entry point
public class Application {

    public static void main(String[] args) {
        // Create implementations
        OrderRepository repo = new JpaOrderRepository(dataSource());
        EmailService email = new SmtpEmailService("mail.example.com");
        PaymentGateway payment = new StripePaymentGateway(apiKey());

        // Wire them together
        OrderService orderService = new OrderService(repo, email, payment);
        OrderController controller = new OrderController(orderService);
```

```
        // Start the application
        WebServer server = new WebServer(controller);
        server.start(8080);
    }
}
```

```python
# COMPOSITION ROOT: Wire everything together at the application entry point
def main():
    # Create implementations
    repo = PostgresOrderRepository(connection_string=os.getenv("DB_URL"))
    email = SmtpEmailService(smtp_host=os.getenv("SMTP_HOST"))
    payment = StripePaymentGateway(api_key=os.getenv("STRIPE_KEY"))

    # Wire them together
    order_service = OrderService(
        repository=repo,
        email_service=email,
        payment_gateway=payment,
    )

    # Start the application
    app = create_web_app(order_service=order_service)
    app.run(port=8080)


if __name__ == "__main__":
    main()
```

```
THE COMPOSITION ROOT PRINCIPLE:

  +-------------------------------------------------------+
  |                                                       |
  |  Application Entry Point (Composition Root)          |
  |                                                       |
  |  Creates: Repo, EmailSvc, PaymentGw                   |
  |  Wires:   OrderService(Repo, EmailSvc, PaymentGw)    |
  |  Starts:  WebServer(OrderController(OrderService))   |
  |                                                       |
  +-------------------------------------------------------+
       |            |              |
       v            v              v
  +---------+  +---------+   +---------+
  | Repo    |  | Email   |   | Payment |
  +---------+  +---------+   +---------+

  Rule: Dependency creation happens in ONE place.
        All other code receives its dependencies.
```

Common Mistakes

Mistake 1: Too Many Constructor Parameters

If your constructor takes more than 4-5 parameters, your class probably does too much. This is a signal to split the class.

```java
// BAD: Too many dependencies = too many responsibilities
public class OrderService(
    OrderRepository repo,
    EmailService email,
    PaymentGateway payment,
    InventoryService inventory,
    TaxCalculator tax,
    ShippingCalculator shipping,
    AuditLogger audit,
    NotificationService notifications,
    DiscountEngine discounts
) { ... }

// GOOD: Split into focused classes
public class OrderPlacer(OrderRepository repo, PaymentGateway payment) { ... }
public class OrderNotifier(EmailService email, NotificationService notif) { ... }
public class OrderPricer(TaxCalculator tax, DiscountEngine discounts) { ... }
```

Mistake 2: Injecting the Container Itself

Never inject the DI container into a class. That is just a Service Locator in disguise.

```java
// BAD: Injecting the container
public class OrderService {
    private final ApplicationContext context;  // Spring container

    public OrderService(ApplicationContext context) {
        this.context = context;
    }

    public void placeOrder(Order order) {
        // This is a Service Locator!
        OrderRepository repo = context.getBean(OrderRepository.class);
    }
}

// GOOD: Inject the actual dependencies
public class OrderService {
    private final OrderRepository repository;
```

```
    public OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

Mistake 3: Using `new` for Services Inside Business Logic

If you use `new` to create a service (not a value object or data structure), you are creating a hard-coded dependency.

```python
# BAD: Creating services with new inside business logic
class OrderService:
    def place_order(self, order):
        validator = OrderValidator()           # Hard-coded!
        pricer = PricingEngine()               # Hard-coded!
        validator.validate(order)
        total = pricer.calculate(order)

# GOOD: Inject the services
class OrderService:
    def __init__(self, validator: OrderValidator, pricer: PricingEngine):
        self.validator = validator
        self.pricer = pricer

    def place_order(self, order):
        self.validator.validate(order)
        total = self.pricer.calculate(order)
```

Mistake 4: Circular Dependencies

If A depends on B and B depends on A, you have a design problem. DI will expose this, which is a feature, not a bug.

Fix: Extract the shared logic into a third class that both A and B depend on.

Best Practices

1. Use constructor injection for required dependencies. Make them final/readonly. This is the default choice.
2. Use setter injection only for optional dependencies with sensible defaults.

3. Program to interfaces, not implementations. Inject `OrderRepository`, not `MySqlOrderRepository`.
4. Keep constructors simple. No logic, no computation -- just assignment.
5. Wire everything in the Composition Root. One place for all dependency creation.
6. Avoid the Service Locator pattern. It hides dependencies and makes code harder to understand.
7. Watch for too many constructor parameters. More than 4-5 means your class has too many responsibilities.
8. Never inject the DI container itself. That is a Service Locator in disguise.

Quick Summary

```
DEPENDENCY INJECTION AT A GLANCE:

  What:    Providing dependencies from outside rather than
           creating them inside a class.

  Why:     Makes code testable, flexible, and loosely coupled.

  How:     Constructor injection (preferred), setter injection
           (optional deps), DI containers (Spring, dependency-injector).

  +---------------------------+-------------------------------+
  | WITHOUT DI                | WITH DI                       |
  +---------------------------+-------------------------------+
  | Class creates deps        | Class receives deps           |
  | Rigid, tightly coupled    | Flexible, loosely coupled     |
  | Hard to test              | Easy to test with fakes       |
  | Changes ripple everywhere | Changes are local             |
  | new MySqlRepo() inside    | OrderRepository interface     |
  +---------------------------+-------------------------------+
```

Key Points

- Dependency Injection means receiving dependencies from outside rather than creating them with `new` inside the class.
- Constructor injection is the preferred method: it makes dependencies explicit, required, and immutable.

- Setter injection is for optional dependencies that have sensible defaults.
- DI containers (Spring, dependency-injector, FastAPI Depends) automate wiring but are not required for small apps.
- The Service Locator pattern is an anti-pattern because it hides dependencies and introduces global state.
- The Composition Root is the single place in your application where all dependencies are created and wired together.
- Too many constructor parameters signal that your class has too many responsibilities and should be split.
- The biggest benefit of DI is testability: you can substitute fakes and mocks without changing production code.

Practice Questions

1. What is the key difference between Dependency Injection and the Service Locator pattern? Why is the Service Locator considered an anti-pattern?
2. A class has a constructor with eight parameters. What does this tell you about the class design, and what would you do to fix it?
3. When would you use setter injection instead of constructor injection? Give a concrete example.
4. Explain what a Composition Root is and why all dependency wiring should happen there. What problems arise if you scatter `new` calls throughout your codebase?
5. You are reviewing code that uses `@Autowired` on a private field in a Spring application (field injection). Why is this problematic, and what would you recommend instead?

Exercises

Exercise 1: Refactor to Use DI

Refactor the following class to use constructor injection. Create the necessary interfaces and write a unit test using fake implementations.

```java
public class UserRegistrationService {
    private final UserDatabase db = new MySqlUserDatabase();
    private final PasswordHasher hasher = new BcryptPasswordHasher();
    private final WelcomeEmailSender sender = new SmtpWelcomeEmailSender();

    public void register(String email, String password) {
        if (db.findByEmail(email) != null) {
            throw new EmailAlreadyExistsException(email);
        }
        String hashed = hasher.hash(password);
        db.save(new User(email, hashed));
        sender.sendWelcome(email);
    }
}
```

Exercise 2: Build a Composition Root

Create a Composition Root for a simple blogging application with these components: `PostService` (depends on `PostRepository` and `NotificationService`), `PostRepository` (depends on `DatabaseConnection`), `NotificationService` (depends on `EmailSender`). Wire everything together manually, then show how you would override `EmailSender` for testing.

Exercise 3: Identify the Anti-Pattern

The following code uses a DI container but still has a problem. Identify the anti-pattern and fix it.

```python
class ReportService:
    def __init__(self, container):
        self.container = container

    def generate_report(self, report_type):
        if report_type == "sales":
            repo = self.container.get(SalesRepository)
            formatter = self.container.get(SalesFormatter)
        elif report_type == "inventory":
            repo = self.container.get(InventoryRepository)
            formatter = self.container.get(InventoryFormatter)
        data = repo.fetch_all()
        return formatter.format(data)
```

What Is Next?

Now that you know how to inject dependencies cleanly, the next chapter -- Design for Change -- explores how to build software that anticipates and accommodates change. You will learn about abstraction boundaries, feature flags, the strategy pattern, and how to balance flexibility with simplicity.

Chapter 25: Design for Change

What You Will Learn

- How to build software that anticipates and accommodates change without over-engineering
- Abstraction boundaries: where to put the seams in your code
- Configuration over code: externalizing decisions that change frequently
- Feature flags: deploying incomplete features safely
- The Strategy pattern: swapping behavior at runtime
- Event-driven architecture: decoupling producers from consumers
- How to balance flexibility with simplicity (the YAGNI tension)

Why This Chapter Matters

Here is an uncomfortable truth about software: the only thing you can predict about requirements is that they will change. The payment provider will switch. The business rules will evolve. A new regulation will arrive. A feature you built last month will need to work differently next quarter.

The question is not whether change will happen. The question is: when change arrives, will it take you two hours or two weeks?

Code that is designed for change has clear seams -- places where you can swap, extend, or modify behavior without rewriting everything. Code that is not designed for change is brittle: one change causes a cascade of modifications across dozens of files.

But there is a trap. Some developers, fearing future change, add abstraction layers for every possible scenario. They build frameworks instead of features. They add flexibility that nobody ever uses. This is over-engineering, and it is just as harmful as under-engineering.

This chapter teaches you to find the balance: build seams where change is likely, keep things simple where it is not.

Abstraction Boundaries: Where to Put the Seams

An abstraction boundary is a point in your code where you can change the implementation without affecting the rest of the system. The most common tool for creating abstraction boundaries is the interface.

```
ABSTRACTION BOUNDARIES:

  Without boundaries (rigid):

  +------------+     +------------+     +------------+
  | Controller |---->| Service    |---->| MySQL      |
  |            |     |            |     | Repository |
  +------------+     +------------+     +------------+

  Changing MySQL to PostgreSQL requires changing Service.

  With boundaries (flexible):

  +------------+     +------------+     +----------------+
  | Controller |---->| Service    |---->| <<interface>>  |
  |            |     |            |     | Repository     |
  +------------+     +------------+     +-------+--------+
                                                |
                                          +--------+--------+
                                          |                 |
                                     +----+------+    +-----+------+
                                     | MySQL     |    | Postgres   |
                                     | Repository|    | Repository |
                                     +-----------+    +------------+

  Changing MySQL to PostgreSQL requires only:
  1. Writing the new implementation
  2. Changing the wiring in the Composition Root
  Service code does not change at all.
```

Where to Place Boundaries

Not every dependency needs an abstraction boundary. Place them where change is likely:

```
WHERE TO ADD ABSTRACTION BOUNDARIES:

  HIGH likelihood of change (ADD boundary):
  +-----------------------------------------+
  | - Database technology                   |
  | - External APIs (payment, shipping, etc) |
  | - Notification channels (email, SMS)    |
  | - File storage (local, S3, Azure Blob)  |
  | - Business rules that vary by client    |
  | - Authentication providers              |
  +-----------------------------------------+

  LOW likelihood of change (SKIP boundary):
  +-----------------------------------------+
  | - String utility functions              |
  | - Math calculations                     |
  | - Data structures (lists, maps)         |
  | - Language standard library             |
  | - Internal value objects                |
  +-----------------------------------------+

  Rule of thumb: If you can imagine the implementation
  changing within the next year, add a boundary.
  If not, keep it simple.
```

Java Example: Strategic Abstraction

```java
// BOUNDARY: Notification channel (likely to change)
// Today it is email. Tomorrow it might be SMS, push, or Slack.
public interface NotificationChannel {
    void send(String recipient, String subject, String body);
}

public class EmailNotification implements NotificationChannel {
    private final SmtpClient smtpClient;

    public EmailNotification(SmtpClient smtpClient) {
        this.smtpClient = smtpClient;
    }

    @Override
    public void send(String recipient, String subject, String body) {
        smtpClient.sendEmail(recipient, subject, body);
    }
}

public class SlackNotification implements NotificationChannel {
    private final SlackWebhook webhook;

    public SlackNotification(SlackWebhook webhook) {
```

```
        this.webhook = webhook;
    }

    @Override
    public void send(String recipient, String subject, String body) {
        webhook.postMessage("#" + recipient, subject + "\n" + body);
    }
}

// The service does not care HOW notifications are sent
public class OrderService {
    private final NotificationChannel notifications;

    public OrderService(NotificationChannel notifications) {
        this.notifications = notifications;
    }

    public void placeOrder(Order order) {
        // ... business logic ...
        notifications.send(
            order.getCustomerEmail(),
            "Order Confirmed",
            "Your order " + order.getId() + " has been placed."
        );
    }
}
```

```python
# BOUNDARY: Notification channel (likely to change)
from abc import ABC, abstractmethod

class NotificationChannel(ABC):
    @abstractmethod
    def send(self, recipient: str, subject: str, body: str) -> None:
        pass

class EmailNotification(NotificationChannel):
    def __init__(self, smtp_client):
        self.smtp_client = smtp_client

    def send(self, recipient: str, subject: str, body: str) -> None:
        self.smtp_client.send_email(recipient, subject, body)

class SlackNotification(NotificationChannel):
    def __init__(self, webhook):
        self.webhook = webhook

    def send(self, recipient: str, subject: str, body: str) -> None:
        self.webhook.post_message(f"#{recipient}", f"{subject}\n{body}")

# The service does not care HOW notifications are sent
class OrderService:
    def __init__(self, notifications: NotificationChannel):
```

```
        self.notifications = notifications

    def place_order(self, order) -> None:
        # ... business logic ...
        self.notifications.send(
            order.customer_email,
            "Order Confirmed",
            f"Your order {order.id} has been placed.",
        )
```

Configuration Over Code

Some decisions change frequently but do not require new logic -- just new values. These belong in configuration, not code.

```
DECISIONS THAT BELONG IN CONFIGURATION:

  +-----------------------------------------+
  | - Database connection strings           |
  | - API keys and endpoints                |
  | - Feature flag states (on/off)          |
  | - Retry counts and timeout values       |
  | - Rate limits                           |
  | - Logging levels                        |
  | - Email templates                       |
  | - Tax rates by region                   |
  | - Currency formatting rules             |
  +-----------------------------------------+

  DECISIONS THAT BELONG IN CODE:

  +-----------------------------------------+
  | - Business logic and rules              |
  | - Data validation                       |
  | - Algorithms                            |
  | - Error handling strategies             |
  | - Security checks                       |
  +-----------------------------------------+
```

Before and After

```java
// BEFORE: Values hard-coded (bad)
public class RetryPolicy {
    public void executeWithRetry(Runnable action) {
        int maxRetries = 3;           // Hard-coded
        long delayMs = 1000;          // Hard-coded
        for (int i = 0; i <= maxRetries; i++) {
            try {
                action.run();
                return;
            } catch (Exception e) {
                if (i == maxRetries) throw e;
                Thread.sleep(delayMs * (i + 1));  // Hard-coded backoff
            }
        }
    }
}

// AFTER: Values from configuration (good)
public class RetryPolicy {
    private final int maxRetries;
    private final long baseDelayMs;

    public RetryPolicy(RetryConfig config) {
        this.maxRetries = config.getMaxRetries();
        this.baseDelayMs = config.getBaseDelayMs();
    }

    public void executeWithRetry(Runnable action) {
        for (int i = 0; i <= maxRetries; i++) {
            try {
                action.run();
                return;
            } catch (Exception e) {
                if (i == maxRetries) throw e;
                Thread.sleep(baseDelayMs * (i + 1));
            }
        }
    }
}
```

```python
# BEFORE: Values hard-coded (bad)
class RetryPolicy:
    def execute_with_retry(self, action):
        max_retries = 3          # Hard-coded
        base_delay = 1.0         # Hard-coded
        for attempt in range(max_retries + 1):
            try:
                return action()
            except Exception:
                if attempt == max_retries:
                    raise
                time.sleep(base_delay * (attempt + 1))
```

```
# AFTER: Values from configuration (good)
class RetryPolicy:
    def __init__(self, max_retries: int = 3, base_delay: float = 1.0):
        self.max_retries = max_retries
        self.base_delay = base_delay

    def execute_with_retry(self, action):
        for attempt in range(self.max_retries + 1):
            try:
                return action()
            except Exception:
                if attempt == self.max_retries:
                    raise
                time.sleep(self.base_delay * (attempt + 1))
```

Feature Flags: Deploy Without Risk

Feature flags let you deploy code that is not yet ready for all users. You can enable features for specific users, gradually roll them out, or instantly disable them if something goes wrong.

```
FEATURE FLAGS FLOW:

  Code is deployed with the feature HIDDEN behind a flag:

  +-------------------------------------------+
  |  if feature_flags.is_enabled("new_checkout"):
  |      show_new_checkout()
  |  else:
  |      show_old_checkout()
  +-------------------------------------------+

  Flag states:
  +------------+--------------------+----------------------------+
  | State      | Who sees it        | Use case                   |
  +------------+--------------------+----------------------------+
  | OFF        | Nobody             | Not ready yet              |
  | INTERNAL   | Team only          | Internal testing           |
  | BETA       | Opt-in users       | Early feedback             |
  | PERCENTAGE | 10% of users       | Gradual rollout            |
  | ON         | Everyone           | Fully launched             |
  +------------+--------------------+----------------------------+
```

Java Example

```java
// Simple feature flag implementation
public class FeatureFlags {
    private final Map<String, Boolean> flags;

    public FeatureFlags(Map<String, Boolean> flags) {
        this.flags = flags;
    }

    public boolean isEnabled(String featureName) {
        return flags.getOrDefault(featureName, false);
    }
}

// Usage in service code
public class CheckoutService {
    private final FeatureFlags featureFlags;
    private final OldCheckoutProcess oldProcess;
    private final NewCheckoutProcess newProcess;

    public CheckoutService(FeatureFlags featureFlags,
                           OldCheckoutProcess oldProcess,
                           NewCheckoutProcess newProcess) {
        this.featureFlags = featureFlags;
        this.oldProcess = oldProcess;
        this.newProcess = newProcess;
    }

    public CheckoutResult checkout(Cart cart) {
        if (featureFlags.isEnabled("new_checkout_flow")) {
            return newProcess.execute(cart);
        }
        return oldProcess.execute(cart);
    }
}
```

```python
# Simple feature flag implementation
class FeatureFlags:
    def __init__(self, flags: dict[str, bool]):
        self._flags = flags

    def is_enabled(self, feature_name: str) -> bool:
        return self._flags.get(feature_name, False)

    @classmethod
    def from_config(cls, config_path: str) -> "FeatureFlags":
        import json
        with open(config_path) as f:
            return cls(json.load(f))

# Usage in service code
class CheckoutService:
    def __init__(self, feature_flags: FeatureFlags,
```

```
                 old_process: OldCheckoutProcess,
                 new_process: NewCheckoutProcess):
        self.feature_flags = feature_flags
        self.old_process = old_process
        self.new_process = new_process

    def checkout(self, cart) -> CheckoutResult:
        if self.feature_flags.is_enabled("new_checkout_flow"):
            return self.new_process.execute(cart)
        return self.old_process.execute(cart)
```

Feature Flag Best Practices

1. Remove flags after full rollout. Dead flags become technical debt.
2. Keep flag checks at the boundary (controllers, services), not deep in business logic.
3. Test both paths. Every flag creates two code paths. Test them both.
4. Use a naming convention. Prefix flags with the team or feature area.

The Strategy Pattern: Swapping Behavior at Runtime

The Strategy pattern encapsulates an algorithm behind an interface, letting you swap algorithms without changing the code that uses them.

```
STRATEGY PATTERN:

  +------------------+       +--------------------+
  |  PricingService  |------>| <<interface>>      |
  |                  |       | DiscountStrategy   |
  |  applyDiscount() |       +---------+----------+
  +------------------+                 |
                              +-------+-------+
                              |       |       |
                         +----+--+ +--+---+ +-+---------+
                         |Percent| |Fixed | |Tiered     |
                         |Discount|Discount|Discount    |
                         +-------+ +------+ +----------+

  The PricingService does not know which discount
  strategy it uses. It just calls strategy.apply(price).
```

Java Example

```java
// Strategy interface
public interface DiscountStrategy {
    Money apply(Money originalPrice, Customer customer);
}

// Concrete strategies
public class PercentageDiscount implements DiscountStrategy {
    private final double percentage;

    public PercentageDiscount(double percentage) {
        this.percentage = percentage;
    }

    @Override
    public Money apply(Money originalPrice, Customer customer) {
        return originalPrice.multiply(1 - percentage / 100);
    }
}

public class TieredDiscount implements DiscountStrategy {
    @Override
    public Money apply(Money originalPrice, Customer customer) {
        double discount = switch (customer.getLoyaltyTier()) {
            case GOLD     -> 0.15;
            case SILVER   -> 0.10;
            case BRONZE   -> 0.05;
            default       -> 0.0;
        };
        return originalPrice.multiply(1 - discount);
    }
}

public class NoDiscount implements DiscountStrategy {
    @Override
    public Money apply(Money originalPrice, Customer customer) {
        return originalPrice;
    }
}

// The service uses whichever strategy is injected
public class PricingService {
    private final DiscountStrategy discountStrategy;

    public PricingService(DiscountStrategy discountStrategy) {
        this.discountStrategy = discountStrategy;
    }

    public Money calculateFinalPrice(Money basePrice, Customer customer) {
        return discountStrategy.apply(basePrice, customer);
    }
}
```

```
// Switching strategies requires only changing the wiring:
// new PricingService(new PercentageDiscount(20))
// new PricingService(new TieredDiscount())
// new PricingService(new NoDiscount())
```

```python
# Strategy using Python's Protocol (duck typing)
from typing import Protocol

class DiscountStrategy(Protocol):
    def apply(self, original_price: float, customer) -> float:
        ...

class PercentageDiscount:
    def __init__(self, percentage: float):
        self.percentage = percentage

    def apply(self, original_price: float, customer) -> float:
        return original_price * (1 - self.percentage / 100)

class TieredDiscount:
    def apply(self, original_price: float, customer) -> float:
        tier_discounts = {
            "gold": 0.15,
            "silver": 0.10,
            "bronze": 0.05,
        }
        discount = tier_discounts.get(customer.loyalty_tier, 0.0)
        return original_price * (1 - discount)

class NoDiscount:
    def apply(self, original_price: float, customer) -> float:
        return original_price

# The service uses whichever strategy is injected
class PricingService:
    def __init__(self, discount_strategy: DiscountStrategy):
        self.discount_strategy = discount_strategy

    def calculate_final_price(self, base_price: float, customer) -> float:
        return self.discount_strategy.apply(base_price, customer)

# Python bonus: you can also use simple functions as strategies
def holiday_discount(original_price: float, customer) -> float:
    return original_price * 0.75

# pricing = PricingService(holiday_discount)  # Functions work too!
```

Event-Driven Architecture: Decoupling Producers from Consumers

Events decouple the code that does something from the code that reacts to it. The producer publishes an event. Zero, one, or many consumers react to it. The producer does not know or care who is listening.

```
DIRECT COUPLING vs EVENT-DRIVEN:

  DIRECT COUPLING (rigid):

  +---------------+     +------------------+
  | OrderService  |---->| InventoryService |
  |               |---->| EmailService     |
  |               |---->| AnalyticsService |
  |               |---->| LoyaltyService   |
  +---------------+     +------------------+

  Adding a new reaction to "order placed" requires
  modifying OrderService. Open/Closed Principle violated.

  EVENT-DRIVEN (flexible):

  +---------------+     +------------+     +------------------+
  | OrderService  |---->| Event Bus  |---->| InventoryHandler |
  |               |     |            |---->| EmailHandler     |
  | publishes:    |     |            |---->| AnalyticsHandler |
  | OrderPlaced   |     |            |---->| LoyaltyHandler   |
  +---------------+     +------------+     +------------------+

  Adding a new reaction means adding a new handler.
  OrderService does not change. Open/Closed Principle upheld.
```

Java Example

```java
// Simple event system
public interface DomainEvent {
    Instant occurredAt();
}

public record OrderPlacedEvent(
    String orderId,
    String customerId,
    Money total,
    Instant occurredAt
) implements DomainEvent {}
```

```
public interface EventHandler<T extends DomainEvent> {
    void handle(T event);
}

public class EventBus {
    private final Map<Class<?>, List<EventHandler<?>>> handlers = new HashMap<>();

    public <T extends DomainEvent> void subscribe(
            Class<T> eventType, EventHandler<T> handler) {
        handlers.computeIfAbsent(eventType, k -> new ArrayList<>()).add(handler);
    }

    @SuppressWarnings("unchecked")
    public void publish(DomainEvent event) {
        List<EventHandler<?>> eventHandlers =
            handlers.getOrDefault(event.getClass(), List.of());
        for (EventHandler handler : eventHandlers) {
            handler.handle(event);
        }
    }
}

// OrderService only publishes -- it does not know who listens
public class OrderService {
    private final OrderRepository repository;
    private final EventBus eventBus;

    public OrderService(OrderRepository repository, EventBus eventBus) {
        this.repository = repository;
        this.eventBus = eventBus;
    }

    public void placeOrder(Order order) {
        order.place();
        repository.save(order);
        eventBus.publish(new OrderPlacedEvent(
            order.getId(), order.getCustomerId(),
            order.getTotal(), Instant.now()
        ));
    }
}

// Handlers are registered separately
public class InventoryReservationHandler implements EventHandler<OrderPlacedEvent> {
    @Override
    public void handle(OrderPlacedEvent event) {
        // Reserve inventory for the order
    }
}

public class OrderConfirmationEmailHandler implements EventHandler<OrderPlacedEvent> {
    @Override
    public void handle(OrderPlacedEvent event) {
        // Send confirmation email
```

```
    }
}
```

```python
# Simple event system in Python
from dataclasses import dataclass, field
from datetime import datetime
from typing import Callable
from collections import defaultdict

@dataclass(frozen=True)
class OrderPlacedEvent:
    order_id: str
    customer_id: str
    total: float
    occurred_at: datetime = field(default_factory=datetime.utcnow)

class EventBus:
    def __init__(self):
        self._handlers: dict[type, list[Callable]] = defaultdict(list)

    def subscribe(self, event_type: type, handler: Callable) -> None:
        self._handlers[event_type].append(handler)

    def publish(self, event) -> None:
        for handler in self._handlers.get(type(event), []):
            handler(event)

# OrderService only publishes -- it does not know who listens
class OrderService:
    def __init__(self, repository, event_bus: EventBus):
        self.repository = repository
        self.event_bus = event_bus

    def place_order(self, order) -> None:
        order.place()
        self.repository.save(order)
        self.event_bus.publish(OrderPlacedEvent(
            order_id=order.id,
            customer_id=order.customer_id,
            total=order.total,
        ))

# Handlers are registered separately
def reserve_inventory(event: OrderPlacedEvent) -> None:
    # Reserve inventory for the order
    ...

def send_confirmation_email(event: OrderPlacedEvent) -> None:
    # Send confirmation email
    ...

# Wiring in the Composition Root
```

```
event_bus = EventBus()
event_bus.subscribe(OrderPlacedEvent, reserve_inventory)
event_bus.subscribe(OrderPlacedEvent, send_confirmation_email)
```

Balancing Flexibility vs Simplicity

The biggest danger in designing for change is over-engineering -- adding flexibility you never use. Here is a framework for deciding:

```
THE FLEXIBILITY DECISION FRAMEWORK:

  Ask yourself these questions:

  1. Has this part of the system changed before?
     YES --> Add an abstraction boundary
     NO  --> Continue to question 2

  2. Is the stakeholder actively discussing changes to this area?
     YES --> Add an abstraction boundary
     NO  --> Continue to question 3

  3. Are there multiple known implementations (e.g., MySQL AND Postgres)?
     YES --> Add an abstraction boundary
     NO  --> Keep it simple. Add the boundary WHEN you need it.

  +-----------------------------------------------------+
  |                                                     |
  |  "You Ain't Gonna Need It" (YAGNI) says:            |
  |                                                     |
  |  Do not add flexibility for hypothetical future     |
  |  needs. Add it when you actually need it.            |
  |                                                     |
  |  But ALSO:                                          |
  |                                                     |
  |  Write code that is EASY to add flexibility to      |
  |  later. Small classes, single responsibility,       |
  |  clean interfaces.                                  |
  |                                                     |
  +-----------------------------------------------------+
```

The Spectrum of Flexibility

```
TOO RIGID              JUST RIGHT              TOO FLEXIBLE
(hard to change)       (easy to change)        (hard to understand)

+------------------+   +------------------+   +------------------+
| Hard-coded       |   | Interface for    |   | 5 layers of      |
| everything.      |   | things that      |   | abstraction.     |
| One massive      |   | actually change. |   | Plugin system    |
| class.           |   | Config for       |   | for everything.  |
| No interfaces.   |   | values that      |   | XML config for   |
| No config.       |   | vary.            |   | every decision.  |
| No patterns.     |   | Simple patterns  |   | Factory of       |
|                  |   | where needed.    |   | factory of       |
|                  |   |                  |   | abstract factory.|
+------------------+   +------------------+   +------------------+

  "I can't change     "Change is            "I can't understand
   anything without    straightforward        what this code
   breaking things."   and localized."        actually does."
```

Practical Rules

```java
// OVER-ENGINEERED: Abstract factory for something that never changes
public interface StringFormatterFactory {
    StringFormatter createFormatter(String type);
}
public interface StringFormatter {
    String format(String input);
}
public class UpperCaseFormatterFactory implements StringFormatterFactory { ... }
// This is just: input.toUpperCase()!

// JUST RIGHT: Direct call for stable operations
public String formatName(String name) {
    return name.trim().toUpperCase();
}

// JUST RIGHT: Interface for something that actually varies
public interface TaxCalculator {
    Money calculateTax(Money subtotal, Address address);
}
// Tax rules vary by country, state, and change regularly.
// An interface here is justified.
```

Common Mistakes

Mistake 1: Abstracting Everything

Creating interfaces for every class, even when there is only one implementation and no foreseeable need for another.

Fix: Apply the Rule of Three -- add an abstraction when you have or foresee at least two implementations.

Mistake 2: Feature Flags That Never Get Removed

Deploying feature flags and forgetting to clean them up. Over time, the code fills with dead branches.

Fix: Set an expiration date for every flag. Add a tech debt ticket when the flag is created. Review flags monthly.

Mistake 3: Configuration for Logic

Putting business logic in configuration files (XML workflows, JSON rule engines) instead of code. Configuration should drive values, not behavior.

Fix: Keep business logic in code where it can be tested, debugged, and refactored. Use configuration for values that change without new logic.

Mistake 4: Events Everywhere

Using events for every method call, even within a single module. This makes the code flow impossible to follow.

Fix: Use events for cross-module or cross-context communication. Within a module, direct method calls are clearer.

Mistake 5: Premature Optimization for Change

Designing for changes that never come. "What if we need to support five databases?" when you only ever use one.

Fix: Design for the changes you know about. Make the code clean enough that adding flexibility later is easy.

Best Practices

1. Place abstraction boundaries at integration points. Database, external APIs, messaging systems -- these are where change is most likely.
2. Use configuration for values, code for logic. Connection strings, retry counts, and feature flags go in config. Business rules go in code.
3. Remove feature flags promptly. Set a cleanup deadline when creating any flag.
4. Use the Strategy pattern when behavior varies. Pricing rules, tax calculations, and notification channels are natural fits.
5. Use events for cross-boundary communication. Events decouple producers from consumers and follow the Open/Closed Principle.
6. Follow YAGNI, but write clean code. Do not add flexibility you do not need. But write code that is easy to make flexible later.
7. Apply the Rule of Three. If you have done something similar three times, it is time to abstract. Before that, duplication may be cheaper than the wrong abstraction.
8. Prefer composition over inheritance. Injecting strategies and collaborators gives you more flexibility than deep class hierarchies.

Quick Summary

```
DESIGN FOR CHANGE AT A GLANCE:

  Tool                    When to use it
  +--------------------+-------------------------------------+
  | Abstraction        | At integration points where        |
  | Boundaries         | implementations are likely to      |
  |                    | change (DB, APIs, messaging).      |
  +--------------------+-------------------------------------+
  | Configuration      | For values that change without     |
  | Over Code          | new logic (timeouts, URLs, keys).  |
  +--------------------+-------------------------------------+
  | Feature Flags      | To deploy incomplete features      |
  |                    | safely and roll out gradually.     |
  +--------------------+-------------------------------------+
  | Strategy Pattern   | When algorithm or behavior needs   |
  |                    | to vary (pricing, discounts, tax). |
  +--------------------+-------------------------------------+
  | Event-Driven       | For cross-module reactions where   |
  |                    | the producer should not know       |
  |                    | about consumers.                   |
  +--------------------+-------------------------------------+

  The golden rule: Make change easy, but do not pay for
  flexibility you do not need yet.
```

Key Points

- Abstraction boundaries (interfaces) create seams where you can swap implementations without changing calling code.
- Place boundaries at integration points where change is most likely: databases, external APIs, notification channels.
- Use configuration for values that change frequently (timeouts, URLs, keys) and code for business logic.
- Feature flags let you deploy incomplete code safely, but remove them promptly after full rollout.
- The Strategy pattern encapsulates varying algorithms behind an interface for easy swapping.
- Event-driven architecture decouples producers from consumers, making it easy to add new reactions without modifying existing code.
- Balance flexibility with simplicity: follow YAGNI, but write clean code that is easy to extend later.
- The Rule of Three helps decide when to abstract: wait for two or three similar cases before generalizing.

Practice Questions

1. You are building a reporting system that currently generates PDF reports. The product team mentions that CSV and Excel exports might be needed in the future. Should you create an abstraction boundary now, or wait? Justify your answer.
2. A developer on your team has created interfaces for every class in the project, even for utility classes like `StringHelper` and `DateFormatter` that have exactly one implementation. What would you tell them?
3. Your application has 47 feature flags. Only 12 are actively being tested. The rest were for features that shipped months ago. What is the problem, and how would you address it?
4. Compare the Strategy pattern with using `if/else` chains for handling different discount types. When would you choose one approach over the other?
5. An event-driven system publishes 200 different event types. Debugging is difficult because you cannot trace the flow of a single operation. What went wrong, and how

would you fix it?

Exercises

Exercise 1: Add an Abstraction Boundary

You have a `UserService` that directly calls `SmtpEmailClient.send()` to send welcome emails. Refactor it to use an abstraction boundary so that the notification method (email, SMS, push notification) can be swapped without changing `UserService`.

Exercise 2: Implement Feature Flags

Build a simple feature flag system that:

- Loads flag states from a JSON file
- Supports boolean flags (on/off)
- Supports percentage-based rollout (enabled for X% of users based on user ID hash)
- Can be updated at runtime without restarting the application

Exercise 3: Event-Driven Refactoring

You have an `OrderService.placeOrder()` method that directly calls `inventoryService.reserve()`, `emailService.sendConfirmation()`, `analyticsService.trackPurchase()`, and `loyaltyService.addPoints()`. Refactor this to use an event-driven approach. What are the trade-offs?

What Is Next?

Now that you know how to design code that accommodates change gracefully, the next chapter focuses on API Design Principles -- how to design interfaces that other developers love to use. You will learn about the principle of least surprise, consistent naming, backward compatibility, versioning, and building APIs that are hard to misuse.

Chapter 26: API Design Principles

What You Will Learn

- The Principle of Least Surprise: design APIs that behave as users expect
- Consistent naming conventions that make APIs intuitive
- Backward compatibility strategies that protect existing users
- API versioning approaches and their trade-offs
- How to design clear, helpful error responses
- Pagination patterns for large data sets
- Idempotency: making APIs safe to retry
- A practical API review checklist

Why This Chapter Matters

An API is a contract. Whether it is a REST endpoint, a library interface, or a class's public methods, an API is a promise you make to other developers: "Call me this way, and I will do this for you."

A well-designed API is a joy to use. You read the method name, and you know what it does. You pass the obvious parameters, and it works. Error messages tell you exactly what went wrong and how to fix it. You can call it twice by accident, and nothing bad happens.

A poorly-designed API is a nightmare. You guess what the parameters mean. You discover undocumented side effects. Error messages say "Error 500" and nothing else. Calling an endpoint twice creates duplicate records that haunt you forever.

The difference between these two is not talent -- it is a set of principles that anyone can learn and apply. This chapter teaches those principles.

The Principle of Least Surprise

The most important rule in API design: an API should behave the way its users expect it to behave. If someone reads the name of your method or endpoint and forms an expectation about what it does, the actual behavior should match that expectation.

```
PRINCIPLE OF LEAST SURPRISE:

  When a developer reads your API signature,
  their first guess about what it does should be correct.

  +-----------------------------+---------------------------+
  | SURPRISING (bad)            | UNSURPRISING (good)       |
  +-----------------------------+---------------------------+
  | user.save()                 | user.save()               |
  |   --> also sends email      |   --> only saves           |
  |   --> also logs analytics   |                           |
  |   --> also clears cache     | email_service.send(user)  |
  |                             |   --> only sends email     |
  +-----------------------------+---------------------------+
  | list.sort()                 | list.sort()               |
  |   --> returns new list      |   --> sorts in place       |
  |   --> original unchanged    |                           |
  |                             | sorted(list)              |
  |                             |   --> returns new list     |
  +-----------------------------+---------------------------+
  | cart.getTotal()             | cart.getTotal()            |
  |   --> applies discount      |   --> returns current total|
  |   --> modifies cart state   |                           |
  |                             | cart.applyDiscount(code)  |
  |                             |   --> modifies cart state  |
  +-----------------------------+---------------------------+
```

Rules for Least Surprise

1. Methods named `get` should not modify state. Getters should be read-only.
2. Methods named `save` should only save. Do not add side effects.
3. Boolean parameters should not change behavior drastically. Instead, use separate methods.
4. Error behavior should be consistent. If one method throws on invalid input, all methods should.

Java -- Before and After

```java
// SURPRISING: save() does three hidden things
public class UserRepository {
    public void save(User user) {
        database.insert(user);
        emailService.sendWelcome(user);     // Surprise!
        analyticsService.trackSignup(user); // Surprise!
    }
}

// UNSURPRISING: each method does one thing
public class UserRepository {
    public void save(User user) {
        database.insert(user);  // Just saves. Nothing else.
    }
}

public class UserRegistrationService {
    public void register(User user) {
        userRepository.save(user);           // Save
        emailService.sendWelcome(user);      // Explicit: send email
        analyticsService.trackSignup(user);  // Explicit: track
    }
}
```

```python
# SURPRISING: save() does three hidden things
class UserRepository:
    def save(self, user):
        self.database.insert(user)
        self.email_service.send_welcome(user)       # Surprise!
        self.analytics_service.track_signup(user)    # Surprise!

# UNSURPRISING: each method does one thing
class UserRepository:
    def save(self, user):
        self.database.insert(user)  # Just saves. Nothing else.

class UserRegistrationService:
    def register(self, user):
        self.user_repository.save(user)              # Save
        self.email_service.send_welcome(user)         # Explicit
        self.analytics_service.track_signup(user)     # Explicit
```

Consistent Naming

Consistency makes APIs predictable. If you call it `getUser` in one place, do not call it `fetchCustomer` in another. Pick a convention and stick to it.

Naming Conventions for REST APIs

```
REST API NAMING CONVENTIONS:

  Resource naming:
  +---------------------------+---------------------------+
  | BAD                       | GOOD                      |
  +---------------------------+---------------------------+
  | /getUsers                 | GET /users                |
  | /createUser               | POST /users               |
  | /deleteUser/123           | DELETE /users/123          |
  | /user_list                | GET /users                |
  | /Users                    | GET /users  (lowercase)   |
  +---------------------------+---------------------------+

  Use nouns for resources, HTTP verbs for actions:
  +---------------------------+---------------------------+
  | Action                    | Endpoint                  |
  +---------------------------+---------------------------+
  | List all orders           | GET    /orders            |
  | Get one order             | GET    /orders/{id}       |
  | Create an order           | POST   /orders            |
  | Update an order           | PUT    /orders/{id}       |
  | Partially update          | PATCH  /orders/{id}       |
  | Delete an order           | DELETE /orders/{id}       |
  +---------------------------+---------------------------+

  Sub-resources:
  +---------------------------+---------------------------+
  | Action                    | Endpoint                  |
  +---------------------------+---------------------------+
  | List items in order       | GET /orders/{id}/items    |
  | Add item to order         | POST /orders/{id}/items   |
  +---------------------------+---------------------------+

  Filtering, sorting, pagination via query parameters:
  GET /orders?status=pending&sort=created_at&page=2
```

Naming Conventions for Code APIs

```java
// CONSISTENT NAMING in Java
// Pick a verb convention and stick to it

// For queries (read-only):
User findById(String id);          // Returns User or throws
Optional<User> findByEmail(String email);  // Returns Optional
List<User> findAllByRole(Role role);       // Returns list
boolean existsById(String id);             // Returns boolean
```

```
// For commands (write):
void save(User user);               // Persist
void delete(User user);             // Remove
void update(User user);             // Modify existing

// Do NOT mix:
//   getUser(), fetchUser(), loadUser(), retrieveUser()
// Pick ONE verb and use it everywhere.
```

```python
# CONSISTENT NAMING in Python
# Pick a convention and stick to it

class UserRepository:
    def find_by_id(self, user_id: str) -> User: ...
    def find_by_email(self, email: str) -> Optional[User]: ...
    def find_all_by_role(self, role: str) -> list[User]: ...
    def exists_by_id(self, user_id: str) -> bool: ...

    def save(self, user: User) -> None: ...
    def delete(self, user: User) -> None: ...
    def update(self, user: User) -> None: ...

    # Do NOT mix: get_user(), fetch_user(), load_user()
    # Pick ONE verb and use it everywhere.
```

Backward Compatibility

Once you publish an API, people depend on it. Changing it in ways that break existing users is one of the most expensive mistakes you can make.

```
BACKWARD COMPATIBILITY RULES:

  SAFE CHANGES (backward compatible):
  +-----------------------------------------+
  | - Adding a new optional field to response|
  | - Adding a new endpoint                  |
  | - Adding a new optional query parameter  |
  | - Adding a new HTTP method to a resource |
  | - Widening an input type (int -> long)   |
  +-----------------------------------------+

  BREAKING CHANGES (not backward compatible):
  +-----------------------------------------+
  | - Removing a field from response         |
  | - Renaming a field                       |
```

```
  | - Changing a field's type                |
  | - Removing an endpoint                   |
  | - Adding a required field to request     |
  | - Changing error codes or formats        |
  | - Narrowing an input type (long -> int)  |
  +------------------------------------------+
```

Strategies for Evolving APIs Without Breaking Them

```java
// STRATEGY 1: Add optional fields, never remove required ones
// Version 1 response:
{
    "id": "123",
    "name": "Jane Smith",
    "email": "jane@example.com"
}

// Version 1.1 response (backward compatible):
{
    "id": "123",
    "name": "Jane Smith",
    "email": "jane@example.com",
    "phone": "+1-555-0123"      // NEW optional field -- old clients ignore it
}


// STRATEGY 2: Use @JsonIgnoreProperties to handle unknown fields
@JsonIgnoreProperties(ignoreUnknown = true)
public class UserResponse {
    private String id;
    private String name;
    private String email;
    // Old clients will not break if new fields are added
}
```

```python
# STRATEGY: Accept unknown fields gracefully
from dataclasses import dataclass, field
from typing import Any

@dataclass
class UserResponse:
    id: str
    name: str
    email: str
    # New fields have defaults so old code still works
    phone: str = ""
    extra: dict[str, Any] = field(default_factory=dict)
```

```
    @classmethod
    def from_dict(cls, data: dict) -> "UserResponse":
        known_fields = {"id", "name", "email", "phone"}
        extra = {k: v for k, v in data.items() if k not in known_fields}
        return cls(
            id=data["id"],
            name=data["name"],
            email=data["email"],
            phone=data.get("phone", ""),
            extra=extra,
        )
```

API Versioning

When breaking changes are unavoidable, use versioning to protect existing users.

```
API VERSIONING STRATEGIES:

  1. URL PATH VERSIONING (most common):
     GET /api/v1/users
     GET /api/v2/users

     Pros: Simple, visible, easy to route
     Cons: URL changes for every version

  2. HEADER VERSIONING:
     GET /api/users
     Accept: application/vnd.myapp.v2+json

     Pros: Clean URLs
     Cons: Hidden, harder to test in browser

  3. QUERY PARAMETER VERSIONING:
     GET /api/users?version=2

     Pros: Easy to add
     Cons: Optional params can be forgotten

  RECOMMENDATION: Use URL path versioning.
  It is the most transparent and widely understood.

  +-----------------------------------------+
  |  VERSIONING TIMELINE:                   |
  |                                         |
  |  v1 launched -----> v2 launched         |
  |  |                  |                   |
  |  |  v1 supported   |  v1 deprecated     |
  |  |  v2 available   |  v2 supported      |
```

```
  |  |                  |                    |
  |  |                  |  v1 sunset date    |
  |  |                  |  announced         |
  |  |                  |                    |
  |  |                  |  v1 removed        |
  +-----------------------------------------+
```

Java Example: Supporting Multiple Versions

```java
// Version 1: Original response format
@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 {

    @GetMapping("/{id}")
    public UserResponseV1 getUser(@PathVariable String id) {
        User user = userService.findById(id);
        return new UserResponseV1(user.getId(), user.getName(), user.getEmail());
    }
}

// Version 2: Enhanced response format
@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 {

    @GetMapping("/{id}")
    public UserResponseV2 getUser(@PathVariable String id) {
        User user = userService.findById(id);
        return new UserResponseV2(
            user.getId(),
            user.getFullName(),          // Changed: name -> fullName
            user.getEmail(),
            user.getPhone(),             // Added
            user.getCreatedAt()          // Added
        );
    }
}
```

```python
# Version 1: Original response
@app.get("/api/v1/users/{user_id}")
def get_user_v1(user_id: str):
    user = user_service.find_by_id(user_id)
    return {
        "id": user.id,
        "name": user.name,
        "email": user.email,
    }
```

```
# Version 2: Enhanced response
@app.get("/api/v2/users/{user_id}")
def get_user_v2(user_id: str):
    user = user_service.find_by_id(user_id)
    return {
        "id": user.id,
        "full_name": user.full_name,     # Changed
        "email": user.email,
        "phone": user.phone,             # Added
        "created_at": user.created_at,   # Added
    }
```

Error Responses

Good error responses tell the user what went wrong, why it went wrong, and how to fix it.

```
BAD vs GOOD ERROR RESPONSES:

  BAD:
  {
      "error": "Bad Request"
  }

  What went wrong? No idea. How to fix it? Good luck.

  BAD:
  {
      "code": 422,
      "message": "Validation failed"
  }

  Which field? What was wrong with it?

  GOOD:
  {
      "error": {
          "code": "VALIDATION_ERROR",
          "message": "The request contains invalid fields.",
          "details": [
              {
                  "field": "email",
                  "issue": "Must be a valid email address.",
                  "received": "not-an-email"
              },
              {
                  "field": "age",
                  "issue": "Must be between 0 and 150.",
```

```
                "received": -5
            }
        ],
        "documentation": "https://api.example.com/docs/errors#VALIDATION_ERROR"
    }
}
```

Standard Error Response Structure

```java
// A consistent error response structure
public class ApiError {
    private final String code;           // Machine-readable error code
    private final String message;        // Human-readable summary
    private final List<FieldError> details;  // Specific field errors
    private final String documentation;  // Link to docs

    // ... constructor and getters
}

public class FieldError {
    private final String field;
    private final String issue;
    private final Object received;

    // ... constructor and getters
}

// Controller advice for consistent error handling
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ValidationException.class)
    @ResponseStatus(HttpStatus.UNPROCESSABLE_ENTITY)
    public ApiError handleValidation(ValidationException e) {
        return new ApiError(
            "VALIDATION_ERROR",
            "The request contains invalid fields.",
            e.getFieldErrors().stream()
                .map(f -> new FieldError(f.getField(), f.getMessage(), f.getValue()))
                .toList(),
            "https://api.example.com/docs/errors#VALIDATION_ERROR"
        );
    }

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError handleNotFound(NotFoundException e) {
        return new ApiError(
            "NOT_FOUND",
            "The requested resource was not found.",
```

```
            List.of(),
            "https://api.example.com/docs/errors#NOT_FOUND"
        );
    }
}
```

```python
# A consistent error response structure
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse
from pydantic import BaseModel

class FieldError(BaseModel):
    field: str
    issue: str
    received: str | None = None

class ApiError(BaseModel):
    code: str
    message: str
    details: list[FieldError] = []
    documentation: str = ""

app = FastAPI()

@app.exception_handler(ValidationError)
async def validation_error_handler(request, exc):
    error = ApiError(
        code="VALIDATION_ERROR",
        message="The request contains invalid fields.",
        details=[
            FieldError(field=e["field"], issue=e["message"], received=e.get("value"))
            for e in exc.errors
        ],
        documentation="https://api.example.com/docs/errors#VALIDATION_ERROR",
    )
    return JSONResponse(status_code=422, content=error.dict())
```

HTTP Status Code Guide

```
COMMON HTTP STATUS CODES:

  Success:
  +-------+------------------+-------------------------------+
  | Code  | Name             | When to use                   |
  +-------+------------------+-------------------------------+
  | 200   | OK               | Successful GET, PUT, PATCH    |
  | 201   | Created          | Successful POST (resource     |
  |       |                  | created)                      |
  | 204   | No Content       | Successful DELETE             |
```

```
+-------+------------------+-------------------------------+

Client Errors:
+-------+------------------+-------------------------------+
| 400   | Bad Request      | Malformed request syntax      |
| 401   | Unauthorized     | Missing or invalid auth       |
| 403   | Forbidden        | Authenticated but not allowed |
| 404   | Not Found        | Resource does not exist       |
| 409   | Conflict         | Resource state conflict       |
| 422   | Unprocessable    | Valid syntax but invalid data |
| 429   | Too Many Requests| Rate limit exceeded           |
+-------+------------------+-------------------------------+

Server Errors:
+-------+------------------+-------------------------------+
| 500   | Internal Error   | Unexpected server failure     |
| 502   | Bad Gateway      | Upstream service failure      |
| 503   | Service Unavail  | Server temporarily down       |
+-------+------------------+-------------------------------+
```

Pagination

Any endpoint that returns a list should support pagination. Returning all records at once is a performance disaster.

```
PAGINATION STRATEGIES:

  1. OFFSET-BASED (simple, most common):
     GET /orders?page=2&size=20

     Response:
     {
         "data": [...],
         "page": 2,
         "size": 20,
         "total_items": 243,
         "total_pages": 13
     }

     Pros: Simple, random access to any page
     Cons: Skips/duplicates if data changes between requests

  2. CURSOR-BASED (better for large/changing data):
     GET /orders?cursor=eyJpZCI6MTAwfQ&size=20

     Response:
     {
```

```
        "data": [...],
        "next_cursor": "eyJpZCI6MTIwfQ",
        "has_more": true
    }

    Pros: No skips/duplicates, better performance
    Cons: No random access, forward-only navigation
```

Java Example: Offset-Based Pagination

```java
// Pagination request
public record PageRequest(int page, int size) {
    public PageRequest {
        if (page < 1) throw new IllegalArgumentException("Page must be >= 1");
        if (size < 1 || size > 100) throw new IllegalArgumentException(
            "Size must be between 1 and 100"
        );
    }

    public int offset() {
        return (page - 1) * size;
    }
}

// Pagination response wrapper
public record PageResponse<T>(
    List<T> data,
    int page,
    int size,
    long totalItems,
    int totalPages
) {
    public static <T> PageResponse<T> of(List<T> data, PageRequest request, long total) {
        return new PageResponse<>(
            data,
            request.page(),
            request.size(),
            total,
            (int) Math.ceil((double) total / request.size())
        );
    }
}

// Controller
@GetMapping("/orders")
public PageResponse<OrderResponse> listOrders(
        @RequestParam(defaultValue = "1") int page,
        @RequestParam(defaultValue = "20") int size) {
    PageRequest pageRequest = new PageRequest(page, size);
    List<Order> orders = orderService.findAll(pageRequest);
```

```
    long total = orderService.count();
    return PageResponse.of(
        orders.stream().map(OrderResponse::from).toList(),
        pageRequest,
        total
    );
}
```

```python
# Pagination response
from dataclasses import dataclass
from math import ceil
from typing import Generic, TypeVar, List

T = TypeVar("T")

@dataclass
class PageResponse:
    data: list
    page: int
    size: int
    total_items: int
    total_pages: int

    @classmethod
    def of(cls, data: list, page: int, size: int, total: int):
        return cls(
            data=data,
            page=page,
            size=size,
            total_items=total,
            total_pages=ceil(total / size) if size > 0 else 0,
        )

# Endpoint
@app.get("/orders")
def list_orders(page: int = 1, size: int = 20):
    if page < 1:
        raise ValueError("Page must be >= 1")
    if size < 1 or size > 100:
        raise ValueError("Size must be between 1 and 100")

    offset = (page - 1) * size
    orders = order_service.find_all(offset=offset, limit=size)
    total = order_service.count()

    return PageResponse.of(
        data=[order.to_dict() for order in orders],
        page=page,
        size=size,
        total=total,
    )
```

Idempotency: Safe to Retry

An idempotent operation produces the same result whether you call it once or multiple times. This is crucial for reliability -- networks fail, requests get retried, and your API must handle this gracefully.

```
IDEMPOTENCY:

  Calling it once:       Calling it three times:
  PUT /users/123         PUT /users/123  (same result)
  {"name": "Jane"}       PUT /users/123  (same result)
                         PUT /users/123  (same result)

  Result is the same regardless of how many times you call it.

  NATURALLY IDEMPOTENT:
  +--------+-----------------------------------------------+
  | Method | Why                                           |
  +--------+-----------------------------------------------+
  | GET    | Reading does not change state                 |
  | PUT    | Replaces entire resource to known state       |
  | DELETE | Deleting already-deleted resource is harmless |
  +--------+-----------------------------------------------+

  NOT NATURALLY IDEMPOTENT:
  +--------+-----------------------------------------------+
  | Method | Why                                           |
  +--------+-----------------------------------------------+
  | POST   | Each call may create a new resource           |
  | PATCH  | "increment by 1" is not idempotent            |
  +--------+-----------------------------------------------+
```

Making POST Idempotent with Idempotency Keys

```java
// The client sends a unique key with each request
// If the same key is sent again, return the original result
@PostMapping("/payments")
public ResponseEntity<PaymentResponse> createPayment(
        @RequestHeader("Idempotency-Key") String idempotencyKey,
        @RequestBody PaymentRequest request) {

    // Check if we already processed this key
    Optional<PaymentResponse> existing = idempotencyStore.find(idempotencyKey);
    if (existing.isPresent()) {
```

```
        return ResponseEntity.ok(existing.get());  // Return cached result
    }

    // Process the payment
    PaymentResponse response = paymentService.processPayment(request);

    // Store the result keyed by idempotency key
    idempotencyStore.save(idempotencyKey, response);

    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

```python
# Making POST idempotent with idempotency keys
@app.post("/payments")
def create_payment(
    request: PaymentRequest,
    idempotency_key: str = Header(..., alias="Idempotency-Key"),
):
    # Check if we already processed this key
    existing = idempotency_store.find(idempotency_key)
    if existing is not None:
        return existing  # Return cached result

    # Process the payment
    response = payment_service.process_payment(request)

    # Store the result
    idempotency_store.save(idempotency_key, response)

    return response
```

API Review Checklist

Use this checklist when designing or reviewing any API:

```
API REVIEW CHECKLIST:

  NAMING AND CONSISTENCY:
  [ ] Resource names are plural nouns (users, orders, products)
  [ ] Naming is consistent across all endpoints
  [ ] No verbs in URLs (use HTTP methods instead)
  [ ] Snake_case or camelCase is used consistently (not mixed)

  REQUESTS:
  [ ] Required vs optional fields are clearly documented
  [ ] Input validation is thorough with clear error messages
```

```
[ ] Sensible defaults for optional parameters
[ ] Maximum limits on list sizes and string lengths

RESPONSES:
[ ] Consistent response structure across all endpoints
[ ] Error responses include code, message, and details
[ ] Successful responses include only relevant data
[ ] Dates use ISO 8601 format (2024-01-15T10:30:00Z)
[ ] Monetary values include currency information

PAGINATION:
[ ] List endpoints are paginated
[ ] Page size has a maximum limit (e.g., 100)
[ ] Response includes total count and navigation info

ERRORS:
[ ] Appropriate HTTP status codes used
[ ] Error format is consistent and documented
[ ] No sensitive information leaked in errors
[ ] Rate limiting returns 429 with Retry-After header

COMPATIBILITY:
[ ] No breaking changes to existing fields
[ ] New fields have default values
[ ] Versioning strategy is defined
[ ] Deprecation policy is communicated

SAFETY:
[ ] GET requests are read-only (no side effects)
[ ] PUT and DELETE are idempotent
[ ] POST endpoints support idempotency keys where needed
[ ] Authentication and authorization are enforced
[ ] Input is sanitized to prevent injection attacks

DOCUMENTATION:
[ ] Every endpoint is documented with examples
[ ] Request/response schemas are defined
[ ] Error codes are listed and explained
[ ] Rate limits are documented
```

Common Mistakes

Mistake 1: Verbs in URLs

Using `/getUsers` or `/createOrder` instead of letting HTTP methods convey the action.

Fix: Use `GET /users` and `POST /orders`. The HTTP method is the verb.

Mistake 2: Inconsistent Error Formats

Some endpoints return `{"error": "..."}`, others return `{"message": "..."}`, and others return plain strings.

Fix: Use a single error response structure everywhere. Enforce it with a global exception handler.

Mistake 3: No Pagination

Returning all 50,000 records in a single response because "it works with test data."

Fix: Always paginate list endpoints. Default to a reasonable page size (20-50). Set a maximum (100).

Mistake 4: Breaking Changes Without Versioning

Renaming a response field from `name` to `fullName` without versioning, breaking all existing clients.

Fix: Either keep backward compatibility (add `fullName` alongside `name`) or introduce a new API version.

Mistake 5: Exposing Internal Models Directly

Using your database entity as your API response, leaking internal fields like `password_hash` or `internal_notes`.

Fix: Create separate response DTOs that contain only the fields API consumers should see.

Best Practices

1. Follow the Principle of Least Surprise. APIs should do what their names suggest, nothing more.
2. Be consistent. Pick naming conventions and stick to them everywhere.
3. Never break backward compatibility without versioning and a deprecation period.
4. Design error responses for humans. Include the field, the issue, and a link to documentation.
5. Paginate all list endpoints. No exceptions. Default page sizes are your friend.

6. Make unsafe operations idempotent where possible, using idempotency keys for POST requests.
7. Create response DTOs. Never expose internal models directly through your API.
8. Use your own API. The best way to find usability issues is to be your own first user.
9. Review APIs before implementing them. Changing an API after clients depend on it is expensive.
10. Document with examples. Show complete request/response pairs, not just schemas.

Quick Summary

```
API DESIGN PRINCIPLES AT A GLANCE:

  Principle             What it means
  +-------------------+-------------------------------------+
  | Least Surprise    | API behaves as the name suggests.   |
  |                   | No hidden side effects.             |
  +-------------------+-------------------------------------+
  | Consistency       | Same conventions everywhere.        |
  |                   | Pick one style, use it always.      |
  +-------------------+-------------------------------------+
  | Backward Compat   | Adding is safe. Removing breaks.    |
  |                   | Use versioning for breaking change.|
  +-------------------+-------------------------------------+
  | Clear Errors      | Code + message + details + docs.    |
  |                   | Tell users what went wrong and how |
  |                   | to fix it.                          |
  +-------------------+-------------------------------------+
  | Pagination        | Never return unbounded lists.       |
  |                   | Always paginate.                    |
  +-------------------+-------------------------------------+
  | Idempotency       | Safe to retry. Same call, same      |
  |                   | result. Use idempotency keys.       |
  +-------------------+-------------------------------------+
```

Key Points

- The Principle of Least Surprise is the foundation of good API design: APIs should behave as their names suggest.

- Consistent naming (plural nouns for resources, HTTP methods for actions) makes APIs predictable and learnable.
- Backward compatibility protects existing users: add new fields, never remove or rename existing ones without versioning.
- Use URL path versioning (e.g., `/api/v1/users`) as the most transparent versioning strategy.
- Error responses should include a machine-readable code, human-readable message, field-level details, and a documentation link.
- Always paginate list endpoints with a maximum page size limit.
- Idempotency makes APIs safe to retry: use idempotency keys for POST endpoints where duplicate processing is dangerous.
- Review APIs using the checklist before implementation, not after.

Practice Questions

1. A developer proposes this endpoint: `POST /api/users/getByEmail`. What are two things wrong with this design, and how would you fix it?
2. Your API currently returns user responses with a `name` field. A new requirement asks you to split this into `firstName` and `lastName`. How would you handle this without breaking existing clients?
3. Explain the difference between offset-based and cursor-based pagination. When would you choose each?
4. A POST endpoint to create orders sometimes receives the same request twice due to network retries, resulting in duplicate orders. How would you solve this?
5. Your team uses these error response formats across different endpoints: `{"error": "..."}`, `{"message": "..."}`, `{"err_code": 1, "err_msg": "..."}`. What is the problem and how would you fix it?

Exercises

Exercise 1: Design a REST API

Design a complete REST API for a library management system. Define endpoints for books (list, get, create, update, delete), members (list, get, create), and loans (borrow, return, list

active). Include: URL paths, HTTP methods, request/response formats, error responses, and pagination. Apply all principles from this chapter.

Exercise 2: Build Error Handling

Implement a global error handler (in Java or Python) that catches common exceptions (`NotFoundException`, `ValidationException`, `UnauthorizedException`, `RateLimitException`) and returns consistent error responses with the structure defined in this chapter.

Exercise 3: Add Idempotency

You have a `POST /transfers` endpoint that moves money between bank accounts. Implement idempotency using an idempotency key so that network retries do not cause double transfers. Include the key storage mechanism and the duplicate detection logic.

What Is Next?

With strong API design principles in your toolkit, we jump ahead to Chapter 29: Code Review Culture -- how teams use code reviews not just to catch bugs, but to maintain quality, share knowledge, and build a culture of continuous improvement. You will learn what to look for in a review, how to give and receive feedback constructively, and how to keep pull requests at a reviewable size.

Chapter 27: Logging and Observability

What You Will Learn

- What to log and, critically, what never to log
- How to use log levels (DEBUG, INFO, WARN, ERROR) effectively
- How structured logging (JSON) makes logs searchable and actionable
- How correlation IDs let you trace a request across multiple services
- The difference between metrics, logs, and traces
- How to replace println debugging with proper logging
- How log rotation prevents disk space disasters
- How centralized logging (ELK stack) works
- How to set up alerting so errors wake the right people

Why This Chapter Matters

Your application is running in production. A customer reports that their order failed, but they cannot tell you when or what they ordered. You open the server and find a single file called `app.log` that is 47 gigabytes. You search for "error" and get 200,000 results. None of them include the customer ID.

This scenario is painfully common. Good logging is the difference between diagnosing a production issue in five minutes and spending an entire weekend guessing. Observability goes further: it means your system can tell you what is wrong before your customers do.

Yet most developers learn logging by accident. They scatter `System.out.println` or `print()` calls throughout their code, leave them in production, and wonder why their logs are useless. This chapter teaches you to treat logging as a first-class engineering concern.

Section 1: Why println Debugging Fails in Production

Every developer starts here:

BEFORE: println Debugging

```java
// Java - The "I'll just print it" approach
public class OrderService {
    public void processOrder(Order order) {
        System.out.println("Processing order");
        System.out.println("Order: " + order);

        try {
            validateOrder(order);
            System.out.println("Validation passed");

            chargePayment(order);
            System.out.println("Payment charged");

            sendConfirmation(order);
            System.out.println("Confirmation sent");

        } catch (Exception e) {
            System.out.println("Something went wrong: " + e);
        }
    }
}
```

```python
# Python - The "I'll just print it" approach
class OrderService:
    def process_order(self, order):
        print("Processing order")
        print(f"Order: {order}")

        try:
            self.validate_order(order)
            print("Validation passed")

            self.charge_payment(order)
            print("Payment charged")

            self.send_confirmation(order)
            print("Confirmation sent")
```

```
        except Exception as e:
            print(f"Something went wrong: {e}")
```

This produces output like:

```
Processing order
Order: Order@3f2a1c
Validation passed
Payment charged
Something went wrong: java.lang.NullPointerException
Processing order
Order: Order@7b4a2d
Validation passed
Payment charged
Confirmation sent
```

Problems with this approach:

```
+-------------------------------------------------------------+
|  Why println Fails in Production                            |
+-------------------------------------------------------------+
|                                                             |
|  1. No timestamps   - When did the error happen?            |
|  2. No severity     - Is this fatal or informational?       |
|  3. No context      - Which user? Which order?              |
|  4. No filtering    - Cannot turn off noisy messages        |
|  5. No structure    - Cannot search or aggregate            |
|  6. Goes to stdout  - Lost when process restarts            |
|  7. Not thread-safe - Messages interleave randomly          |
|  8. No rotation     - File grows until disk is full         |
|                                                             |
+-------------------------------------------------------------+
```

AFTER: Proper Logging

```java
// Java - Professional logging with SLF4J + Logback
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;

public class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public void processOrder(Order order) {
        MDC.put("orderId", order.getId());
        MDC.put("customerId", order.getCustomerId());
```

```
        log.info("Starting order processing");

        try {
            validateOrder(order);
            log.debug("Order validation passed");

            chargePayment(order);
            log.info("Payment charged successfully, amount={}",
                     order.getTotal());

            sendConfirmation(order);
            log.info("Order processing completed");

        } catch (PaymentDeclinedException e) {
            log.warn("Payment declined for order, reason={}",
                     e.getReason());
            throw e;

        } catch (Exception e) {
            log.error("Unexpected error processing order", e);
            throw new OrderProcessingException(
                "Failed to process order " + order.getId(), e);

        } finally {
            MDC.clear();
        }
    }
}
```

```python
# Python - Professional logging
import logging
import structlog

log = structlog.get_logger()

class OrderService:
    def process_order(self, order):
        logger = log.bind(
            order_id=order.id,
            customer_id=order.customer_id
        )

        logger.info("starting_order_processing")

        try:
            self.validate_order(order)
            logger.debug("order_validation_passed")

            self.charge_payment(order)
            logger.info("payment_charged",
                        amount=order.total)
```

```
            self.send_confirmation(order)
            logger.info("order_processing_completed")

        except PaymentDeclinedError as e:
            logger.warning("payment_declined",
                           reason=str(e))
            raise

        except Exception as e:
            logger.error("unexpected_error_processing_order",
                        error=str(e),
                        exc_info=True)
            raise OrderProcessingError(
                f"Failed to process order {order.id}"
            ) from e
```

This produces:

```json
{
  "timestamp": "2024-01-15T10:23:45.123Z",
  "level": "INFO",
  "logger": "com.shop.OrderService",
  "message": "Payment charged successfully",
  "orderId": "ORD-12345",
  "customerId": "CUST-789",
  "amount": 49.99,
  "thread": "http-nio-8080-exec-3"
}
```

Now you can answer: What happened? When? To whom? How bad was it?

Section 2: What to Log and What NEVER to Log

What to Log

```
+-------------------------------------------------------------------+
|  LOG THIS                            |  EXAMPLE                    |
+--------------------------------------+-----------------------------+
|  Request received                    |  "Received POST /orders"    |
|  Request completed (with timing)     |  "Completed in 234ms"       |
|  Business events                     |  "Order placed"             |
|  State transitions                   |  "Order PENDING -> SHIPPED" |
|  Integration calls (start/end)       |  "Calling payment gateway"  |
```

```
|  Authentication events              |  "User logged in"            |
|  Errors and exceptions              |  Full stack trace             |
|  Configuration at startup           |  "DB pool size: 10"          |
|  Decisions with reasons             |  "Retry #2, backoff 400ms"   |
+-------------------------------------+------------------------------+
```

What NEVER to Log

This is not optional. Logging sensitive data can violate laws (GDPR, HIPAA, PCI-DSS) and create security vulnerabilities.

```
+-------------------------------------------------------------------+
|  NEVER LOG THIS                     |  WHY                         |
+-------------------------------------+------------------------------+
|  Passwords                          |  Obvious security risk       |
|  API keys / tokens                  |  Attacker reads logs = pwned |
|  Credit card numbers                |  PCI-DSS violation           |
|  Social security numbers            |  Identity theft risk         |
|  Session IDs / JWTs                 |  Session hijacking           |
|  Full request bodies (user data)    |  May contain PII             |
|  Database connection strings        |  Contains credentials        |
|  Encryption keys                    |  Defeats encryption          |
|  Health information                 |  HIPAA violation             |
|  Personal email / phone             |  GDPR / privacy violation    |
+-------------------------------------+------------------------------+
```

BEFORE: Logging Sensitive Data (DANGEROUS)

```java
// Java - NEVER DO THIS
public class AuthService {
    public User login(String username, String password) {
        // SECURITY VULNERABILITY: password in logs!
        log.info("Login attempt: user={}, password={}", username, password);

        User user = userRepository.findByUsername(username);

        // SECURITY VULNERABILITY: token in logs!
        String token = generateToken(user);
        log.info("Generated token: {}", token);

        // SECURITY VULNERABILITY: full user object may contain PII
        log.debug("User details: {}", user);

        return user;
    }
}
```

```python
# Python - NEVER DO THIS
class AuthService:
    def login(self, username, password):
        # SECURITY VULNERABILITY: password in logs!
        logger.info(f"Login attempt: user={username}, password={password}")

        user = self.user_repository.find_by_username(username)

        # SECURITY VULNERABILITY: token in logs!
        token = self.generate_token(user)
        logger.info(f"Generated token: {token}")

        # SECURITY VULNERABILITY: full user object may contain PII
        logger.debug(f"User details: {user}")

        return user
```

AFTER: Safe Logging

```java
// Java - Safe logging practices
public class AuthService {
    public User login(String username, String password) {
        // Log the event, not the credentials
        log.info("Login attempt, username={}", username);

        User user = userRepository.findByUsername(username);
        if (user == null) {
            log.warn("Login failed: unknown username={}", username);
            throw new AuthenticationException("Invalid credentials");
        }

        if (!passwordEncoder.matches(password, user.getPasswordHash())) {
            log.warn("Login failed: invalid password for username={}",
                     username);
            throw new AuthenticationException("Invalid credentials");
        }

        String token = generateToken(user);
        // Log that a token was generated, not the token itself
        log.info("Login successful, username={}, tokenPrefix={}",
                 username, token.substring(0, 8) + "...");

        return user;
    }
}
```

```python
# Python - Safe logging practices
class AuthService:
```

```
    def login(self, username: str, password: str) -> User:
        # Log the event, not the credentials
        logger.info("login_attempt", username=username)

        user = self.user_repository.find_by_username(username)
        if user is None:
            logger.warning("login_failed_unknown_user",
                           username=username)
            raise AuthenticationError("Invalid credentials")

        if not verify_password(password, user.password_hash):
            logger.warning("login_failed_invalid_password",
                           username=username)
            raise AuthenticationError("Invalid credentials")

        token = self.generate_token(user)
        # Log that a token was generated, not the token itself
        logger.info("login_successful",
                    username=username,
                    token_prefix=token[:8] + "...")

        return user
```

Rule of thumb: Before logging any variable, ask yourself: "If an attacker read this log line, could they use this information to compromise the system or a user?" If yes, do not log it.

Section 3: Log Levels -- When to Use Each One

Log levels are not decoration. They are a filtering mechanism that lets you control how much detail you see in different environments.

```
+-----------------------------------------------------------------+
|  Log Level Pyramid                                              |
+-----------------------------------------------------------------+
|                                                                 |
|        /\          FATAL  - Application cannot continue         |
|       /  \         (Use sparingly: JVM out of memory,           |
|      /    \         unrecoverable startup failure)              |
|     /------\                                                    |
|    / ERROR  \      Something broke that should not have.        |
|   /   (Fix   \     An operation failed. Needs attention.        |
|  /   ASAP)    \    Example: Payment gateway unreachable.        |
| /--------------\                                                |
| |    WARN      |   Something unexpected but recoverable.        |
| |  (Investigate|   Example: Retry succeeded on attempt 3.       |
| |   soon)      |   Example: Deprecated API still in use.        |
```

```
| |--------------|                                            |
| |    INFO      |   Normal business events.                  |
| |  (Default    |   Example: Order placed, user logged in.   |
| |   in prod)   |   This is your production baseline.        |
| |--------------|                                            |
| |    DEBUG     |   Detailed technical information.           |
| |  (Dev/       |   Example: SQL queries, cache hits/misses. |
| |   staging)   |   Too noisy for production normally.       |
| |--------------|                                            |
| |    TRACE     |   Extremely detailed: method entry/exit,   |
| |  (Rarely     |   variable values at each step.            |
| |   used)      |   Only for deep debugging sessions.        |
| |______________|                                            |
+-------------------------------------------------------------+
```

Setting Log Levels by Environment

```java
// Java - logback.xml configuration
// Production: INFO and above
// <root level="INFO">

// Staging: DEBUG for your code, INFO for libraries
// <logger name="com.yourcompany" level="DEBUG" />
// <logger name="org.springframework" level="INFO" />
// <logger name="org.hibernate" level="WARN" />
```

```python
# Python - logging configuration
import logging

# Production
logging.basicConfig(level=logging.INFO)

# Development
logging.basicConfig(level=logging.DEBUG)

# Suppress noisy libraries
logging.getLogger("urllib3").setLevel(logging.WARNING)
logging.getLogger("botocore").setLevel(logging.WARNING)
```

Common Log Level Mistakes

```java
// WRONG: Using ERROR for expected business cases
log.error("User not found: {}", username);  // This is a WARN or INFO
```

```
// WRONG: Using INFO for debugging details
log.info("SQL query: {}", sql);  // This is DEBUG

// WRONG: Using DEBUG for critical failures
log.debug("Database connection failed", e);  // This is ERROR

// CORRECT usage:
log.info("User login successful, userId={}", userId);
log.warn("Payment retry attempt {}/3", attempt);
log.error("Database connection pool exhausted", e);
log.debug("Cache miss for key={}, fetching from DB", key);
```

Section 4: Structured Logging

Traditional logs are strings. Structured logs are data.

BEFORE: Unstructured Logging

```
2024-01-15 10:23:45 INFO Processing order #12345 for customer John Smith, total $49.99
2024-01-15 10:23:45 ERROR Failed to charge payment for order #12345: Connection timeout
```

Try searching these logs for "all failed payments over $100 in the last hour." You would need complex regex, and it would break when someone changes the message format.

AFTER: Structured Logging (JSON)

```json
{
  "timestamp": "2024-01-15T10:23:45.123Z",
  "level": "INFO",
  "event": "order_processing_started",
  "orderId": "12345",
  "customerId": "CUST-789",
  "customerName": "John Smith",
  "total": 49.99,
  "currency": "USD",
  "service": "order-service",
  "host": "prod-web-03"
}
{
  "timestamp": "2024-01-15T10:23:46.456Z",
  "level": "ERROR",
```

```
  "event": "payment_charge_failed",
  "orderId": "12345",
  "customerId": "CUST-789",
  "total": 49.99,
  "error": "Connection timeout",
  "gateway": "stripe",
  "retryCount": 0,
  "service": "order-service",
  "host": "prod-web-03"
}
```

Now the query "all failed payments over $100" is trivial:

```
event:payment_charge_failed AND total:>100
```

Setting Up Structured Logging

```java
// Java - Logback with JSON encoder (logback.xml)
// Add dependency: net.logstash.logback:logstash-logback-encoder

/*
<appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdcKeyName>orderId</includeMdcKeyName>
        <includeMdcKeyName>customerId</includeMdcKeyName>
        <includeMdcKeyName>correlationId</includeMdcKeyName>
    </encoder>
</appender>
*/

// In code, use MDC for context:
import org.slf4j.MDC;

public class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public void processOrder(Order order) {
        MDC.put("orderId", order.getId());
        MDC.put("customerId", order.getCustomerId());

        log.info("Order processing started");
        // Output: {"timestamp":"...","level":"INFO",
        //          "message":"Order processing started",
        //          "orderId":"12345","customerId":"CUST-789"}
    }
}
```

```python
# Python - structlog for structured logging
import structlog

structlog.configure(
    processors=[
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.add_log_level,
        structlog.processors.StackInfoRenderer(),
        structlog.processors.format_exc_info,
        structlog.processors.JSONRenderer()
    ]
)

log = structlog.get_logger()

class OrderService:
    def process_order(self, order):
        logger = log.bind(
            order_id=order.id,
            customer_id=order.customer_id
        )
        logger.info("order_processing_started", total=order.total)
        # Output: {"timestamp":"...","level":"info",
        #          "event":"order_processing_started",
        #          "order_id":"12345","customer_id":"CUST-789",
        #          "total":49.99}
```

Section 5: Correlation IDs -- Tracing Requests Across Services

In a microservices architecture, a single user request may pass through five or more services. Without a correlation ID, connecting the logs from each service is nearly impossible.

```
+-----------------------------------------------------------------+
|  Request Flow Without Correlation ID                            |
+-----------------------------------------------------------------+
|                                                                 |
|  API Gateway     Order Service     Payment Service    Email   |
|  ----------      -------------     ---------------    -----   |
|  "Request in"    "Processing"      "Charging card"    "Send"  |
|  "Request in"    "Processing"      "Charging card"    "Send"  |
|  "Request in"    "Validating"      "Refunding"        "Send"  |
|                                                                 |
|  Which "Charging card" goes with which "Request in"?            |
|  IMPOSSIBLE TO TELL.                                            |
+-----------------------------------------------------------------+
```

```
+-----------------------------------------------------------------+
|  Request Flow WITH Correlation ID                               |
+-----------------------------------------------------------------+
|                                                                 |
|  API Gateway        Order Service        Payment Service       |
|  [corr=abc-123]     [corr=abc-123]       [corr=abc-123]       |
|  "Request in"  -->  "Processing"    -->  "Charging $49.99"     |
|                                                                 |
|  [corr=def-456]     [corr=def-456]       [corr=def-456]       |
|  "Request in"  -->  "Processing"    -->  "Charging $29.99"     |
|                                                                 |
|  Search for corr=abc-123 and get the COMPLETE story.            |
+-----------------------------------------------------------------+
```

Implementing Correlation IDs

```java
// Java - Servlet filter that creates/propagates correlation IDs
import javax.servlet.*;
import javax.servlet.http.*;
import org.slf4j.MDC;
import java.util.UUID;

public class CorrelationIdFilter implements Filter {

    private static final String CORRELATION_HEADER = "X-Correlation-ID";

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
                         FilterChain chain) throws Exception {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        // Use existing ID from upstream service, or generate new one
        String correlationId = request.getHeader(CORRELATION_HEADER);
        if (correlationId == null || correlationId.isBlank()) {
            correlationId = UUID.randomUUID().toString();
        }

        // Put it in MDC so every log message includes it automatically
        MDC.put("correlationId", correlationId);

        // Pass it downstream in the response
        response.setHeader(CORRELATION_HEADER, correlationId);

        try {
            chain.doFilter(req, res);
        } finally {
            MDC.clear();
        }
```

```
    }
}
```

```python
# Python - Middleware for correlation IDs (FastAPI example)
import uuid
from contextvars import ContextVar
from starlette.middleware.base import BaseHTTPMiddleware
import structlog

correlation_id_var: ContextVar[str] = ContextVar("correlation_id",
                                                 default="")

class CorrelationIdMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        # Use existing ID or generate a new one
        correlation_id = request.headers.get(
            "X-Correlation-ID",
            str(uuid.uuid4())
        )

        # Store in context variable
        correlation_id_var.set(correlation_id)

        # Bind to structlog so every log includes it
        structlog.contextvars.clear_contextvars()
        structlog.contextvars.bind_contextvars(
            correlation_id=correlation_id
        )

        response = await call_next(request)
        response.headers["X-Correlation-ID"] = correlation_id

        return response
```

When calling another service, always forward the correlation ID:

```java
// Java - Forwarding correlation ID to downstream services
public class PaymentClient {
    public PaymentResult charge(Order order) {
        String correlationId = MDC.get("correlationId");

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(paymentUrl + "/charge"))
            .header("X-Correlation-ID", correlationId)
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(toJson(order)))
            .build();

        return httpClient.send(request, ...);
    }
}
```

Section 6: Metrics vs Logs vs Traces

These three tools are called the "three pillars of observability." They serve different purposes and complement each other.

```
+-----------------------------------------------------------------+
|  The Three Pillars of Observability                             |
+-----------------------------------------------------------------+
|                                                                 |
|  LOGS            METRICS          TRACES                       |
|  ----            -------          ------                       |
|  What happened   How much/many    Where time was spent         |
|                                                                 |
|  Discrete        Aggregated       Distributed                  |
|  events          numbers          request flows                |
|                                                                 |
|  "Order 123      "50 orders/min"  "Order 123 took 2.3s:      |
|   failed at       "p99 = 1.2s"     - API: 50ms               |
|   10:23:45        "error rate 2%"  - Validation: 20ms          |
|   because..."                      - DB query: 800ms          |
|                                     - Payment: 1400ms"         |
|                                                                 |
|  High detail,    Low detail,      Medium detail,               |
|  high volume     low volume       medium volume                |
|                                                                 |
|  grep / search   Dashboards /     Flame graphs /               |
|  (ELK, Splunk)   alerts           waterfall charts             |
|                  (Prometheus,     (Jaeger, Zipkin)              |
|                   Datadog)                                      |
+-----------------------------------------------------------------+
```

When to Use Each

- Logs: "Why did order 12345 fail?" -- You need the specific details.
- Metrics: "Are we healthy right now?" -- You need aggregated numbers.
- Traces: "Why is the checkout page slow?" -- You need to see where time

is spent across services.

Basic Metrics Example

```java
// Java - Micrometer metrics (works with Prometheus, Datadog, etc.)
import io.micrometer.core.instrument.*;

public class OrderService {
    private final Counter ordersPlaced;
    private final Counter ordersFailed;
    private final Timer orderProcessingTime;

    public OrderService(MeterRegistry registry) {
        this.ordersPlaced = Counter.builder("orders.placed")
            .description("Total orders placed")
            .register(registry);

        this.ordersFailed = Counter.builder("orders.failed")
            .description("Total failed orders")
            .tag("reason", "unknown")
            .register(registry);

        this.orderProcessingTime = Timer.builder("orders.processing.time")
            .description("Time to process an order")
            .register(registry);
    }

    public void processOrder(Order order) {
        orderProcessingTime.record(() -> {
            try {
                doProcessOrder(order);
                ordersPlaced.increment();
            } catch (Exception e) {
                ordersFailed.increment();
                throw e;
            }
        });
    }
}
```

```python
# Python - Prometheus metrics
from prometheus_client import Counter, Histogram

orders_placed = Counter(
    "orders_placed_total",
    "Total number of orders placed"
)
orders_failed = Counter(
    "orders_failed_total",
    "Total number of failed orders",
    ["reason"]
)
order_processing_seconds = Histogram(
    "order_processing_seconds",
    "Time spent processing orders"
)
```

```
class OrderService:
    @order_processing_seconds.time()
    def process_order(self, order):
        try:
            self._do_process_order(order)
            orders_placed.inc()
        except PaymentError as e:
            orders_failed.labels(reason="payment").inc()
            raise
        except Exception as e:
            orders_failed.labels(reason="unknown").inc()
            raise
```

Section 7: Log Rotation -- Preventing Disk Disasters

Without log rotation, your log files grow until the disk fills up. When the disk is full, your application crashes. This is not a theoretical concern; it happens regularly.

```
+-----------------------------------------------------------------+
|  Log Rotation Strategy                                          |
+-----------------------------------------------------------------+
|                                                                 |
|  app.log  (current, max 100MB)                                 |
|    |                                                            |
|    | When full, rotate:                                         |
|    v                                                            |
|  app.2024-01-15.log.gz  (compressed, ~10MB)                    |
|  app.2024-01-14.log.gz                                         |
|  app.2024-01-13.log.gz                                         |
|  ...                                                            |
|  app.2024-01-01.log.gz                                         |
|    |                                                            |
|    | After 30 days, auto-delete                                 |
|    v                                                            |
|  (deleted)                                                      |
|                                                                 |
+-----------------------------------------------------------------+
```

Java - Logback Rolling File Configuration

```xml
<!-- logback.xml -->
<appender name="FILE"
          class="ch.qos.logback.core.rolling.RollingFileAppender">
    <file>logs/app.log</file>

    <rollingPolicy
        class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
        <!-- Daily rotation -->
        <fileNamePattern>
            logs/app.%d{yyyy-MM-dd}.%i.log.gz
        </fileNamePattern>
        <!-- Each file max 100MB -->
        <maxFileSize>100MB</maxFileSize>
        <!-- Keep 30 days of history -->
        <maxHistory>30</maxHistory>
        <!-- Total max 5GB -->
        <totalSizeCap>5GB</totalSizeCap>
    </rollingPolicy>

    <encoder class="net.logstash.logback.encoder.LogstashEncoder" />
</appender>
```

Python - Rotating File Handler

```python
import logging
from logging.handlers import RotatingFileHandler, TimedRotatingFileHandler

# Size-based rotation
handler = RotatingFileHandler(
    "logs/app.log",
    maxBytes=100 * 1024 * 1024,  # 100MB
    backupCount=30               # Keep 30 old files
)

# Time-based rotation (daily)
handler = TimedRotatingFileHandler(
    "logs/app.log",
    when="midnight",
    interval=1,
    backupCount=30
)

logging.basicConfig(
    level=logging.INFO,
    handlers=[handler],
    format="%(asctime)s %(levelname)s %(name)s %(message)s"
)
```

Section 8: Centralized Logging with the ELK Stack

When you have 20 servers, you cannot SSH into each one to read log files. You need centralized logging.

```
+-----------------------------------------------------------------+
|  ELK Stack Architecture                                         |
+-----------------------------------------------------------------+
|                                                                 |
|  Server 1 ----\                                                 |
|  (app.log)     \                                                |
|                 \    +-----------+    +---------------+         |
|  Server 2 ------+--> | Logstash  |--> | Elasticsearch |        |
|  (app.log)      |    | (process  |    | (store and    |        |
|                 /    |  & parse) |    |  index)       |        |
|  Server 3 ----/     +-----------+    +-------+-------+         |
|  (app.log)                                     |                |
|                                        +------+------+         |
|                                        |   Kibana    |         |
|                                        | (visualize  |         |
|                                        |  & search)  |         |
|                                        +-------------+         |
|                                                                 |
|  Modern alternative: Filebeat (lightweight shipper)             |
|  Server -> Filebeat -> Elasticsearch -> Kibana                  |
+-----------------------------------------------------------------+
```

Elasticsearch stores and indexes logs for fast searching. Logstash (or Filebeat) ships logs from your servers to Elasticsearch. Kibana provides a web UI for searching and visualizing logs.

Modern alternatives include:

- Grafana Loki (lightweight, indexes labels not content)
- Datadog (managed service)
- AWS CloudWatch (if on AWS)
- Google Cloud Logging (if on GCP)

Section 9: Alerting on Errors

Logs are useless if nobody reads them. Alerting bridges the gap between "something went wrong" and "someone knows about it."

What to Alert On

```
+-----------------------------------------------------------------+
|  Alert Severity Guide                                           |
+-----------------------------------------------------------------+
|                                                                 |
|  PAGE (wake someone up):                                        |
|    - Error rate > 5% for 5 minutes                              |
|    - Response time p99 > 5 seconds for 10 minutes               |
|    - Health check failing on 2+ servers                         |
|    - Zero orders processed in 30 minutes (during business       |
|      hours)                                                     |
|    - Database connection pool exhausted                         |
|                                                                 |
|  TICKET (fix during business hours):                            |
|    - Error rate > 1% for 15 minutes                             |
|    - Disk usage > 80%                                           |
|    - Deprecated API calls detected                              |
|    - Certificate expiring within 30 days                        |
|                                                                 |
|  LOG ONLY (review periodically):                                |
|    - Individual request failures (retried successfully)         |
|    - Cache miss rates                                           |
|    - Slow queries under threshold                               |
|                                                                 |
+-----------------------------------------------------------------+
```

Alert Fatigue

The biggest risk with alerting is alert fatigue. If your team gets 50 alerts a day and most are false positives, they will start ignoring all alerts -- including the real ones.

Rules for healthy alerting:

1. Every alert must be actionable. If nobody needs to do anything, it is not an alert.
2. Every alert should have a runbook. "What do I do when I get this alert?"
3. Review and tune alerts monthly. Delete alerts nobody acts on.
4. Use escalation policies. If the primary on-call does not acknowledge in 15 minutes, escalate.

Section 10: Putting It All Together -- A Real-World Logging Setup

Here is a complete example showing how all these concepts work together in a production-ready service.

```java
// Java - Complete production logging setup
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;
import io.micrometer.core.instrument.*;

public class OrderService {
    private static final Logger log =
        LoggerFactory.getLogger(OrderService.class);

    private final Timer processingTimer;
    private final Counter successCounter;
    private final Counter failureCounter;

    public OrderService(MeterRegistry metrics) {
        this.processingTimer = Timer.builder("order.processing.duration")
            .description("Order processing duration")
            .register(metrics);
        this.successCounter = Counter.builder("order.processed")
            .tag("result", "success")
            .register(metrics);
        this.failureCounter = Counter.builder("order.processed")
            .tag("result", "failure")
            .register(metrics);
    }

    public OrderResult processOrder(Order order) {
        // Structured context for all log messages in this operation
        MDC.put("orderId", order.getId());
        MDC.put("customerId", order.getCustomerId());
        MDC.put("orderTotal", String.valueOf(order.getTotal()));

        long startTime = System.currentTimeMillis();
        log.info("Order processing started");

        try {
            // Step 1: Validate
            log.debug("Validating order, itemCount={}",
                      order.getItems().size());
            validateOrder(order);

            // Step 2: Calculate price
            BigDecimal finalPrice = calculatePrice(order);
            log.info("Price calculated, originalTotal={}, finalTotal={}, " +
```

```
                    "discountApplied={}",
                    order.getTotal(), finalPrice,
                    !finalPrice.equals(order.getTotal()));

            // Step 3: Charge payment
            log.info("Initiating payment charge");
            PaymentResult payment = chargePayment(order, finalPrice);
            log.info("Payment completed, transactionId={}, gateway={}",
                    payment.getTransactionId(), payment.getGateway());

            // Step 4: Send confirmation
            sendConfirmation(order, payment);

            // Record metrics
            long duration = System.currentTimeMillis() - startTime;
            processingTimer.record(duration, TimeUnit.MILLISECONDS);
            successCounter.increment();

            log.info("Order processing completed, durationMs={}", duration);

            return OrderResult.success(payment.getTransactionId());

        } catch (ValidationException e) {
            failureCounter.increment();
            log.warn("Order validation failed, reason={}", e.getMessage());
            return OrderResult.failure("VALIDATION_ERROR", e.getMessage());

        } catch (PaymentDeclinedException e) {
            failureCounter.increment();
            log.warn("Payment declined, reason={}, gateway={}",
                    e.getReason(), e.getGateway());
            return OrderResult.failure("PAYMENT_DECLINED", e.getReason());

        } catch (Exception e) {
            failureCounter.increment();
            log.error("Unexpected error processing order", e);
            return OrderResult.failure("INTERNAL_ERROR",
                                       "An unexpected error occurred");
        } finally {
            MDC.clear();
        }
    }
}
```

```python
# Python - Complete production logging setup
import time
import structlog
from prometheus_client import Counter, Histogram

log = structlog.get_logger()

processing_duration = Histogram(
```

```
    "order_processing_duration_seconds",
    "Order processing duration"
)
orders_processed = Counter(
    "orders_processed_total",
    "Orders processed",
    ["result"]
)

class OrderService:
    def process_order(self, order) -> OrderResult:
        logger = log.bind(
            order_id=order.id,
            customer_id=order.customer_id,
            order_total=float(order.total)
        )

        start_time = time.monotonic()
        logger.info("order_processing_started")

        try:
            # Step 1: Validate
            logger.debug("validating_order",
                         item_count=len(order.items))
            self.validate_order(order)

            # Step 2: Calculate price
            final_price = self.calculate_price(order)
            logger.info("price_calculated",
                        original_total=float(order.total),
                        final_total=float(final_price),
                        discount_applied=final_price != order.total)

            # Step 3: Charge payment
            logger.info("initiating_payment_charge")
            payment = self.charge_payment(order, final_price)
            logger.info("payment_completed",
                        transaction_id=payment.transaction_id,
                        gateway=payment.gateway)

            # Step 4: Send confirmation
            self.send_confirmation(order, payment)

            # Record metrics
            duration = time.monotonic() - start_time
            processing_duration.observe(duration)
            orders_processed.labels(result="success").inc()

            logger.info("order_processing_completed",
                        duration_seconds=round(duration, 3))

            return OrderResult.success(payment.transaction_id)

        except ValidationError as e:
            orders_processed.labels(result="failure").inc()
```

```
            logger.warning("order_validation_failed", reason=str(e))
            return OrderResult.failure("VALIDATION_ERROR", str(e))

        except PaymentDeclinedError as e:
            orders_processed.labels(result="failure").inc()
            logger.warning("payment_declined",
                           reason=e.reason, gateway=e.gateway)
            return OrderResult.failure("PAYMENT_DECLINED", e.reason)

        except Exception as e:
            orders_processed.labels(result="failure").inc()
            logger.error("unexpected_error_processing_order",
                         error=str(e), exc_info=True)
            return OrderResult.failure("INTERNAL_ERROR",
                                       "An unexpected error occurred")
```

Common Mistakes

1. Logging sensitive data. Passwords, tokens, credit card numbers, and PII

must never appear in logs. This is a security vulnerability and often a legal violation.

2. Using the wrong log level. Logging expected business events (like "user

not found") as ERROR creates alert fatigue. Save ERROR for things that genuinely need developer attention.

3. Logging without context. A message that says "Error occurred" is useless.

Always include what was being attempted, for whom, and with what parameters (excluding sensitive data).

4. Not using structured logging. Plain text logs are nearly impossible to

search and aggregate at scale. Switching to JSON structured logging is one of the highest-value improvements you can make.

5. Forgetting log rotation. Logs that grow without bound will eventually

fill the disk and crash your application.

6. Logging too much in production. Setting DEBUG level in production creates

enormous volumes of data, slows the application, and makes it harder to find important messages.

7. No correlation IDs. Without a way to trace a request across services,

debugging distributed systems is guesswork.

8. Alert fatigue. Too many alerts, or alerts that are not actionable, teach the team to ignore alerts entirely.

Best Practices

1. Use a logging framework, never println/print. Logging frameworks give you levels, formatting, rotation, and output control.
2. Always include correlation IDs. Generate one at the entry point and propagate it through every service call.
3. Log at the right level. INFO for business events, WARN for recoverable issues, ERROR for failures needing attention, DEBUG for development details.
4. Use structured logging from day one. It is much harder to retrofit later.
5. Set up centralized logging early. Even a simple ELK or Loki setup saves hours of debugging.
6. Create actionable alerts. Every alert should have a runbook and a clear owner.
7. Review what you log. Periodically audit log statements to ensure no sensitive data has crept in.
8. Log the "why," not just the "what." Instead of "Retrying request," log "Retrying request, attempt 2/3, previous failure: connection timeout, backoff: 400ms."

Quick Summary

Logging and observability transform your application from a black box into a system you can understand, debug, and operate with confidence. Replace println debugging with structured logging using proper log levels. Never log sensitive data. Use correlation IDs to trace requests across services. Combine logs with metrics and traces for complete observability. Set up centralized logging and meaningful alerts so problems are found before customers report them.

Key Points

- Structured logging (JSON) makes logs searchable and machine-readable.
- Log levels (DEBUG, INFO, WARN, ERROR) are a filtering mechanism, not decoration. Use them consistently.
- Never log sensitive data: passwords, tokens, credit cards, PII.
- Correlation IDs let you trace a single request across multiple services.
- Metrics answer "how much," logs answer "what happened," traces answer "where did time go."
- Log rotation prevents disk space disasters.
- Centralized logging (ELK, Loki, Datadog) is essential for multi-server deployments.
- Alerts must be actionable. If nobody needs to respond, it should not be an alert.

Practice Questions

1. You are debugging a production issue where a customer's order failed. Your logs show the error, but you cannot determine which of the 50 concurrent requests it belongs to. What concept would solve this problem, and how would you implement it?
2. A junior developer adds `log.info("User password: " + password)` to the authentication service for debugging. Explain why this is dangerous and what they should log instead.
3. Your team receives 80 alerts per day and has started ignoring them. Three real incidents were missed last month. What steps would you take to fix the alerting system?
4. Explain the difference between metrics, logs, and traces. Give a specific scenario where each one is the best tool for the job.
5. Your production log file is 200GB and the disk is 95% full. The application

is at risk of crashing. What immediate steps do you take, and what long-term solution do you implement?

Exercises

Exercise 1: Add Proper Logging to a Service

Take the following println-based code and refactor it to use proper logging with appropriate levels, structured context, and safe practices.

```java
public class UserService {
    public User register(String email, String password, String name) {
        System.out.println("Registering user: " + email +
                           ", password: " + password);
        try {
            User user = new User(email, password, name);
            database.save(user);
            System.out.println("User saved: " + user);
            emailService.sendWelcome(email);
            System.out.println("Welcome email sent");
            return user;
        } catch (Exception e) {
            System.out.println("Registration failed: " + e);
            return null;
        }
    }
}
```

Exercise 2: Design a Logging Strategy

You are building an e-commerce platform with the following services: API Gateway, User Service, Product Service, Order Service, Payment Service, and Notification Service. Design a complete logging strategy covering: what each service should log, what log levels to use, how to implement correlation IDs, what metrics to track, and what alerts to set up.

Exercise 3: Set Up Structured Logging

Create a small application (in Java or Python) that:

1. Uses structured JSON logging
2. Includes a correlation ID middleware/filter

3. Logs request start and completion with timing
4. Demonstrates proper use of all log levels
5. Includes at least two custom metrics (counter, histogram)

What Is Next?

Now that you understand how to make your systems observable, the next chapter tackles a concept every developer encounters but few handle well: Technical Debt. You will learn how shortcuts taken today become interest payments tomorrow, how to measure and communicate debt, and when taking on debt is actually the right decision.

Chapter 28: Technical Debt

What You Will Learn

- What technical debt is and why the metaphor matters
- The difference between intentional and unintentional debt
- The four types of debt: code, architecture, test, and documentation
- Martin Fowler's Technical Debt Quadrant (reckless/prudent x deliberate/inadvertent)
- How to measure and track technical debt
- Strategies for paying it down: refactoring sprints, the boy scout rule, and the strangler fig pattern
- How to communicate debt to non-technical stakeholders
- When taking on debt is the right decision

Why This Chapter Matters

Every software project accumulates shortcuts. A quick hack to meet a deadline. A copy-paste instead of a proper abstraction. A test that was "going to be written later." These shortcuts are not free. Like financial debt, they accrue interest: every future change takes longer, bugs appear more frequently, and new developers take longer to become productive.

The term "technical debt" was coined by Ward Cunningham specifically so that developers could explain this phenomenon to business stakeholders in terms they understand. If you cannot articulate why your team is slowing down, you will never get the time to fix the underlying problems.

This chapter teaches you to recognize debt, measure it, communicate it, and pay it down systematically -- without stopping feature development.

Section 1: What Is Technical Debt?

Technical debt is the implied cost of future rework caused by choosing a quick or easy solution now instead of a better approach that would take longer.

The Financial Metaphor

```
+-----------------------------------------------------------------+
|  Financial Debt vs Technical Debt                               |
+-----------------------------------------------------------------+
|                                                                 |
|  FINANCIAL DEBT              TECHNICAL DEBT                     |
|  --------------              ---------------                    |
|  Borrow money                Take a shortcut                    |
|  Get something now           Ship a feature faster              |
|  Pay interest over time      Spend extra time on every          |
|                              future change                      |
|  Pay down principal          Refactor the shortcut properly     |
|  Go bankrupt                 Project becomes unmaintainable     |
|                                                                 |
|  KEY INSIGHT: Just like financial debt, technical debt          |
|  is not inherently bad. A mortgage lets you live in a           |
|  house while paying for it. A startup shipping an MVP           |
|  with shortcuts lets them validate the market before            |
|  investing in perfect code.                                     |
|                                                                 |
|  The problem is UNMANAGED debt.                                 |
+-----------------------------------------------------------------+
```

A Concrete Example

```java
// The "quick" solution (taking on debt)
public class OrderService {
    public void processOrder(Order order) {
        // TODO: Extract to separate validator class
        // TODO: Add proper validation rules
        if (order.getTotal() > 0 && order.getItems() != null) {
            // TODO: Handle multiple payment methods
            // Hardcoded to credit card for now
            chargeCreditCard(order);

            // TODO: Make this async
            // TODO: Support SMS notifications
            sendEmail(order.getCustomerEmail(), "Order confirmed");
```

```
            // Direct database call, no repository pattern
            // TODO: Add caching
            database.execute(
                "INSERT INTO orders VALUES ('" + order.getId() + "', " +
                "'" + order.getCustomerName() + "', " +
                order.getTotal() + ")"
            );
        }
    }
}
```

Every one of those TODO comments represents debt. Each one means that the next developer who touches this code will have to work around limitations, and each future feature (like adding PayPal support or SMS notifications) will require untangling this mess first.

The Interest Compounds

```
+-----------------------------------------------------------------+
|  How Technical Debt Compounds Over Time                         |
+-----------------------------------------------------------------+
|                                                                 |
|  Velocity                                                       |
|  ^                                                              |
|  |  * * *                                                       |
|  |        * * *            <-- Without debt                     |
|  |              * * * * *                                       |
|  |                                                              |
|  |  * * *                                                       |
|  |        * *                                                   |
|  |           *  *          <-- With accumulating debt           |
|  |               *  *                                           |
|  |                   *                                          |
|  |                    * *                                       |
|  +----------------------------------------------> Time        |
|                                                                 |
|  At first, shortcuts make you FASTER.                           |
|  Over time, they make you MUCH SLOWER.                          |
+-----------------------------------------------------------------+
```

Section 2: Intentional vs Unintentional Debt

Not all technical debt is created equal. Understanding the difference helps you manage it.

Intentional Debt (Deliberate)

You know you are taking a shortcut. You make a conscious decision to do so.

```python
# Intentional debt: hardcoded for launch, refactor in sprint 3
# TECH-DEBT: Ticket JIRA-1234 - Replace with configurable tax rates
TAX_RATE = 0.08  # Only works for California

class PriceCalculator:
    def calculate_total(self, subtotal: float) -> float:
        # TECH-DEBT: JIRA-1234 - Need tax lookup by state
        tax = subtotal * TAX_RATE
        return subtotal + tax
```

This is managed debt. You know it exists, you have a ticket for it, and you plan to fix it.

Unintentional Debt (Accidental)

You did not realize you were creating a problem. This happens when:

- You did not know a better approach existed
- Requirements changed and the design no longer fits
- The codebase grew beyond the original architecture's capacity

```java
// Unintentional debt: seemed fine for 10 products,
// breaks down at 10,000 products
public List<Product> searchProducts(String query) {
    List<Product> allProducts = productRepository.findAll();
    List<Product> results = new ArrayList<>();

    for (Product product : allProducts) {
        if (product.getName().toLowerCase()
                    .contains(query.toLowerCase())) {
            results.add(product);
        }
    }
    return results;
}
// Nobody realized this loads ALL products into memory.
// It worked fine with 50 products. Now there are 50,000.
```

Section 3: The Technical Debt Quadrant

Martin Fowler's Technical Debt Quadrant classifies debt along two dimensions: reckless vs prudent, and deliberate vs inadvertent.

```
+-----------------------------------------------------------------+
|  Martin Fowler's Technical Debt Quadrant                        |
+-----------------------------------------------------------------+
|                                                                 |
|              DELIBERATE              INADVERTENT                 |
|          (We know it's debt)    (We didn't realize)              |
|                                                                 |
|  RECKLESS  "We don't have       "What's layered                 |
|            time for design."     architecture?"                  |
|                                                                 |
|            - Skipping tests      - Junior dev doesn't            |
|              because deadline      know patterns                 |
|            - Copy-pasting        - No code review                |
|              everywhere            caught it                     |
|            - No error handling   - Accidental coupling           |
|                                                                 |
|  --------+----------------------+-------------------------     |
|                                                                 |
|  PRUDENT   "We must ship now     "Now we know how we             |
|            and deal with the      should have done it."          |
|            consequences."                                         |
|                                                                 |
|            - Hardcoded config    - Outgrew the original          |
|              for MVP launch        architecture                  |
|            - Monolith before     - Learned better patterns       |
|              microservices         after shipping                 |
|            - Manual deployment   - Requirements changed          |
|              for first release     the ideal design              |
|                                                                 |
+-----------------------------------------------------------------+
```

Reckless-Deliberate is the most dangerous: you know you are making a mess and do not care. This leads to projects that become unmaintainable.

Prudent-Deliberate is often the best strategy: you make a conscious trade-off, document it, and plan to fix it.

Prudent-Inadvertent is unavoidable: you only learn the right design after you have built the wrong one. This is normal and expected.

Reckless-Inadvertent is prevented through learning, mentorship, and code reviews.

Section 4: Types of Technical Debt

Code Debt

The most visible kind. Messy code, poor naming, duplicated logic, long methods, god classes.

```java
// BEFORE: Code debt - duplicated validation logic
public class UserController {
    public Response createUser(UserRequest req) {
        if (req.getEmail() == null || req.getEmail().isEmpty()) {
            return Response.error("Email required");
        }
        if (!req.getEmail().contains("@")) {
            return Response.error("Invalid email");
        }
        if (req.getName() == null || req.getName().length() < 2) {
            return Response.error("Name must be at least 2 characters");
        }
        // ... create user
    }

    public Response updateUser(UserRequest req) {
        // SAME validation copied here
        if (req.getEmail() == null || req.getEmail().isEmpty()) {
            return Response.error("Email required");
        }
        if (!req.getEmail().contains("@")) {
            return Response.error("Invalid email");
        }
        if (req.getName() == null || req.getName().length() < 2) {
            return Response.error("Name must be at least 2 characters");
        }
        // ... update user
    }
}
```

```java
// AFTER: Debt paid off - extracted validation
public class UserValidator {
    public ValidationResult validate(UserRequest req) {
        List<String> errors = new ArrayList<>();

        if (req.getEmail() == null || req.getEmail().isEmpty()) {
            errors.add("Email is required");
        } else if (!req.getEmail().contains("@")) {
            errors.add("Email format is invalid");
        }

        if (req.getName() == null || req.getName().length() < 2) {
            errors.add("Name must be at least 2 characters");
        }

        return errors.isEmpty()
            ? ValidationResult.valid()
```

```
            : ValidationResult.invalid(errors);
    }
}
```

Architecture Debt

The system's structure no longer fits its needs.

```
+-----------------------------------------------------------------+
|  Architecture Debt Example                                      |
+-----------------------------------------------------------------+
|                                                                 |
|  What we built (Year 1):           What we need (Year 3):       |
|                                                                 |
|  +------------------+              +---------+  +---------+     |
|  |   Monolith       |              | Orders |  | Users  |     |
|  |   - Orders       |              +---------+  +---------+     |
|  |   - Users        |              +---------+  +---------+     |
|  |   - Products     |     ==>      |Products|  |Payments|     |
|  |   - Payments     |              +---------+  +---------+     |
|  |   - Notifications|              +---------+                  |
|  +------------------+              | Notify |                  |
|                                    +---------+                  |
|  Works fine at 100 req/s           Need 10,000 req/s for        |
|                                    products, only 10 req/s      |
|                                    for user registration        |
+-----------------------------------------------------------------+
```

Test Debt

Missing tests, brittle tests, slow tests, or no tests at all.

```python
# Test debt: no tests for critical business logic
class DiscountCalculator:
    def calculate(self, order, customer):
        # 15 complex discount rules
        # No tests for any of them
        # Nobody dares change this code
        # Last person who tried broke checkout for 4 hours
        if customer.is_premium and order.total > 100:
            discount = order.total * 0.15
        elif customer.order_count > 10:
            discount = order.total * 0.10
        # ... 13 more rules ...
        return discount
```

Documentation Debt

No README, no architecture decision records, no API documentation, tribal knowledge that exists only in people's heads.

```
+-----------------------------------------------------------------+
|  Documentation Debt Symptoms                                    |
+-----------------------------------------------------------------+
|                                                                 |
|  "Ask Sarah, she wrote that part."                              |
|  "I think this config does something important, don't touch." |
|  "The onboarding process takes 3 months."                       |
|  "Nobody knows why we have two user tables."                    |
|  "The deployment process is in a Slack thread from 2022."       |
|                                                                 |
+-----------------------------------------------------------------+
```

Section 5: Measuring Technical Debt

You cannot manage what you cannot measure. Here are practical ways to quantify debt.

Code-Level Metrics

```python
# Python - Simple cyclomatic complexity checker concept
# Real tools: radon (Python), SonarQube (Java), CodeClimate

# These metrics indicate potential debt:
#
# 1. Cyclomatic Complexity
#    Low debt:  1-10 per method
#    Moderate:  11-20 per method
#    High debt: 21+ per method
#
# 2. Code Duplication
#    Low debt:  < 3% duplicate blocks
#    Moderate:  3-5%
#    High debt: > 5%
#
# 3. Method Length
#    Low debt:  < 20 lines
#    Moderate:  20-50 lines
#    High debt: > 50 lines
#
```

```
# 4. Test Coverage (for critical paths)
#    Low debt:  > 80%
#    Moderate:  50-80%
#    High debt: < 50%
```

Team-Level Indicators

```
+-----------------------------------------------------------------+
|  Signs Your Team Is Drowning in Debt                            |
+-----------------------------------------------------------------+
|                                                                 |
|  METRIC                           HEALTHY       DEBT-LADEN      |
|  ------                           -------       ----------      |
|  Time to onboard new dev          1-2 weeks     2-3 months      |
|  Bug fix time (average)           Hours         Days/weeks      |
|  Feature delivery estimate        Predictable   "It depends"    |
|  Ratio of bug fixes to features   20/80         60/40           |
|  Deployment frequency             Daily/weekly  Monthly+        |
|  Fear of changing code            Low           High            |
|  "Just don't touch that file"     Never said    Said weekly      |
|                                                                 |
+-----------------------------------------------------------------+
```

Tracking Debt Formally

```java
// Java - Tag debt in code with annotations
@TechnicalDebt(
    description = "Tax calculation is hardcoded to US rates",
    type = DebtType.CODE,
    severity = Severity.MEDIUM,
    ticket = "JIRA-4567",
    createdDate = "2024-01-15",
    estimatedEffort = "3 days"
)
public BigDecimal calculateTax(BigDecimal amount, String country) {
    // Only handles US tax rates
    return amount.multiply(new BigDecimal("0.08"));
}
```

```python
# Python - Track debt with structured comments and a registry
# TECH-DEBT: [MEDIUM] [JIRA-4567] [3 days]
# Tax calculation is hardcoded to US rates.
# Need to support EU VAT and other tax jurisdictions.
def calculate_tax(amount: float, country: str) -> float:
    return amount * 0.08  # Only US rates
```

Keep a tech debt register (a spreadsheet or board is fine):

```
+-----------------------------------------------------------------+
|  Technical Debt Register                                        |
+-----------------------------------------------------------------+
|                                                                 |
|  ID    | Area        | Severity | Effort | Impact  | Status   |
|  ------+-------------+----------+--------+---------+--------  |
|  TD-01 | Tax calc    | Medium   | 3 days | Orders  | Open     |
|  TD-02 | Search      | High     | 5 days | Perf    | Planned  |
|  TD-03 | Auth module | Critical | 2 wks  | Securty | In prog  |
|  TD-04 | No CI/CD    | High     | 1 week | Deploy  | Open     |
|  TD-05 | Tests: Cart | Medium   | 3 days | Quality | Open     |
|                                                                 |
+-----------------------------------------------------------------+
```

Section 6: Paying Down Debt

Strategy 1: The Boy Scout Rule

"Leave the code cleaner than you found it." -- Robert C. Martin

Every time you touch a file, make one small improvement. Rename a variable. Extract a method. Add a missing test. Over time, the most-touched files get steadily cleaner.

```python
# BEFORE: You are adding a new feature to this file
# and notice the poor naming
def proc(d):
    t = 0
    for i in d:
        if i['s'] == 'A':
            t += i['p'] * i['q']
    return t

# AFTER: You clean it up while you are here anyway
def calculate_active_order_total(orders):
    total = 0
    for order in orders:
        if order['status'] == 'ACTIVE':
            total += order['price'] * order['quantity']
    return total
```

This works because the files you touch most are the files that matter most.

Strategy 2: Refactoring Sprints

Dedicate a percentage of each sprint to debt reduction. Common ratios:

```
+-----------------------------------------------------------------+
|  Sprint Allocation Models                                       |
+-----------------------------------------------------------------+
|                                                                 |
|  80/20 Rule:  80% features, 20% debt reduction                 |
|  (Most common, sustainable, easy to justify)                    |
|                                                                 |
|  Debt Sprint: Every 4th sprint is 100% debt reduction           |
|  (Good for catching up, harder to justify to business)          |
|                                                                 |
|  Continuous: Each ticket includes time for related cleanup      |
|  (Best long-term, but requires discipline)                      |
|                                                                 |
+-----------------------------------------------------------------+
```

Strategy 3: Strangler Fig Pattern

Gradually replace the old system with new code instead of a risky "big bang" rewrite.

```
+-----------------------------------------------------------------+
|  Strangler Fig Pattern                                          |
+-----------------------------------------------------------------+
|                                                                 |
|  Phase 1: All traffic goes to old system                        |
|                                                                 |
|  Request --> [Old Monolith]                                     |
|                                                                 |
|  Phase 2: New feature routes to new service                     |
|                                                                 |
|  Request --> [Router] --orders--> [Old Monolith]                |
|                       --search--> [New Search Service]          |
|                                                                 |
|  Phase 3: More routes migrated                                  |
|                                                                 |
|  Request --> [Router] --orders--> [New Order Service]           |
|                       --search--> [New Search Service]          |
|                       --users---> [Old Monolith]                |
|                                                                 |
|  Phase 4: Old system fully replaced                             |
|                                                                 |
|  Request --> [Router] --orders--> [New Order Service]           |
|                       --search--> [New Search Service]          |
|                       --users---> [New User Service]            |
|                                                                 |
|  Old monolith decommissioned.                                   |
+-----------------------------------------------------------------+
```

```java
// Java - Strangler fig with a routing facade
public class OrderFacade {
    private final LegacyOrderService legacyService;
    private final NewOrderService newService;
    private final FeatureFlags featureFlags;

    public OrderResult processOrder(Order order) {
        if (featureFlags.isEnabled("new-order-processing")) {
            // New path: gradually migrate traffic
            return newService.processOrder(order);
        } else {
            // Old path: legacy system
            return legacyService.processOrder(order);
        }
    }
}
```

```python
# Python - Strangler fig with gradual migration
class OrderFacade:
    def __init__(self, legacy_service, new_service, feature_flags):
        self.legacy_service = legacy_service
        self.new_service = new_service
        self.feature_flags = feature_flags

    def process_order(self, order):
        if self.feature_flags.is_enabled("new-order-processing"):
            return self.new_service.process_order(order)
        else:
            return self.legacy_service.process_order(order)
```

Section 7: Communicating Debt to Non-Technical Stakeholders

This is one of the most important skills a developer can have. Business people do not understand (and should not need to understand) code quality. But they understand money, risk, and speed.

Language That Works

```
+-----------------------------------------------------------------+
|  How to Translate Technical Debt for Business People           |
```

```
+-----------------------------------------------------------------+
|                                                                 |
|  DON'T SAY                    DO SAY                            |
|  ---------                    ------                            |
|  "The code is messy"          "New features take 3x longer      |
|                                than they should"                |
|                                                                 |
|  "We need to refactor"        "We can cut development time      |
|                                by 40% with a 2-week             |
|                                investment"                       |
|                                                                 |
|  "We have technical debt"     "We have a $200K/year hidden      |
|                                cost that's growing"             |
|                                                                 |
|  "The architecture is wrong"  "We cannot scale past 1000        |
|                                users without changes"           |
|                                                                 |
|  "We skipped tests"           "Each release has a 30% chance    |
|                                of a customer-facing bug"        |
|                                                                 |
|  "We need to rewrite this"    "We're spending 60% of our time   |
|                                working around limitations"      |
|                                                                 |
+-----------------------------------------------------------------+
```

Quantify the Cost

Instead of saying "we have technical debt," calculate what it actually costs:

```
+-----------------------------------------------------------------+
|  Debt Cost Calculation Example                                  |
+-----------------------------------------------------------------+
|                                                                 |
|  Feature that should take:        3 developer-days              |
|  Feature actually takes:          9 developer-days              |
|  Overhead per feature:            6 developer-days              |
|                                                                 |
|  Features delivered per quarter:  12                            |
|  Total overhead per quarter:      72 developer-days             |
|                                                                 |
|  Average developer daily cost:    $600                          |
|  Quarterly cost of debt:          $43,200                       |
|  Annual cost of debt:             $172,800                      |
|                                                                 |
|  Cost to fix (one-time):          $30,000 (2 devs, 5 weeks)  |
|  ROI payback period:              ~2 months                     |
|                                                                 |
|  NOW the business understands.                                  |
+-----------------------------------------------------------------+
```

Section 8: When Debt Is Acceptable

Technical debt is not always bad. Sometimes it is the right strategic choice.

Good Reasons to Take On Debt

```
+-----------------------------------------------------------------+
|  When Technical Debt Makes Sense                                |
+-----------------------------------------------------------------+
|                                                                 |
|  SCENARIO                    RATIONALE                          |
|  --------                    ---------                          |
|  MVP / prototype             Validate idea before investing     |
|                              in quality code                    |
|                                                                 |
|  Competitive deadline        Missing a market window costs      |
|                              more than the debt interest        |
|                                                                 |
|  Throwaway code              Proof of concept that will be      |
|                              rewritten regardless               |
|                                                                 |
|  Learning / exploration      First version teaches you what     |
|                              the right design should be         |
|                                                                 |
|  Emergency hotfix            Stop the bleeding first,           |
|                              proper fix in next sprint          |
|                                                                 |
+-----------------------------------------------------------------+
```

Bad Reasons to Take On Debt

```
+-----------------------------------------------------------------+
|  When Technical Debt Is Just Laziness                            |
+-----------------------------------------------------------------+
|                                                                 |
|  "We don't have time for tests"                                 |
|  --> You don't have time NOT to write tests. Bugs cost more.    |
|                                                                 |
|  "Nobody will ever change this code"                            |
|  --> They will. They always will.                               |
|                                                                 |
|  "It works, so why clean it up?"                                |
|  --> It works TODAY. The next person who touches it will         |
|      spend hours understanding it.                              |
|                                                                 |
```

```
|  "We'll fix it later"                                          |
|  --> Later never comes unless you put it on the backlog        |
|      with a priority and a date.                                |
|                                                                 |
+-----------------------------------------------------------------+
```

The Golden Rule

If you take on debt deliberately, you must:

1. Document it (code comment + ticket in your tracker)
2. Estimate the interest (how much will it slow us down?)
3. Set a pay-down date (when will we fix it?)
4. Get agreement (the team agrees to the trade-off)

Common Mistakes

1. Ignoring debt until it is too late. By the time the team can barely ship features, the cost to fix everything is enormous.
2. Treating all debt as equal. A hardcoded config value is not the same as a fundamental architecture problem. Prioritize by impact and cost.
3. Trying to fix everything at once. A "stop the world and rewrite" approach almost always fails. Pay debt incrementally.
4. Not tracking debt. If debt is not in your issue tracker, it does not exist in the eyes of planning. "I'll remember" is not a strategy.
5. Using "technical debt" to mean "code I don't like." Debt is about trade-offs that cost you in the future, not stylistic preferences.
6. Never communicating debt to stakeholders. If the business does not understand why things are slowing down, they will not allocate time to fix the root causes.
7. Taking on reckless debt habitually. Prudent, deliberate debt with a paydown plan is fine. "We don't have time for design" is not.

Best Practices

1. Make debt visible. Track it in your issue tracker with clear labels, estimated effort, and business impact.
2. Pay continuously. Use the boy scout rule and allocate 15-20% of each sprint to debt reduction.
3. Prioritize by interest rate. Fix the debt that slows you down the most first, not the debt that annoys you the most.
4. Communicate in business terms. Translate technical problems into cost, risk, and speed.
5. Use the strangler fig pattern for large debts. Replace systems incrementally rather than attempting big-bang rewrites.
6. Document deliberate debt. Every intentional shortcut needs a comment, a ticket, and a planned resolution date.
7. Prevent reckless debt through code reviews. A good review process catches shortcuts before they become permanent.
8. Celebrate debt reduction. Make paying down debt as valued as shipping features.

Quick Summary

Technical debt is the future cost of shortcuts taken today. Like financial debt, it can be strategic (an MVP that validates a market) or reckless (skipping tests because you cannot be bothered). The key is to make it visible, track it, communicate it in business terms, and pay it down continuously. Use the boy scout rule for small improvements, dedicated sprints for medium debts, and the strangler fig pattern for large architectural debts. Never let debt accumulate silently.

Key Points

- Technical debt = shortcuts now, interest payments later. The longer debt remains unpaid, the more it costs.
- Intentional debt is manageable. Track it, estimate it, plan to fix it.
- Unintentional debt is prevented through learning, code reviews, and experience.
- The debt quadrant helps classify debt: reckless/prudent x deliberate/inadvertent.
- Four types of debt: code, architecture, test, and documentation.
- Measure debt using code metrics, team velocity trends, and cost calculations.
- Communicate in business language: cost, risk, speed -- not "the code is messy."
- Pay debt continuously: boy scout rule, 20% sprint allocation, strangler fig for large changes.

Practice Questions

1. A product manager asks, "Why can't we just add the new payment method? It should only take a day." The real answer is that the payment module was hardcoded for credit cards three years ago and has accumulated significant debt. How do you explain this in business terms?
2. Your team inherits a codebase with no tests, inconsistent naming, and no documentation. Classify this debt using the Technical Debt Quadrant. What paydown strategy would you recommend?
3. A startup founder says, "We don't have time for clean code; we need to ship the MVP by next month." Is this a valid position? Under what conditions? What should they do after the MVP ships?
4. You calculate that technical debt is costing your team 60 developer-days per quarter. A complete refactoring would take 40 developer-days but would eliminate 80% of the overhead. Build the business case.

5. Compare the boy scout rule, refactoring sprints, and the strangler fig pattern. When is each strategy most appropriate?

Exercises

Exercise 1: Debt Audit

Take a project you are working on (or an open-source project you know) and perform a technical debt audit:

1. Identify at least 5 instances of technical debt
2. Classify each one on the debt quadrant
3. Estimate the "interest rate" (how much time does it cost per month?)
4. Prioritize them by business impact
5. Propose a paydown plan

Exercise 2: Communicate Debt to Stakeholders

Write a one-page summary for a non-technical product manager explaining:

- What technical debt the team has accumulated
- How much it is costing in developer time and quality
- What the proposed solution is
- What the ROI of fixing it would be

Use only business language. No code, no jargon.

Exercise 3: Strangler Fig Implementation

Given this legacy function, implement a strangler fig migration:

```python
# Legacy function: does too much, hard to test, no error handling
def process_order(order_data):
    # Validates, calculates price, charges payment, sends email,
    # saves to database -- all in one giant function (200 lines)
    ...
```

1. Create an interface/facade that can route to old or new code
2. Extract the validation logic into a new, well-tested module

3. Keep the rest routing to the legacy code
4. Write tests that prove the new validation matches the old behavior

---

What Is Next?

You now understand how to recognize, measure, and manage technical debt. In the next chapter, we tackle one of the most powerful tools for preventing debt from accumulating in the first place: Code Review Culture. You will learn how to give and receive feedback effectively, what to look for in a review, and how to build a team culture where reviews are valued, not dreaded.

Chapter 29: Code Review Culture

What You Will Learn

- Why code reviews matter more than most teams realize
- What to look for in a code review (and what NOT to focus on)
- How to give constructive feedback that improves code AND relationships
- How to receive feedback without taking it personally
- Optimal pull request size and how to keep PRs small
- A practical code review checklist
- How to use automated checks to free reviewers for higher-level thinking

Why This Chapter Matters

Code review is not about catching bugs. Well, it catches some bugs. But its real value runs much deeper.

Code review is how a team maintains quality standards. It is how junior developers learn from seniors, and how seniors discover blind spots they did not know they had. It is how knowledge spreads across the team so that no one person is the only one who understands a critical system. It is how your codebase stays consistent instead of becoming a patchwork of different styles and approaches.

Teams without code review gradually accumulate inconsistencies, shortcuts, and knowledge silos. One developer writes beautiful code. Another writes spaghetti. Nobody notices until the spaghetti developer goes on vacation and nobody can maintain their code.

Code review is the team's immune system. This chapter teaches you how to make it effective, constructive, and sustainable.

Why Code Reviews Matter

```
THE BENEFITS OF CODE REVIEW:

  +-------------------------------------------------------+
  |                                                       |
  |  1. KNOWLEDGE SHARING                                 |
  |     Every review teaches something. The author       |
  |     learns from feedback. The reviewer learns        |
  |     from reading the code.                           |
  |                                                       |
  |  2. CONSISTENCY                                       |
  |     Reviews enforce shared standards. The codebase   |
  |     reads like it was written by one team, not       |
  |     twelve individuals.                              |
  |                                                       |
  |  3. BUG PREVENTION                                    |
  |     A second pair of eyes catches logic errors,      |
  |     edge cases, and security issues that tests miss. |
  |                                                       |
  |  4. DESIGN IMPROVEMENT                                |
  |     Reviewers spot over-engineering, missed          |
  |     abstractions, and simpler alternatives.          |
  |                                                       |
  |  5. TEAM OWNERSHIP                                    |
  |     When everyone reviews each other's code,         |
  |     the entire team owns the entire codebase.        |
  |     No knowledge silos.                              |
  |                                                       |
  +-------------------------------------------------------+
```

The Cost of Skipping Reviews

```
WITHOUT CODE REVIEW:

  Developer A                Developer B
  +------------------+      +------------------+
  | Uses camelCase   |      | Uses snake_case  |
  | Handles errors   |      | Ignores errors   |
  | Writes tests     |      | Skips tests      |
  | Small methods    |      | 200-line methods |
  +------------------+      +------------------+

  6 months later:
  +-----------------------------------------+
  | Codebase is a mess                      |
  | - Mixed naming conventions              |
  | - Some parts tested, some not           |
  | - Nobody can maintain Developer B's code |
```

```
  | - New hires are confused by the chaos    |
  +-------------------------------------------+

WITH CODE REVIEW:

  Developer A reviews B    Developer B reviews A
  +------------------+      +------------------+
  | "Use camelCase   |      | "This abstraction|
  |  like the rest   |      |  is clever --    |
  |  of the project" |      |  let me learn    |
  |                  |      |  this approach"  |
  | "Add error       |      |                  |
  |  handling here"  |      | "Why not use the |
  |                  |      |  factory pattern |
  | "This needs a    |      |  here?"          |
  |  test"           |      |                  |
  +------------------+      +------------------+

  6 months later:
  +-------------------------------------------+
  | Codebase is consistent                    |
  | - Unified conventions                     |
  | - Comprehensive test coverage             |
  | - Both developers can maintain all code   |
  | - New hires onboard quickly               |
  +-------------------------------------------+
```

What to Look for in a Code Review

Not all review comments are equally valuable. Focus on the things that matter most:

```
REVIEW PRIORITY PYRAMID:

                    /\
                   /  \
                  / BUG \
                 / LOGIC \        <-- HIGHEST PRIORITY
                / SECURITY \       Correctness and safety
               /   ISSUES   \
              +--------------+
              |   DESIGN     |    <-- HIGH PRIORITY
              |  Structure,  |     Architecture, patterns
              |  abstractions|
              +--------------+
              | MAINTAINAB.  |    <-- MEDIUM PRIORITY
              | Readability, |     Names, complexity
              | clarity      |
              +--------------+
```

```
               |    STYLE     |    <-- LOW PRIORITY
               | Formatting,  |     Automate this!
               | whitespace   |
               +--------------+

  Spend most of your review energy at the TOP of the pyramid.
  Automate the BOTTOM with linters and formatters.
```

High-Priority Review Items

1. Correctness

Does the code actually do what it is supposed to do? Check edge cases and boundary conditions.

```java
// REVIEW CATCH: Off-by-one error
// The developer wrote:
for (int i = 0; i <= items.size(); i++) {  // Bug: <= should be <
    process(items.get(i));  // ArrayIndexOutOfBoundsException!
}

// Review comment:
// "This loop will throw on the last iteration. Use i < items.size()"
```

2. Error Handling

What happens when things go wrong? Are errors handled, or silently swallowed?

```python
# REVIEW CATCH: Swallowed exception
try:
    result = external_api.call(data)
except Exception:
    pass  # Silently swallowed! Bug reports will be impossible.

# Review comment:
# "This catches and ignores all exceptions. At minimum, log the
# error. Better: handle specific exceptions and re-raise unexpected ones."
```

3. Security

Is user input validated? Are there SQL injection or XSS vulnerabilities?

```java
// REVIEW CATCH: SQL injection
String query = "SELECT * FROM users WHERE name = '" + userName + "'";
// If userName is "'; DROP TABLE users; --", disaster.

// Review comment:
// "Use parameterized queries to prevent SQL injection:
// PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE name = ?");
// ps.setString(1, userName);"
```

4. Design

Is the solution well-structured? Is it too complex? Too clever?

```python
# REVIEW CATCH: Over-engineering
class AbstractStrategyFactoryProvider:
    def create_strategy_factory(self, context):
        return StrategyFactory(context.get_provider())

# Review comment:
# "This adds two layers of indirection for a feature that
# has exactly one implementation. Can we simplify to a direct
# function call? We can add abstraction when we need it."
```

Low-Priority Items (Automate These)

- Formatting: Spaces, tabs, line length. Use a formatter (Prettier, Black, google-java-format).
- Import order: Use an import sorter.
- Naming conventions: Use a linter rule.
- Unused variables: Use a linter.

```
AUTOMATE THE STYLE ARGUMENTS:

  +-----------+                   +-----------+
  | Developer | "You used tabs!" | Developer |
  |     A     |<---------------->|     B     |
  +-----------+ "Spaces are      +-----------+
                 better!"

  This argument has been going on since 1970.
  End it with a formatter.

  +-----------+     +----------+     +-----------+
  | Developer |---->| Prettier |---->| Formatted |
  |     A     |     | / Black  |     | Code      |
  +-----------+     +----------+     +-----------+
```

```
  Nobody argues with the formatter. Problem solved.
```

Giving Feedback: The Art of Constructive Review

How you say something matters as much as what you say. A review comment can teach and inspire, or it can demoralize and antagonize.

Principles for Giving Feedback

1. Comment on the Code, Not the Person

```
BAD:  "You clearly don't understand how exceptions work."
GOOD: "This catch block swallows the exception. Consider logging
       it or re-throwing it so we can diagnose failures."

BAD:  "This is wrong."
GOOD: "This will throw a NullPointerException if the user has no
       email set. Could we add a null check here?"

BAD:  "Why would you do it this way?"
GOOD: "I think using a Map here instead of a switch would make
       it easier to add new cases. What do you think?"
```

2. Explain the WHY, Not Just the WHAT

```
BAD:  "Use final here."
GOOD: "Making this field final ensures it cannot be reassigned
       after construction, which prevents a category of bugs
       where the dependency is accidentally replaced."

BAD:  "Extract a method."
GOOD: "This block of 30 lines handles validation, and the rest
       of the method handles processing. Extracting validate()
       would make the flow clearer and let us test validation
       independently."
```

3. Ask Questions Instead of Making Demands

```
DEMANDING: "Change this to use a stream."
ASKING:    "Have you considered using a stream here?
```

```
             It might make the filtering more readable."

DEMANDING: "This needs to be refactored."
ASKING:    "Do you think this method is doing too much?
             I count three distinct responsibilities."
```

4. Distinguish Between Blocking and Non-Blocking Comments

```
BLOCKING (must fix before merge):
"Bug: This will cause a NullPointerException when user.getEmail()
 returns null, which happens for unverified accounts."

NON-BLOCKING (nice to have, author decides):
"Nit: Could rename `proc` to `processPayment` for clarity.
 Not a blocker -- feel free to take or leave."

"Optional: We could use an enum here instead of string constants.
 Happy to merge as-is if you prefer."

Use prefixes:
  "Bug:"      -- Must fix
  "Security:" -- Must fix
  "Nit:"      -- Nice to have
  "Optional:" -- Author's choice
  "Question:" -- Asking for understanding
```

5. Praise Good Code

Reviews should not be only about problems. Call out things the author did well.

```
"Nice use of the builder pattern here. It makes the test
 setup much more readable than the previous approach."

"Great test coverage! The edge case for empty lists
 is exactly the kind of thing that catches real bugs."

"I learned something from this approach. I hadn't
 considered using Optional.flatMap() for this case."
```

Receiving Feedback: Checking Your Ego

Receiving code review feedback is hard. Your code is a reflection of your thinking, and criticism can feel personal. Here is how to handle it well:

Principles for Receiving Feedback

```
HOW TO RECEIVE FEEDBACK WELL:

  1. ASSUME GOOD INTENT
     The reviewer is trying to make the code better,
     not attack you personally.

  2. SEPARATE CODE FROM IDENTITY
     "This code could be clearer" is not
     "You are not a good developer."

  3. ASK QUESTIONS, DON'T DEFEND
     If you disagree, ask: "Can you explain why
     you prefer that approach?" -- not "My way is fine."

  4. SAY THANK YOU
     "Good catch, I missed that edge case."
     "I hadn't thought of that approach, thanks!"

  5. PICK YOUR BATTLES
     If the suggestion is minor and you have no strong
     opinion, just accept it. Save your energy for
     things that matter.
```

How to Respond to Different Types of Comments

```
TYPE: Bug report
  Comment: "This will NPE when email is null."
  Response: "Good catch! Fixed in the latest commit."

TYPE: Design suggestion
  Comment: "Have you considered using a strategy pattern here?"
  Response: "Interesting idea. I went with the switch because
             there are only two cases today. If we add more,
             I agree the strategy pattern would be better.
             Want me to refactor now or add a TODO?"

TYPE: Nit
  Comment: "Nit: rename `x` to `orderTotal`."
  Response: "Done." (No need for a long discussion.)

TYPE: Disagreement
  Comment: "I think we should use inheritance here."
  Response: "I chose composition because [specific reason].
             But I see the inheritance angle. Let's discuss
             in standup if you feel strongly about it."
```

Pull Request Size: Smaller Is Better

The single most impactful thing you can do for code review quality is to keep pull requests small.

```
PR SIZE AND REVIEW QUALITY:

  Lines Changed    Review Quality    Time to Review
  +-------------+-----------------+-----------------+
  |    < 100    | Thorough,       | 15-30 minutes   |
  |             | catches bugs    |                 |
  +-------------+-----------------+-----------------+
  |  100 - 300  | Good, some      | 30-60 minutes   |
  |             | things missed   |                 |
  +-------------+-----------------+-----------------+
  |  300 - 500  | Surface-level,  | 60-90 minutes   |
  |             | many things     |                 |
  |             | missed          |                 |
  +-------------+-----------------+-----------------+
  |    > 500    | "LGTM" stamp,   | 2 minutes       |
  |             | nothing actually | (reviewer gives |
  |             | reviewed        |  up)            |
  +-------------+-----------------+-----------------+

  The sweet spot: 100-300 lines changed.
  Anything over 500 lines is almost never reviewed properly.
```

Strategies for Keeping PRs Small

1. One concern per PR. A PR should do one thing: fix one bug, add one feature, or refactor one area.
2. Separate refactoring from features. If you need to refactor before adding a feature, make two PRs: one for the refactoring, one for the feature.
3. Stack PRs. Break a large feature into sequential PRs that each build on the last.
4. Ship the skeleton first. Create the interfaces and structure first, then fill in the implementation.

```
STACKING PRs:

  Instead of one massive PR:

  +-----------------------------------------+
  | PR #42: Add checkout feature            |
  | - New database tables                   |
  | - Repository classes                    |
  | - Service layer                         |
```

```
| - Controller                              |
| - Frontend changes                        |
| - Tests                                   |
| 1,200 lines changed                       |
+-------------------------------------------+

Stack smaller PRs:

PR #42: Add checkout database schema (80 lines)
              |
PR #43: Add checkout repository + tests (120 lines)
              |
PR #44: Add checkout service + tests (150 lines)
              |
PR #45: Add checkout controller + tests (100 lines)
              |
PR #46: Add checkout frontend (200 lines)

Each PR is reviewable. Each can be merged independently.
```

The Code Review Checklist

Use this checklist when reviewing a pull request:

```
CODE REVIEW CHECKLIST:

  FUNCTIONALITY:
  [ ] Code does what the PR description says it does
  [ ] Edge cases are handled (null, empty, boundary values)
  [ ] Error paths are handled appropriately
  [ ] No unintended side effects

  DESIGN:
  [ ] Follows existing patterns in the codebase
  [ ] No unnecessary complexity or over-engineering
  [ ] Responsibilities are in the right places
  [ ] Dependencies point in the right direction

  READABILITY:
  [ ] Names are clear and descriptive
  [ ] Complex logic has explanatory comments
  [ ] Methods are focused (single responsibility)
  [ ] No dead code or commented-out code

  TESTING:
  [ ] New code has tests
  [ ] Tests cover happy path AND error paths
  [ ] Tests are readable and maintainable
```

```
  [ ] Edge cases are tested

  SECURITY:
  [ ] User input is validated and sanitized
  [ ] No hardcoded secrets or credentials
  [ ] Authentication/authorization is enforced
  [ ] No SQL injection or XSS vulnerabilities

  PERFORMANCE:
  [ ] No obvious N+1 query issues
  [ ] Large collections are not loaded entirely into memory
  [ ] No unnecessary database calls in loops

  PR QUALITY:
  [ ] PR description explains WHAT and WHY
  [ ] PR is a reasonable size (< 300 lines ideal)
  [ ] Related changes are grouped logically
  [ ] No unrelated changes mixed in
```

Automated Checks: Let Robots Handle the Boring Stuff

Automated checks handle consistency and style so reviewers can focus on logic, design, and correctness.

```
WHAT TO AUTOMATE vs WHAT HUMANS DO:

  AUTOMATE (CI/CD pipeline):            HUMAN REVIEW:
  +-------------------------------+     +----------------------------+
  | - Code formatting             |     | - Business logic           |
  | - Linting rules               |     |   correctness              |
  | - Import ordering             |     | - Design decisions         |
  | - Unit test execution         |     | - Architecture fit         |
  | - Code coverage thresholds    |     | - Naming clarity           |
  | - Static analysis             |     | - Error handling strategy  |
  | - Dependency vulnerability    |     | - Performance concerns     |
  |   scanning                    |     | - Security review          |
  | - Build verification          |     | - Knowledge sharing        |
  | - Type checking               |     | - Mentoring and teaching   |
  +-------------------------------+     +----------------------------+
```

Setting Up Automated Checks

```
CI/CD PIPELINE FOR CODE REVIEW:

  Developer pushes PR
         |
         v
  +-------------------+
  | 1. Lint           |  Checks naming, style, import order
  +-------------------+
         |
         v
  +-------------------+
  | 2. Format Check   |  Verifies code is formatted
  +-------------------+
         |
         v
  +-------------------+
  | 3. Build          |  Compiles and resolves dependencies
  +-------------------+
         |
         v
  +-------------------+
  | 4. Unit Tests     |  Runs all tests, reports coverage
  +-------------------+
         |
         v
  +-------------------+
  | 5. Static Analysis|  FindBugs, SonarQube, mypy, etc.
  +-------------------+
         |
         v
  +-------------------+
  | 6. Security Scan  |  Dependency vulnerabilities
  +-------------------+
         |
  All pass? --> Human review begins
  Any fail? --> Author fixes before review
```

Java CI Example

```yaml
# .github/workflows/pr-checks.yml
name: PR Checks
on: [pull_request]

jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Format check
```

```
        run: mvn spotless:check

      - name: Build and test
        run: mvn verify

      - name: Static analysis
        run: mvn spotbugs:check

      - name: Coverage check
        run: |
          mvn jacoco:report
          # Fail if coverage drops below 80%
```

Python CI Example

```yaml
# .github/workflows/pr-checks.yml
name: PR Checks
on: [pull_request]

jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install dependencies
        run: pip install -r requirements.txt -r requirements-dev.txt

      - name: Format check
        run: black --check .

      - name: Lint
        run: ruff check .

      - name: Type check
        run: mypy src/

      - name: Tests with coverage
        run: pytest --cov=src --cov-fail-under=80
```

Code Review Workflow

A healthy code review workflow looks like this:

```
CODE REVIEW WORKFLOW:

  1. AUTHOR prepares:
     - Write clean, small PR (< 300 lines)
     - Add description: WHAT changed, WHY, HOW to test
     - Self-review before requesting reviews
     - Ensure CI passes

  2. REVIEWER reviews:
     - Read the PR description first
     - Understand the context and goal
     - Review the code, focusing on the pyramid
     - Leave clear, constructive comments
     - Approve, request changes, or comment

  3. AUTHOR responds:
     - Address all comments
     - Fix blocking issues
     - Discuss non-blocking suggestions
     - Request re-review if changes were significant

  4. MERGE:
     - All reviewers approve
     - CI passes
     - Squash or merge per team convention

  Timeline goal: Review within 24 hours of request.
  Stale PRs lose context and become harder to review.
```

Writing a Good PR Description

```
PR DESCRIPTION TEMPLATE:

  ## What
  Brief description of what this PR does.

  ## Why
  Why this change is needed. Link to issue/ticket.

  ## How
  High-level approach. What alternatives were considered?

  ## Testing
  How was this tested? What scenarios were covered?

  ## Screenshots (if UI change)
  Before and after screenshots.

  ## Checklist
  - [ ] Tests added
  - [ ] Documentation updated
```

```
  - [ ] No breaking changes
```

Common Mistakes

Mistake 1: Rubber-Stamp Reviews

Approving PRs without reading them. This defeats the entire purpose of code review.

Fix: Set a minimum review time expectation. If a PR took hours to write, it deserves more than 30 seconds of review.

Mistake 2: Nitpicking Without Substance

Leaving 20 comments about formatting while missing a critical bug.

Fix: Use the review priority pyramid. Automate style checks. Focus on logic and design.

Mistake 3: Gatekeeping

One senior developer blocks all PRs with personal preferences disguised as "standards."

Fix: Document standards explicitly. If it is not in the style guide, it is a preference, not a requirement. Preferences should be suggestions, not blockers.

Mistake 4: Enormous PRs

PRs with 1,000+ lines that nobody can actually review properly.

Fix: Set a soft limit (e.g., 300 lines). If a PR is larger, the author must explain why and consider splitting it.

Mistake 5: Delayed Reviews

PRs sitting for days without review, losing context and blocking the author.

Fix: Set a team SLA (e.g., first review within 24 hours). Use a rotation or assign reviewers proactively.

Best Practices

1. Automate style and formatting. Never argue about tabs vs spaces again. Let the formatter decide.
2. Keep PRs small. Under 300 lines is ideal. Over 500 lines is a red flag.
3. Review within 24 hours. Stale PRs lose context and block other work.
4. Comment on the code, not the person. Always assume good intent.
5. Explain the WHY. A comment that explains reasoning teaches more than a comment that just says "change this."
6. Distinguish blocking from non-blocking. Use prefixes like "Bug:", "Nit:", "Optional:" to set expectations.
7. Praise good work. Reviews should not be only about problems.
8. Self-review before requesting. Read your own PR as if someone else wrote it. You will catch obvious issues.
9. Write clear PR descriptions. Tell reviewers what, why, and how to test.
10. Rotate reviewers. Spread knowledge across the team. Everyone should review and be reviewed.

Quick Summary

```
CODE REVIEW CULTURE AT A GLANCE:

  Purpose:  Knowledge sharing, consistency, quality, team ownership

  Focus:    Logic and bugs > Design > Readability > Style

  PR Size:  < 300 lines ideal, > 500 lines is a red flag

  Feedback: Constructive, explains WHY, asks questions,
            comments on code not person

  Automate: Formatting, linting, tests, static analysis

  Timeline: First review within 24 hours
```

Key Points

- Code review is primarily about knowledge sharing and consistency, not just bug catching.
- Focus review energy on correctness, security, and design. Automate style checks.
- Give feedback that is constructive, specific, and explains the reasoning.
- Receive feedback with curiosity, not defensiveness. Assume good intent.
- Keep pull requests under 300 lines for thorough reviews. Over 500 lines means reviewers give up.
- Use prefixes (Bug, Nit, Optional) to distinguish blocking from non-blocking comments.
- Automate everything that can be automated: formatting, linting, tests, static analysis, security scans.
- Review within 24 hours. Stale PRs lose context and block progress.
- Write clear PR descriptions that explain what, why, and how to test.
- Praise good code. Reviews should recognize excellent work, not just problems.

Practice Questions

1. A developer on your team submits a 1,500-line PR that adds a complete new feature. The code looks correct, but you cannot review it thoroughly. What do you do?
2. You receive a review comment that says "This is wrong." No explanation, no suggestion. How do you respond constructively?
3. Your team spends 40% of review time debating code formatting (tabs vs spaces, brace placement). How would you eliminate these debates?
4. A junior developer's PR has correct logic but uses a different pattern than the rest of the codebase. How do you give feedback that teaches without discouraging?
5. Your team has no code review process. Management says it would "slow things down." What arguments would you make for introducing code reviews?

Exercises

Exercise 1: Review This Code

Review the following code and write review comments. For each comment, indicate whether it is blocking or non-blocking, and explain why.

```java
public class UserService {
    public User createUser(String name, String email, String pw) {
        User u = new User();
        u.setName(name);
        u.setEmail(email);
        u.setPassword(pw);  // stored as plain text
        db.save(u);
        try {
            emailService.sendWelcome(email);
        } catch (Exception e) {
            // ignore
        }
        return u;
    }
}
```

Exercise 2: Rewrite These Comments

Rewrite these code review comments to be constructive and helpful:

1. "This is terrible code."
2. "Why would you use a for loop here?"
3. "You obviously didn't test this."
4. "This is not how we do things."
5. "Wrong."

Exercise 3: Set Up Automated Checks

Configure a CI pipeline for a project of your choice that runs the following checks on every PR: code formatting verification, linting, unit tests with coverage reporting (minimum 80%), and static analysis. Document which tools you chose and why.

What Is Next?

With a strong code review culture in place, the next chapter tackles one of the hardest challenges in software development: working with Legacy Code. You will learn techniques for safely modifying code you did not write and do not fully understand -- characterization tests, the strangler fig pattern, branch by abstraction, and more.

Chapter 30: Working with Legacy Code

What You Will Learn

- What legacy code really is (hint: it is not just "old code")
- Characterization tests: documenting what code actually does before changing it
- The Strangler Fig pattern: gradually replacing a legacy system without a big-bang rewrite
- Branch by Abstraction: swapping implementations behind an interface
- The Sprout Method/Class technique: adding new behavior without modifying risky code
- When to rewrite vs when to refactor
- How to make untestable legacy code testable

Why This Chapter Matters

Every developer will eventually face legacy code. Maybe you inherited a codebase from a developer who left. Maybe your own code from three years ago has become legacy. Maybe you joined a company with a ten-year-old monolith that nobody fully understands.

Michael Feathers defined legacy code simply: code without tests. Without tests, you cannot change the code with confidence. Every modification is a gamble. You fix one bug and introduce two more. You add a feature and break something unrelated. You refactor and hope for the best.

This chapter gives you practical techniques for working safely with code you did not write and do not fully understand. These techniques let you make changes with confidence, gradually improve the codebase, and avoid the temptation of the catastrophic big-bang rewrite.

What Is Legacy Code?

```
LEGACY CODE IS NOT JUST "OLD CODE":

  Old code WITH tests,       Old code WITHOUT tests,
  clear structure,           no documentation,
  good documentation:        tangled dependencies:
  +------------------+       +------------------+
  | Mature Code      |       | Legacy Code      |
  | (not legacy)     |       | (dangerous to    |
  |                  |       |  change)         |
  +------------------+       +------------------+

  New code WITHOUT tests,
  written last month,
  no one understands:
  +------------------+
  | Also Legacy Code |
  | (age does not    |
  |  matter)         |
  +------------------+

  LEGACY CODE = Code you are afraid to change.

  The fear comes from:
  - No tests to catch regressions
  - No documentation to explain intent
  - Tangled dependencies that make changes ripple
  - Knowledge silos (only one person understood it)
```

Step 1: Understand Before You Change

Before modifying legacy code, you must understand what it currently does. Not what it should do, not what the documentation says -- what it actually does right now.

Reading Strategies

```
STRATEGIES FOR UNDERSTANDING LEGACY CODE:

  1. START FROM THE OUTSIDE
     Find the entry points (controllers, main methods,
     API endpoints). Trace the flow inward.

  2. DRAW THE DEPENDENCY MAP
```

```
      +----------+     +-----------+     +--------+
      | Controller| --> | Service   | --> | DAO    |
      |          | --> | Validator | --> | Cache  |
      |          |     | Helper    |     | Logger |
      +----------+     +-----------+     +--------+
      Who calls whom? Who depends on what?

   3. FIND THE HOTSPOTS
      Use version control to find files that change
      most often. Frequently changed files are where
      bugs live and where understanding matters most.

      git log --format=format: --name-only | sort | uniq -c | sort -rn | head -20

   4. LOOK AT THE TESTS (if any exist)
      Tests are documentation. They show how the code
      is expected to behave.

   5. TALK TO PEOPLE
      Former developers, users, support staff -- anyone
      who knows how the system behaves.
```

Step 2: Characterization Tests

A characterization test documents the current behavior of the code, whether that behavior is correct or not. It is not about testing what the code should do -- it is about pinning down what it actually does.

Why Characterization Tests?

```
THE CHARACTERIZATION TEST MINDSET:

  Normal test:           Characterization test:
  "The code SHOULD       "The code CURRENTLY
   return 42"             returns 42"

  If the test fails:     If the test fails:
  "The code has a bug"   "I accidentally changed
                          the behavior"

  Purpose:               Purpose:
  Verify correctness     Detect unintended changes
```

Java Example

```java
// You have this legacy method. You do not fully understand it.
// You need to modify it but are afraid of breaking something.
public class LegacyTaxCalculator {

    public double calculateTax(double amount, String stateCode, boolean isExempt) {
        if (isExempt) return 0;
        double rate = 0.0;
        if (stateCode.equals("CA")) rate = 0.0725;
        else if (stateCode.equals("TX")) rate = 0.0625;
        else if (stateCode.equals("NY")) rate = 0.08;
        else if (stateCode.equals("OR")) rate = 0.0;
        else rate = 0.05;

        double tax = amount * rate;
        if (amount > 1000) tax *= 0.95;  // Mysterious discount?

        return Math.round(tax * 100.0) / 100.0;
    }
}

// CHARACTERIZATION TESTS: Pin down the current behavior
class LegacyTaxCalculatorTest {

    private LegacyTaxCalculator calculator = new LegacyTaxCalculator();

    @Test
    void california_standard_tax() {
        // We OBSERVED this result. We document it, not judge it.
        assertEquals(7.25, calculator.calculateTax(100.0, "CA", false));
    }

    @Test
    void california_over_1000_gets_discount() {
        // Discovered: amounts over $1000 get 5% off the tax
        assertEquals(103.36, calculator.calculateTax(1500.0, "CA", false));
    }

    @Test
    void exempt_returns_zero() {
        assertEquals(0.0, calculator.calculateTax(500.0, "CA", true));
    }

    @Test
    void texas_standard_tax() {
        assertEquals(6.25, calculator.calculateTax(100.0, "TX", false));
    }

    @Test
    void unknown_state_uses_default_rate() {
        // Default rate is 5%
        assertEquals(5.0, calculator.calculateTax(100.0, "ZZ", false));
    }
```

```
    @Test
    void oregon_has_no_tax() {
        assertEquals(0.0, calculator.calculateTax(100.0, "OR", false));
    }

    @Test
    void zero_amount() {
        assertEquals(0.0, calculator.calculateTax(0.0, "CA", false));
    }

    @Test
    void negative_amount() {
        // Discovered: negative amounts produce negative tax!
        // This might be a bug, but we document the current behavior.
        assertEquals(-7.25, calculator.calculateTax(-100.0, "CA", false));
    }
}
```

```python
# CHARACTERIZATION TESTS in Python
# Pin down current behavior before making any changes

class LegacyTaxCalculator:
    def calculate_tax(self, amount, state_code, is_exempt):
        if is_exempt:
            return 0
        rates = {"CA": 0.0725, "TX": 0.0625, "NY": 0.08, "OR": 0.0}
        rate = rates.get(state_code, 0.05)
        tax = amount * rate
        if amount > 1000:
            tax *= 0.95  # Mysterious discount
        return round(tax, 2)


# Characterization tests
def test_california_standard():
    calc = LegacyTaxCalculator()
    assert calc.calculate_tax(100.0, "CA", False) == 7.25

def test_california_over_1000_gets_discount():
    calc = LegacyTaxCalculator()
    assert calc.calculate_tax(1500.0, "CA", False) == 103.36

def test_exempt_returns_zero():
    calc = LegacyTaxCalculator()
    assert calc.calculate_tax(500.0, "CA", True) == 0

def test_unknown_state_uses_default():
    calc = LegacyTaxCalculator()
    assert calc.calculate_tax(100.0, "ZZ", False) == 5.0

def test_negative_amount():
    calc = LegacyTaxCalculator()
```

```
    # Documenting current behavior, not asserting correctness
    assert calc.calculate_tax(-100.0, "CA", False) == -7.25
```

Writing Characterization Tests: The Process

```
HOW TO WRITE CHARACTERIZATION TESTS:

  1. Pick an input you want to understand
  2. Call the method with that input
  3. Observe the actual output
  4. Write a test that asserts the actual output

  Do NOT guess what the output should be.
  Run the code and OBSERVE what it does.

  Repeat with different inputs until you have
  covered the important paths through the code.

  +-----+     +--------+     +--------+     +------+
  | Pick|---->| Call   |---->| Observe|---->| Write|
  |Input|     | Method |     | Output |     | Test |
  +-----+     +--------+     +--------+     +------+
     ^                                          |
     |                                          |
     +------------- repeat ---------------------+
```

The Strangler Fig Pattern

Named after the strangler fig tree that slowly grows around another tree until it replaces it entirely. Apply the same idea to legacy systems: gradually replace parts while the old system continues running.

```
STRANGLER FIG PATTERN:

  Phase 1: New system handles 0%, old handles 100%

  +--------+     +------------------+
  | Users  |---->| OLD SYSTEM       |
  |        |     | (100% of traffic)|
  +--------+     +------------------+

  Phase 2: Route SOME requests to new system
```

```
+--------+     +----------+     +------------------+
| Users  |---->| ROUTER   |---->| OLD SYSTEM       |
|        |     |          |     | (80% of traffic) |
+--------+     |          |     +------------------+
               |          |
               |          |     +------------------+
               |          +---->| NEW SYSTEM       |
               +----------+     | (20% of traffic) |
                                +------------------+

Phase 3: New system handles most traffic

+--------+     +----------+     +------------------+
| Users  |---->| ROUTER   |---->| OLD SYSTEM       |
|        |     |          |     | (10% of traffic) |
+--------+     |          |     +------------------+
               |          |
               |          |     +------------------+
               |          +---->| NEW SYSTEM       |
               +----------+     | (90% of traffic) |
                                +------------------+

Phase 4: Old system decommissioned

+--------+     +------------------+
| Users  |---->| NEW SYSTEM       |
|        |     | (100% of traffic)|
+--------+     +------------------+
```

Java Example: Strangler Fig with a Router

```java
// STEP 1: Create a router that delegates to old or new system
public class OrderServiceRouter implements OrderService {

    private final LegacyOrderService legacyService;
    private final NewOrderService newService;
    private final FeatureFlags featureFlags;

    public OrderServiceRouter(LegacyOrderService legacyService,
                              NewOrderService newService,
                              FeatureFlags featureFlags) {
        this.legacyService = legacyService;
        this.newService = newService;
        this.featureFlags = featureFlags;
    }

    @Override
    public Order placeOrder(OrderRequest request) {
        if (featureFlags.isEnabled("new_order_service")) {
            return newService.placeOrder(request);
```

```
        }
        return legacyService.placeOrder(request);
    }

    @Override
    public Order getOrder(String orderId) {
        // Maybe migrate one method at a time
        // getOrder uses new system already
        return newService.getOrder(orderId);
    }

    @Override
    public void cancelOrder(String orderId) {
        // cancelOrder still uses legacy
        legacyService.cancelOrder(orderId);
    }
}
```

```python
# STEP 1: Create a router that delegates to old or new system
class OrderServiceRouter:
    def __init__(self, legacy_service, new_service, feature_flags):
        self.legacy_service = legacy_service
        self.new_service = new_service
        self.feature_flags = feature_flags

    def place_order(self, request):
        if self.feature_flags.is_enabled("new_order_service"):
            return self.new_service.place_order(request)
        return self.legacy_service.place_order(request)

    def get_order(self, order_id):
        # Already migrated to new system
        return self.new_service.get_order(order_id)

    def cancel_order(self, order_id):
        # Still on legacy
        self.legacy_service.cancel_order(order_id)
```

Branch by Abstraction

Branch by Abstraction lets you replace an implementation behind an interface without creating long-lived feature branches in version control.

```
BRANCH BY ABSTRACTION:
```

```
Step 1: Code directly uses the legacy implementation

+----------+     +-------------------+
| Client   |---->| LegacyPaymentProc |
| Code     |     | (concrete class)  |
+----------+     +-------------------+

Step 2: Insert an interface (abstraction layer)

+----------+     +----------------+     +-------------------+
| Client   |---->| PaymentProc    |<----| LegacyPaymentProc |
| Code     |     | (interface)    |     | (implements)      |
+----------+     +----------------+     +-------------------+

Step 3: Build new implementation alongside

+----------+     +----------------+     +-------------------+
| Client   |---->| PaymentProc    |<----| LegacyPaymentProc |
| Code     |     | (interface)    |     +-------------------+
+----------+     +-------+--------+
                         |              +-------------------+
                         +<-------------| NewPaymentProc    |
                                        | (new, in progress)|
                                        +-------------------+

Step 4: Switch to new implementation and remove legacy

+----------+     +----------------+     +-------------------+
| Client   |---->| PaymentProc    |<----| NewPaymentProc    |
| Code     |     | (interface)    |     | (complete)        |
+----------+     +----------------+     +-------------------+

The client code NEVER changes. Only the wiring in the
Composition Root changes.
```

Java Example

```java
// STEP 1: Legacy code is used directly everywhere
// public class OrderService {
//     private final LegacyPaymentProcessor processor = new LegacyPaymentProcessor();
//     ...
// }

// STEP 2: Extract interface
public interface PaymentProcessor {
    PaymentResult processPayment(String customerId, Money amount);
    void refund(String transactionId, Money amount);
}

// STEP 3: Legacy class implements the interface
```

```
public class LegacyPaymentProcessor implements PaymentProcessor {
    // Existing code, unchanged
    @Override
    public PaymentResult processPayment(String customerId, Money amount) {
        // All the legacy code stays exactly the same
        // ...
    }

    @Override
    public void refund(String transactionId, Money amount) {
        // ...
    }
}

// STEP 4: Build new implementation
public class StripePaymentProcessor implements PaymentProcessor {
    private final StripeClient stripe;

    public StripePaymentProcessor(StripeClient stripe) {
        this.stripe = stripe;
    }

    @Override
    public PaymentResult processPayment(String customerId, Money amount) {
        // Clean, new implementation
        var charge = stripe.charges().create(
            customerId, amount.toCents(), amount.getCurrency()
        );
        return new PaymentResult(charge.getId(), PaymentStatus.COMPLETED);
    }

    @Override
    public void refund(String transactionId, Money amount) {
        stripe.refunds().create(transactionId, amount.toCents());
    }
}

// STEP 5: Switch in Composition Root
// Before: new OrderService(new LegacyPaymentProcessor())
// After:  new OrderService(new StripePaymentProcessor(stripeClient))
```

```python
# STEP 2: Extract interface (using Protocol in Python)
from typing import Protocol

class PaymentProcessor(Protocol):
    def process_payment(self, customer_id: str, amount: float) -> dict: ...
    def refund(self, transaction_id: str, amount: float) -> None: ...

# STEP 3: Legacy class (unchanged, already matches the protocol)
class LegacyPaymentProcessor:
    def process_payment(self, customer_id, amount):
        # All the legacy code, untouched
```

```
        ...

    def refund(self, transaction_id, amount):
        ...

# STEP 4: New implementation
class StripePaymentProcessor:
    def __init__(self, stripe_client):
        self.stripe = stripe_client

    def process_payment(self, customer_id, amount):
        charge = self.stripe.charges.create(
            customer=customer_id, amount=int(amount * 100)
        )
        return {"id": charge.id, "status": "completed"}

    def refund(self, transaction_id, amount):
        self.stripe.refunds.create(charge=transaction_id, amount=int(amount * 100))

# STEP 5: Switch in Composition Root
# Before: order_service = OrderService(LegacyPaymentProcessor())
# After:  order_service = OrderService(StripePaymentProcessor(stripe_client))
```

Sprout Method and Sprout Class

When you need to add new behavior to legacy code, the Sprout technique lets you write new, clean code without modifying the risky legacy code.

Sprout Method

```java
// LEGACY METHOD: 150 lines of tangled code
// You need to add validation, but touching this method is risky.
public class LegacyOrderProcessor {

    public void processOrder(Order order) {
        // 150 lines of legacy code that you cannot easily test
        // and are afraid to modify
        // ...
    }
}

// SPROUT METHOD: Write new behavior in a new, tested method
public class LegacyOrderProcessor {

    public void processOrder(Order order) {
```

```
        // NEW: Call the sprouted method at the right point
        validateOrderForProcessing(order);

        // All the original 150 lines stay untouched
        // ...
    }

    // SPROUTED METHOD: New, clean, fully tested
    // This method was written with TDD -- tests first.
    void validateOrderForProcessing(Order order) {
        if (order.getItems().isEmpty()) {
            throw new EmptyOrderException("Order must have at least one item");
        }
        if (order.getTotal().isNegative()) {
            throw new InvalidOrderException("Order total cannot be negative");
        }
        for (OrderItem item : order.getItems()) {
            if (item.getQuantity() <= 0) {
                throw new InvalidOrderException(
                    "Item quantity must be positive: " + item.getProductId()
                );
            }
        }
    }
}
```

```python
# SPROUT METHOD in Python
class LegacyOrderProcessor:

    def process_order(self, order):
        # NEW: Call the sprouted method
        self._validate_order_for_processing(order)

        # All the original legacy code stays untouched
        # ...

    # SPROUTED METHOD: New, clean, fully tested
    def _validate_order_for_processing(self, order):
        if not order.items:
            raise EmptyOrderError("Order must have at least one item")
        if order.total < 0:
            raise InvalidOrderError("Order total cannot be negative")
        for item in order.items:
            if item.quantity <= 0:
                raise InvalidOrderError(
                    f"Item quantity must be positive: {item.product_id}"
                )
```

Sprout Class

When the new behavior is complex enough to warrant its own class:

```java
// SPROUT CLASS: Extract new behavior into a separate, clean class
public class OrderFraudDetector {
    private final FraudRuleEngine ruleEngine;
    private final CustomerRiskProfile riskProfile;

    public OrderFraudDetector(FraudRuleEngine ruleEngine,
                              CustomerRiskProfile riskProfile) {
        this.ruleEngine = ruleEngine;
        this.riskProfile = riskProfile;
    }

    public FraudCheckResult checkForFraud(Order order) {
        // Clean, well-tested code
        RiskScore score = riskProfile.scoreFor(order.getCustomerId());
        List<FraudRule> violations = ruleEngine.evaluate(order);

        if (score.isHigh() || violations.size() > 2) {
            return FraudCheckResult.flagged(violations);
        }
        return FraudCheckResult.clear();
    }
}

// Then add ONE line to the legacy code:
public class LegacyOrderProcessor {
    public void processOrder(Order order) {
        // ONE new line -- minimal change to legacy code
        FraudCheckResult fraudCheck = fraudDetector.checkForFraud(order);
        if (fraudCheck.isFlagged()) {
            throw new FraudDetectedException(fraudCheck.getViolations());
        }

        // Original 150 lines unchanged
        // ...
    }
}
```

Making Legacy Code Testable

The biggest obstacle to testing legacy code is dependencies. Legacy classes often create their dependencies internally, making it impossible to substitute test doubles.

Technique: Extract and Override

```java
// LEGACY: Untestable because it creates its own dependencies
public class InvoiceGenerator {

    public Invoice generate(Order order) {
        // Creates its own connection -- cannot test without a real database
        DatabaseConnection db = new DatabaseConnection("prod-db:5432");
        TaxRate taxRate = db.getTaxRate(order.getState());

        // Creates its own email client -- cannot test without SMTP server
        EmailClient email = new EmailClient("smtp.company.com");

        Invoice invoice = new Invoice(order, taxRate);
        email.send(order.getCustomerEmail(), invoice.toPdf());
        return invoice;
    }
}

// STEP 1: Extract dependency creation into protected methods
public class InvoiceGenerator {

    public Invoice generate(Order order) {
        DatabaseConnection db = createDatabaseConnection();
        TaxRate taxRate = db.getTaxRate(order.getState());

        EmailClient email = createEmailClient();

        Invoice invoice = new Invoice(order, taxRate);
        email.send(order.getCustomerEmail(), invoice.toPdf());
        return invoice;
    }

    // Protected so tests can override
    protected DatabaseConnection createDatabaseConnection() {
        return new DatabaseConnection("prod-db:5432");
    }

    protected EmailClient createEmailClient() {
        return new EmailClient("smtp.company.com");
    }
}

// STEP 2: In tests, override the factory methods
class InvoiceGeneratorTest {

    @Test
    void generates_invoice_with_correct_tax() {
        // Create a testable subclass that returns fakes
        InvoiceGenerator generator = new InvoiceGenerator() {
            @Override
            protected DatabaseConnection createDatabaseConnection() {
                return new FakeDatabaseConnection();
            }
            @Override
            protected EmailClient createEmailClient() {
```

```
                return new FakeEmailClient();
            }
        };

        Invoice invoice = generator.generate(testOrder);
        assertEquals(expectedTotal, invoice.getTotal());
    }
}
```

```python
# LEGACY: Untestable
class InvoiceGenerator:
    def generate(self, order):
        db = DatabaseConnection("prod-db:5432")
        tax_rate = db.get_tax_rate(order.state)
        email = EmailClient("smtp.company.com")
        invoice = Invoice(order, tax_rate)
        email.send(order.customer_email, invoice.to_pdf())
        return invoice

# STEP 1: Extract dependency creation
class InvoiceGenerator:
    def generate(self, order):
        db = self._create_db_connection()
        tax_rate = db.get_tax_rate(order.state)
        email = self._create_email_client()
        invoice = Invoice(order, tax_rate)
        email.send(order.customer_email, invoice.to_pdf())
        return invoice

    def _create_db_connection(self):
        return DatabaseConnection("prod-db:5432")

    def _create_email_client(self):
        return EmailClient("smtp.company.com")

# STEP 2: In tests, override or use dependency injection
def test_generates_invoice_with_correct_tax():
    generator = InvoiceGenerator()
    generator._create_db_connection = lambda: FakeDatabaseConnection()
    generator._create_email_client = lambda: FakeEmailClient()

    invoice = generator.generate(test_order)
    assert invoice.total == expected_total
```

When to Rewrite vs When to Refactor

```
REWRITE vs REFACTOR DECISION:

  REFACTOR when:                       REWRITE when:
  +--------------------------------+   +--------------------------------+
  | - Core logic is sound          |   | - Technology is obsolete       |
  | - Architecture is salvageable |   |   (no security updates)        |
  | - Team understands the domain |   | - Architecture fundamentally   |
  | - Business cannot pause for    |   |   wrong (cannot be evolved)    |
  |   a rewrite                    |   | - Codebase is tiny enough      |
  | - Risk tolerance is low        |   |   to rewrite quickly           |
  |                                |   | - Business can tolerate        |
  |                                |   |   parallel development         |
  +--------------------------------+   +--------------------------------+

  THE BIG REWRITE TRAP:

  Plan:     "We'll rewrite from scratch in 6 months!"
  Reality:  +----+----+----+----+----+----+----+----+-->
            Month 1: "Going great!"
            Month 3: "Harder than expected..."
            Month 6: "Not done yet, but old system got
                       6 months of new features we
                       now need to implement too"
            Month 9: "We are trying to hit a moving target"
            Month 12: "Project cancelled. Back to the
                        old system."

  PREFER: Strangler Fig pattern. Replace piece by piece.
          The old system keeps running. The new system
          grows gradually. No big-bang cutover.
```

Common Mistakes

Mistake 1: Changing Legacy Code Without Tests First

Making changes to code you do not have tests for. When things break, you will not know if the bug is old or new.

Fix: Write characterization tests before making any changes. Pin down the current behavior, then modify with confidence.

Mistake 2: The Big-Bang Rewrite

Stopping all feature development to rewrite the entire system from scratch. This almost always fails because the old system keeps getting new requirements.

Fix: Use the Strangler Fig pattern. Replace the system incrementally while both systems run in parallel.

Mistake 3: Trying to Understand Everything First

Spending weeks reading every line of legacy code before making any changes. You will forget the beginning by the time you reach the end.

Fix: Focus on the area you need to change. Write characterization tests for that area. Understand locally, not globally.

Mistake 4: Refactoring Without a Safety Net

Refactoring legacy code without tests. "I'll just clean this up" leads to subtle behavior changes.

Fix: Characterization tests first. Always. No exceptions.

Mistake 5: Gold-Plating the Legacy Code

Trying to make legacy code perfect. Rewriting entire modules that are stable and working just because they are "ugly."

Fix: Only improve code you need to change. If ugly code works and nobody needs to modify it, leave it alone. Focus your energy where it provides value.

Best Practices

1. Write characterization tests before changing anything. Document what the code does before modifying it.
2. Use the Strangler Fig pattern for system-level replacements. Avoid big-bang rewrites.
3. Use Branch by Abstraction for component-level replacements. Insert an interface, build behind it, switch.
4. Use Sprout Method/Class for adding new features. Write new code cleanly without touching risky legacy code.
5. Make the smallest possible change. Each change should be independently testable and deployable.
6. Improve code you touch, leave the rest. The Boy Scout Rule applied to legacy code.
7. Extract and Override to break dependencies for testing. Move `new` calls into overridable methods.

8. Use version control hotspot analysis to find the most frequently changed files. Those are your highest-value targets for improvement.

Quick Summary

```
WORKING WITH LEGACY CODE AT A GLANCE:

  Technique              When to use it
  +--------------------+-------------------------------------+
  | Characterization   | Before ANY change to legacy code.   |
  | Tests              | Pin down current behavior.          |
  +--------------------+-------------------------------------+
  | Strangler Fig      | Replacing an entire system          |
  |                    | gradually. No big-bang rewrite.     |
  +--------------------+-------------------------------------+
  | Branch by          | Replacing a component. Insert       |
  | Abstraction        | interface, build new, switch.       |
  +--------------------+-------------------------------------+
  | Sprout Method/     | Adding new behavior. Write clean    |
  | Sprout Class       | code, call it from legacy code.     |
  +--------------------+-------------------------------------+
  | Extract and        | Making legacy code testable by      |
  | Override           | moving dependency creation into     |
  |                    | overridable methods.                |
  +--------------------+-------------------------------------+
```

Key Points

- Legacy code is code you are afraid to change, usually because it lacks tests. Age is not the defining factor.
- Characterization tests document what code actually does, not what it should do. Write them before making any changes.
- The Strangler Fig pattern replaces a system gradually: route traffic to the new system piece by piece while the old system continues running.
- Branch by Abstraction lets you replace implementations behind interfaces without long-lived feature branches.
- Sprout Method and Sprout Class let you add new, clean, tested code without modifying risky legacy code.

- Extract and Override breaks hard-coded dependencies by moving `new` calls into overridable factory methods.
- Prefer refactoring over rewriting. Big-bang rewrites almost always fail because the old system is a moving target.
- Only improve code you need to change. Stable, working legacy code that nobody touches does not need to be "cleaned up."

Practice Questions

1. You need to add a new validation rule to a 500-line method that has no tests. Describe the steps you would take, in order, to make this change safely.
2. Your team wants to replace a legacy payment system with a new one. The legacy system processes 10,000 transactions per day. Would you recommend a big-bang rewrite or the Strangler Fig pattern? Why?
3. What is the difference between a characterization test and a unit test? When would you write each?
4. A legacy class creates a `new DatabaseConnection()` inside its constructor, making it impossible to test without a real database. Describe two techniques for making this class testable.
5. A colleague says: "This legacy code is terrible. Let's spend a month cleaning it all up." What are the risks of this approach, and what would you suggest instead?

Exercises

Exercise 1: Write Characterization Tests

Given this legacy method, write at least six characterization tests that document its current behavior:

```java
public double calculateShipping(double weight, String destination, boolean express) {
    double base = weight * 0.5;
    if (destination.startsWith("US")) {
        base += 5.0;
    } else if (destination.startsWith("EU")) {
```

```
        base += 15.0;
    } else {
        base += 25.0;
    }
    if (express) {
        base *= 2;
    }
    if (weight > 50) {
        base += 10;  // heavy surcharge
    }
    return Math.round(base * 100.0) / 100.0;
}
```

Exercise 2: Apply Sprout Method

You have a legacy `ReportGenerator.generate()` method that produces monthly reports. A new requirement says that reports for amounts over $100,000 must include an "auditor's note" section. Apply the Sprout Method technique: write the new behavior in a separate, tested method and add a single call to it in the legacy method. Write the new method with full test coverage.

Exercise 3: Plan a Strangler Fig Migration

Your company has a legacy monolith that handles: user authentication, product catalog, ordering, payments, and shipping. Draw a migration plan using the Strangler Fig pattern. Which module would you extract first and why? How would you route traffic between old and new systems? What risks should you mitigate?

What Is Next?

Now that you have the techniques for safely working with legacy code, the next chapter brings everything together in a complete Project Refactoring. You will take a messy, 300-line "god class" and refactor it step by step -- applying every principle from this book -- into clean, well-structured code spread across multiple focused files.

Chapter 31: Project Refactoring -- From God Class to Clean Architecture

What You Will Learn

- How to recognize and dismantle a "god class" -- a single class that does everything
- A step-by-step refactoring process that applies every principle from this book
- How to go from one messy 300-line file to six clean, focused files
- How to maintain working code at every step (no big-bang rewrites)
- Practical application of SRP, DIP, DI, strategy pattern, and clean architecture

Why This Chapter Matters

Throughout this book, you have learned principles: Single Responsibility, Open/Closed, Dependency Inversion, clean naming, small functions, testability. But learning principles and applying them are two different things.

This chapter is where everything comes together. We take a realistic, messy `OrderProcessor` god class -- the kind you would find in any production codebase -- and refactor it step by step into clean, well-structured code. Every step applies a principle you have already learned. Every step keeps the code working.

This is not a toy example. The original code is 300 lines of tangled logic: validation mixed with pricing, payment processing mixed with notification, database calls mixed with business rules. By the end, you will have six clean files, each with a single responsibility, fully testable, and easy to modify.

The Starting Point: The God Class

Here is the `OrderProcessor` in its original form. This is one file, one class, one massive method. Read through it and notice how many different responsibilities are tangled together.

BEFORE: OrderProcessor.java (One File, ~300 Lines)

```java
import java.sql.*;
import java.util.*;
import java.time.LocalDateTime;

public class OrderProcessor {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/shop";
    private static final String DB_USER = "root";
    private static final String DB_PASS = "password123";
    private static final double TAX_RATE = 0.08;
    private static final double FREE_SHIPPING_THRESHOLD = 50.0;
    private static final double STANDARD_SHIPPING = 5.99;
    private static final String SMTP_HOST = "smtp.company.com";

    public Map<String, Object> processOrder(Map<String, Object> orderData) {
        Map<String, Object> result = new HashMap<>();

        // ---- VALIDATION ----
        String customerEmail = (String) orderData.get("email");
        if (customerEmail == null || customerEmail.isEmpty()) {
            result.put("success", false);
            result.put("error", "Email is required");
            return result;
        }
        if (!customerEmail.contains("@")) {
            result.put("success", false);
            result.put("error", "Invalid email format");
            return result;
        }

        List<Map<String, Object>> items = (List<Map<String, Object>>) orderData.get("it...
        if (items == null || items.isEmpty()) {
            result.put("success", false);
            result.put("error", "Order must have at least one item");
            return result;
        }

        for (Map<String, Object> item : items) {
            int qty = (int) item.get("quantity");
            if (qty <= 0) {
                result.put("success", false);
                result.put("error", "Quantity must be positive for item: " + item.get("...
                return result;
```

```
            }
            double price = (double) item.get("price");
            if (price < 0) {
                result.put("success", false);
                result.put("error", "Price cannot be negative for item: " + item.get("n...
                return result;
            }
        }

        // ---- PRICING ----
        double subtotal = 0;
        for (Map<String, Object> item : items) {
            subtotal += (double) item.get("price") * (int) item.get("quantity");
        }

        // Apply discount
        String discountCode = (String) orderData.get("discountCode");
        double discount = 0;
        if (discountCode != null) {
            if (discountCode.equals("SAVE10")) {
                discount = subtotal * 0.10;
            } else if (discountCode.equals("SAVE20")) {
                discount = subtotal * 0.20;
            } else if (discountCode.equals("FLAT5")) {
                discount = 5.0;
            } else {
                // Check database for custom discount codes
                try {
                    Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_P...
                    PreparedStatement ps = conn.prepareStatement(
                        "SELECT discount_percent FROM discount_codes WHERE code = ? AND...
                    ps.setString(1, discountCode);
                    ResultSet rs = ps.executeQuery();
                    if (rs.next()) {
                        discount = subtotal * (rs.getDouble("discount_percent") / 100);
                    }
                    conn.close();
                } catch (SQLException e) {
                    System.out.println("DB error checking discount: " + e.getMessage());
                }
            }
        }

        double afterDiscount = subtotal - discount;
        double tax = afterDiscount * TAX_RATE;
        double shipping = afterDiscount >= FREE_SHIPPING_THRESHOLD ? 0 : STANDARD_SHIPP...
        double total = afterDiscount + tax + shipping;

        // ---- CHECK INVENTORY ----
        try {
            Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
            for (Map<String, Object> item : items) {
                PreparedStatement ps = conn.prepareStatement(
                    "SELECT stock FROM inventory WHERE product_id = ?");
                ps.setString(1, (String) item.get("productId"));
```

```
                ResultSet rs = ps.executeQuery();
                if (rs.next()) {
                    int stock = rs.getInt("stock");
                    int qty = (int) item.get("quantity");
                    if (stock < qty) {
                        result.put("success", false);
                        result.put("error", "Insufficient stock for: " + item.get("name...
                        conn.close();
                        return result;
                    }
                } else {
                    result.put("success", false);
                    result.put("error", "Product not found: " + item.get("productId"));
                    conn.close();
                    return result;
                }
            }
            conn.close();
        } catch (SQLException e) {
            result.put("success", false);
            result.put("error", "Database error: " + e.getMessage());
            return result;
        }

        // ---- PROCESS PAYMENT ----
        boolean paymentSuccess = false;
        String paymentMethod = (String) orderData.get("paymentMethod");
        String paymentId = null;
        try {
            if ("credit_card".equals(paymentMethod)) {
                // Simulate credit card processing
                String cardNumber = (String) orderData.get("cardNumber");
                if (cardNumber == null || cardNumber.length() < 13) {
                    result.put("success", false);
                    result.put("error", "Invalid card number");
                    return result;
                }
                paymentId = "CC-" + System.currentTimeMillis();
                paymentSuccess = true;
            } else if ("paypal".equals(paymentMethod)) {
                String paypalEmail = (String) orderData.get("paypalEmail");
                paymentId = "PP-" + System.currentTimeMillis();
                paymentSuccess = true;
            } else {
                result.put("success", false);
                result.put("error", "Unknown payment method: " + paymentMethod);
                return result;
            }
        } catch (Exception e) {
            result.put("success", false);
            result.put("error", "Payment processing failed: " + e.getMessage());
            return result;
        }

        // ---- SAVE ORDER TO DATABASE ----
```

```
        String orderId = "ORD-" + System.currentTimeMillis();
        try {
            Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS);
            PreparedStatement ps = conn.prepareStatement(
                "INSERT INTO orders (id, customer_email, subtotal, discount, tax, " +
                "shipping, total, payment_id, status, created_at) " +
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
            ps.setString(1, orderId);
            ps.setString(2, customerEmail);
            ps.setDouble(3, subtotal);
            ps.setDouble(4, discount);
            ps.setDouble(5, tax);
            ps.setDouble(6, shipping);
            ps.setDouble(7, total);
            ps.setString(8, paymentId);
            ps.setString(9, "CONFIRMED");
            ps.setString(10, LocalDateTime.now().toString());
            ps.executeUpdate();

            // Update inventory
            for (Map<String, Object> item : items) {
                PreparedStatement updatePs = conn.prepareStatement(
                    "UPDATE inventory SET stock = stock - ? WHERE product_id = ?");
                updatePs.setInt(1, (int) item.get("quantity"));
                updatePs.setString(2, (String) item.get("productId"));
                updatePs.executeUpdate();
            }

            conn.close();
        } catch (SQLException e) {
            result.put("success", false);
            result.put("error", "Failed to save order: " + e.getMessage());
            return result;
        }

        // ---- SEND CONFIRMATION EMAIL ----
        try {
            Properties props = new Properties();
            props.put("mail.smtp.host", SMTP_HOST);
            // ... email sending code ...
            System.out.println("Email sent to " + customerEmail + " for order " + order...
        } catch (Exception e) {
            // Log but don't fail the order
            System.out.println("Failed to send email: " + e.getMessage());
        }

        // ---- BUILD RESPONSE ----
        result.put("success", true);
        result.put("orderId", orderId);
        result.put("subtotal", subtotal);
        result.put("discount", discount);
        result.put("tax", tax);
        result.put("shipping", shipping);
        result.put("total", total);
        result.put("paymentId", paymentId);
```

```
        return result;
    }
}
```

What Is Wrong With This Code?

```
PROBLEMS IN THE GOD CLASS:

  +-------------------------------------------------------+
  | 1. SINGLE RESPONSIBILITY VIOLATION                    |
  |    One class does: validation, pricing, inventory,    |
  |    payment, persistence, notifications                |
  |                                                       |
  | 2. NO ABSTRACTION                                    |
  |    Raw Maps instead of domain objects                 |
  |    Raw JDBC instead of repositories                   |
  |    Hard-coded SMTP instead of notification interface |
  |                                                       |
  | 3. UNTESTABLE                                        |
  |    Cannot test pricing without a database             |
  |    Cannot test validation without payment processing |
  |    Cannot test anything in isolation                  |
  |                                                       |
  | 4. HARD-CODED CONFIGURATION                          |
  |    Database credentials in source code                |
  |    SMTP host hard-coded                               |
  |    Tax rate hard-coded                                |
  |                                                       |
  | 5. POOR ERROR HANDLING                                |
  |    Some exceptions caught, some ignored               |
  |    Database connections not properly closed           |
  |    No transaction management                          |
  |                                                       |
  | 6. NO DOMAIN MODEL                                   |
  |    Map<String, Object> everywhere                     |
  |    No type safety                                     |
  |    Business rules hidden in procedural code           |
  +-------------------------------------------------------+
```

The Refactoring Plan

We will refactor in six steps. Each step produces working code. Each step applies specific principles from this book.

```
REFACTORING PLAN:

  Step 1: Create Domain Objects       (Ch 2: Meaningful Names)
          Replace Map<String, Object>   (Ch 7: Classes)
          with typed classes

  Step 2: Extract OrderValidator       (Ch 8: Single Responsibility)
          All validation in one place

  Step 3: Extract PricingService       (Ch 8: Single Responsibility)
          Subtotal, discount, tax,      (Ch 25: Strategy Pattern)
          shipping calculations

  Step 4: Extract OrderRepository      (Ch 12: Dependency Inversion)
          Database access behind        (Ch 24: Dependency Injection)
          an interface

  Step 5: Extract PaymentProcessor     (Ch 12: Dependency Inversion)
          Payment logic behind          (Ch 24: Dependency Injection)
          an interface

  Step 6: Extract NotificationService  (Ch 12: Dependency Inversion)
          Email behind an interface     (Ch 24: Dependency Injection)

  Final: Wire everything in            (Ch 24: Composition Root)
         OrderService
```

Step 1: Create Domain Objects

Replace the raw `Map<String, Object>` with proper domain objects.

Principles applied: Meaningful Names (Ch 2), Classes (Ch 7)

File 1: Domain Objects

```java
// domain/OrderItem.java
public record OrderItem(
    String productId,
    String name,
    double price,
    int quantity
) {
    public OrderItem {
        if (productId == null || productId.isBlank())
            throw new IllegalArgumentException("Product ID is required");
```

```
        if (price < 0)
            throw new IllegalArgumentException("Price cannot be negative");
        if (quantity <= 0)
            throw new IllegalArgumentException("Quantity must be positive");
    }

    public double lineTotal() {
        return price * quantity;
    }
}

// domain/Order.java
public class Order {
    private final String id;
    private final String customerEmail;
    private final List<OrderItem> items;
    private final String discountCode;
    private final String paymentMethod;
    private final Map<String, String> paymentDetails;
    private OrderStatus status;
    private OrderPricing pricing;
    private String paymentId;

    public Order(String customerEmail, List<OrderItem> items,
                 String discountCode, String paymentMethod,
                 Map<String, String> paymentDetails) {
        this.id = "ORD-" + System.currentTimeMillis();
        this.customerEmail = customerEmail;
        this.items = List.copyOf(items);
        this.discountCode = discountCode;
        this.paymentMethod = paymentMethod;
        this.paymentDetails = paymentDetails != null
            ? Map.copyOf(paymentDetails) : Map.of();
        this.status = OrderStatus.PENDING;
    }

    public String getId() { return id; }
    public String getCustomerEmail() { return customerEmail; }
    public List<OrderItem> getItems() { return items; }
    public String getDiscountCode() { return discountCode; }
    public String getPaymentMethod() { return paymentMethod; }
    public Map<String, String> getPaymentDetails() { return paymentDetails; }
    public OrderStatus getStatus() { return status; }
    public OrderPricing getPricing() { return pricing; }
    public String getPaymentId() { return paymentId; }

    public void confirm(OrderPricing pricing, String paymentId) {
        this.pricing = pricing;
        this.paymentId = paymentId;
        this.status = OrderStatus.CONFIRMED;
    }
}

// domain/OrderPricing.java
public record OrderPricing(
```

```
    double subtotal,
    double discount,
    double tax,
    double shipping,
    double total
) {}

// domain/OrderStatus.java
public enum OrderStatus {
    PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED
}

// domain/OrderResult.java
public record OrderResult(
    boolean success,
    String orderId,
    OrderPricing pricing,
    String paymentId,
    String error
) {
    public static OrderResult success(Order order) {
        return new OrderResult(true, order.getId(), order.getPricing(),
                               order.getPaymentId(), null);
    }

    public static OrderResult failure(String error) {
        return new OrderResult(false, null, null, null, error);
    }
}
```

Step 2: Extract OrderValidator

Move all validation logic into its own class.

Principles applied: Single Responsibility (Ch 8), Clean Functions (Ch 3)

File 2: OrderValidator.java

```java
// validation/OrderValidator.java
public class OrderValidator {

    public List<String> validate(Order order) {
        List<String> errors = new ArrayList<>();

        validateEmail(order.getCustomerEmail(), errors);
```

```
        validateItems(order.getItems(), errors);
        validatePaymentMethod(order.getPaymentMethod(), errors);

        return errors;
    }

    private void validateEmail(String email, List<String> errors) {
        if (email == null || email.isBlank()) {
            errors.add("Email is required");
        } else if (!email.contains("@")) {
            errors.add("Invalid email format");
        }
    }

    private void validateItems(List<OrderItem> items, List<String> errors) {
        if (items == null || items.isEmpty()) {
            errors.add("Order must have at least one item");
        }
        // Note: Individual item validation is now in the OrderItem constructor
    }

    private void validatePaymentMethod(String method, List<String> errors) {
        if (method == null || method.isBlank()) {
            errors.add("Payment method is required");
        }
    }
}
```

Step 3: Extract PricingService

Move all pricing logic (subtotal, discounts, tax, shipping) into its own service.

Principles applied: Single Responsibility (Ch 8), Strategy Pattern (Ch 25), Open/Closed (Ch 9)

File 3: PricingService.java

```java
// pricing/PricingService.java
public class PricingService {

    private final DiscountStrategy discountStrategy;
    private final double taxRate;
    private final double freeShippingThreshold;
    private final double standardShipping;

    public PricingService(DiscountStrategy discountStrategy,
```

```
                            double taxRate,
                            double freeShippingThreshold,
                            double standardShipping) {
        this.discountStrategy = discountStrategy;
        this.taxRate = taxRate;
        this.freeShippingThreshold = freeShippingThreshold;
        this.standardShipping = standardShipping;
    }

    public OrderPricing calculatePricing(List<OrderItem> items, String discountCode) {
        double subtotal = calculateSubtotal(items);
        double discount = discountStrategy.calculateDiscount(subtotal, discountCode);
        double afterDiscount = subtotal - discount;
        double tax = afterDiscount * taxRate;
        double shipping = calculateShipping(afterDiscount);
        double total = afterDiscount + tax + shipping;

        return new OrderPricing(subtotal, discount, tax, shipping, total);
    }

    private double calculateSubtotal(List<OrderItem> items) {
        return items.stream()
            .mapToDouble(OrderItem::lineTotal)
            .sum();
    }

    private double calculateShipping(double afterDiscount) {
        return afterDiscount >= freeShippingThreshold ? 0 : standardShipping;
    }
}

// pricing/DiscountStrategy.java
public interface DiscountStrategy {
    double calculateDiscount(double subtotal, String discountCode);
}

// pricing/StandardDiscountStrategy.java
public class StandardDiscountStrategy implements DiscountStrategy {

    private final DiscountCodeRepository discountCodeRepo;

    public StandardDiscountStrategy(DiscountCodeRepository discountCodeRepo) {
        this.discountCodeRepo = discountCodeRepo;
    }

    @Override
    public double calculateDiscount(double subtotal, String discountCode) {
        if (discountCode == null || discountCode.isBlank()) {
            return 0;
        }

        return switch (discountCode) {
            case "SAVE10" -> subtotal * 0.10;
            case "SAVE20" -> subtotal * 0.20;
            case "FLAT5"  -> 5.0;
```

```
            default       -> lookupCustomDiscount(subtotal, discountCode);
        };
    }

    private double lookupCustomDiscount(double subtotal, String code) {
        return discountCodeRepo.findActiveByCode(code)
            .map(discount -> subtotal * (discount.percent() / 100))
            .orElse(0.0);
    }
}
```

Step 4: Extract OrderRepository

Move all database access behind an interface.

Principles applied: Dependency Inversion (Ch 12), Dependency Injection (Ch 24)

File 4: OrderRepository.java

```java
// repository/OrderRepository.java
public interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String orderId);
}

// repository/InventoryRepository.java
public interface InventoryRepository {
    int getStock(String productId);
    void reduceStock(String productId, int quantity);
}

// repository/DiscountCodeRepository.java
public interface DiscountCodeRepository {
    Optional<DiscountCode> findActiveByCode(String code);
}

// repository/mysql/MySqlOrderRepository.java
public class MySqlOrderRepository implements OrderRepository {

    private final DataSource dataSource;

    public MySqlOrderRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }
```

```
    @Override
    public void save(Order order) {
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement ps = conn.prepareStatement(
                "INSERT INTO orders (id, customer_email, subtotal, discount, " +
                "tax, shipping, total, payment_id, status, created_at) " +
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
            ps.setString(1, order.getId());
            ps.setString(2, order.getCustomerEmail());
            ps.setDouble(3, order.getPricing().subtotal());
            ps.setDouble(4, order.getPricing().discount());
            ps.setDouble(5, order.getPricing().tax());
            ps.setDouble(6, order.getPricing().shipping());
            ps.setDouble(7, order.getPricing().total());
            ps.setString(8, order.getPaymentId());
            ps.setString(9, order.getStatus().name());
            ps.setString(10, LocalDateTime.now().toString());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new PersistenceException("Failed to save order: " + e.getMessage(), e);
        }
    }

    @Override
    public Optional<Order> findById(String orderId) {
        // Implementation omitted for brevity
        return Optional.empty();
    }
}

// repository/mysql/MySqlInventoryRepository.java
public class MySqlInventoryRepository implements InventoryRepository {

    private final DataSource dataSource;

    public MySqlInventoryRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public int getStock(String productId) {
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement ps = conn.prepareStatement(
                "SELECT stock FROM inventory WHERE product_id = ?");
            ps.setString(1, productId);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) return rs.getInt("stock");
            throw new ProductNotFoundException(productId);
        } catch (SQLException e) {
            throw new PersistenceException("Failed to check stock: " + e.getMessage(), e);
        }
    }

    @Override
    public void reduceStock(String productId, int quantity) {
```

```
        try (Connection conn = dataSource.getConnection()) {
            PreparedStatement ps = conn.prepareStatement(
                "UPDATE inventory SET stock = stock - ? WHERE product_id = ?");
            ps.setInt(1, quantity);
            ps.setString(2, productId);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new PersistenceException("Failed to update stock: " + e.getMessage(),...
        }
    }
}
```

Step 5: Extract PaymentProcessor

Move payment logic behind an interface.

Principles applied: Dependency Inversion (Ch 12), Strategy Pattern (Ch 25)

File 5: PaymentProcessor.java

```java
// payment/PaymentProcessor.java
public interface PaymentProcessor {
    PaymentResult processPayment(double amount, String method,
                                 Map<String, String> details);
}

// payment/PaymentResult.java
public record PaymentResult(String paymentId, boolean success, String error) {
    public static PaymentResult success(String paymentId) {
        return new PaymentResult(paymentId, true, null);
    }
    public static PaymentResult failure(String error) {
        return new PaymentResult(null, false, error);
    }
}

// payment/DefaultPaymentProcessor.java
public class DefaultPaymentProcessor implements PaymentProcessor {

    @Override
    public PaymentResult processPayment(double amount, String method,
                                        Map<String, String> details) {
        return switch (method) {
            case "credit_card" -> processCreditCard(amount, details);
            case "paypal"      -> processPayPal(amount, details);
```

```
            default            -> PaymentResult.failure(
                                      "Unknown payment method: " + method);
        };
    }

    private PaymentResult processCreditCard(double amount,
                                            Map<String, String> details) {
        String cardNumber = details.get("cardNumber");
        if (cardNumber == null || cardNumber.length() < 13) {
            return PaymentResult.failure("Invalid card number");
        }
        String paymentId = "CC-" + System.currentTimeMillis();
        return PaymentResult.success(paymentId);
    }

    private PaymentResult processPayPal(double amount,
                                        Map<String, String> details) {
        String paymentId = "PP-" + System.currentTimeMillis();
        return PaymentResult.success(paymentId);
    }
}
```

Step 6: Extract NotificationService

Move email notification behind an interface.

Principles applied: Dependency Inversion (Ch 12)

File 6: NotificationService.java

```java
// notification/NotificationService.java
public interface NotificationService {
    void sendOrderConfirmation(String email, String orderId, OrderPricing pricing);
}

// notification/EmailNotificationService.java
public class EmailNotificationService implements NotificationService {

    private final String smtpHost;

    public EmailNotificationService(String smtpHost) {
        this.smtpHost = smtpHost;
    }

    @Override
```

```
    public void sendOrderConfirmation(String email, String orderId,
                                      OrderPricing pricing) {
        try {
            // Email sending implementation
            System.out.println("Confirmation email sent to " + email
                               + " for order " + orderId);
        } catch (Exception e) {
            // Log but don't fail the order
            System.err.println("Failed to send email: " + e.getMessage());
        }
    }
}
```

The Final Result: OrderService

Now the `OrderService` orchestrates clean, focused components. Compare this to the original 300-line god class.

AFTER: OrderService.java (The Orchestrator)

```java
// service/OrderService.java
public class OrderService {

    private final OrderValidator validator;
    private final PricingService pricingService;
    private final InventoryRepository inventoryRepo;
    private final PaymentProcessor paymentProcessor;
    private final OrderRepository orderRepo;
    private final NotificationService notifications;

    public OrderService(OrderValidator validator,
                        PricingService pricingService,
                        InventoryRepository inventoryRepo,
                        PaymentProcessor paymentProcessor,
                        OrderRepository orderRepo,
                        NotificationService notifications) {
        this.validator = validator;
        this.pricingService = pricingService;
        this.inventoryRepo = inventoryRepo;
        this.paymentProcessor = paymentProcessor;
        this.orderRepo = orderRepo;
        this.notifications = notifications;
    }

    public OrderResult processOrder(Order order) {
```

```
        // Step 1: Validate
        List<String> errors = validator.validate(order);
        if (!errors.isEmpty()) {
            return OrderResult.failure(String.join("; ", errors));
        }

        // Step 2: Check inventory
        for (OrderItem item : order.getItems()) {
            int stock = inventoryRepo.getStock(item.productId());
            if (stock < item.quantity()) {
                return OrderResult.failure(
                    "Insufficient stock for: " + item.name());
            }
        }

        // Step 3: Calculate pricing
        OrderPricing pricing = pricingService.calculatePricing(
            order.getItems(), order.getDiscountCode());

        // Step 4: Process payment
        PaymentResult payment = paymentProcessor.processPayment(
            pricing.total(), order.getPaymentMethod(),
            order.getPaymentDetails());
        if (!payment.success()) {
            return OrderResult.failure("Payment failed: " + payment.error());
        }

        // Step 5: Confirm and save
        order.confirm(pricing, payment.paymentId());
        orderRepo.save(order);

        // Step 6: Reduce inventory
        for (OrderItem item : order.getItems()) {
            inventoryRepo.reduceStock(item.productId(), item.quantity());
        }

        // Step 7: Notify customer
        notifications.sendOrderConfirmation(
            order.getCustomerEmail(), order.getId(), pricing);

        return OrderResult.success(order);
    }
}
```

Before vs After: Side by Side

```
BEFORE (1 file, ~300 lines):          AFTER (6+ files, each focused):
```

```
+-----------------------------------+  +-----------------------------+
| OrderProcessor.java               |  | domain/                     |
|                                   |  |   Order.java          (45)  |
|  - Validation        (30 lines) |  |   OrderItem.java      (20)  |
|  - Pricing            (40 lines) |  |   OrderPricing.java   (10)  |
|  - Discount lookup    (20 lines) |  |   OrderResult.java    (15)  |
|  - Inventory check    (30 lines) |  |   OrderStatus.java     (5)  |
|  - Payment processing (35 lines) |  |                             |
|  - Order persistence  (35 lines) |  | validation/                 |
|  - Inventory update   (15 lines) |  |   OrderValidator.java (30)  |
|  - Email notification (15 lines) |  |                             |
|  - Response building  (15 lines) |  | pricing/                    |
|  - Error handling     (scattered)|  |   PricingService.java (35)  |
|  - DB connection      (repeated) |  |   DiscountStrategy.java (5) |
|  - Configuration      (hard-coded) |   StandardDiscount.java(25)|
|                                   |  |                             |
|  Total: ~300 lines                |  | repository/                 |
|  Responsibilities: 8+             |  |   OrderRepository.java  (5) |
|  Testable: NO                     |  |   InventoryRepository   (5) |
|                                   |  |   MySqlOrderRepo.java  (35) |
+-----------------------------------+  |   MySqlInventoryRepo   (30) |
                                       |                             |
                                       | payment/                    |
                                       |   PaymentProcessor.java  (5)|
                                       |   PaymentResult.java   (10) |
                                       |   DefaultPaymentProc   (30) |
                                       |                             |
                                       | notification/               |
                                       |   NotificationService    (5)|
                                       |   EmailNotification     (20)|
                                       |                             |
                                       | service/                    |
                                       |   OrderService.java    (50) |
                                       |                             |
                                       | Each file: 5-50 lines       |
                                       | Each: ONE responsibility    |
                                       | Testable: YES               |
                                       +-----------------------------+
```

Testing the Refactored Code

The refactored code is trivially testable because every dependency is injected.

```java
class OrderServiceTest {

    private OrderValidator validator = new OrderValidator();
```

```
    private FakeInventoryRepository inventoryRepo = new FakeInventoryRepository();
    private FakePaymentProcessor paymentProcessor = new FakePaymentProcessor();
    private FakeOrderRepository orderRepo = new FakeOrderRepository();
    private FakeNotificationService notifications = new FakeNotificationService();
    private PricingService pricingService = new PricingService(
        new StandardDiscountStrategy(new FakeDiscountCodeRepository()),
        0.08, 50.0, 5.99
    );

    private OrderService service = new OrderService(
        validator, pricingService, inventoryRepo,
        paymentProcessor, orderRepo, notifications
    );

    @Test
    void successful_order_flow() {
        inventoryRepo.setStock("WIDGET-1", 100);
        paymentProcessor.willSucceed();

        Order order = new Order(
            "jane@example.com",
            List.of(new OrderItem("WIDGET-1", "Widget", 25.0, 2)),
            null, "credit_card",
            Map.of("cardNumber", "4111111111111111")
        );

        OrderResult result = service.processOrder(order);

        assertTrue(result.success());
        assertNotNull(result.orderId());
        assertEquals(50.0, result.pricing().subtotal());
        assertTrue(orderRepo.wasSaved(result.orderId()));
        assertTrue(notifications.wasSentTo("jane@example.com"));
    }

    @Test
    void rejects_order_with_invalid_email() {
        Order order = new Order(
            "not-an-email",
            List.of(new OrderItem("WIDGET-1", "Widget", 25.0, 1)),
            null, "credit_card", Map.of()
        );

        OrderResult result = service.processOrder(order);

        assertFalse(result.success());
        assertTrue(result.error().contains("Invalid email"));
    }

    @Test
    void rejects_order_when_insufficient_stock() {
        inventoryRepo.setStock("WIDGET-1", 0);  // Out of stock

        Order order = new Order(
            "jane@example.com",
```

```
            List.of(new OrderItem("WIDGET-1", "Widget", 25.0, 5)),
            null, "credit_card",
            Map.of("cardNumber", "4111111111111111")
        );

        OrderResult result = service.processOrder(order);

        assertFalse(result.success());
        assertTrue(result.error().contains("Insufficient stock"));
    }

    @Test
    void applies_discount_correctly() {
        inventoryRepo.setStock("WIDGET-1", 100);
        paymentProcessor.willSucceed();

        Order order = new Order(
            "jane@example.com",
            List.of(new OrderItem("WIDGET-1", "Widget", 100.0, 1)),
            "SAVE10", "credit_card",
            Map.of("cardNumber", "4111111111111111")
        );

        OrderResult result = service.processOrder(order);

        assertTrue(result.success());
        assertEquals(10.0, result.pricing().discount());
    }
}
```

The Python Version (After)

Here is the same refactored structure in Python:

```python
# domain.py
from dataclasses import dataclass
from enum import Enum
from typing import Optional

class OrderStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"

@dataclass(frozen=True)
class OrderItem:
    product_id: str
```

```
    name: str
    price: float
    quantity: int

    def line_total(self) -> float:
        return self.price * self.quantity

@dataclass
class OrderPricing:
    subtotal: float
    discount: float
    tax: float
    shipping: float
    total: float

@dataclass
class Order:
    customer_email: str
    items: list[OrderItem]
    discount_code: Optional[str]
    payment_method: str
    payment_details: dict
    id: str = ""
    status: OrderStatus = OrderStatus.PENDING
    pricing: Optional[OrderPricing] = None
    payment_id: Optional[str] = None

    def confirm(self, pricing: OrderPricing, payment_id: str):
        self.pricing = pricing
        self.payment_id = payment_id
        self.status = OrderStatus.CONFIRMED

# validation.py
class OrderValidator:
    def validate(self, order: Order) -> list[str]:
        errors = []
        if not order.customer_email or "@" not in order.customer_email:
            errors.append("Valid email is required")
        if not order.items:
            errors.append("Order must have at least one item")
        return errors

# pricing.py
from typing import Protocol

class DiscountStrategy(Protocol):
    def calculate_discount(self, subtotal: float, code: str) -> float: ...

class PricingService:
    def __init__(self, discount_strategy, tax_rate=0.08,
                 free_shipping_threshold=50.0, standard_shipping=5.99):
        self.discount_strategy = discount_strategy
        self.tax_rate = tax_rate
        self.free_shipping_threshold = free_shipping_threshold
        self.standard_shipping = standard_shipping
```

```
    def calculate_pricing(self, items, discount_code) -> OrderPricing:
        subtotal = sum(item.line_total() for item in items)
        discount = self.discount_strategy.calculate_discount(subtotal, discount_code)
        after_discount = subtotal - discount
        tax = after_discount * self.tax_rate
        shipping = 0 if after_discount >= self.free_shipping_threshold \
                     else self.standard_shipping
        total = after_discount + tax + shipping
        return OrderPricing(subtotal, discount, tax, shipping, total)

# service.py
class OrderService:
    def __init__(self, validator, pricing_service, inventory_repo,
                 payment_processor, order_repo, notifications):
        self.validator = validator
        self.pricing_service = pricing_service
        self.inventory_repo = inventory_repo
        self.payment_processor = payment_processor
        self.order_repo = order_repo
        self.notifications = notifications

    def process_order(self, order: Order) -> dict:
        errors = self.validator.validate(order)
        if errors:
            return {"success": False, "error": "; ".join(errors)}

        for item in order.items:
            stock = self.inventory_repo.get_stock(item.product_id)
            if stock < item.quantity:
                return {"success": False,
                        "error": f"Insufficient stock for: {item.name}"}

        pricing = self.pricing_service.calculate_pricing(
            order.items, order.discount_code)

        payment = self.payment_processor.process_payment(
            pricing.total, order.payment_method, order.payment_details)
        if not payment["success"]:
            return {"success": False,
                    "error": f"Payment failed: {payment['error']}"}

        order.confirm(pricing, payment["payment_id"])
        self.order_repo.save(order)

        for item in order.items:
            self.inventory_repo.reduce_stock(item.product_id, item.quantity)

        self.notifications.send_order_confirmation(
            order.customer_email, order.id, pricing)

        return {"success": True, "order_id": order.id,
                "pricing": pricing, "payment_id": order.payment_id}
```

Principles Applied: Summary

```
PRINCIPLES APPLIED IN THIS REFACTORING:

  +-------------------------------+--------------------------------+
  | Principle                     | Where it was applied           |
  +-------------------------------+--------------------------------+
  | Single Responsibility (Ch 8)  | Each class has ONE job:        |
  |                               | validate, price, pay, save,    |
  |                               | notify, orchestrate.           |
  +-------------------------------+--------------------------------+
  | Open/Closed (Ch 9)            | DiscountStrategy interface:    |
  |                               | add new discounts without      |
  |                               | modifying PricingService.      |
  +-------------------------------+--------------------------------+
  | Dependency Inversion (Ch 12)| OrderService depends on        |
  |                               | interfaces, not MySQL or SMTP.|
  +-------------------------------+--------------------------------+
  | Dependency Injection (Ch 24)| All dependencies injected      |
  |                               | via constructor.               |
  +-------------------------------+--------------------------------+
  | Strategy Pattern (Ch 25)      | DiscountStrategy and           |
  |                               | PaymentProcessor are swappable|
  +-------------------------------+--------------------------------+
  | Meaningful Names (Ch 2)       | OrderItem, PricingService,     |
  |                               | InventoryRepository -- clear.  |
  +-------------------------------+--------------------------------+
  | Clean Functions (Ch 3)        | Each method does one thing.    |
  |                               | No method exceeds 20 lines.    |
  +-------------------------------+--------------------------------+
  | Error Handling (Ch 6)         | Exceptions with context.       |
  |                               | Resources properly closed.     |
  +-------------------------------+--------------------------------+
  | Testability                   | Every component testable in    |
  |                               | isolation with fake deps.      |
  +-------------------------------+--------------------------------+
```

Common Mistakes

Mistake 1: Refactoring Everything at Once

Trying to do all six steps in one commit. If anything breaks, you cannot tell which step caused it.

Fix: One step per commit. Run tests after each step. Keep the code working at every point.

Mistake 2: Refactoring Without Tests

Starting to refactor the god class without characterization tests. You will not know if you changed behavior.

Fix: Write characterization tests for the god class first (Chapter 30). Then refactor with confidence.

Mistake 3: Creating Too Many Tiny Classes

Splitting so aggressively that each class has only one method and you need to trace through fifteen files to understand a flow.

Fix: Aim for cohesive classes with 3-10 methods, not one-method classes. Group related behavior.

Best Practices

1. Refactor in small, verifiable steps. Each step should produce working, tested code.
2. Write characterization tests first. Know what the code does before changing it.
3. Extract the easiest piece first. Build momentum with a simple extraction (validation is often easiest).
4. Keep the orchestrator thin. The final service should only coordinate, not contain business logic.
5. Use interfaces for external boundaries. Databases, APIs, email -- these should be behind interfaces.
6. Commit after each step. If something goes wrong, you can revert to the last working state.

Quick Summary

```
PROJECT REFACTORING AT A GLANCE:
```

```
BEFORE: 1 file, 300 lines, 8+ responsibilities, untestable

AFTER:  6+ files, each < 50 lines, 1 responsibility each, testable

Steps:
1. Create domain objects (replace Maps with types)
2. Extract validation
3. Extract pricing (with strategy for discounts)
4. Extract repositories (database behind interfaces)
5. Extract payment processing (behind interface)
6. Extract notifications (behind interface)

Result: OrderService orchestrates clean components.
        Every component is independently testable.
        Every dependency is injected.
```

Key Points

- A god class is a single class with too many responsibilities. It is rigid, untestable, and painful to modify.
- Refactoring a god class is done in small, safe steps -- never as a big-bang rewrite.
- Create domain objects first to replace untyped data structures like Maps.
- Extract each responsibility into its own class: validation, pricing, persistence, payment, notification.
- Use interfaces for external dependencies (database, payment, email) to enable testing and flexibility.
- Use the Strategy pattern for behavior that varies (discounts, payment methods).
- The final service class should only orchestrate -- it should contain no business logic of its own.
- Write characterization tests before refactoring and run tests after every step.

Practice Questions

1. Why is it important to refactor in small steps rather than rewriting the entire god class at once?
2. The original `OrderProcessor` uses `Map<String, Object>` for order data. What specific problems does this cause, and how do domain objects solve them?

3. Why did we extract `DiscountStrategy` as an interface instead of putting all discount logic directly in `PricingService`?
4. After the refactoring, the `OrderService` has six constructor parameters. Is this a problem? Why or why not?
5. How would you add a new payment method (e.g., Bitcoin) to the refactored code? How many files would you need to change compared to the original god class?

Exercises

Exercise 1: Add a New Feature

Using the refactored code, add support for a "loyalty points" discount. Loyal customers earn points on each purchase, and those points can be redeemed as a discount on future orders. Implement this as a new `DiscountStrategy` without modifying any existing discount code.

Exercise 2: Refactor a Python God Class

Take this Python god class and refactor it following the same six-step process:

```python
class ReportGenerator:
    def generate(self, report_type, date_range, format_type, recipients):
        # 1. Validate inputs (30 lines)
        # 2. Query database for raw data (40 lines)
        # 3. Transform and aggregate data (50 lines)
        # 4. Format as PDF/CSV/Excel (60 lines)
        # 5. Save to file system (20 lines)
        # 6. Email to recipients (30 lines)
        # Total: ~230 lines, all in one method
        pass
```

Exercise 3: Write Tests for the Orchestrator

Write a complete test suite for the `OrderService` orchestrator using fake implementations. Test: successful order, validation failure, insufficient stock, payment failure, and discount application.

What Is Next?

Congratulations -- you have just completed the most comprehensive refactoring in this book. The final chapter provides a Glossary of all the terms, patterns, and principles you have learned, along with recommended reading to continue your journey into clean code and software architecture.

Chapter 32: Glossary and Recommended Reading

What You Will Learn

- Quick-reference definitions for 80+ terms used throughout this book
- Where each term was introduced and how it connects to other concepts
- Recommended books to continue your learning journey

Why This Chapter Matters

Software development has its own vocabulary. Terms like "coupling," "cohesion," "dependency inversion," and "bounded context" carry precise meanings that differ from everyday English. Misunderstanding a term can lead to misapplying a principle.

This glossary gives you a single place to look up any term from the book. Each definition is concise, practical, and linked to the chapter where it was covered in depth.

Glossary

Abstraction. Hiding implementation details behind a simpler interface. A method signature is an abstraction -- callers know what it does without knowing how. (Chapters 3, 12, 25)

Abstraction Boundary. A seam in your code (usually an interface) where you can swap implementations without affecting calling code. Place boundaries at integration points where change is likely. (Chapter 25)

Aggregate. In Domain-Driven Design, a cluster of domain objects treated as a single unit for data changes. The aggregate root controls access and enforces consistency rules. (Chapter 22)

Anemic Domain Model. A domain model where objects contain only data (getters/setters) and no behavior. Business logic lives in separate service classes instead of in the domain objects where it belongs. The opposite of a Rich Domain Model. (Chapter 22)

Anti-Corruption Layer (ACL). A translation layer that protects your domain model from an external system's model. It converts the external system's types and language into your own. (Chapter 23)

Anti-Pattern. A common but counterproductive practice. Looks like a good idea but causes more problems than it solves. Examples: God Class, Service Locator, Singleton abuse. (Various chapters)

Backward Compatibility. The property of an API or system that allows older clients to continue working after an update. Adding optional fields is backward compatible; removing fields is not. (Chapter 26)

Bounded Context. An explicit boundary within which a particular domain model applies. Inside the boundary, every term has one clear meaning. Different bounded contexts may use the same word (e.g., "Customer") to mean different things. (Chapter 23)

Branch by Abstraction. A technique for replacing an implementation by inserting an interface, building a new implementation behind it, and switching over -- all without long-lived feature branches. (Chapter 30)

Characterization Test. A test that documents the current behavior of existing code, whether correct or not. Used before modifying legacy code to detect unintended changes. (Chapter 30)

Circular Dependency. When A depends on B and B depends on A. This creates tight coupling and makes both classes harder to test, understand, and deploy independently. (Chapter 24)

Clean Architecture. An architectural approach where business rules are at the center, independent of frameworks, databases, and UI. Dependencies point inward toward the domain. (Chapter 18)

Code Coverage. The percentage of code lines or branches executed by tests. Useful as a guide, but 100% coverage does not guarantee correct code. (Chapter 16)

Code Smell. A surface indication in code that usually corresponds to a deeper problem. Examples: long methods, large classes, feature envy, shotgun surgery. (Chapter 14)

Code Review. The practice of having other developers examine code before it is merged. Improves quality, shares knowledge, and maintains consistency. (Chapter 29)

Cohesion. The degree to which the elements inside a module belong together. High cohesion means everything in the class is related to its single purpose. (Chapter 7)

Command-Query Separation (CQS). The principle that a method should either change state (command) or return data (query), but not both. Queries should have no side effects. (Chapter 3)

Composition Root. The single place in an application where all dependencies are created and wired together. Usually near the application entry point. (Chapter 24)

Conformist. A bounded context relationship where the downstream context adopts the upstream context's model without translation. Used when you cannot change the upstream system. (Chapter 23)

Constructor Injection. Providing dependencies through a class's constructor. The preferred form of Dependency Injection because it makes dependencies explicit, required, and immutable. (Chapter 24)

Context Map. A diagram showing how bounded contexts relate to each other, including the direction of data flow and the type of relationship. (Chapter 23)

Coupling. The degree of interdependence between modules. Tight coupling means a change in one module forces changes in others. Loose coupling is the goal. (Chapter 7)

Customer-Supplier. A bounded context relationship where the upstream context provides data that the downstream context needs. The teams can negotiate the shared interface. (Chapter 23)

Cyclomatic Complexity. A measure of the number of independent paths through a function. Higher complexity means more paths, more tests needed, and harder comprehension. (Chapter 3)

Dead Code. Code that is never executed. It adds confusion and maintenance burden. Delete it; version control remembers. (Chapter 14)

Dependency Injection (DI). Providing dependencies to a class from the outside rather than having the class create them internally. Enables testability and loose coupling. (Chapter 24)

Dependency Inversion Principle (DIP). High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details. The "D" in SOLID. (Chapter 12)

DI Container. A framework that automates dependency creation and injection. Examples: Spring (Java), dependency-injector (Python), FastAPI Depends. (Chapter 24)

Domain-Driven Design (DDD). An approach to software development that focuses on the business domain, uses Ubiquitous Language, and structures code around domain concepts.

(Chapter 22)

Domain Event. A record of something significant that happened in the domain. Used to decouple parts of a system -- one context publishes an event, others react. (Chapters 22, 25)

DRY (Don't Repeat Yourself). Every piece of knowledge should have a single, authoritative representation. But wrong abstraction is worse than duplication. (Chapter 13)

Entity. A domain object with a unique identity that persists over time. Two entities are equal if they have the same ID, regardless of other attributes. (Chapter 22)

Event-Driven Architecture. A pattern where components communicate through events rather than direct calls. The producer publishes events; consumers react independently. (Chapter 25)

Extract Method. A refactoring technique that moves a block of code into a new method with a descriptive name. Improves readability and enables reuse. (Chapter 15)

Feature Envy. A code smell where a method accesses data from another class more than its own. The method probably belongs in the other class. (Chapter 14)

Feature Flag. A mechanism to enable or disable features without deploying new code. Allows safe deployment of incomplete features and gradual rollouts. (Chapter 25)

God Class. A class that does too many things. It has too many fields, too many methods, and too many responsibilities. The solution is to split it into focused classes. (Chapters 14, 31)

Guard Clause. An early return at the start of a method that handles edge cases or invalid inputs, reducing nesting in the main logic. (Chapter 3)

Hexagonal Architecture. An architectural style (also called Ports and Adapters) where business logic is at the center, surrounded by ports (interfaces) and adapters (implementations). (Chapter 20)

Idempotency. An operation that produces the same result whether called once or multiple times. Critical for API reliability. GET, PUT, and DELETE are naturally idempotent; POST typically is not. (Chapter 26)

Immutability. The property of an object that cannot be changed after creation. Immutable objects are safer in concurrent code and easier to reason about. (Chapter 7)

Integration Test. A test that verifies the interaction between two or more components, including real external resources like databases. Slower than unit tests but verifies real integration. (Chapter 16)

Interface. A contract that defines what methods a class must implement without specifying how. In Java, the `interface` keyword. In Python, abstract base classes or Protocols. (Chapter 11)

Interface Segregation Principle (ISP). Clients should not be forced to depend on interfaces they do not use. Many specific interfaces are better than one general-purpose interface. The "I" in SOLID. (Chapter 11)

KISS (Keep It Simple, Stupid). Prefer the simplest solution that works. Complexity should be justified by real requirements, not hypothetical future needs. (Chapter 13)

Layered Architecture. An architecture where code is organized into horizontal layers (presentation, business, data access). Each layer depends only on the layer below it. (Chapter 19)

Legacy Code. Code you are afraid to change, usually because it lacks tests. Not necessarily old code -- new code without tests is also legacy. (Chapter 30)

Liskov Substitution Principle (LSP). Objects of a subtype should be replaceable with objects of the base type without breaking the program. Subtypes must honor the contracts of their parent types. The "L" in SOLID. (Chapter 10)

Long Method. A code smell. A method that is too long to understand at a glance. Extract methods to break it into named, focused steps. (Chapter 14)

Magic Number. A literal number in code without explanation. Replace with a named constant that explains its purpose. (Chapter 2)

Modular Monolith. A monolith with clear module boundaries. Each module has its own domain model and communicates through defined interfaces. A good stepping stone toward microservices. (Chapter 23)

Mutation Testing. A testing technique that introduces small changes (mutations) into the code and checks whether tests catch them. Measures the effectiveness of your test suite. (Chapter 16)

Null Object Pattern. Replacing null checks with an object that implements the expected interface but does nothing. Eliminates NullPointerExceptions and simplifies calling code. (Chapter 6)

Onion Architecture. An architectural style with concentric layers: Domain Model at the center, surrounded by Domain Services, Application Services, and Infrastructure. Dependencies point inward. (Chapter 21)

Open/Closed Principle (OCP). Software entities should be open for extension but closed for modification. Add new behavior by adding new code, not by changing existing code. The "O" in SOLID. (Chapter 9)

Over-Engineering. Adding unnecessary abstractions, patterns, or flexibility for hypothetical future needs that never materialize. Just as harmful as under-engineering. (Chapter 25)

Pagination. Returning large result sets in smaller pages rather than all at once. Essential for API performance. Two styles: offset-based and cursor-based. (Chapter 26)

Port. In Hexagonal Architecture, an interface that defines how the application communicates with the outside world. Ports are defined by the application core. (Chapter 20)

Adapter. In Hexagonal Architecture, an implementation of a port that connects to a specific external system (database, API, UI). (Chapter 20)

Primitive Obsession. A code smell where primitive types (strings, ints) are used instead of small domain objects. Replace "String email" with an "EmailAddress" value object. (Chapter 14)

Pull Request (PR). A request to merge code changes into a shared branch. The unit of code review. Keep PRs small (under 300 lines) for effective review. (Chapter 29)

Refactoring. Changing the structure of code without changing its behavior. Done to improve readability, reduce complexity, or enable future changes. Always maintain passing tests. (Chapter 15)

Repository Pattern. An abstraction that makes data access look like an in-memory collection. Hides database details from business logic. (Chapters 22, 24)

Rich Domain Model. A domain model where objects contain both data and the business behavior that operates on that data. The opposite of an Anemic Domain Model. (Chapter 22)

Rule of Three. Wait until you have three similar cases before abstracting. Two occurrences may be coincidence; three suggest a pattern worth generalizing. (Chapters 13, 25)

Separation of Concerns. Organizing code so that each section addresses a distinct concern. Related to Single Responsibility but applied at a broader architectural level. (Chapter 7)

Service Locator. An anti-pattern where classes fetch dependencies from a global registry instead of receiving them through injection. Hides dependencies and makes testing harder. (Chapter 24)

Setter Injection. Providing dependencies through setter methods after construction. Use only for optional dependencies with sensible defaults. (Chapter 24)

Shared Kernel. A bounded context relationship where two contexts share a small, jointly maintained model. Keep it minimal to avoid coupling. (Chapter 23)

Shotgun Surgery. A code smell where a single change requires modifications in many classes scattered across the codebase. Indicates a responsibility is spread too thin. (Chapter 14)

Single Responsibility Principle (SRP). A class should have only one reason to change. Each class should have one job, one focus, one responsibility. The "S" in SOLID. (Chapter 8)

SOLID. An acronym for five object-oriented design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. (Chapters 8-12)

Sprout Method. A technique for adding new behavior to legacy code by writing it in a new, tested method and calling it from the legacy code with minimal modification. (Chapter 30)

Sprout Class. Like Sprout Method but for larger new behaviors. Create an entirely new, clean class and instantiate it from the legacy code. (Chapter 30)

Strangler Fig Pattern. Gradually replacing a legacy system by routing traffic to a new system piece by piece, while the old system continues running. Avoids big-bang rewrites. (Chapter 30)

Strategy Pattern. A design pattern that defines a family of algorithms, encapsulates each one behind an interface, and makes them interchangeable. The calling code does not know which strategy it uses. (Chapter 25)

Technical Debt. The implied cost of future rework caused by choosing an expedient solution now instead of a better approach. Like financial debt -- you pay interest over time. (Chapter 28)

Test Double. A generic term for any object used in place of a real object during testing. Includes fakes, stubs, mocks, and dummies. (Chapter 16)

Test-Driven Development (TDD). A development practice where you write a failing test first, then write the minimum code to make it pass, then refactor. Red-Green-Refactor cycle. (Chapter 17)

Ubiquitous Language. In Domain-Driven Design, the shared vocabulary used by developers and business experts. The same terms appear in conversations, documentation, and code. (Chapter 22)

Unit Test. A test that verifies a single unit of behavior in isolation, without external dependencies. Should be fast, independent, and repeatable. (Chapter 16)

Value Object. A domain object defined by its attributes rather than a unique identity. Two value objects with the same attributes are equal. Examples: Money, Address, DateRange. Typically immutable. (Chapter 22)

YAGNI (You Ain't Gonna Need It). Do not add functionality until it is actually needed. Resist the urge to build for hypothetical future requirements. (Chapter 13)

Recommended Reading

These books will deepen your understanding of clean code, software architecture, and software design. They are listed in a suggested reading order, from most accessible to most advanced.

Clean Code and Craftsmanship

Clean Code: A Handbook of Agile Software Craftsmanship by Robert C. Martin. The foundational text on writing readable, maintainable code. Covers naming, functions, comments, formatting, error handling, and testing. If you read only one book on this list, make it this one.

Refactoring: Improving the Design of Existing Code by Martin Fowler. The definitive catalog of refactoring techniques. Each refactoring is named, explained, and demonstrated step by step. Essential for anyone who works with existing codebases.

The Pragmatic Programmer by David Thomas and Andrew Hunt. Practical advice on the craft of software development, from coding to career. Covers a wide range of topics with actionable tips.

A Philosophy of Software Design by John Ousterhout. A concise and opinionated take on software complexity. Introduces the concept of "deep modules" -- simple interfaces hiding complex implementations.

Software Architecture

Clean Architecture: A Craftsman's Guide to Software Structure and Design by Robert C. Martin. Explains how to structure software so that business rules are independent of frameworks, databases, and UI. Covers the Dependency Rule, hexagonal architecture, and boundary design.

Patterns of Enterprise Application Architecture by Martin Fowler. A comprehensive catalog of architectural patterns for enterprise applications: Repository, Unit of Work, Data Mapper, Service Layer, and many more. A reference you will return to throughout your career.

Building Microservices by Sam Newman. A practical guide to designing, building, and operating microservices. Covers decomposition strategies, inter-service communication, data management, and deployment.

Domain-Driven Design

Domain-Driven Design: Tackling Complexity in the Heart of Software by Eric Evans. The original DDD book. Introduces Ubiquitous Language, Bounded Contexts, Entities, Value Objects, Aggregates, and strategic design patterns. Dense but transformative.

Implementing Domain-Driven Design by Vaughn Vernon. A more practical companion to Evans's book. Shows how to apply DDD concepts using real code and real architectures. Covers bounded contexts, aggregates, domain events, and CQRS.

Testing

Working Effectively with Legacy Code by Michael Feathers. The essential guide to making changes in codebases without tests. Introduces characterization tests, seam-based techniques, and strategies for breaking dependencies. Every developer should read this.

Test-Driven Development: By Example by Kent Beck. The original TDD book by its creator. Walks through two complete TDD examples, teaching the Red-Green-Refactor rhythm by doing.

Design Patterns

Head First Design Patterns by Eric Freeman and Elisabeth Robson. An accessible, visual introduction to the most important design patterns. Uses humor and exercises to make patterns memorable. A great starting point before reading the Gang of Four book.

Design Patterns: Elements of Reusable Object-Oriented Software by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (the "Gang of Four"). The classic catalog of 23 design patterns. More of a reference than a cover-to-cover read. Look up patterns as you encounter the problems they solve.

Congratulations

You have completed the journey through Clean Code and Software Architecture. You started with the basics -- naming variables, writing small functions, formatting code -- and progressed through SOLID principles, architecture patterns, domain-driven design, testing strategies, and real-world refactoring.

Here is what you have learned:

```
YOUR JOURNEY THROUGH THIS BOOK:

  Foundations (Chapters 1-6):
  +-----------------------------------------+
  | Why clean code matters                  |
  | Meaningful names                        |
  | Small, focused functions                |
  | Comments that add value                 |
  | Consistent formatting                   |
  | Error handling without surprises        |
  +-----------------------------------------+

  Object-Oriented Design (Chapters 7-13):
  +-----------------------------------------+
  | Cohesive classes                        |
  | SOLID principles (SRP, OCP, LSP, ISP, DIP)|
  | DRY, KISS, YAGNI                        |
  +-----------------------------------------+

  Code Quality (Chapters 14-17):
  +-----------------------------------------+
  | Recognizing code smells                 |
  | Refactoring techniques                  |
  | Unit testing principles                 |
  | Test-Driven Development                 |
  +-----------------------------------------+

  Architecture (Chapters 18-23):
  +-----------------------------------------+
  | Clean Architecture                      |
  | Layered Architecture                    |
  | Hexagonal Architecture                  |
  | Onion Architecture                      |
  | Domain-Driven Design                    |
  | Bounded Contexts                        |
  +-----------------------------------------+

  Advanced Practices (Chapters 24-31):
  +-----------------------------------------+
  | Dependency Injection                    |
  | Designing for change                    |
  | API design principles                   |
  | Logging and observability               |
  | Managing technical debt                 |
  | Code review culture                     |
  | Working with legacy code                |
  | Complete project refactoring            |
  +-----------------------------------------+
```

But finishing this book is not the end. It is the beginning. The principles you have learned only become powerful through practice. The next time you write a function, think about its name, its size, its single responsibility. The next time you design a class, think about its dependencies and its cohesion. The next time you review someone's code, give feedback that teaches and

inspires.

Clean code is not about perfection. It is about respect -- respect for your future self, respect for your teammates, and respect for the craft of software development.

Write code that others can read. Build systems that others can maintain. Leave the codebase better than you found it.

Happy coding.